



**DISTRIBUIÇÃO
INTELIGENTE**

TREINAMENTO FPGA ALTERA



CIDADES INTELIGENTES



SOLUÇÕES IOT E AI



**SUORTE E
DESENVOLVIMENTO**

MACNICA
Macnica DHW

altera
An Intel Company

DIREITOS AUTORAIS

Este material didático é protegido por Direitos Autorais, sendo vedada a reprodução, distribuição, comercialização de qualquer material ou conteúdo dele obtido, sem a prévia e expressa autorização da Macnica DHW.

O material didático disponibilizado, se constitui em trabalho intelectual, cujos direitos autorais pertencem à Macnica DHW e a Intel, e está protegido por leis nacionais e tratados internacionais, qualquer reprodução ou distribuição, ou utilização comercial, de qualquer conteúdo deste material didático é expressamente proibida e o transgressor será responsabilizado civil e criminalmente na forma da lei.

SOBRE

2005

A DHW surgiu como Design House especializada em Altera

2005

Foi designada pela Altera como o Centro Oficial de Treinamento Altera e responsável pelo Programa Universitário Altera.

2009

Tornou-se representante exclusiva da Altera na América do Sul.

2012

A DHW foi adquirida por um dos maiores grupos de Distribuição de componentes do Japão, a Macnica Inc., tornando-se Macnica DHW e ampliando o seu foco de negócio para:

- Distribuição de semicondutores
- Desenvolvimento de projetos para Soluções IoT
- *Smart Cities*

2019

A Macnica DHW é gestora do Perini City Lab, o primeiro *One Stop Shop* do Brasil. Empresas podem desenvolver, testar e demonstrar tecnologias alinhadas com a temática de *Smart Cities* dentro de um ambiente privado e controlado.

2020

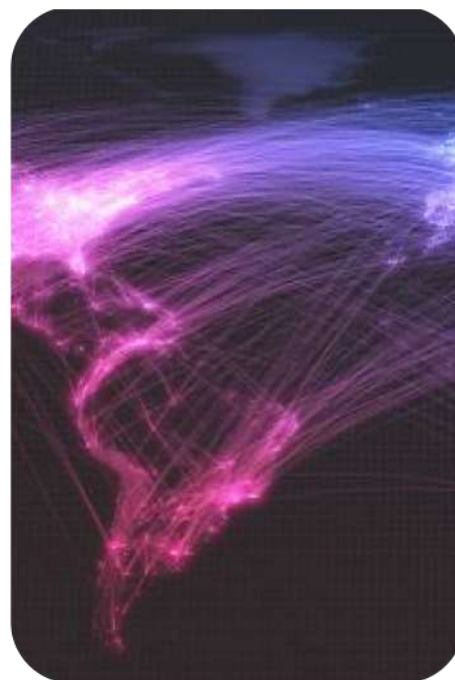
A Macnica DHW tornou-se o Centro de Excelência Intel no Brasil para Inteligência Artificial e Plataformas de Aceleração por Hardware (baseadas em FPGA).



Fundada em 1972.
Maior distribuidora de
Semicondutores do
Japão



Provedora de
Soluções
para *Cybersecurity*
e IoT & AI



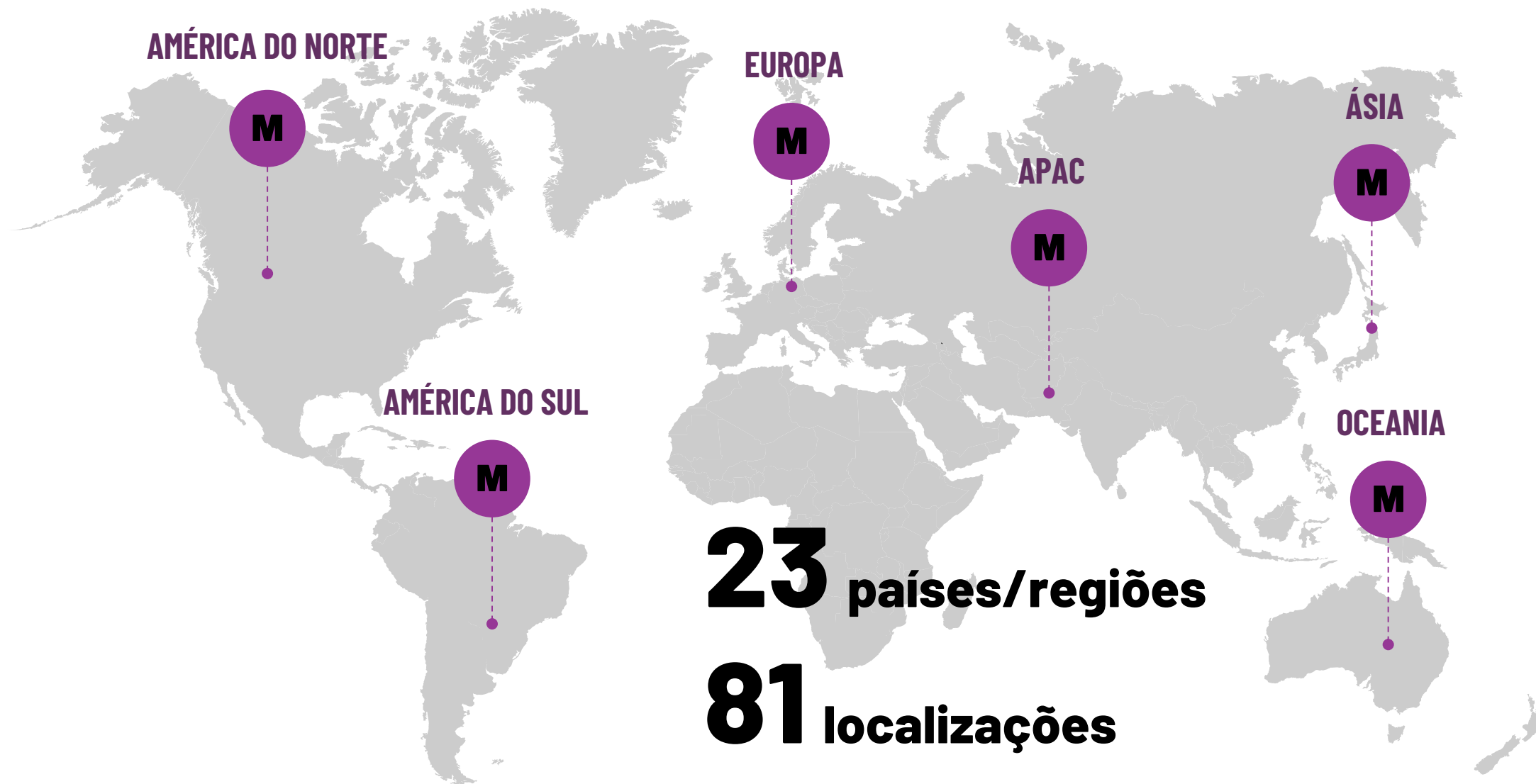
Mais de 80 escritórios
na Ásia, Europa
e Américas



4.200 funcionários
onde 1 a cada 3
é engenheiro



Receita em 2022
US\$ 10 Bilhões



DISTRIBUIÇÃO INTELIGENTE

Faz parte de todo o **ciclo de desenvolvimento** dos projetos e produtos da sua empresa, utilizando tecnologia de ponta e componentes de alta qualidade.

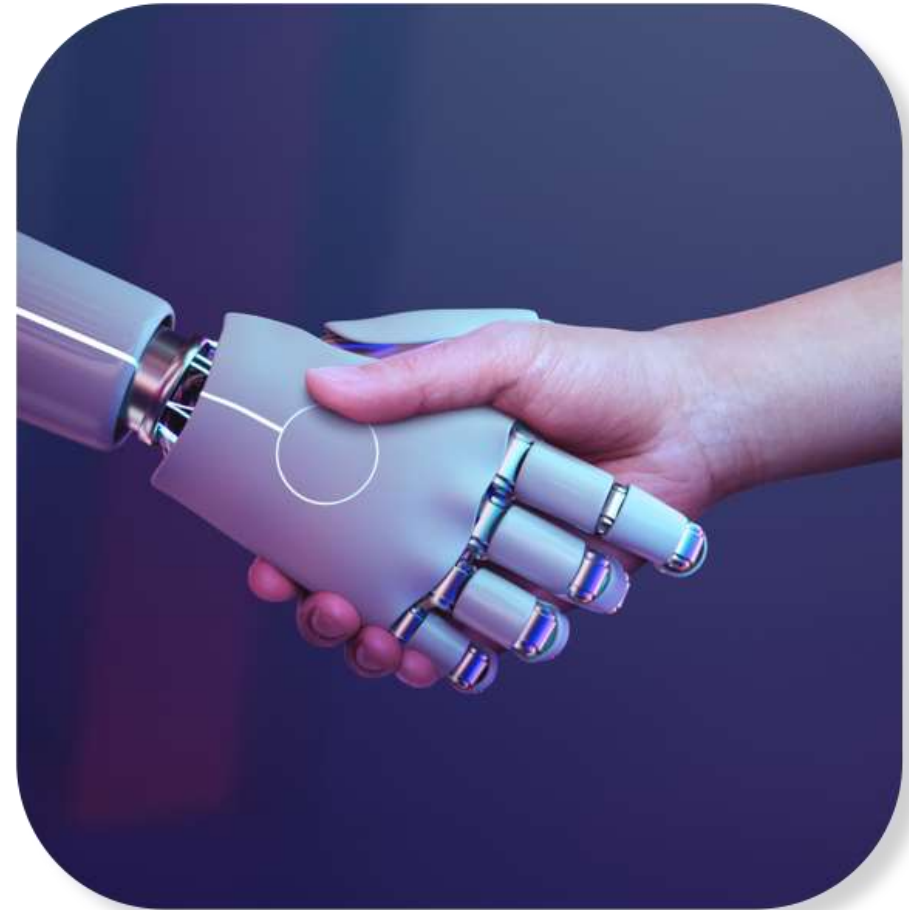
Oferece **suporte técnico local e especializado** com conhecimento em todas as tecnologias do seu portfólio.



SOLUÇÕES IoT & IA

Aceleramos a transformação digital de sua empresa implementando IoT e IA em seus produtos, operações e negócios.

Como uma **parceira de confiança** trabalhamos em estreita colaboração para o sucesso **em sua jornada de transformação**



TREINAMENTO FPGA ALTERA

Somos o **Centro Oficial de Treinamento Altera** na América do Sul, além de ser a única empresa capacitada para ministrar **todos os módulos FPGA** oferecidos pela Altera.

Com mais de **15 anos de experiência em projetos utilizando FPGA Altera**.

Já **capacitamos** cerca de 3.500 pessoas entre **engenheiros, desenvolvedores de hardware, professores e alunos**.



INTRODUCTION TO FPGA



Introduction to Intel FPGA Technology

- What is an FPGA?
- Where FPGAs are used?
- How to start?



What is an FPGA?

FPGA stands for **Field Programmable Gate Array**

- **FPGA** is a semiconductor integrated circuit where electrical functionality is customized to accelerate key workloads.
- Altera® FPGAs offer a wide variety of configurable embedded SRAM, high-speed transceivers, high-speed I/Os, logic blocks, and routing.
- Also, built-in intellectual property (IP) combined with outstanding software tools **lower FPGA development time, power, and cost.**

What is an FPGA?

FPGA is like a whiteboard when compared with to ASICs and ASSPs.



Where FPGAs are used?

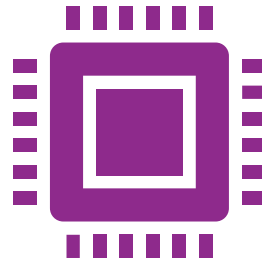
FPGAs are present in many growing industries: Data Center, Wireless Communications, Military & Industrial, Healthcare, Finance, Retail, and Automotive just to name a few.



Why FPGA?

ReProgrammable
& Flexible

Reduced
time-to-Market

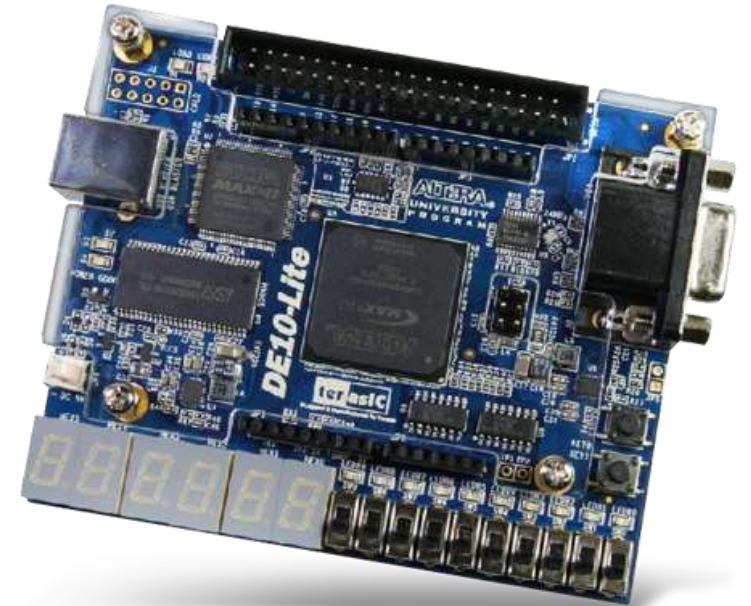


Product
Longevity

Market-Size
Optimized

How to start?

- Review concepts
- Install Quartus Prime software
- Development and test in the hardware (FPGA Board)

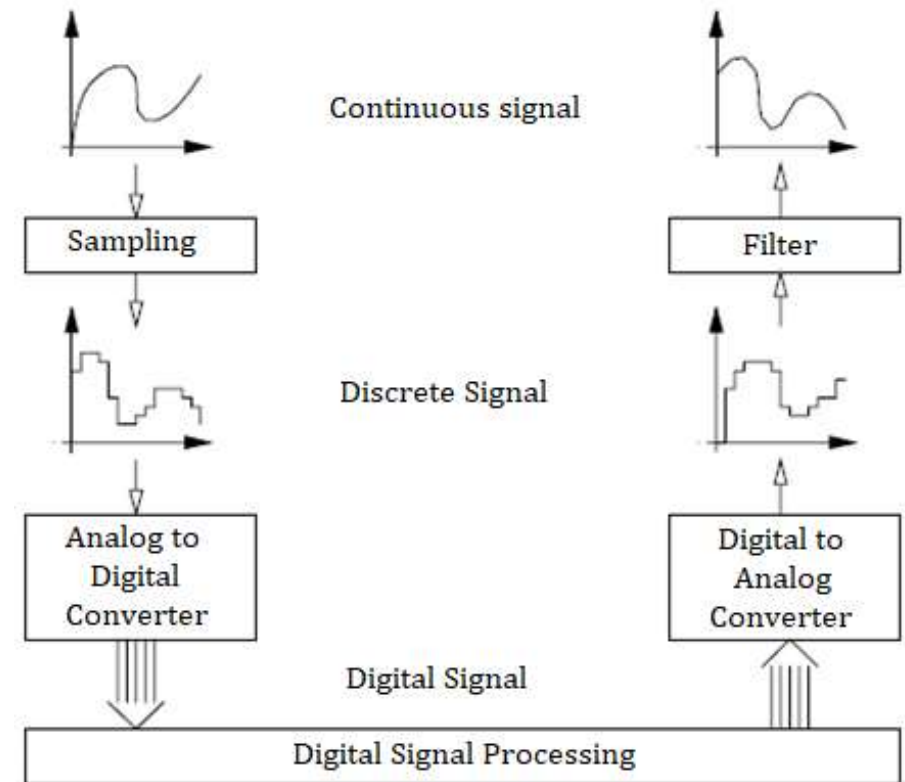


DIGITAL LOGIC REVIEW



INTRODUCTION

- The "real world" is mostly analog, made up of continuous quantities
- To work in the digital world, we need to make it discrete.



NUMERAL SYSTEM

- The binary system or base 2 system is a method of numeral expression, which uses only two symbols, each digit being referred to as a bit: **0** and **1**.
- The binary numbers can be decomposed the same way as a decimal number, expressing it in the base 2 numeral system:

$$2534 = 2 \times 10^3 + 5 \times 10^2 + 3 \times 10^1 + 4 \times 10^0$$

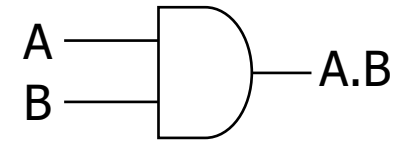
$$100110_2 = 1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

BOOLEAN ALGEBRA

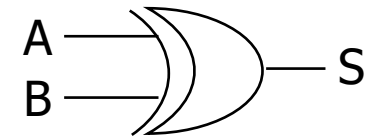
Basic Operations

The basic operations of Boolean Algebra are:

NOT, AND, and OR.

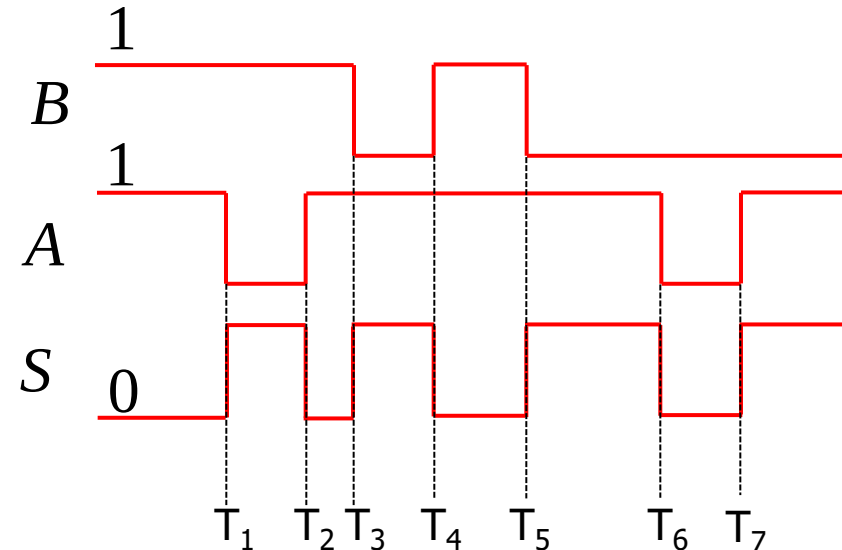


A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1



Representation

- Logic Gates
- Truth Table
- Timing Diagram



A	B	S
0	0	0
0	1	1
1	0	1
1	1	0

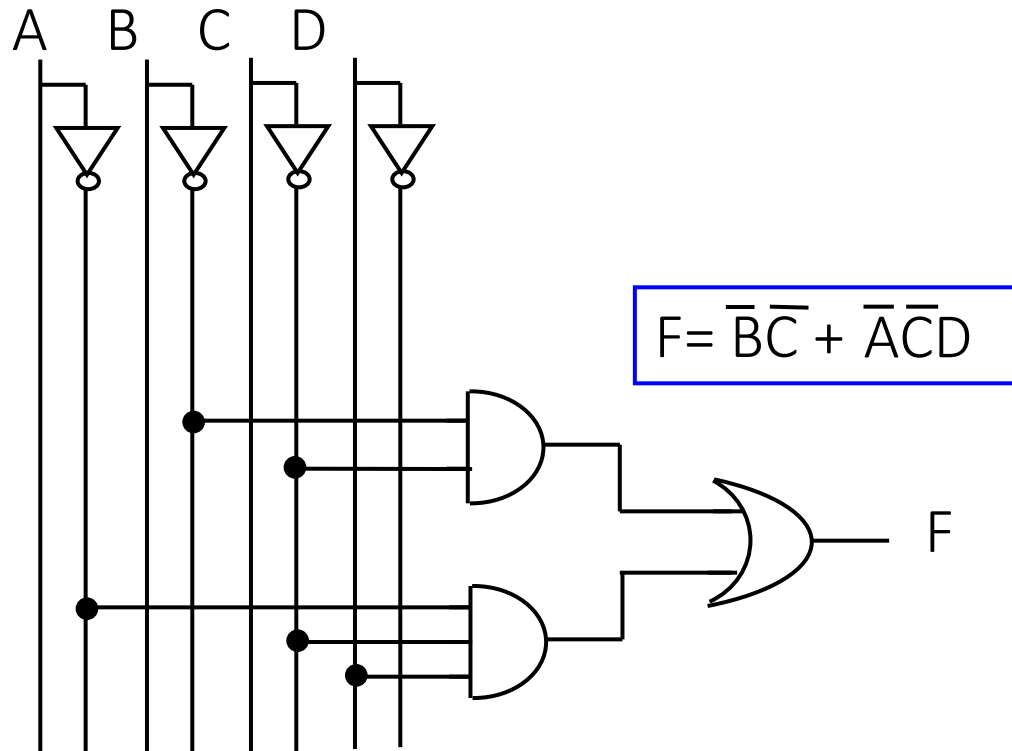
BOOLEAN ALGEBRA

Boolean algebra simplification

We can get a simplified form of a given Boolean expression by using Boolean algebra theorems and axioms.

$$\begin{aligned} Z &= \overline{(\bar{A} + C).(\bar{B} + \bar{D})} \\ &= \overline{(\bar{A} + C)} + \overline{(\bar{B} + \bar{D})} \\ &= (\bar{\bar{A}}.\bar{C}) + (\bar{\bar{B}}.\bar{\bar{D}}) \end{aligned}$$

$$Z = A\bar{C} + \bar{B}D$$



BOOLEAN ALGEBRA

Boolean Algebra Simplification

Karnaugh map, a diagram that can be used to simplify a Boolean algebra expression.

		AB			
		00	01	11	10
CD	00			X	
	01	1	1	X	1
	11			X	X
	10	1	1	X	X

$$F = \overline{A}\overline{C}D + \overline{B}\overline{C}D + \overline{A}C\overline{D}$$

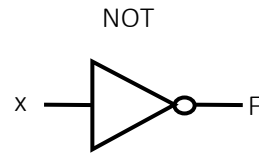
$$F = \overline{C}D + C\overline{D}$$

MICROELECTRONICS – LOGIC GATES CONSTRUCTION

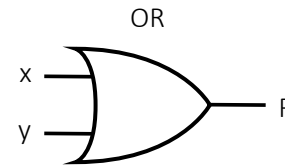
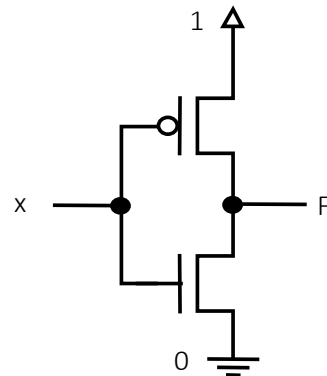
Logic Gate
(symbol)

Truth
Table

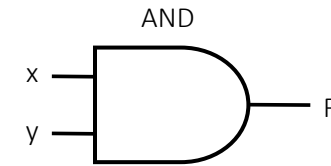
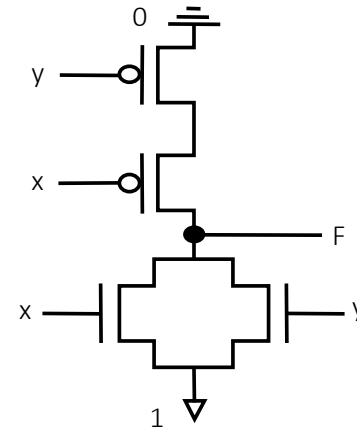
Physical
Implementation
(circuit)



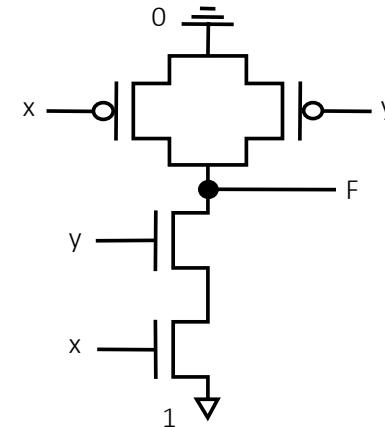
x	F
0	1
1	0



x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



x	y	F
0	0	0
0	1	0
1	0	0
1	1	1



COMBINATIONAL LOGIC CIRCUITS

- Output value depends only on the inputs value (no “internal state”)
- Output value is updated “immediately” (after a propagation time)
- The combinational logic circuit can be described using truth tables or Boolean expressions
- The Boolean expressions are optimized and simplified before implementing the circuit
- Examples:
 - Converters
 - Decoders
 - Encoders
 - Multiplexers
 - Adders
 - Comparators
 - Others arithmetic operators
 - ...

COMBINATIONAL LOGIC CIRCUITS

➤ BINARY TO 7 SEGMENT DECODER EXAMPLE

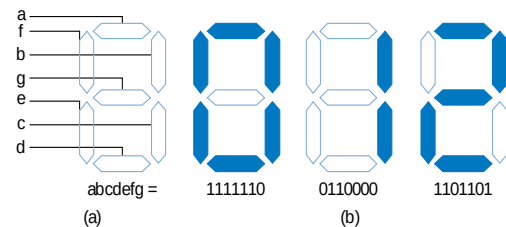


TABLE 2-4 4-bit binary number to seven-segment display truth table

D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0



$$\begin{aligned}
 a &= m_0 + m_2 + m_3 + m_5 + m_6 + m_7 + m_8 + m_9 \\
 b &= m_0 + m_1 + m_2 + m_3 + m_4 + m_7 + m_8 + m_9 \\
 c &= m_0 + m_1 + m_3 + m_4 + m_5 + m_6 + m_7 + m_8 + m_9 \\
 d &= m_0 + m_2 + m_3 + m_5 + m_6 + m_8 + m_9 \\
 e &= m_0 + m_2 + m_6 + m_8 \\
 f &= m_0 + m_4 + m_5 + m_6 + m_8 + m_9 \\
 g &= m_2 + m_3 + m_4 + m_5 + m_6 + m_8 + m_9
 \end{aligned}$$



$$a = m_0 + m_2 + m_3 + m_5 + m_6 + m_7 + m_8 + m_9$$

		DC			
		00	01	11	10
BA	00	1		X	1
	01		1	X	1
	11	1	1	X	X
	10	1	1	X	X

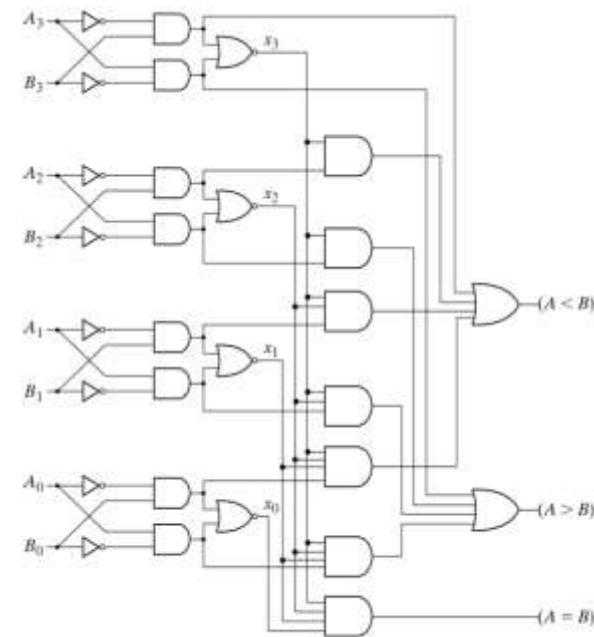
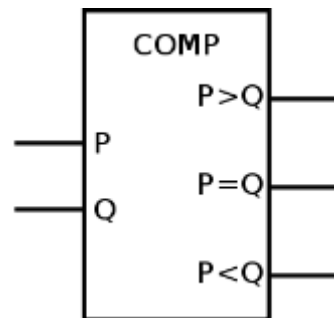
$$a = B + D + C.A + \bar{C}.\bar{A}$$

... b, c, d, e f, g = ...

COMBINATIONAL LOGIC CIRCUITS

➤ COMPARATOR

- There are different types of arithmetic comparators
- Used to compare single bits or bit vectors
 - Bit vectors can optionally be compared as signed values
- The output shows if the comparison is true or false



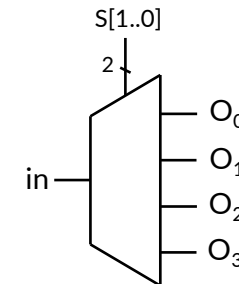
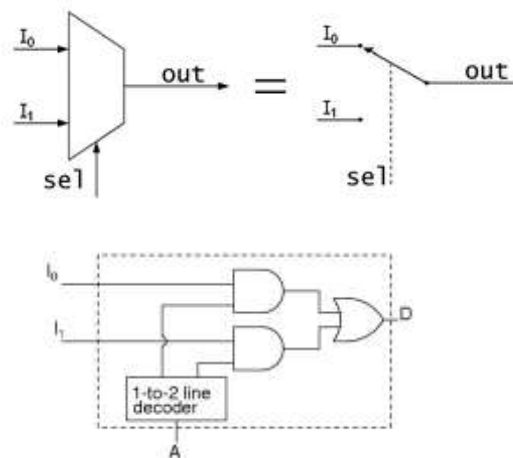
4-Bit Magnitude Comparator

COMBINATIONAL LOGIC CIRCUITS

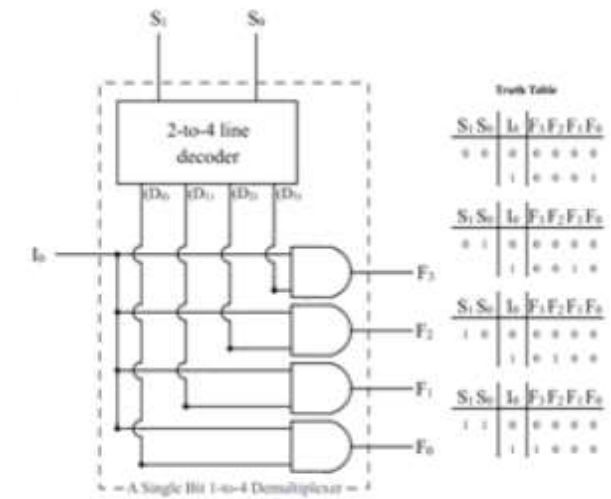
- **Multiplexer**
 - Selects which of the input signals will be presented at the output.
- **Demultiplexer**
 - It selects on which of the outputs the input signal will be presented.

The control signal for both is binary encoded as an unsigned number

Multiplexer 2x1



Demultiplexer 1x4



COMBINATIONAL LOGIC CIRCUITS

➤ Adder

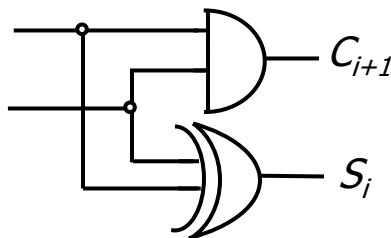
- It executes the addition or subtraction operations using Two's Complement
- From a two-bit adder, it is possible to build larger adders
- Similar to the addition in decimal base, the sum of two bits may “not fit” in one bit, resulting in the well-known “go one”, called **Carry-Out**
- So, the next sum operation occurs with three bits: two operands and a Carry-Out from the previous sum operation, called **Carry-In**

Half adder: adder without Carry-In

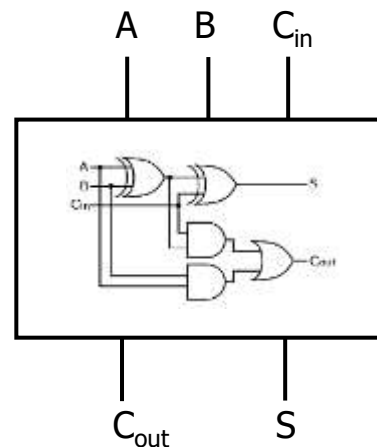
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B$$

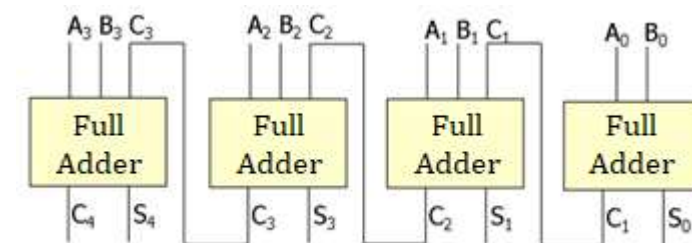
$$C = A \cdot B$$



Full adder



Ripple Carry Adder: Two or more adders are cascaded together to add an N-bit number, the Carry-Out of each full adder is the Carry-In of the succeeding next full adder.



BEFORE PROGRAMMABLE LOGIC DEVICE

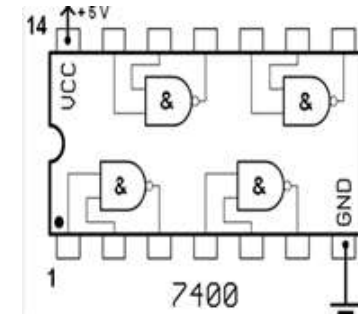
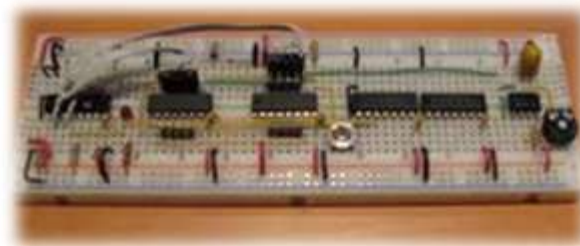
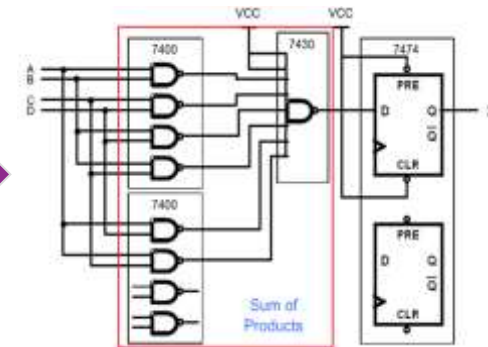
- TTL devices are bulky and consumes a lot of board area and power
- Everything must be done manually

A	B	C	D	X
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

AB \ CD	CD			
	00	01	11	10
00	0	0	1	0
01	0	1	1	1
11	1	1	1	1
10	0	1	1	1

$$X = AB + CD + BD + BC + AD + AC$$

$$X = \overline{(AB \cdot \overline{CD} \cdot \overline{BD} \cdot \overline{BC} \cdot \overline{AD} \cdot \overline{AC})}$$



BEFORE PROGRAMABLE LOGIC DEVICE

- ASIC development is expensive
 - High initial cost (unit cost is low for large volumes)
 - Prototyping is very costly
 - Errors must be avoided at all costs!
 - Simulation takes a significant amount of time
 - Requires a larger development team

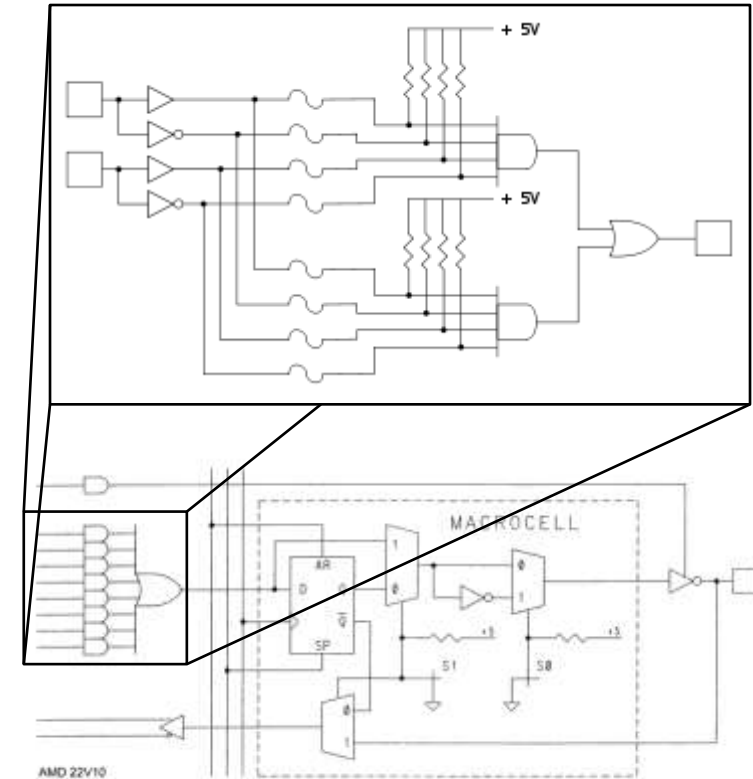
- Hardware design is just as important as logic and architecture

- Fabrication schedule is fixed

- Time to market is long

FIRST PROGRAMMABLE LOGIC DEVICE

- A single programmable logic device could implement different logic functions
- PAL stands for Programmable Array Logic
- Devices with arrays of PALs and additional routing resources started to become available – the CPLDs
- **CPLD** stands for **Complex Programmable Logic Device**
- Nowadays CPLDs are more complex and very similar to FPGAs' architecture
- **FPGA** stands for **Field Programmable Gate Arrays**



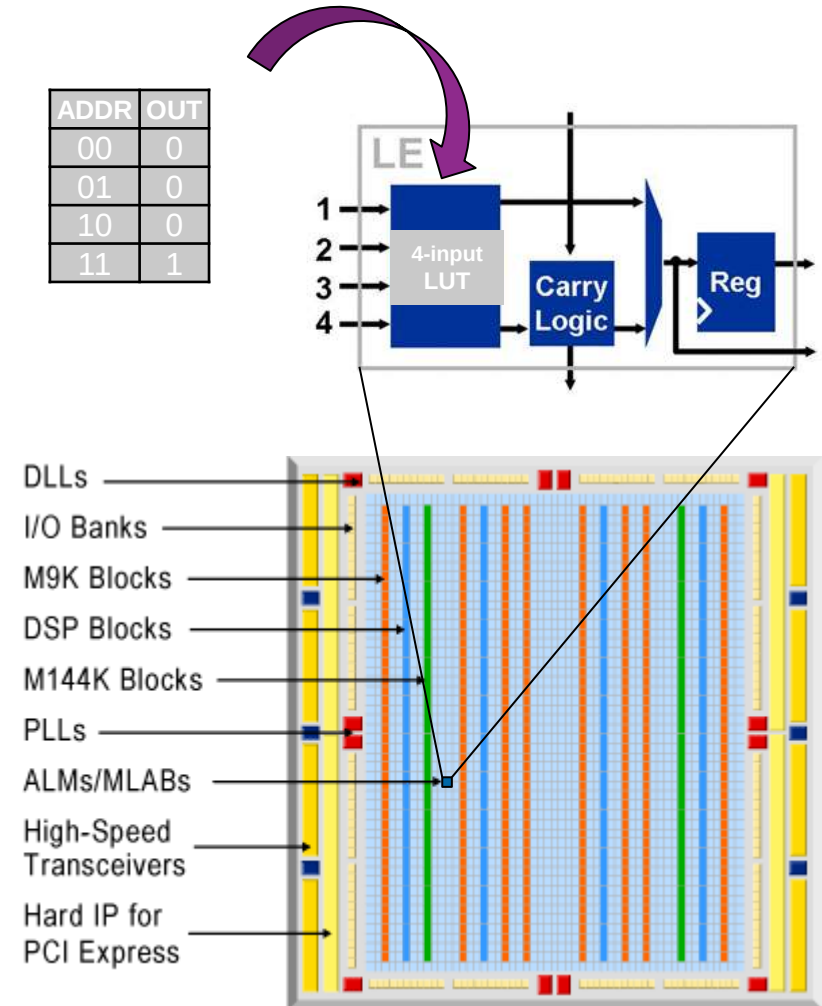
CPLDs AND FPGAs

- More advanced devices with more resources made **more complex designs** possible
- Early hardware description languages were **too “low level”**
- Higher level hardware description languages were created, namely:
 - **VHDL** based on ADA software development language
 - **Verilog** based on C software development language

Both have very similar features but very different syntax

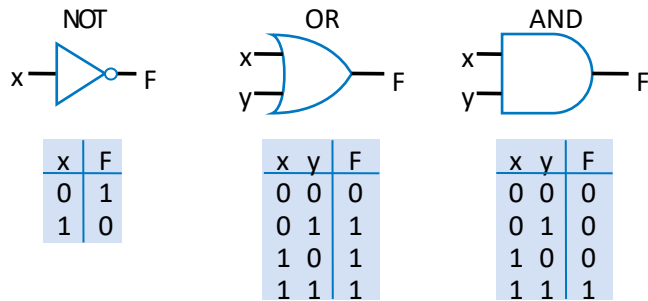
FPGA INTERNALS

- FPGA basic elements are Look-Up Tables (**LUT**)
- Logic Elements (**LE** or **ALM**) contain LUTs, registers, and glue logic
- Besides LEs an FPGA also has **SRAM memory blocks**, **multipliers**, **PLLs**, hardware interfaces, and a lot of interconnect
- LUT outputs and other peripherals configurations are stored in **SRAM**



FPGA INTERNALS

- A look-up table can model any logic gate!

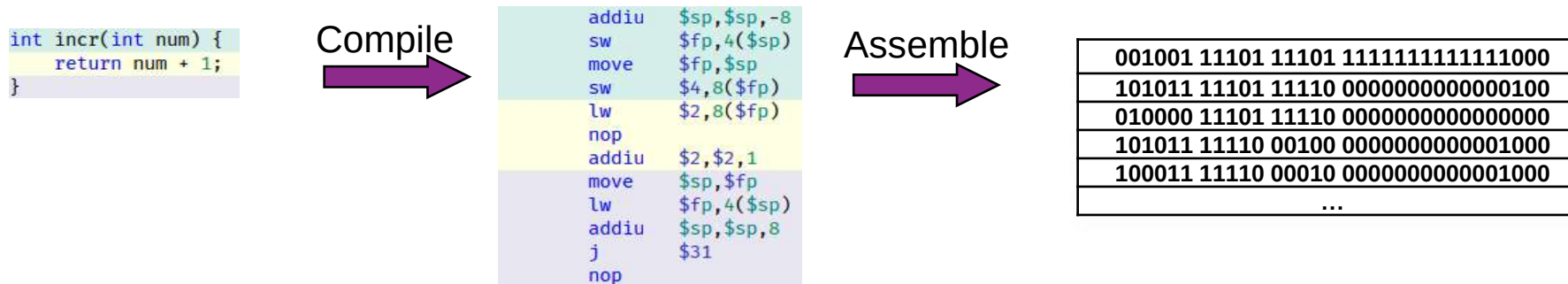


- Different FPGA families have different LUT architecture (input connections and number of inputs, ranging from 4 to 8)
- A LUT can be split in two (or more, depending on the FPGA family) for different operation modes

CPU SOFTWARE vs. FPGA RTL

COMPUTER ARCHITETURE

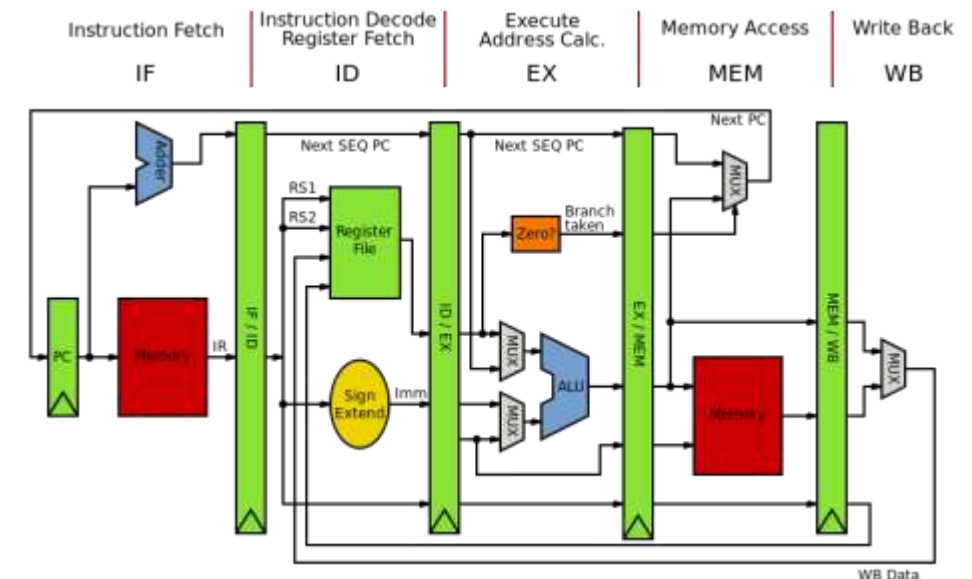
- The processor runs operations from the instructions
- Instructions are generated by the compiler from source code.
 - Source code is compiled to assembly and the assembly code is translated directly to binary instructions
- Each processor architecture has a specific set of instruction



Instrução	FORMATO DA INSTRUÇÃO						AÇÃO
	31 - 26	25 - 21	20 - 16	15 - 11	10 - 6	5 - 0	
ADDU Rd, Ra, Rt	00	Ra	Rt	Rd	00	21	$Rd \leftarrow Ra + Rt$
SUBU Rd, Ra, Rt	00	Ra	Rt	Rd	00	23	$Rd \leftarrow Ra - Rt$
ADDIU Rt, Ra, lmed16	09	Ra	Rt	lmed16			$Rt \leftarrow Ra + (lmed16 \text{ com sinal estendido})$
ANDI Rt, Ra, lmed16	0C	Ra	Rt	lmed16			$Rt \leftarrow Ra \text{ and } (0x0000 \& (lmed16))$
LW Rt, lmed16(Ra)	23	Ra	Rt	lmed16			$Rt \leftarrow PMEMD(lmed16 \text{ com sinal estendido} + Ra) \text{ (4 bytes)}$
SB Rt, lmed16(Ra)	28	Ra	Rt	lmed16			$PMEMD(lmed16 \text{ com sinal estendido} + Ra) \leftarrow Rt [7:0] \text{ (1 byte)}$
SW Rt, lmed16(Ra)	2B	Ra	Rt	lmed16			$PMEMD(lmed16 \text{ com sinal estendido} + Ra) \leftarrow Rt \text{ (4 bytes)}$
J rótulo	02	lmed26					$PC \leftarrow NPC[31:28] \& lmed26 \& "00"$
JAL rótulo	03	lmed26					$\$31 \leftarrow NPC; PC \leftarrow NPC[31:28] \& lmed26 \& "00"$

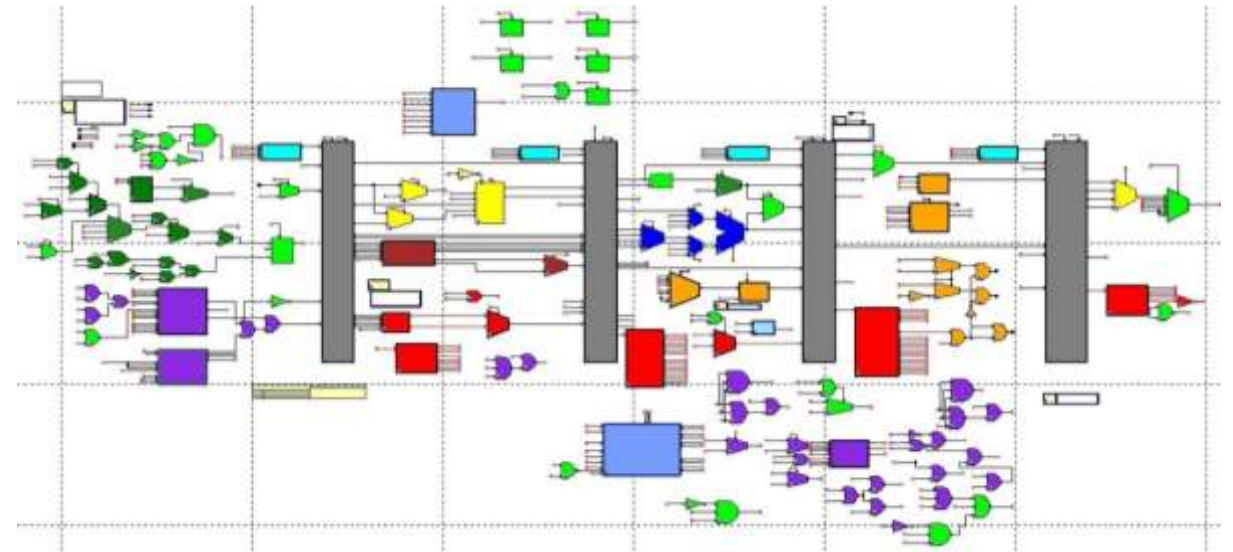
COMPUTER ARCHITETURE

- Instructions are read from a specific memory address
- The **Program Counter (PC)** register holds the memory address of the next instruction to be run
- Upon **reset**, the PC register returns to the **default value**, which is the address where the initialization software is found (ex: BIOS, bootloader, boot copier)



COMPUTER ARCHITETURE

- The processor's components are built from **digital logic** (logic gates, registers, memory)
- The processor's operations are controlled by a **clock** (synchronous digital circuit)
- A programmable logic device such as a **FPGA** can be **configured to be used as a processor** (or multiple processors!)

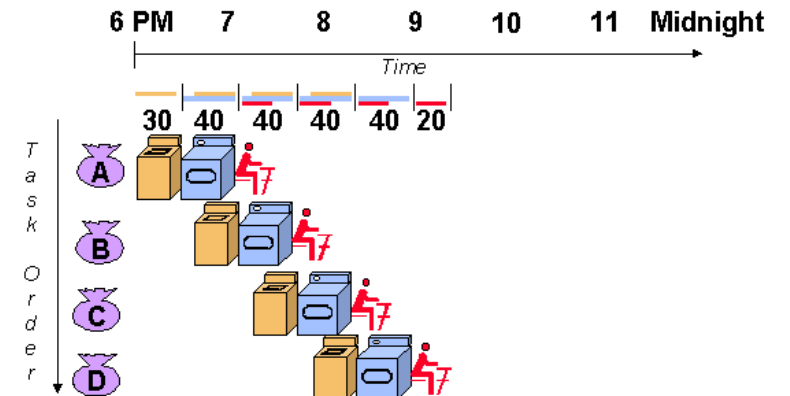
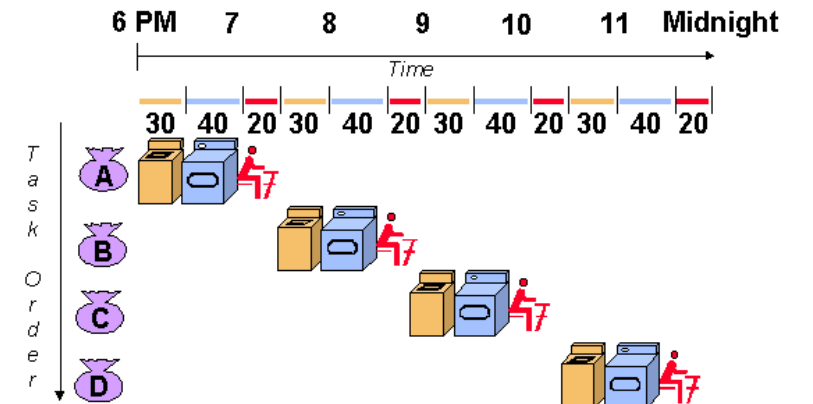


COMPUTER ARCHITETURE

- In computer processors, **pipelines** are used to increase instruction *throughput*
 - The processor is divided in stages
 - Instructions goes through every stage during execution
 - Different instructions are running at the same time on different pipeline stages.

Pipelines are also used in other digital logic structures, including FPGA designs

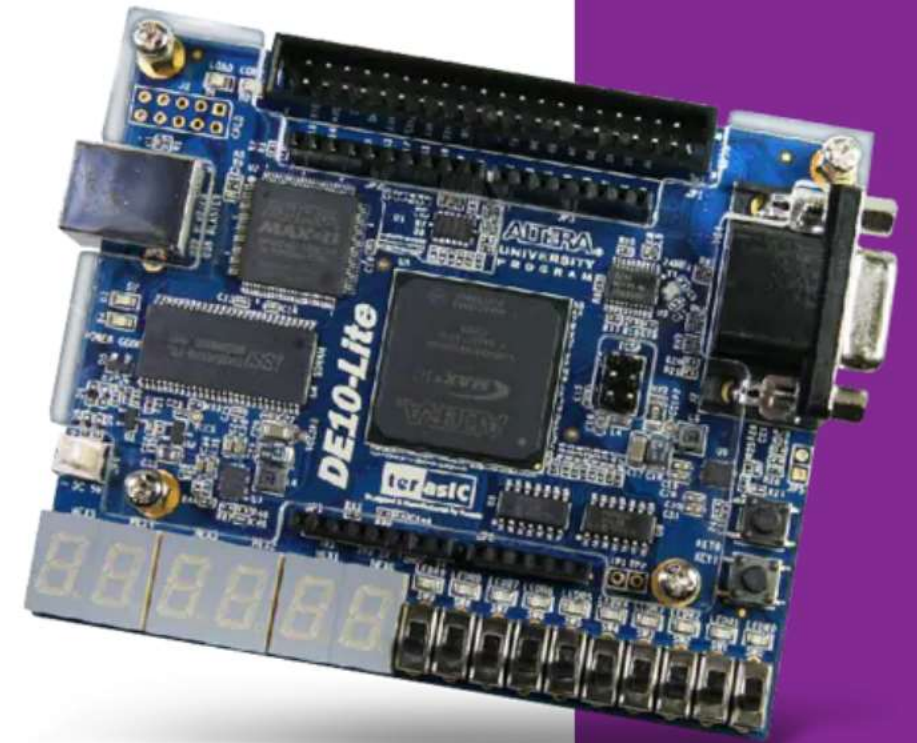
- Ex: complex arithmetic operators, like division



FPGA ARCHITECTURE:

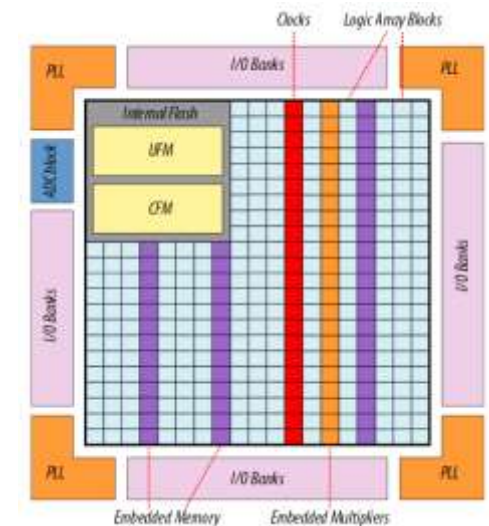
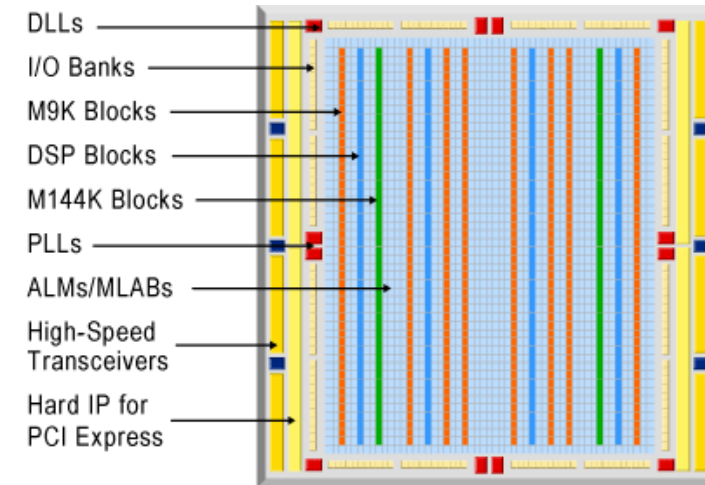
HARDWARE RESOURCES:

- LOOK-UP TABLES
- REGISTERS
- MULTIPLIERS
- INTERNAL MEMORY



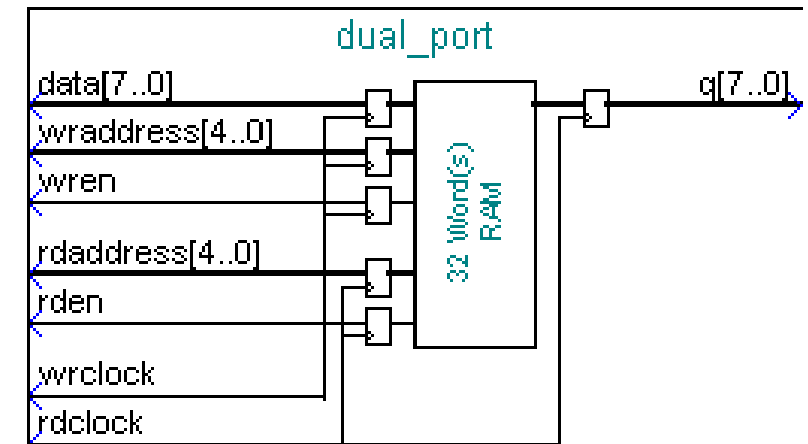
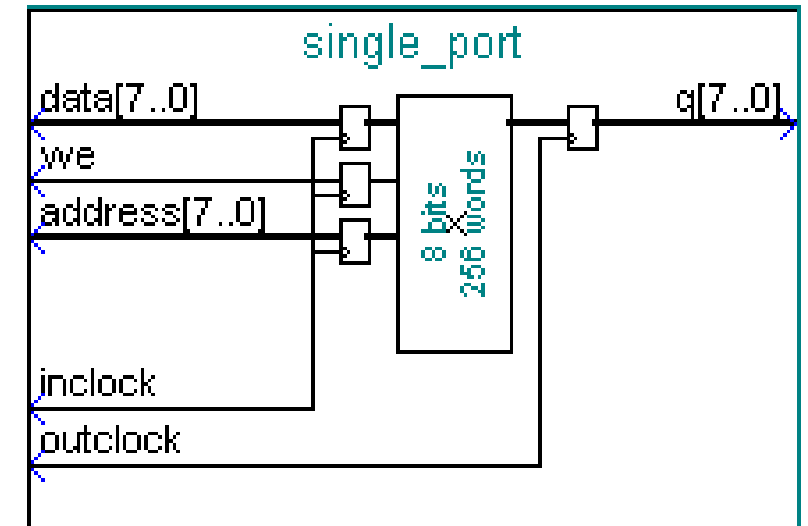
FPGA RESOURCES

- LE and ALM: the basic construction block, containing one (LE) or more (ALM) LUTs and registers
- M4K, M9K, MLAB: on-chip SRAM memory blocks, which can work with different widths and can be grouped
- Multipliers and DSP blocks: hardware specialized in multiplication and multiply-accumulate operations
- PLL and FPLL: clock synthesis hardware, can be used to generate clock frequencies other than the supplied ones
- I/O cells: connected to each FPGA pin, implements input and output buffers, DDR I/O, different signaling levels, etc
- Hardware interfaces: DDR interface, PCI-express, transceivers...



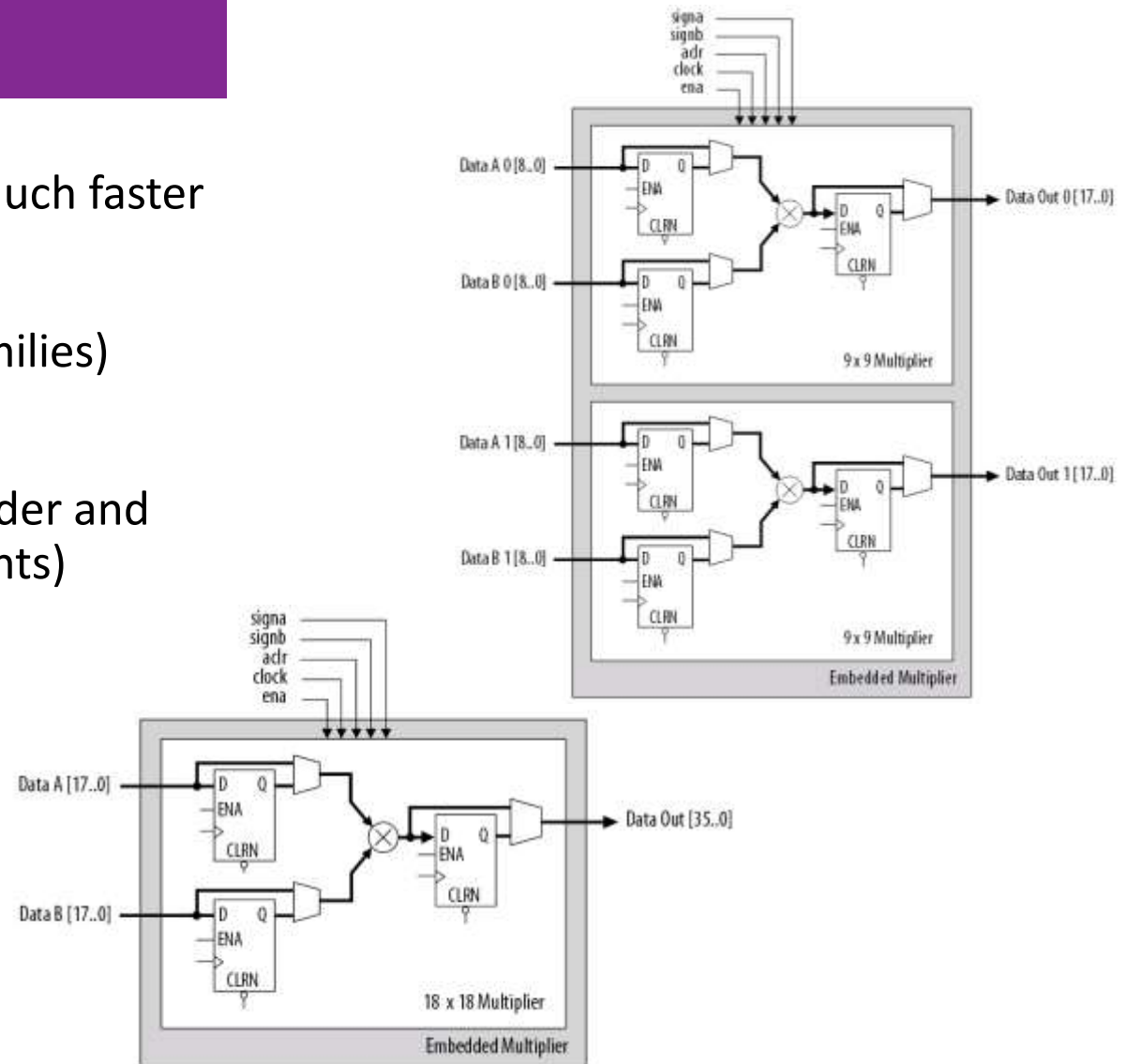
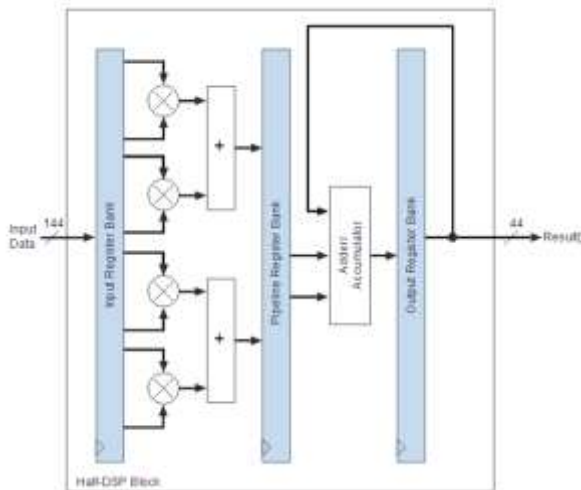
BLOCK MEMORY

- Internal SRAM memory is divided in fixed size blocks
- Memory blocks can be combined to create a larger memory
- Data width can be selected from fixed values
- Blocks can operate in single or dual port mode
- Several features can be configured at instantiation time



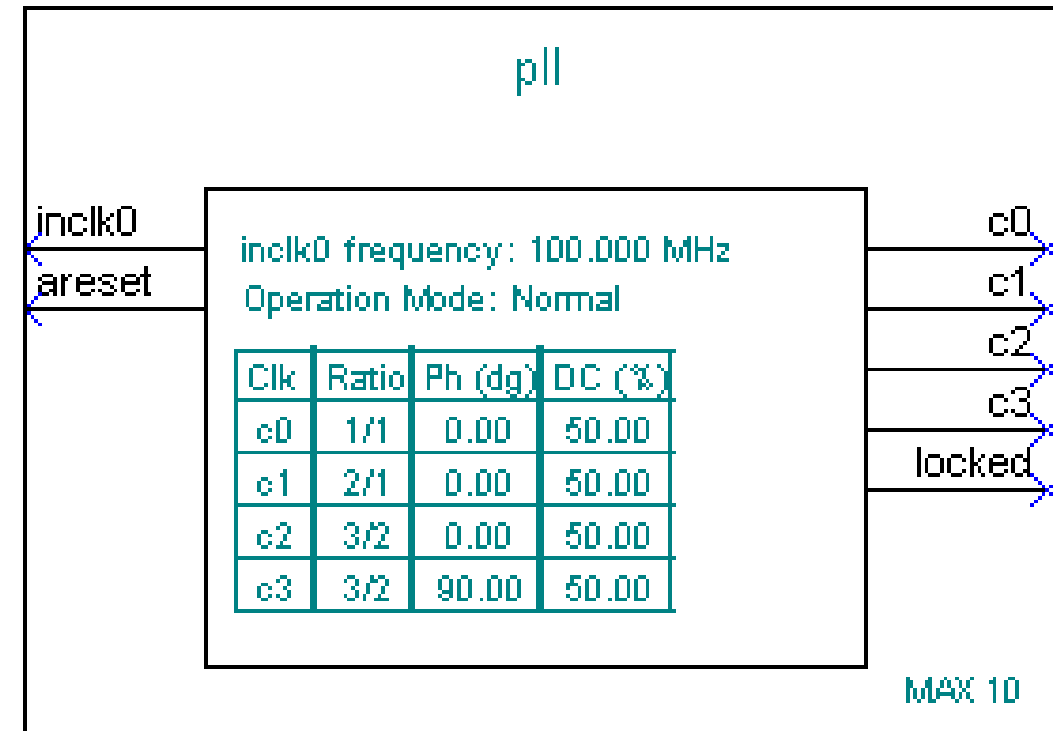
MULTIPLIERS

- Dedicated hardware which performs much faster
- Can be grouped in couples (in most families)
- Some devices have four multipliers, adder and register (a MAC, also called DSP Elements)



PLLs

- Can multiply and divide external clock frequencies
- Each PLL can generate several output clocks
- Clock duty cycle and phase can be adjusted
- Several clock compensation modes



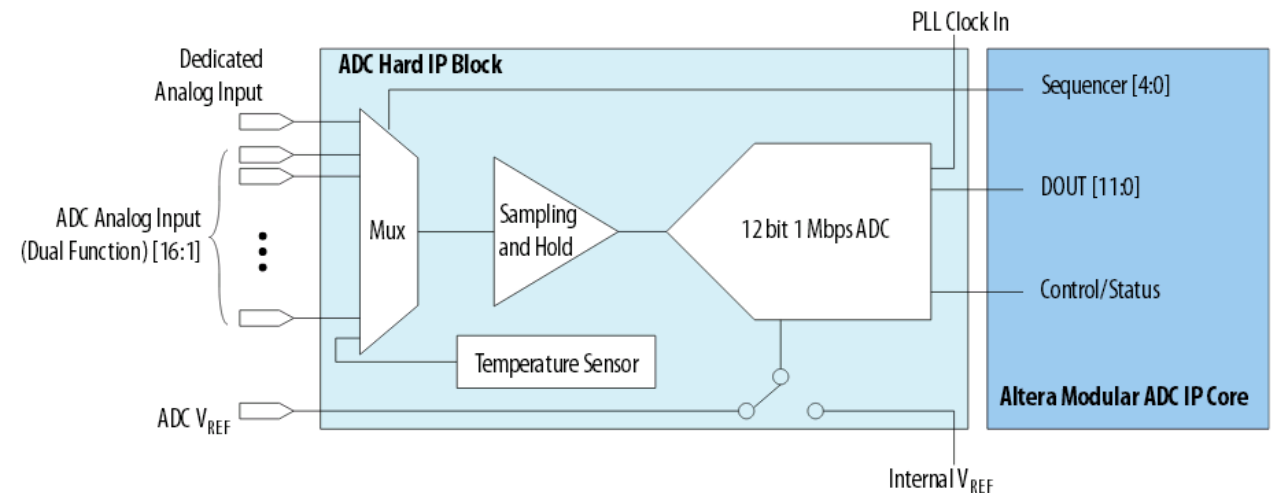
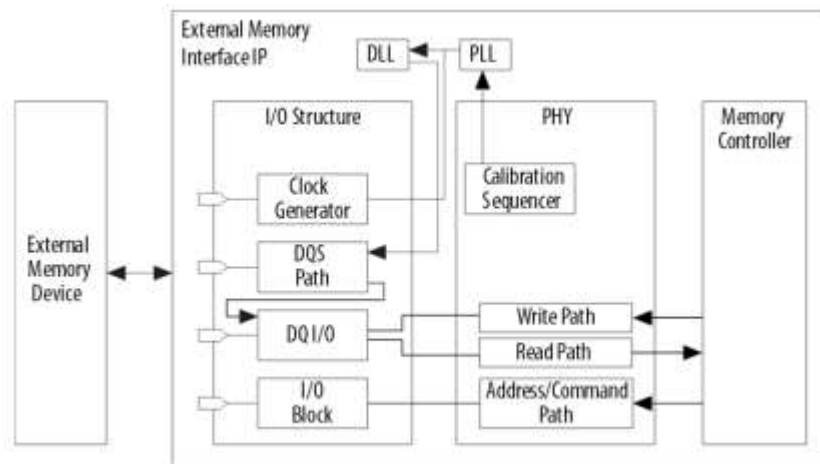
HARD-IPs

➤ Hard-IP examples:

- Dedicated Memory Controller can deal with higher frequency memories
- Dedicated PCI-Express controller

➤ Save Logic Elements for other uses

➤ Internal Analog to Digital Converter

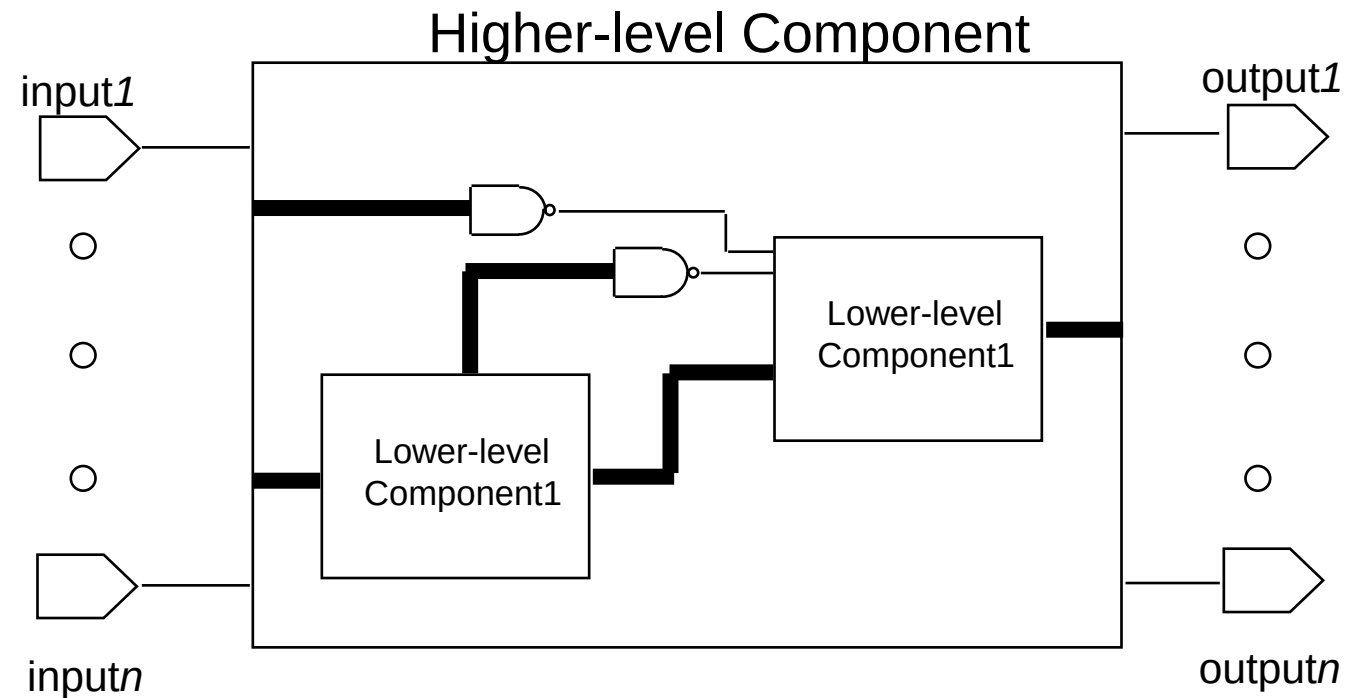


HARDWARE DESIGN TERMINOLOGY

- **HDL (Hardware Description Language):** A software programming language that is used to model a piece of hardware
- **Behavior Modeling:** A component is described by its input/output response
- **Structural Modeling:** A component is described by interconnecting lower-level components/primitives

STRUCTURAL MODELING

- Functionality and structure of the circuit
- Call out the specific hardware



VERILOG - BASIC MODELING STRUCTURE

module *module_name*
 (*port_declarations*);

data type declarations

circuit functionality

timing specifications

endmodule

Verilog syntax

- Begins with keyword **module** & ends with keyword **endmodule**
- Case-sensitive
- All keywords are lowercase
- Whitespace () is used for readability
- Semicolon (;) is the statement terminator
- **//** : Single line comment
- **/* */** : Multi-line comment
- Timing specification is for simulation (not discussed)

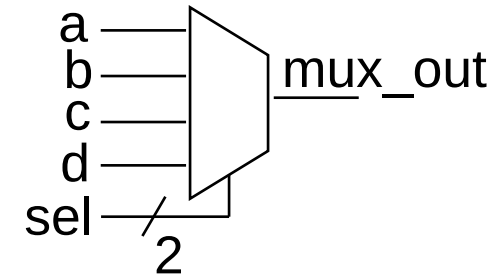
MORE TERMINOLOGY

- **Register Transfer Level (RTL):** A type of behavioral modeling, for the purpose of synthesis
 - Hardware is implied or inferred
 - Synthesizable
- **Synthesis (logic):** Translating HDL to a circuit (gate-level description) and then optimizing the represented circuit
- **RTL Synthesis (physical):** Translating a RTL model of hardware into an optimized technology specific gate level implementation

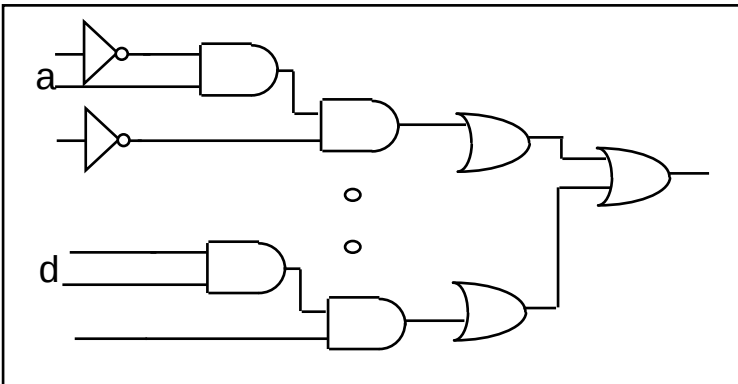
RTL SYNTHESIS

```
always @(a, b, c, d, sel)
  case (sel)
    2'b00: mux_out = a;
    2'b01: mux_out = b;
    2'b10: mux_out = c;
    2'b11: mux_out = d;
  endcase
```

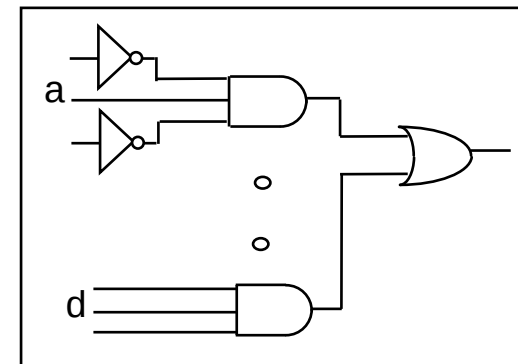
Logic synthesis



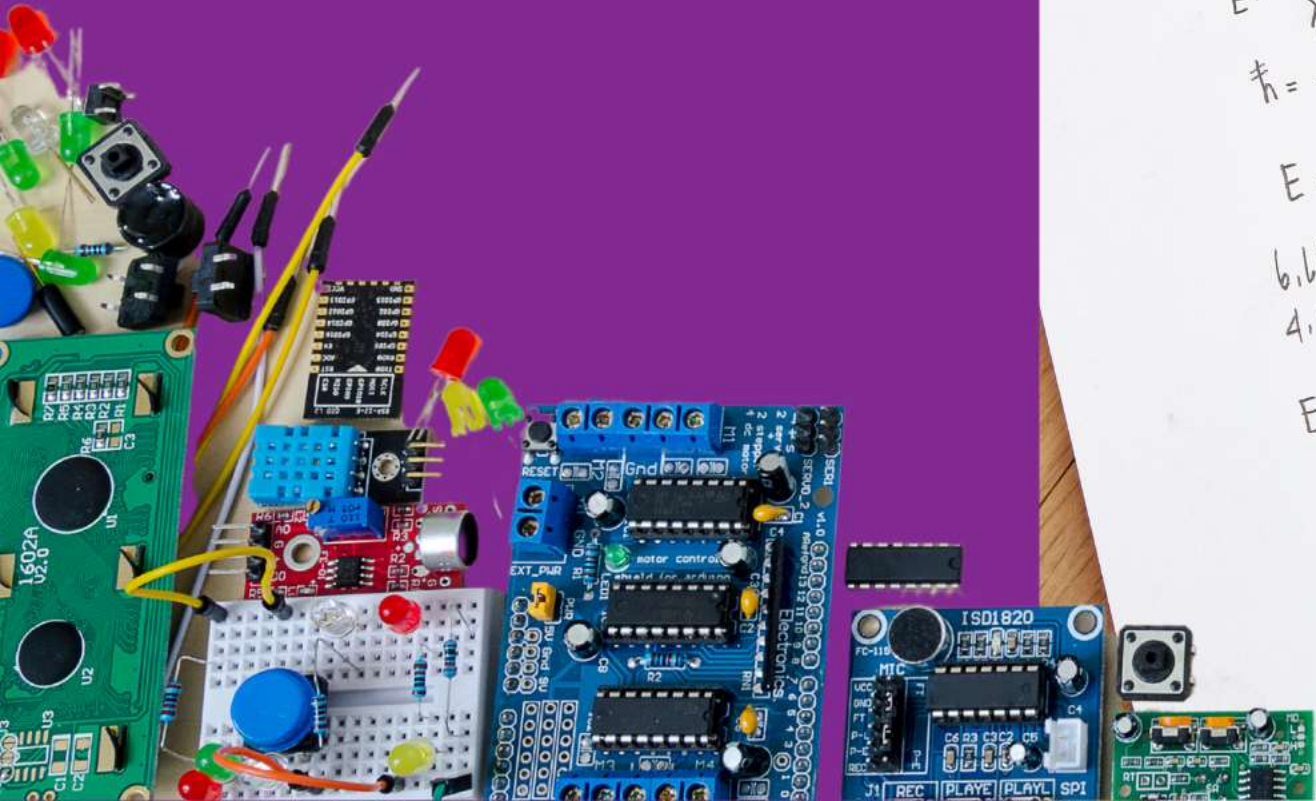
Physical synthesis



Optimization



VERILOG CONSTANT NUMBERS



PLANCK CONSTANT

$$E = h\nu$$

$$E = \frac{hc}{\lambda}$$

$$\hbar = \frac{h}{2\pi}$$

$$E = \hbar\omega$$

$$6.62606896(33) \times 10^{-34} \text{ J}\cdot\text{s}$$

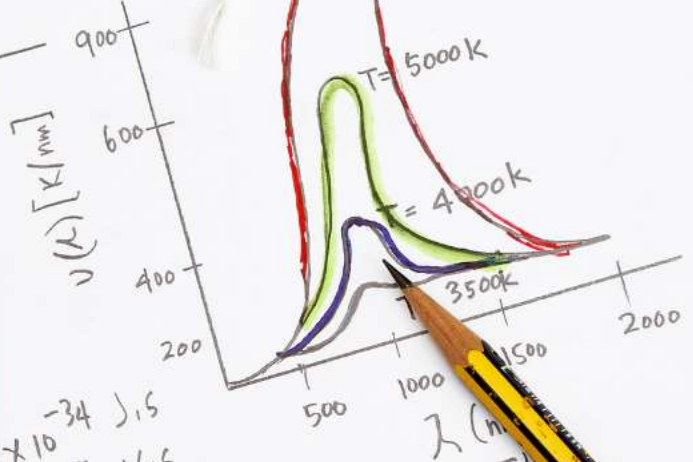
$$4.13566733(10) \times 10^{-15} \text{ eV}\cdot\text{s}$$

$$E_n = \frac{hcR_\infty}{n^2}$$

$$\Delta x \Delta p \geq \frac{1}{2}\hbar$$

$$e = \sqrt{\frac{2\alpha\hbar}{m_0c_0}}$$

$$M_n = \mu_B \frac{A_r(e)}{A_r(p)}$$



BLACK BODY RADIATION

$$R_\infty = \frac{m_e e^4}{8\epsilon_0^2 h^3 c_0} = \frac{m_e c_0 \alpha^2}{2\hbar}$$

$$m_e = \frac{2R_\infty \hbar}{c_0 \alpha^2}$$

$$\sigma = \frac{\pi^2}{15} \frac{k_B^4}{15\hbar^3 c_0^2}$$

ASSIGNING VALUES - NUMBERS

- Are **sized** or **unsized**: **<size>'<base format><number>**
 - **Sized** example: **3'b010** = 3-bit wide binary number
 - The prefix (3) indicates the size of number
 - **Unsized** example: **123** = 32-bit wide decimal number by default
 - **Defaults**
 - No specified <base format> defaults to **decimal**
 - No specified <size> defaults to **32-bit** wide number
- Base Formats
 - Decimal ('d or 'D) **16'd255** = 16-bit wide decimal number
 - Hexadecimal ('h or 'H) **8'h9a** = 8-bit wide hexadecimal number
 - Binary ('b or 'B) **'b1010** = 32-bit wide binary number
 - Octal ('o or 'O) **'o21** = 32-bit wide octal number
 - Signed ('s or 'S) **16'shFA** = signed 16-bit hex value

NUMBERS

- Negative numbers - specified by putting a minus sign before the <size>
 - **Legal:** `-8'd3` = 8-bit negative number stored as 2's complement of 3
 - **Illegal:** `4'd-2` = **ERROR!!**

- Special Number Characters
 - `'_'` (underscore): used for readability
 - Example: `32'h21_65_bc_fe` = 32-bit hexadecimal number
 - `'x'` or `'X'` (unknown value)
 - Example: `12'h12x` = 12-bit hexadecimal number; LSBs unknown
 - `'z'` or `'Z'` (high impedance value)
 - Example: `1'bz` = 1-bit high impedance number

NUMBERS EXTENSION

- If MSB is **0**, **x**, or **z**, number is extended to fill MSBs with **0**, **x**, or **z**, respectively
 - Examples
 - **3'b01** is equal to **3'b001**
 - **3'bx1** is equal to **3'bxx1**
 - **3'bz** is equal to **3'bzzz**

- If MSB is **1**, number is extended to fill MSBs with **0**
 - Example
 - **3'b1** is equal to **3'b001**



SHORT QUIZ

What is the actual value
for 4'd017 in binary?

SHORT QUIZ ANSWER

What is the actual value for 4'd017 in binary?

4'b0001, MSB is truncated (10001)



SHORT QUIZ 2

```
module mult_acc (  
    input [7:0] data_in,  
    output [7:0] data_out  
);  
assign undeclared_wire = data_in;  
assign data_out = undeclared_wire;  
...
```

What is the actual value for **data_out**
when **data_in** is 8'd163 ?

Tip:

8'd163 = 8'b10100011

SHORT QUIZ 2 ANSWER

```
module mult_acc (  
    input [7:0] data_in,  
    output [7:0] data_out  
);  
assign undeclared_wire = data_in;  
assign data_out = undeclared_wire;  
...
```

What is the actual value for **data_out**
when **data_in** is 8'd163 ?

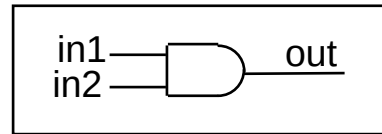
8'b00000001,
as the undeclared_wire has width=1,
everything else is truncated!

A blue ballpoint pen is positioned diagonally across the frame, resting on a technical circuit diagram. The diagram features various electronic components, including integrated circuits labeled '2 JA3 SE' and '2 JA3', and several logic gates. Some gates are labeled 'TM2' and '4B'. The diagram also includes numerical values like '10', '12', and '15', and some text like 'PI-HI*SI-HO*=.1"'. The background is a light yellowish-green, and the overall image has a soft, slightly blurred aesthetic.

VERILOG “LOGIC GATES”: OPERATORS

RTL SYNTHESIS

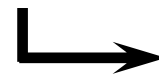
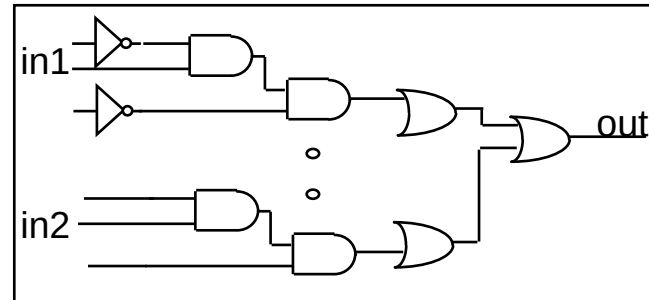
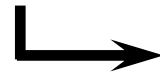
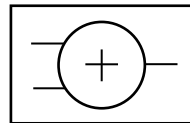
```
assign out = in1 && in2;
```



2-input LUT

ADDR	OUT
00	0
01	0
10	0
11	1

```
assign out = in1 + in2;
```



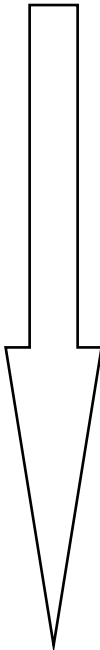
ADDR	OUT
00	0
01	0
10	0
11	1

OPERATORS

- Arithmetic
- Bitwise
- Reduction
- Relational
- Equality
- Logical
- Shift
- Miscellaneous

*Used in **expressions** to describe behavior model*

OPERATOR PRECEDENCE

Operator(s)	Priority	
+ - ! ~ & ~& etc. (unary* operators)	High  Low	
**		
* / %		
+ - (binary operators)		
<< >> <<< >>>		
< > <= >=		
== != === !==		
& (binary operator)		
^ ~^ ^~ (binary operators)		
(binary operator)		
&&		
?:		
{ } { } { }		* <i>Unary operators have only one operand</i>

() used to override default and provide clarity

REDUCTION OPERATORS

For Your Reference

Operator Symbol	Functionality	Examples ain = 4'b1010 ; bin = 4'b10xz ; cin = 4'b111z		
&	AND all bits	&ain ⇒ 1'b0	&bin ⇒ 1'b0	&cin ⇒ 1'bx
~&	NAND all bits	~&ain ⇒ 1'b1	~&bin ⇒ 1'b1	~&cin ⇒ 1'bx
 	OR all bits	ain ⇒ 1'b1	bin ⇒ 1'b1	cin ⇒ 1'b1
~ 	NOR all bits	~ ain ⇒ 1'b0	~ bin ⇒ 1'b0	~ cin ⇒ 1'b0
^	XOR all bits	^ain ⇒ 1'b0	^bin ⇒ 1'bx	^cin ⇒ 1'bx
^~ or ~^	XNOR all bits	~^ain ⇒ 1'b1	~^bin ⇒ 1'bx	~^cin ⇒ 1'bx

- Reduces a vector to a single bit value
- X or Z are both considered unknown in operands, but result maybe a known value

BITWISE OPERATOR

For Your Reference

Operator Symbol	Functionality	Examples ain = 3'b101 ; bin = 3'b110 ; cin = 3'b01x	
~	Invert each bit	$\sim \text{ain} \Rightarrow 3\text{b}'010$	$\sim \text{cin} \Rightarrow 3\text{b}'10\text{x}$
&	AND each bit	$\text{ain} \& \text{bin} \Rightarrow 3\text{b}'100$	$\text{bin} \& \text{cin} \Rightarrow 3\text{b}'010$
	OR each bit	$\text{ain} \text{bin} \Rightarrow 3\text{b}'111$	$\text{bin} \text{cin} \Rightarrow 3\text{b}'11\text{x}$
^	XOR each bit	$\text{ain} \wedge \text{bin} \Rightarrow 3\text{b}'011$	$\text{bin} \wedge \text{cin} \Rightarrow 3\text{b}'10\text{x}$
$\wedge \sim$ or $\sim \wedge$	XNOR each bit	$\text{ain} \wedge \sim \text{bin} \Rightarrow 3\text{b}'100$	$\text{bin} \sim \wedge \text{cin} \Rightarrow 3\text{b}'01\text{x}$

- Operates on each bit or bit pairing of the operand(s)
- Result is the size of the largest operand
- X or Z are both considered unknown in operands, but result maybe a known value
- Operands are left-extended if sizes are different

ARITHMETIC OPERATORS

For Your Reference

Operator Symbol	Functionality	Examples <code>ain = 5 ; bin = 10 ; cin = 2'b01 ; din = 2'b0z</code>		
+	Add, Positive	<code>bin + cin ⇒ 11</code>	<code>+bin ⇒ 10</code>	<code>ain + din ⇒ x</code>
-	Subtract, Negate	<code>bin - cin ⇒ 9</code>	<code>-bin ⇒ -10</code>	<code>ain - din ⇒ x</code>
*	Multiply	<code>ain * bin ⇒ 50</code>		
/	Divide*	<code>bin / ain ⇒ 2</code>		
%	Modulus	<code>bin % ain ⇒ 0</code>		
**	Exponent*	<code>ain ** 2 ⇒ 25</code>		

- Treats vectors as a whole value
- Results unknown if any operand is Z or X
- Carry bit(s) handled automatically if result wider than operands
- Carry bit lost if operands and results are same size

* Check synthesis tool for support

RELATIONAL OPERATORS

For Your Reference

Operator Symbol	Functionality	Examples ain = 3'b101 ; bin = 3'b110 ; cin = 3'b01x	
>	Greater than	ain > bin $\Rightarrow$ 1'b0	bin > cin $\Rightarrow$ 1'bx
<	Less than	ain < bin $\Rightarrow$ 1'b1	bin < cin $\Rightarrow$ 1'bx
>=	Greater than or equal to	ain >= bin $\Rightarrow$ 1'b0	bin >= cin $\Rightarrow$ 1'bx
<=	Less than or equal to	ain <= bin $\Rightarrow$ 1'b1	bin <= cin $\Rightarrow$ 1'bx

- Used to compare values
- Returns a 1 bit scalar value of Boolean true (1) / false (0)
- X or Z are both considered unknown in operands and result is always unknown

EQUALITY OPERATORS

For Your Reference

Operator Symbol	Functionality	Examples ain = 3'b101 ; bin = 3'b110 ; cin = 3'b01x	
==	Equality	ain == bin $\Rightarrow$ 1'b0	cin == cin $\Rightarrow$ 1'bx
!=	Inequality	ain != bin $\Rightarrow$ 1'b1	cin != cin $\Rightarrow$ 1'bx
===	Case equality	ain === bin $\Rightarrow$ 1'b0	cin === cin $\Rightarrow$ 1'b1
!==	Case inequality	ain !== bin $\Rightarrow$ 1'b1	cin !== cin $\Rightarrow$ 1'b0

- Used to compare values
- Returns a 1 bit scalar value of Boolean true (1) / false (0)
- For equality/inequality, X or Z are both considered unknown in operands and result is always unknown
- For case equality/case inequality, X or Z are both considered distinct values and operands must match completely

LOGICAL OPERATORS

For Your Reference

Operator Symbol	Functionality	Examples ain = 3'b101 ; bin = 3'b000 ; cin = 3'b01x		
!	Expression not true	!ain $\Rightarrow$ 1'b0	!bin $\Rightarrow$ 1'b1	!cin $\Rightarrow$ 1'bx
&&	AND of two expressions	ain && bin $\Rightarrow$ 1'b0		bin && cin $\Rightarrow$ 1'bx
	OR of two expressions	ain bin $\Rightarrow$ 1'b1		bin cin $\Rightarrow$ 1'bx

- Used to evaluate single expression or compare multiple expressions
 - Each operand is considered a single expression
 - Expressions with a zero value are viewed as false (0)
 - Expressions with a non-zero value are viewed as true (1)
- Returns a 1 bit scalar value of Boolean true (1) / false (0)
- X or Z are both considered unknown in operands and result is always unknown

SHIFT OPERATORS

For Your Reference

Operator Symbol	Functionality	Examples ain = 3'b101 ; bin = 3'b01x	
<<	Logical shift left	ain << 2 $\Rightarrow$ 3'b100	bin << 2 $\Rightarrow$ 3'bx00
>>	Logical shift right	ain >> 2 $\Rightarrow$ 3'b001	bin >> 2 $\Rightarrow$ 3'b000
<<<	Arithmetic shift left	ain <<< 2 $\Rightarrow$ 3'b100	bin <<< 2 $\Rightarrow$ 3'bx00
>>>	Arithmetic shift right	ain >>> 2 $\Rightarrow$ 3'b111 (signed)	bin >>> 2 $\Rightarrow$ 3'b000 (signed)

- Shifts a vector left or right some defined number of bits
- Left shifts (logical or arithmetic): Vacated positions always filled with zero
- Right shifts
 - Logical: Vacated positions always filled with zero
 - Arithmetic (unsigned): Vacated positions filled with zero
 - Arithmetic (signed): Vacated position filled with sign bit value (MSB value)
- Shifted bits are lost
- Shifts by values of X or Z (right operand) return unknown

MISCELLANEOUS OPERATORS

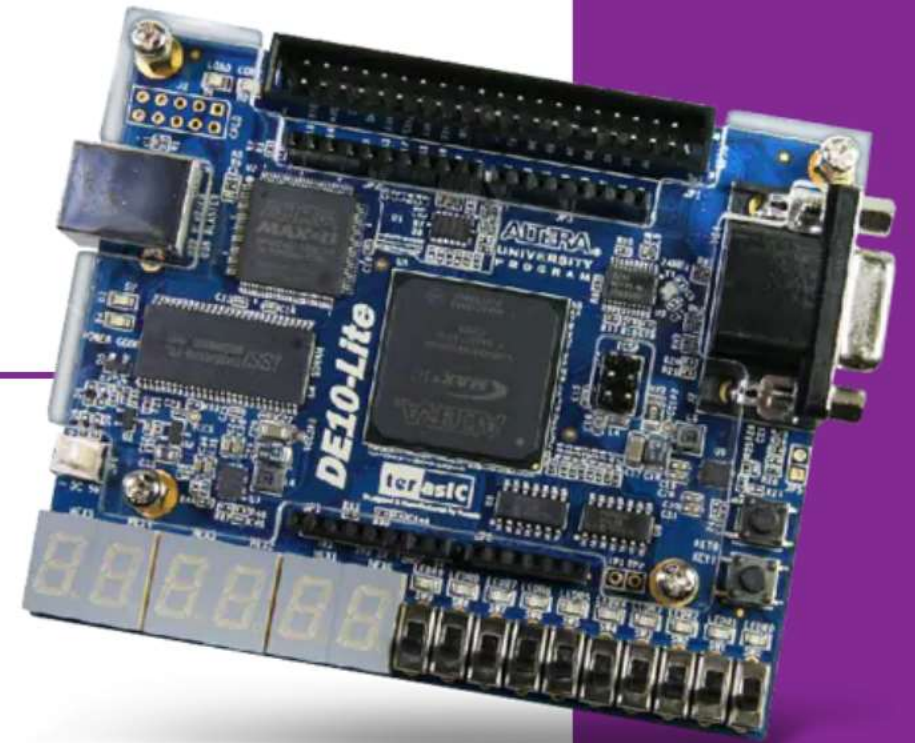
For Your Reference

VERY USEFUL!

Operator Symbol	Functionality	Format & Examples
?:	Conditional test	<i>(condition) ? true_value : false_value</i> sig_out = (sel == 2'b01) ? a : b
{ }	Concatenate	ain = 3'b010 ; bin = 3'b110 {ain,bin} ⇒ 6'b010110
{ { } }	Replicate	{3 {3'b101} } ⇒ 9'b101101101

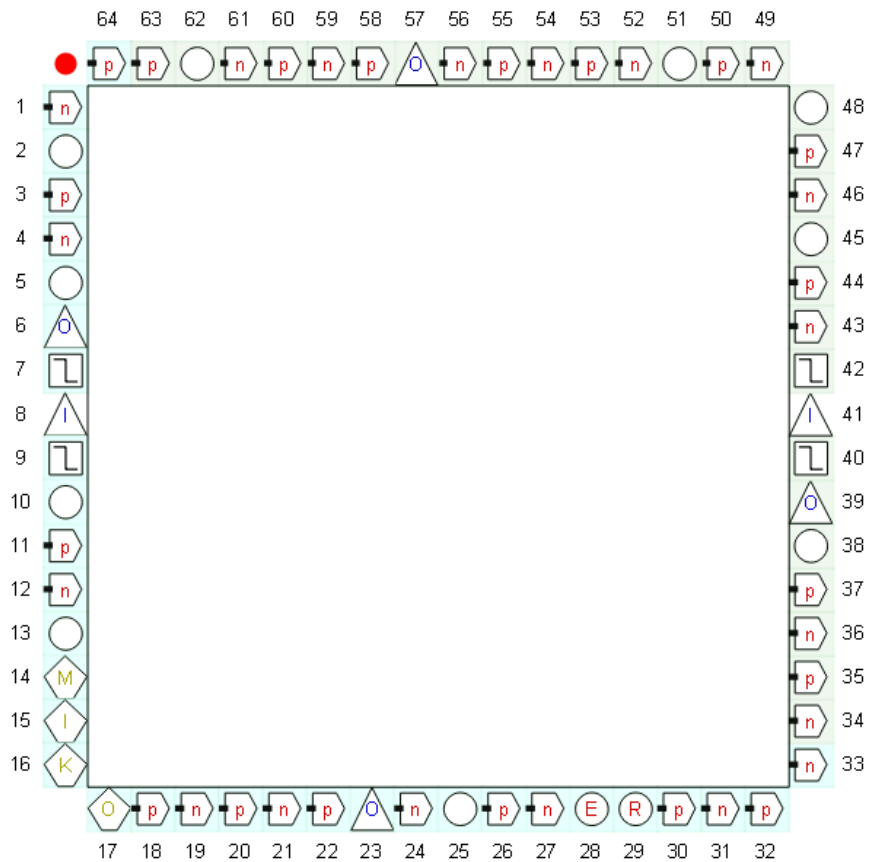
FPGA ARCHITECTURE:

- I/O PINS

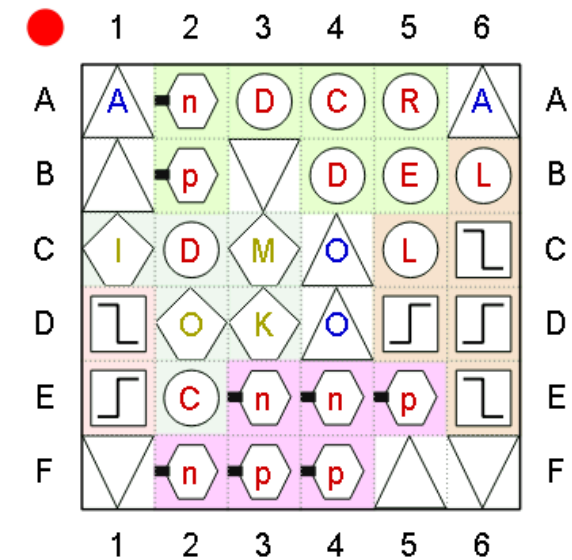


PIN NUMBERING

➤ QFP: single number

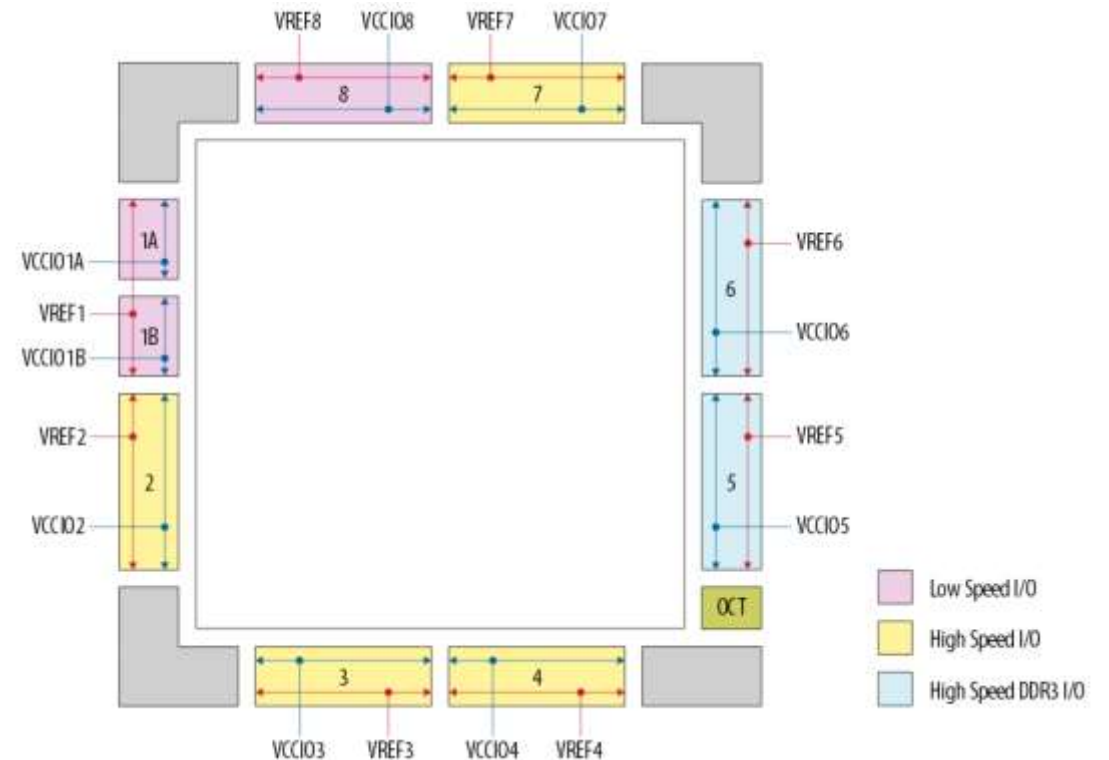


➤ BGA: letter and number

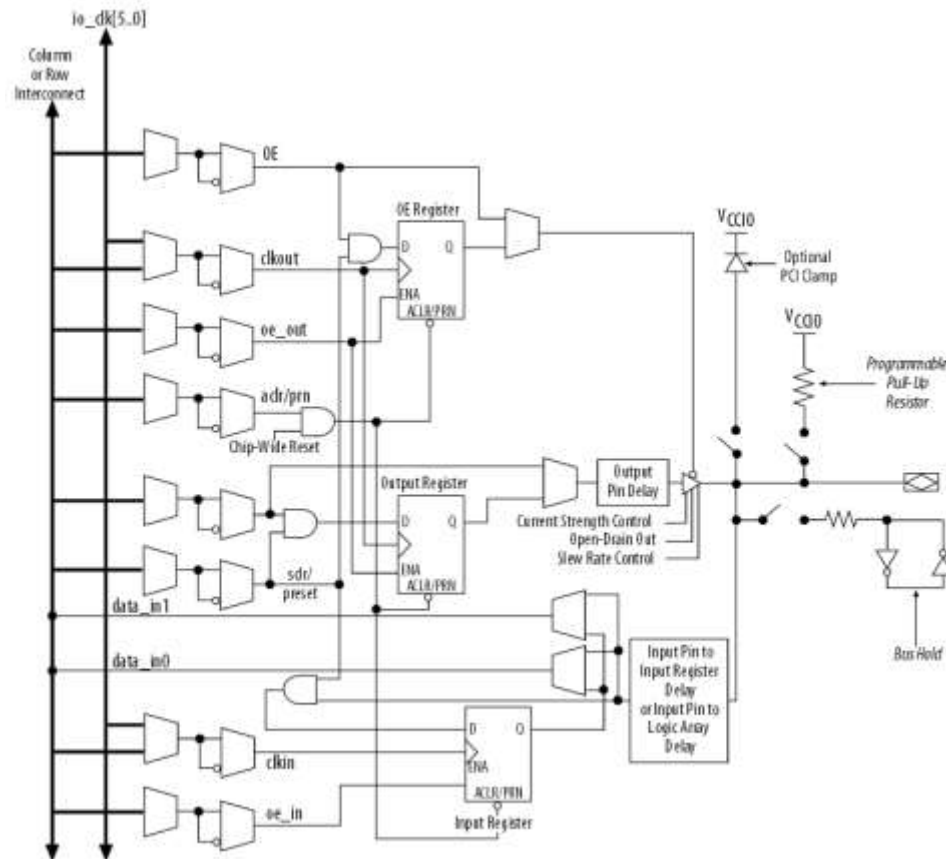


PIN FEATURES

- Each bank has a dedicated supply voltage VCCIO input
- Voltage referenced I/O standards have an additional VREF input
- Some devices need a pre-drive supply VCCPD



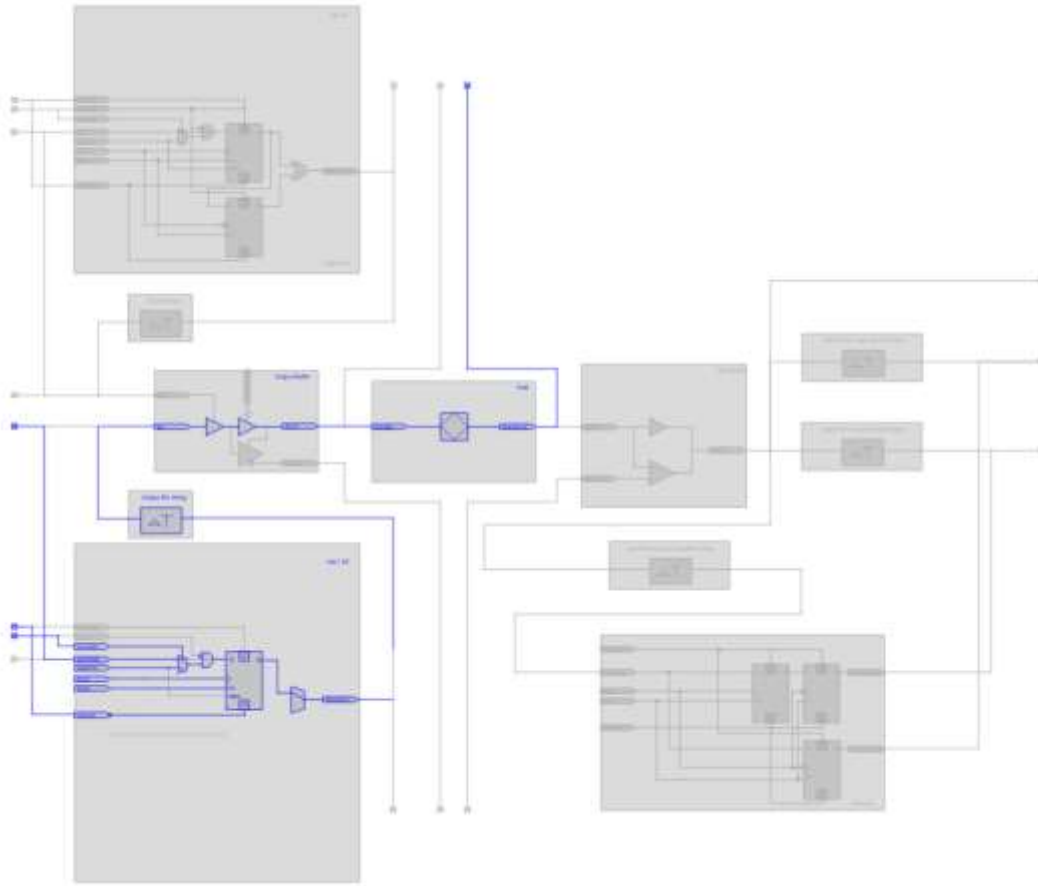
PIN FEATURES



- FPGA pins have a lot of programmable features
 - DDIO
 - Programmable Delay
 - Tri-state buffer
 - Internal pull-up
 - Fast IO registers

- Some pins may have specific purposes or a dedicated purpose

PIN FEATURES



- IO Buffer view from Chip Planner
 - Output only pin
 - Used logic showed in blue
 - DDIO input capable pin

VERILOG:

INPUT AND OUTPUT PORTS:

MODULE ()



PORT DECLARATION

Defines the names, sizes, types & directions for all ports

➤ Format

```
<port_type> port_name;
```

➤ Port types

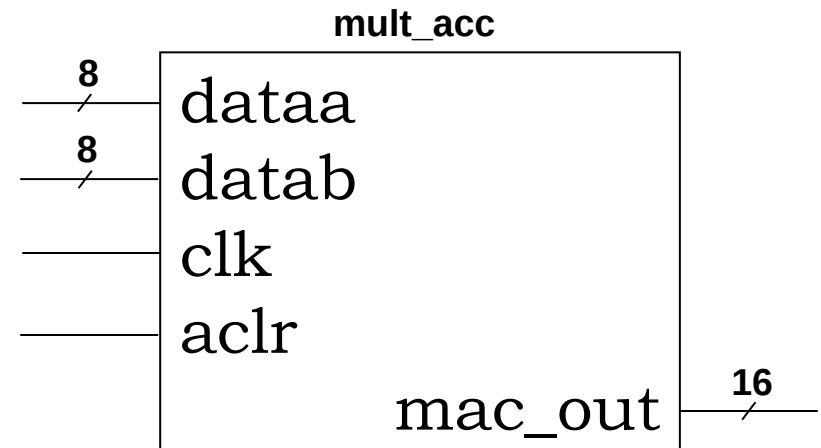
- **input** ⇒ input port
- **output** ⇒ output port
- **inout** ⇒ bidirectional port

➤ Examples

```
input [7:0] dataa, datab;  
input clk, aclr;  
output [15:0] mac_out;
```

MODULE/PORT DECLARATION

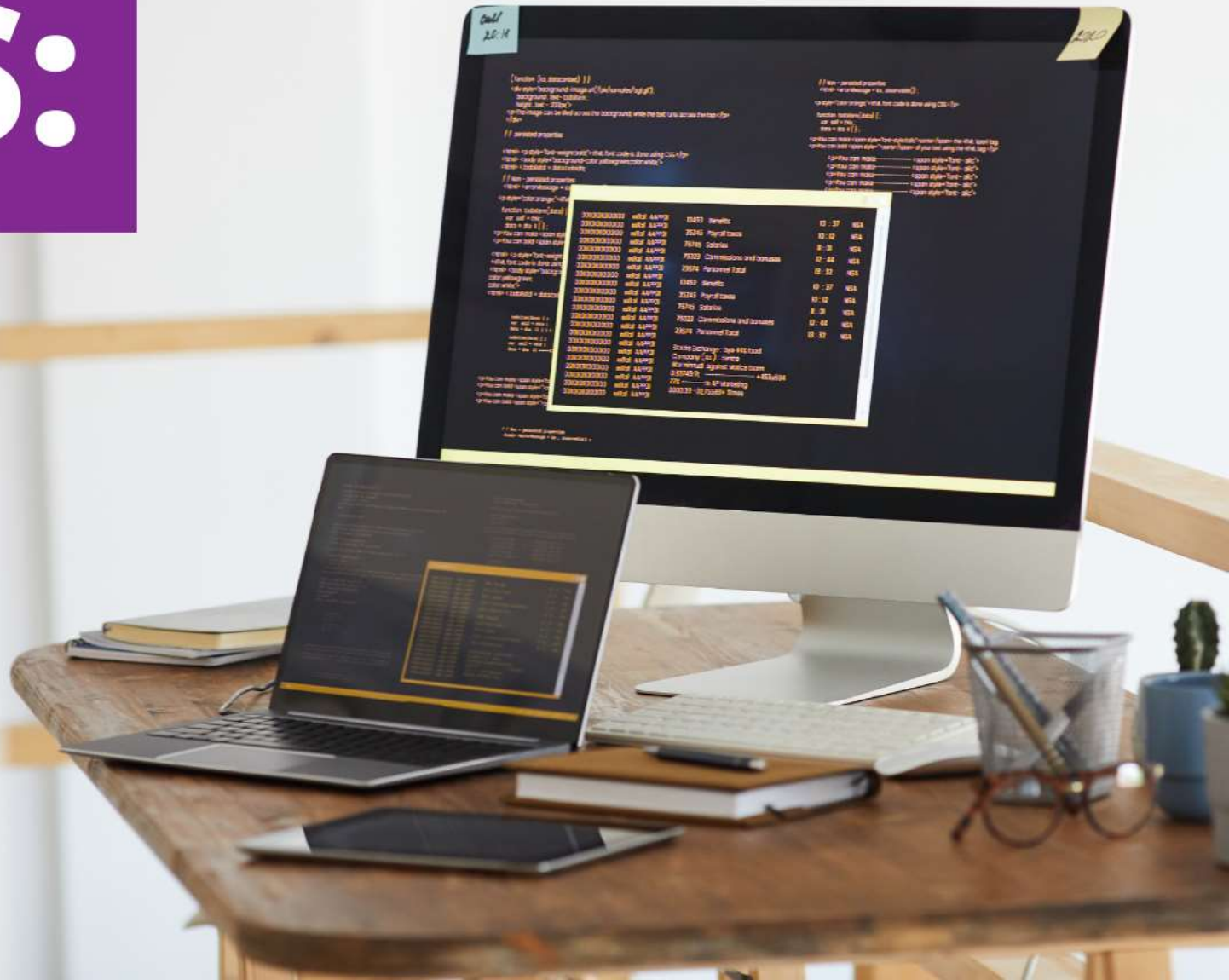
```
module mult_acc (  
    input [7:0] dataa, datab,  
    input clk, aclr,  
    output [15:0] mac_out  
);  
  
endmodule
```



QUARTUS:

INPUT AND OUTPUT
PINS:

PIN PLANNER

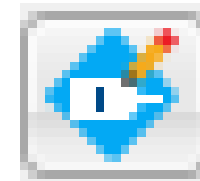


CREATING I/O-RELATED ASSIGNMENTS

- Pin Planner
- Import from spreadsheet in **.csv** format
- Type directly into **.qsf** file
- Directly in HDL code
- Scripting

PIN PLANNER

- Interactive graphical tool for assigning pins
 - Drag & drop pin assignments
 - Set pin I/O standards
 - Reserve future I/O locations
- Default window panes
 - Package View
 - All Pins list
 - Groups list
 - Tasks window
 - Report window



Assignments menu → **Pin Planner**, toolbar, or **Tasks** window

PIN PLANNER WINDOW

Toolbar

The screenshot shows the Pin Planner window with the following components:

- Groups list:** A table listing node names and their directions.

Node Name	Direction
dataa[15..0]	Input Group
datab[15..0]	
q[31..0]	
rdaddress[5..1]	
waddress[5..0]	Input Group
<<new group>>	
- Package View:** A top-down view of the chip package showing pin locations and assignments. The title is "Top View - Flip Chip" and the device is "Arria 10 - 10AX115S2F45I15G".
- Tasks pane:** A list of tasks for pin planning, including:
 - Early Pin Planning
 - Early Pin Planning...
 - Run I/O Assignment Analysis
 - Export Pin Assignments...
 - Pin Finder...
 - Highlight Pins
 - I/O Banks
 - Clock Pins
 - Clock
 - PLL/DLL Input
 - PLL/DLL Output
 - Memory Pins
 - Hard Memory Interface
 - Hard IP Interface
 - Lanes
 - DQ/DQS
- All Pins list:** A table listing all pins and their properties.

Node Name	Direction	Location	I/O Bank	Fitter Location	I/O Standard	Reserved	Current Strength
clk1	Input			PIN_Q28	1.8 V (default)		12mA (default)
dataa[15]	Input			PIN_J28	1.8 V (default)		12mA (default)
dataa[14]	Input			PIN_H29	1.8 V (default)		12mA (default)
dataa[13]	Input			PIN_K31	1.8 V (default)		12mA (default)
dataa[12]	Input			PIN_H25	1.8 V (default)		12mA (default)
dataa[11]	Input			PIN_B30	1.8 V (default)		12mA (default)
dataa[10]	Input			PIN_A28	1.8 V (default)		12mA (default)
dataa[9]	Input			PIN_K30	1.8 V (default)		12mA (default)

PIN PLANNER WINDOW PANES

► Package View

- Displays graphical representation of chip package
- Locate, make, or edit I/O assignments

➤ All Pins list

- Displays I/O pins (signals) in design
- Edit pin assignments

➤ Groups list

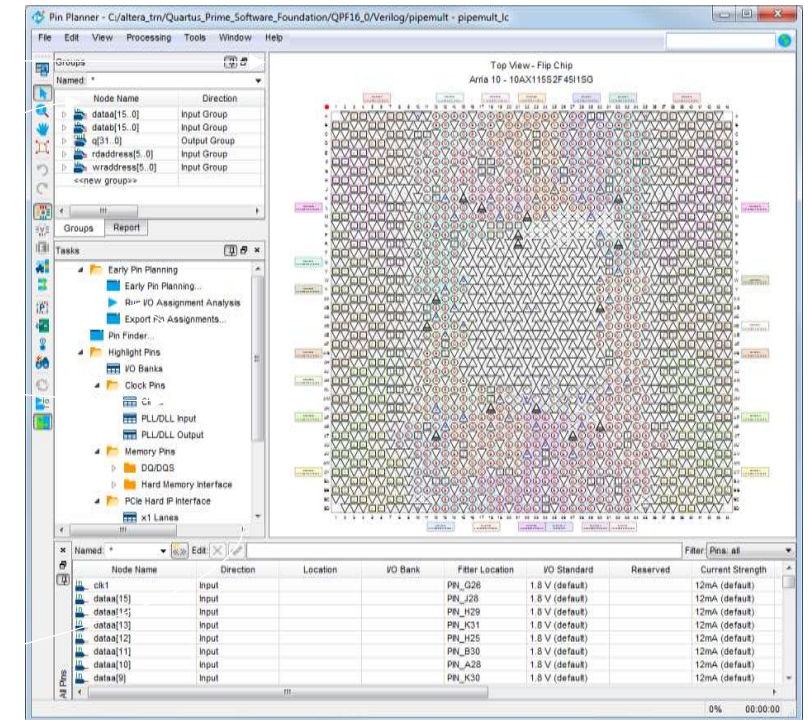
- Similar to All Pins list displaying only groups & buses
- Make bus and group assignments
- Create new user-defined groups

➤ Tasks pane

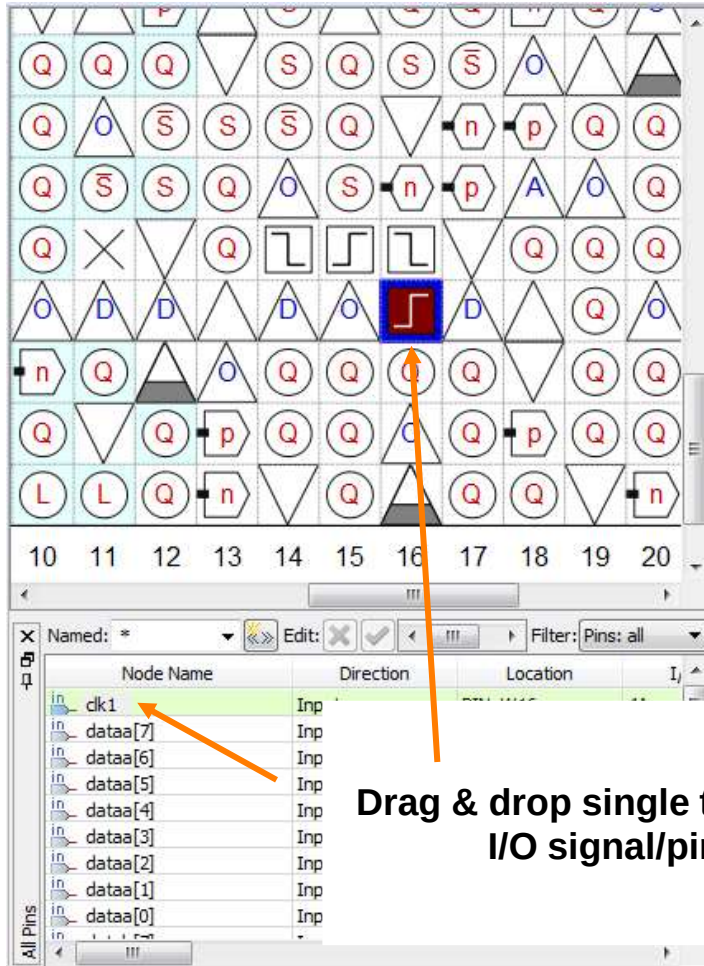
- Perform tasks such as Early Pin Planning and pin highlight reports

➤ Report Pane

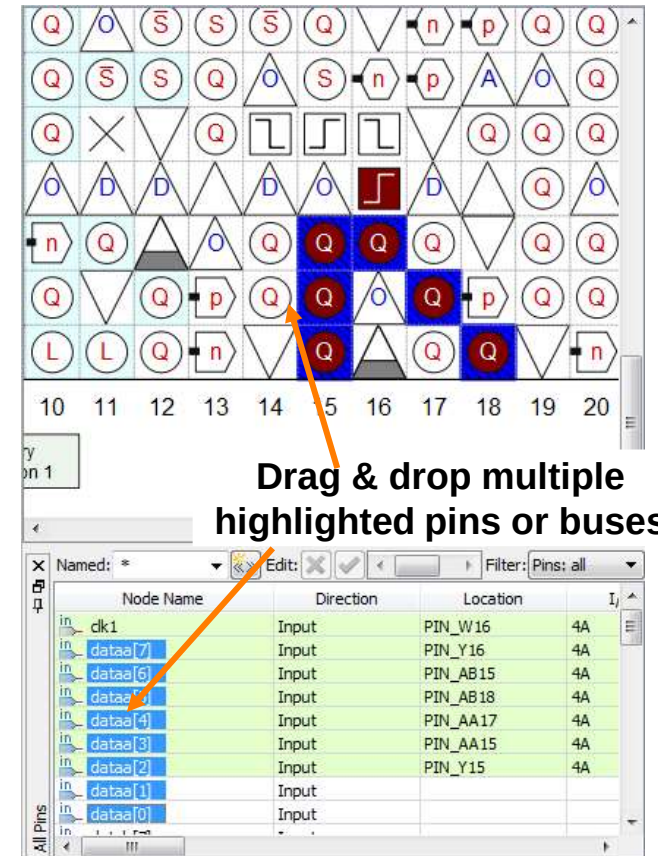
- Quickly enable/disable reports generated in the Tasks pane



ASSIGNING PIN LOCATIONS USING PIN PLANNER



Choose one-by-one or pin alignment direction (Edit menu)



ASSIGNING PIN LOCATIONS USING PIN PLANNER

Drag & drop to I/O bank, VREF block, or device edge

Click pin or I/O bank to display pin properties

The screenshot displays the Altera Pin Planner interface. At the top, there are I/O bank selection buttons: IOBANK_6A (0 total) and IOBANK_7A (7 assigned/64 total). Below these is a grid of pins, each with a letter (Q, n, p, D) and a number. A blue arrow points from the text 'Drag & drop to I/O bank, VREF block, or device edge' to the IOBANK_7A button. Another blue arrow points from the text 'Click pin or I/O bank to display pin properties' to a pin labeled 'Q' in the grid. To the right, a 'Pin Properties' window is open, showing details for pin PDI_B17, including its I/O bank (7A), VREF group (B7A_n0), edge (TOP), and general function (Column I/O). Below the grid, a table titled 'All Pins' lists the assigned pins. A blue arrow points from the text 'Filter nodes displayed in All Pins list' to the 'Filter: Pins: all' dropdown menu in the table's header.

Node Name	Direction	Location	I/O Bank	VREF Group	I/O Standard	Reserved	Current Strength	Slew Rate	Differential
d[0]	Input	PDI_W36	4A	B4A_n0	2.5 V (default)		12mA (default)		
d[1]	Input	IOBANK_7A	7A		2.5 V (default)		12mA (default)		
d[2]	Input	IOBANK_7A	7A		2.5 V (default)		12mA (default)		
d[3]	Input	IOBANK_7A	7A		2.5 V (default)		12mA (default)		
d[4]	Input	IOBANK_7A	7A		2.5 V (default)		12mA (default)		
d[5]	Input	IOBANK_7A	7A		2.5 V (default)		12mA (default)		
d[6]	Input	IOBANK_7A	7A		2.5 V (default)		12mA (default)		
d[7]	Input	IOBANK_7A	7A		2.5 V (default)		12mA (default)		
d[8]	Input	IOBANK_7A	7A		2.5 V (default)		12mA (default)		
d[9]	Input	IOBANK_7A	7A		2.5 V (default)		12mA (default)		

Filter nodes displayed in All Pins list

ASSIGNING PIN LOCATIONS USING PIN PLANNER

Select available locations from list of pins color-coded by I/O bank

Named: * Edit: PIN_AA13 Filter: Pins: all

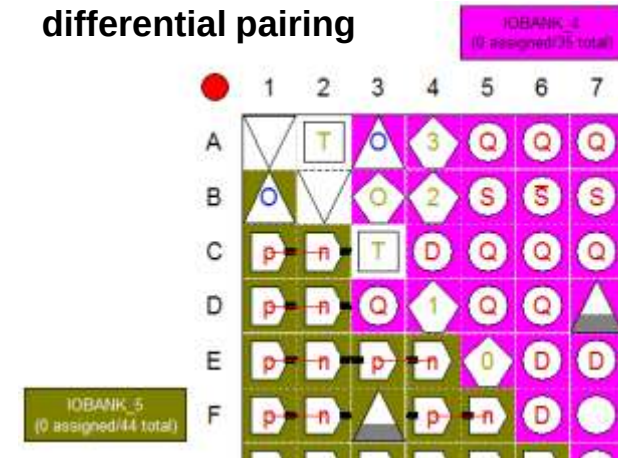
Node Name	Direction	Location	I/O Bank	VREF Group	I/O Standard	Reserved	Current Strength	Slew Rate	Differential
clk1	Input	PIN_W16	4A	B4A_N0	2.5 V (default)		12mA (default)		
dataa[7]	Input	PIN_AA13							
dataa[6]	Input	PIN_AA20	IOBANK_4A	Column I/O	DIFFIO_TX_B61p, DIFFOUT_B61p, DQ8B				
dataa[5]	Input	PIN_AA22	IOBANK_4A	Column I/O	DIFFIO_TX_B64p, DIFFOUT_B64p, DQ8B				
dataa[4]	Input	PIN_AB5	IOBANK_3B	Column I/O	DIFFIO_TX_B33p, DIFFOUT_B33p, DQ5B				
dataa[3]	Input	PIN_AB6	IOBANK_3B	Column I/O	DIFFIO_TX_B33n, DIFFOUT_B33n				
dataa[2]	Input	PIN_AB7	IOBANK_3B	Column I/O	DIFFIO_TX_B36p, DIFFOUT_B36p				
dataa[1]	Input	PIN_AB8	IOBANK_3B	Column I/O	DIFFIO_TX_B37p, DIFFOUT_B37p, DQ5B				
dataa[0]	Input	PIN_AB10	IOBANK_3B	Column I/O	FPLL_BL_CLKOUT1, FPLL_BL_CLKOUTn, DIFFIO_TX_B45n, DIFFOUT_B45n, DQ6B				
		PIN_AB11	IOBANK_3B	Column I/O	FPLL_BL_CLKOUT0, FPLL_BL_CLKOUTp, FPLL_BL_FB, DIFFIO_TX_B45p, DIFFOUT_B45p, DQ6B				
		PIN_AB12	IOBANK_4A	Column I/O	DIFFIO_TX_B49p, DIFFOUT_B49p, DQ7B				
		PIN_AB13	IOBANK_4A	Column I/O	RZQ_0, DIFFIO_TX_B49n, DIFFOUT_B49n				

00:00:00

PIN LEGEND AND PIN PLANNER VIEWS

- Displays (**View** menu → **Show**, toolbar buttons, or right-click in Package View)
 - Device edges
 - I/O banks
 - VREF groups
 - Differential pin pairing
 - DQ/DQS and hard memory interface pins
 - PCIe hard IP pins
- Easy-to-read pin legend

I/O banks & differential pairing

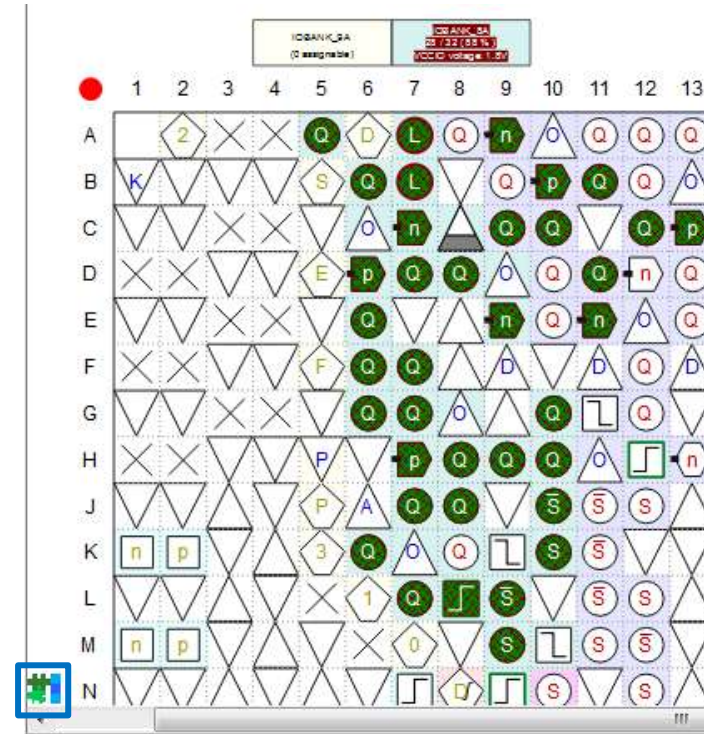


View menu → **Pin Legend**
or in **Toolbar**



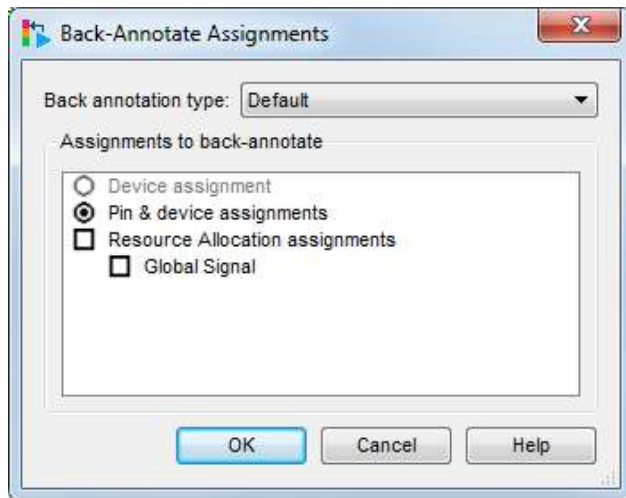
Symbol	Pin Type
	User I/O
	User assigned I/O
	Fitter assigned I/O
	Unbonded pad
	Reserved pin
	DEV_OE
	DIFF_n
	DIFF_p
	DIFF_n output
	DIFF_p output
	DQ
	DQS
	DQSB
	CLK_n
	CLK_p
	GX_X*n
	GX_X*p
	Other PLL
	MSEL0
	MSEL1
	MSEL2
	MSEL3

**View → Show
or in toolbar**



BACK-ANNOTATION

Assignments menu in main
Quartus Prime window



➤ Use to lock Fitter-chosen pin assignments
for future compilations

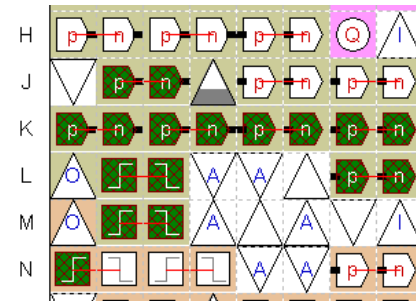
- Copies device & resource locations
chosen by fitter into **.qsf** file

Pins

Logic

Routing

➤ “Locks down” locations in Pin Planner



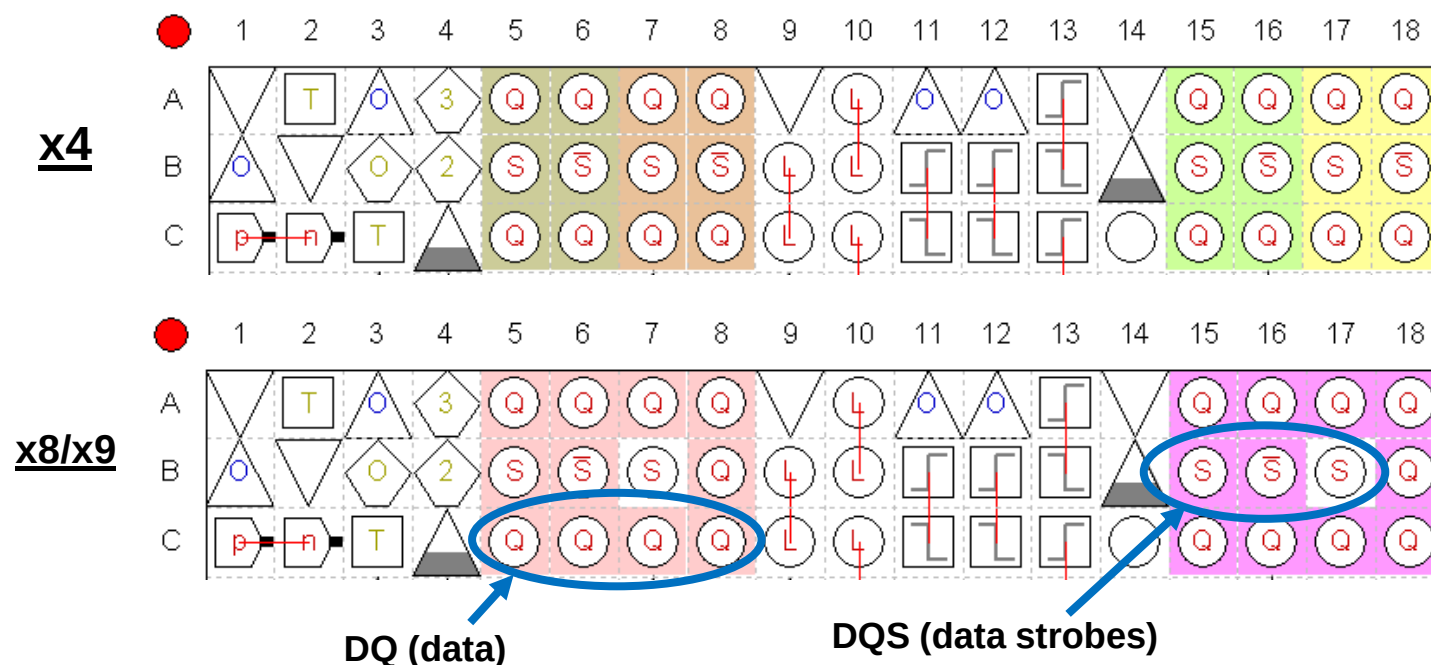
Green/brown pattern
indicates back-annotation

I/O PLANNING NEED

- I/O standards increasing in complexity
- FPGA/CPLD I/O structure increasing in complexity
 - Results in increased pin placement guidelines
- PCB development performed simultaneously with FPGA design
- Pin assignments need to be verified earlier in design cycle

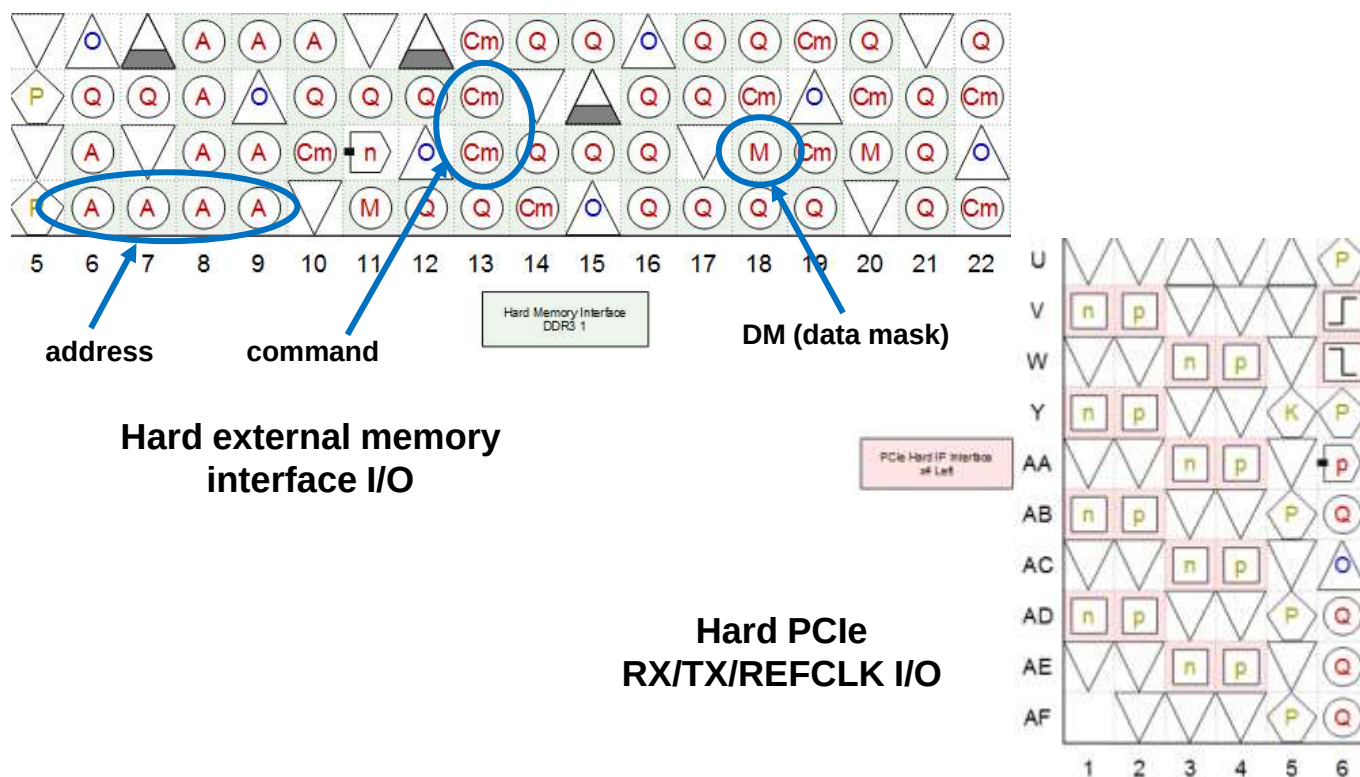
EXAMPLE VIEW: SHOW DQ/DQS PINS

Show color-coded DQ/DQS sets in x4, x8/x9, x16/x18, or x32/x36 modes in the Package View for DDR interfaces



EXAMPLE VIEW:

SHOW DEDICATED HARD RESOURCES I/O



VERIFYING I/O ASSIGNMENTS

- I/O Assignment Analysis
 - Checks legality of all I/O assignments without full compilation
- Minimal requirements for running
 - I/O declaration
 - HDL port declaration
 - Reserved pin
 - Pin-related assignments
 - I/O standard
 - Current strength
 - Pin location (pin, bank, edge)
 - Toggle rate

Pin Planner Processing menu → **Start I/O Assignment Analysis**
or toolbar

Run from Pin Planner toolbar



I/O RULES CHECKED

- No internal logic
 - Checks I/O locations & constraints with respect to other I/O & I/O banks
 - e.g. Each I/O bank supports a single VCCIO
- I/O connected to logic
 - Checks I/O locations & constraints with respect to other I/O, I/O banks, & internal resources
 - e.g. PLL must be driven by a dedicated clock input pin

Note: When working with design files, synthesize design before running I/O Assignment Analysis

I/O ASSIGNMENT ANALYSIS OUTPUT

Compilation Report - C:/altera_trn/Quartus_Prime_Software_Foundation/QPF16_0/Verilog/pipemult - pipemult_lc

File Edit Tools Window Help Search altera.com

Table of Contents

- Flow Summary
- Flow Settings
- Flow Non-Default Global Settings
- Flow Elapsed Time
- Flow OS Summary
- Flow Log
- Synthesis
- Fitter
 - Summary
 - Settings
 - Parallel Compilation
 - Device Options
 - I/O Assignment Warnings
 - Netlist Optimizations
 - Plan Stage
 - I/O Rules Details
 - I/O Rules Matrix**
 - I/O Rules Summary
 - Ignored Assignments
 - Messages
 - Suppressed Messages
 - Flow Messages
 - Flow Suppressed Messages

I/O Rules Matrix

	Pin/Rules	IO_000001	IO_000004	IO_000007	IO_000009	IO_000010	IO_000011
1	Total Pass	78	0	78	78	78	0
2	Total Unchecked	0	0	0	0	0	0
3	Total Inapplicable	0	78	0	0	0	78
4	Total Fail	0	0	0	0	0	0
5	q[0]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
6	q[1]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
7	q[2]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
8	q[3]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
9	q[4]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
10	q[5]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
11	q[6]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
12	q[7]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
13	q[8]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
14	q[9]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
15	q[10]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
16	q[11]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
17	q[12]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
18	q[13]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
19	q[14]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
20	q[15]	Pass	Inapplicable	Pass	Pass	Pass	Inapplicable
21	q[16]	Pass	Inapplicable				
22	q[17]	Pass	Inapplicable				
23	q[18]	Pass	Inapplicable				
24	q[19]	Pass	Inapplicable				
25	q[20]	Pass	Inapplicable				
26	q[21]	Pass	Inapplicable				

Messages on I/O assignment issues

- Compiler assumptions
- Device & pin migration issues
- I/O bank voltages & standards

VALIDATING I/O PIN-OUT

➤ Completed design

- Run full compilation
- Enable option to **Run I/O Assignment Analysis before compilation** (Settings dialog box → **Compilation Process Settings**)

➤ Incomplete design with completed top-level design file

- Add I/O-related IP cores and instantiate, even if final connections are not complete
- Run I/O Assignment Analysis on design

QUARTUS:

SOFTWARE OVERVIEW



QUARTUS PRIME DESIGN SOFTWARE

➤ Fully-integrated development tool

- Multiple design entry methods
- Logic synthesis
- Place & route
- Device programming

➤ Simulation

- Supports standard HDL simulation tools
- Includes ModelSim®-Altera Starter Edition tool
Optional upgrade to ModelSim-Altera Edition tool
- See comparison

<https://www.intel.com/content/www/us/en/software/programmable/quartus-prime/model-sim.html>

QUARTUS PRIME DESIGN SOFTWARE FEATURES

For Your Reference

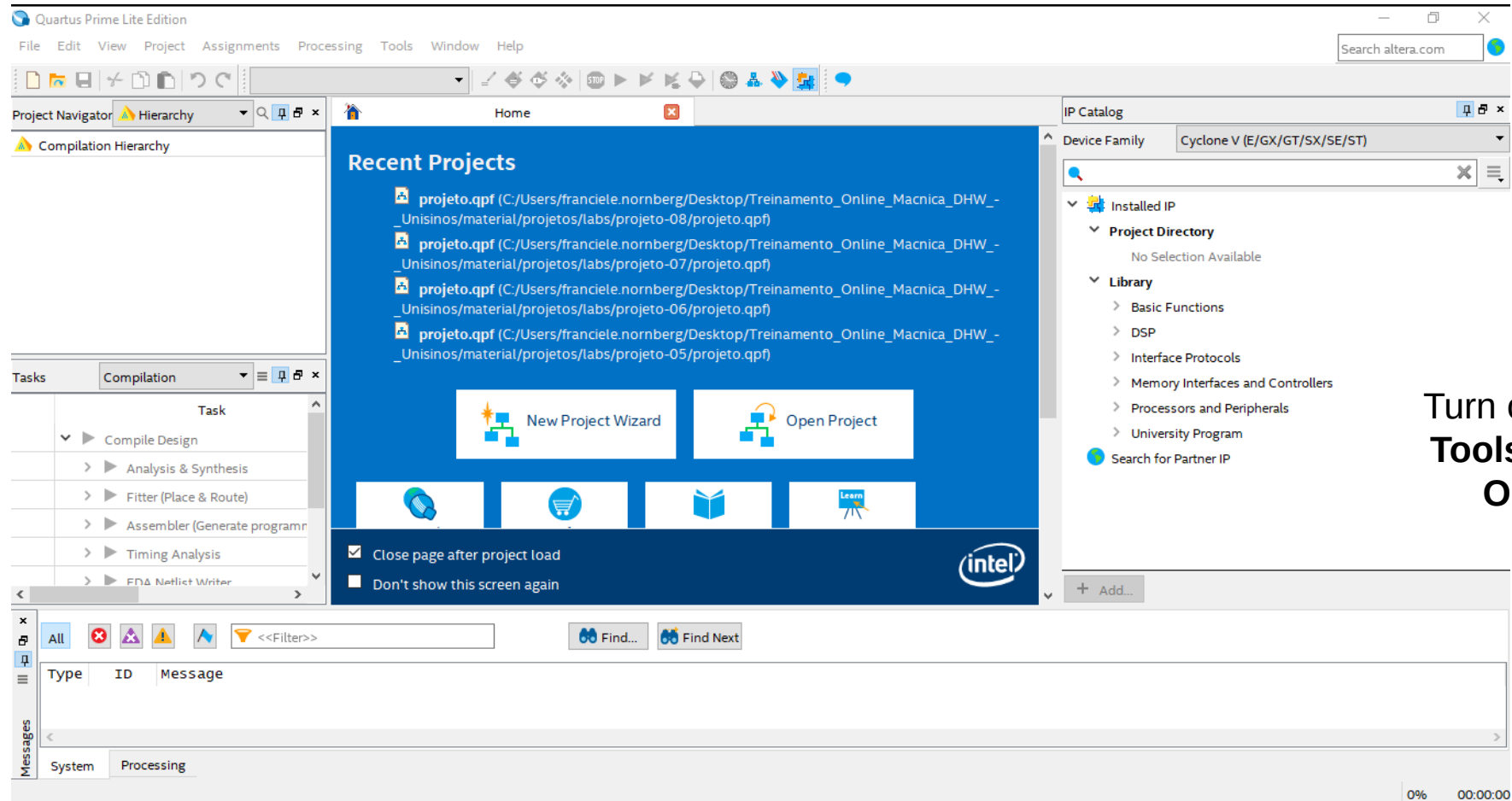
	Quartus Prime Tool/Feature
Operating system	64-bit Windows and Linux support
Licensing	Node-locked and network licensing support
Project creation	New Project Wizard
Design entry	- Text Editor (HDL support) - Schematic Editor - State Machine Editor - IP Catalog & IP Parameter Editor - Platform Designer or Platform Designer Pro system design tool - DSP Builder Standard/Advanced Blockset - OpenCL support 3rd-party design entry tool support
Constraint (assignment) entry	- Assignment Editor - Text Editor support of Synopsys Design Constraints (SDC) - Pin Planner - Blueprint Platform Designer (<i>Pro Edition only</i>) - Scripting (Tcl) support
Design processing/compilation (synthesis and fitting)	- Quartus Integrated Synthesis (QIS) or Spectra-Q™ Synthesis (<i>Pro</i>) 3rd party EDA synthesis tool support - Quartus Prime Fitter (<i>Lite & Standard</i>) - Spectra-Q Hybrid Placer & Router (<i>Standard & Pro</i>)
Design evaluation and debugging	- RTL Viewer - Technology Map Viewers - State Machine Viewer

QUARTUS PRIME DESIGN SOFTWARE FEATURES

	Quartus Prime Tool/Feature	
Power analysis	PowerPlay power analyzer	
Static timing analysis	TimeQuest timing analyzer	
Simulation	ModelSim-Altera Starter Edition ModelSim-Altera Edition 3 rd -party EDA simulation tool support	
Chip layout viewing and modification	Chip Planner Resource Property Editor	
Programming file generation	Quartus Prime Assembler	
FPGA/CPLD programming	Quartus Prime Programmer	
Hardware debugging tools	SignalTap II embedded logic analyzer In-System Sources and Probes SignalProbe incremental routing	System Console Transceiver Toolkit In-System Memory Content Editor
Design optimization and productivity improvement	Design Assistant Rapid Recompile Quartus Prime incremental compilation (<i>Standard Edition only</i>) Incremental Optimization (<i>Pro Edition only</i>) Physical synthesis optimization Design Space Explorer II (DSE)	

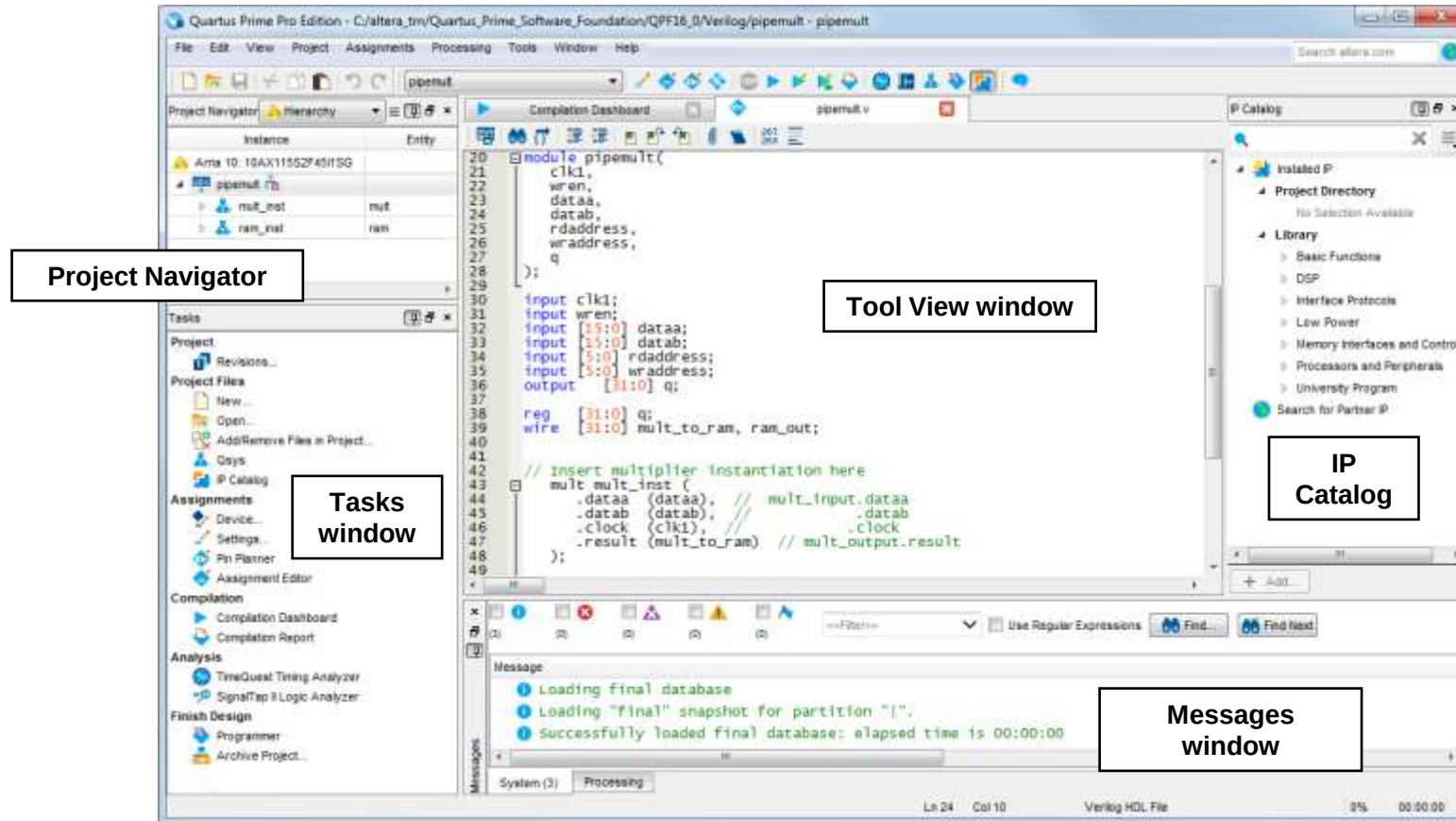
For Your Reference

WELCOME TO THE QUARTUS PRIME SOFTWARE!

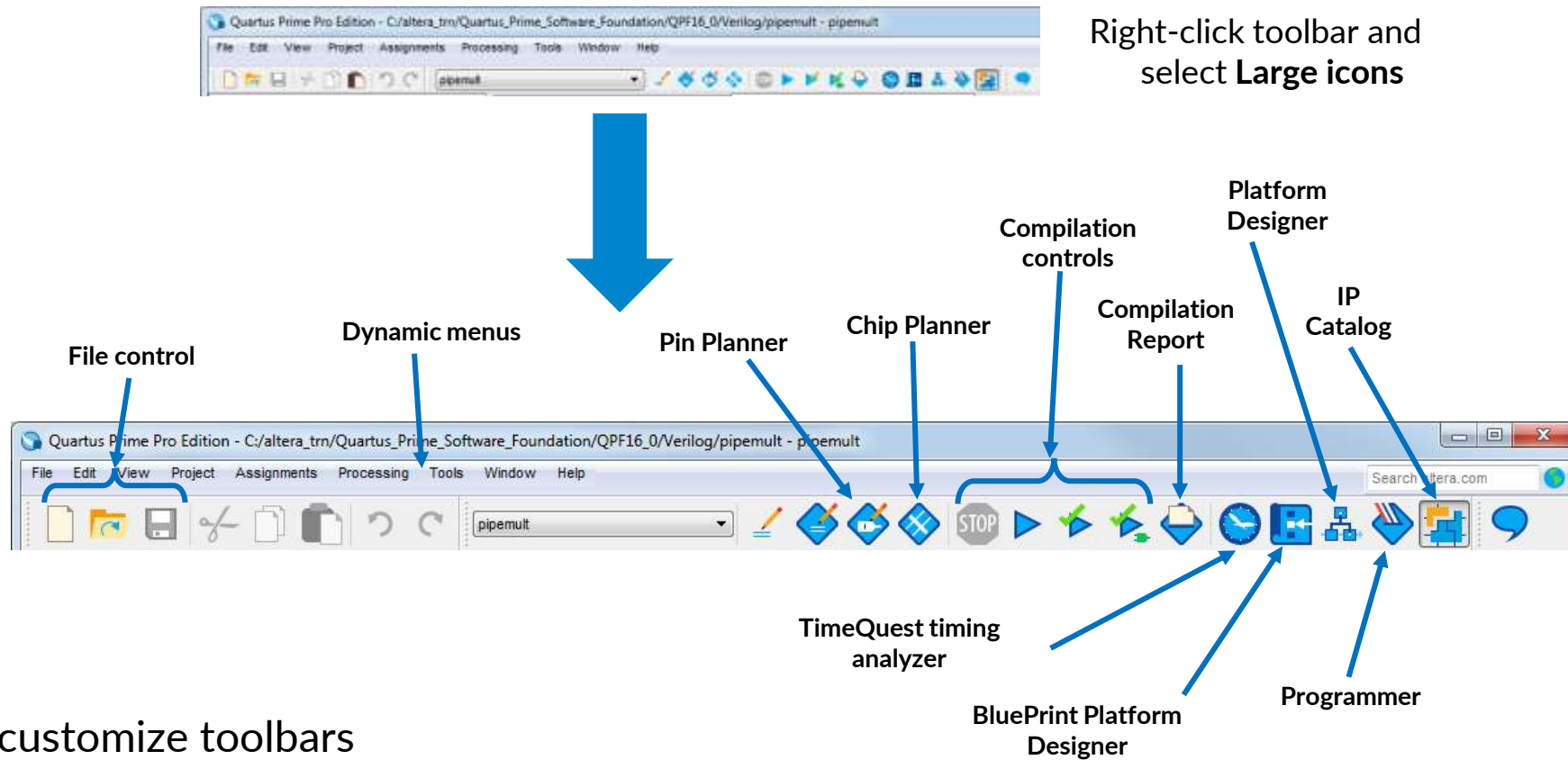


Turn on or off in
Tools menu →
Options

QUARTUS PRIME DEFAULT OPERATING ENVIRONMENT



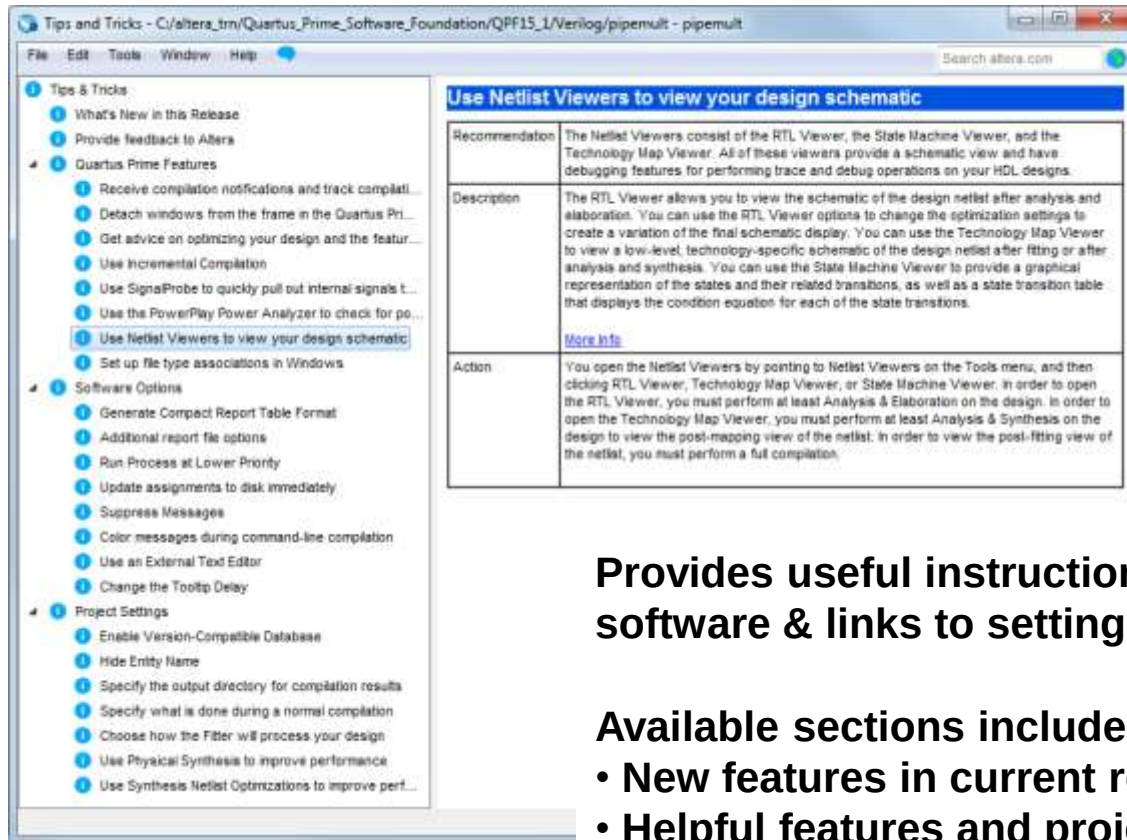
MAIN TOOLBAR



To customize toolbars
Tools menu → Customize...

TIPS & TRICK ADVISOR

Help menu → Tips & Tricks

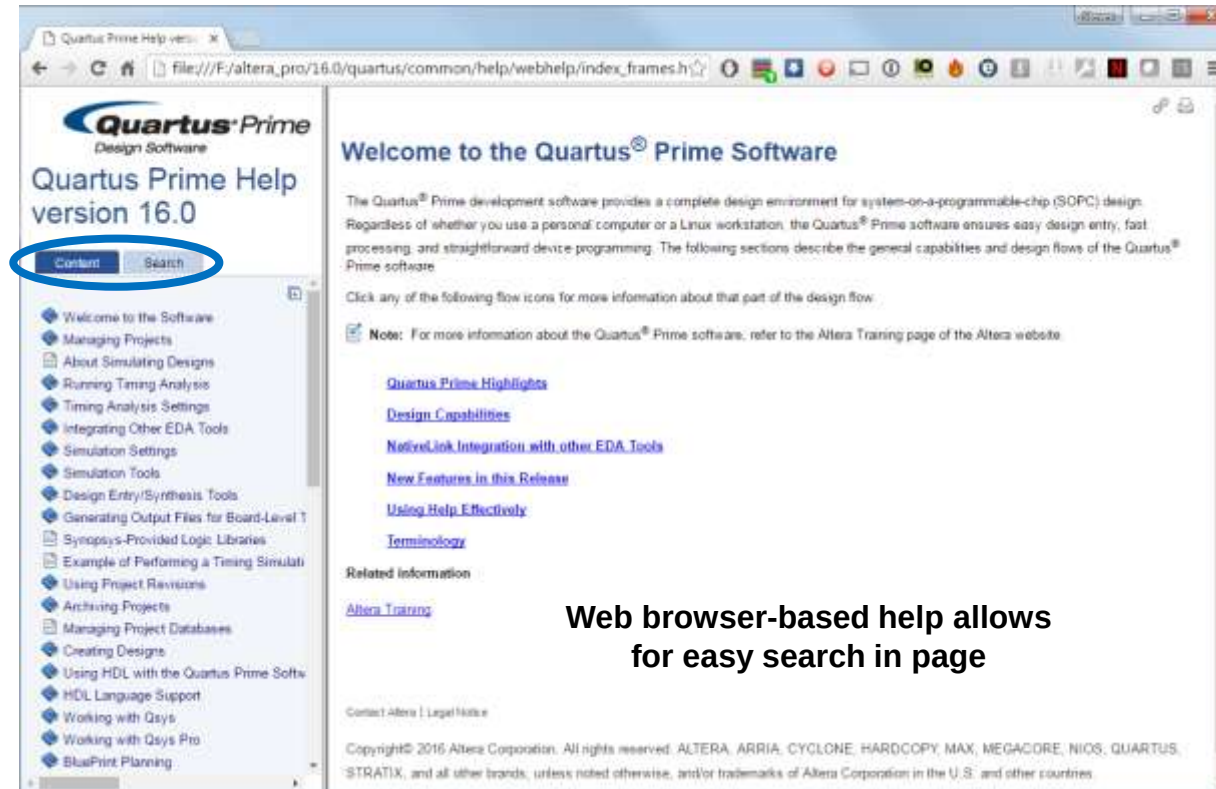
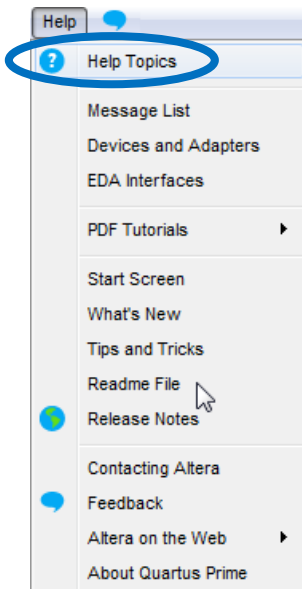


Provides useful instructions on using the Quartus Prime software & links to settings

Available sections include:

- New features in current release
- Helpful features and project settings available to designers

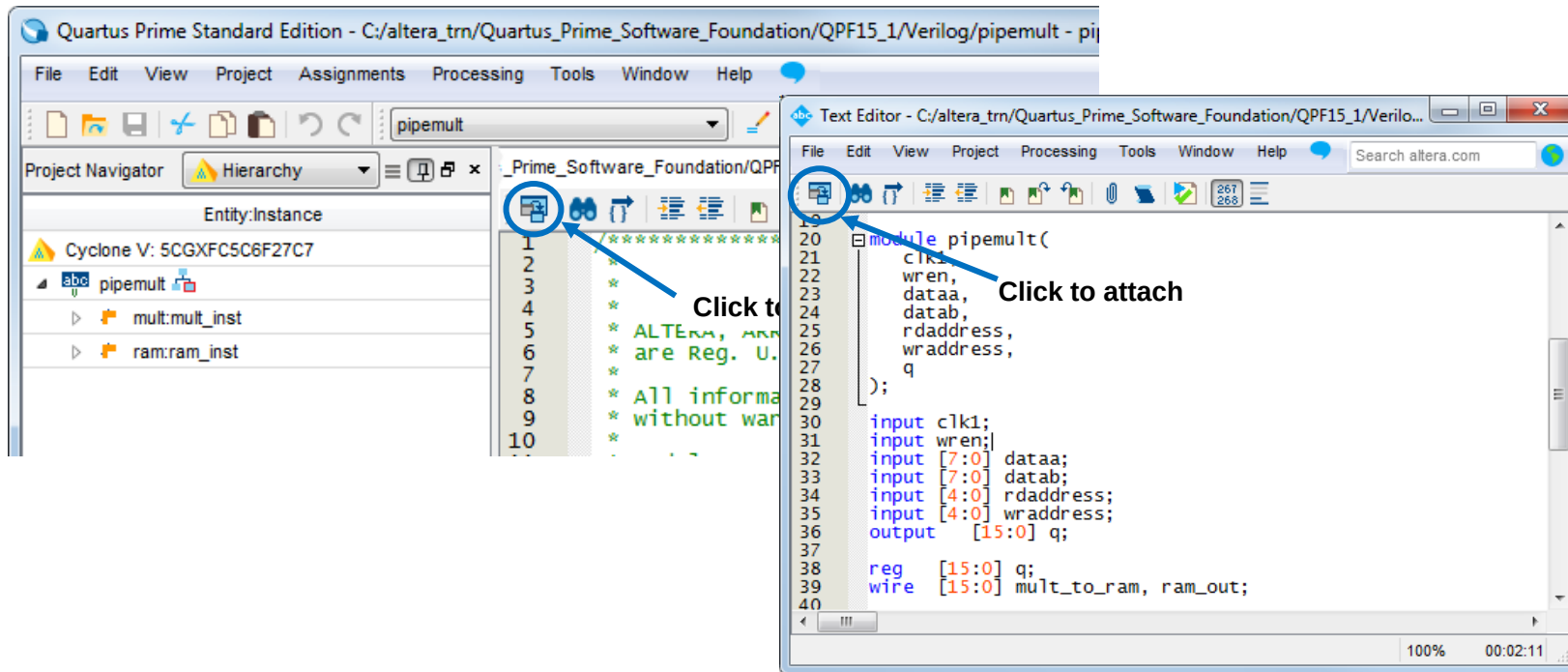
BUILT-IN HELP SYSTEM



Web browser-based help allows
for easy search in page

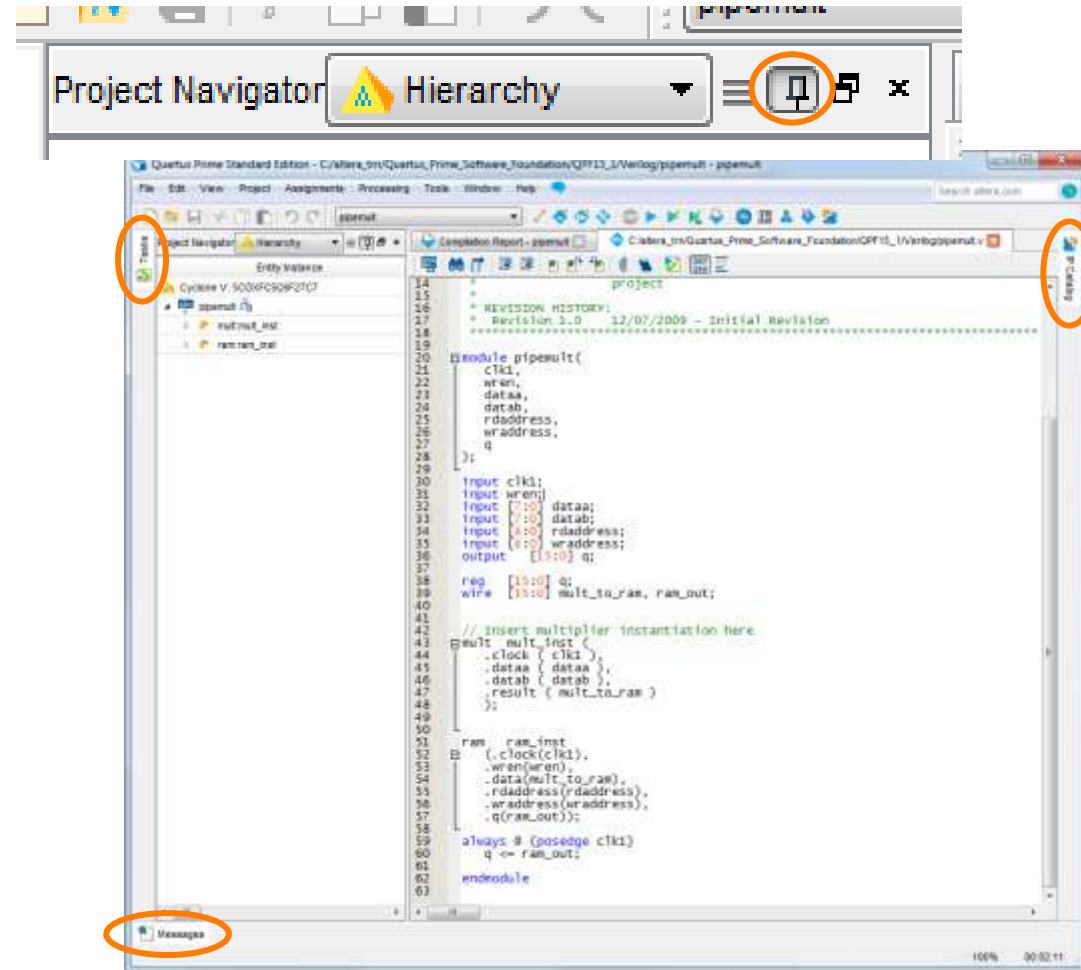
DETACHABLE WINDOWS

Separate child windows from the Quartus Prime GUI frame
(**Window menu → Detach/Attach Window**)

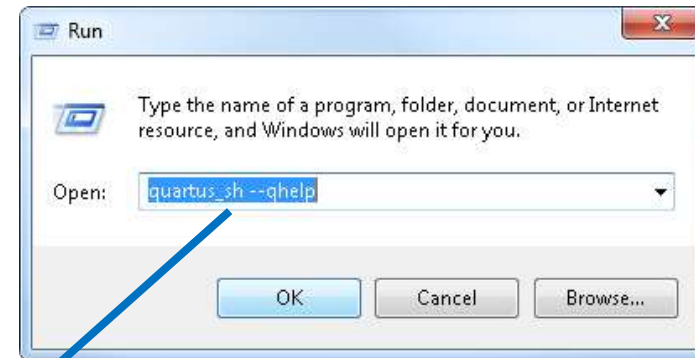
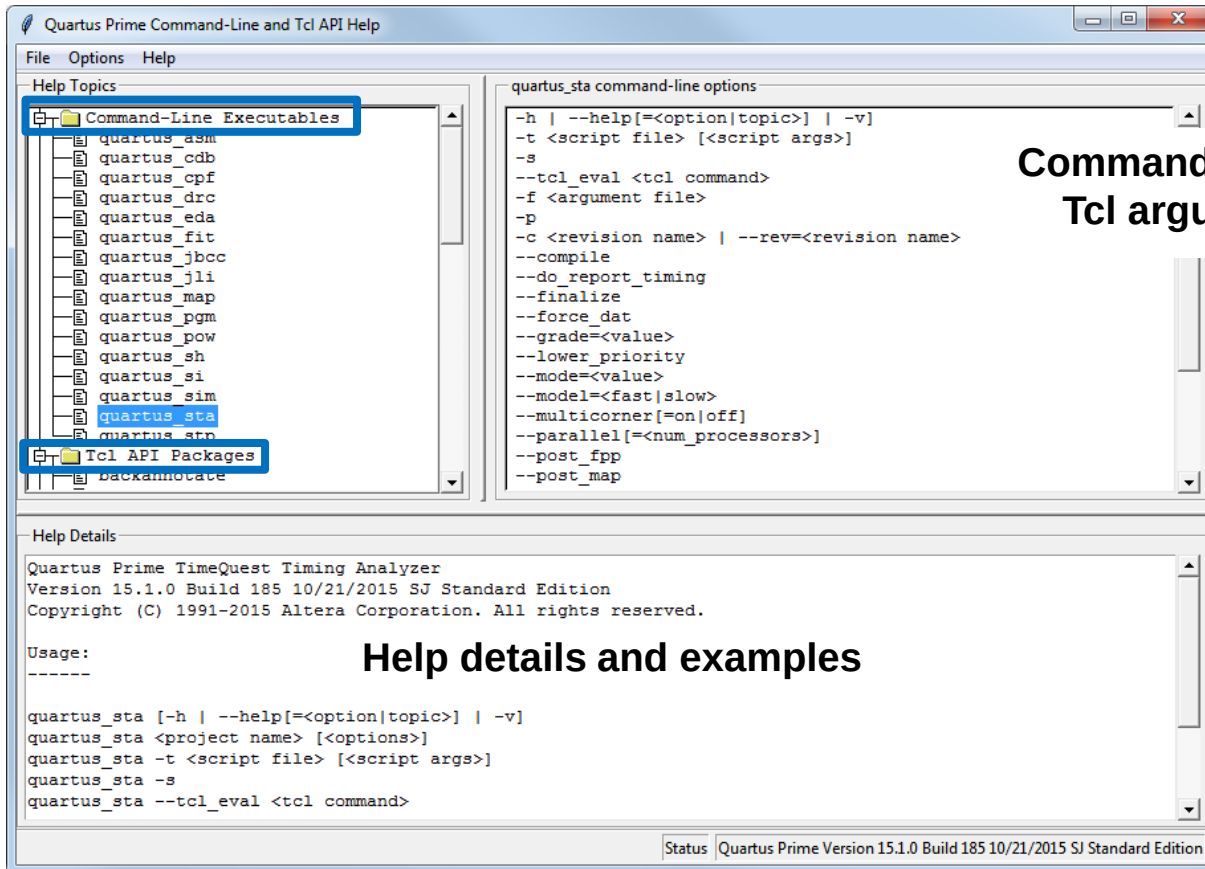


AUTO HIDE/SHOW DOCKABLE WINDOWS

- Quickly hide/unhide dockable windows by “unpinning”
- Hover over a window’s tab to make it temporarily visible
- Re-pin to keep the window visible



TCL AND COMMAND LINE HELP



C:\<install dir>\quartus\bin64\quartus_sh --qhhelp

VERILOG:

VALUE ASSIGNMENT:
ASSIGN



CONTINUOUS ASSIGNMENTS

- Model the behavior of combinatorial logic by using expressions and operators
 - Continuous assignments can be made by using the **assign** statement
 - Implicit assign can be made when the **wire** or **reg** is declared (discussed later)

```
assign adder_out = mult_out + out;
```

CONTINUOUS ASSIGNMENTS CHARACTERISTICS

- 1) Left-hand side of an assignment (LHS) must be a net data type
- 2) **Always active:** When one of the operands on the right-hand side of an assignment (RHS) changes, expression is evaluated and net on LHS is updated immediately
- 3) RHS can be an expression containing net data type, variable data type or function call (or combination of)
- 4) Delay values can be assigned to model gate delays (discussed later)

```
assign adder_out = mult_out + out;
```

(LHS) Left-Hand Side

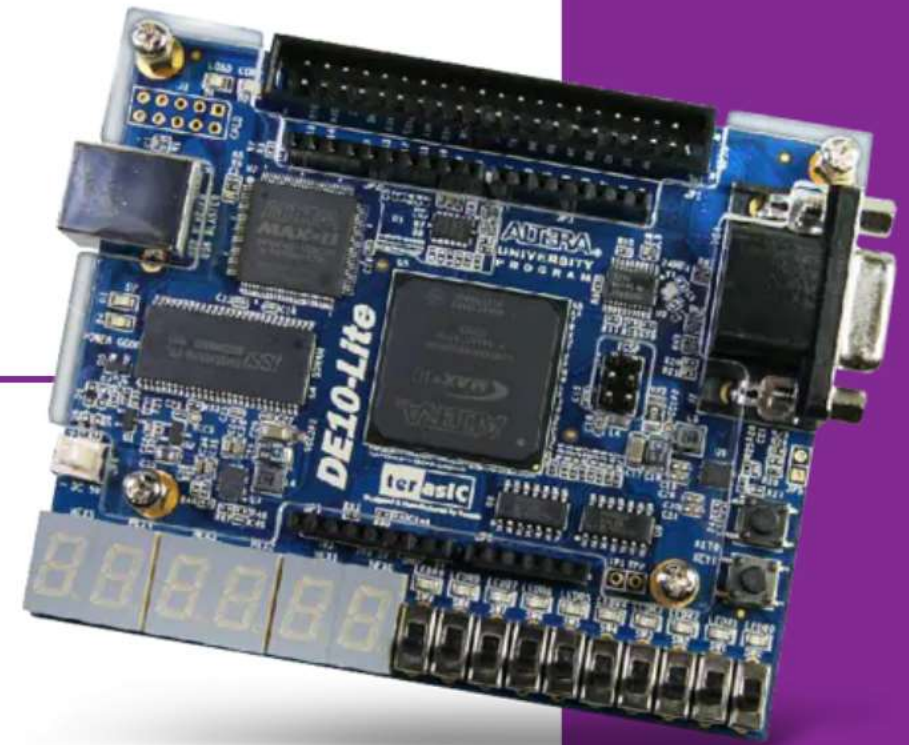
Right-Hand Side (RHS)

CONTINUOUS ASSIGNMENTS - EXAMPLE

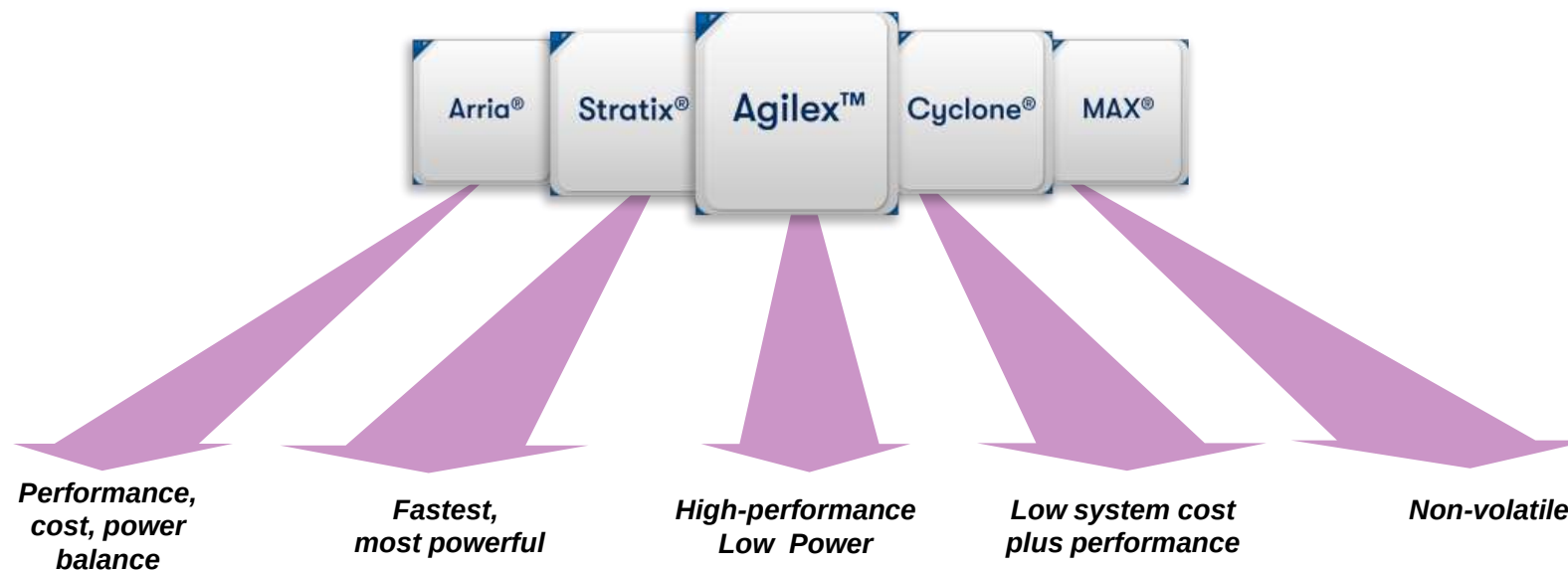
```
module and_func (  
    input [7:0] ina, inb,  
    output [7:0] out  
);  
  
assign out = ina & inb;  
  
endmodule
```

KIT DE10-LITE:

AVAILABLE RESOURCES AND COMPLEMENTARY INFORMATION



PRODUCT PORTFOLIO OVERVIEW



RESOURCES

Embedded Soft and Hard Processors

Nios®

ARM

Design Software

Quartus Prime
Design Software

Altera® SDK
for OpenCL

DSPBuilder

Development Kits

Intellectual Property (IP)

OpenCore Plus
IP Evaluation Tool

Altera IP

Design Solutions Network

FPGAs

For Your Reference

Intel Agilex FPGAs

Enable customized acceleration and connectivity for a wide range of compute and bandwidth intensive applications, while providing an improvement in performance and reduction in power.

Intel Stratix Series

Enables high-performance, state-of-the-art products go to market faster with lower risk and higher productivity.

Intel Arria Series

Performance and power efficiency in the midrange

Intel Cyclone Series

Meet low-power, cost-sensitive design needs, enabling you to get to market faster.

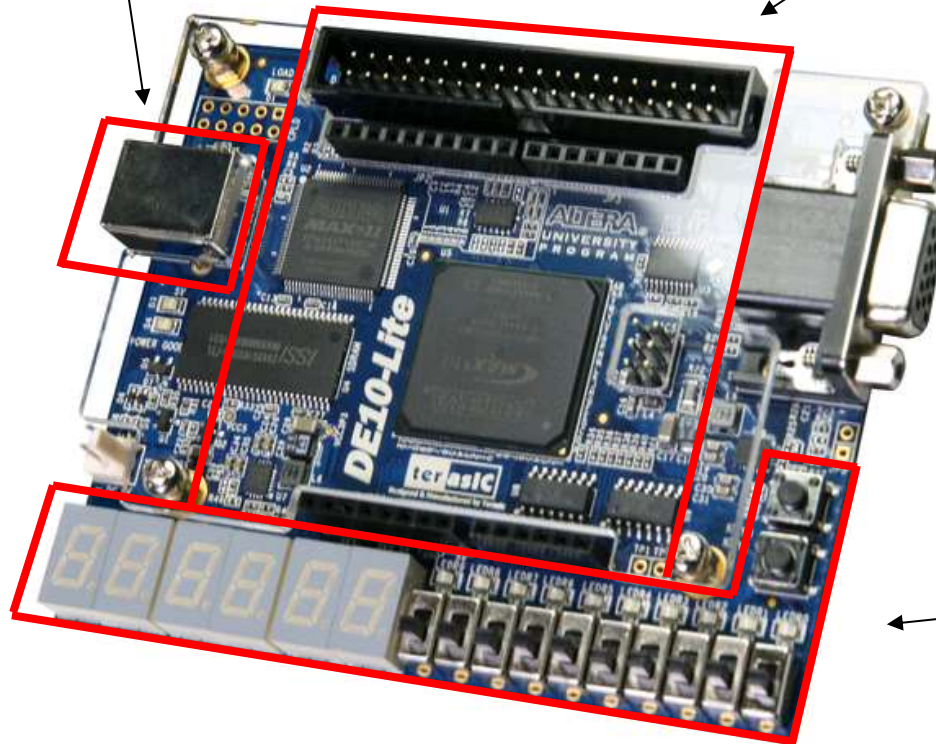
Intel MAX Series

MAX 10 FPGAs revolutionize non-volatile integration by delivering advanced processing capabilities in a low-cost, single-chip small form.

KIT DE10-LITE: EXTERNAL INTERFACES

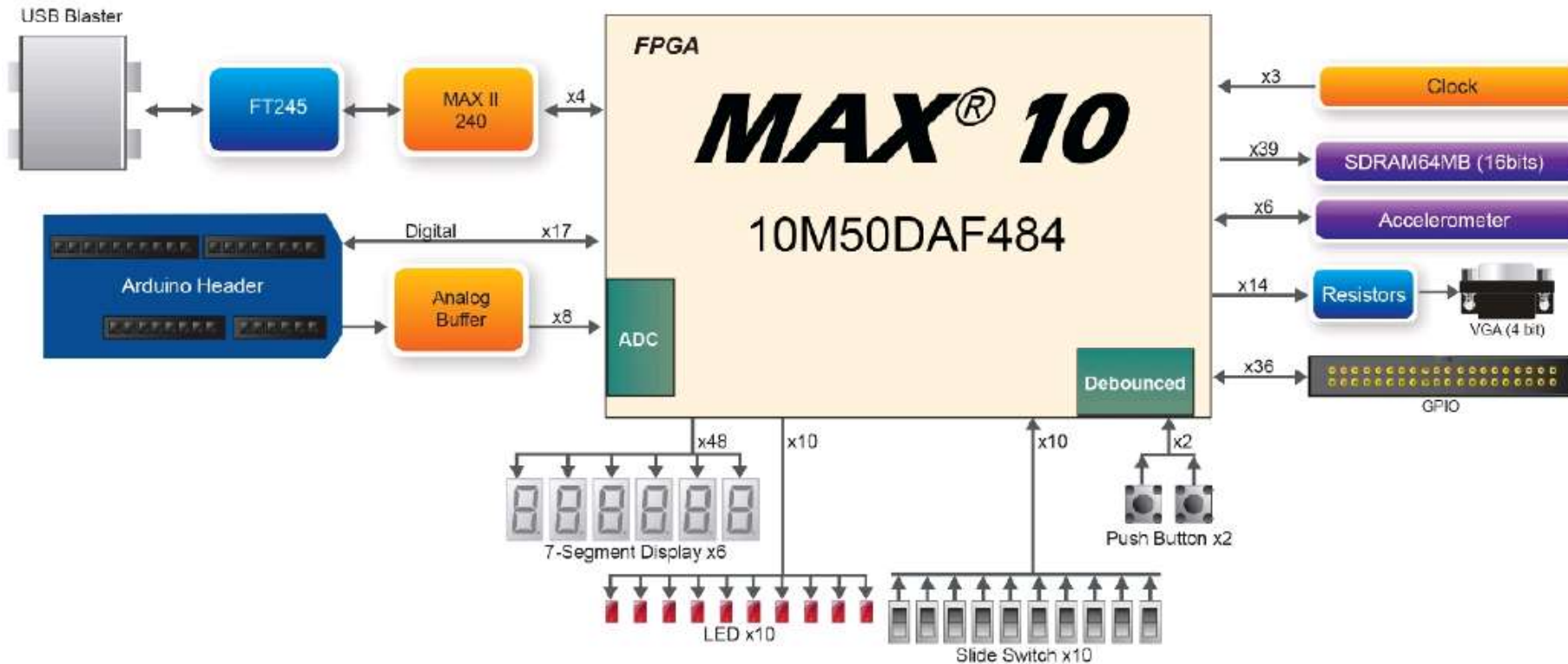
- USB Powered
- USB Programmable

- 40-Pin header support several Terasic Daughter-Cards
- Arduino-like interface support tons of off-the-shelf shields



- Human interface
- Quick prototyping
- Fast learning

KIT DE10-LITE: BLOCK DIAGRAM

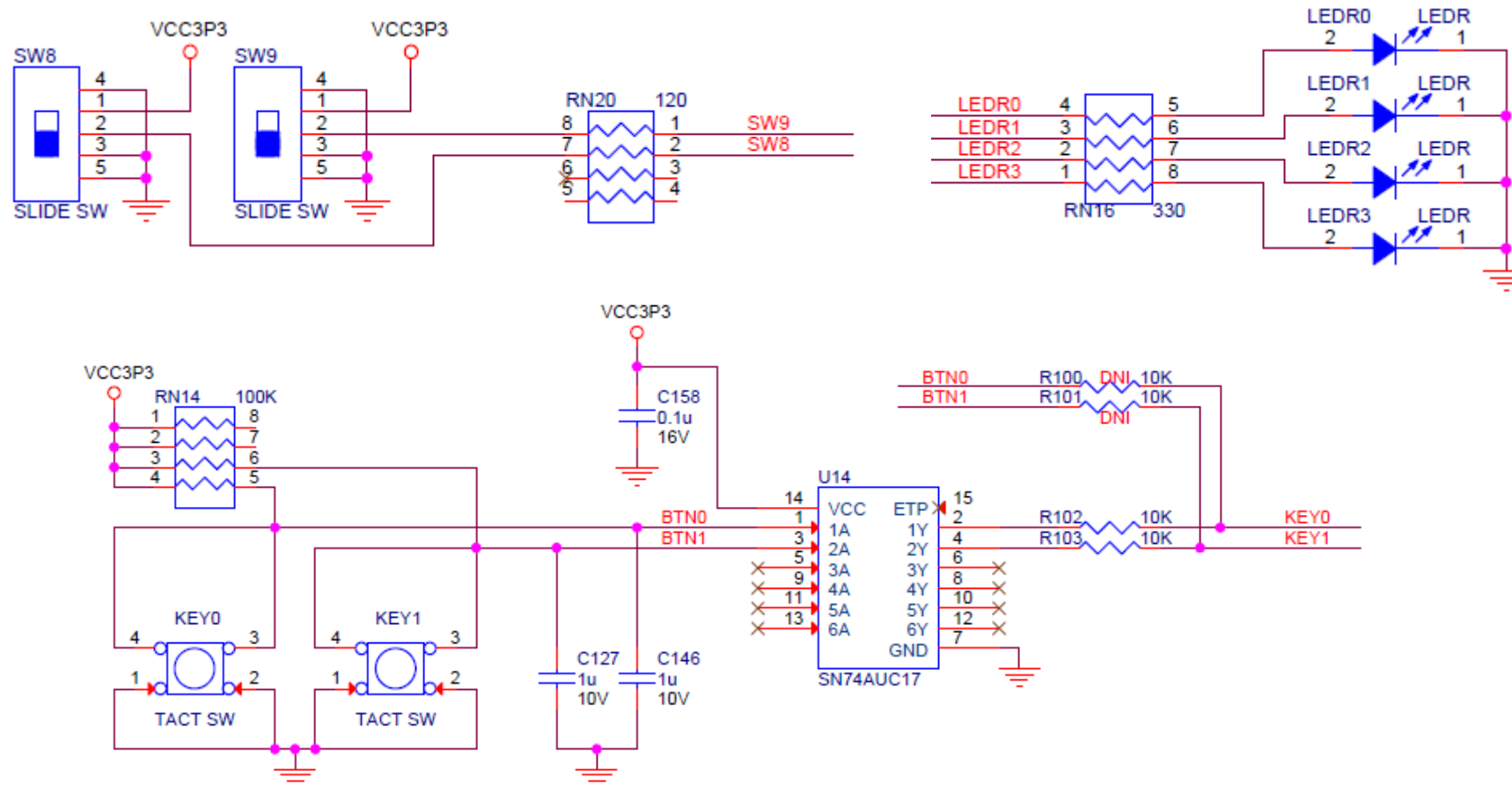


- 50K Logic Elements
- 1.5 Mbit internal memory
- 288 9-bit multipliers
- Up to 360 User I/O pins
- Integrated ADC (2x 12bit 1MSPS)
- Internal Flash Memory (10 Mbit+)
- 4 independent PLLs

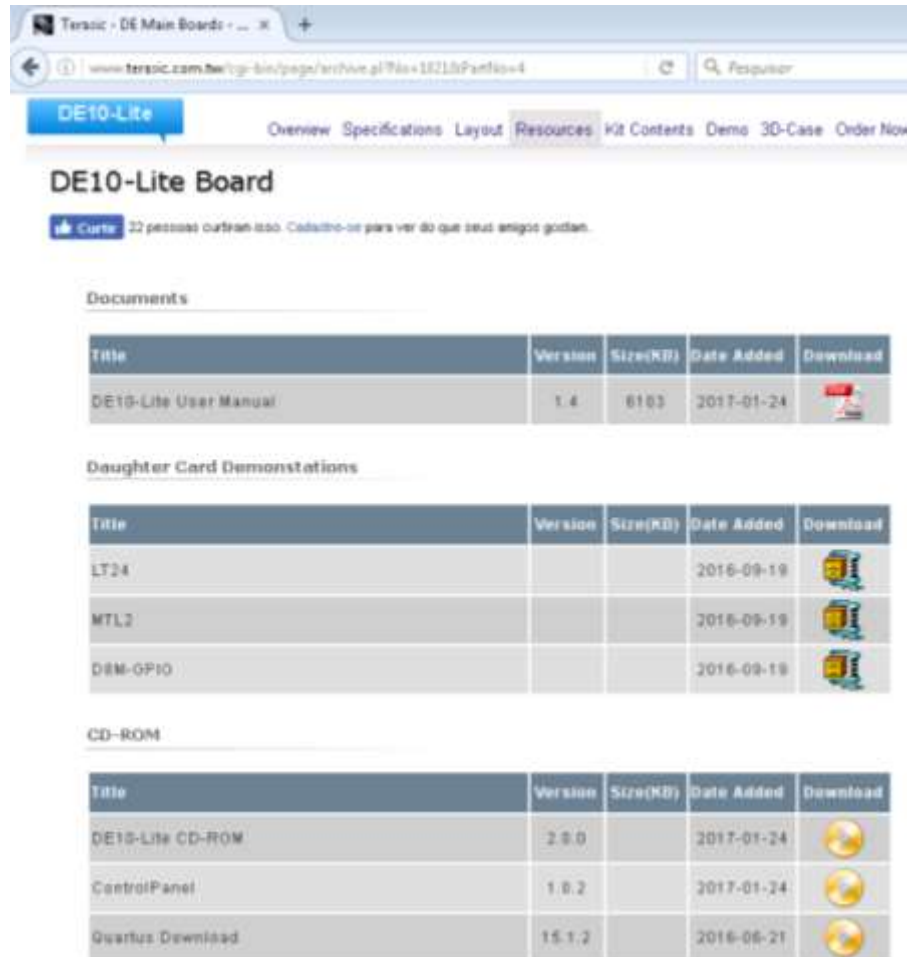
KIT DE10-LITE:

SWITCHES, KEYS AND LEDS ELECTRICAL SCHEMATIC

Don't forget! KEYs are active-low (grounded when depressed)



EXTRA RESOURCES



DE10-Lite Board

Curto 22 pessoas curtiram isso. Cadastre-se para ver do que seus amigos gostam.

Documents

Title	Version	Size(KB)	Date Added	Download
DE10-Lite User Manual	1.4	6163	2017-01-24	

Daughter Card Demonstrations

Title	Version	Size(KB)	Date Added	Download
LT24			2016-09-19	
MTL2			2016-09-19	
DSM-GPIO			2016-09-19	

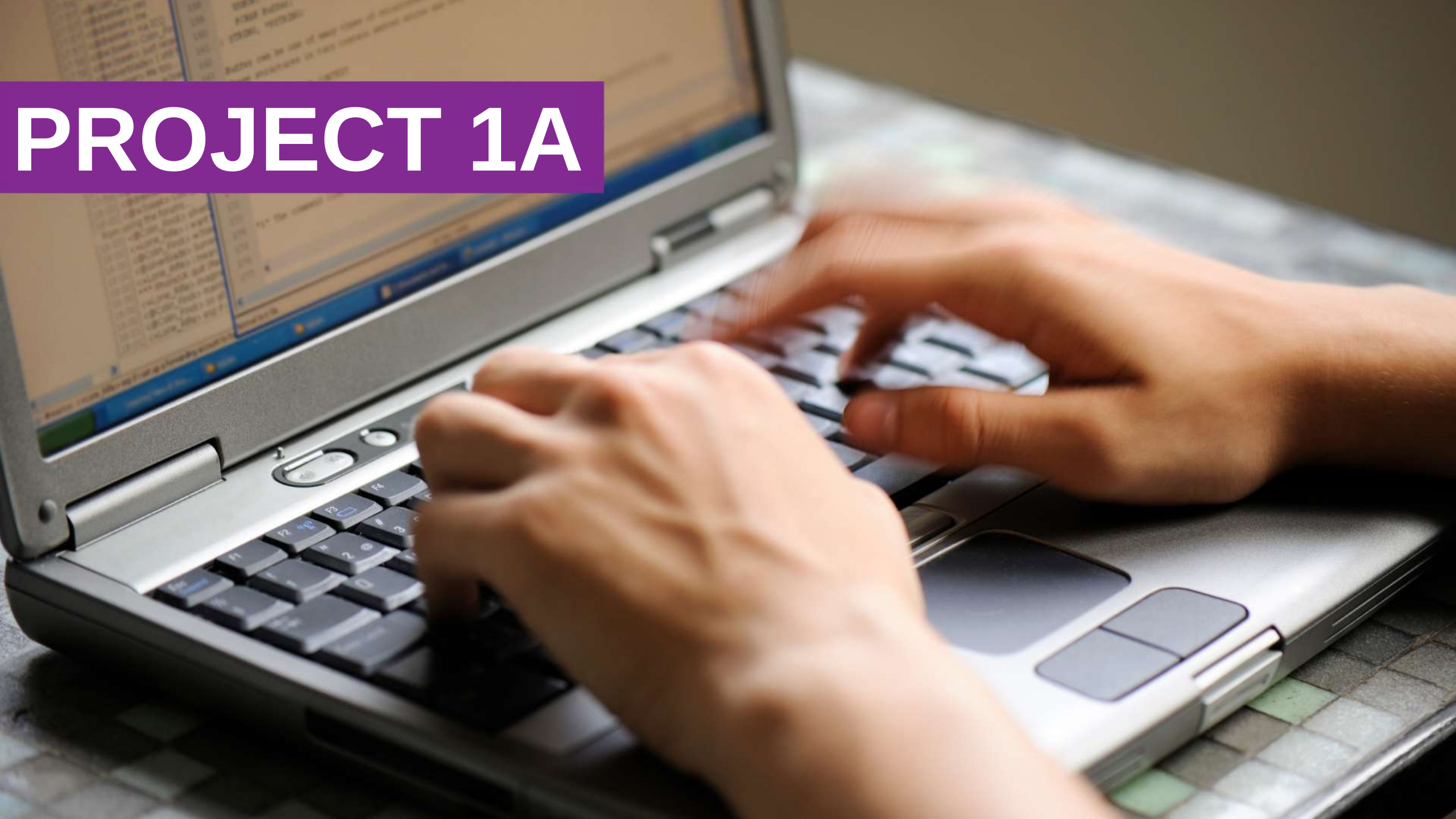
CD-ROM

Title	Version	Size(KB)	Date Added	Download
DE10-Lite CD-ROM	2.0.0		2017-01-24	
ControlPanel	1.0.2		2017-01-24	
Quartus Download	15.1.2		2016-06-21	

- Download the **DE10-Lite CD-ROM** for complete support:
 - User Manual
 - Board Schematic
 - Design Examples
 - Device Datasheets
 - Tools
 - ...and more!

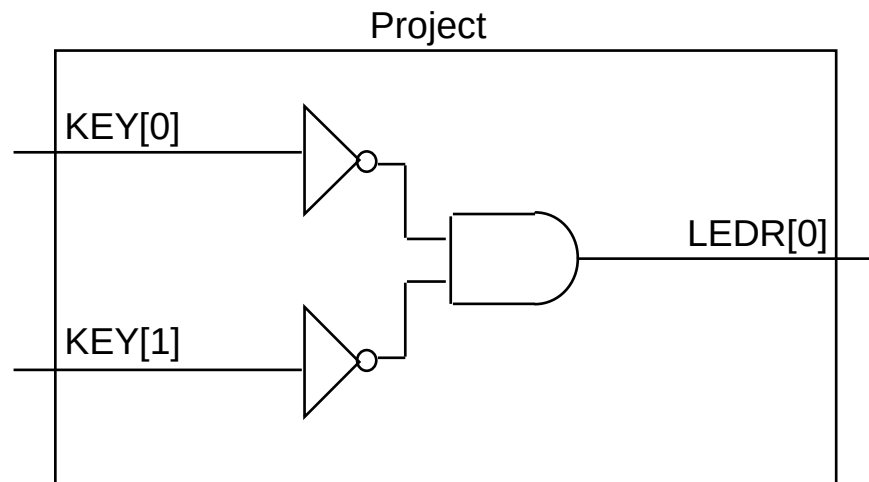
<http://www.terasic.com.tw/cgi-bin/page/archive.pl?No=1021&PartNo=4>

PROJECT 1A



PROJECT 1A

- Open the given **projeto-01** and open the **Pin Planner**
- Create the **pins** with the same names as the diagram below
- Assign the pin's **position** as it can be seen in the schematic below
- Set the pin **voltage** as 3.3-V
- Describe the **module** with their **inputs** and **outputs** as it can be seen in the diagram below
- Use **assign** and **operators** to connect the inputs and outputs



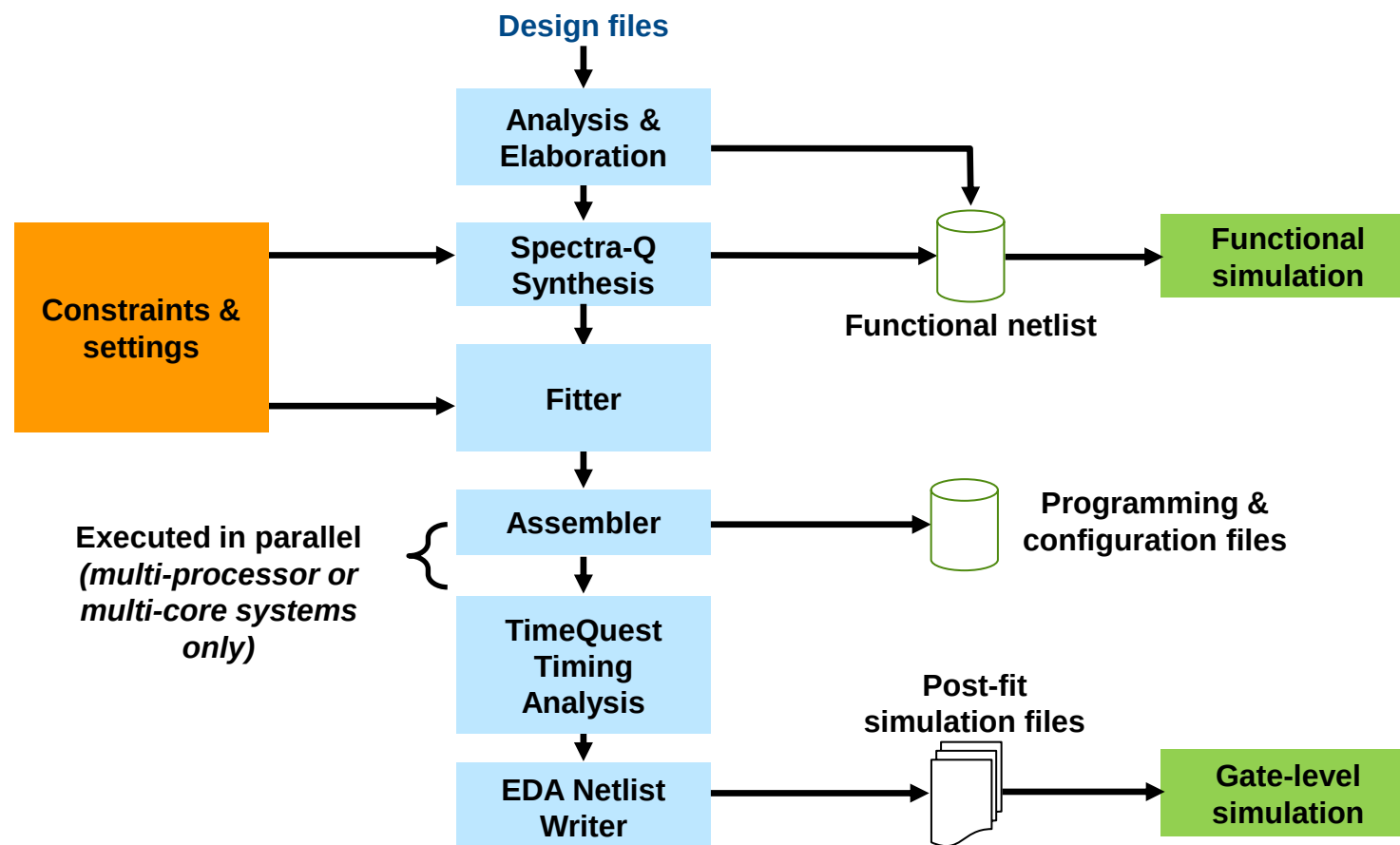
	J11	DIFFIO_RX_T28N
	H12	DIFFIO_RX_T28P
KEY0	B8	DIFFIO_RX_T31N
LEDR1	A9	DIFFIO_RX_T31P
HEX06	C17	DIFFIO_RX_T2N
HEX05	D17	DIFFIO_RX_T2P
	C9	DIFFIO_RX_T30N
LEDR3	B10	DIFFIO_RX_T30P
KEY1	A7	DIFFIO_RX_T29P
LEDR0	A8	DIFFIO_RX_T29N
SW9	F15	DIFFIO_RX_T5N

QUARTUS:

COMPILATION

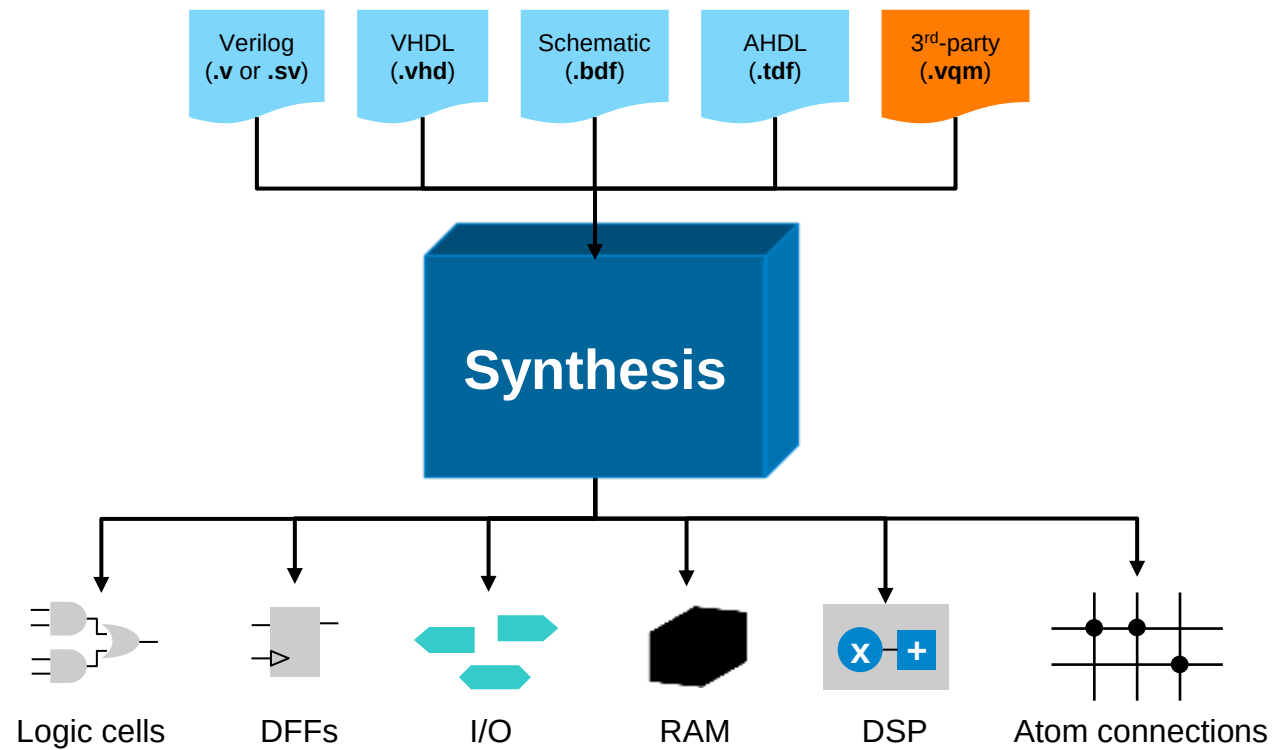


QUARTUS PRIME FULL COMPILATION



WHAT IS SYNTHESIS?

Translates HDL source files into an **atom netlist**



FITTER STAGES

➤ Plan

- Periphery (I/O) placement and routing, clock resource selection

➤ Place

- Core resource placement (logic elements, registers, DSP, RAM)

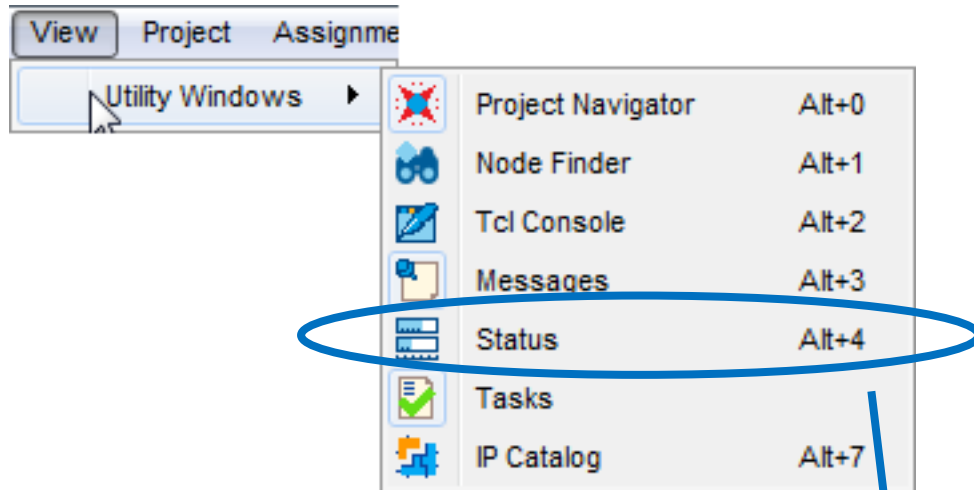
➤ Route

- Core routing connections made

➤ Finalize

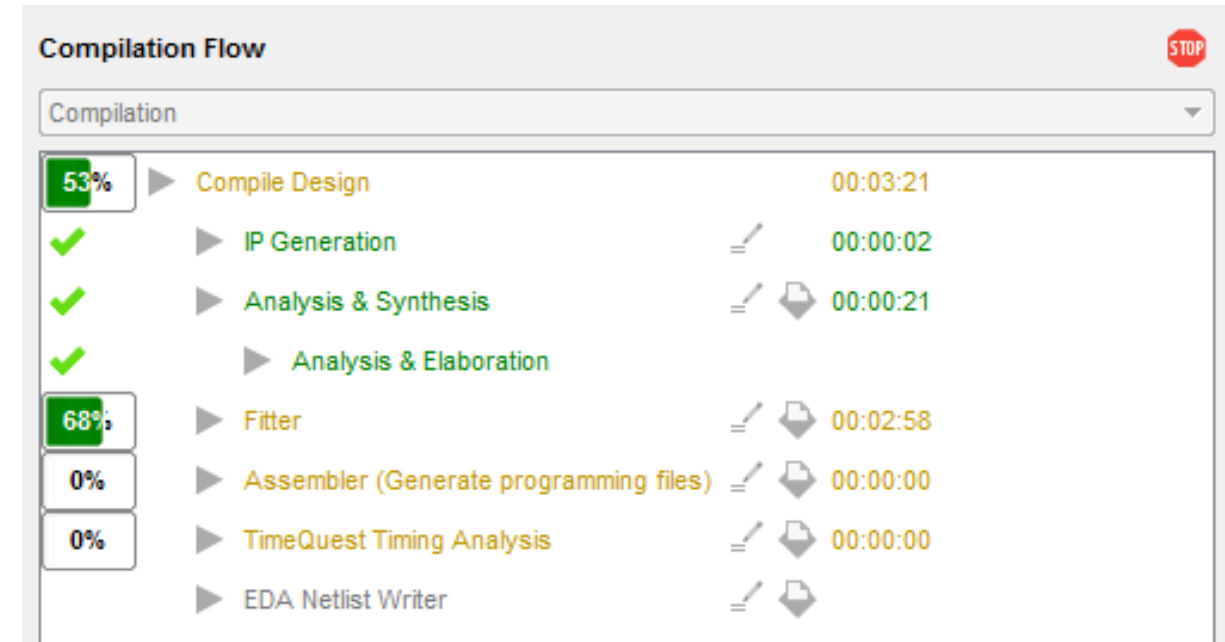
- Post-routing optimizations

STATUS WINDOW AND COMPILATION DASHBOARD



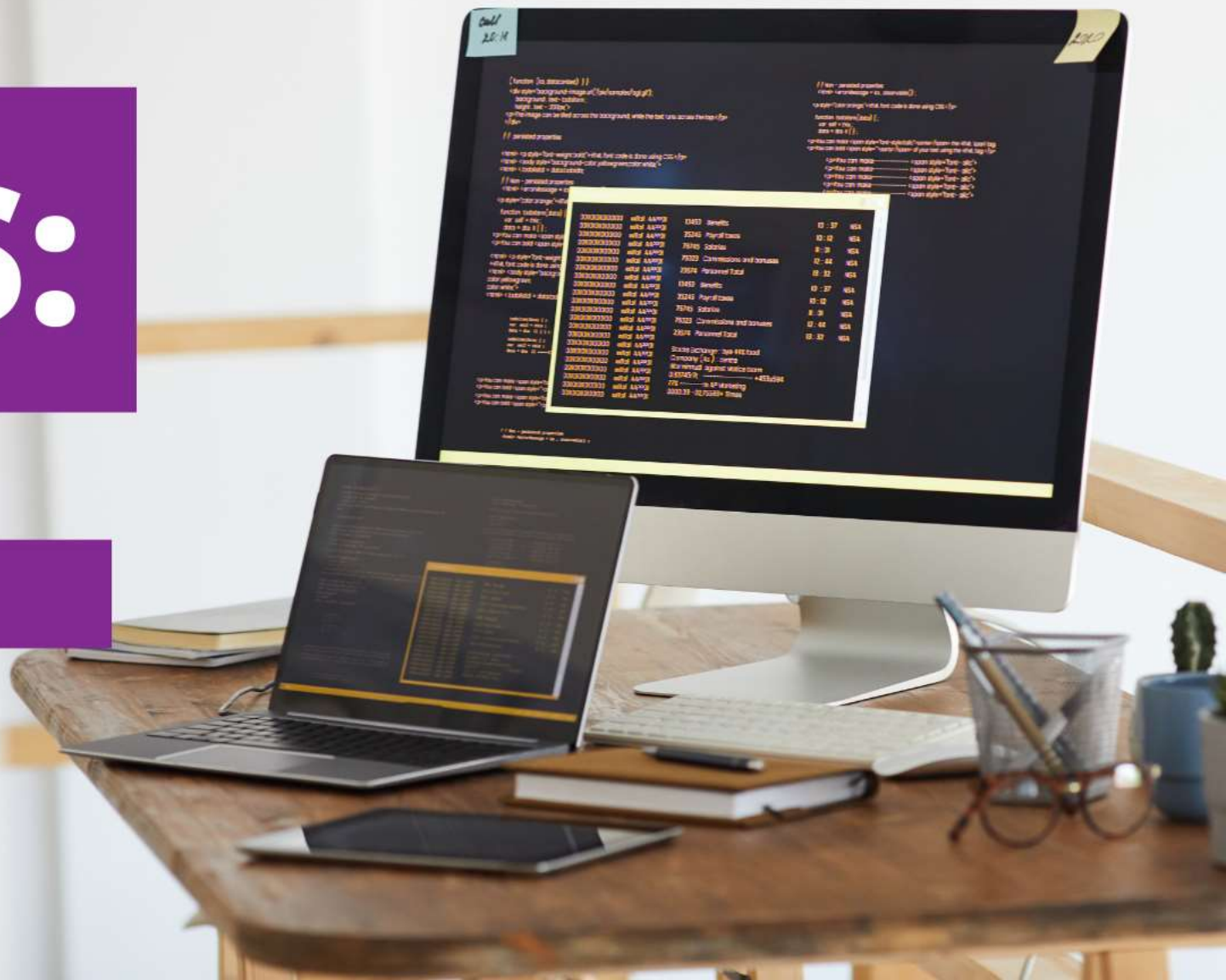
Status

Module	%	Progress	Time
Full Compilation	27%		00:00:57
Analysis & Synthesis	100%		00:00:34
Fitter	35%		00:00:23
Assembler	0%		00:00:00
TimeQuest Timing Analyzer	0%		00:00:00
Post Compilation Processing	0%		00:00:00



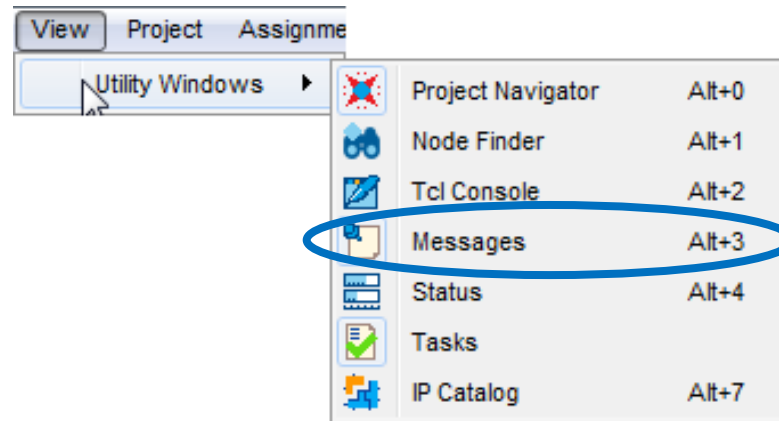
QUARTUS:

COMPILATION MESSAGES



MESSAGES WINDOW

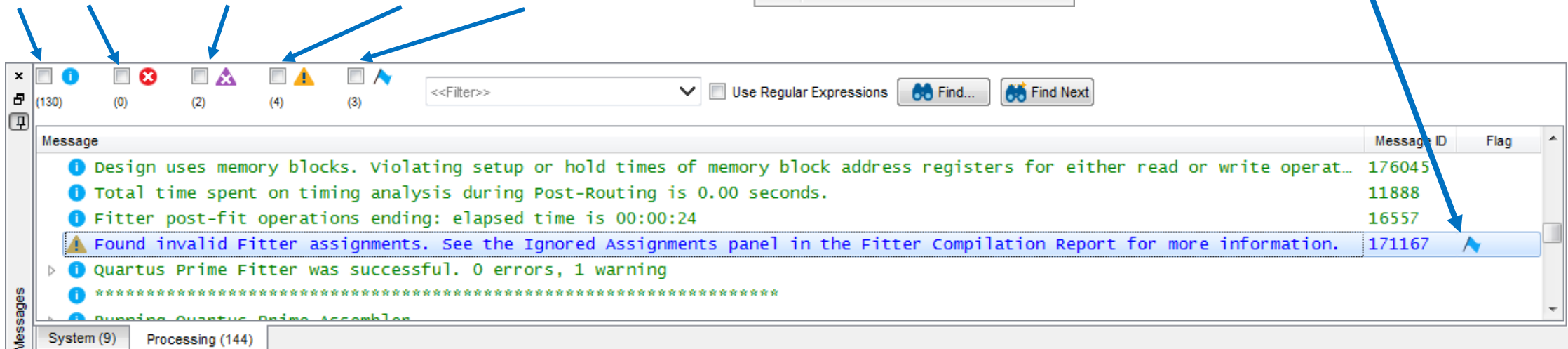
Message window displays informational, warning, and error messages



Filter messages by type

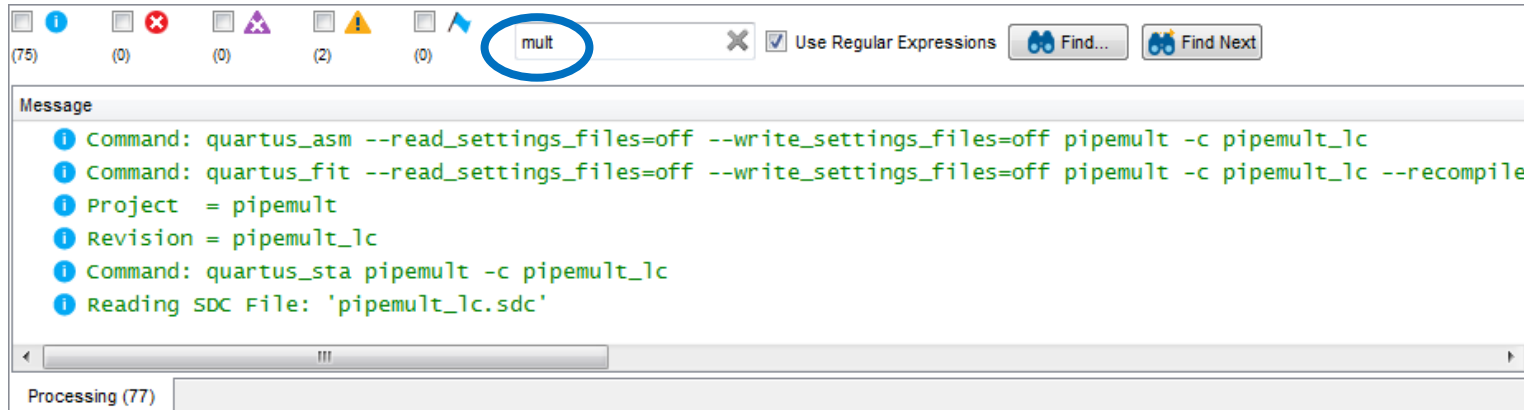
Info Error Critical Warning Warning Flagged

Right-click to flag selected messages for later review

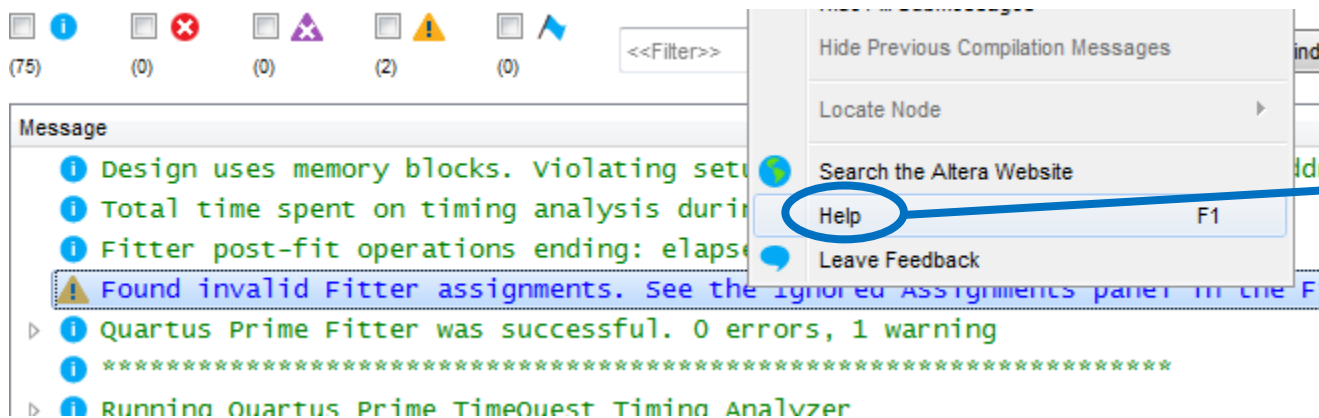


MESSAGE SEARCHING & HELP

Search for keywords in the Messages window



Right-click to search built-in help



MESSAGE IDs

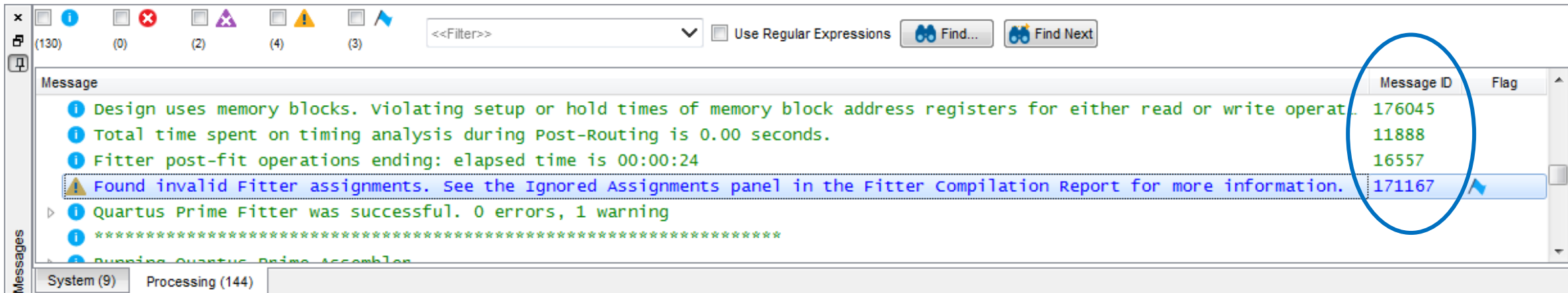
➤ Most messages have IDs

➤ Benefits

- More easily reference messages in service requests
- Search for collateral documentation (self-help)
- Disable specific info or warning messages globally (MESSAGE_DISABLE assignment)

➤ Benefits for IP developers

- Disable info or warning messages specifically for an IP core
- Ship “warning-free” IP



The screenshot shows the Messages window in Quartus Prime. The window has a toolbar with icons for message types (info, error, warning, etc.) and a search bar. The message list is displayed below, with columns for Message ID and Flag. The message 'Found invalid Fitter assignments. See the Ignored Assignments panel in the Fitter Compilation Report for more information.' is highlighted, and its ID, 171167, is circled in blue.

Message	Message ID	Flag
Design uses memory blocks. Violating setup or hold times of memory block address registers for either read or write operation.	176045	
Total time spent on timing analysis during Post-Routing is 0.00 seconds.	11888	
Fitter post-fit operations ending: elapsed time is 00:00:24	16557	
Found invalid Fitter assignments. See the Ignored Assignments panel in the Fitter Compilation Report for more information.	171167	
Quartus Prime Fitter was successful. 0 errors, 1 warning		

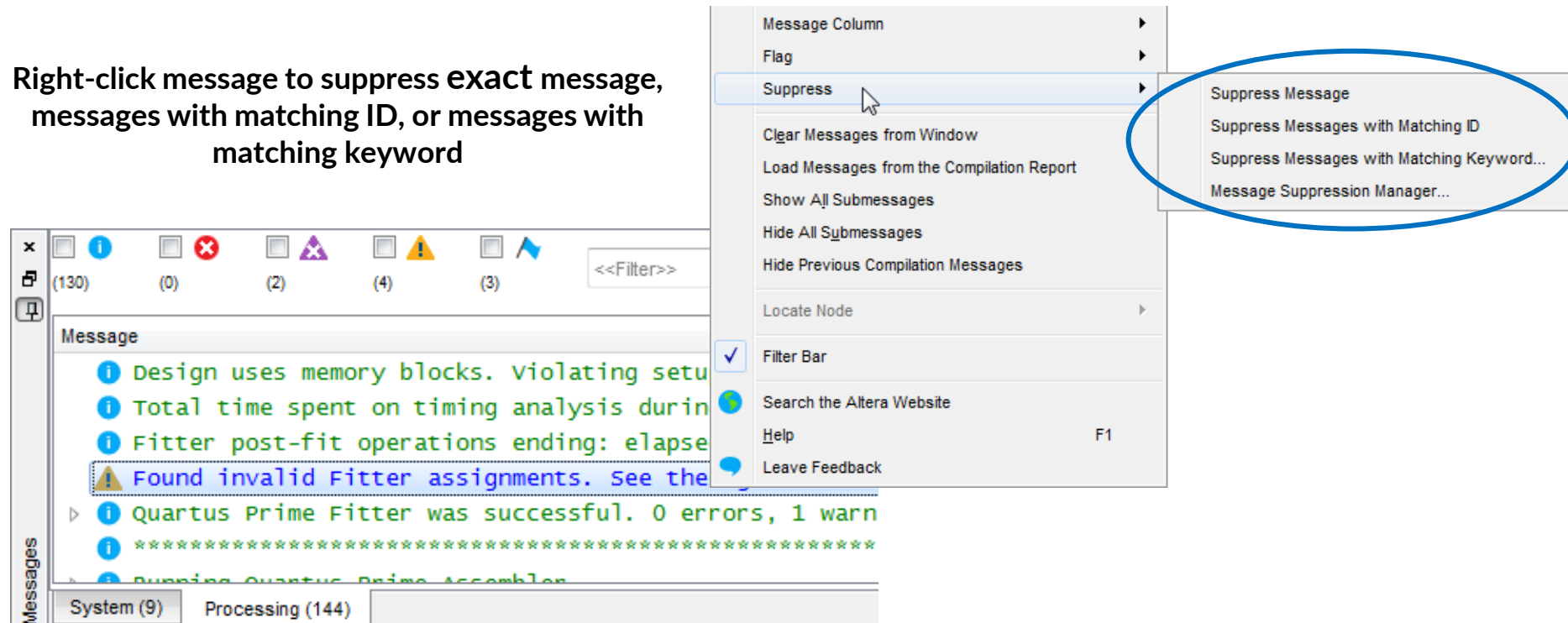
Running Quartus Prime Assembler		

System (9) Processing (144)

MESSAGE SUPPRESSION

- Hide messages from current and future compiles
 - E.g. known synthesis warning message already investigated
- Store suppression rules in `<revision_name>.srf` file

Right-click message to suppress exact message, messages with matching ID, or messages with matching keyword



QUARTUS:

VIEWING COMPILATION RESULTS:

- NETLIST VIEWER
- CHIP PLANNER



NETLIST VIEWERS

➤ RTL Viewer

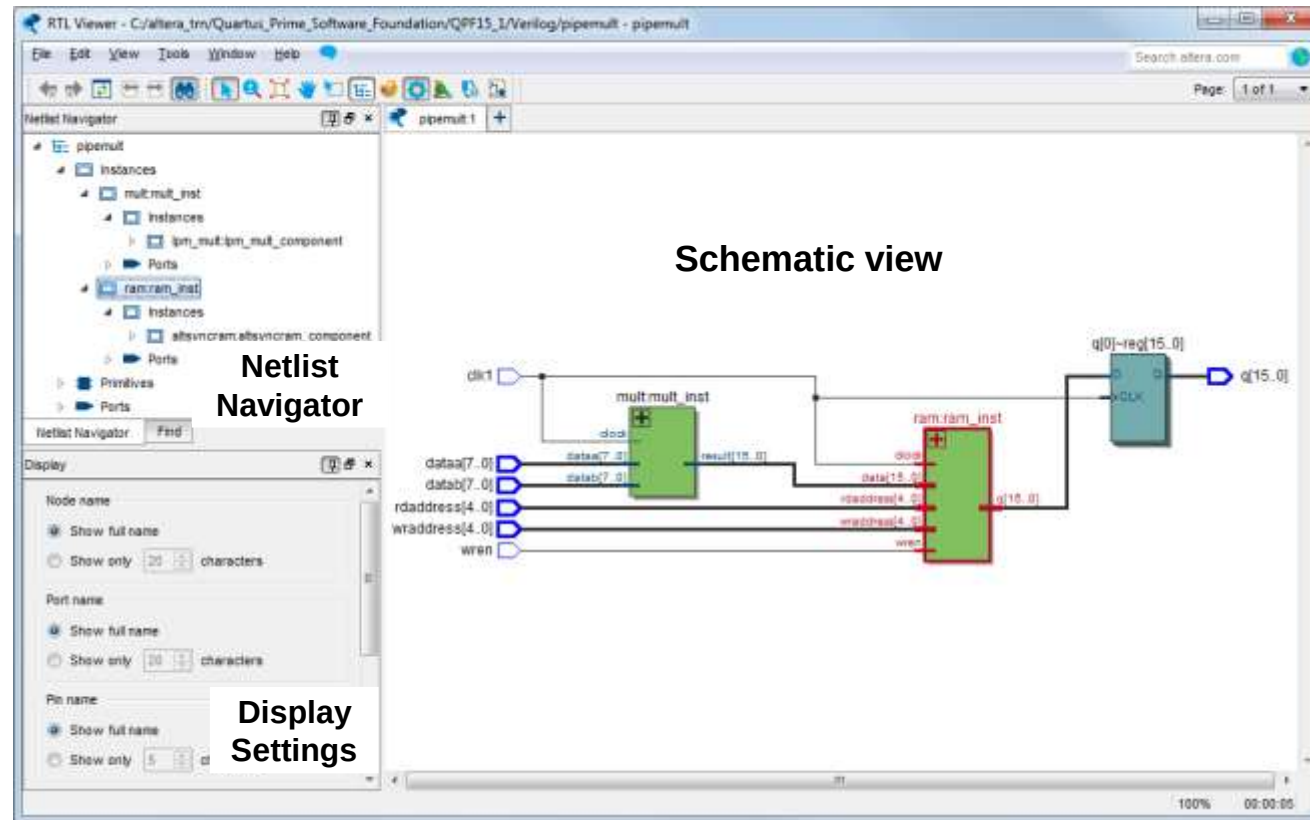
- Schematic of design after Analysis & Elaboration
- Visually check initial HDL before synthesis optimizations
- Locate synthesized nodes for assigning constraints
- Debug verification issues

➤ Technology Map Viewers (Post-Mapping or Post-Fitting)

- Graphically represents results of mapping (post-synthesis) & fitting
- Analyze critical timing paths graphically
- Locate nodes & node names after optimizations (cross-probing)

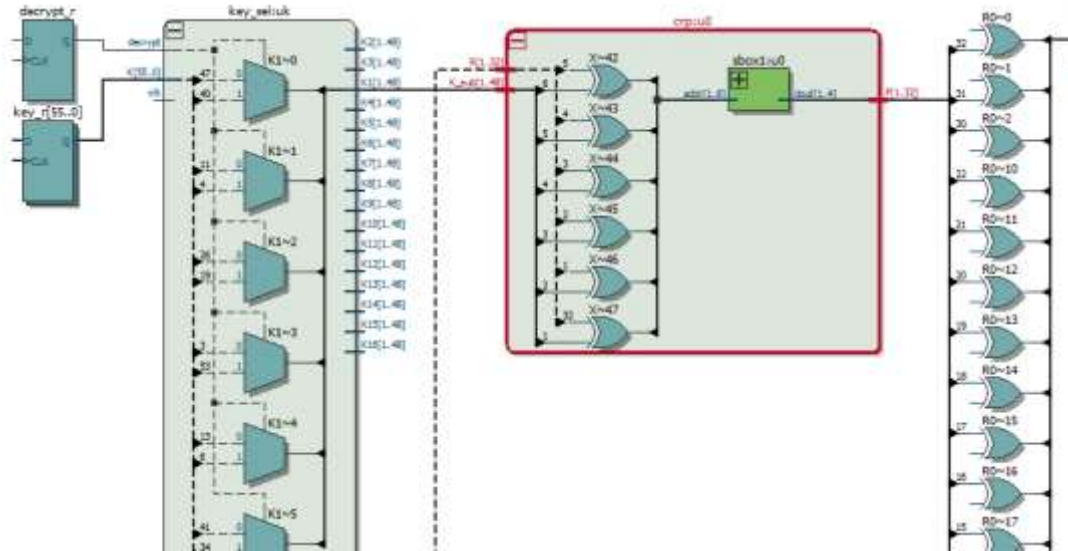
NETLIST VIEWERS

Tools menu → Netlist Viewers



Note: Must perform elaboration first (e.g. Analysis & Elaboration OR Analysis & Synthesis)

SCHEMATIC VIEW (RTL VIEWER)



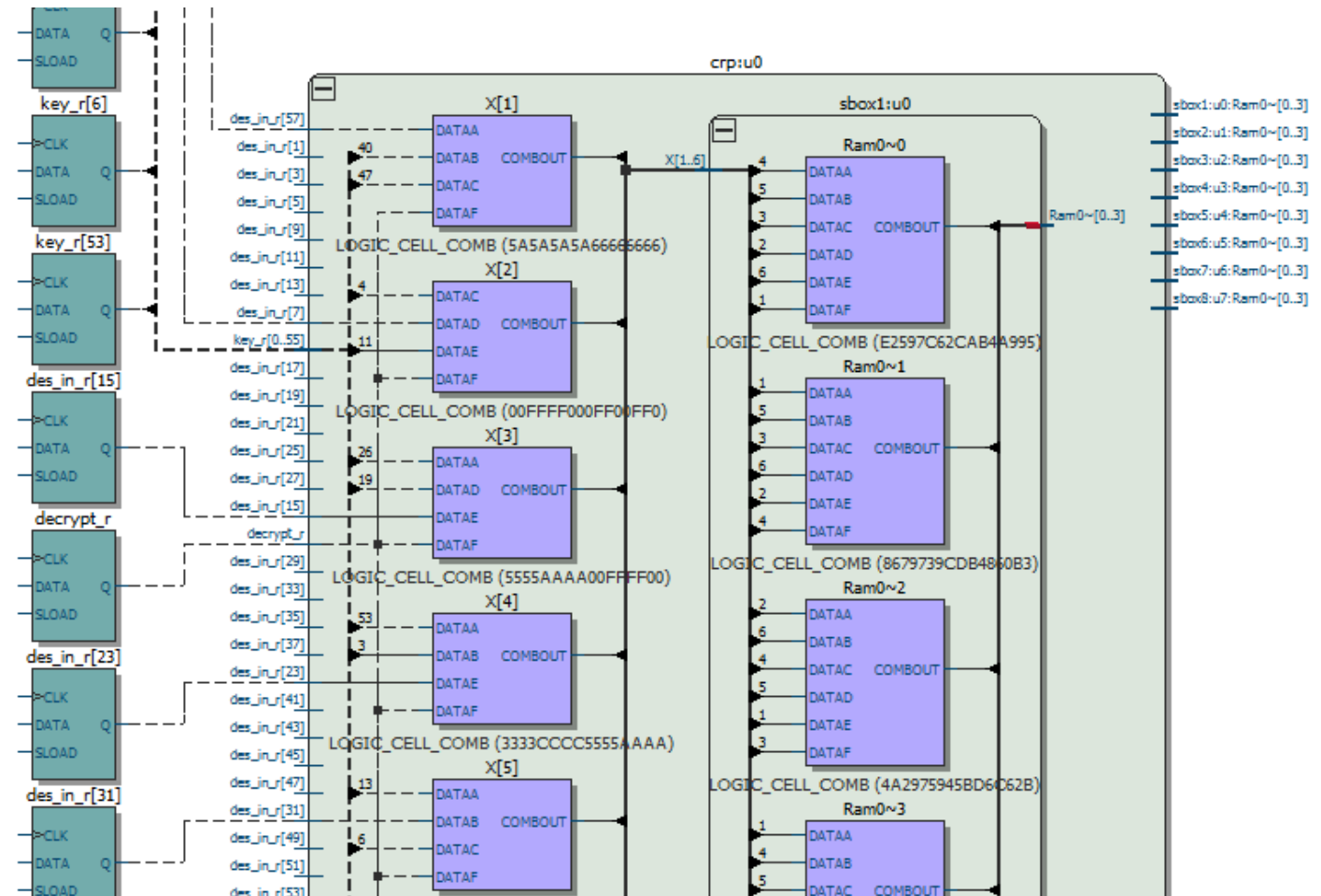
➤ Represents design using logic blocks & nets

- I/O pins
- Registers
- Muxes
- Gates (AND, OR, etc.)
- Operators (adders, multipliers, etc.)

SCHEMATIC VIEW (TECHNOLOGY MAP VIEWER)

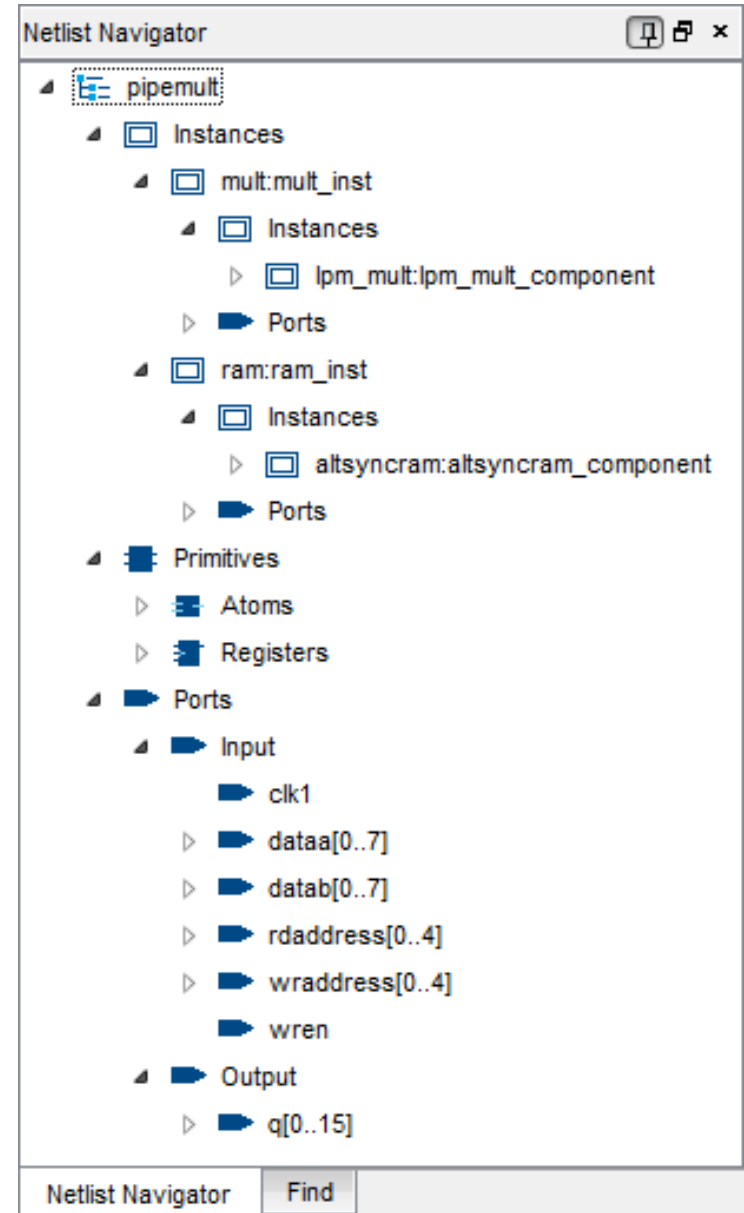
➤ Represents design using atoms

- I/O pins & cells
- Logic cells (Lcells)
- Memory blocks
- MAC (DSP blocks)

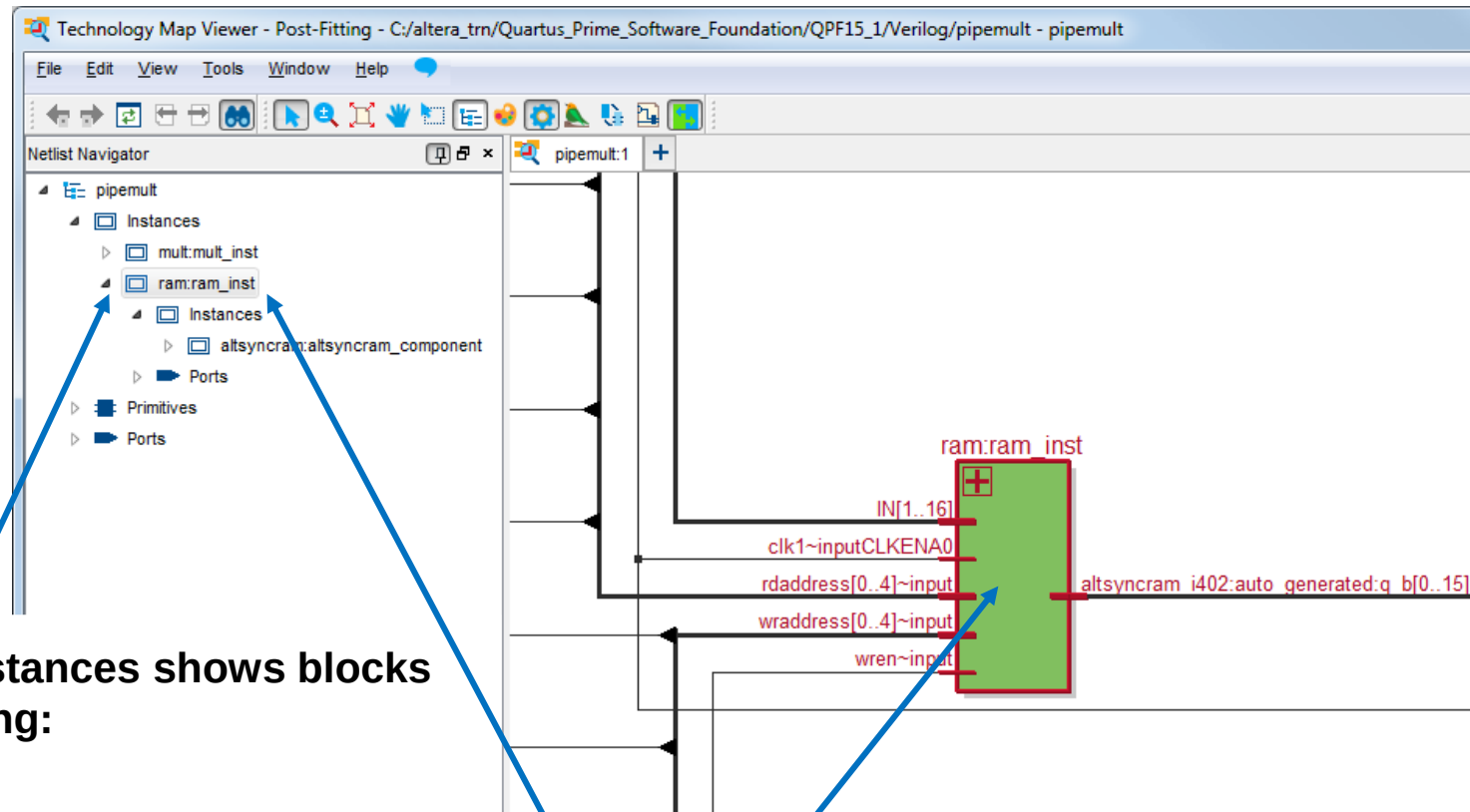


NETLIST NAVIGATOR (HIERARCHY LIST)

- Traverse between levels of design hierarchy
- View logic schematic for each hierarchical level
- Break down each hierarchical level into netlist elements or atoms
 - Instances
 - Registers
 - Buffers
 - Logic
 - Operators
 - Atoms
 - I/O
 - Ports
 - State machines



USING THE NETLIST NAVIGATOR



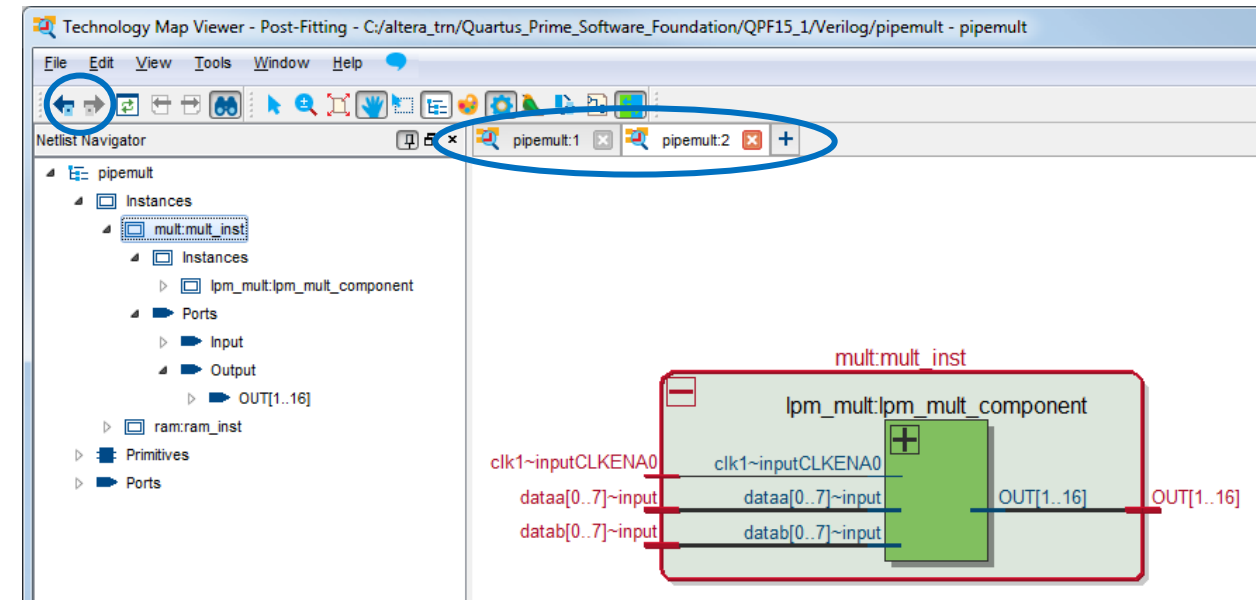
Expanding instances shows blocks within including:

- Instances
- Primitives
- Ports

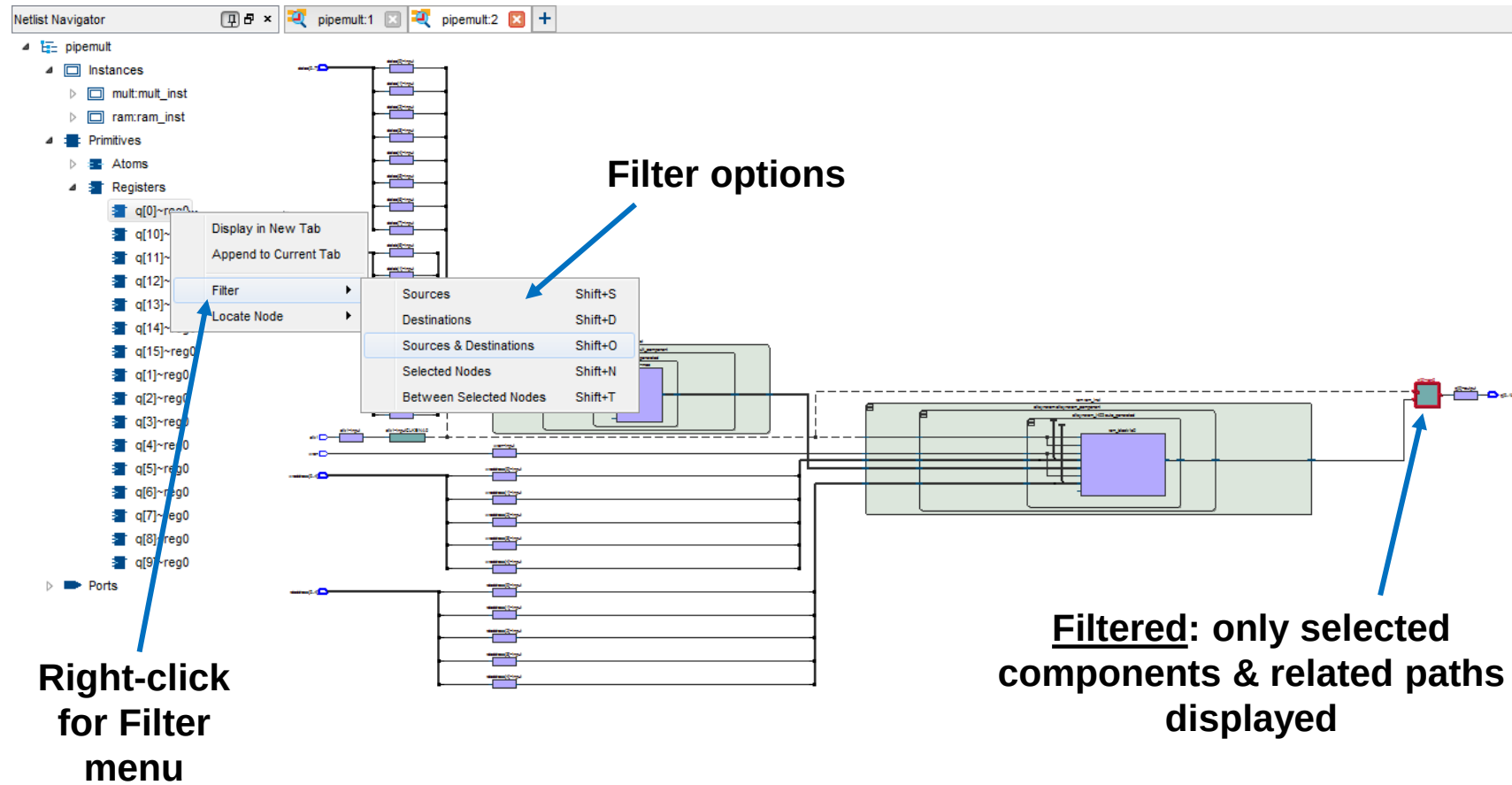
Highlighting object in Netlist Navigator highlights that element in schematic view

USING VIEW TABS

- Open multiple tabs to create and store particular views
- To create new tab:
 - Click +
 - Drag node(s) from Netlist Navigator to new empty tab or
 - Right-click on block in schematic or hierarchy and select **Display in New Tab**
- Tile or cascade tabs to see views simultaneously

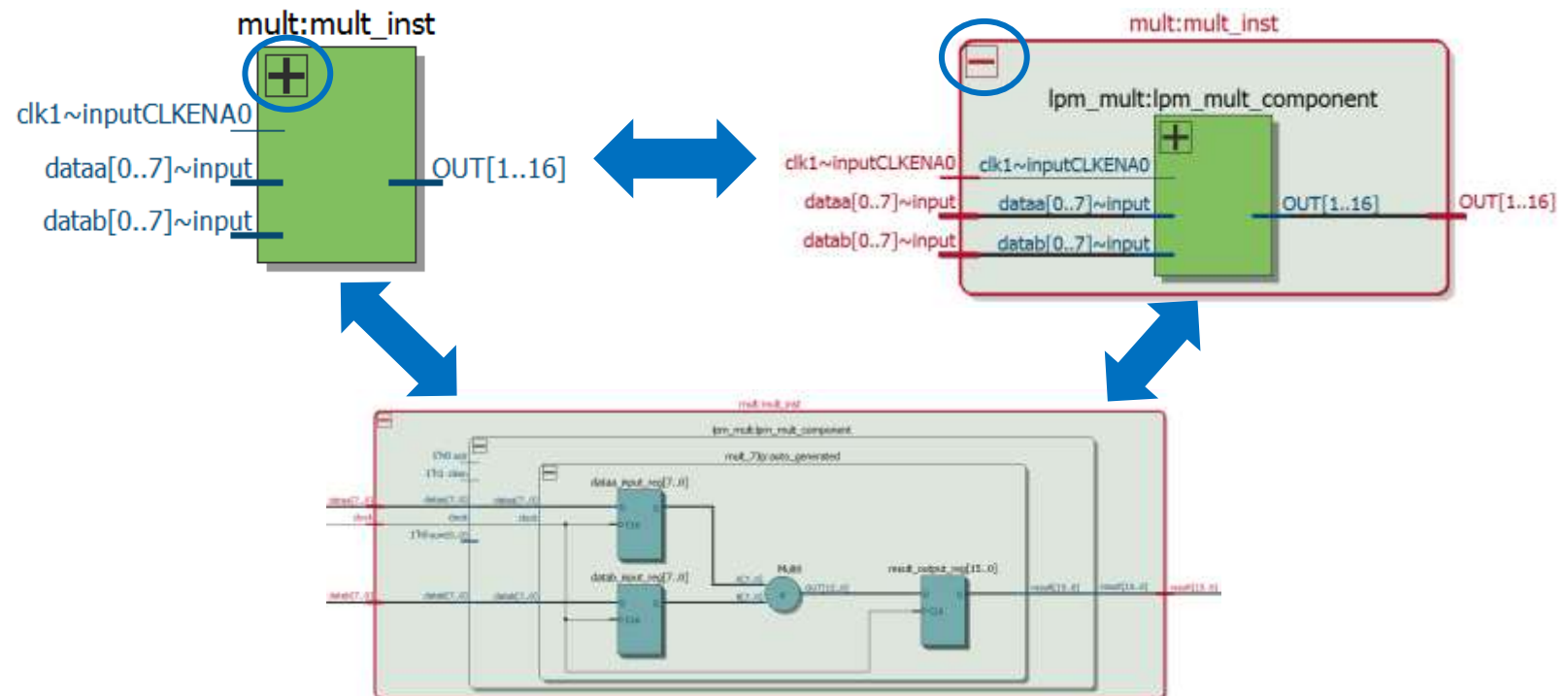


FILTER SCHEMATIC



INSTANCE UNWRAP/WRAP

- Unwrap instance by clicking + to expand and see logic within
- Wrap instance by clicking - to hide expanded logic



OTHER FEATURES

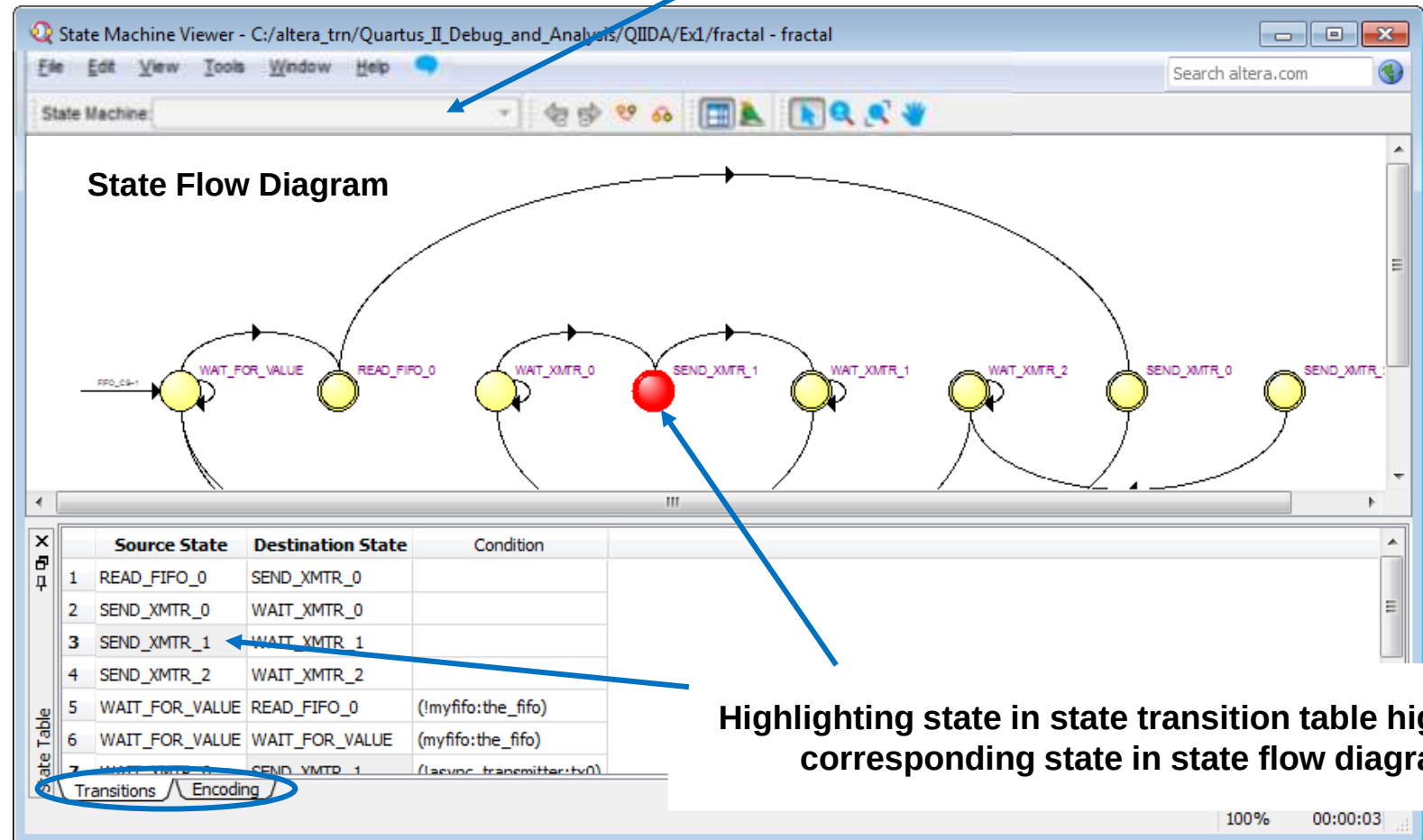
- Bird's Eye View: overall panning view of design
- Block properties view (right-click)
 - Fan-in, fan-out, ports, parameters, logic diagram of atom
- Design search (**Find** in **Edit** menu or toolbar)
- Lookup table (LUT) internal detail as gate schematic or truth table
- Port/pin connectivity details: in-place view of all connections to/from port without filtering
- Shift-click instances in schematic, then drag and drop to reorganize view
- Cross-probing: locate nodes from/to
 - Design files
 - Assignment Editor
 - Chip Planner
 - Resource Property Editor
 - RTL/Technology Map Viewers
 - Pin Planner
 - TimeQuest timing reports
 - SignalTap II logic analyzer

STATE MACHINE VIEWER

Tools menu → Netlist Viewers

(State machines will be discussed later)

Use drop-down to select from multiple state machines



Highlighting state in state transition table highlights corresponding state in state flow diagram

CHIP PLANNER

➤ Graphical view of design resource usage in target device

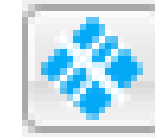
➤ Displays

- Graphical layout of device resources
- Routing channels between device resources
- Global clock regions

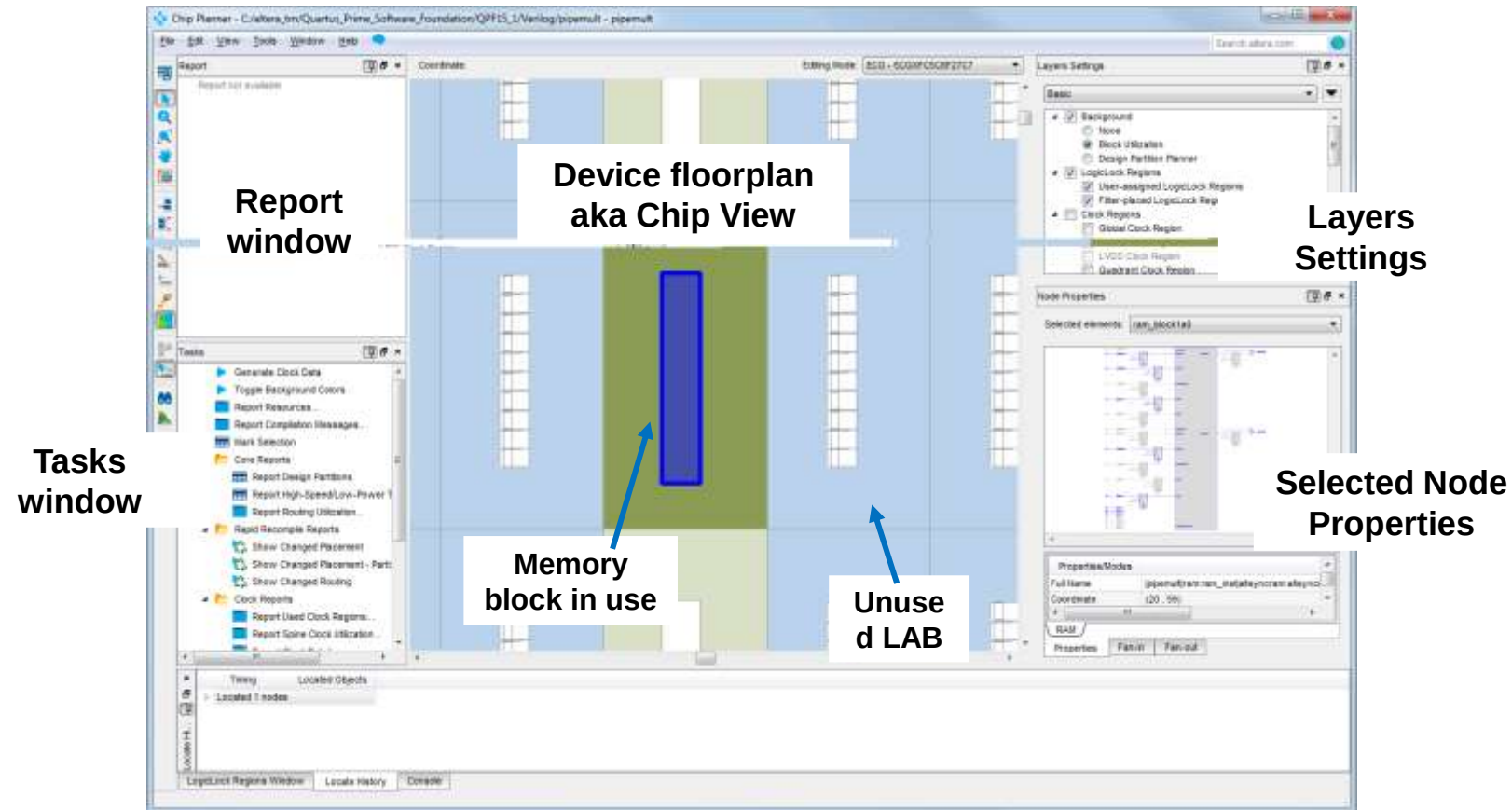
➤ Uses

- View placement of design logic
- View connectivity between resources used in design
- Make placement assignments
- Debugging placement-related issues

CHIP PLANNER

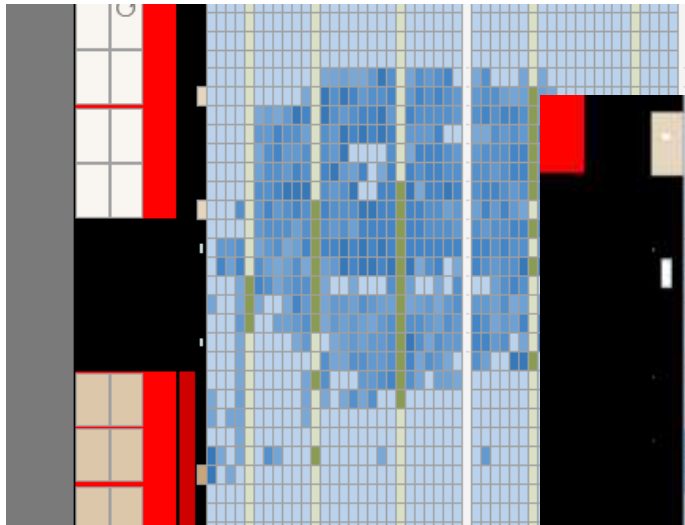


Tools menu or toolbar

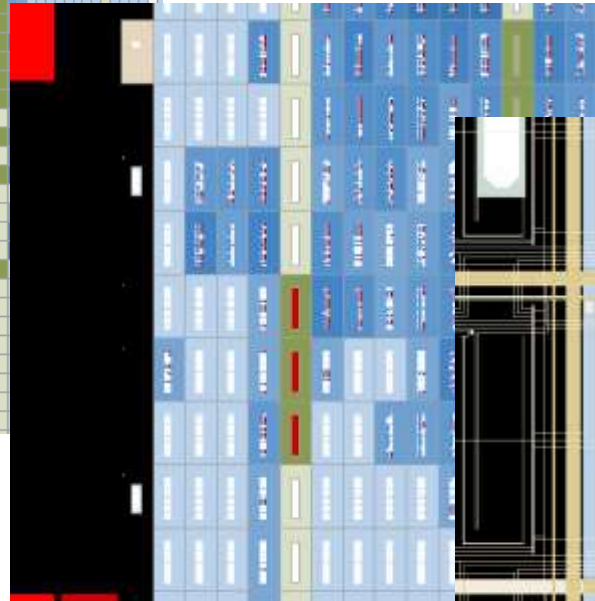


FLOORPLAN VIEWS

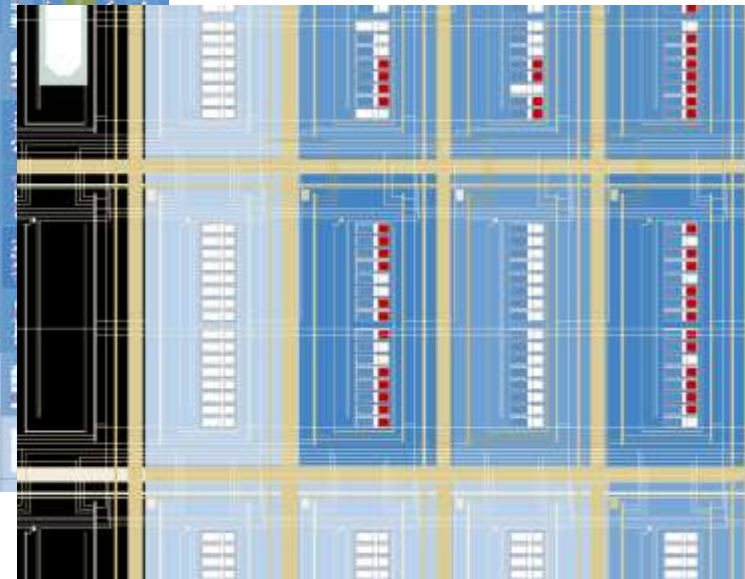
Overall device resource usage



Lower level block usage



Lowest level routing detail



Zoom in for detailed logic implementation & routing usage

BLOCK RESOURCE PROPERTIES

Click any block to view the internal resources detail

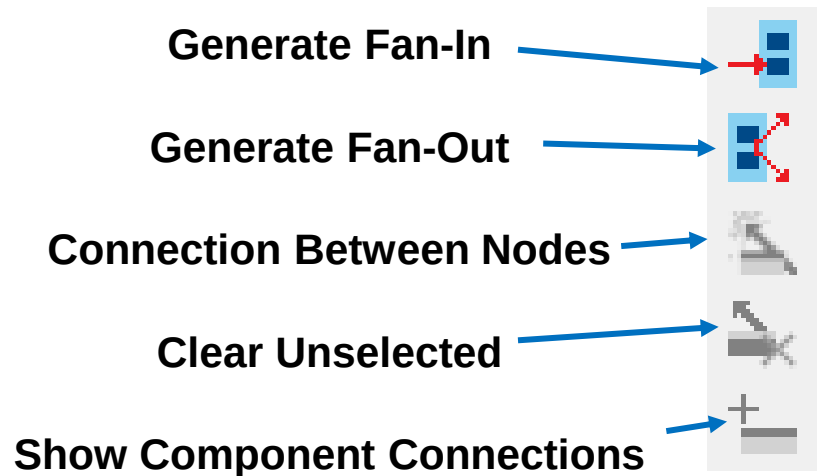
Double-clicking on the resource detail invokes the Resource Property Editor for making low-level changes

The screenshot shows the Altera IDE interface. On the left, a circuit diagram is displayed with a grid of resource blocks. One block is highlighted in blue. An orange arrow points from the text 'Click any block to view the internal resources detail' to this highlighted block. Another orange arrow points from the text 'Double-clicking on the resource detail invokes the Resource Property Editor for making low-level changes' to the same block. The 'Node Properties' window is open on the right, showing the internal resources detail for the selected element 'Add31~13'. The window contains a circuit diagram of the internal resources and a table of properties.

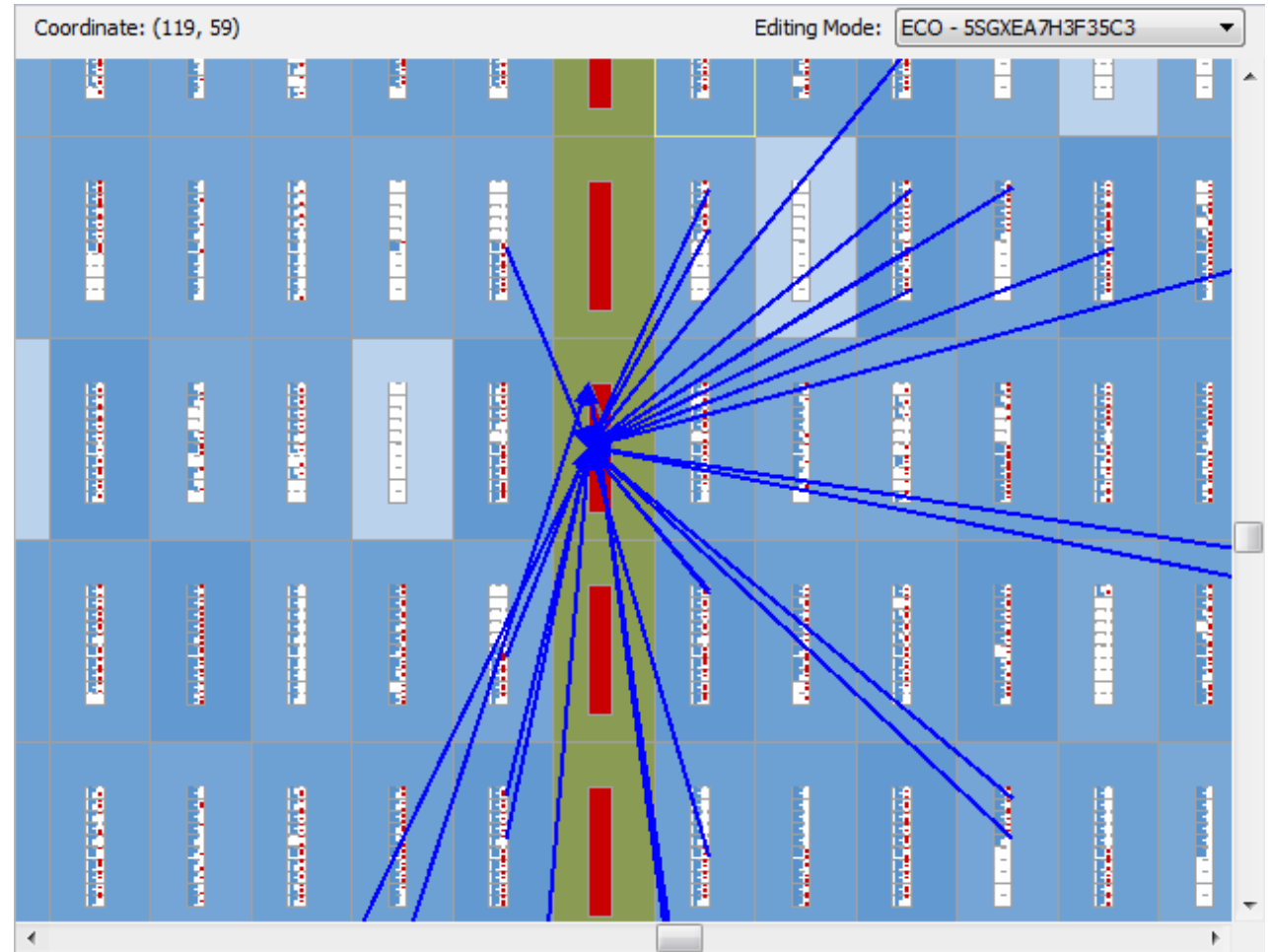
Properties/Modes	Values
Full Name	fractal Add31~13
Coordinate	(123 , 58)
Resource Type	MLAB combinational cell
Location	MLABCELL_X123_Y58_N36
Location Assignment	No
F0 LUT Mask	3333
F1 LUT Mask	3333
F2 LUT Mask	FF00
F3 LUT Mask	FF00
Latch Type	none
F0 LUT Equation	1b
F1 LUT Equation	N/A

At the bottom of the window, there are tabs for 'Top Register', 'Top Combinational', 'Bottom Combinational', and 'Bottom 2 Register'. Below these tabs are buttons for 'Properties', 'Fan-in', and 'Fan-out'.

DISPLAYING FAN-IN & FAN-OUT



- 1.Highlight target blocks
- 2.Click fan-in toolbar button
- 3.Highlight target blocks again
- 4.Click fan-out toolbar button

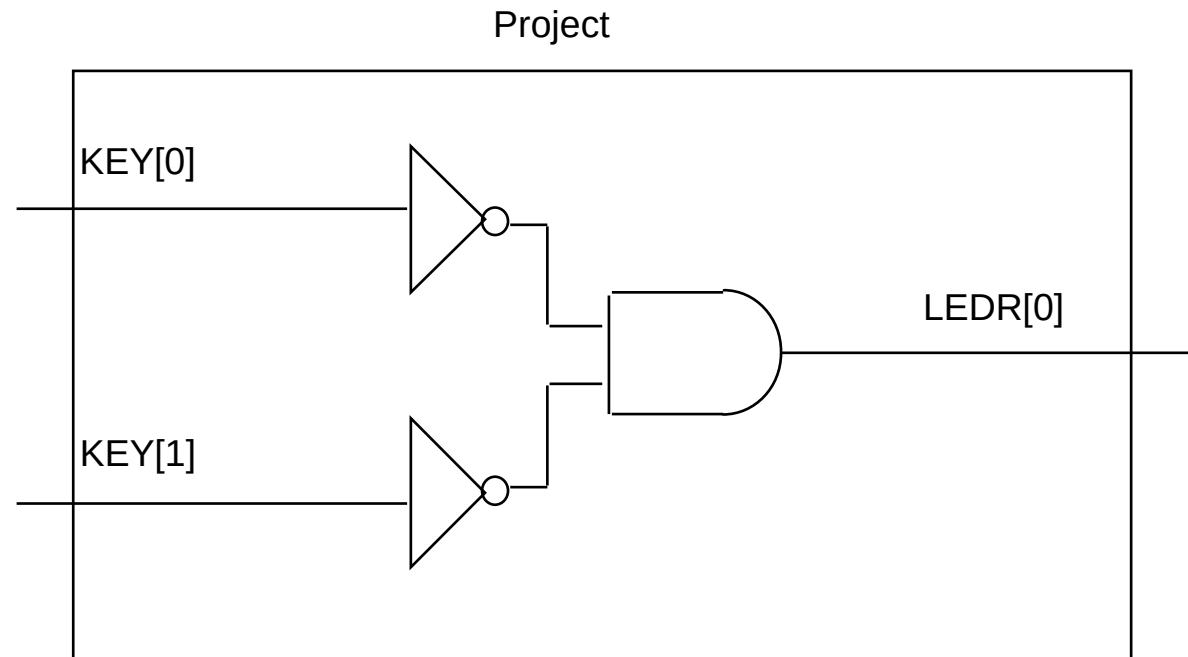


PROJECT 1B



PROJECT 1B

- Compile the **projeto-01** file
- Analyze the compilation **reports**
- Verify the obtained result by **RTL Viewer**



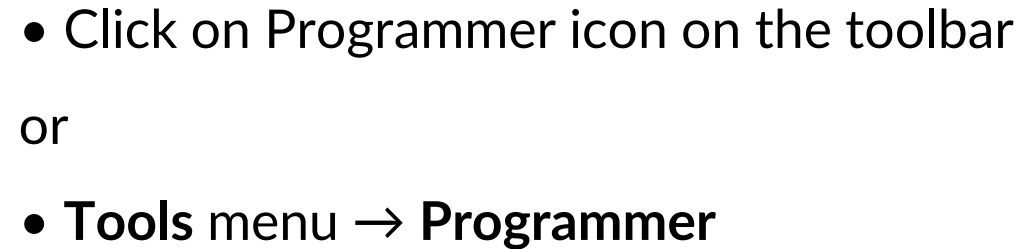
QUARTUS:

HOW TO CONFIGURE
THE FPGA:

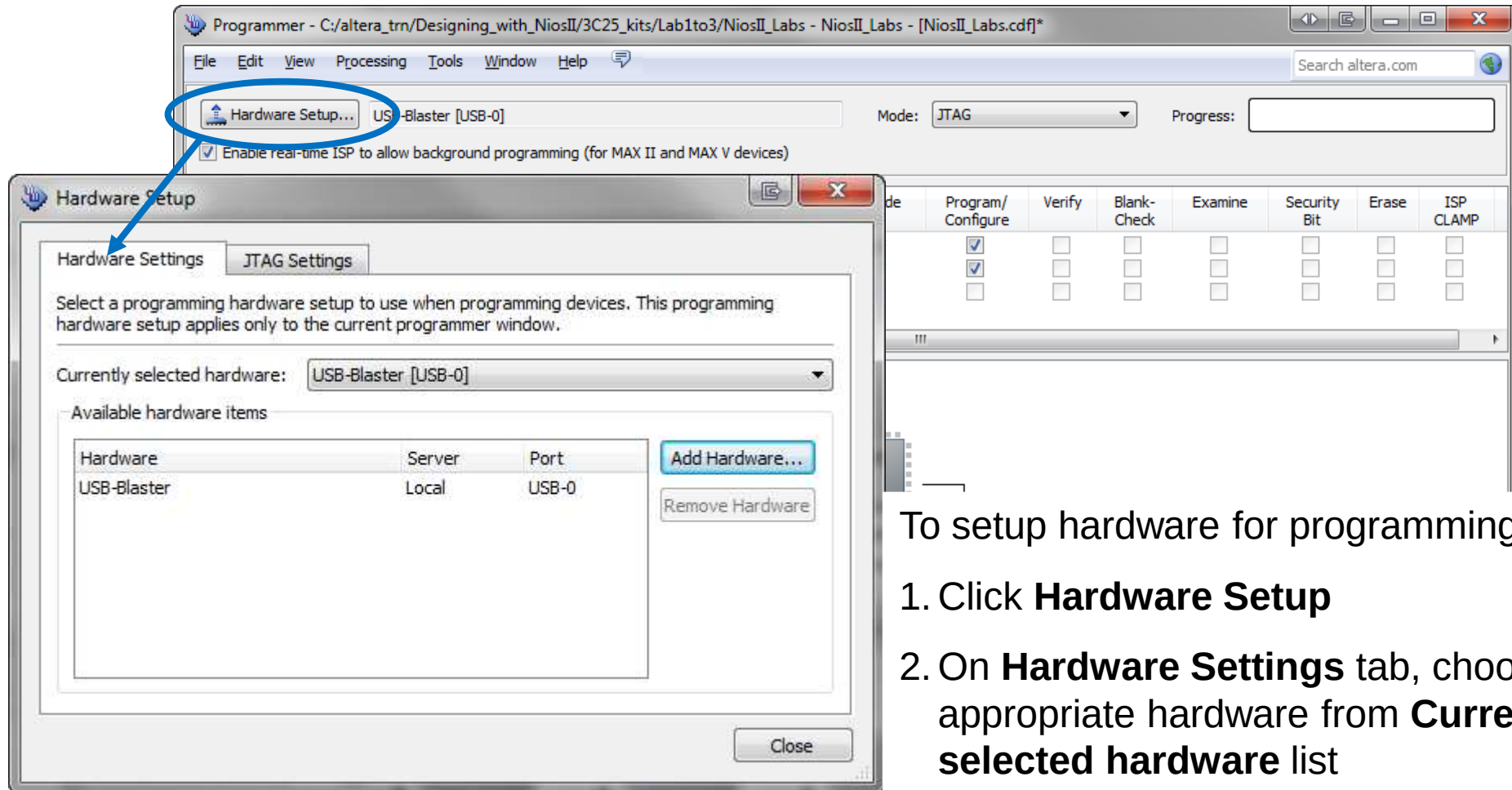
QUARTUS
PROGRAMMER



Use to configure devices in-system



SETTING UP THE PROGRAMMING HARDWARE



To setup hardware for programming:

1. Click **Hardware Setup**
2. On **Hardware Settings** tab, choose appropriate hardware from **Currently selected hardware** list

SUPPORT PROGRAMMING FILES

➤ **.sof** (SRAM Object File)

- Used to configure FPGAs directly from Quartus software through download cable
- Always generated by default during a full compilation

➤ **.pof** (Programming Object File)

- Used to program CPLDs and configuration devices

➤ **.jam/.jbc**

- ASCII file used by processors and test equipment to program devices via JTAG

➤ **.jic** (JTAG Indirect Configuration File)

- Contains programming data for target FPGA
- Used to program EPCS (Altera serial configuration) devices through their dedicated configuration interface connection to an FPGA
- Configuration devices do not have JTAG interfaces!

JTAG CHAIN

When adding files, the device for that file is automatically chosen

Mode: JTAG

File	Device	Checksum	Usercode	Program/Configure	Verify	Blank-Check	Examine	Security Bit	Erase	ISP CLAMP
NiosII_Labs.sof	EP3C25F324	0053074A	FFFFFFFF	☒	☐	☐	☐	☐	☐	☐
../././Quartus_II_Softw...	EP4CE6F17	0008DCE8	FFFFFFFF	☒	☐	☐	☐	☐	☐	☐
<none>	MY_DEVICE	00000000	<none>	☐	☐	☐	☐	☐	☐	☐

Enable programming

JTAG Chain:

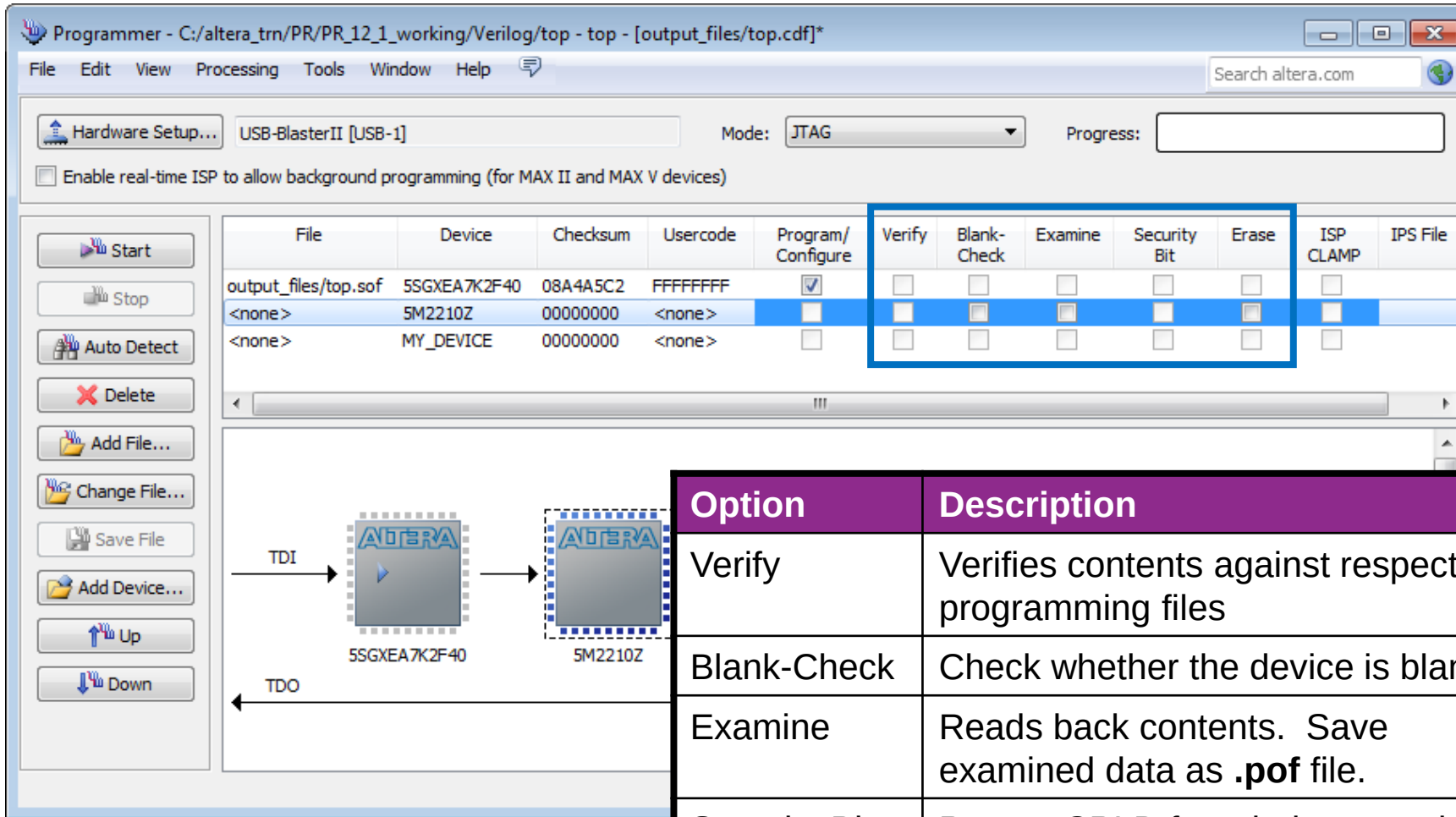
- Can consist of combination of Altera FPGAs and CPLDs as well as non-Altera (or user) devices*
- Devices in the chain will be programmed from top down

Change the order of the device chain

The diagram shows a JTAG chain with three devices: EP3C25F324, EP4CE6F17, and MY_DEVICE. The chain is connected via TDI and TDO lines.

* Non-Altera devices will be issued JTAG BYPASS command

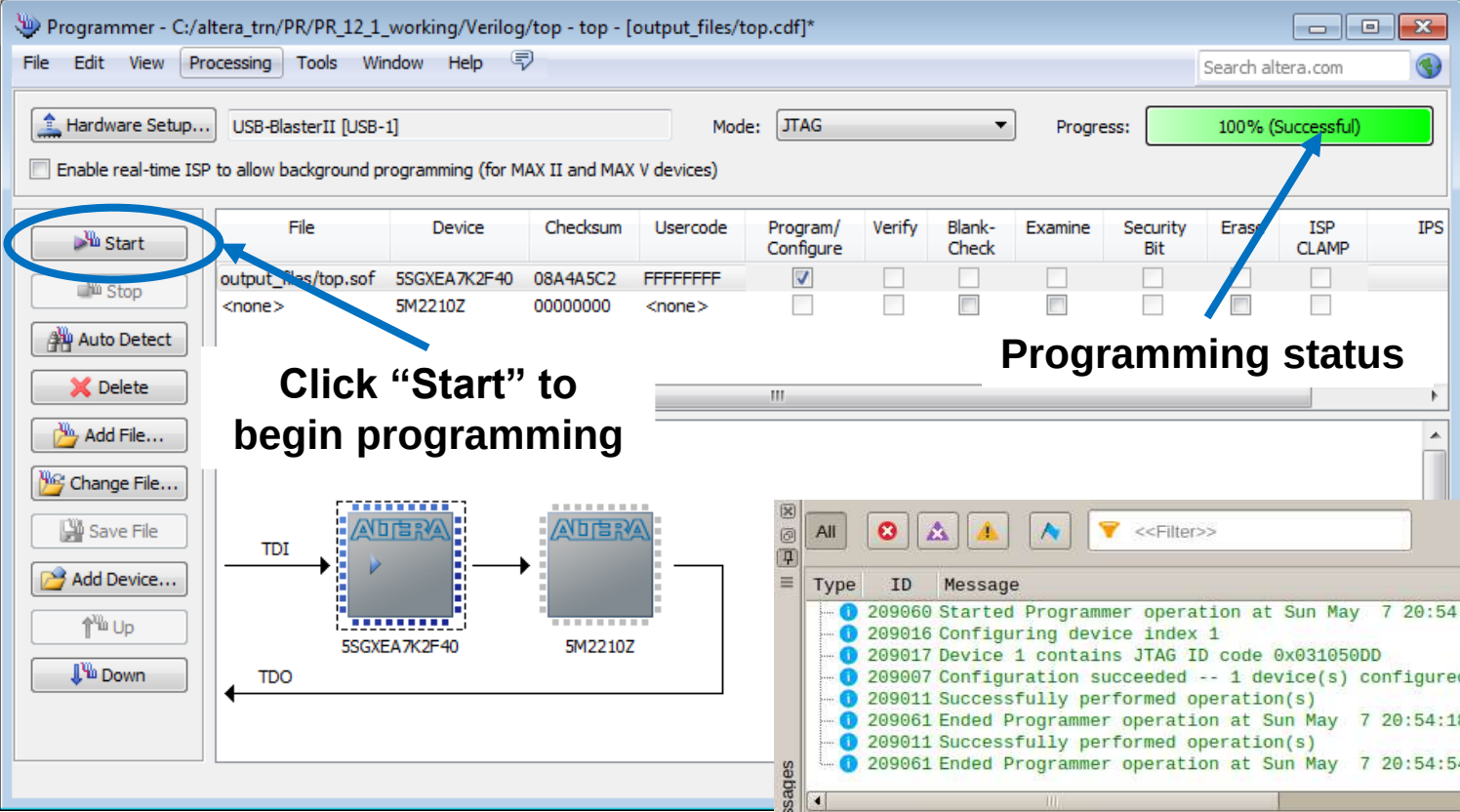
PROGRAMMING OPTIONS FOR CPLDs/CONFIG. DEVICES



Option	Description
Verify	Verifies contents against respective programming files
Blank-Check	Check whether the device is blank
Examine	Reads back contents. Save examined data as .pof file.
Security Bit	Protect CPLD from being examined.
Erase	Erase the contents of devices

PROGRAMMING

Status messages appear in the main Quartus Prime window, inside the messages pane



Programmer - C:/altera_trn/PR/PR_12_1_working/Verilog/top - top - [output_files/top.cdf]*

File Edit View Processing Tools Window Help Search altera.com

Hardware Setup... USB-BlasterII [USB-1] Mode: JTAG Progress: 100% (Successful)

☐ Enable real-time ISP to allow background programming (for MAX II and MAX V devices)

Start Stop Auto Detect Delete Add File... Change File... Save File Add Device... Up Down

File	Device	Checksum	Usercode	Program/Configure	Verify	Blank-Check	Examine	Security Bit	Erase	ISP CLAMP	IPS
output_files/top.sof	5SGXEA7K2F40	08A4A5C2	FFFFFFFF	☒	☐	☐	☐	☐	☐	☐	☐
<none>	5M2210Z	00000000	<none>	☐	☐	☐	☐	☐	☐	☐	☐

Click "Start" to begin programming

Programming status

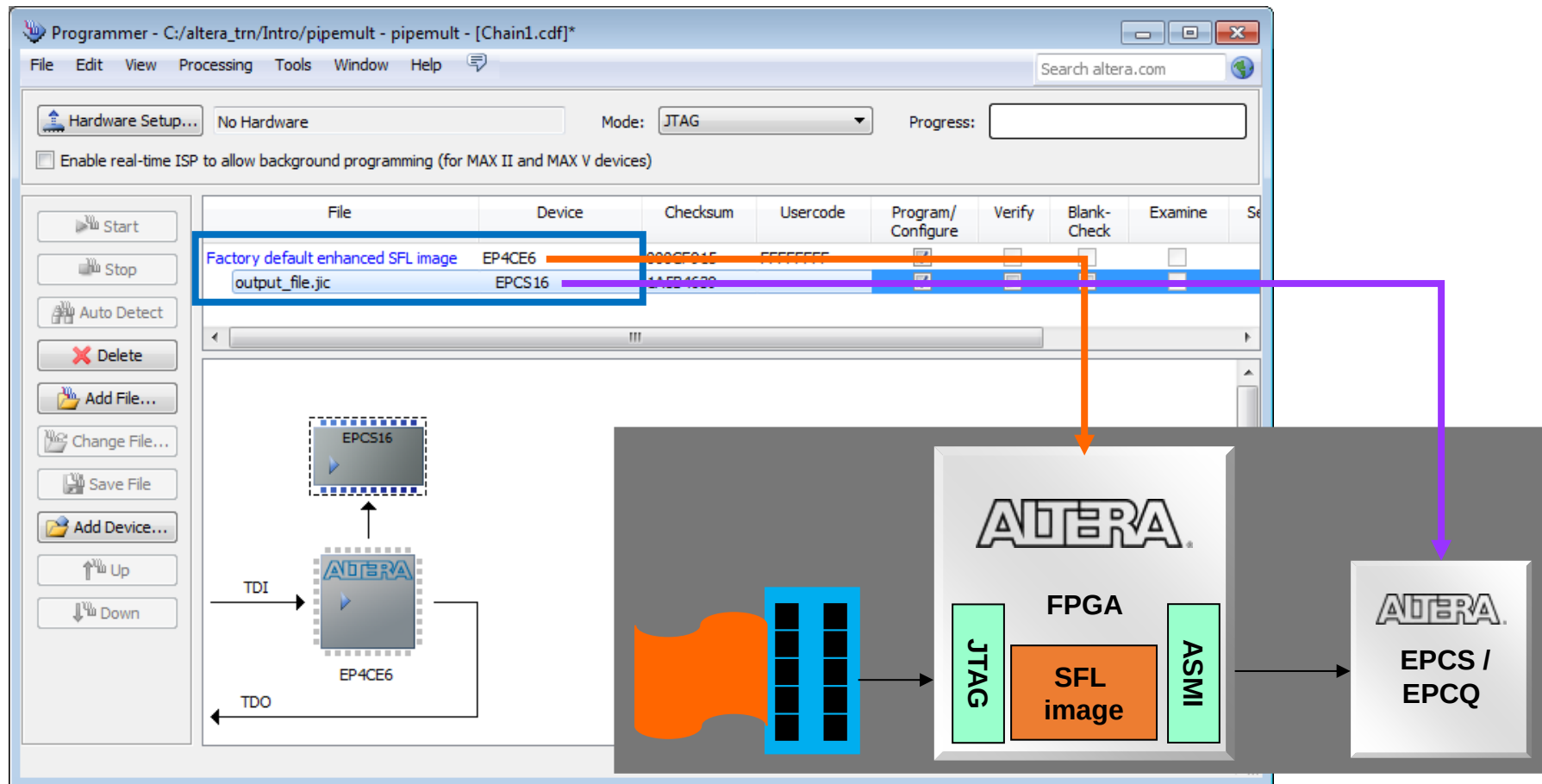
Messages

Type	ID	Message
Info	209060	Started Programmer operation at Sun May 7 20:54:16 2017
Info	209016	Configuring device index 1
Info	209017	Device 1 contains JTAG ID code 0x031050DD
Info	209007	Configuration succeeded -- 1 device(s) configured
Info	209011	Successfully performed operation(s)
Info	209061	Ended Programmer operation at Sun May 7 20:54:18 2017
Info	209011	Successfully performed operation(s)
Info	209061	Ended Programmer operation at Sun May 7 20:54:54 2017

System (26) Processing (220)

PROGRAMMING CONFIG. DEVICE WITH .jic FILE

- EPCS configuration devices have no JTAG interface
- Pass .jic through serial flash loader (SFL) image on FPGA

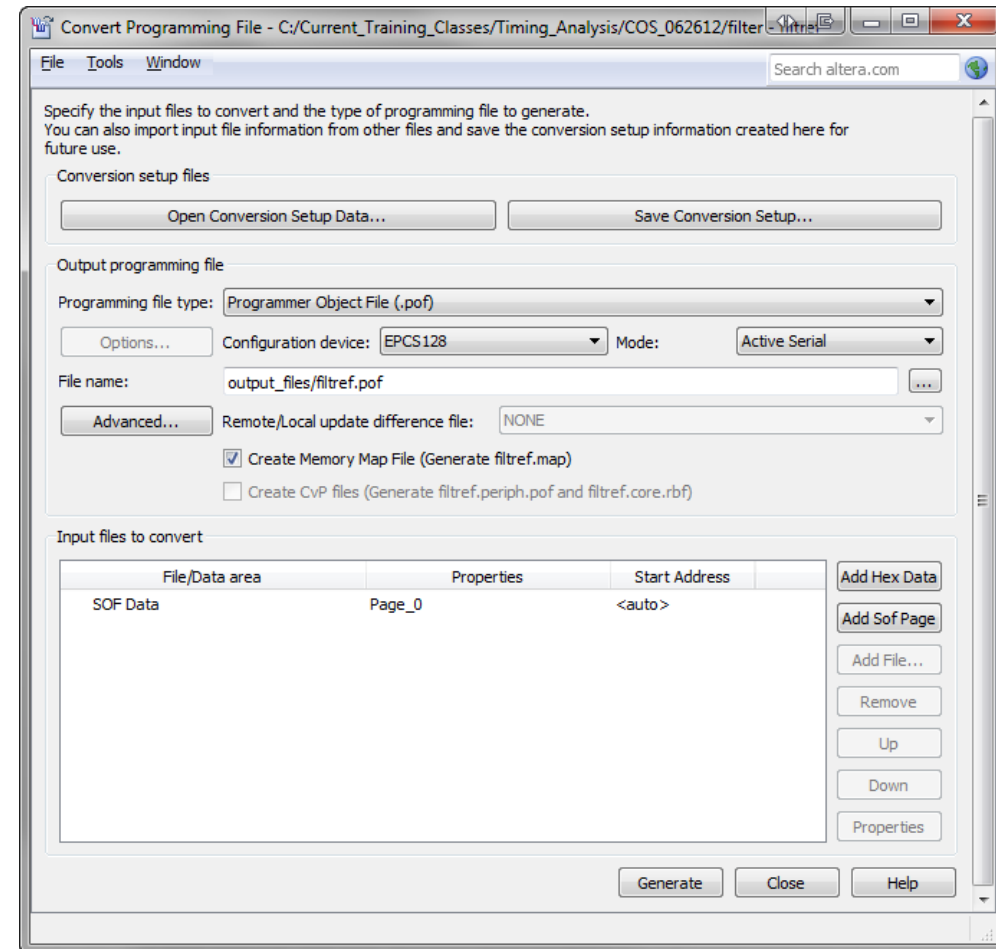
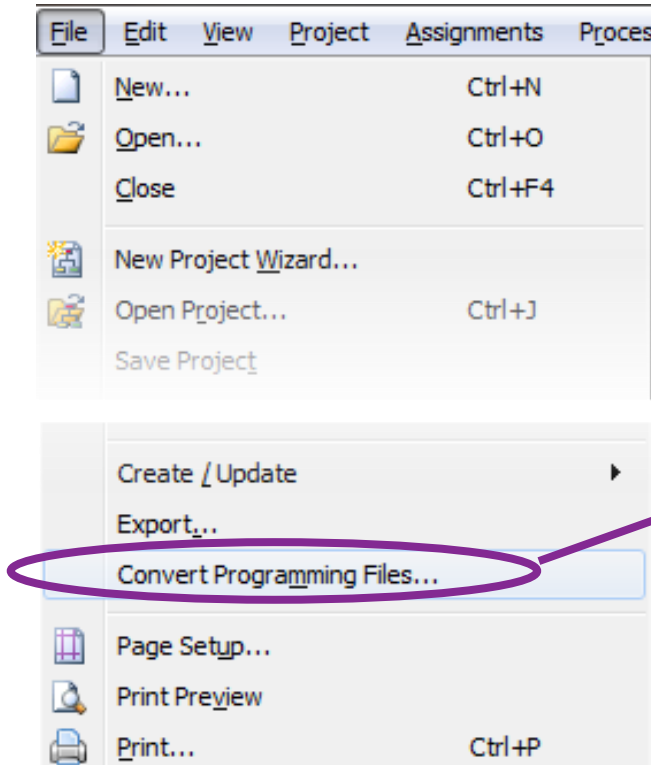


PROGRAMMING FILE CONVERSION

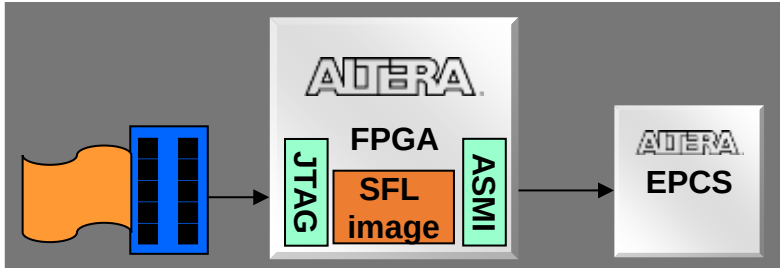
- Quartus Assembler automatically generates **.sof** files for any FPGA
 - May generate additional single-device programming file types from Device settings dialog box (**Device and Pin Options** → **Programming Files**)
- **.sof** files can only be used by the Quartus Programmer to directly configure FPGA
- Other configuration solutions require converting **.sof** file(s) into other file formats
 - **.jic** files
 - Multi-FPGA programming files

CONVERTING PROGRAMMING FILES

Convert **.sof** and **.pof** files to other supported programming files

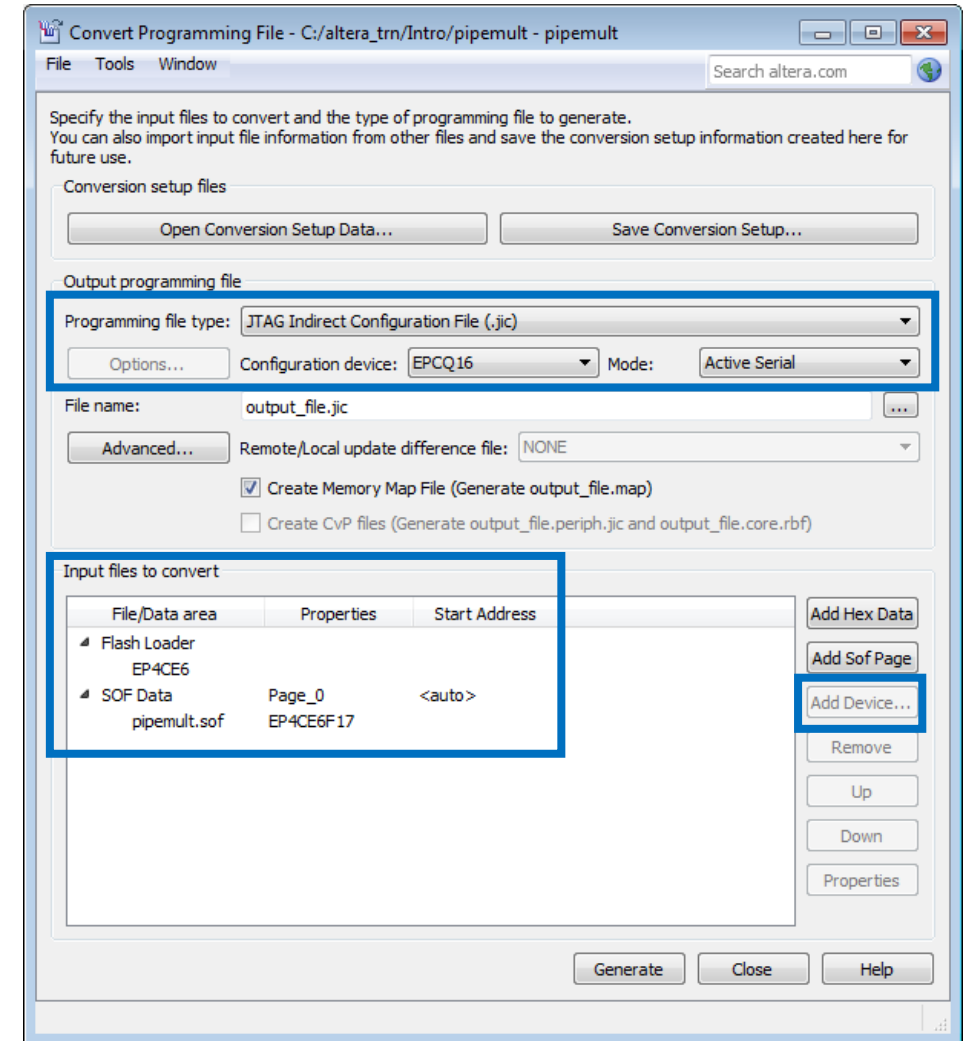


EPCS PROGRAMMING EXAMPLE: .jic GENERATION



To create the .jic file:

1. Select **.jic** as output
Programming file type
2. Select **Flash Loader** in **Input files to convert list** and click **Add Device**
3. Select **SOF Data** and click **Add File**
4. Click **Generate**



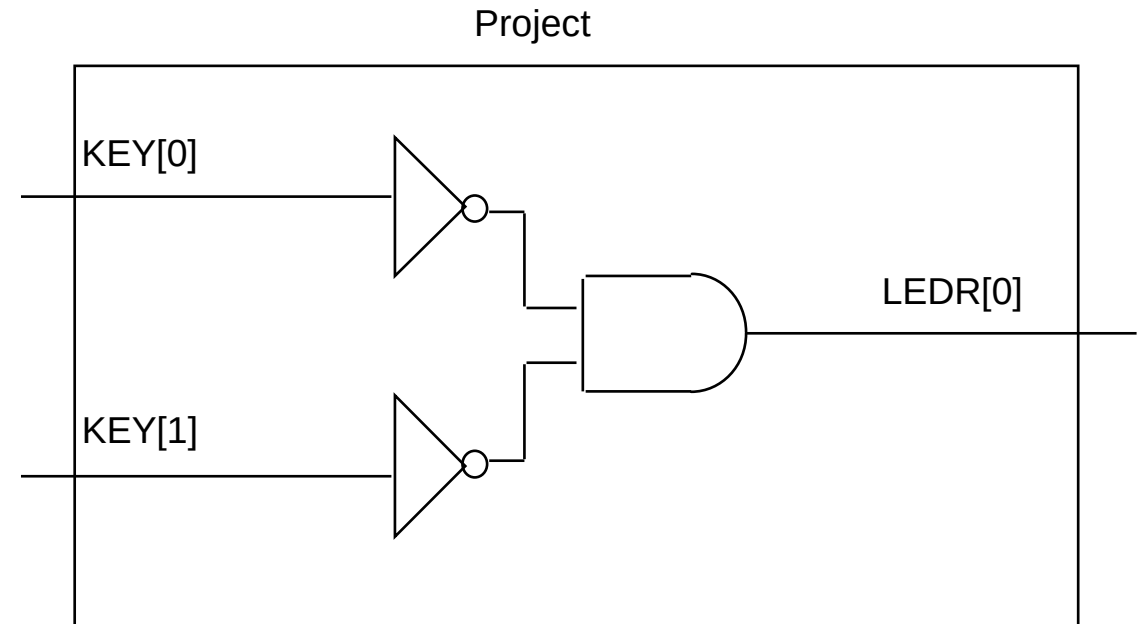
PROJECT 1C



PROJECT 1C

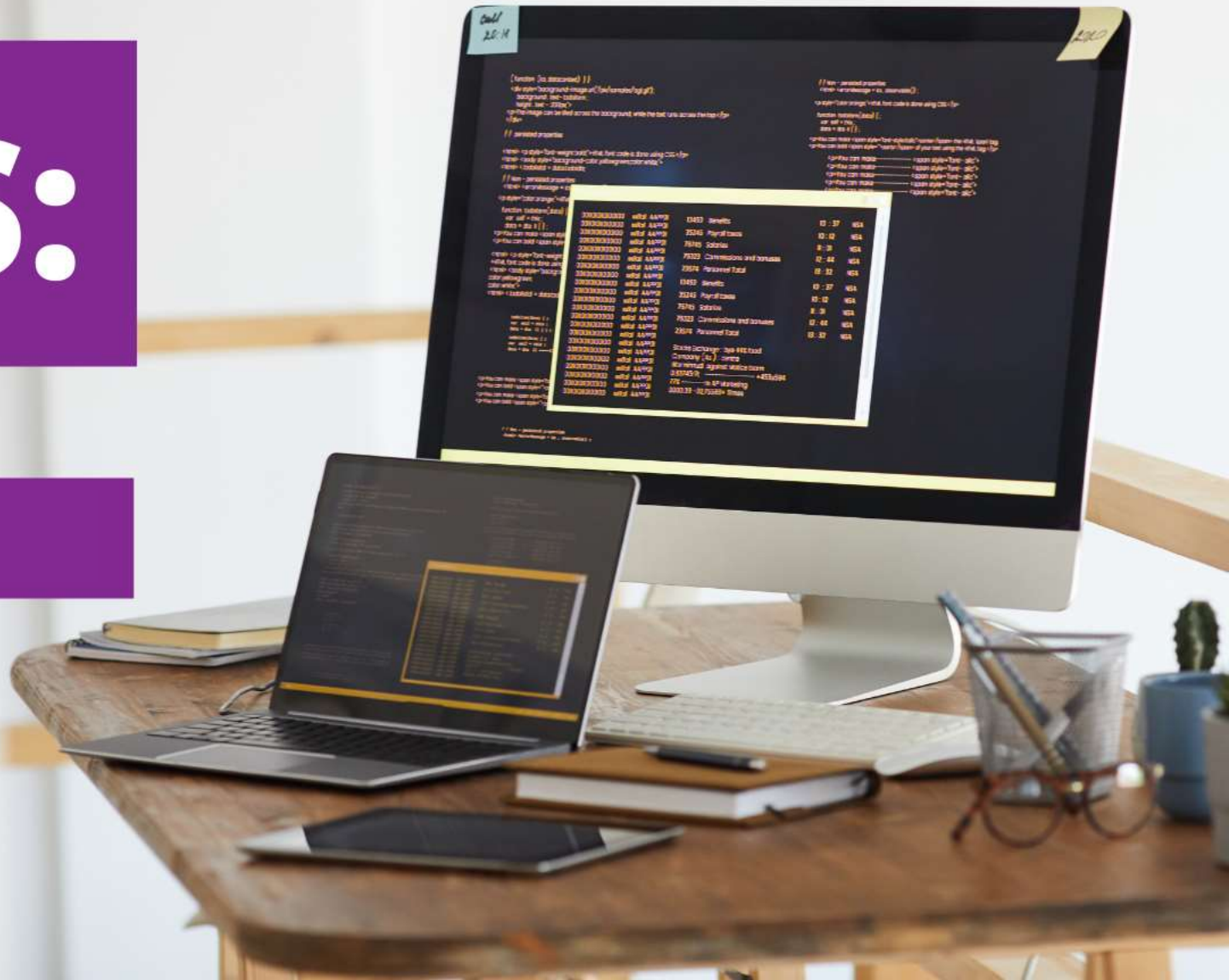
- Connect the DE10-Lite board through a USB cable
- Configure the FPGA using Quartus Programmer
- Press the KEYs and verify if it is working

(The LED must turn on only when both keys are pressed together)



QUARTUS:

CREATING A NEW PROJECT



QUARTUS PROJECTS

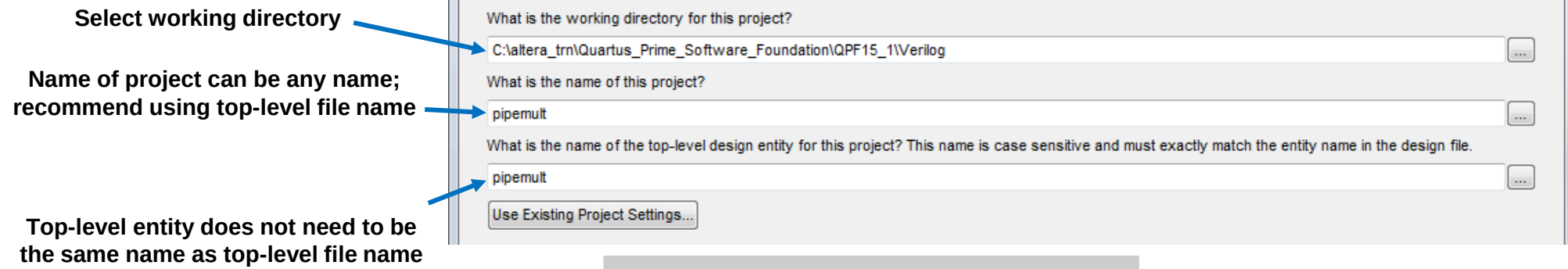
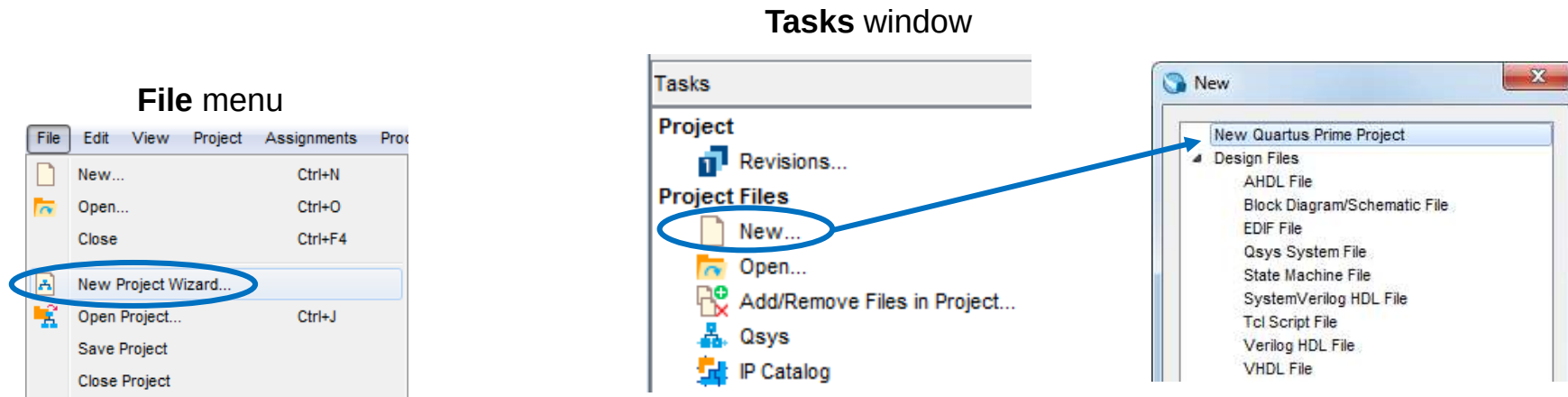
➤ Description

- Collection of related design files & libraries
- Must have a designated top-level entity
- Target a single device
- Store settings in Quartus Prime settings file (.qsf)
- Compiled netlist information stored in **db** folder in project directory

➤ Create new projects with **New Project Wizard**

- Can be created using Tcl scripts

NEW PROJECT WIZARD

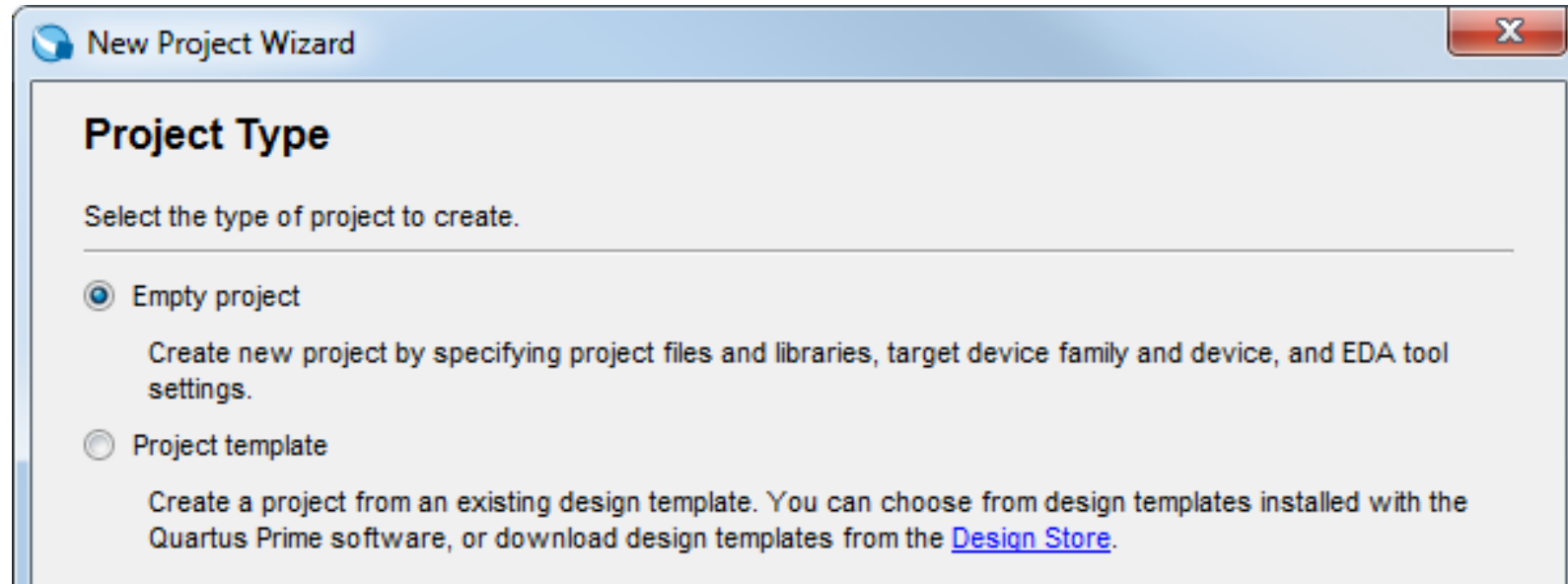


Tcl: `project_new <project_name>`

PROJECT TYPE

Create a blank project or use templates from Intel/Altera Design Store

<https://fpgacloud.intel.com/devstore/>



INTEL DESIGN STORE

- Download complete example design templates for specific development kits
- Design examples include design files, device programming files, and software code as required
- Install .par files and select as template in New Project Wizard

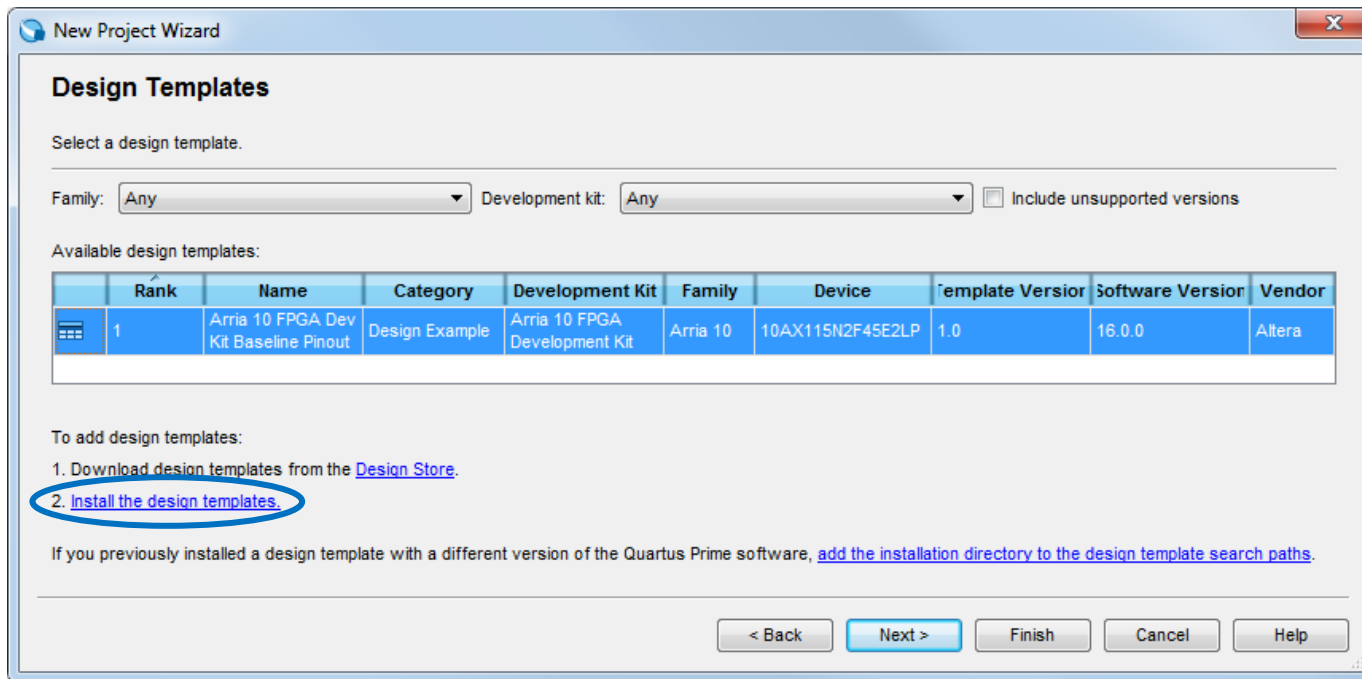
<https://fpgacloud.intel.com/devstore/platform/>

The screenshot shows the Intel Design Store interface. At the top, there are navigation links for SOLUTIONS, DEVELOPERS, and PARTNERS. Below this is a header with a search bar and a 'Login' button. The main content area is titled 'Design Store' and includes tabs for 'Design Examples' and 'Development Kits'. A yellow banner prompts users to find more design examples. Below this is a filter section with dropdown menus for Family (MAX 10), Category (Any), Quartus Prime Version (Any), Development Kit (Any), and IP Core (Any). A table of design examples is displayed below the filters, with columns for Name, Category, Development Kit, Family, Quartus Prime Version, and Vendor.

Name	Category	Development Kit	Family	Quartus Prime Version	Vendor
Accelerated Video Tiny Embedded System	Design Example	Max 10 Specific MAX 10 Design Example	MAX 10	17.0.0 Standard	Synaptic Laboratories Ltd.
Adapting Digilent Prodigy LCD Display to DE10-Lite Development Kit: Arduino Shield Header	Design Example	MAX 10 DE10 - Lite	MAX 10	18.0.0	Intel
Adapting Digilent Prodigy LCD Display to DE10-Lite Development Kit: Arduino Shield Header	Design Example	MAX 10 DE10 - Lite	MAX 10	18.1.0	Intel
ADC and Audio Monitor	Design Example	Chipsky MAX 10 FPGA Kit	MAX 10	14.1.0	Macnica America

INSTALL AND SELECT DESIGN TEMPLATES

- devplatforms folder created at .par installation location
- Templates available for all future project creation



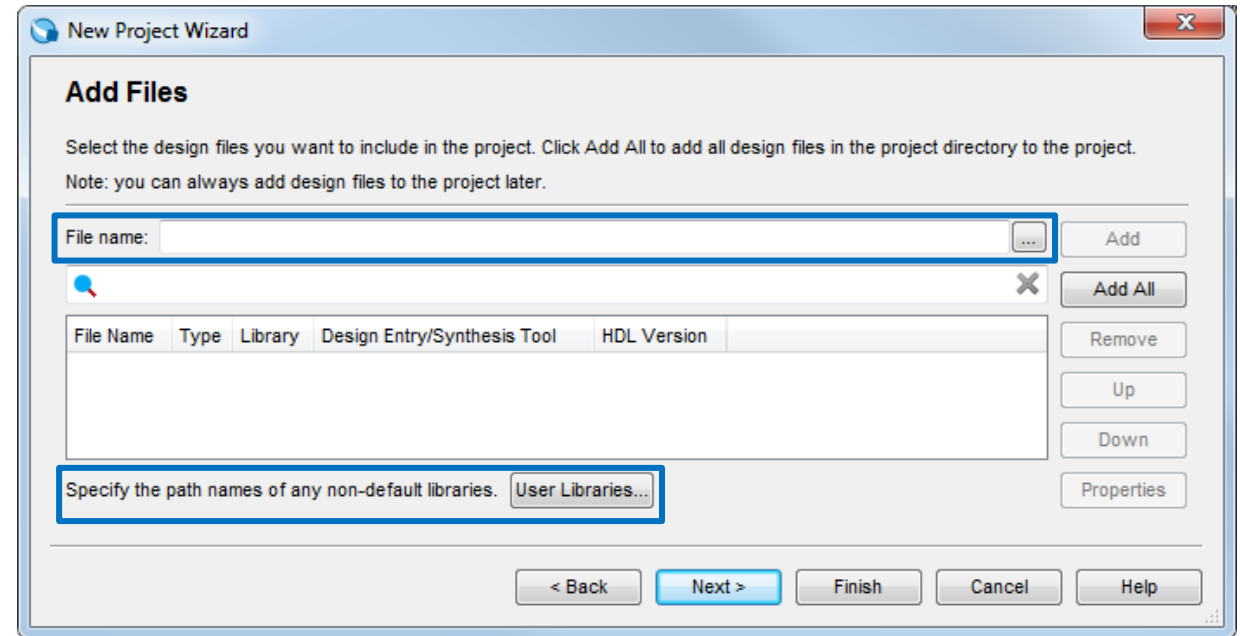
ADD FILES

➤ Add design files

- Graphic
- VHDL
- Verilog
- SystemVerilog
- EDIF
- VQM
- Quartus IP
- Platform Designer

➤ Add library paths

- User libraries
- MegaCore® library
- AMPP™ library
- Pre-compiled VHDL packages



Tcl: set_global_assignment -name VHDL_FILE <filename.vhd>

Tcl: set_global_assignment -name USER_LIBRARIES <library_path_name>

DEVICE SELECTION

Choose device family &
family category
(transceiver options,
SoC options, etc.)

New Project Wizard

Family, Device & Board Settings

Device **Board**

Select the family and device you want to target for compilation.
You can install additional device support with the Install Devices command on the Tools menu.

To determine the version of the Quartus Prime software in which your target device is supported, refer to the [Device Support List](#) webpage.

Device family

Family: Arria 10 (GX/SX)

Devices: Arria 10 GX

Target device

☐ Auto device selected by the Fitter

☒ Specific device selected in 'Available devices' list

☐ Other: n/a

Show in 'Available devices' list

Package: FBGA

Pin count: 1932

Core Speed grade: 1

Transceiver speed grade: 2

Name filter:

☒ Show advanced devices

Available devices:

Name	Core Voltage	ALMs	Total I/Os	GPIOs	HSSI Channels	PCIe Hard IP Blo
10AX115S2F45I1SG	0.9V or 0.95V	427200	960	624	72	4
10AX115S2F45I1SGE2	0.95V	427200	960	624	72	4
10AX115U2F45E1SG	0.9V or 0.95V	427200	928	480	96	4
10AX115U2F45I1SG	0.9V or 0.95V	427200	928	480	96	4

Filter device list

Choose specific part from list

Tcl: `set_global_assignment -name FAMILY "device family name"`
Tcl: `set_global_assignment -name DEVICE <part_number>`

EDA TOOL SETTINGS

- Choose EDA tools and file formats
- Settings can be changed or added later

New Project Wizard

EDA Tool Settings

Specify the other EDA tools used with the Quartus Prime software to develop your project.

EDA tools:

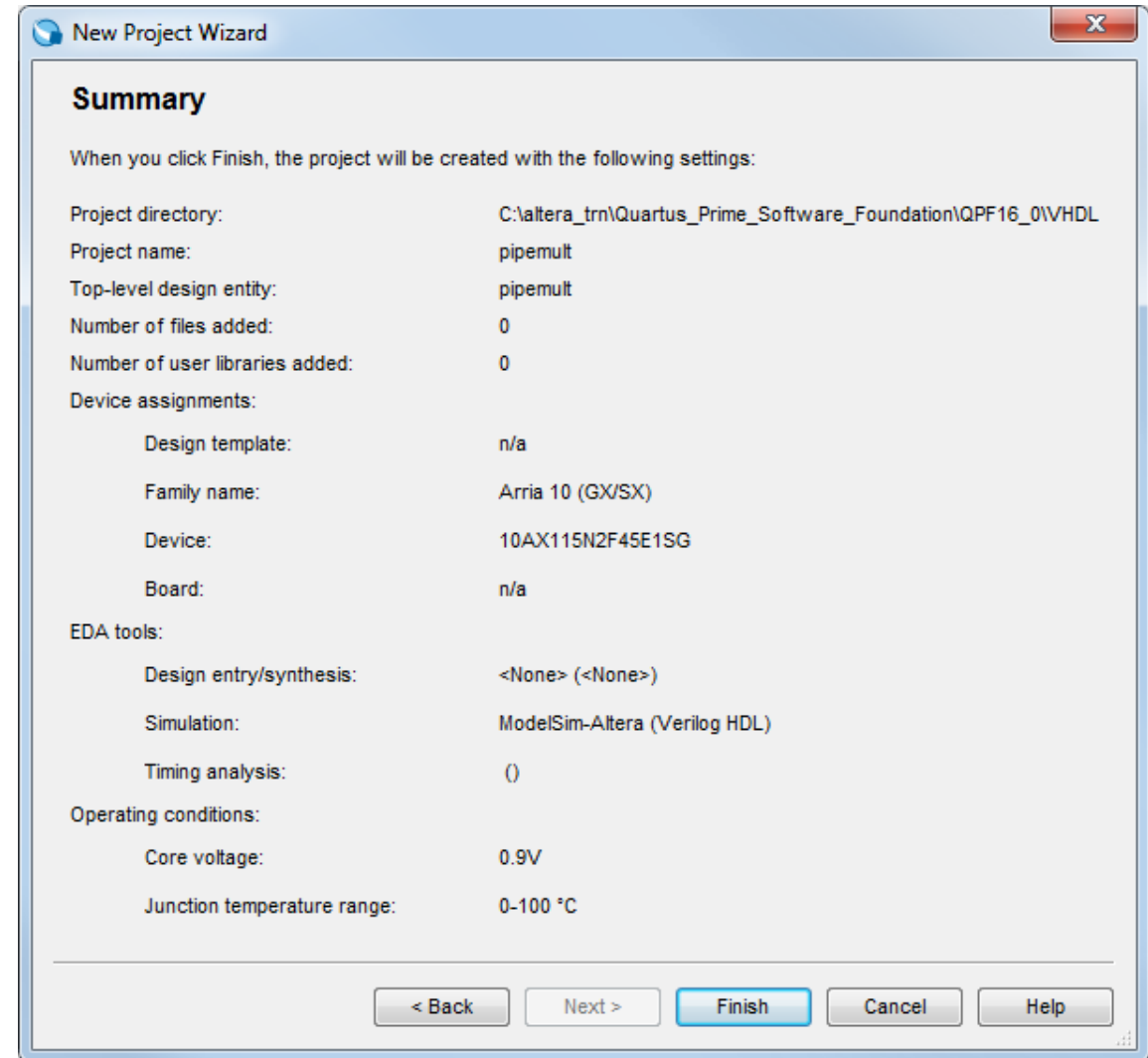
Tool Type	Tool Name	Format(s)
Design Entry/Synthesis	<None>	<None>
Simulation	ModelSim-Altera	Verilog HDL
Board-Level	Signal Integrity	<None>

< Back Next > Finish Cancel Help

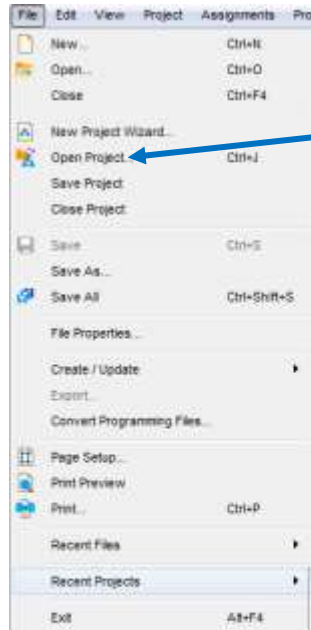
See handbook for Tcl command format

DONE!

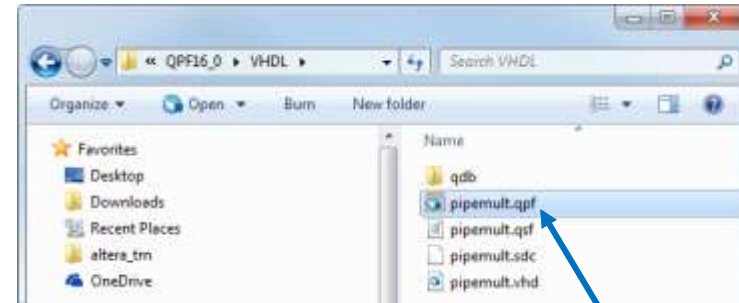
Review results & click **Finish** when done



OPENING AND EXISTING PROJECT

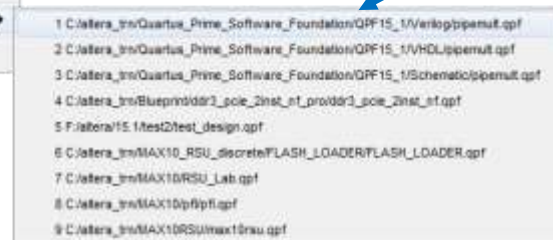


From File menu

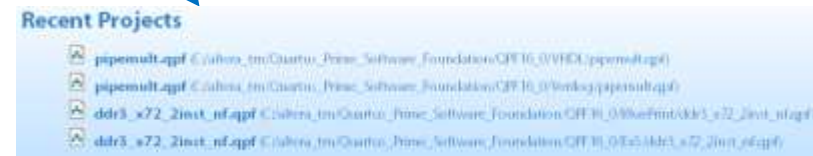


Double-click .qpf file

Select from recent projects



Home screen

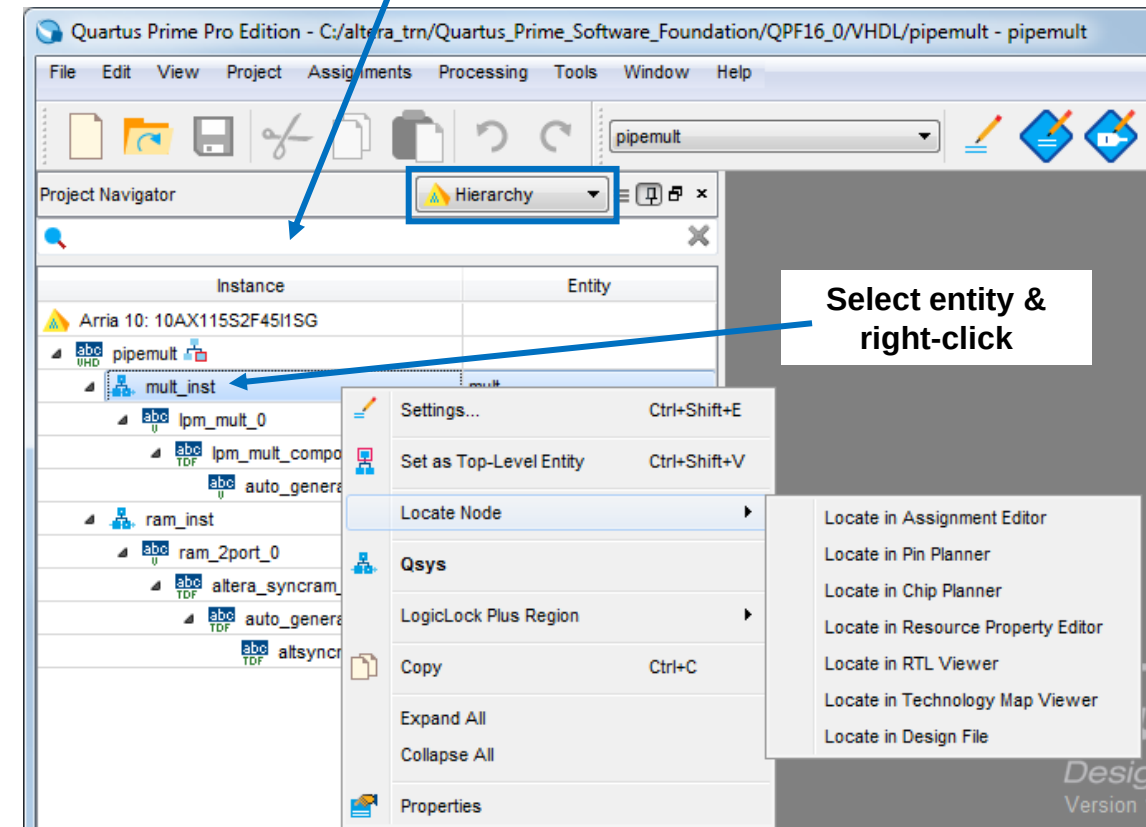


Tcl: project_open <project_name>

PROJECT NAVIGATOR: HIERARCHY

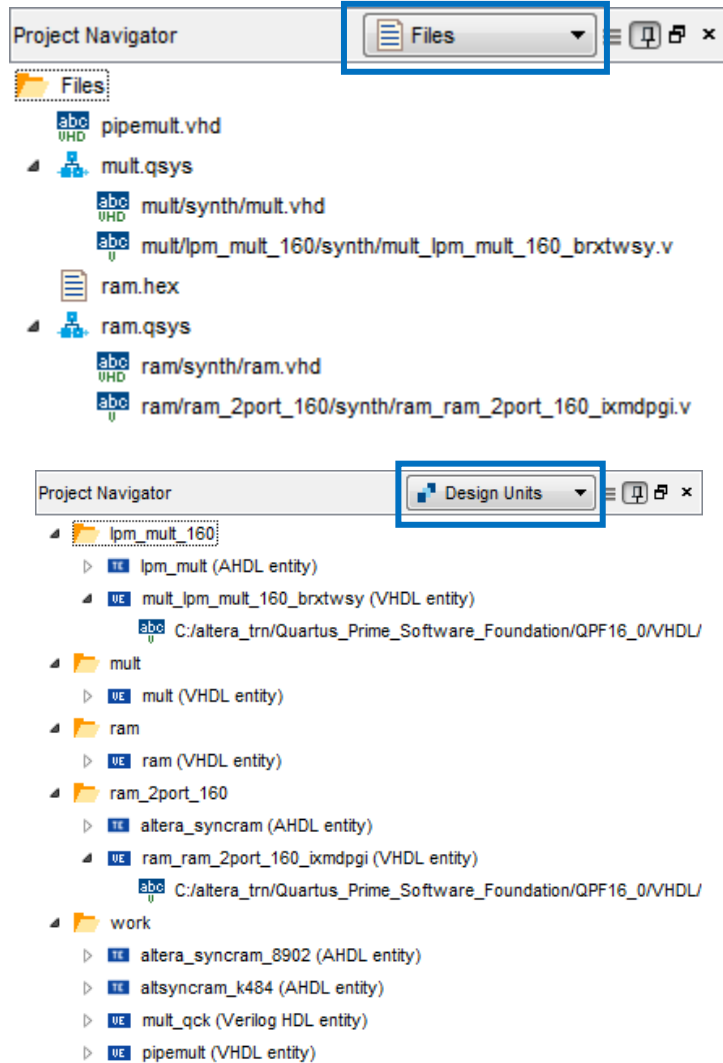
- Displays project hierarchy after project is analyzed
- Uses
 - Set top-level entity
 - Make entity-level assignments
 - View resource usage
 - Locate in design file or viewers/floorplans (cross-probing)

Filter project hierarchy
(click  to enable)



Full compilation or
Processing menu → **Start** → **Start Analysis & Elaboration**

PROJECT NAVIGATOR: FILES & DESIGN UNITS



■ Files

- Shows files explicitly added to project
 - Open files
 - Remove files from project
 - Set new top-level entity
 - Specify VHDL library
 - Select file-specific synthesis tool
- Can also use **Project** menu → **Add/Remove Files in Project...**

■ Design Units

- Displays design unit & type
 - VHDL entity and architecture
 - Verilog module
 - AHDL (Altera HDL) subdesign
 - Block diagram filename
- Expanded unit displays file which instantiates design unit

QUARTUS PROJECT FILES & FOLDERS

- **Quartus Project File (.qpf)**
- Quartus Defaults File (.qdf)
- **Quartus Settings File (.qsf)**
- Synopsys® Design Constraints (.sdc)
 - Contains timing constraints
 - Learn about SDC and using the TimeQuest timing analyzer
 - <https://www.intel.com/content/www/us/en/programmable/support/training/course/ods1115.html>
- **db folder**
 - Contains compiled design information
- **output_files folder** (customize location/name in project settings)
 - Generated compilation report files
 - Programming files generated by the Quartus Prime Assembler

PROJECT & DEFAULTS FILES

- Quartus Project File (.qpf)
 - Quartus version
 - Time stamp
 - Active revision(s)
 - *Each revision is a separate qsf file*
 - *Discussed later*

fir_filter.qpf

```
QUARTUS_VERSION = "16.0"  
DATE = "13:57:59  June 8, 2016"  
  
# Revisions  
  
PROJECT_REVISION = "filtref"  
PROJECT_REVISION = "filtref_new"
```

QUARTUS SETTINGS FILE (.qsf)

- Stores all settings & assignments *except* timing
- Uses Tcl syntax
- Can be edited manually by user

User comments start with #

Source other Tcl/.qsf files
to organize assignments

```

18 #
19 #
20 # Quartus Prime
21 # Version 15.1.0 Build 185 10/21/2015 S3 Standard Edition
22 # Date created = 13:03:15 November 10, 2015
23 #
24 # Notes:
25 #
26 # 1) The default values for assignments are stored in the file:
27 #    pipemult_ic_assignment_defaults.qdf
28 #    If this file doesn't exist, see file:
29 #    assignment_defaults.qdf
30 #
31 # 2) Altera recommends that you do not modify this file. This
32 #    file is updated automatically by the Quartus Prime software
33 #    and any changes you make may be lost or overwritten.
34 #
35 #
36 #
37 #
38 #
39 source "location_assignments.tcl"
40 #
41 set_global_assignment -name FAMILY "Cyclone V"
42 set_global_assignment -name DEVICE 5CGXFC5C6F27C7
43 set_global_assignment -name TOP_LEVEL_ENTITY pipemult
44 set_global_assignment -name ORIGINAL_QUARTUS_VERSION 15.1.0
45 set_global_assignment -name PROJECT_CREATION_TIME_DATE "12:42:26 NOVEMBER 11, 2015"
46 set_global_assignment -name LAST_QUARTUS_VERSION 15.1.0
47 set_global_assignment -name PROJECT_OUTPUT_DIRECTORY output_files
48 set_global_assignment -name MIN_CORE_JUNCTION_TEMP 0
49 set_global_assignment -name MAX_CORE_JUNCTION_TEMP 85
50 set_global_assignment -name DEVICE_FILTER_PACKAGE FBGA
51 set_global_assignment -name DEVICE_FILTER_PIN_COUNT 672
  
```

See the **Quartus Settings File Reference Manual**

(https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/manual/mnl_qsf_reference.pdf) for more details on QSF assignments & syntax

QUARTUS:

IMPORTING CONFIGURATIONS
FROM ANOTHER PROJECT

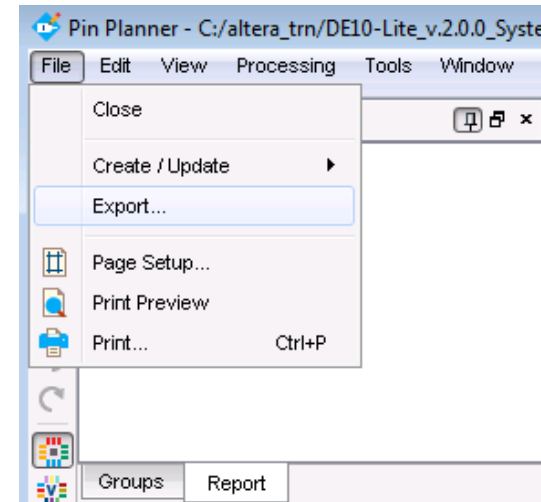


EXPORT VIA CSV

From Pin Planner Window

- Use spreadsheet Comma Separated Value (.csv) file to export your I/O locations
- Export Pin Assignments only

Pin Planner **File** menu

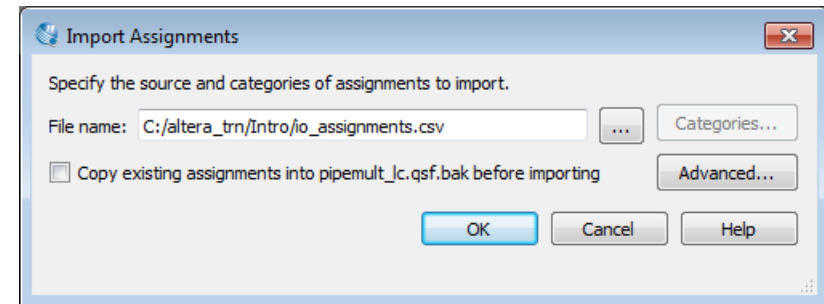


IMPORT VIA CSV

Quartus **Assignments** menu or Pin Planner **File** menu

- Use spreadsheet Comma Separated Value (.csv) file to enter or edit I/O locations
- Convenient for transferring assignments between projects or revisions
- CSV column names must match Pin Planner column headings

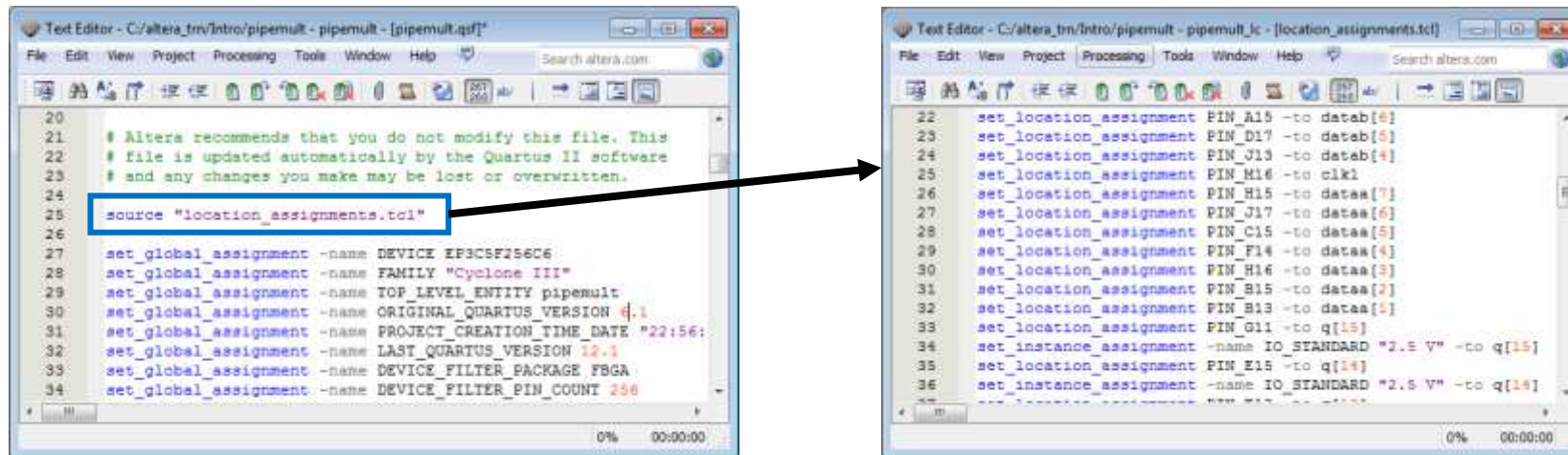
Pin Planner **File** menu or Quartus **Assignments** menu



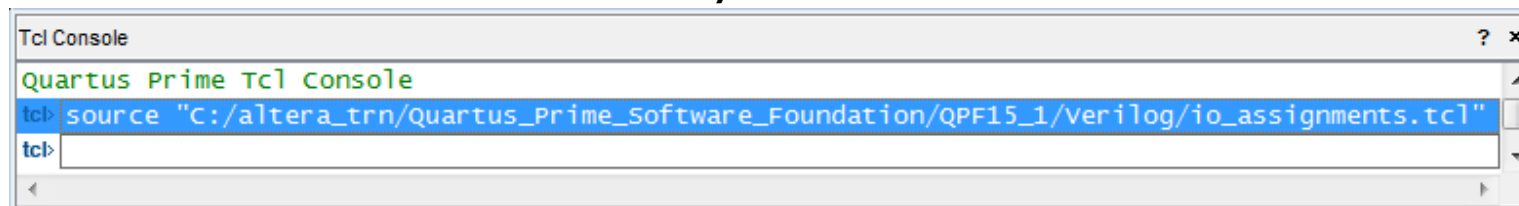
	A	B	C	D	E	F
1	To	Direction	Location	I/O Bank	VREF Group	I/O Standard
2	clk1	Input	PIN_E15	6	B6_N0	1.8 V
3	dataa[7]	Input	PIN_B14	7	B7_N0	2.5 V
4	dataa[6]	Input	PIN_A14	7	B7_N0	2.5 V
5	dataa[5]	Input	PIN_B13	7	B7_N0	2.5 V
6	dataa[4]	Input	PIN_A13	7	B7_N0	2.5 V

.qsf EDITING & SCRIPTING

- Type pin-related assignments directly into .qsf
- Type pin-related assignments into separate Tcl
 - Source Tcl file in project .qsf
 - Execute Tcl file to write assignments into .qsf



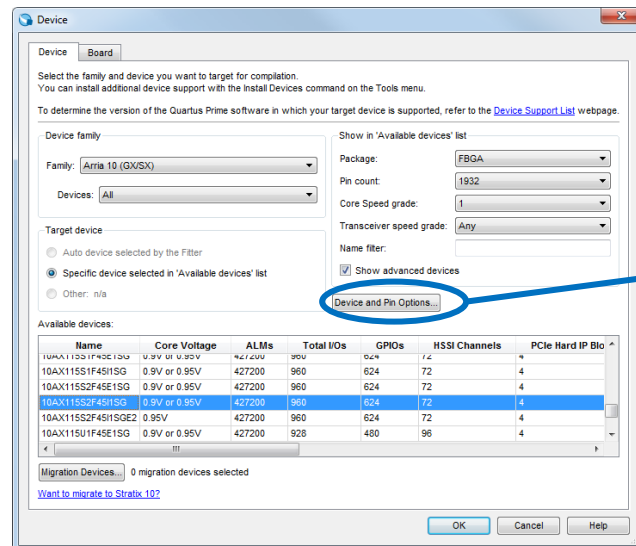
View menu → Utility Windows → Tcl Console



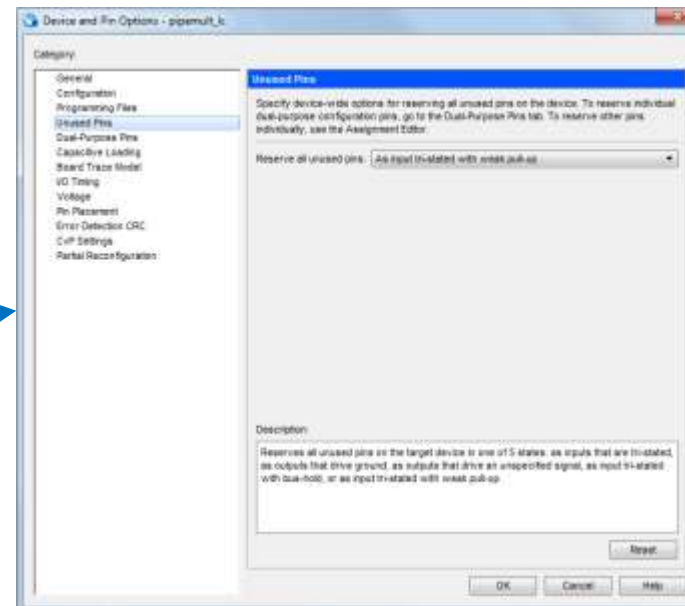
GLOBAL PIN SETTINGS

- Select global settings for pins
 - e.g. unused I/O configuration, dual-purpose configuration pins, device CRC checking
- Assignments in Pin Planner or other tools have precedence over global settings

Device settings (Assignments menu)

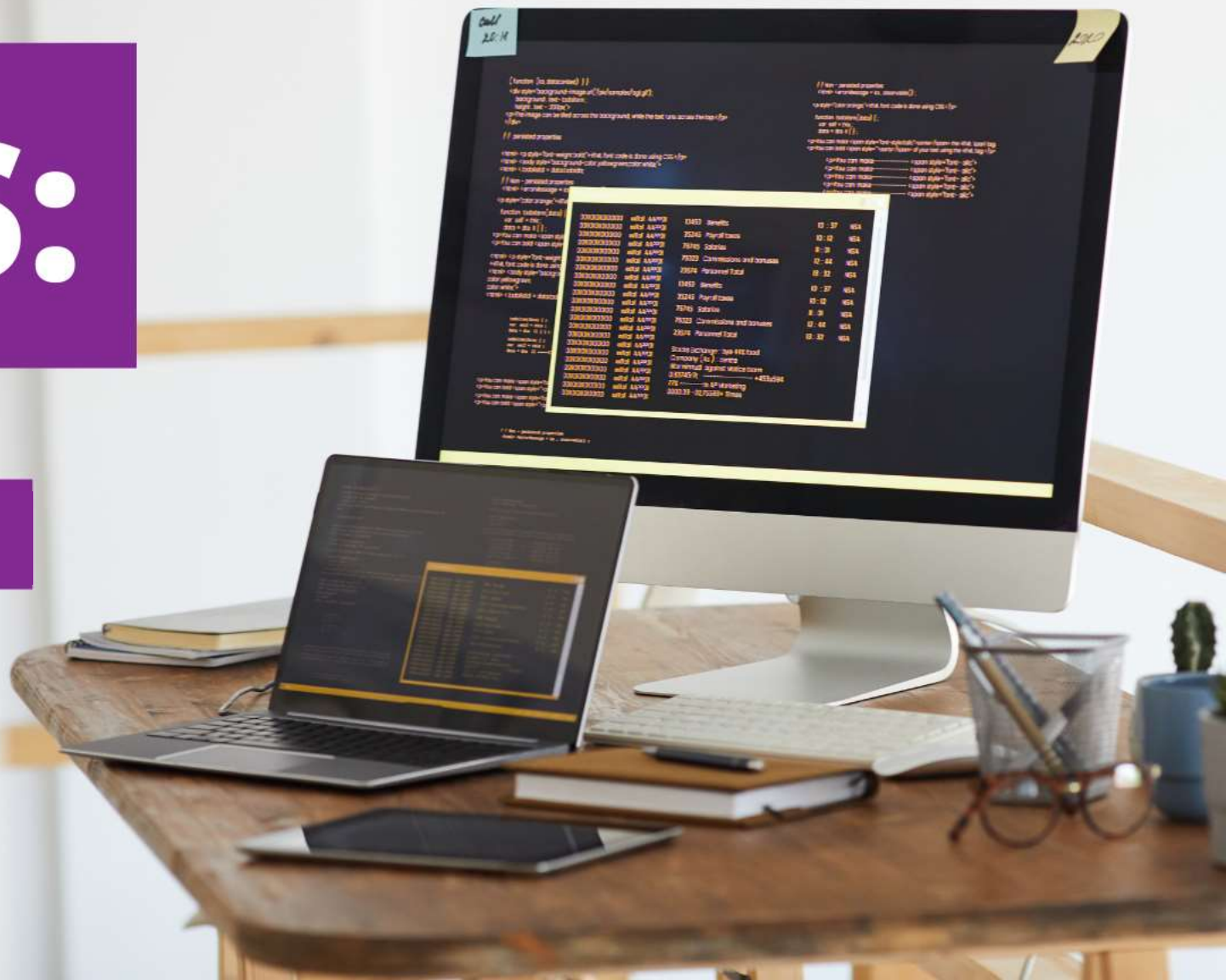


Device and Pin Options



QUARTUS:

PROJECT CONFIGURATION



SYNTHESIS & FITTING CONTROL

Controlled using two methods

Settings: project-wide switches

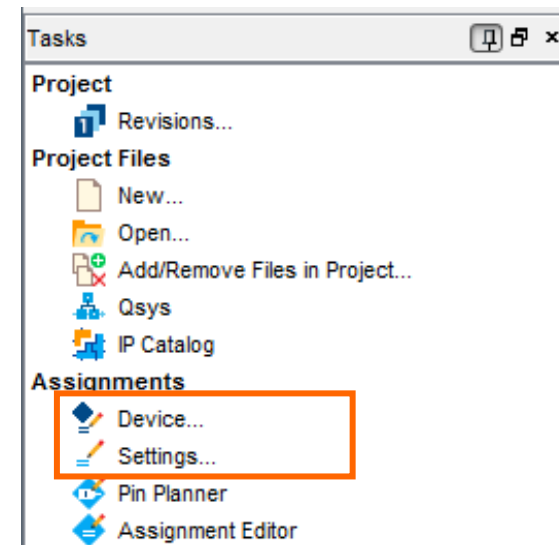
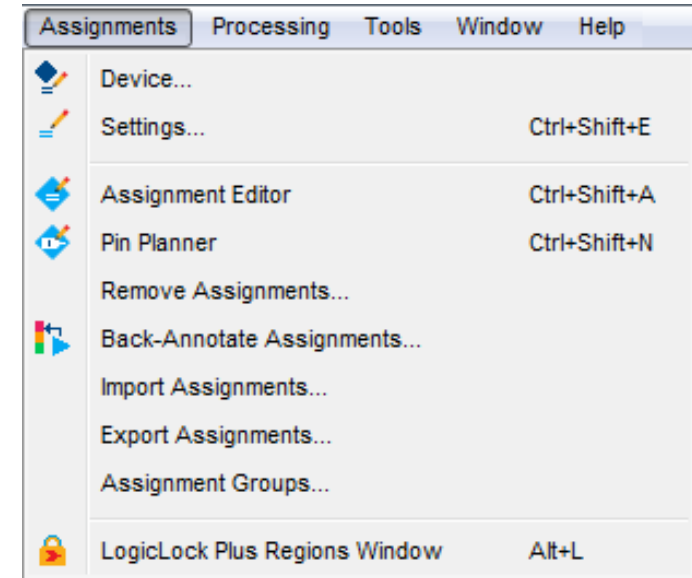
Assignments: individual entity/node controls

Both accessed in **Assignments** menu or **Tasks** window

Stored in **.qsf** file for project/revision

Timing constraints stored in separate **.sdc** file

Discussed later



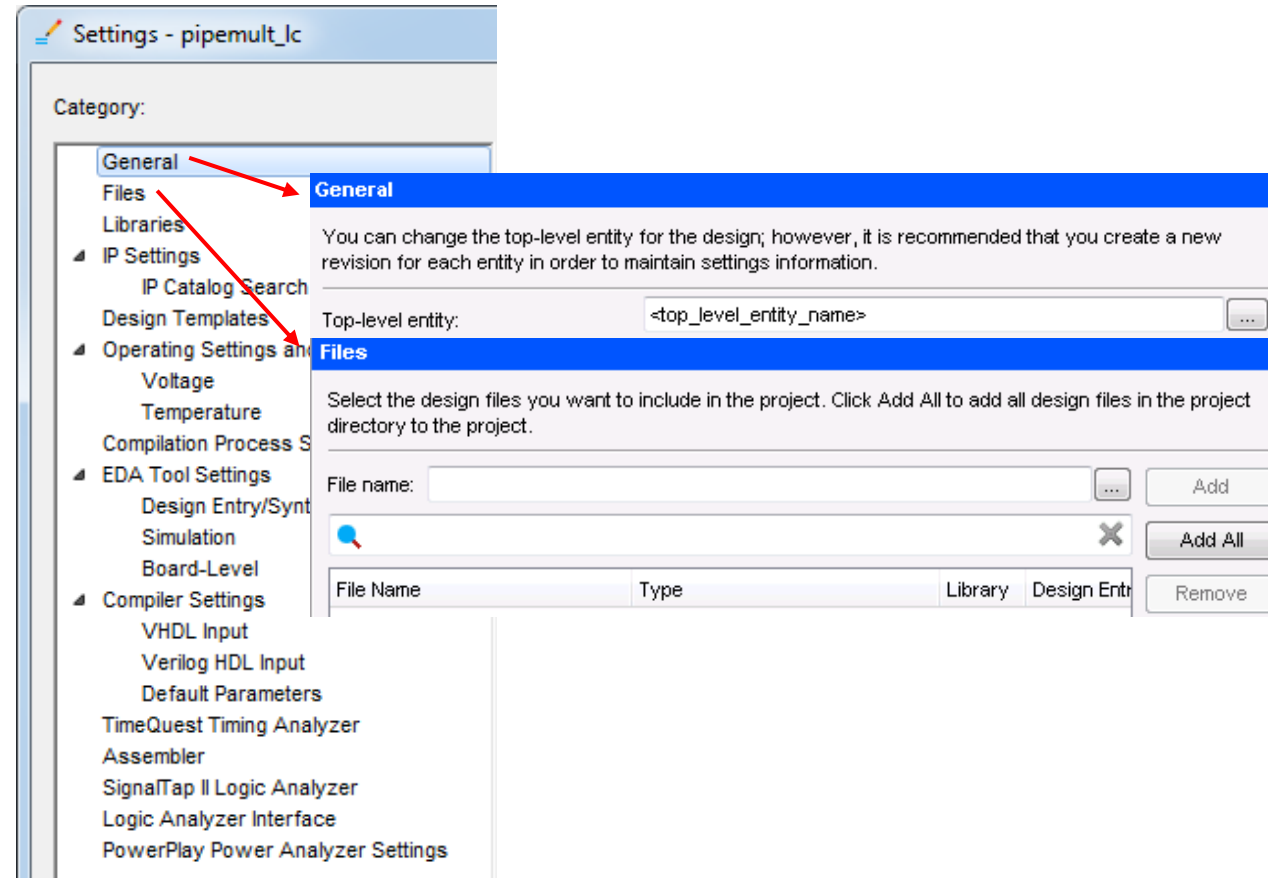
SETTINGS

- Project-wide switches that affect entire design
- Examples
 - Device selection
 - Synthesis optimization
 - Fitter settings
 - Physical synthesis
 - Design Assistant
- Located in **Device** and **Settings** dialog boxes
 - **Assignments** menu
 - **Tasks** window

SETTINGS DIALOG BOX

Change settings

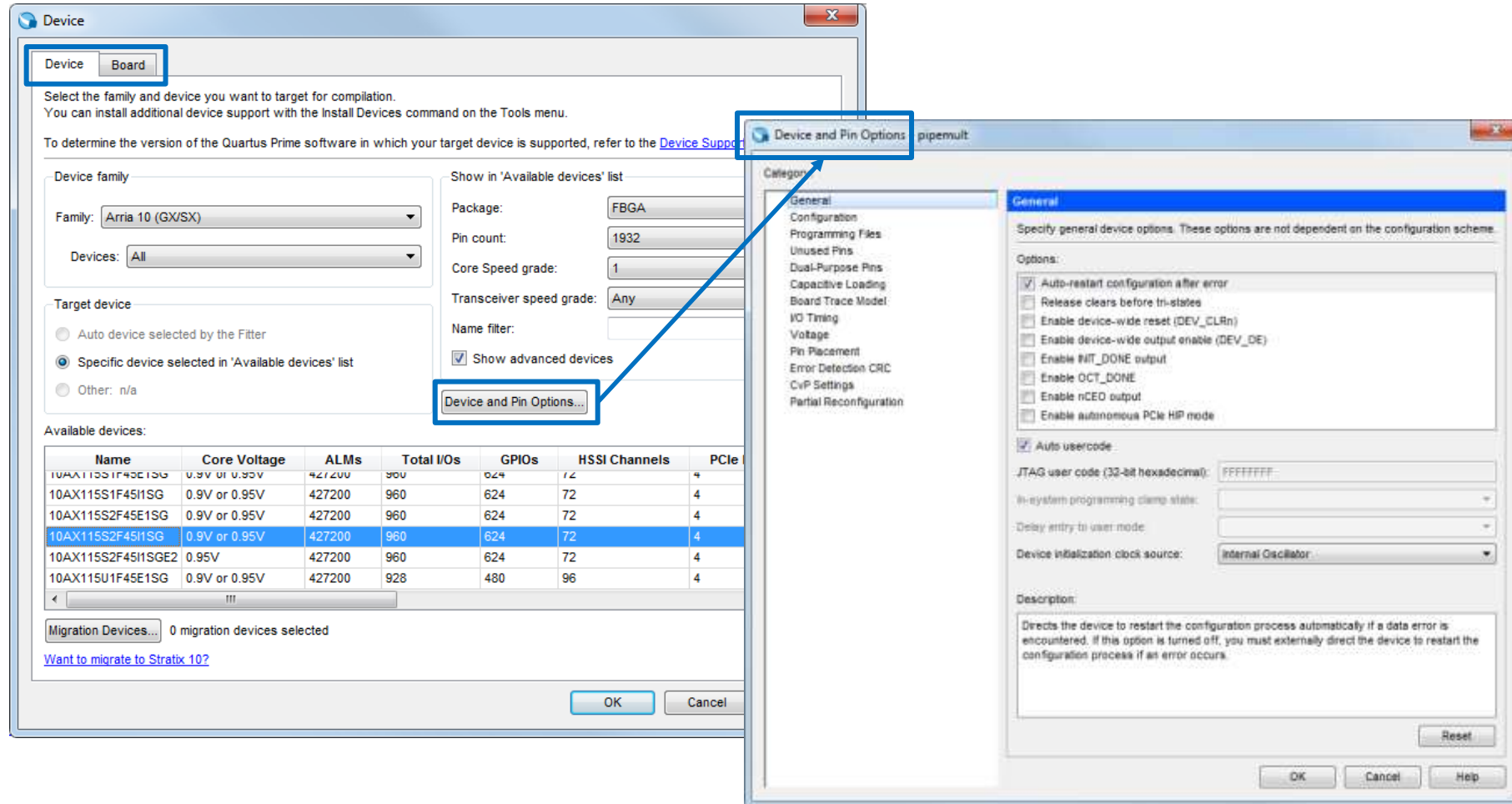
- Top-level entity
- Add/remove files
- Libraries
- Compiler settings
- EDA tool settings
- Synthesis settings
- Fitter settings
- Timing analyzer settings
- Power analysis settings



Tcl: `set_global_assignment -name <assignment_name> <value>`

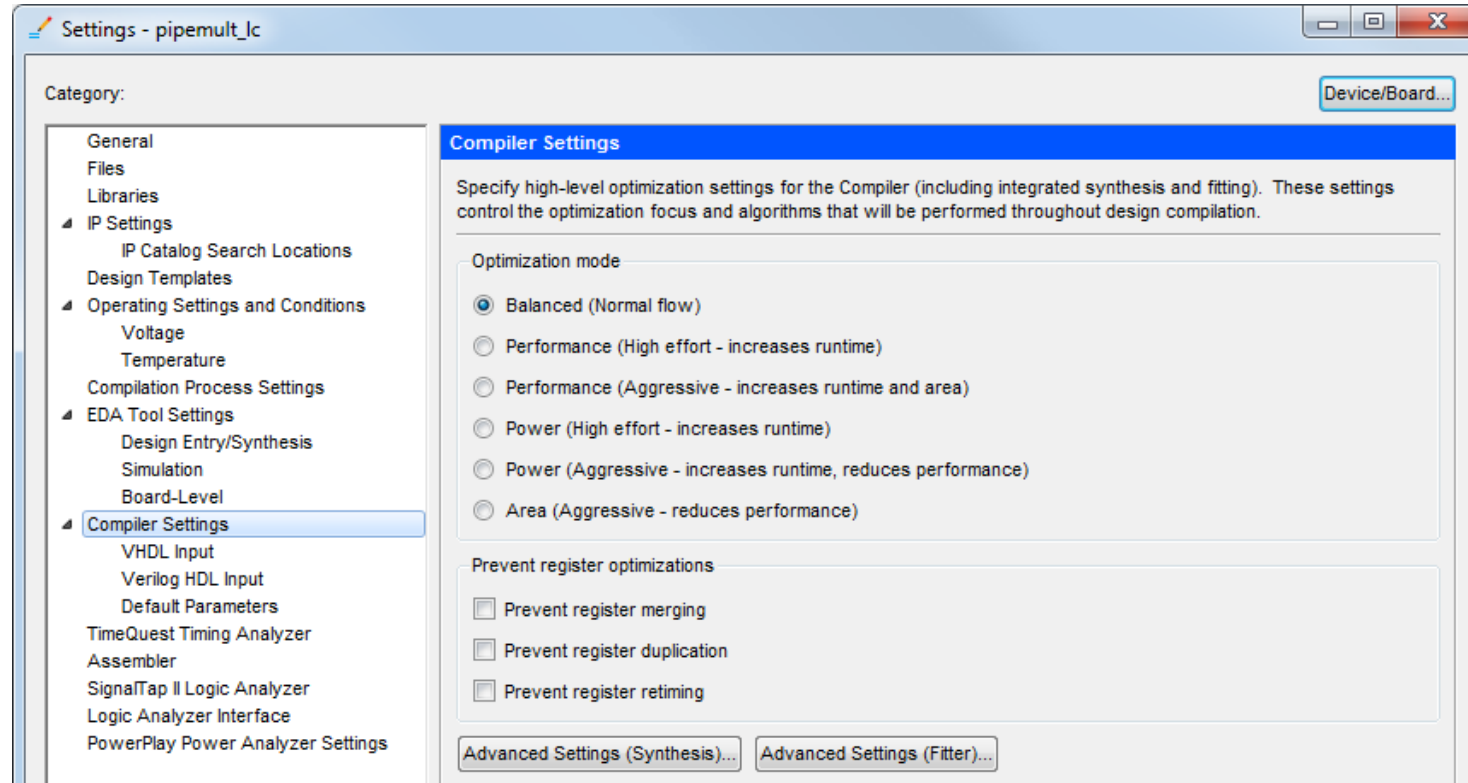
DEVICE SETTINGS

Assignments menu → Device or Tasks window



SETTINGS EXAMPLE: COMPILER OPTIMIZATION

- Choose general options for how compiler should optimize design
- Set specific optimization features under **Advanced Settings** buttons

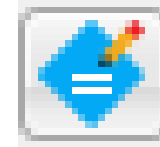


ASSIGNMENTS (AKA LOGIC OPTIONS CONSTRAINTS)

- Individual switches applied
 - I/O
 - Internal nodes
 - Hierarchical blocks (design entities)
- Assignment Editor manages assignments
- Must perform analysis & elaboration
- Example assignment: **Optimization Technique**

ASSIGNMENT EDITOR

- Provides spreadsheet assignment entry & display
 - Copy & paste support
 - Multi-cell editing



Filter nodes Sort on customizable columns Assignments menu, toolbar, or Tasks window

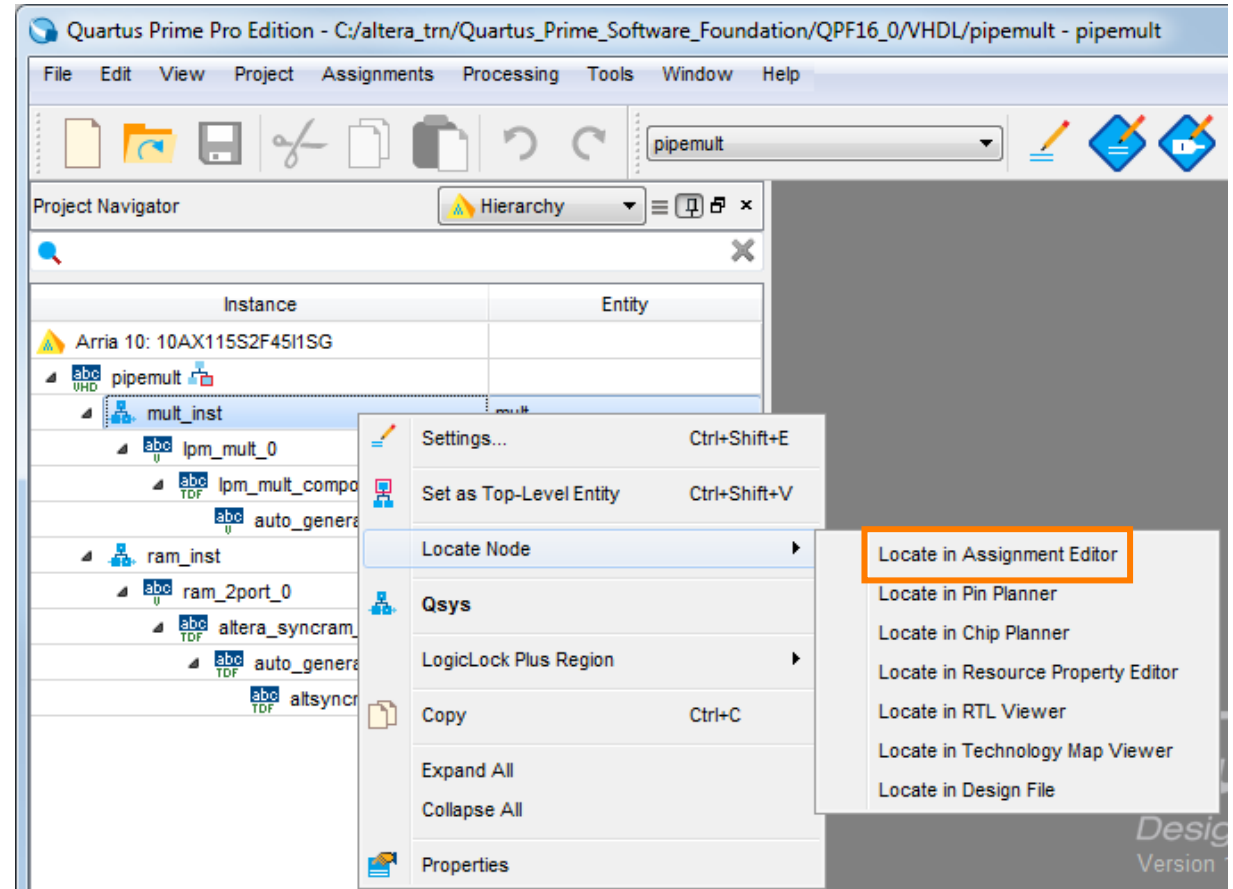
Status	From	To	Assignment Name	Value	Enabled	Entity
1 ✓		mult:mult_inst	DSP Block Balancing	Logic Elements	Yes	pipemult
2 ✓		mult:mult_inst	Optimization Technique	Speed	Yes	pipemult
3	<<new>>	<<new>>	<<new>>			

Information about selected assignment

0% 00:00:00

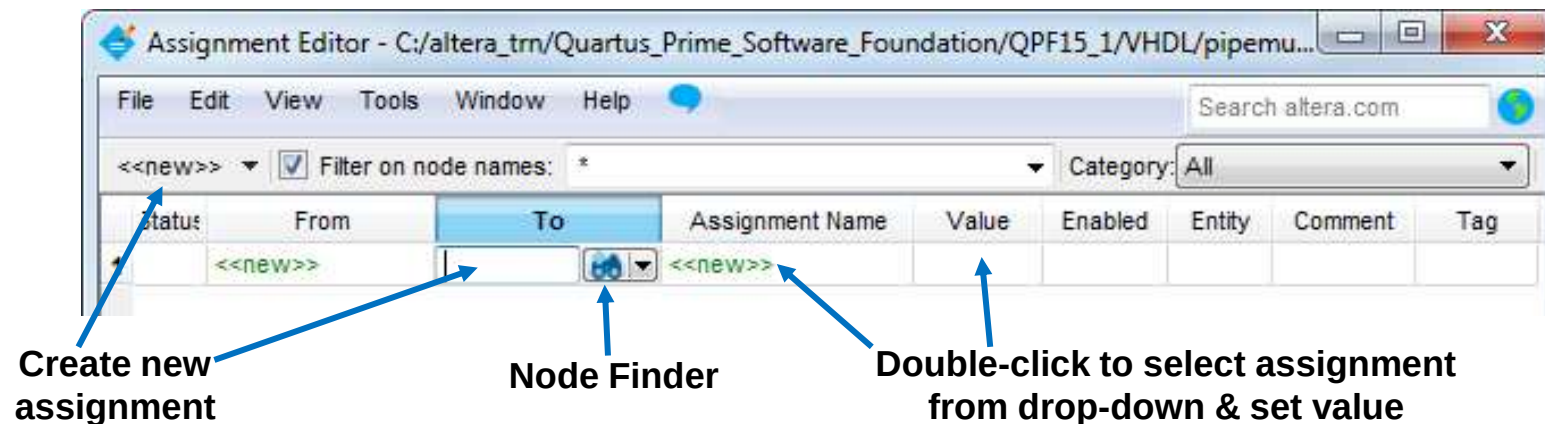
CREATING ASSIGNMENTS: CROSS-PROBING

- Cross-probe (Locate Node) to Assignment Editor from:
 - Project Navigator
 - Message window
 - Compilation Report
 - Design files
- Assignment Editor Node Filter automatically filled in with cross-probed node(s)
- Easiest method for creating assignments

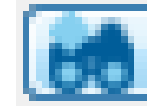


CREATING ASSIGNMENTS: ASSIGNMENT EDITOR

- Cross-probe from other tools to fill in filter
- Click on <<new>> button to create assignment for cross-probed node or entity
- Double-click <<new>> in the **From** or **To** columns to create a new assignment from scratch
 - Manually type in object name
 - Or click **Node Finder** icon to search



NODE FINDER



Search by name using wildcards (? or *)

Use filter to select the type of nodes to search/select

Locate nodes in a certain level of hierarchy

Show/hide filtering options

List of found nodes arranged by hierarchy

Tooltip with hierarchy path to target node

Select node(s) on left & use arrows to move to the right

Node Finder

Named: *

Options

Filter: SignalTap II: pre-synthesis

Look in: |pipemult|. Include subtrees Hierarchy view

Matching Nodes:

Name	Assignments
mult:mult_inst	
clock	Unassigned
lpm_mult:lpm_mult_component	
aclr	Unassigned
clken	Unassigned
clock	Unassigned
mult_p1q:auto_generated	
dataa	Unassigned
datah	Unassigned

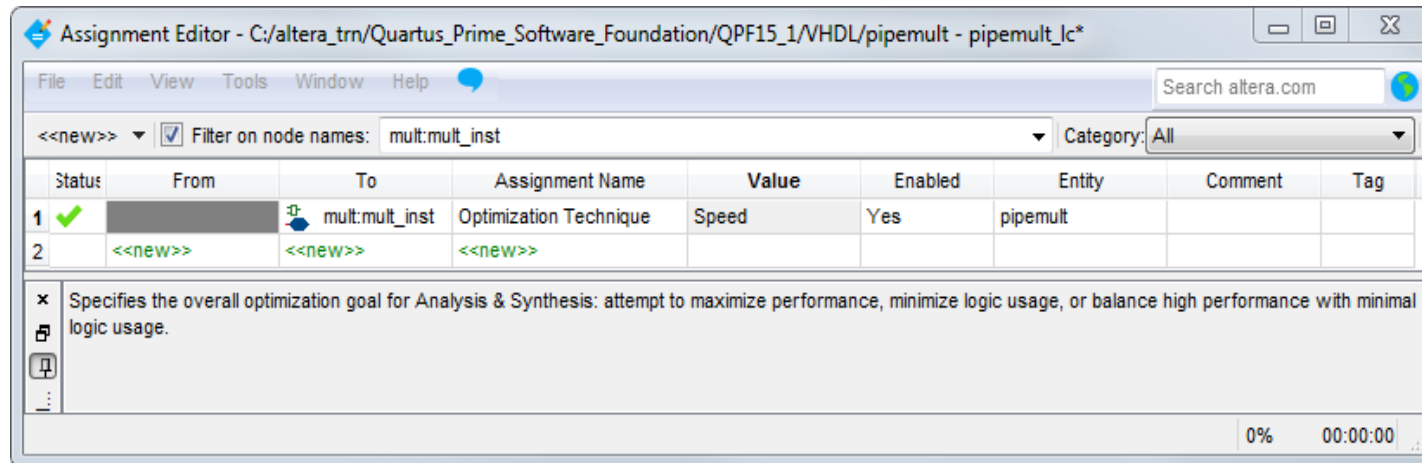
Nodes Found:

Name	Assignments
mult:mult_inst ..._component clken	Unassigned

Tooltip: mult:mult_inst|lpm_mult:lpm_mult_component|clken

ASSIGNMENT EXAMPLE: OPTIMIZATION TECHNIQUE

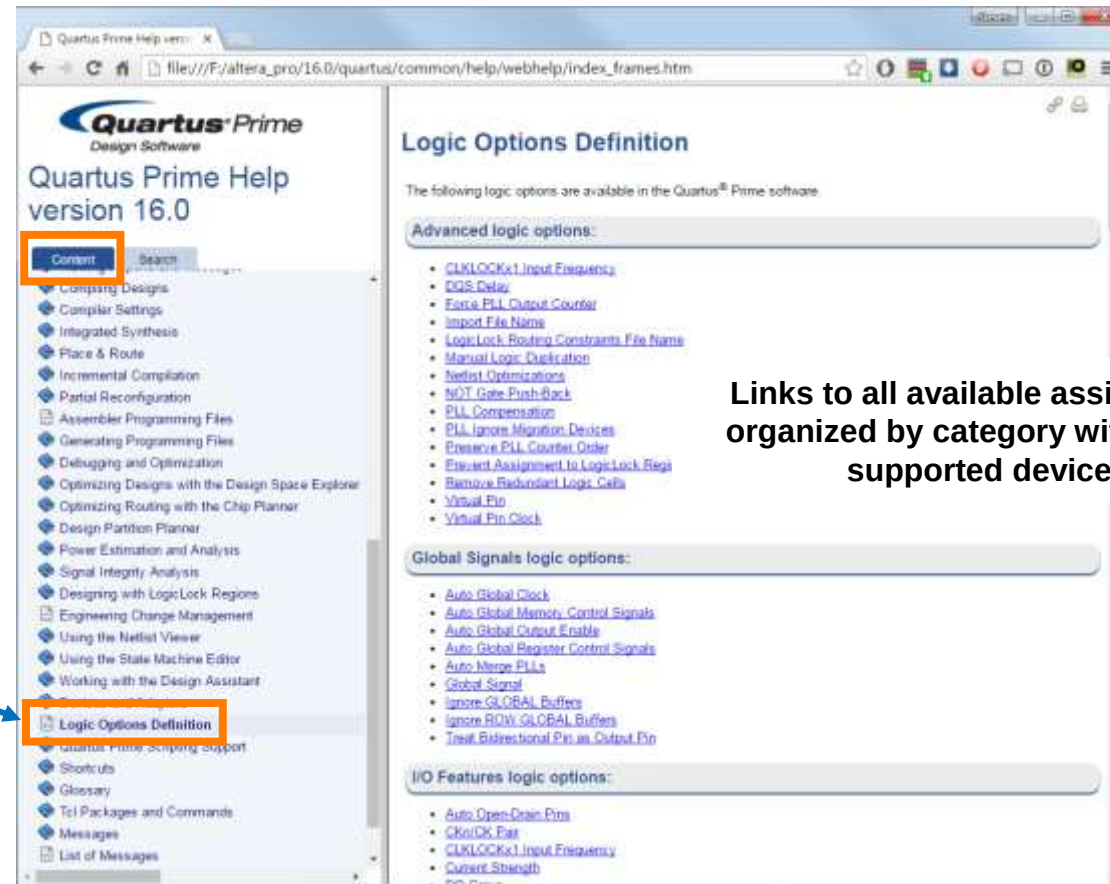
- Selects synthesis optimization goal (*Speed, Balanced, Area*)
- Apply to hierarchical entities
 - Locate (cross-probe) from Project Navigator
 - Or drag and drop into Assignment Editor
- May also apply project-wide in **Advanced Synthesis Settings**
- Affects synthesis & logic mapping; Quartus Prime synthesis only



```
Tcl: set_instance_assignment -name OPTIMIZATION_TECHNIQUE SPEED -to <node name>
```

AVAILABLE LOGIC OPTIONS (ASSIGNMENTS)

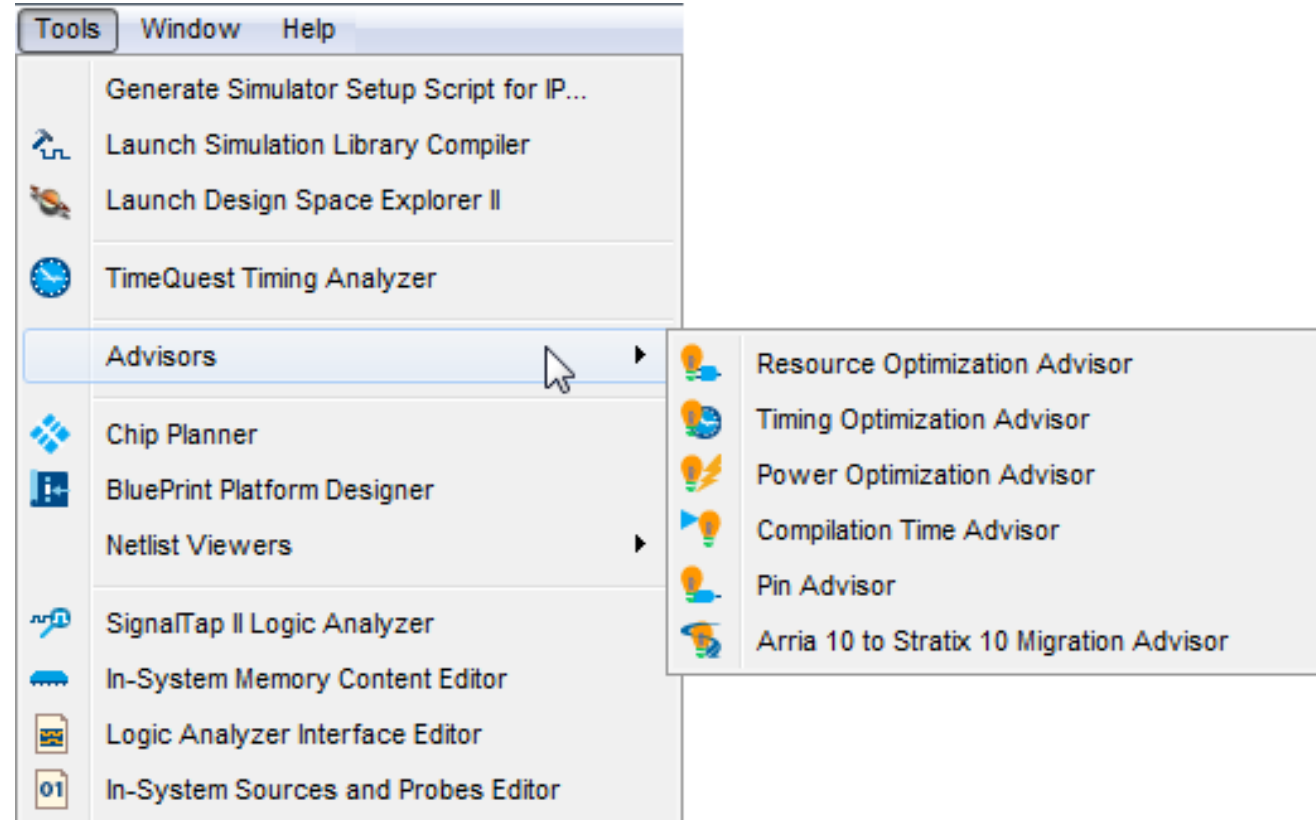
Select “Logic Options Definition” in Quartus Prime Help Contents



Links to all available assignments organized by category with lists of supported devices

ADVISORS

- Provide project-specific recommendations (feedback) on optimizing designs
- Access through **Tools** menu
- Six types
 - Resource usage optimization
 - Timing (performance) optimization
 - Power optimization
 - Compilation time reduction
 - I/O-related recommendations
 - Arria 10 to Stratix 10 migration
- Recommendations can contradict each other between different advisors



ADVISOR EXAMPLE

Green checkmark
indicates
recommendation already
being followed

The screenshot shows the 'Timing Optimization Advisor' window. The left sidebar lists various recommendations, some with green checkmarks indicating they are followed. The main pane shows a detailed recommendation to 'Optimize for speed'. Below the text, a table lists the current settings for a node named 'mult_inst'.

Node Name	Assignment Name	Current Setting
1 mult_inst	Optimization Technique	SPEED

Below the table, it states 'Current Global Settings: Optimization Technique = BALANCED (Recommended: SPEED)'. At the bottom, there are two links: 'Open Settings dialog box - Advanced Analysis & Synthesis Settings dialog box' and 'Open Assignment Editor'. A blue arrow points from the 'Open Assignment Editor' link to the 'mult_inst' row in the table. Another blue arrow points from the text 'Part of design is following recommendation' to the 'SPEED' value in the table.

Part of design is
following
recommendation

Links to locations in software to
change settings/assignments manually

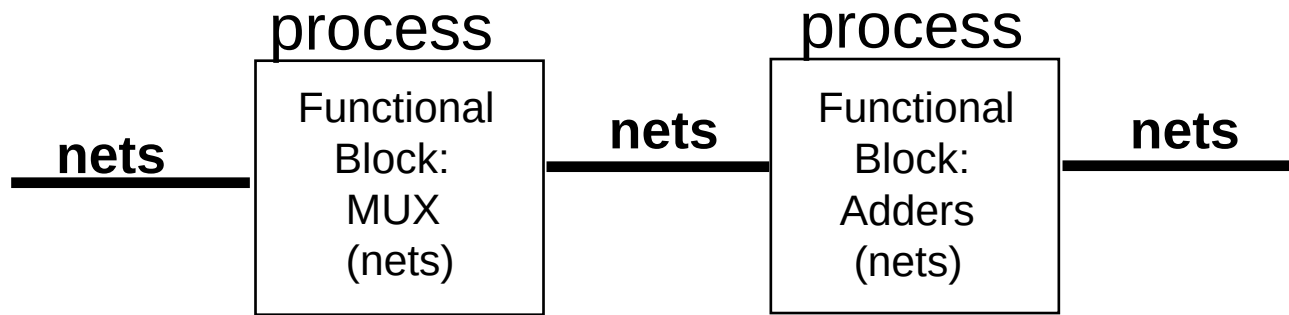
VERILOG:

INTERNAL LOGIC CONNECTION:
WIRE



DATA TYPES

- **Net data type** - represents physical interconnect between structures (**wire**)



- **Variable data type** - represents element to store data temporarily (reg – discussed later)

NET DATA TYPE & NET ARRAYS

- **wire** $\Rightarrow$ represents a node

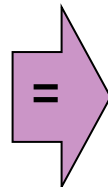
- Bus Declarations:
 - `<data_type> [MSB : LSB] <signal name> ;`
 - `<data_type> [LSB : MSB] <signal name> ;`

- Examples:
 - **wire** clk;
 - **wire** reset, enable;
 - **wire** [15:0] mult_out, adder_out;

DATA TYPE

- Every signal (which includes ports) must have an assigned data type
- Data types for signals must be explicitly declared in the declarations of your module
- Ports are **wire** (net) data types by default if not explicitly declared

```
module mult_acc (  
    input [7:0] dataa, datab,  
    input clk, aclr,  
    output [15:0] mac_out  
);
```



```
module mult_acc (  
    input wire [7:0] dataa, datab,  
    input wire clk, aclr,  
    output wire [15:0] mac_out  
);
```

ASSIGNMENT TO WIRES

- Continuous assignments can be made when the net is declared

OR

- By using the **assign** statement

```
wire [15:0] adder_out = mult_out + out;  
/*implicit continuous assignment */
```

is equivalent to

```
wire [15:0] adder_out;  
assign adder_out = mult_out + out;
```

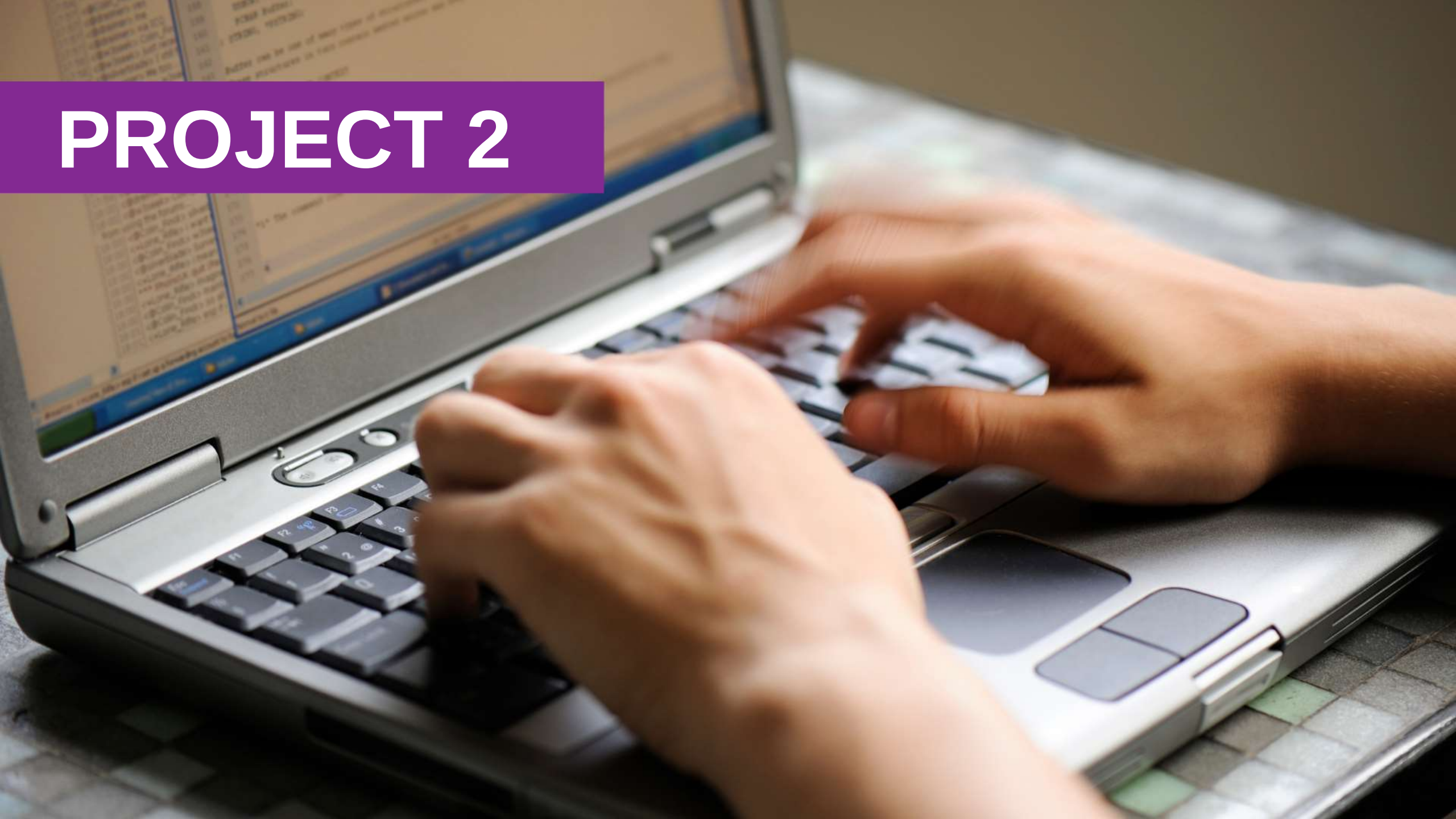
CONTINUOUS ASSIGNMENT - EXAMPLE

```
module and_func (  
    input [7:0] ina, inb,  
    output [7:0] out  
);  
  
wire [7:0] and_ab;  
assign and_ab = ina & inb;  
assign out = and_ab;  
  
endmodule
```

OR

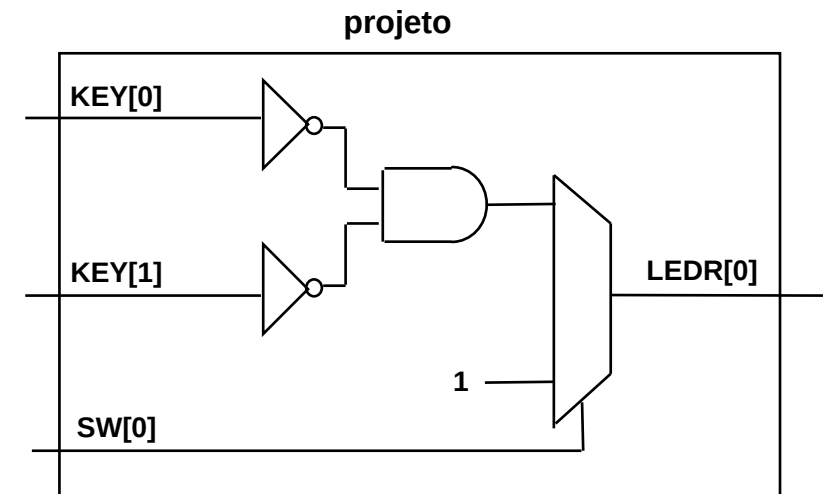
```
module and_func (  
    input [7:0] ina, inb,  
    output [7:0] out  
);  
  
wire [7:0] and_ab = ina & inb;  
  
assign out = and_ab;  
  
endmodule
```

PROJECT 2



PROJECT 2

- Export the configurations from project DE10-Lite Golden Top
- Create a new project on ***New Project Wizard***
- Import the configurations from project DE10-Lite Golden Top
- Copy the projeto.v from Projeto 01 to Projeto 02 folder
- Add the projeto.v to the project, remove all other files
- Edit the projeto.v adding a multiplexer, connect the switch, according the image
 - Tip: use the conditional attribution, ternary operator ?:
- Compile, check RTL Viewer
- Test in DE10-Lite



VERILOG:

COMBINATIONAL PROCESS

EXPLICIT PROCESS: “ALWAYS”



PROCESS STATEMENTS

- Implicit process
 - Types
 - Concurrent signal assignments
 - Component instantiations
 - Process sensitive to all inputs
 - e.g. Read side of signal assignment
- Explicit process (procedural block)
 - ALWAYS keyword defines process boundary

TWO PROCEDURAL BLOCKS

- **initial**

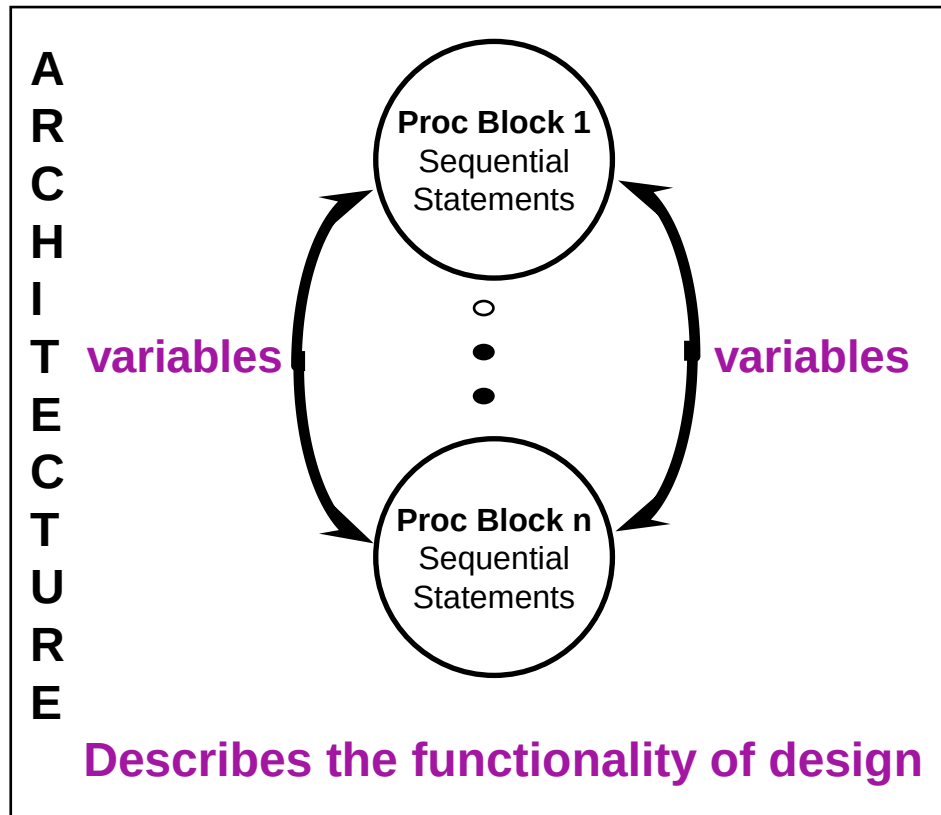
- Used to initialize behavioral statements for simulation (discussed later)

- **always**

- Used to describe the circuit functionality using behavioral statements

- ⇒ Each **always** and **initial** block represents a separate process
- ⇒ Processes “run” in parallel (instantiate independent hardware)
- ⇒ Statements inside a process “execute” sequentially (infer precedence in hardware)
- ⇒ **always** and **initial** blocks cannot be nested

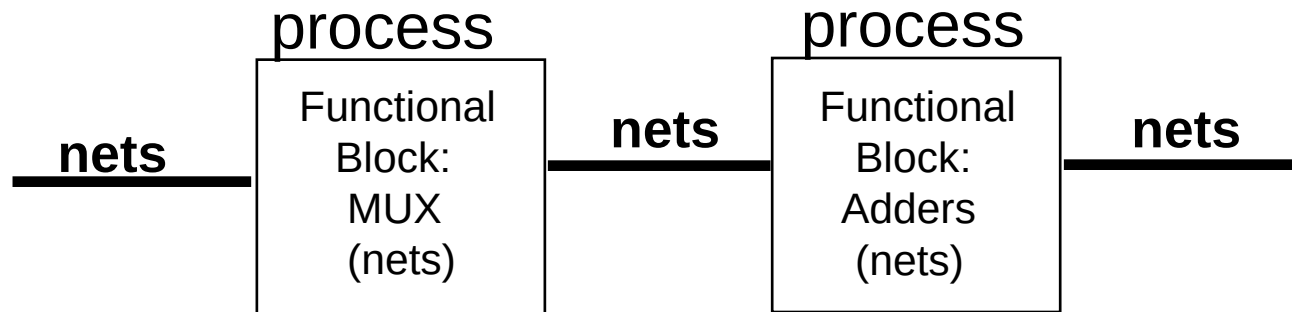
PROCESS BEHAVIOR



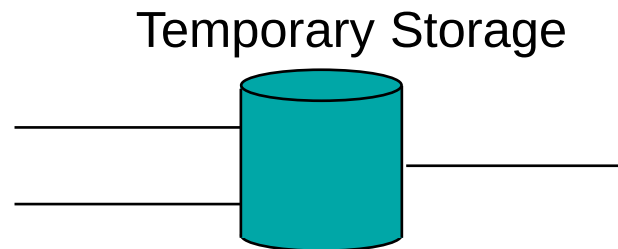
- Each procedural block executes in parallel with other procedural blocks
 - Order of **always/initial** blocks does not matter
- Within a procedural block, the statements are executed sequentially
 - Order of statements within an **always/initial** block does matter

DATA TYPES

- **Net data type** - represents physical interconnect between structures (wire)



- **Variable data type** - represents element to store data temporarily (reg)



VARIABLE DATA TYPES & VARIABLE ARRAYS

- **reg**⁽¹⁾ - unsigned variable of any bit size

Use **reg signed** for a signed implementation⁽²⁾

- **integer** - signed variable (usually 32 bits)

- Bus Declarations:

- **<data_type>** [*MSB* : *LSB*] *<signal name>* ;
- **<data_type>** [*LSB* : *MSB*] *<signal name>* ;

- Examples:

- **reg** *count*;
- **reg** [7 : 0] *out* ;

Notes:

1) Type **reg** does not refer to a physical register

2) The **signed** representation is also supported on net data types

ALWAYS BLOCK

- Is a kind of explicit process
- Consists of behavioral statements
- Each **always** block executes concurrently starting at time 0 and executes continuously in a looping fashion
- Must use keywords **begin** and **end** to group behavioral statements when **always** block contains more than one behavioral statement
- Example uses
 - Modeling a digital circuit
 - Any process or functionality that needs to be executed continuously

EVENT CONTROL & SENSITIVITY LISTS

- Use event control at the beginning of **always** block to control when block begins execution
 - Each execution of always block requires event to be satisfied
 - Forces always block to be “sensitive” to the items in event control
- Also referred to as a **Sensitivity List**
- Supported by synthesis tools

Format

```
always @(sensitivity_list) begin
    -- Statement_1
    -- .....
    -- Statement_N
end
```

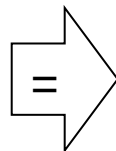
Example

```
// Process executes whenever
// a, b, c or d changes in value
always @(a, b, c, d) begin
    y <= (a ^ b) & (c ~ | d);
end
```


EVENT CONTROL & SENSITIVITY LISTS

- Verilog 2001 allows the use of the wildcard * in the sensitivity list
- The use of the wildcard is equivalent to listing all of the RHS expressions

```
always @(a, b, c, d) begin
    y <= (a ^ b) & (c ~ | d);
end
```



```
always @* begin
    y <= (a ^ b) & (c ~ | d);
end
```

ASSIGNMENTS INSIDE PROCEDURAL BLOCKS

- 1) **LHS must be a variable data type** (e.g. reg, integer, real, time)
- 2) LHS can be a bit-select or part-select
- 3) LHS can be a concatenation of any of the above
- 4) RHS can be expression containing net data type, variable data type or function call (or combination of)

```
wire mult_out, out;  
reg adder_out;
```

```
always @*  
    adder_out <= mult_out + out;
```

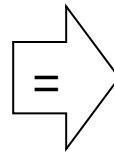
(LHS) Left-Hand Side
Right-Hand Side (RHS)

IMPLICIT AND EXPLICIT PROCESSES EQUIVALENCE

- The **assign** statement infers an implicit process
- The order of assign statements doesn't matter, as well as the order of the always blocks

```
always @* begin
```

```
    y <= (a ^ b) & (c ~ | d);  
end
```



```
assign y = (a ^ b) & (c ~ | d);
```

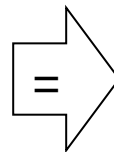
```
assign x = data_in;
```

```
always @* begin
```

```
    y <= x;
```

```
end
```

```
assign data_out = y;
```



```
assign data_out = y;
```

```
assign x = data_in;
```

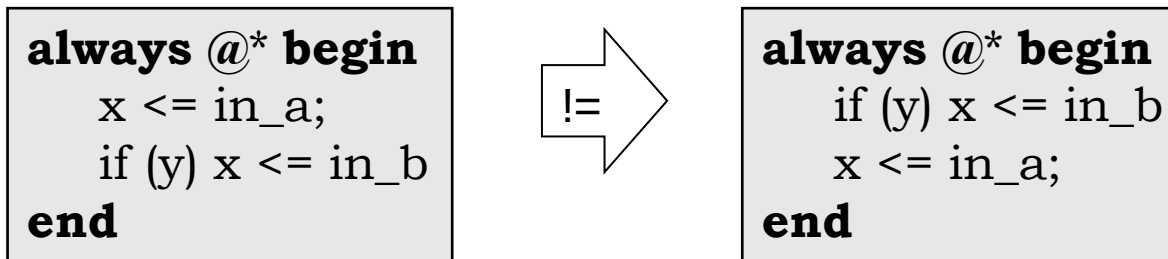
```
always @* begin
```

```
    y <= x;
```

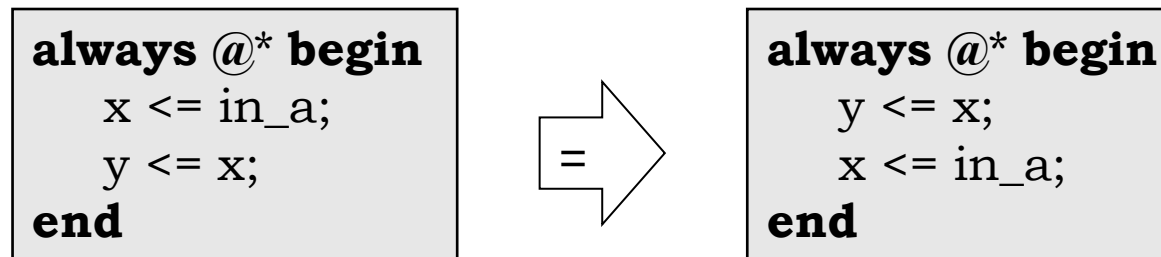
```
end
```

EXECUTION BEHAVIOR INSIDE PROCEDURAL BLOCKS

- The order of assignments inside a procedural block is important



- ...but only for the same LHS



EXPLICIT PROCESS

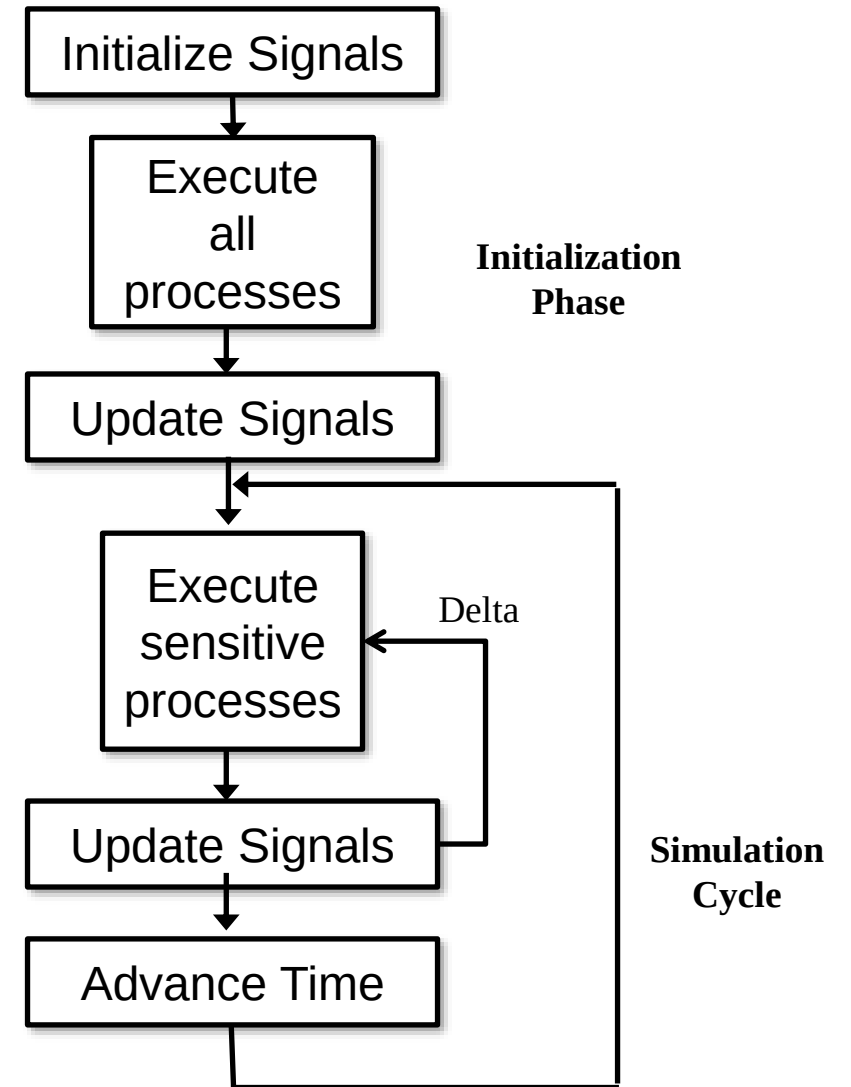
SIMULATION

SIMULATION DELTA AND CYCLE



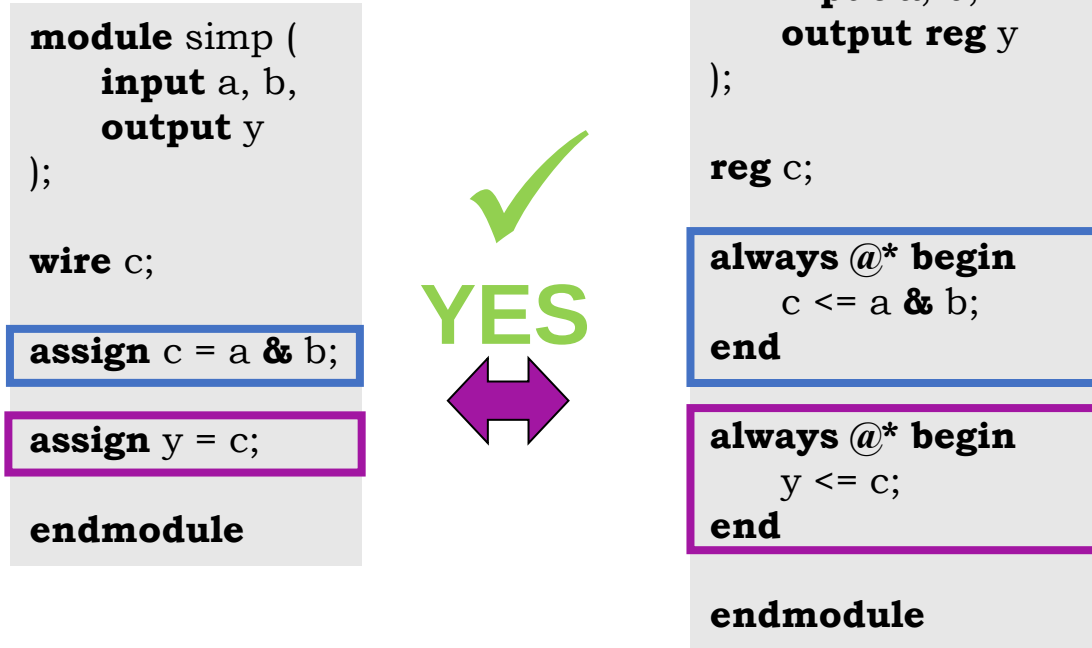
EVALUATING MODEL BEHAVIOR

- Simulation cycle
 - Wall clock time
 - Delta
 - PROCESS execution phase
 - Signal update phase
- When does a delta cycle end?
 - After all processes end execution
 - End of the process (with sensitivity list)
 - Process encounters WAIT statement
 - After which, any signals written to during delta cycle are updated
- When does a simulation cycle end?
 - When updating signals to new values at the end of delta cycle does not cause a brand new delta cycle (i.e. new processes aren't triggered by changing signals)
- **Signals get updated at the end of the delta cycle (delay)**



**Note: Both simulation and synthesis tools evaluate VHDL models the this way.*

EQUIVALENT FUNCTIONS?



c AND y get executed and updated in parallel at the end of the process within one simulation cycle

EQUIVALENT FUNCTIONS?

```

module simp (
    input a, b,
    output y
);

wire c;

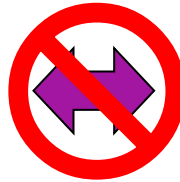
assign c = a & b;

assign y = c;

endmodule

```

NO



```

module simp (
    input a, b,
    output reg y
);

reg c;

always @(a, b) begin
    c <= a & b;

    y <= c;

end

endmodule

```

New value of **c** not available for **y** until next process execution (requires another simulation cycle or transition on **a/b**)

EQUIVALENT FUNCTIONS?

```
module simp (
    input a, b,
    output y
);
```

```
    wire c;
```

```
    assign c = a & b;
```

```
    assign y = c;
```

```
endmodule
```



```
module simp (
    input a, b,
    output reg y
);
```

```
    reg c;
```

```
    always @(a, b, c) begin
```

```
        c <= a & b;
```

```
        y <= c;
```

```
    end
```

```
endmodule
```

Complete sensitivity list solves the problem
always @* would work too!

VERILOG:

SEQUENTIAL STATEMENTS

- IF
- CASE
- LOOP



BEHAVIORAL STATEMENTS

- Describe behavior and express order
- Must be used inside procedural block
- Behavioral Statements
 - **if-else** statement
 - **case** statement
 - Loop statements

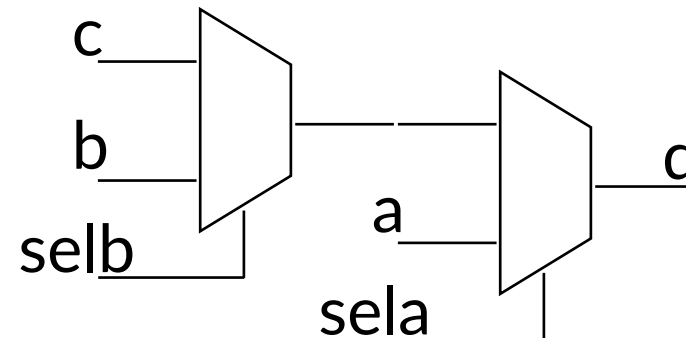
IF-ELSE STATEMENTS

Format:

```
if <condition1> begin
    {sequence of statement(s)}
    ...
end else begin
    {sequence of statement(s)}
    ...
end
```

Example:

```
always @ (sela, selb, a, b, c) begin
    if (sela) begin
        q <= a;
    end else if (selb) begin
        q <= b;
    end else begin
        q <= c;
    end
end
```



IF-ELSE STATEMENTS

- Conditions are evaluated in order from top to bottom
 - Prioritization
- The first condition, that is true, causes the corresponding sequence of statements to be executed
- If all conditions are false, then the sequence of statements associated with the final “**else**” clause are evaluated

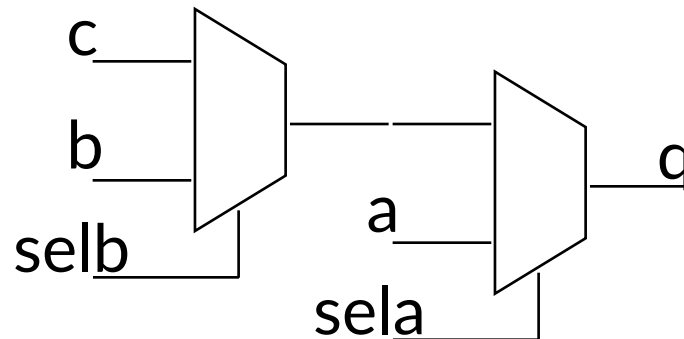
IF-ELSE STATEMENTS

Implicit Process:

```
wire q;  
assign q = (sela) ? a :  
          (selb) ? b : c;
```

Explicit process:

```
reg q;  
always @ (sela, selb, a, b, c) begin  
    if (sela)  
        q <= a;  
    else if (selb)  
        q <= b;  
    else  
        q <= c;  
end
```



LATCHES – COMBINATIONAL LOOP

Level-Sensitive Latch

```

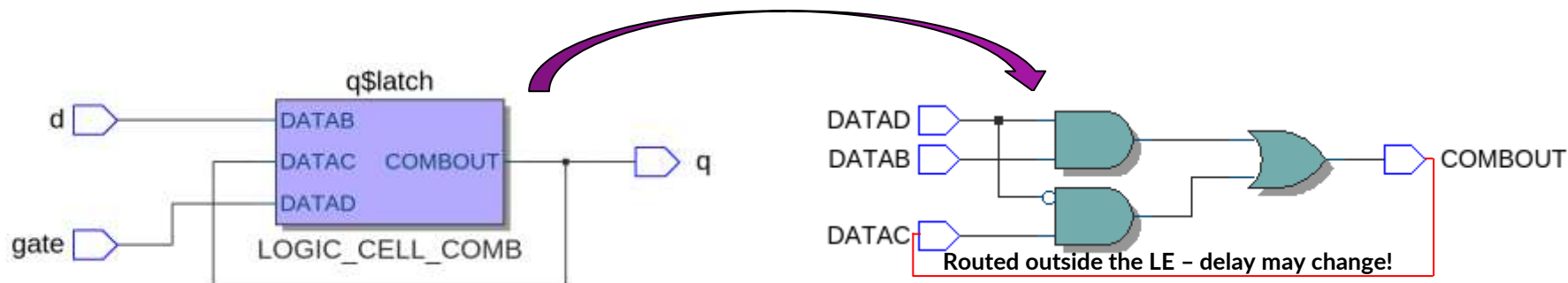
module latch (
  input d, gate,
  output reg q
);

  always @(d, gate)
    if (gate)
      q = d ;

endmodule

```

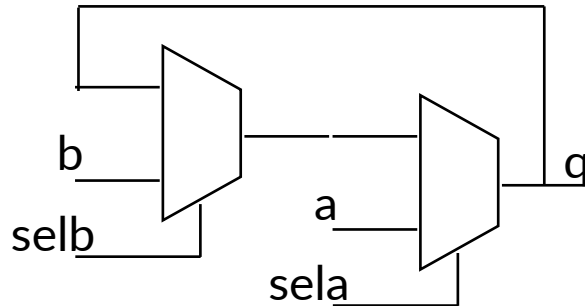
- Altera/Intel FPGAs have no dedicated latches – they will be implemented as a combinational loop
- Combinational loop behavior may depend on propagation delays, which may differ between compilations
- Race conditions may take place
- Some combinational loops may generate an oscillatory output
- Timing Analysis may become tricky (complicated SDC needed to correctly define the desired behavior)



IF-ELSE STATEMENTS – COMBINATIONAL LOOPS

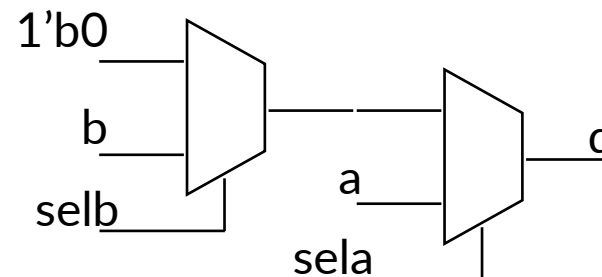
- If statement with conditions not covered generates a combinational loop

```
always @* begin
  if (sela)
    q <= a;
  else if (selb)
    q <= b;
end
```



- Two ways to avoid this:
 - Cover all conditions
 - Set a default value

```
always @* begin
  if (sela)
    q <= a;
  else if (selb)
    q <= b;
  else
    q <= 1'b0;
end
```



```
always @* begin
  q <= 1'b0;
  if (sela)
    q <= a;
  else if (selb)
    q <= b;
end
```

CASE STATEMENTS

Format:

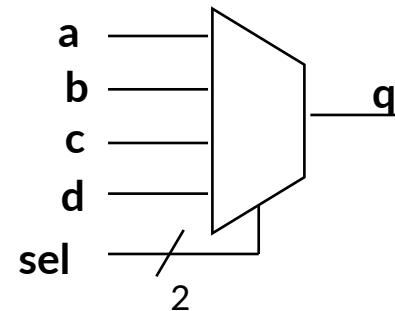
```

case {expression}
  <condition1> :
    {sequence of statements}
  <condition2> :
    {sequence of statements}
  ...
  default : -- (optional)
    {sequence of statements}
endcase
  
```

Example:

```

always @ (sel, a, b, c, d) begin
  case (sel)
    2'b00 : q <= a;
    2'b01 : q <= b;
    2'b10 : q <= c;
    default : q <= d;
  endcase
end
  
```



CASE STATEMENTS

- Conditions are evaluated in order
- First matching value is chosen
- Treats both X and Z as actual logic values
- **default** clause represents all other possible conditions that are not specifically stated
- Verilog does not require (though it is recommended) that
 - All possible conditions be considered
 - All conditions be unique

LOOP STATEMENTS

⇒ Loop statements - used for repetitive operations

- **repeat** loop - executes a fixed number of times
- **for** loop - executes once at the start of the loop and then executes if expression is true

Not synthesizable! (discussed later)

- **forever** loop - executes continually
- **while** loop - executes if expression is true

FOREVER AND REPEAT LOOPS

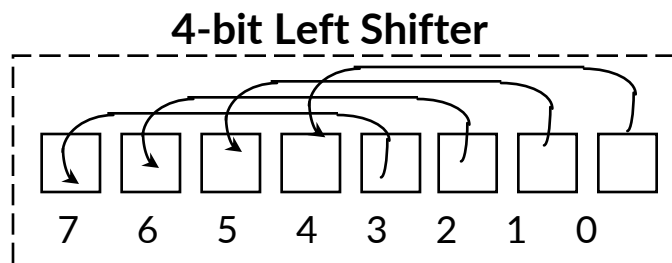
repeat loop - executes a fixed number of times

```
if (rotate == 1)
    repeat (8) begin
        tmp = data[15];
        data = {data << 1, tmp};
    end
```

Repeats a rotate
operation 8 times

FOR LOOP

for loop - executes once at the start of the loop and then executes if expression is true



```
// declare the index for the FOR LOOP  
integer i;
```

```
always @(inp, cnt) begin
```

```
    result[7:4] = 0;
```

```
    result[3:0] = inp;
```

```
    if (cnt == 1) begin
```

```
        for (i = 4; i <= 7; i = i + 1) begin
```

```
            result[i] = result[i-4];
```

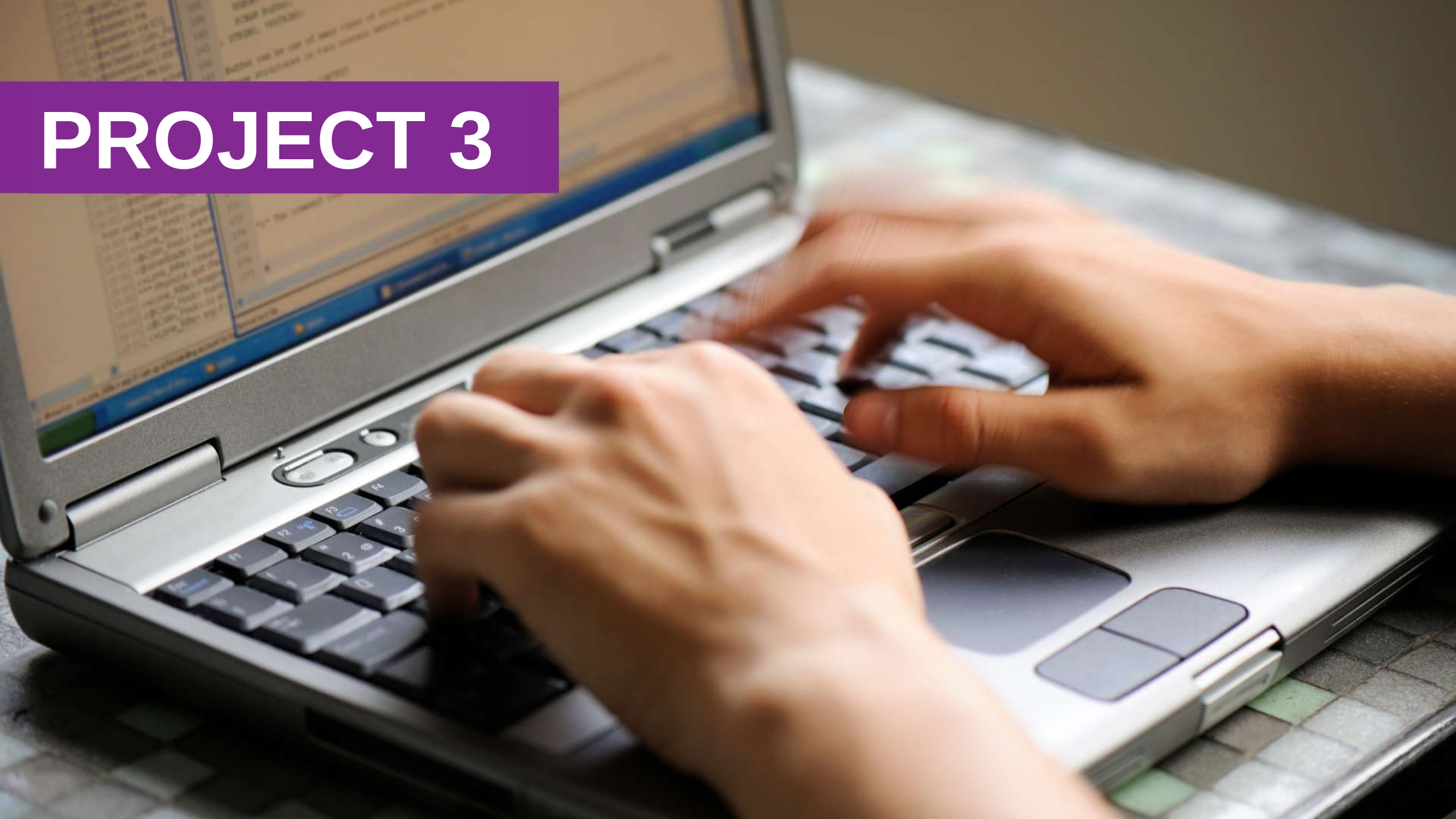
```
        end
```

```
        result[3:0] = 0;
```

```
    end
```

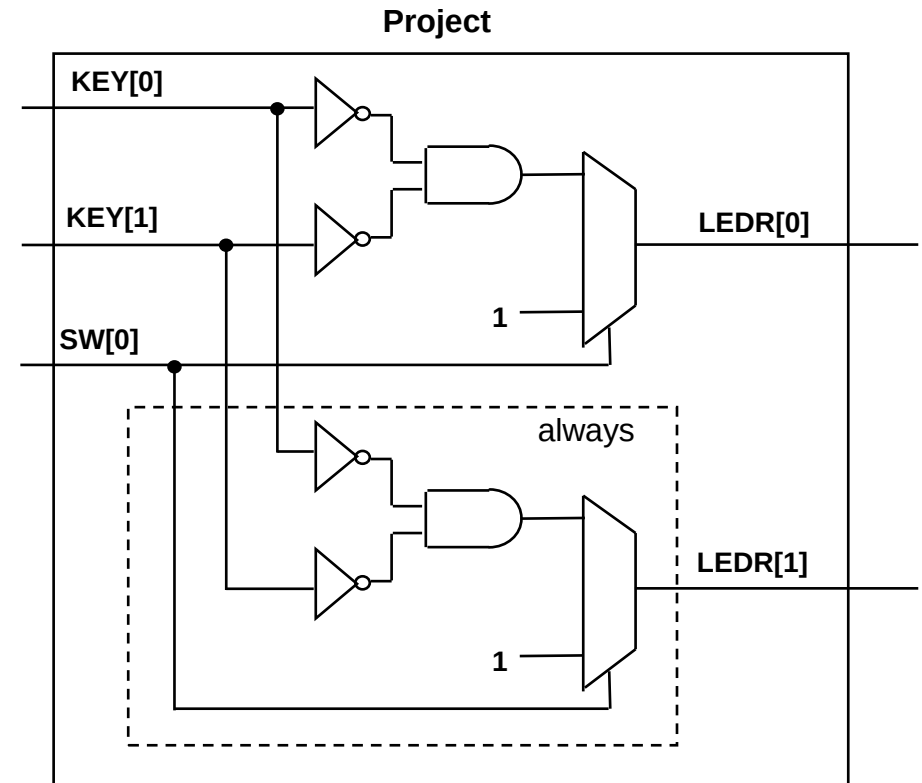
```
end
```


PROJECT 3



PROJECT 3

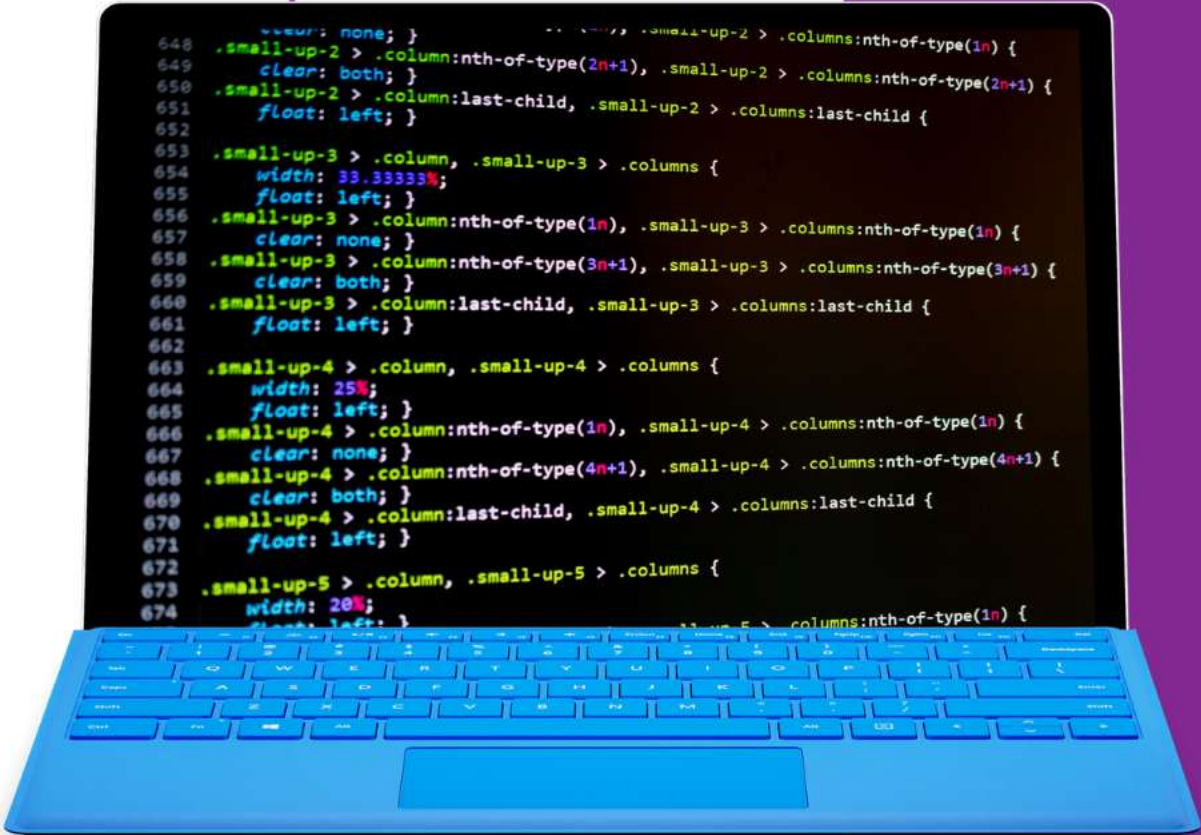
- Add in Projeto 02 an explicit process ***always*** with the same function as the conditional attribution.
- Connect an output from this process in a second LED
- Compile, check the RTL Viewer
- Configure the Board and test it.



Both LEDs must function the same way!

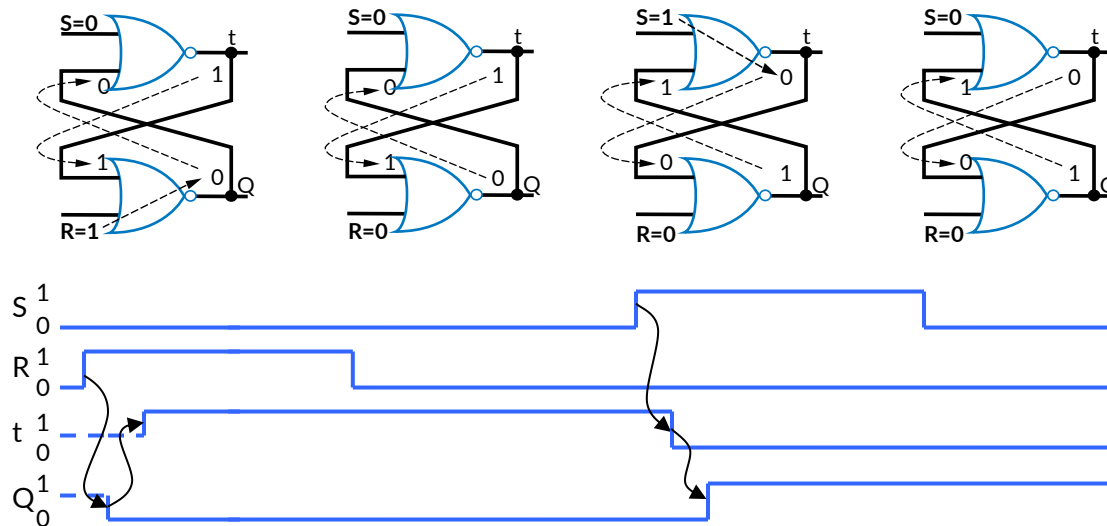
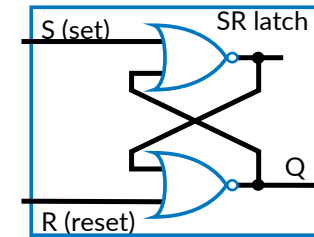
SYNCHRONOUS DESIGN

REGISTERS



SEQUENTIAL CIRCUITS

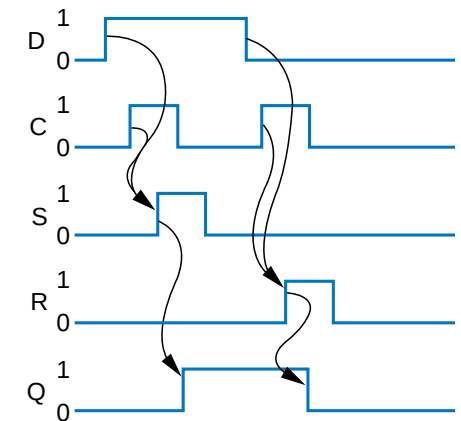
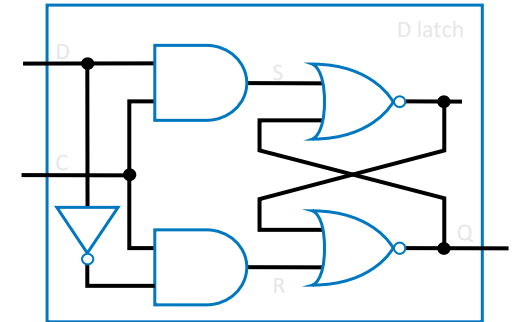
- “Latch Set-Reset” is a simple circuit capable of store a value through a combinational loop.
- The output will become high when the “set” is turned on
- The output will become low when the “reset is turned on



SYNCHRONOUS SEQUENTIAL CIRCUITS

- Adding a clock signal, this memory element can store a bit for a single period of the clock
- Replacing the Set and Reset inputs for a Data input, the illegal state issue is avoided
- This memory component is called “D-Latch” or “Transparent Latch”

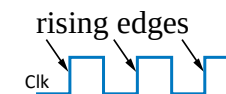
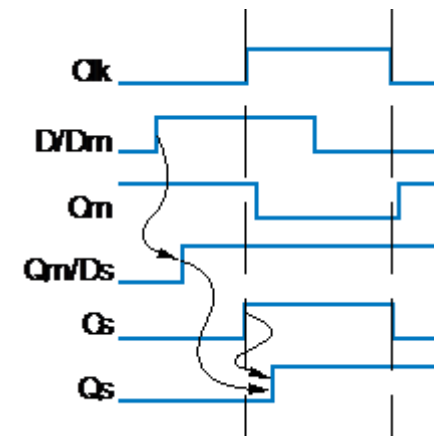
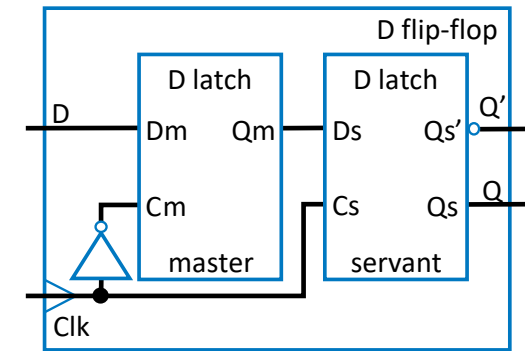
C	D	Q^{n+1}
0	0	Q^n
0	1	Q^n
1	0	0
1	1	1



Adapted from “Digital Design” by Frank Vahid
Copyright 2006 – Frank Vahid

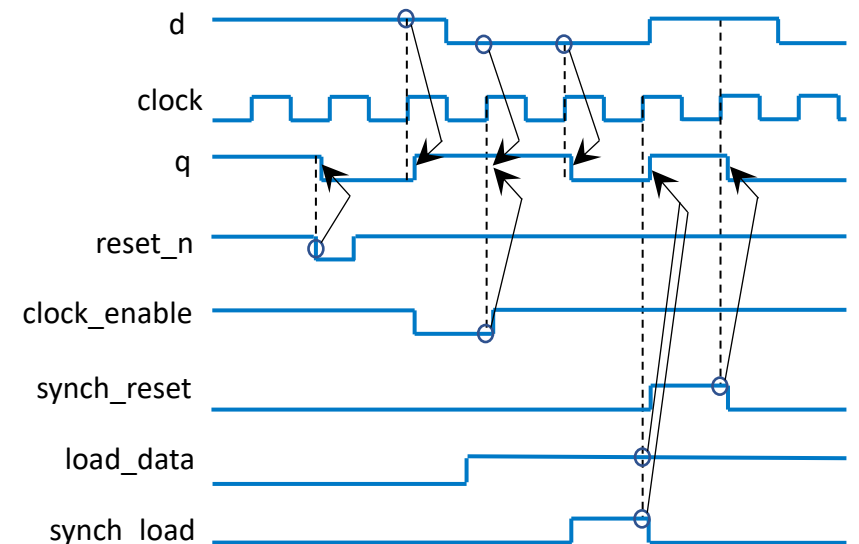
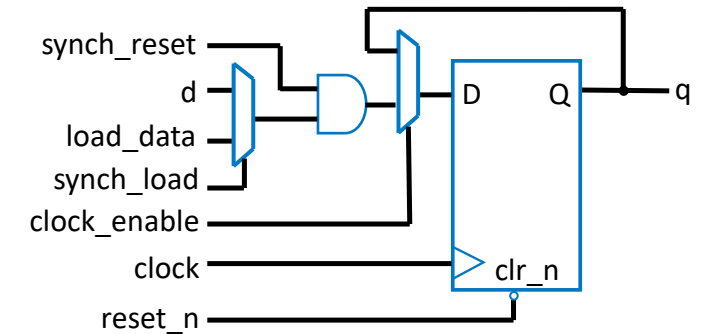
SYNCHRONOUS SEQUENTIAL CIRCUITS

- Most synchronous designs used today, especially in FPGA design, it's not usual to implement latches. A safer component and more practical to work with was developed from the latches, the output signal will only be updated by the input signal in the clock's edge trigger event (clock's rising edge or clock's falling edge). Known as **register** or **flip-flop**.
 - This is one of the multiple possible applications to the D flip-flop. There are other types of registers and other applications for the D-type.



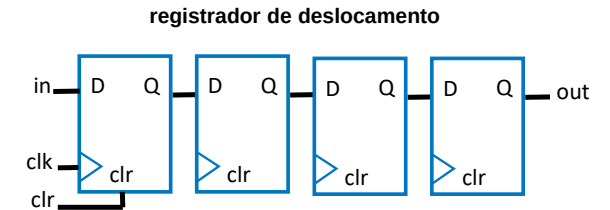
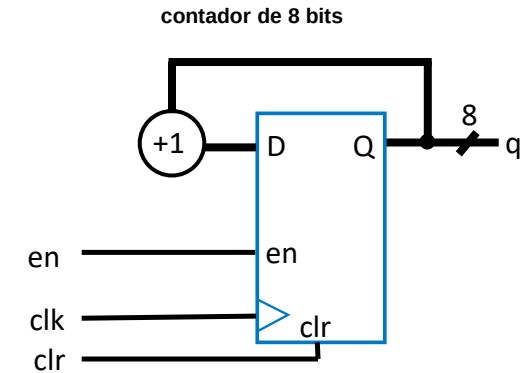
SYNCHRONOUS SEQUENTIAL CIRCUITS

- Register can have other control signals beyond the input, output and clock signal.
- The control signals can be sorted into:
 - Asynchronous control signal: control signal capable of changing the register behavior at any time, regardless the clock signal (ex: reset)
 - Synchronous control signal: its functions only work on clock signal edges (ex: clock_enable, load_data)



SYNCHRONOUS SEQUENTIAL CIRCUITS

- It's possible to build multiple logic blocks using registers
- Some blocks are used frequently
 - Counter – multiple purposes
 - Shift register– can be used in serial communication, for example
- It's possible to store generic control signals using registers



VERILOG:

EXPLICIT PROCESS: “ALWAYS”

**SYNCHRONOUS PROCESS -
REGISTERS**



RTL PROCESSES

- If you remember, RTL is a synthesizable behavioral coding style
- RTL coding style involves two types of procedural blocks (processes)
 - Combinatorial Process
 - Clocked Process

TWO TYPES OF RTL PROCESSES

• Combinatorial Process

- Sensitive to all inputs used in the combinatorial logic

```
always @ (a, b, sel)
always @ *
```

Sensitivity list includes
all inputs used in the
combinatorial logic

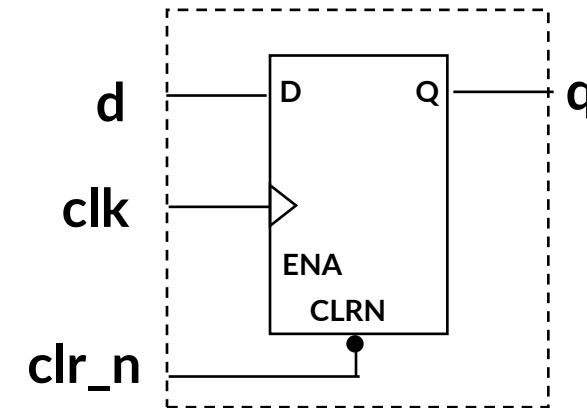
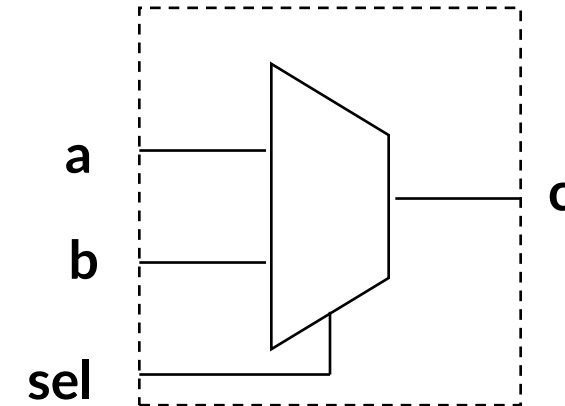
** is a Verilog shortcut to manually having to add all inputs*

• Clocked Process

- Sensitive to a clock or/and control signals

```
always @(posedge clk, negedge clr_n)
```

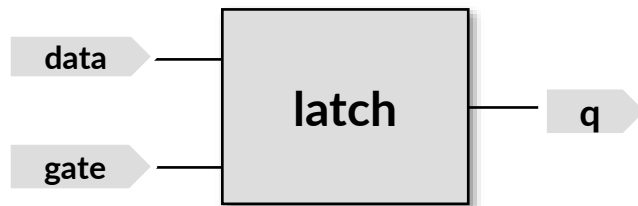
Sensitivity list does not include the **d** input,
only the clock and asynchronous control signals



FUNCTIONAL LATCH VS. FUNCTIONAL FLIPFLOP

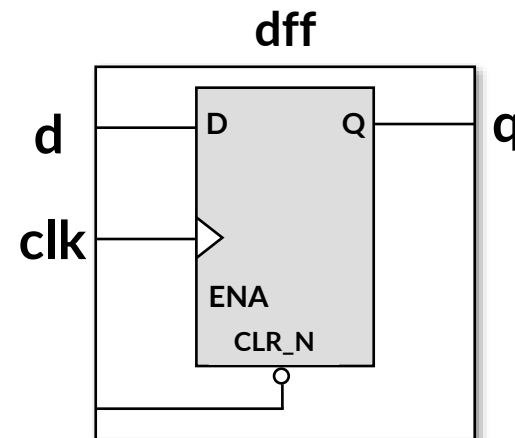
Level-Sensitive Latch

```
module latch (
  input d, gate,
  output reg q
);
  always @(d, gate)
    if (gate)
      q <= d;
endmodule
```



Edge-Triggered Flipflop

```
module dff (
  input d, clk,
  output reg q
);
  always @(posedge clk)
    q <= d;
endmodule
```



SYNCHRONOUS VS. ASYNCHRONOUS

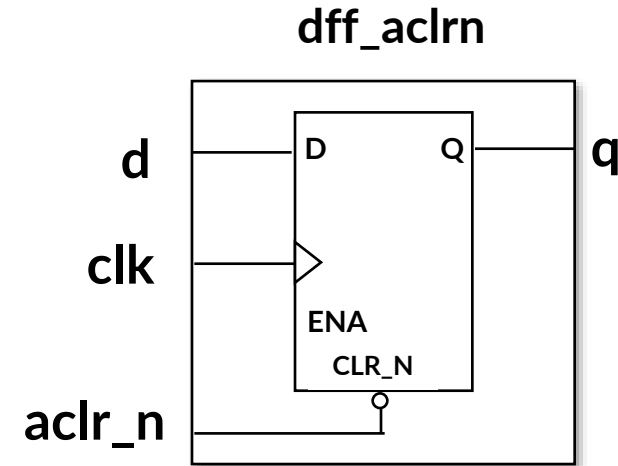
Asynchronous Clear

```

module dff_async (
    input d, clk, aclr_n,
    output reg q
);

    always @(posedge clk,
            negedge aclr_n) begin
        if (~aclr_n)
            q <= 1'b0;
        else
            q <= d;
        end
    endmodule

```



Asynchronous control signals implementation

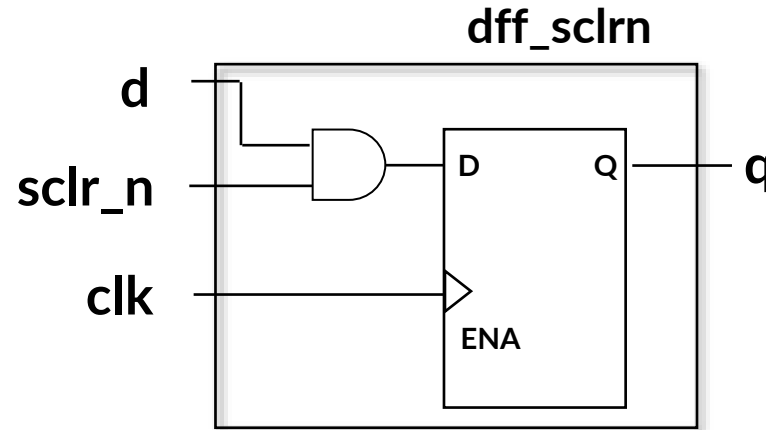
- Asynchronous control signal is included in the sensitivity list and is checked before the clock condition
- **aclr_n='0'** does not depend on the clock

SYNCHRONOUS VS. ASYNCHRONOUS

Synchronous Clear

```
module dff_sync (
    input d, clk, sclr_n, spre,
    output reg q
);

    always @(posedge clk) begin
        if (~sclr_n)
            q <= 1'b0;
        else
            q <= d;
        end
    endmodule
```



Synchronous control signal implementation

- Synchronous control signals are **not** included in the sensitivity list and are checked inside in the clock condition
- **sclr_n='0'** depends on the clock

CLOCK ENABLE

Clock Enable

```

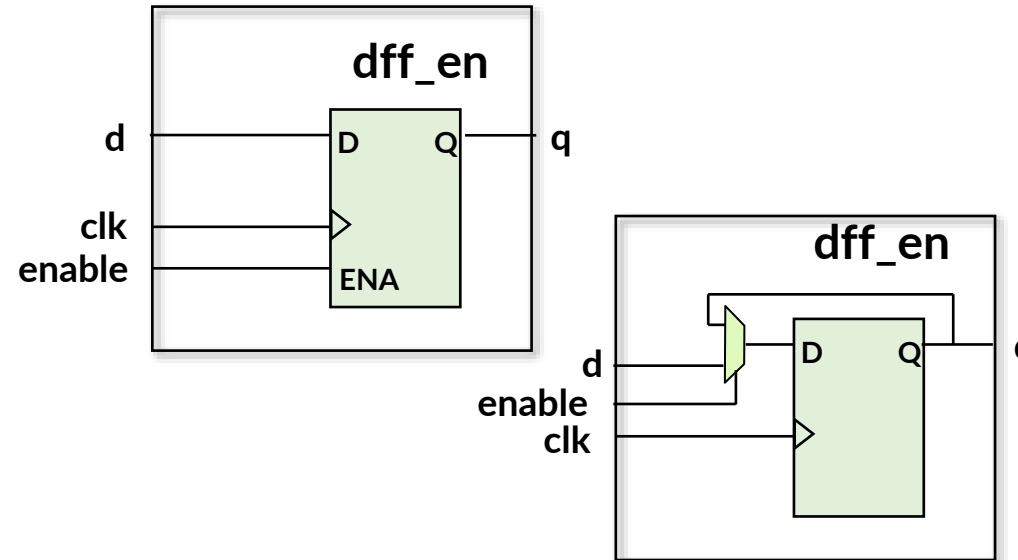
module dff_en (
  input d, enable, clk;
  output reg q
);

/* If clock enable port does not exist in
target technology, then a mux in
front of the d input is generated */

always @( posedge clk )
  if (enable)
    q <= d;

endmodule

```



Clock Enable

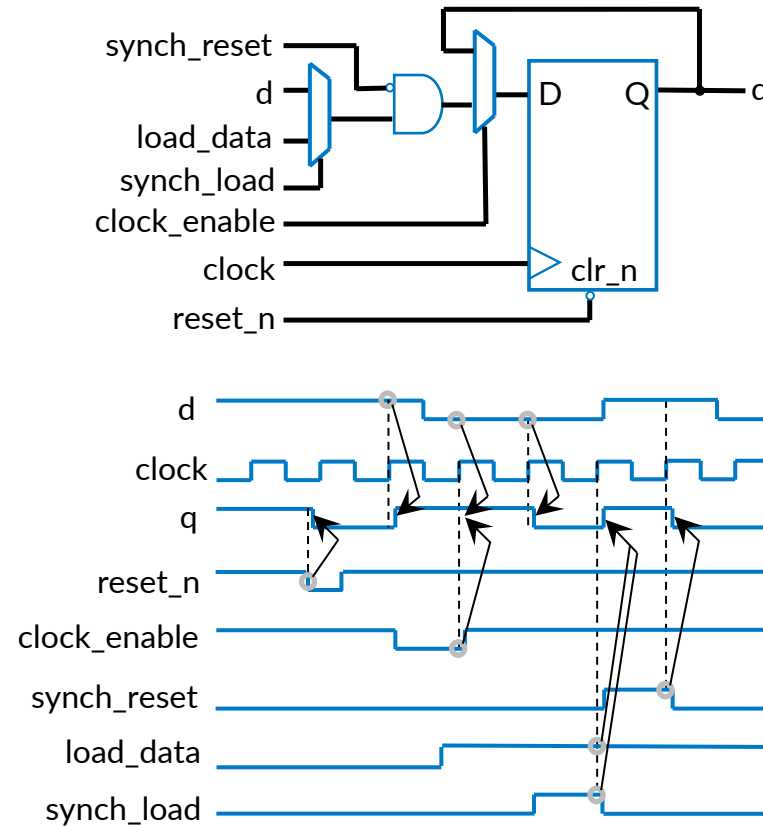
- Enable is a synchronous control signal
- It has precedence over other synchronous control signals
- For some Altera devices, the clock may be entirely disabled by this signal
- Otherwise, a mux may be generated

FULL FEATURED REGISTER

```

reg q;
always @ (posedge clock, negedge reset_n, posedge asynch_load ) begin
    if ( ~reset_n ) begin
        q <= 1'b0;
    end else if ( asynch_load ) begin
        q <= load_data;
    end else begin
        if ( clock_enable ) begin
            if ( synch_reset ) begin
                q <= 1'b0;
            end else if ( synch_load ) begin
                q <= load_data;
            end else begin
                q <= d;
            end
        end
    end
end

```



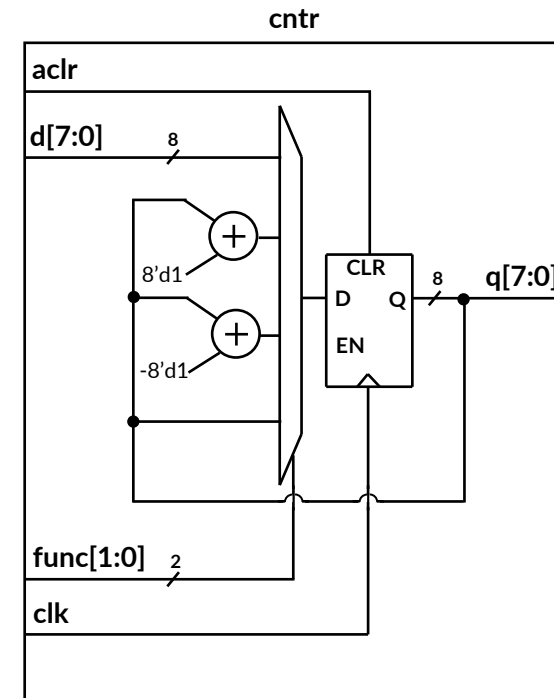
FUNCTIONAL COUNTER

```

module cntr (
    input aclr, clk,
    input [7:0] d,
    input [1:0] func, // Controls functionality
    output reg [7:0] q
);

always @ (posedge clk, posedge aclr) begin
    if (aclr)
        q <= 8'h00;
    else
        case (func)
            2'b00: q <= d; // Loads counter
            2'b01: q <= q + 1; // Counts up
            2'b10: q <= q - 1; // Counts down
        endcase
    end
endmodule

```



SHORT QUIZ:

HOW MANY REGISTERS?

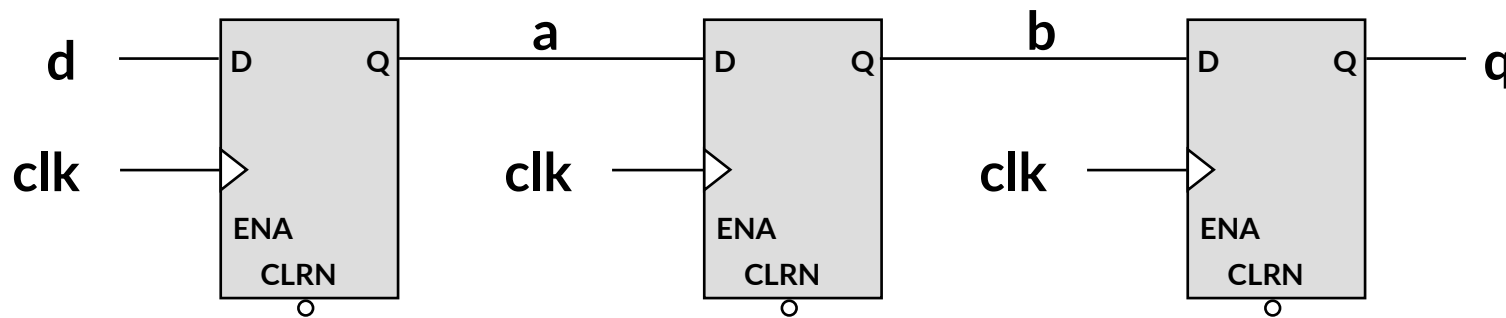


```
module reg1 (  
    input d,  
    input clk,  
    output reg q  
);  
  
reg a, b;  
always @ (posedge clk) begin  
  
    a <= d;  
    b <= a;  
    q <= b;  
  
end  
  
endmodule
```

HOW MANY REGISTERS?

- Assignments inside register constructs infer registers
- Three registers created

```
always @ (posedge clk) begin
    a <= d;
    b <= a;
    q <= b;
end
```



SHORT QUIZ:

HOW MANY REGISTERS?



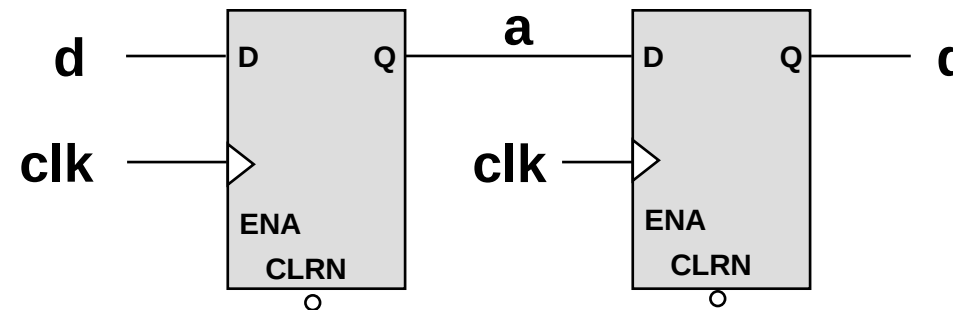
```
module reg1 (  
    input d,  
    input clk,  
    output q  
);  
  
reg a, b;  
always @ (posedge clk) begin  
    a <= d;  
    b <= a;  
end  
  
assign q = b;  
  
endmodule
```

HOW MANY REGISTERS?

- Output q assignment is no longer edge-sensitive because it is outside of register construct

```
reg a, b;
always @ (posedge clk)
begin
    a <= d;
    b <= a;
end

assign q = b;
```



- Two registers created
 - b synthesized out

SHORT QUIZ:

HOW MANY REGISTERS?



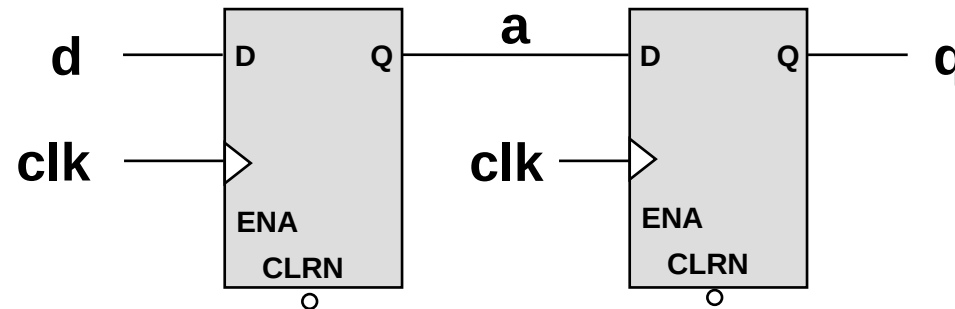
```
module reg1 (  
    input d,  
    input clk,  
    output reg q  
);  
  
reg a, b;  
always @ (posedge clk) begin  
    a <= d;  
    b <= a;  
end  
  
always @* begin  
    q <= b;  
end  
  
endmodule
```

HOW MANY REGISTERS?

- Even though q is declared as a reg it doesn't mean it will synthesize a register. In this case, it must be a reg (variable data type) because it's value is assigned within an always block

```
reg q;
reg a, b;
always @ (posedge clk) begin
    a <= d;
    b <= a;
end

always @* begin
    q <= b;
end
```



- Two registers created
 - b synthesized out

PROCEDURAL ASSIGNMENT TYPES

In Verilog, there are two types of procedural assignment statements

- Blocking
- Nonblocking



PROCEDURAL ASSIGNMENT TYPES (CONT.)

- **Nonblocking** ($\leq$) : Schedules LHS assignments without blocking execution of the statements that follow in a sequential block
 - RHS (inputs) sampled when statement executed
 - LHS (outputs) scheduled to be updated at end of the time step or after delay expires
 - Statements following do not wait until nonblocking LHS assignments are made to begin execution

- **Blocking** ($=$) : Updates LHS assignments blocking execution of other assignments in the procedural block until finished
 - RHS (inputs) sampled when statement executed
 - LHS (outputs) updated immediately or after defined delay
 - Statements following must wait until blocking statement completely finished and LHS assignments are made (including delay) to begin execution

SHORT QUIZ:

HOW MANY REGISTERS?



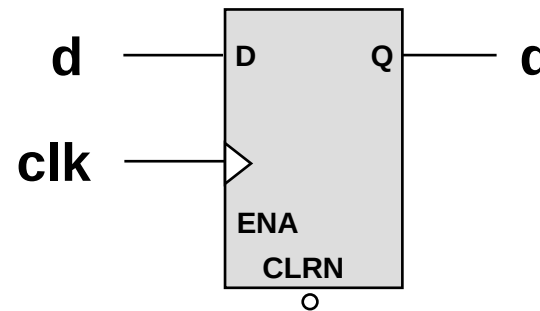
```
module reg1 (  
    input d,  
    input clk,  
    output reg q  
);  
  
reg a, b;  
always @ (posedge clk) begin  
    a = d;  
    b = a;  
    q <= b;  
end  
  
endmodule
```

Using blocking
assignments

HOW MANY REGISTERS?

- Blocking assignments are treated as wires – the value is assigned right away.

```
reg q;  
reg a, b;  
always @ (posedge clk) begin  
    a = d;  
    b = a;  
    q <= b;  
end
```



- One registers created
 - a and b synthesized out

EQUIVALENT FUNCTIONS?

```
module simp (  
  input a, b,  
  output y  
);
```

```
reg z;  
always @* begin
```

```
  z <= a & b;
```

```
  y <= z;
```

```
end
```

```
endmodule
```

✓
YES
↔

```
module simp (  
  input a, b,  
  output y  
);
```

```
reg z;  
always @* begin
```

```
  y <= z;
```

```
  z <= a & b;
```

```
end
```

```
endmodule
```

y and z is scheduled to be updated at the end of simulation delta. Two deltas are necessary to end the cycle.



EQUIVALENT FUNCTIONS?

```
module simp (  
    input a, b,  
    output y  
);
```

```
    reg z;  
    always @* begin
```

```
        z = a & b;
```

```
        y = z;
```

```
    end
```

```
endmodule
```

NO



```
module simp (  
    input a, b,  
    output y  
);
```

```
    reg z;  
    always @* begin
```

```
        y = z;
```

```
        z = a & b;
```

```
    end
```

```
endmodule
```

y and z get executed and updated immediately (no scheduling).



EQUIVALENT FUNCTIONS?

```

module simp (
    input clk,
    input [7:0] x, lim,
    output reg [7:0] y
);

always @(posedge clk) begin
    y <= x;

    if (x > lim[7:2]-((DEC)?1:0)
        y <= lim[7:2]-((DEC)?1:0);

end
endmodule

```



```

module simp (
    input clk,
    input [7:0] x, lim,
    output reg [7:0] y
);

reg [7:0] tmp;
always @(posedge clk) begin
    y <= x;

    tmp <= lim[7:2]-((DEC)?1:0);

    if (x > tmp)
        y <= tmp;

end
endmodule

```

tmp is scheduled to be updated at the end of simulation delta, thus generating one more register.



EQUIVALENT FUNCTIONS?

```
module simp (  
    input clk,  
    input [7:0] x, lim,  
    output reg [7:0] y  
);
```

```
always @(posedge clk) begin  
    y <= x;  
end
```

```
if (x > lim[7:2]-((DEC)?1:0)
```

```
    y <= lim[7:2]-((DEC)?1:0;
```

```
end  
endmodule
```



```
module simp (  
    input clk,  
    input [7:0] x, lim,  
    output reg [7:0] y  
);
```

```
reg [7:0] tmp;  
always @(posedge clk) begin  
    y <= x;
```

```
    tmp = lim[7:2]-((DEC)?1:0;
```

```
if (x > tmp)
```

```
    y <= tmp;
```

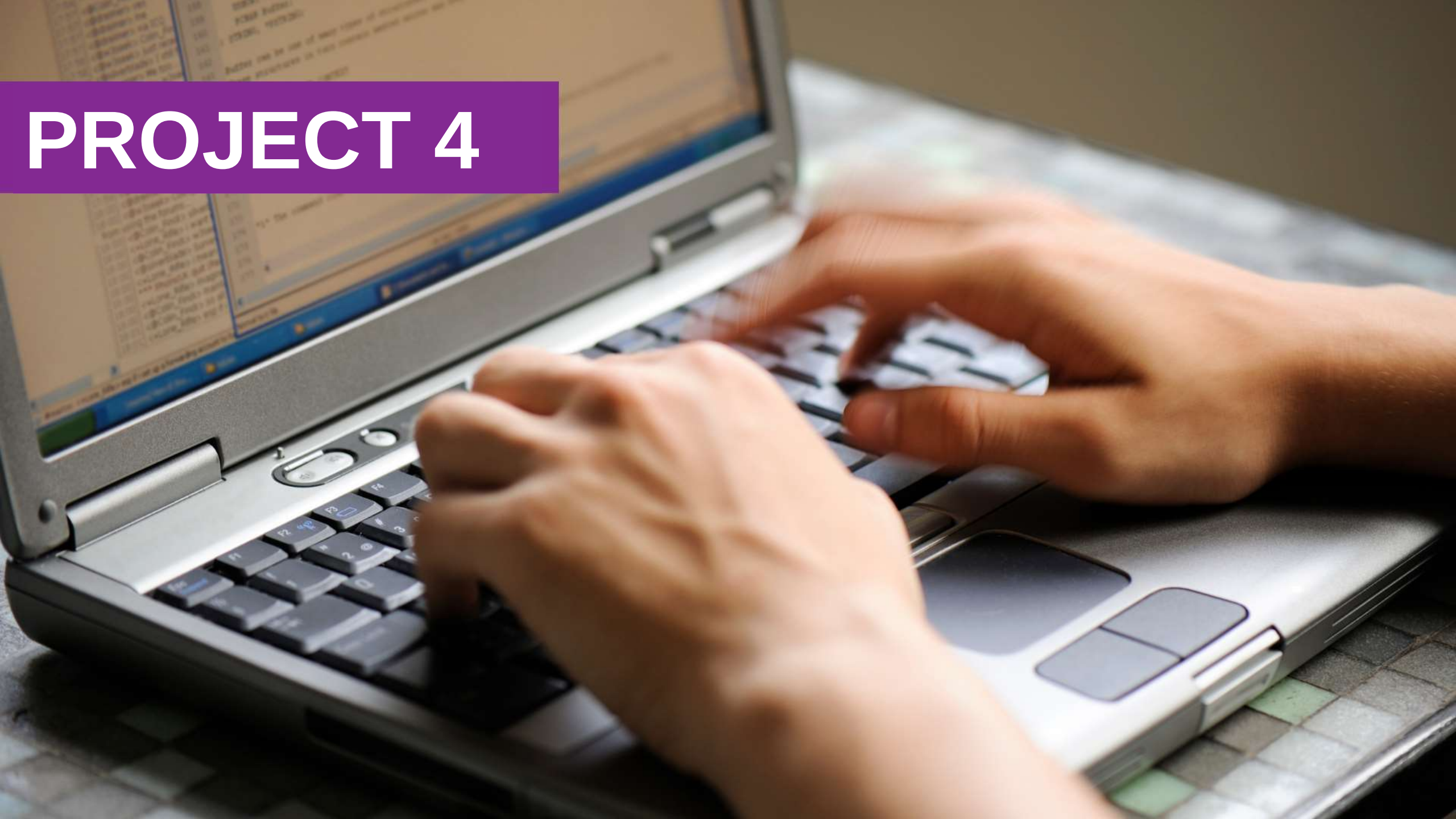
```
end  
endmodule
```

tmp is updated immediately thus no additional register is generated and **tmp** becomes a regular wire



NONBLOCKING VS. BLOCKING

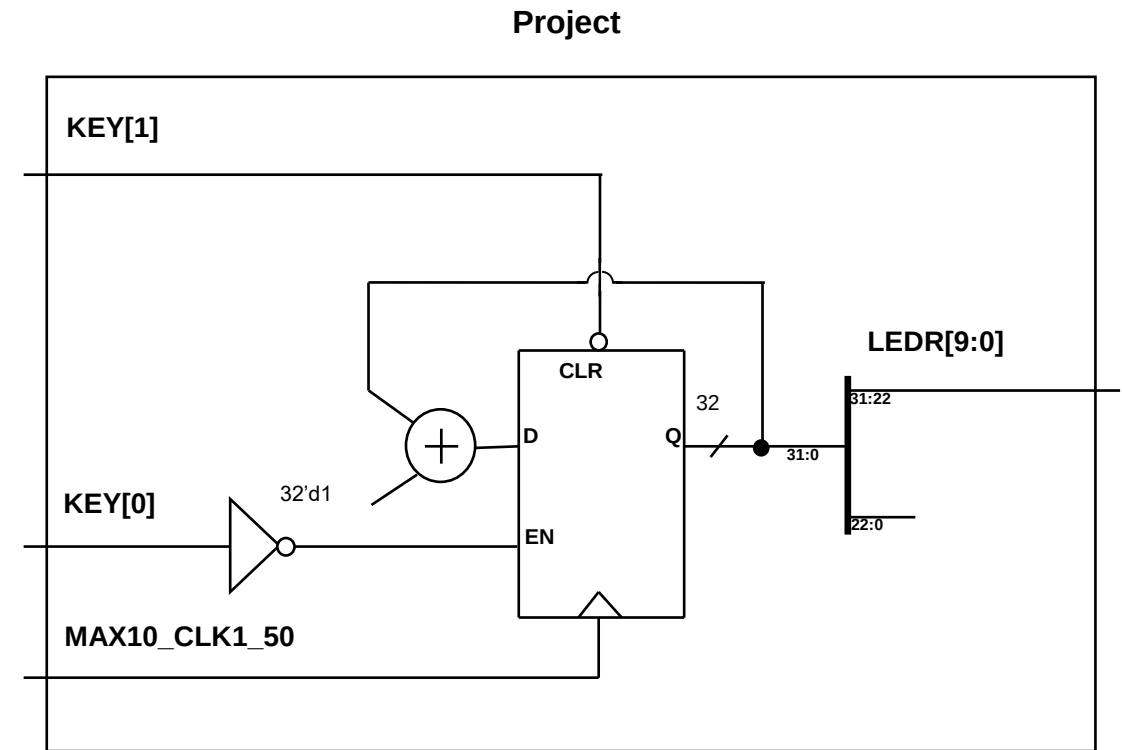
	Nonblocking ($\leq$)	Blocking ($=$)
Assign	assignee $\leq$ assignment	assignee = assignment
Utility	Represent local storage	Represent circuit interconnect
Behavior	Updated at end of current delta cycle (new value not immediately available)	Updated immediately



PROJECT 4

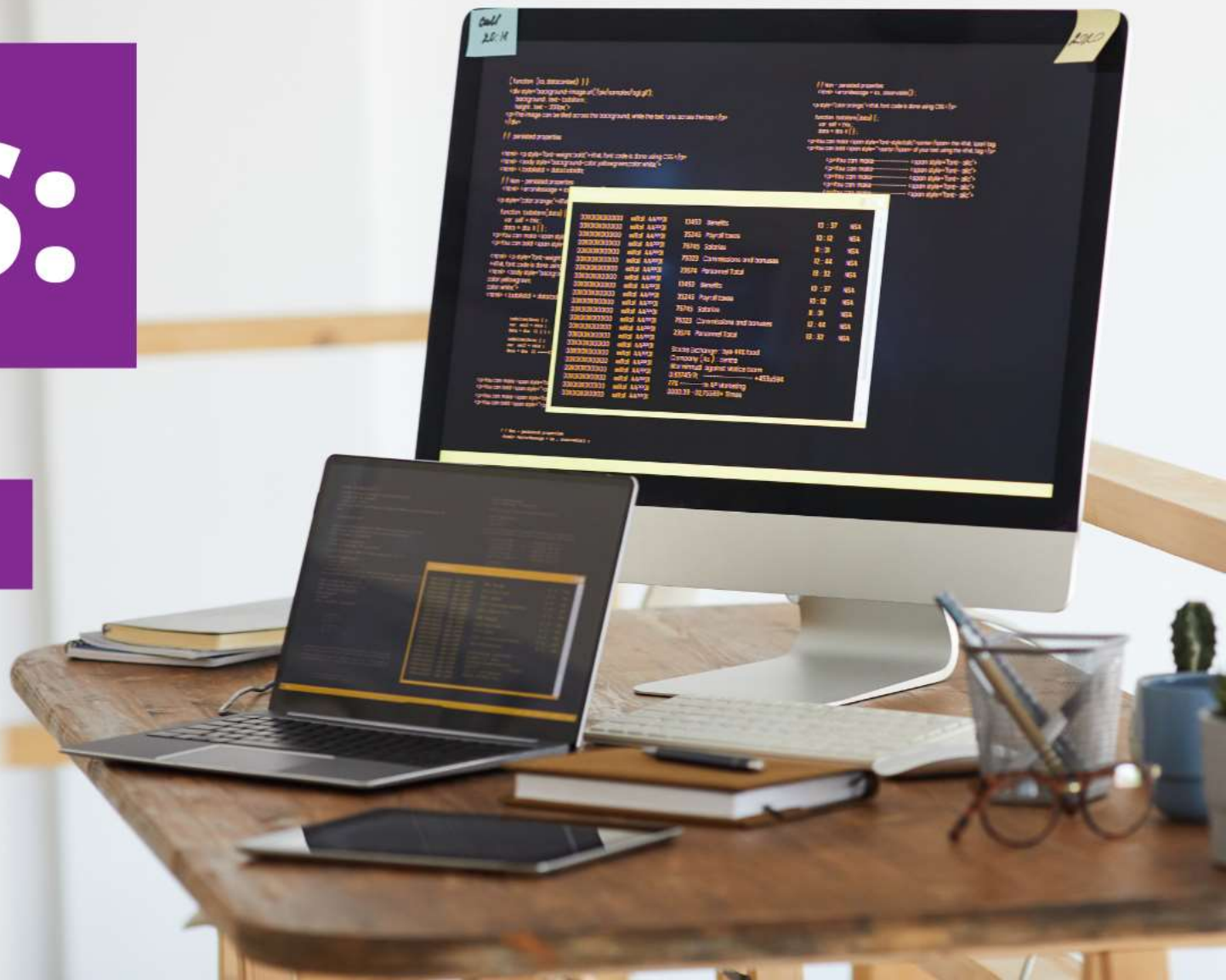
PROJECT 4

- Open projeto-04 and open the Verilog file (projeto.v).
- Add a clock input, two keys and ten LEDs outputs.
- Create a 32-bit counter using a register with an **active low asynchronous reset** signal from the KEY[1] and an **active high enable** signal from the KEY[0].
- Connect the 10 more significative bits from the counter into the LEDs.



QUARTUS:

COMPILATION REPORT



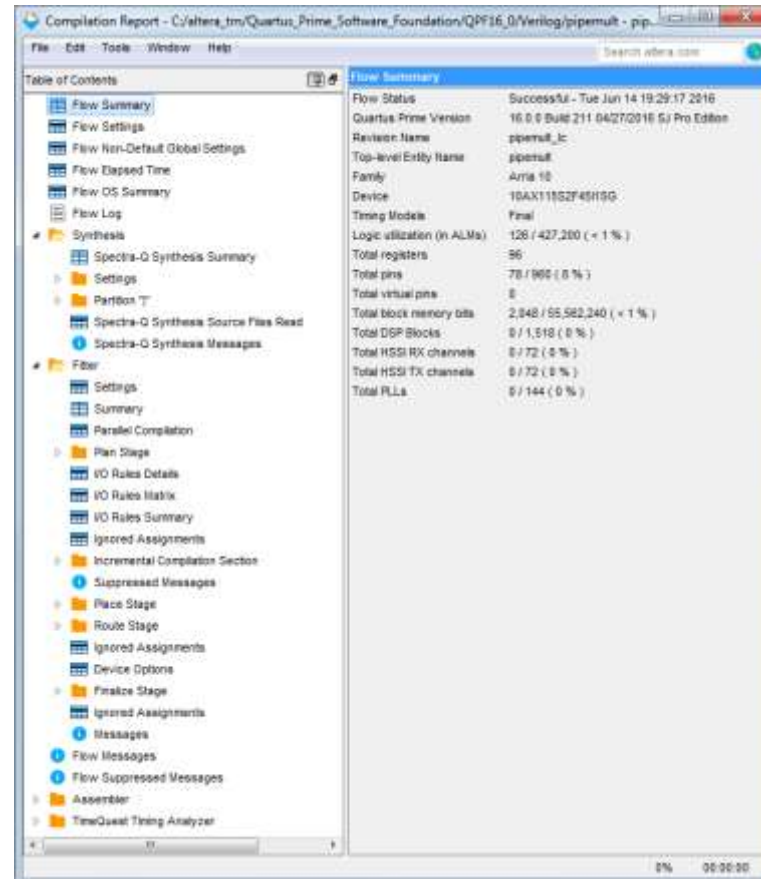
COMPILATION REPORT

- Contains all compilation processing information
 - Resource usage
 - Device pin-out
 - Settings and constraints applied
 - Messages
- Recommendation: Go through report for a design to get sense of information being provided
- Information also available as text files in **output_files** folder in project directory: *<revision_name>.syn.rpt*, *<revision_name>.fit.rpt*, *<revision_name>.fit.plan.rpt*, etc.

COMPILATION REPORT GUI

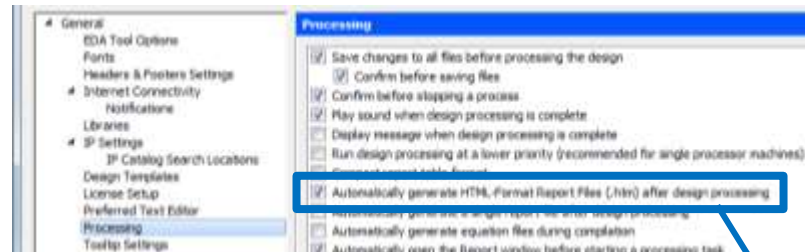
Access from **Processing** menu, toolbar, or Compilation Dashboard

Each compiler process generates separate folder

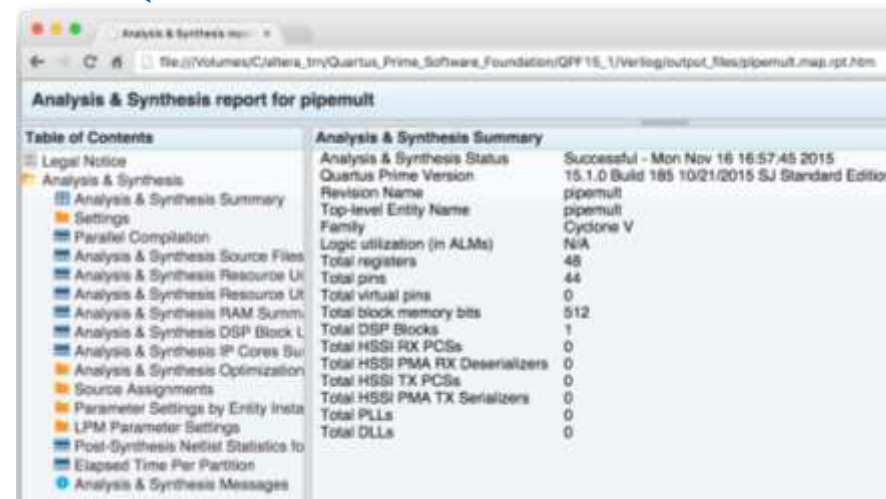


COMPILATION REPORT: HTML VERSION

Tools menu → Options



output_files folder



EXAMPLE: SOURCE FILES READ

List of all design files (user-coded & library) used during last compilation along with files' type and location

Quartus II 64-Bit - C:/altera_trn/Intro/pipemult - pipemult

File Edit View Project Assignments Processing Tools Window Help

Search altera.com

pipemult

Compilation Report - pipemult

Table of Contents

- Flow Summary
- Flow Settings
- Flow Non-Default Global Settings
- Flow Elapsed Time
- Flow OS Summary
- Flow Log
- Analysis & Synthesis
 - Summary
 - Settings
 - Parallel Compilation
 - Source Files Read**
 - Resource Usage Summary
 - Resource Utilization by Entity
 - RAM Summary
 - DSP Block Usage Summary
 - IP Cores Summary
 - Optimization Results
 - Source Assignments
 - Parameter Settings by Entity Instance
 - LPM Parameter Settings
 - Elapsed Time Per Partition
 - Messages
- Fitter
- Assembler

Analysis & Synthesis Source Files Read

	File Name with User-Entered Path	Used in Netlist	File Type	File Name with Absolute Path
1	ram.v	yes	User Wizard-Generated File	C:/altera_trn/Intro/ram.v
2	mult.v	yes	User Wizard-Generated File	C:/altera_trn/Intro/mult.v
3	pipemult.bdf	yes	User Block Diagram/Schematic File	C:/altera_trn/Intro/pipemult.bdf
4	ram.hex	yes	User Hexadecimal (Intel-Format) File	C:/altera_trn/Intro/ram.hex
5	altsyncram.tdf	yes	Megafunction	c:/altera/12.1/quartus/libraries
6	stratix_ram_block.inc	yes	Megafunction	c:/altera/12.1/quartus/libraries
7	lpm_mux.inc	yes	Megafunction	c:/altera/12.1/quartus/libraries
8	lpm_decode.inc	yes	Megafunction	c:/altera/12.1/quartus/libraries
9	aglobal121.inc	yes	Megafunction	c:/altera/12.1/quartus/libraries
10	a_rdenreg.inc	yes	Megafunction	c:/altera/12.1/quartus/libraries
11	altrom.inc	yes	Megafunction	c:/altera/12.1/quartus/libraries
12	altrom.inc	yes	Megafunction	c:/altera/12.1/quartus/libraries
13	altdpram.inc	yes	Megafunction	c:/altera/12.1/quartus/libraries
14	db/altsyncram_psp1.tdf	yes	Auto-Generated Megafunction	C:/altera_trn/Intro/db/altsyncr
15	lpm_mult.tdf	yes	Megafunction	c:/altera/12.1/quartus/libraries
16	lpm_add_sub.inc	yes	Megafunction	c:/altera/12.1/quartus/libraries
17	multcore.inc	yes	Megafunction	c:/altera/12.1/quartus/libraries
18	bypassff.inc	yes	Megafunction	c:/altera/12.1/quartus/libraries
19	altshift.inc	yes	Megafunction	c:/altera/12.1/quartus/libraries
20	db/mult_c9p.tdf	yes	Auto-Generated Megafunction	C:/altera_trn/Intro/db/mult_c9p

100% 00:00:42

EXAMPLE: SOURCE FILES USAGE

- Synthesis resource usage: estimates of FPGA resources required to implement design
- Fitter resource usage: detailed information on all resources used by design for each stage

The screenshot displays the Altera Quartus II software interface, specifically the 'Spectra-Q Synthesis Resource Usage Summary for Partition 'T'' and 'Fitter Resource Usage Summary' windows. The 'Spectra-Q Synthesis' window shows a table of resource usage estimates, while the 'Fitter' window shows a detailed breakdown of resource usage by stage (Plan, Place, Route, Finalize).

Spectra-Q Synthesis Resource Usage Summary for Partition 'T'

Resource	Usage
1 Estimate of Logic utilization (ALMs needed)	121
2	
3 Combinational ALUT usage for logic	219
1 -- 7 input functions	0
2 -- 6 input functions	
3 -- 5 input functions	
4 -- 4 input functions	
5 -- <=3 input functions	
4 Dedicated logic registers	
6	
7 I/O pins	
8 Total MLAB memory bits	
9 Total block memory bits	
10	
11 Total DSP Blocks	
1 -- Total Fixed Point DSP	
2 -- Total Floating Point Q	
12	
13 Maximum fan-out node	
14 Maximum fan-out	
15 Total fan-out	
16 Average fan-out	

Fitter Resource Usage Summary

Resource	Usage	%
1 Logic utilization (ALMs needed / total ALMs on device)	0 / 427,200	0 %
2 ALMs needed (a+b+c)	0	
1 [A] ALMs used in final placement (a+b+c+d)	121 / 427,200	< 1 %
2 [a] ALMs used for LUT logic and registers	22	
3 [b] ALMs used for LUT logic	55	
4 [c] ALMs used for registers	11	
5 [d] ALMs used for memory (up to half of total ALMs)	0	
2 [B] Estimate of ALMs recoverable by dense packing	121 / 427,200	< 1 %
3 [C] Estimate of ALMs unavailable (a+b+c+d)	0 / 427,200	0 %
1 [a] Due to location constrained logic	0	
2 [b] Due to LAB-wide signal conflicts	0	
3 [c] Due to LAB input limits	0	
4 [d] Due to virtual I/Os	0	
5		
4 Difficulty packing design	Low	
5		
6 Total LABs: partially or completely used	23 / 42,720	< 1 %
1 -- Logic LABs	23	
2 -- Memory LABs (up to half of total LABs)	0	
7		
8 Combinational ALUT usage for logic	219	
1 -- 7 input functions	0	
2 -- 6 input functions	0	
3 -- 5 input functions	0	
4 -- 4 input functions	120	
5 -- <=3 input functions	99	
9 Dedicated logic registers	64	
1 -- By type:		
1 -- Primary logic registers	64 / 854,400	< 1 %
2 -- Secondary logic registers	0 / 854,400	0 %

EXAMPLE: TIMING REPORT

- With no clock assignments, the tool creates a default 1 GHz Clock
- This frequency forces the tool to try to optimize the design to achieve this frequency
- Compilation time increases and compilation result is not optimal
- Timing analysis becomes useless

Compilation Report - projeto

Table of Contents

- Flow Suppressed Messages
- Assembler
- TimeQuest Timing Analyzer
 - Summary
 - Parallel Compilation
 - Clocks**
 - Slow 1200mV 85C Model
 - Fmax Summary**
 - Setup Summary**
 - Hold Summary

Clocks

	Clock Name	Type	Period	Frequency	Rise	Fall
1	MAX10_CLK1_50	Base	1.000	1000.0 MHz	0.000	0.500

Slow 1200mV 85C Model Fmax Summary

	Fmax	Restricted Fmax	Clock Name	
1	301.57 MHz	250.0 MHz	MAX10_CLK1_50	limit due to minimum

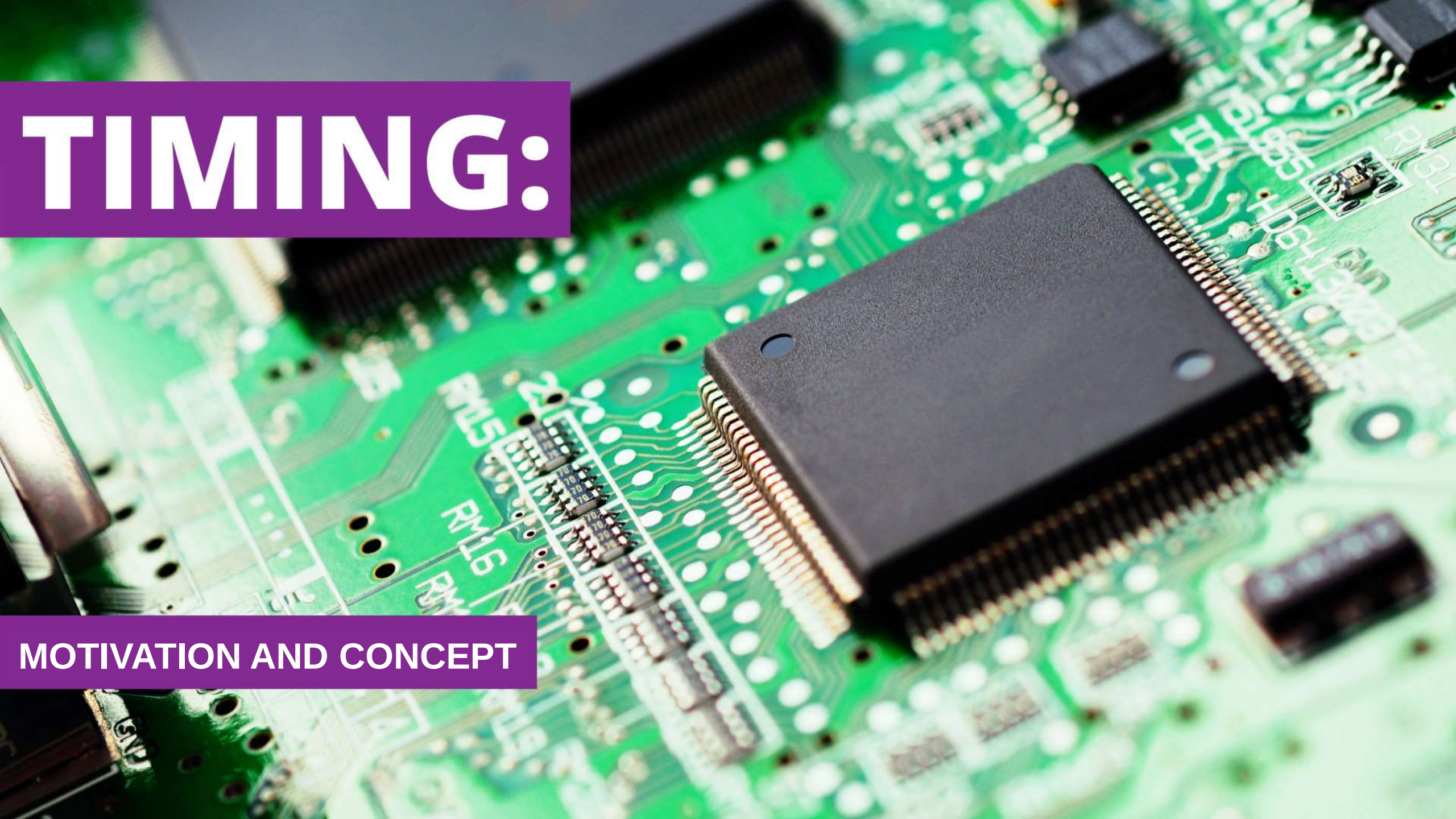
Slow 1200mV 85C Model Setup Summary

	Clock	Slack	End Point TNS
1	MAX10_CLK1_50	-2.316	-50.816

Messages

Type	ID	Message
Warning	332012	Synopsys Design Constraints File file not found: 'projeto.sdc'. A Syno
Warning	332012	Synopsys Design Constraints File file not found: 'projeto.sdc'. A Syno
Error	332148	Timing requirements not met
Error	332148	Timing requirements not met
Error	332148	Timing requirements not met

TIMING:

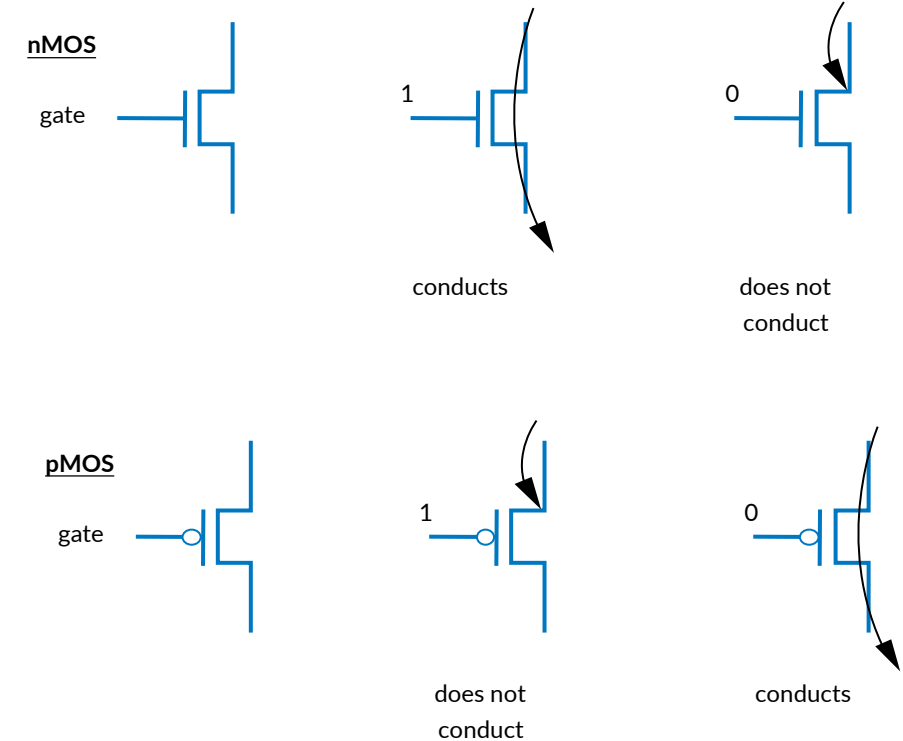
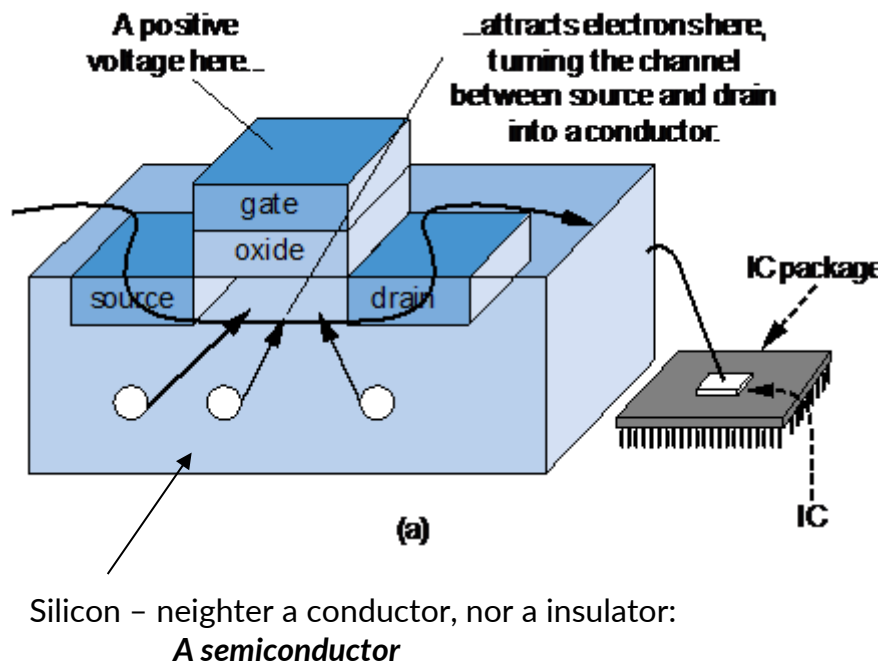


MOTIVATION AND CONCEPT

MICROELECTRONICS – TRANSISTOR

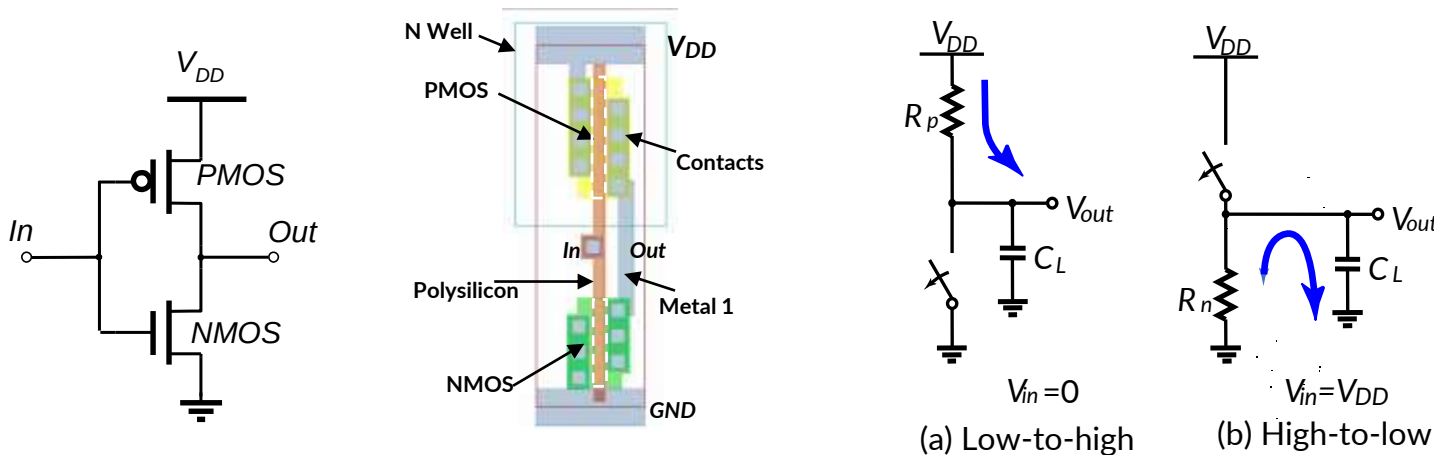
➤ Transistor CMOS

- A basic “switch” used in the current Integrated Circuits (ICs)



MICROELECTRONICS – LOGIC GATES IMPLEMENTATION

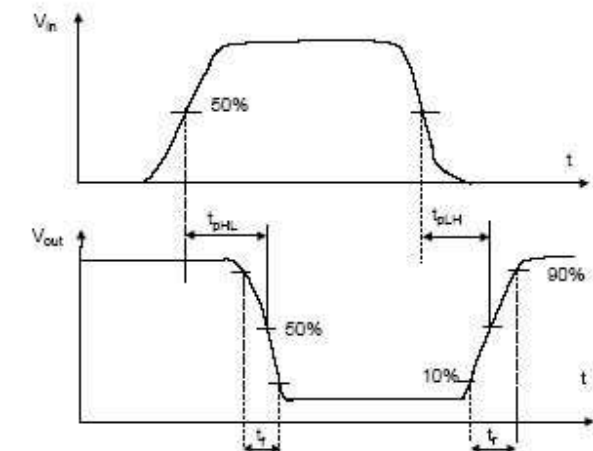
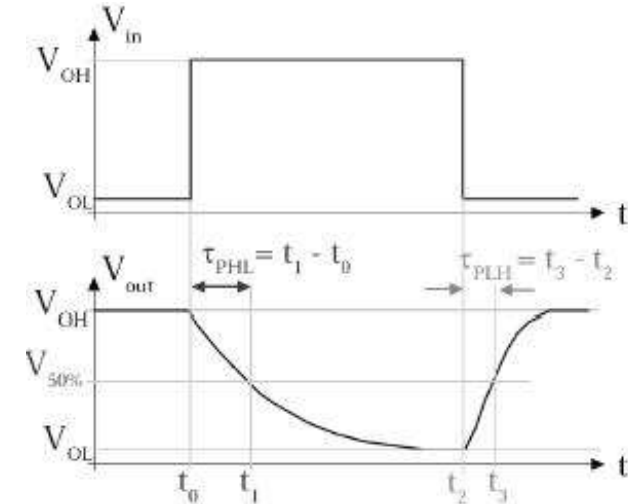
- Currently the CMOS technology is used (Complementary, PMOS+NMOS) to implement physically of general purpose logic gates
- Inverter example:
 - The PMOS transistor conducts from V_{DD} to V_{out} when a low voltage is applied to the gate
 - The NMOS transistor conducts from GND to V_{out} when a high voltage is applied to the gate



MICROELECTRONICS – PHYSICAL IMPLEMENTATION IMPLICATIONS

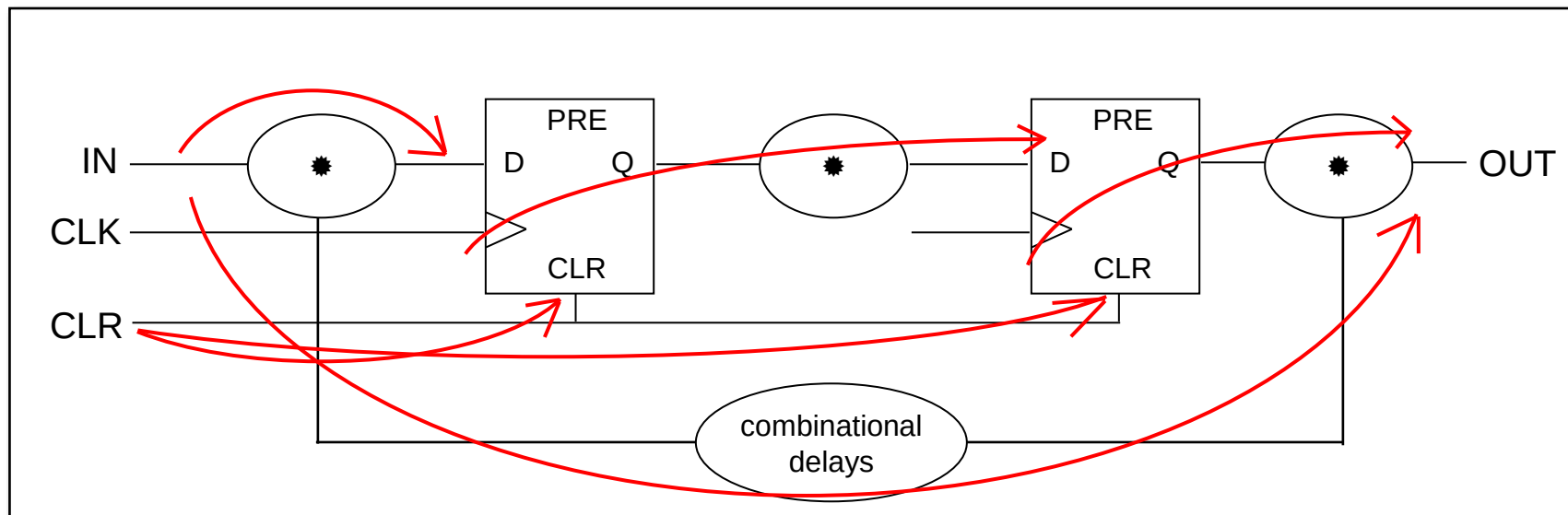
■ Inverter propagation time (delay)

- When the input is a step (ideal) the output is a slope.
- The signal curve depends on the inverter's technical features, the working principle, and the load on the output.
- t_{pHL} and t_{pLH} are the propagation time in the signal transition from 1 to 0 and 0 to 1 respectively.
- The propagation time determines the maximum working frequency a synchronous circuit can have.

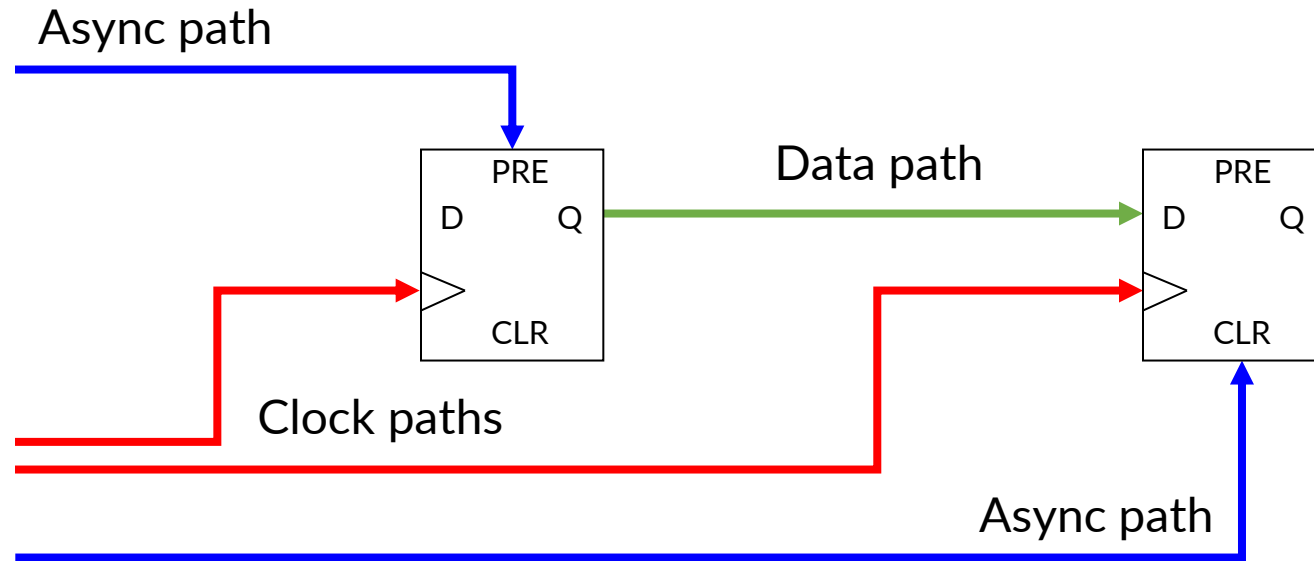


HOW DOES TIMING VERIFICATION WORK?

- Every device path in design must be analyzed with respect to timing specifications/requirements
 - Catch timing-related errors faster and easier than gate-level simulation & board testing
- Designer must enter timing requirements & exceptions
 - Used to guide Fitter during placement & routing
 - Used to compare against actual results (post-fit)



PATH & ANALYSIS TYPES



Three types of Paths:

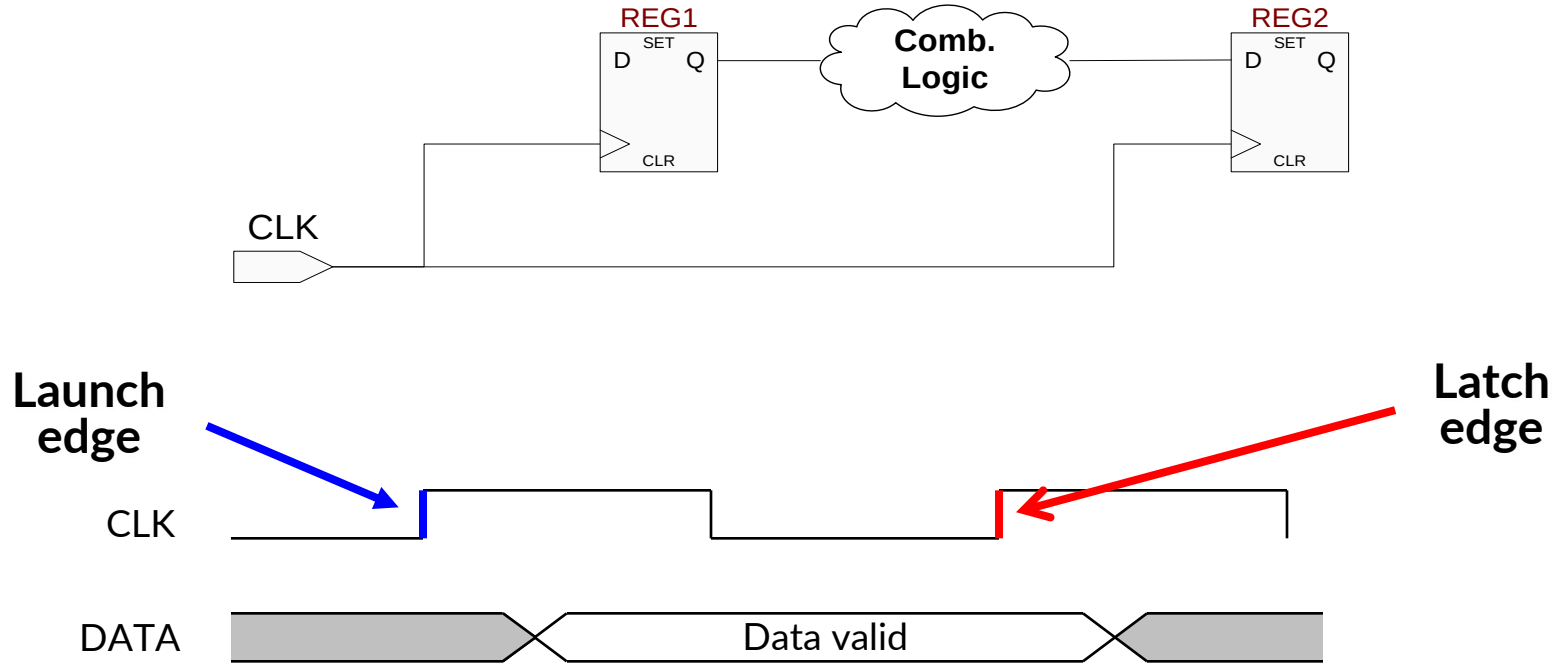
1. Clock paths
2. Data path
3. Asynchronous paths*

Two types of Analysis:

1. Synchronous: clock & data paths
2. Asynchronous*: clock & async paths

**Asynchronous refers to signals feeding the asynchronous control ports of the registers*

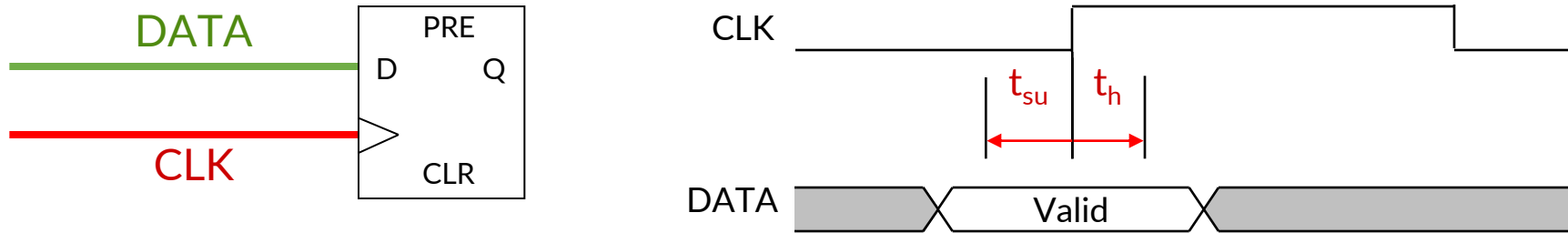
LAUNCH & LATCH EDGES



Launch edge: the edge which “launches” the data from source register

Latch edge: the edge which “latches” the data at destination register (with respect to the launch edge, selected by timing analyzer; typically 1 cycle)

SETUP & HOLD



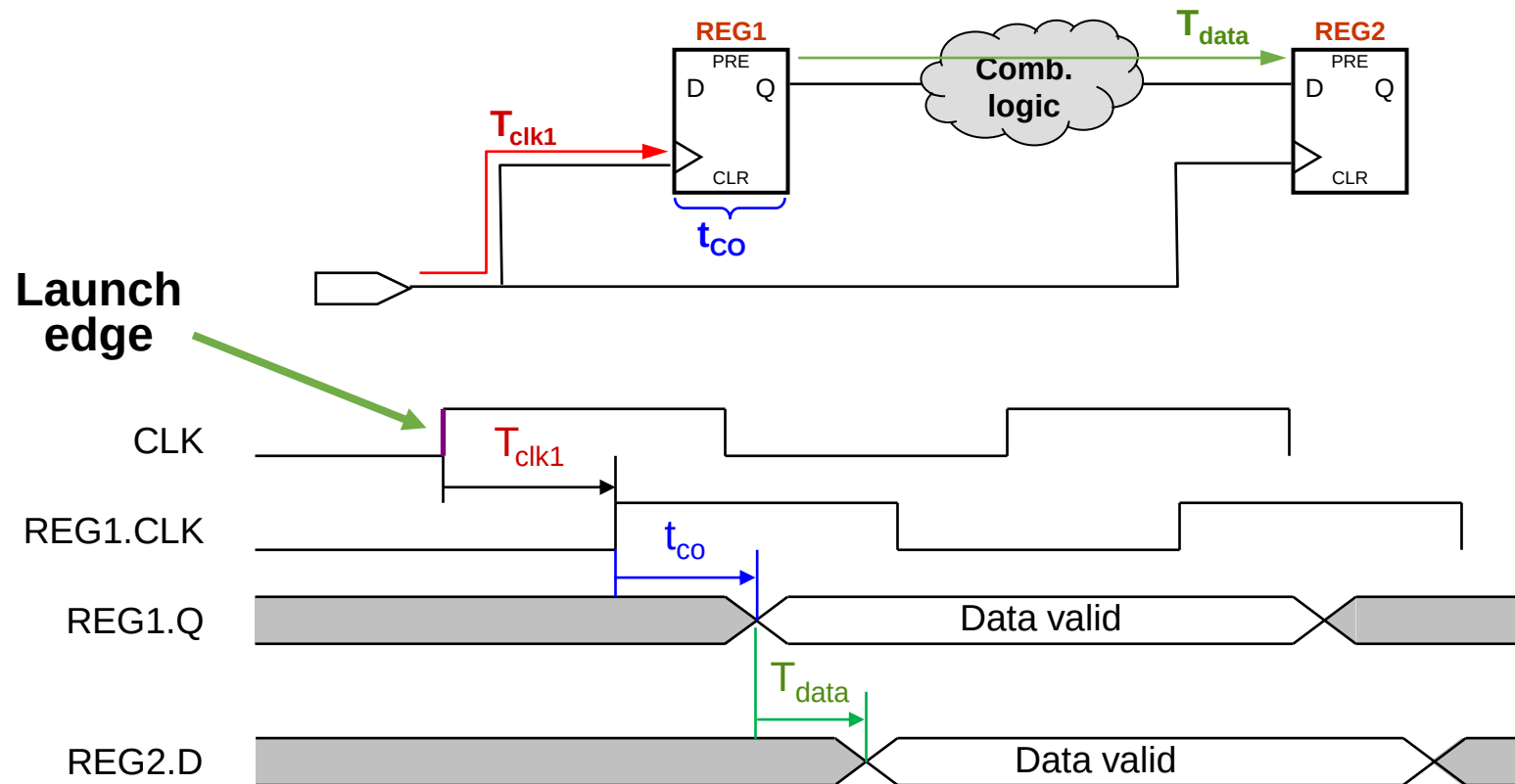
Setup: The minimum time data signal must be stable
BEFORE clock edge

Hold: The minimum time data signal must remain stable
AFTER clock edge

*Together, the setup time and hold time form a Data Required Window,
the time around a clock edge in which data must be stable.*

DATA ARRIVAL TIME

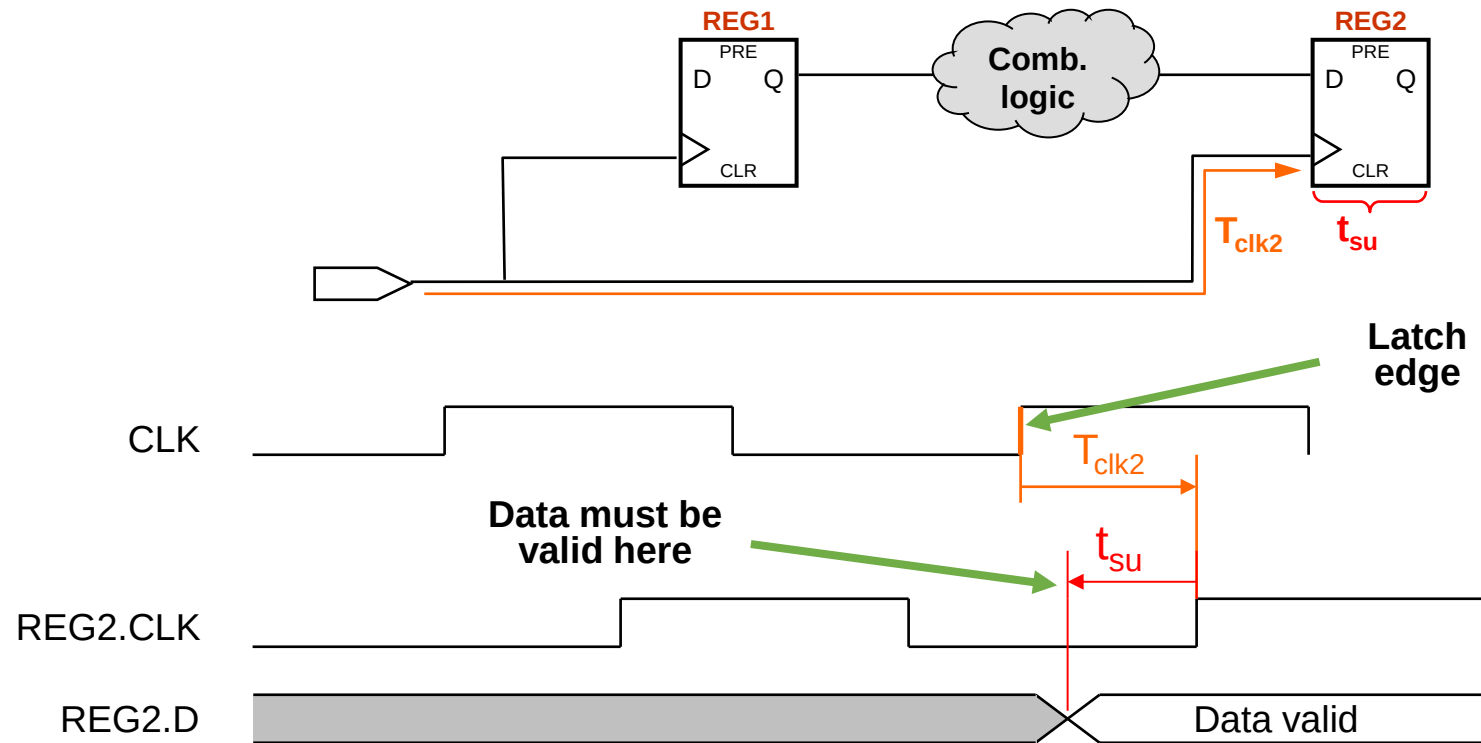
The time for data to arrive at destination register's D input



$$\text{Data arrival time} = \text{launch edge} + T_{clk1} + t_{co} + T_{data}$$

DATA REQUIRED TIME (SETUP)

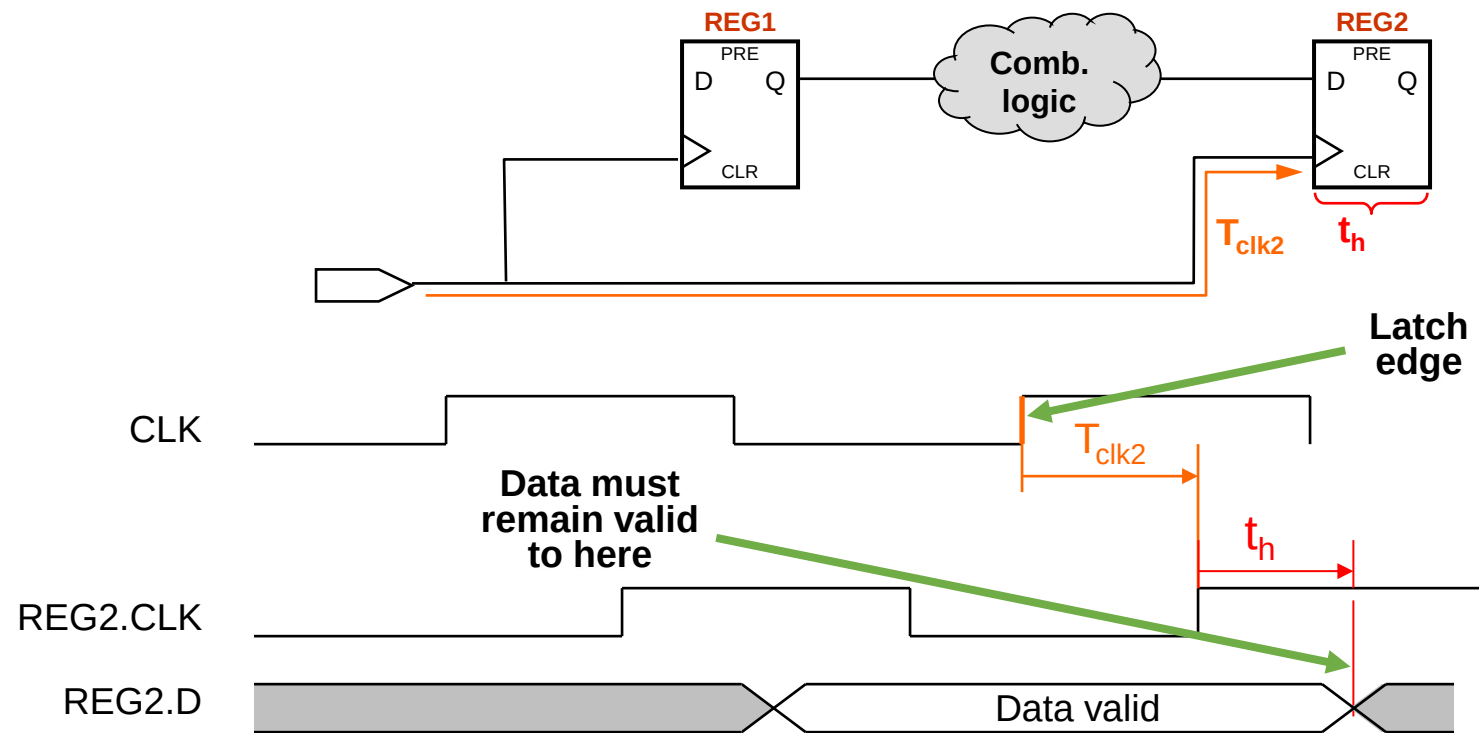
The minimum time required before the latch edge for data to get latched into the destination register.



$$\text{Data required time (Setup)} = \text{Clock arrival time} - t_{su} - \text{Setup uncertainty}$$

DATA REQUIRED TIME (HOLD)

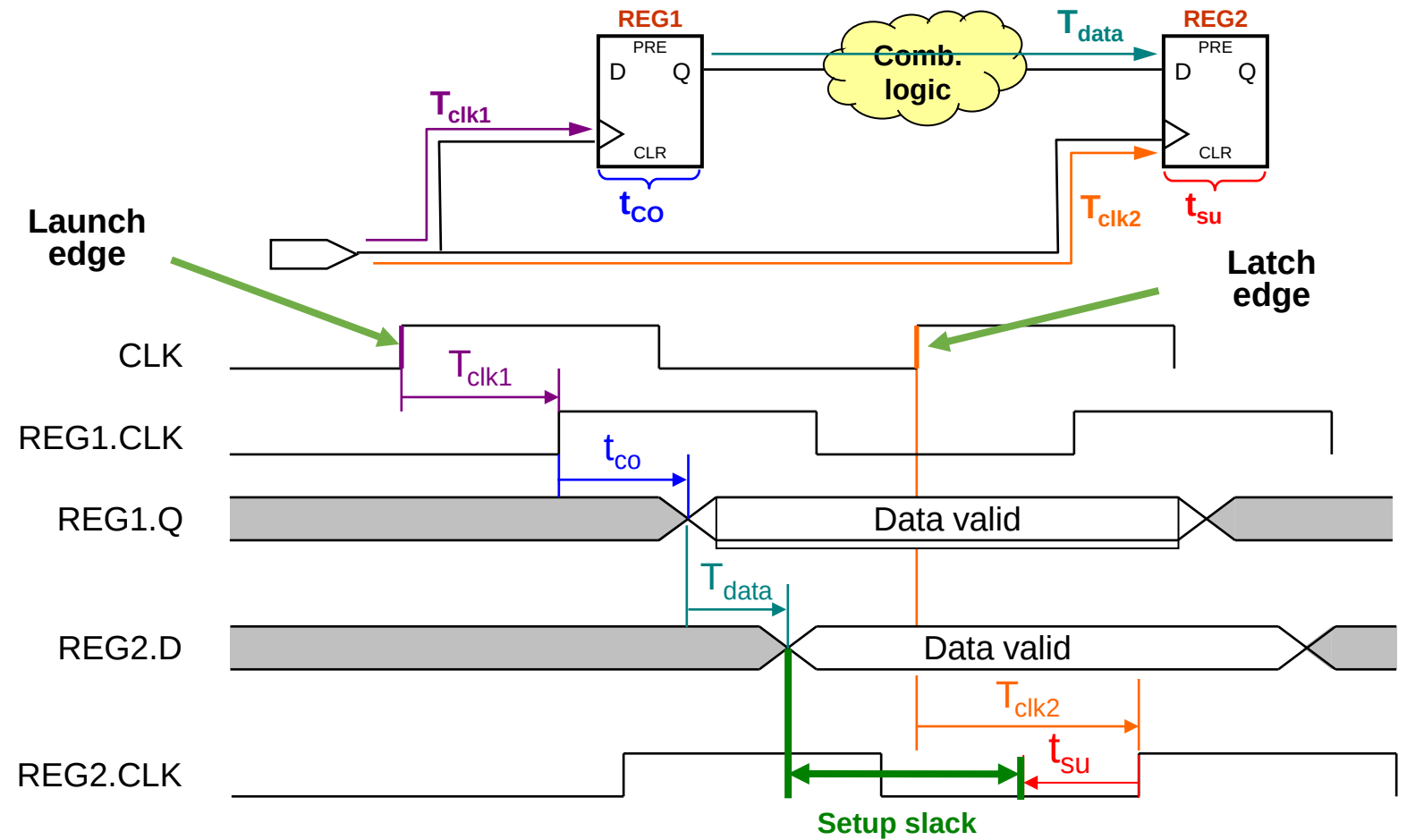
The minimum time required after the latch edge for the data to remain valid for successful latching into the destination register



$$\text{Data required time (Hold)} = \text{Clock arrival time} + t_h + \text{Hold uncertainty}$$

SETUP SLACK

- Margin by which the setup timing requirement is met
- Ensures launched data arrives in time to meet the latching requirement



SETUP SLACK (CONT'D)

$$\text{Setup slack} = \text{Minimum Data Required Time (Setup)} \\ - \text{Maximum Data Arrival Time}$$

Positive slack

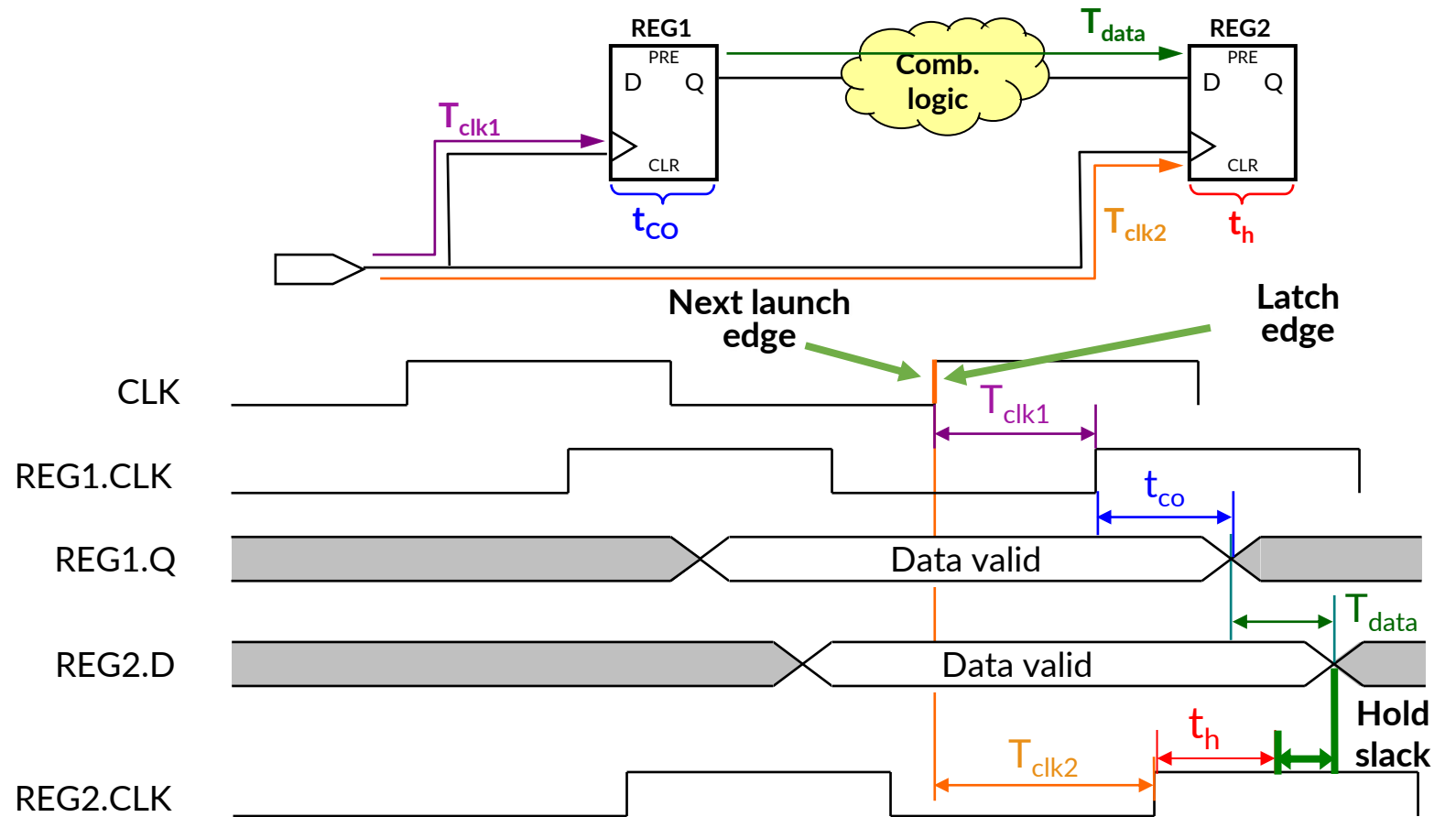
- Timing requirement met

Negative slack

- Timing requirement not met

HOLD SLACK

- Margin by which the hold timing requirement is met
- Ensures latch data is not corrupted by data from another launch edge and prevents “double-clocking”



HOLD SLACK (CONT'D)

*Hold slack = Minimum Data Arrival Time –
Maximum Data Required Time (Hold)*

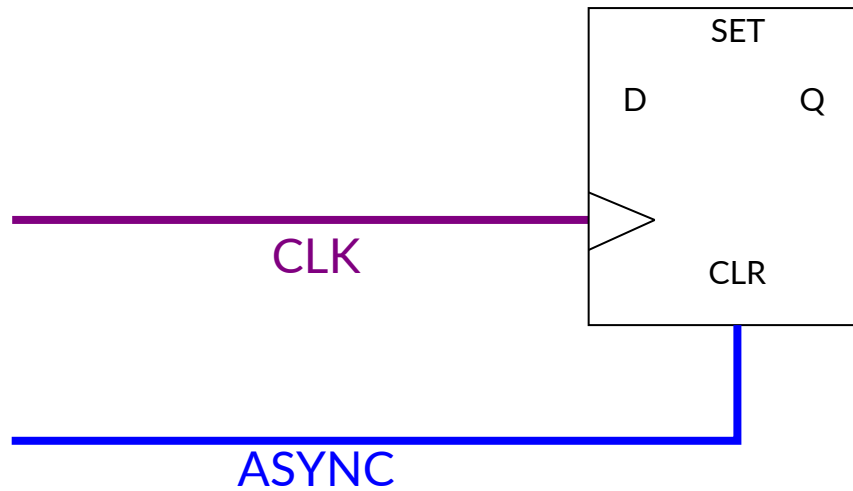
Positive slack

- Timing requirement met

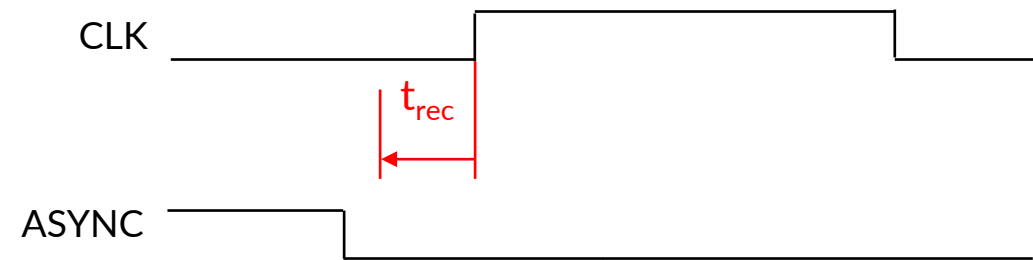
Negative slack

- Timing requirement not met

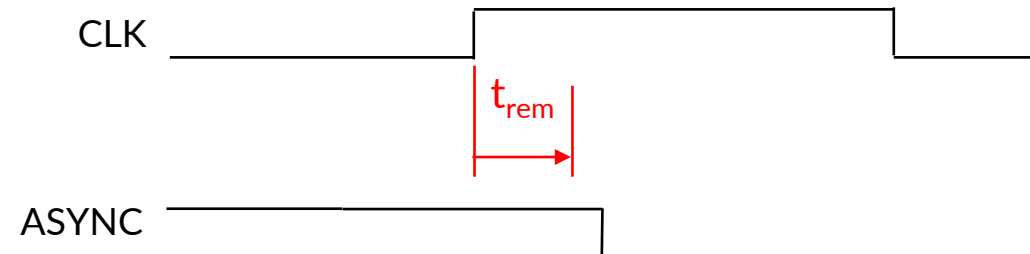
RECOVERY & REMOVAL



Recovery: Minimum time an asynchronous signal must be de-asserted BEFORE clock edge

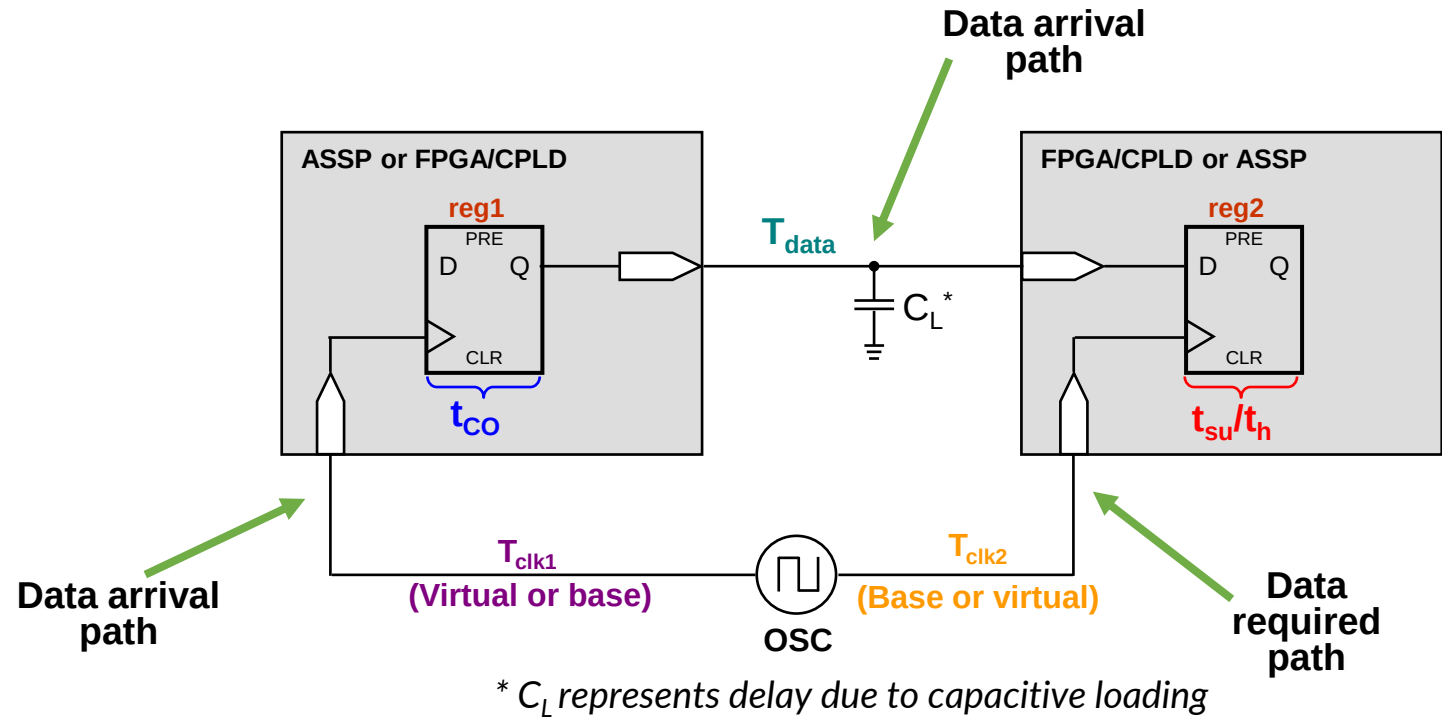


Removal: Minimum time an asynchronous signal must be de-asserted AFTER clock edge



I/O ANALYSIS (COMMON CLOCK SOURCE)

- Analyze I/O performance in a synchronous design using the same slack equations
 - Must include external device & PCB timing parameters
 - Use *virtual clock* for specifying input/output delays
 - Otherwise, difficult to accurately constrain I/Os, especially for source synchronous

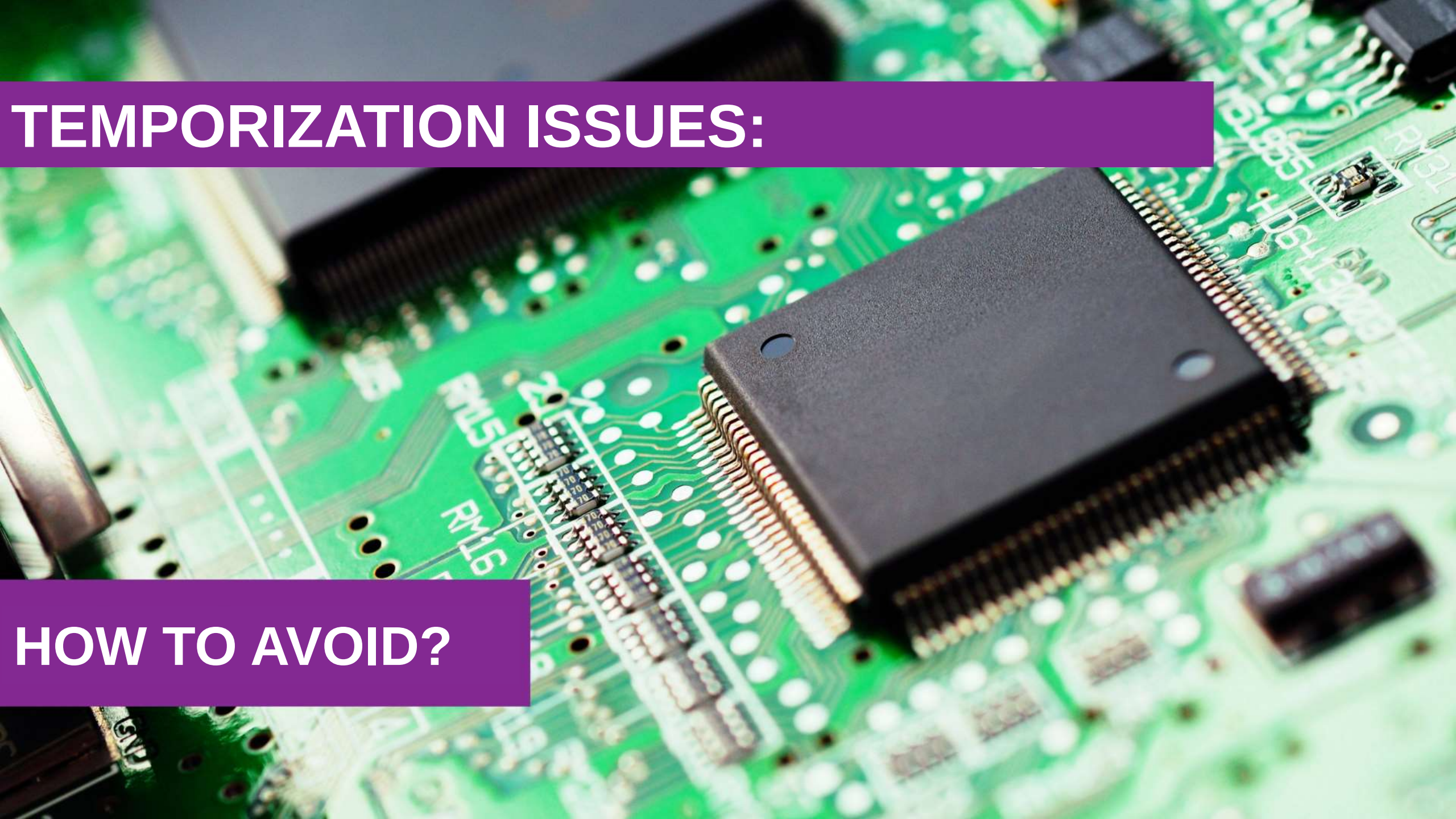


WHY ARE THESE CALCULATIONS IMPORTANT?

- Calculations are important when timing violations occur
 - Need to be able to understand cause of violation
- Example causes
 - Data path too long
 - Timing requirement too short (incorrect analysis)
 - Large clock skew signifying a gated clock, etc.
- TimeQuest timing analyzer uses them
 - Equations used to calculate slack
 - Terminology (launch and latch edges, data arrival path, data required path, etc.)
 - found in timing reports

TIMING MODELS IN DETAIL

- Quartus software models device timing at multiple process voltage temperature (PVT) conditions by default
 - **Slow corner** model
 - Indicates slowest possible performance for any single path
 - Timing for slowest device at maximum operating temperature and VCCMIN
 - **Fast corner** model
 - Indicates fastest possible performance for any single path
 - Timing for fastest device at minimum operating temperature and VCCMAX
 - **2nd slow and 2nd fast** models (temperature inversion phenomenon)
 - Slow timing at minimum operating temperature
 - Fast timing at maximum operating temperature
- Why analyze for multiple corner timing models?
 - Ensure **setup** timing is met in the **slow** models
 - Ensure **hold** timing is met in **fast** model
 - Essential for source synchronous interfaces
- Read the following white paper for more information
 - <https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/wp/wp-01139-timing-model.pdf>



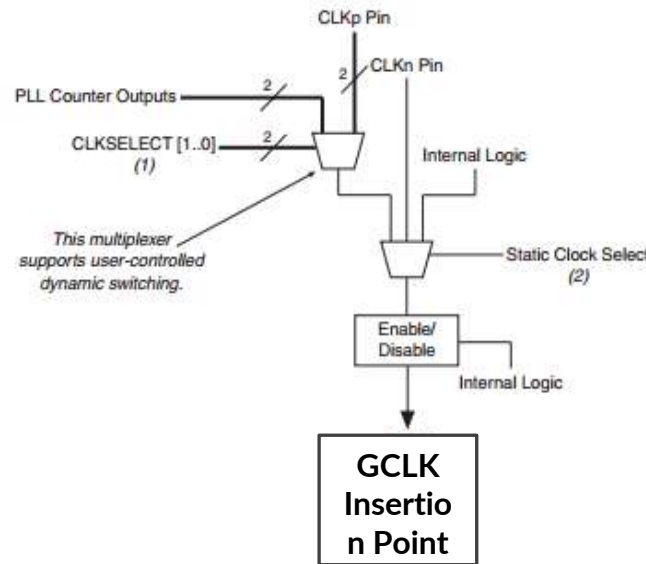
TEMPORIZATION ISSUES:

HOW TO AVOID?

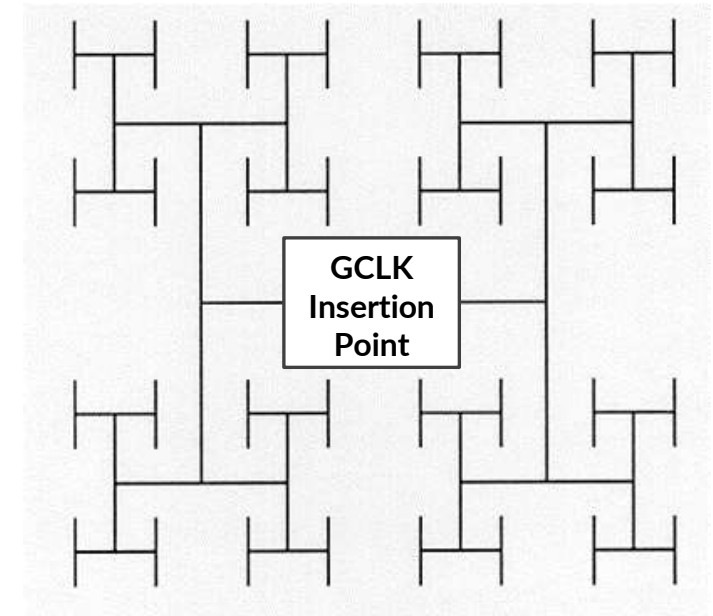
CLOCK ROUTING

- Dedicated clock routing minimizes clock skew
- Propagation delay will be higher due to buffers
- PLL Outputs, Clock Input Pins, and internal logic are routed to the Clock Control Block
- Clock control output is routed to the Clock Tree Insertion Point

CLKCTRL - Clock Control Block

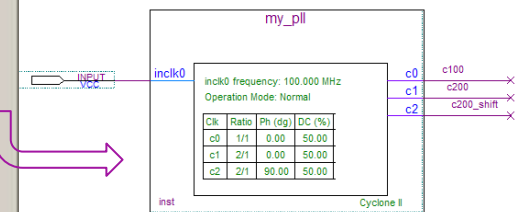
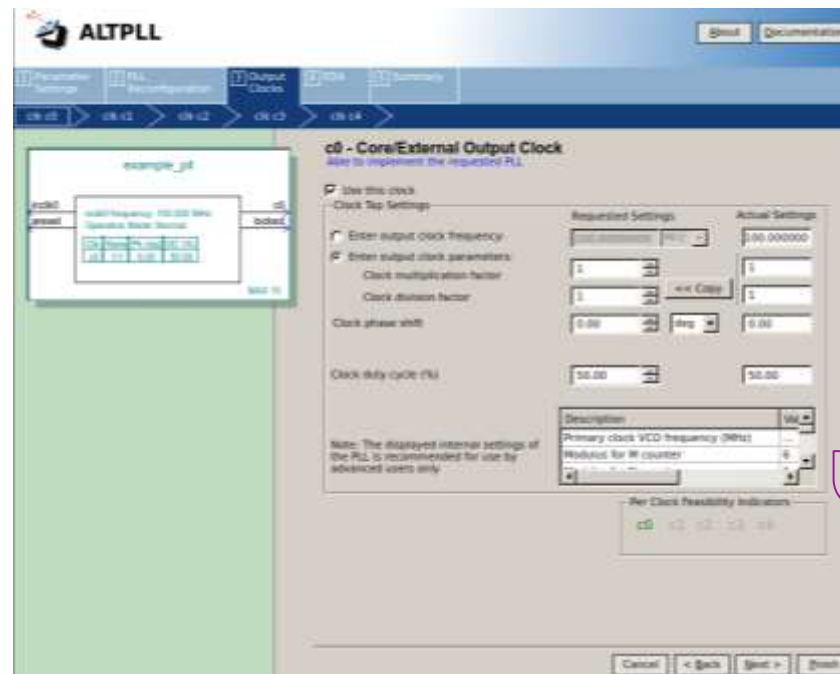
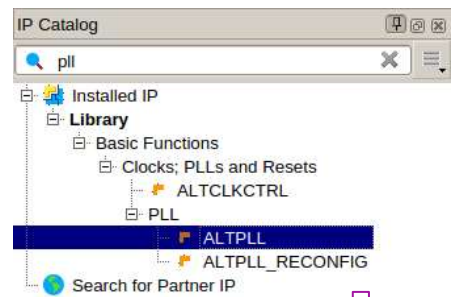


GCLK - Global Clock



PLLs

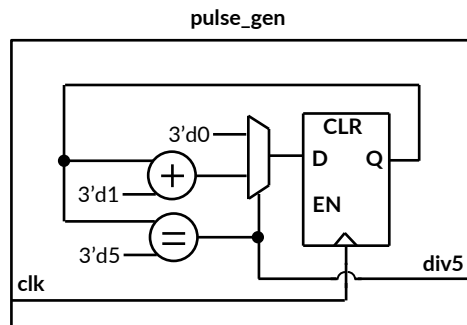
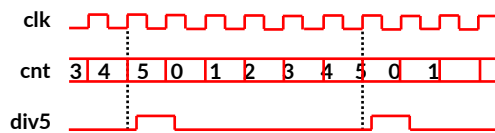
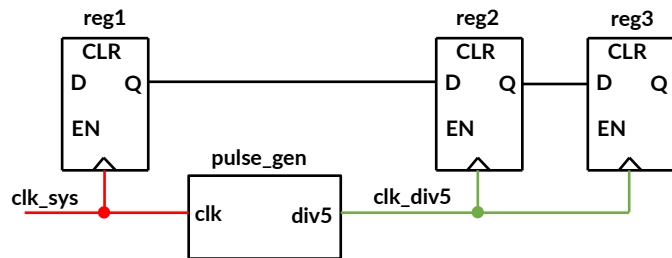
- Use PLLs to generate different clock frequencies
 - PLL outputs are connected directly to a clock control block and global routing
- Use a synchronized reset signal for each clock domain (discussed later)
- Avoid using multiple clock, using the clock enable feature when possible (discussed next)



CLOCK ENABLE

➤ Avoid pulsed clocks, use enable instead

**div5 used as pulsed clock
NOT RECOMMENDED**

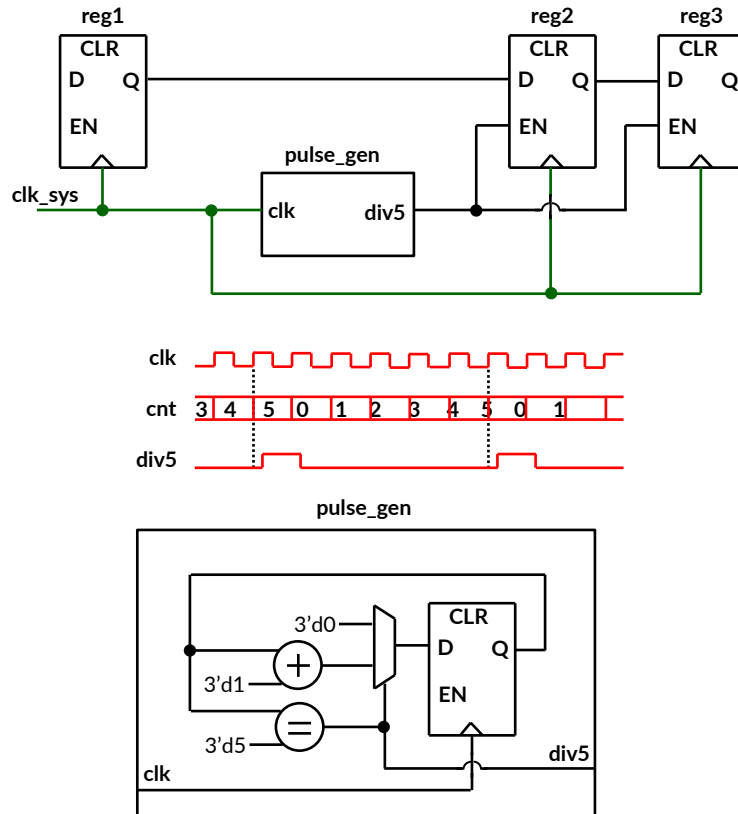


- Clock domain crossing between reg1 and reg2 needs a synchronizer
- If clk_div5 does not use global routing:
 - Timing exception must be defined in the SDC file due to small phase shift from clk_sys to clk_div5
 - Clock skew between reg2 and reg3 can be high (harder for the fitter to close timing)
- if clk_div5 uses global routing:
 - Assignment may be necessary to force clk_div5 to use global routing
 - Huge insertion delay in clk_div5 may cause cross clock domain issues
 - Waste of global resources (if clk_div5 has low fanout)

CLOCK ENABLE

Avoid pulsed clocks, use enable instead

div5 used as clock enable
RECOMMENDED



- There is no clock domain crossing
 - High clock skew issues avoided
 - No synchronization chain needed
 - No additional global resources used
 - No additional SDC command necessary (unless there is some multicycle relationship, then it would be mandatory for any approach)
- Generated div5 signal may have a “skew” as high as one clock period without violating setup or hold

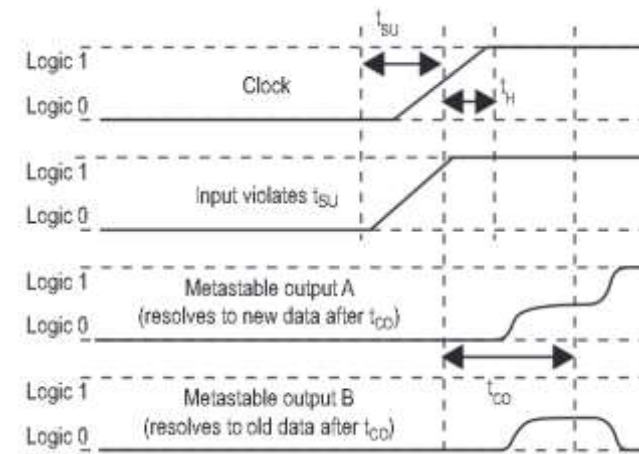
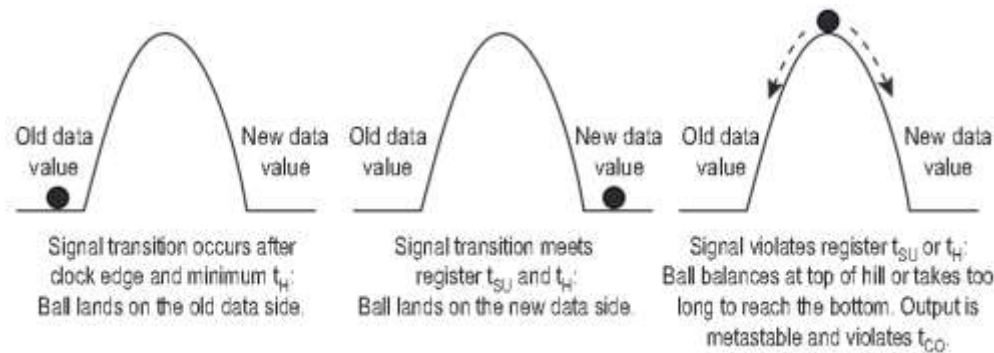


METASTABILITY

WHAT IS IT, WHEN IT OCCURS, HOW TO AVOID?

METASTABILITY DEFINITION

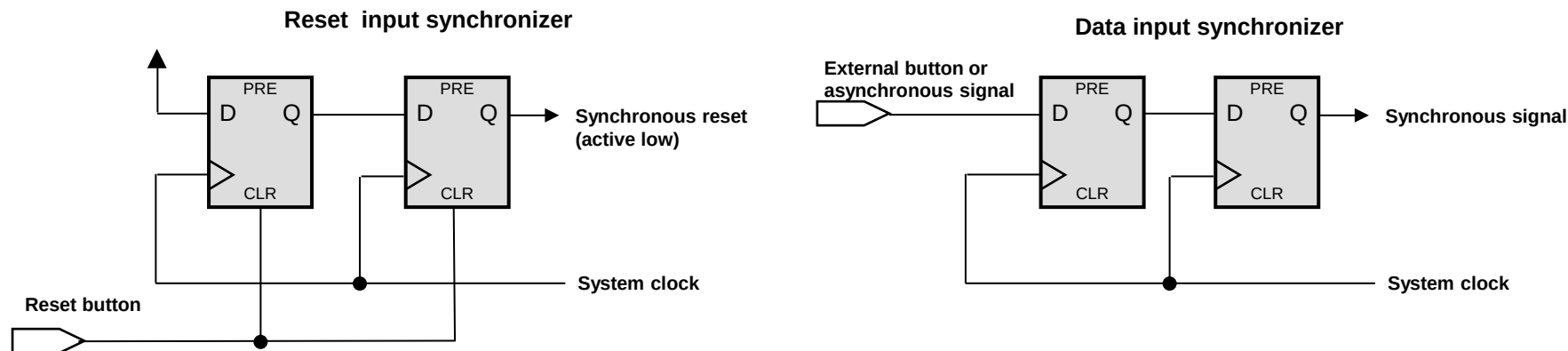
- A metastable state is a internal register state in which it's output voltage does not represent a logic low neither a logic high
- The register enters this state when any input changes within the setup and hold time window
- When a register is in metastable state, it will occasionally settle down to either the old input or the new input
- The amount of time it takes for the output to settle is not deterministic and follows a normal distribution



<https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/wp/wp-01082-quartus-ii-metastability.pdf>

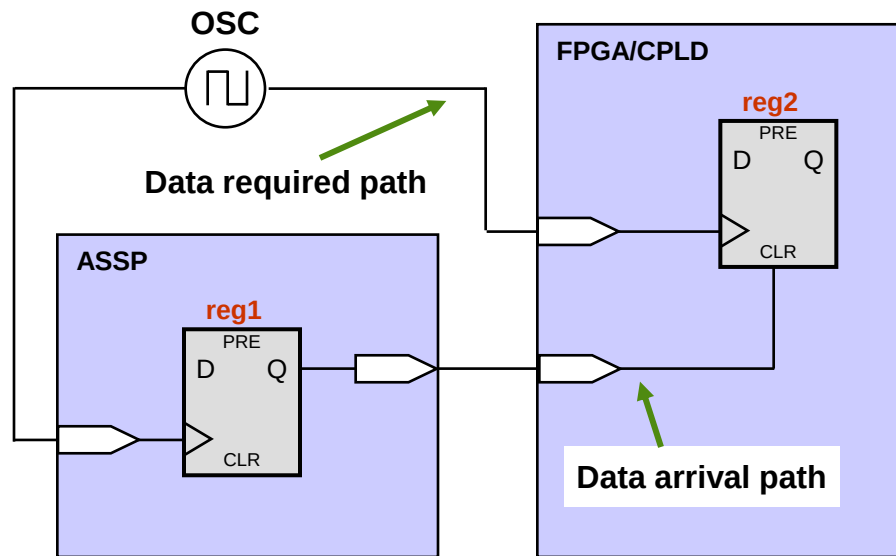
HOW TO AVOID METASTABILITY

- Make sure that all combinational logic within the same clock domain meets timing: combinational + routing delay is shorter than the clock period
- Add synchronizers in asynchronous inputs like buttons and serial interfaces like UART
- Add synchronizers for single bit data crossing clock domains
- Use dual clock FIFOs for buses crossing clock domain
- Synchronize all reset signals using the same clock domain as the register

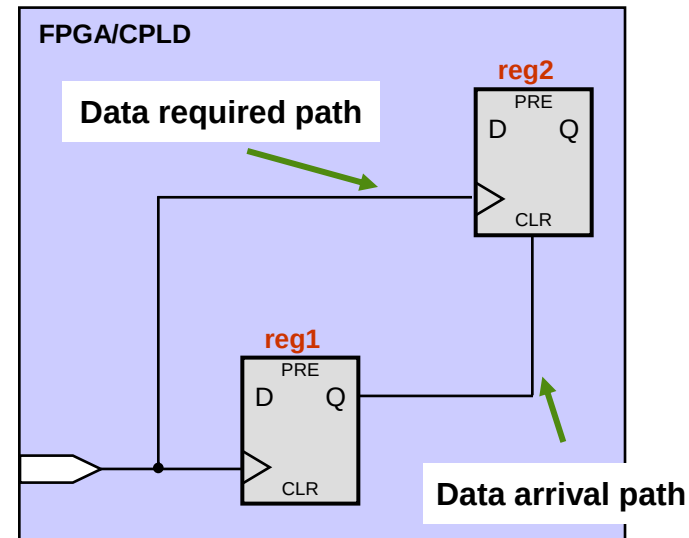


ASYNCHRONOUS = SYNCHRONOUS?

- Asynchronous control signal source is assumed synchronous
 - Slack equations still apply
 - data arrival path = asynchronous control path
 - $t_{su} \approx t_{rec}$; $t_h \approx t_{rem}$
 - External device & board timing parameters may be needed



Externally registered

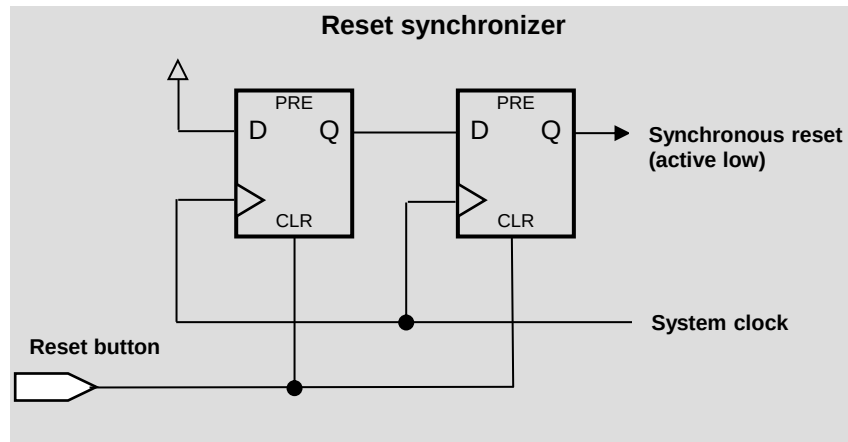


Internally registered

WHAT ABOUT TRULY ASYNCHRONOUS CONTROL INPUTS?

BAD IDEA to use it directly!!!!

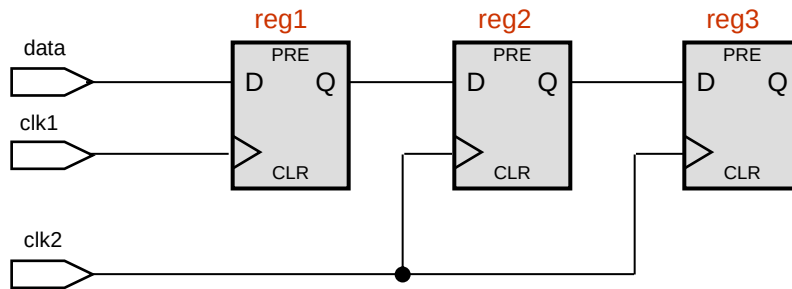
- Solution: synchronize inputs with internal clock
 - Input may then become false path (discussed in next section)



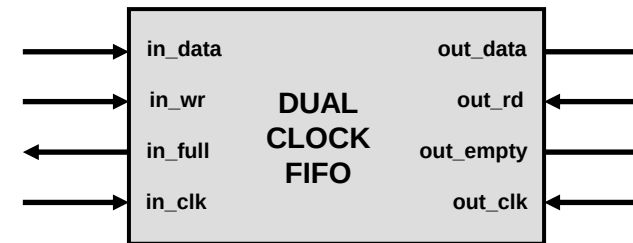
- But if you must...
 - Use false path to remove from timing analysis

SYNCHRONIZERS INSIDE THE DESIGN

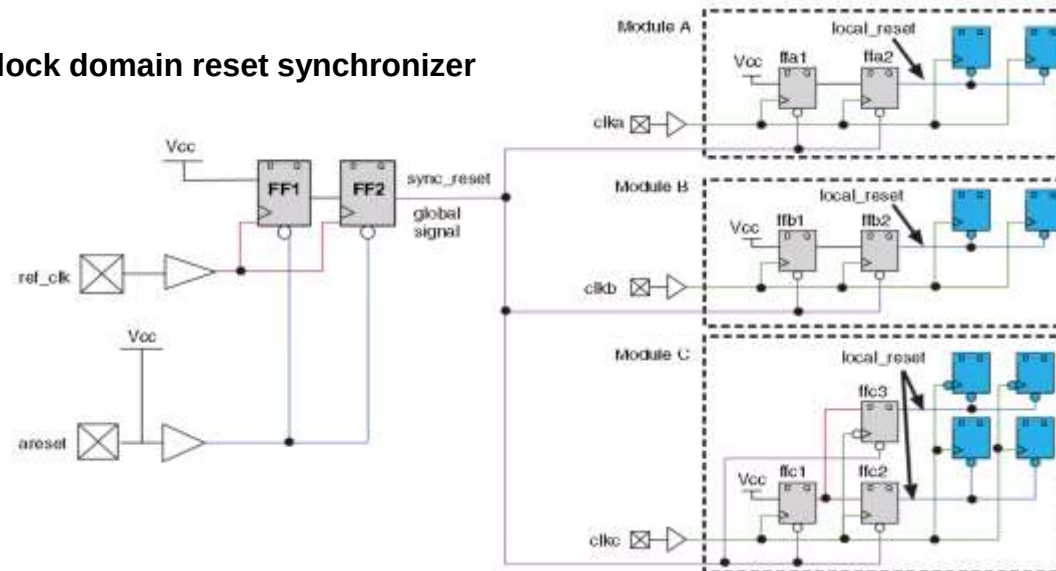
Clock domain crossing synchronizer (single bit)



Clock domain crossing synchronizer (word/bus)

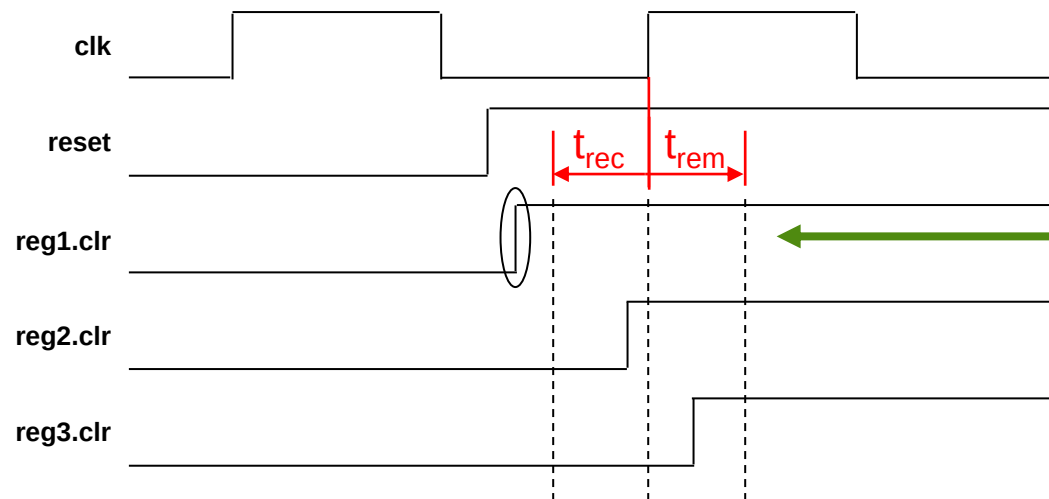
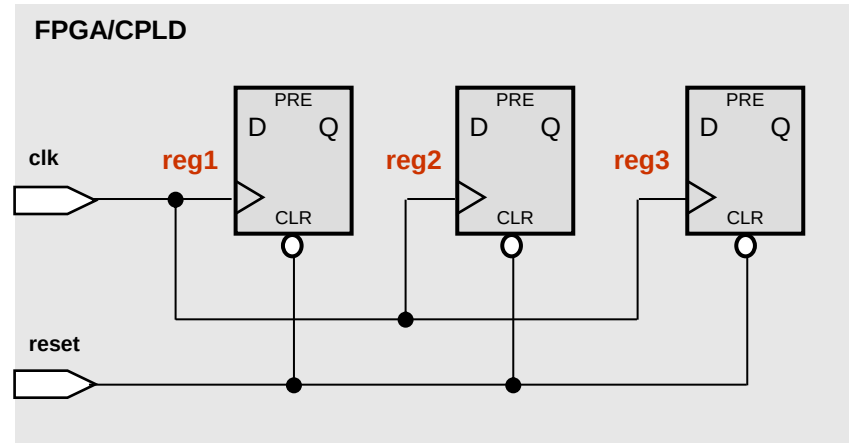


Multiple clock domain reset synchronizer



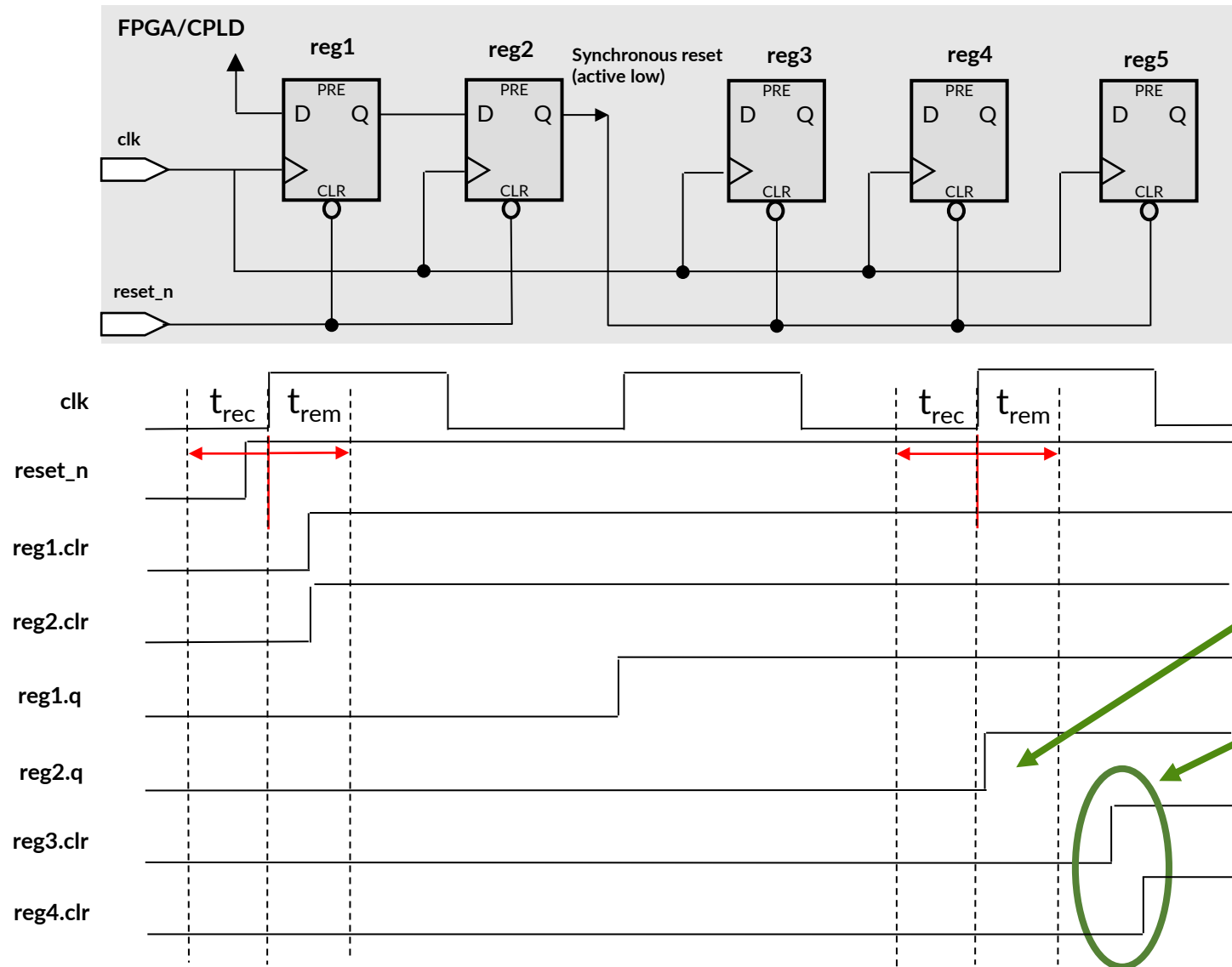
NEED MORE PROOF?

For example, these state machine registers should all be de-asserted together, but...



Due to routing delay (skew) of the clear signal, only reg1 comes out of reset correctly, the rest may de-assert on the next clock cycle. This could mean starting in the wrong state (or even an illegal state).

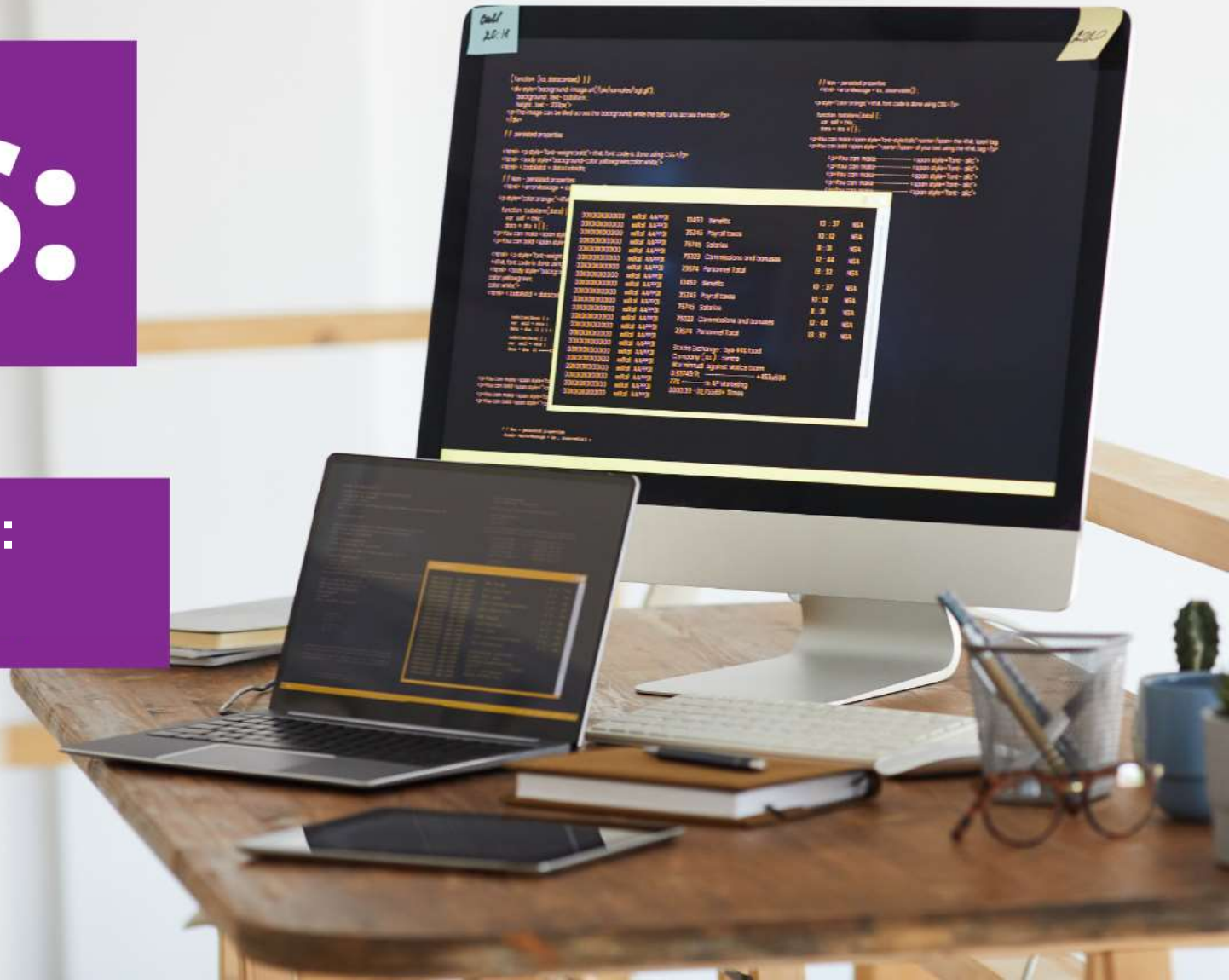
CORRECT METHOD: SYNCHRONIZE RESET



The clear signal is the output of a flip-flop, therefore the tools can ensure that the clear signals meet recovery and removal requirements.

QUARTUS:

TIMING RESTRICTION DEFINITIONS: DESIGNING A BASIC SDC



IMPORTANCE OF CONSTRAINING

- Timing analysis tells how a circuit **WILL** behave
- Providing timing constraints tells tools how you **WANT** the design to behave
 - Constraints paint picture of how design should operate
 - Based on design specs & specs from other devices on PCB
 - Provide goals for Fitter to target during compilation
 - Provide values for comparing with timing results
- TimeQuest timing analyzer performs limited analysis without timing constraints

SDC FILE EDITOR = Quartus TEXT EDITOR

TimeQuest File menu → Open/New SDC File
Quartus File menu → New → Other Files

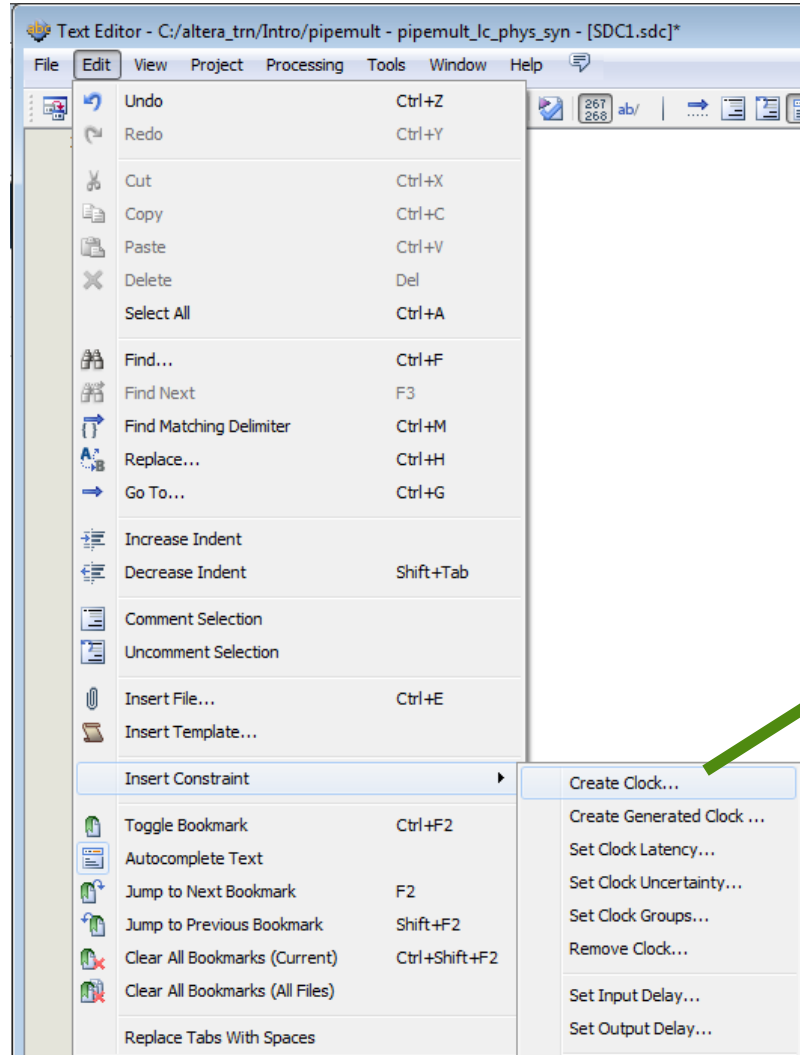
- Use Quartus editor to create and/or edit SDC
- SDC editing unique features (for .sdc files)
 - Access to GUI dialog boxes for constraint entry (**Edit → Insert Constraint**)
 - Syntax coloring
 - Delimiter matching
 - Tooltip syntax help
 - Auto-complete
 - SDC templates

The screenshot shows a text editor window titled 'Text Editor - C:/altera_trn/Intro/pipemult - pipemult_lc_phys_syn - [pipemult.sdc]'. The menu bar includes File, Edit, View, Project, Processing, Tools, Window, and Help. A search bar is present in the top right. The code is syntax-colored, with comments in green and commands in blue. A tooltip is visible over the 'create_clock' command on line 28, providing details about its arguments: `create_clock [-add] [-name <clock_name>] -period <value> [-waveform <edge_list>] [<targets>]`. The tooltip explains that `-add` adds a clock to an existing node, `-name` specifies the clock name, `-period` is the speed in terms of clock period, `-waveform` is a list of edge values, and `<targets>` is a list of targets.

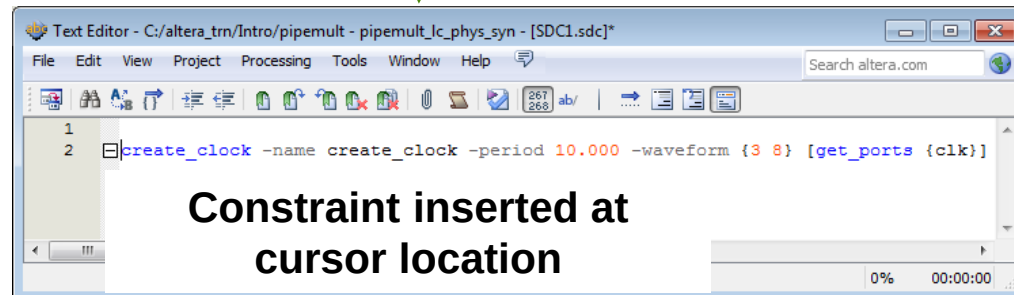
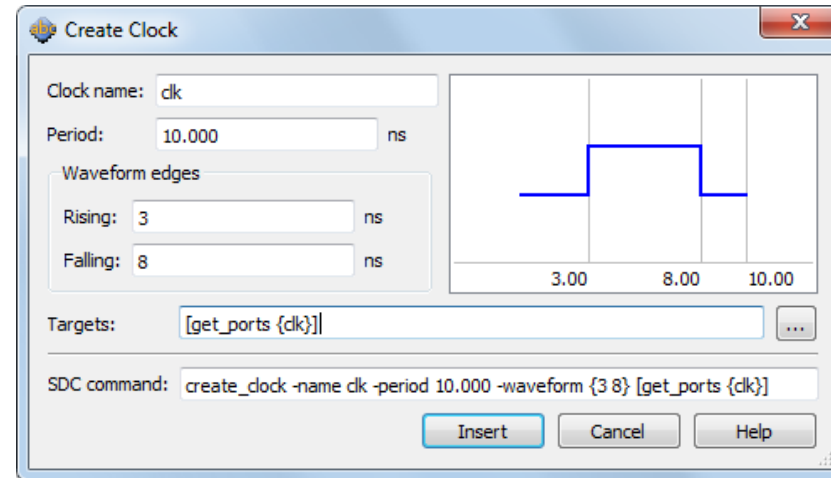
The screenshot shows an auto-complete dropdown menu in the Quartus Text Editor. The text 'deri' is entered in the editor, and the dropdown menu lists the following suggestions: 'derive_clock_uncertainty', 'derive_clocks', and 'derive_pll_clocks'. The list is numbered 5, 6, and 7 respectively.

Place cursor over command to see tooltip

SDC FILE EDITOR GUI CONSTRAINT ENTRY



Construct an SDC file using
TimeQuest graphical
constraint creation tools



SDC TEMPLATES

Quickly add customized constraint templates



SDC “Cookbook” templates from TimeQuest TA Online Resource Center

Preview window: edit before inserting & save as user template

Save User Template

You made changes to an Altera-provided template, however you cannot save changes to the template. If you want to save the changes, you can save the template as a user template.

User template name:

OK Cancel

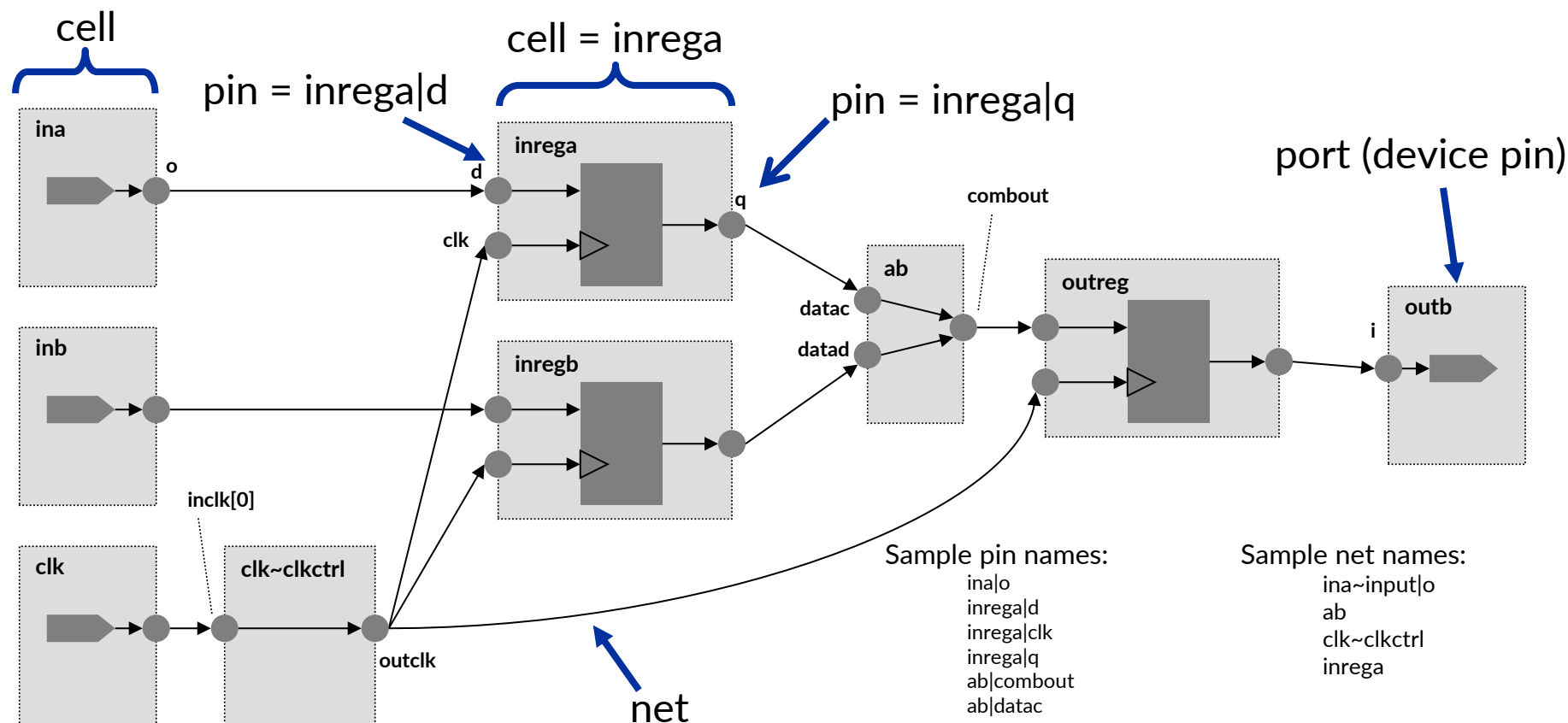
Save Insert Close

Toolbar button or **Edit** menu → **Insert Template**

SDC NETLIST TERMINOLOGY

Term	Definition
Cell	Device building blocks (e.g. look-up tables, registers, embedded multipliers, memory blocks, I/O elements, PLLs, etc.)
Pin	Input or outputs of cells
Net	Connections between pins
Port	Top-level inputs and outputs (e.g. device pins)
Registers	Registers in a design

SDC NETLIST EXAMPLE



Paths defined in constraints by targeted endpoints (pins or ports)

COLLECTIONS

- Searches the timing netlist and returns with a list of names meeting criteria
- Used in SDC commands
 - Some collections searched automatically during a command's usage and may not need to be specified
- Examples
 - get_ports
 - get_pins
 - get_clocks
 - get_cells
 - get_registers
 - all_clocks
 - all_registers
 - all_inputs
 - all_outputs

*See "The Quartus TimeQuest Timing Analyzer"
chapter of the Quartus Software Handbook
(Volume 3) for a complete list & description of each*

NAME FINDER

The screenshot shows the 'Name Finder' dialog box. At the top, the 'Collection' dropdown is set to 'get_ports' and the 'Filter' text box contains an asterisk (*). Below this is an 'Options' section with four checkboxes: 'Case-insensitive' (unchecked), 'Hierarchical' (unchecked), 'Compatibility mode' (checked), and 'No duplicates' (unchecked). A green box highlights the 'Options' section, with an arrow pointing to it from the text 'Options available depend on selected collection (next slide)'. Another green box highlights the 'Collection' dropdown, with an arrow pointing to it from the text 'Select collection to search'. The 'Matches' section shows a list of 74 matches found, with 'clk_in' selected. To the right of the list is a '1 selected name' box containing 'clk_in'. At the bottom, the 'SDC command' text box contains '[get_ports {clk_in}]', highlighted with a green box and an arrow from the text 'Edit command here or in .sdc file to use wildcards'. The dialog box has 'OK', 'Cancel', and 'Help' buttons at the bottom right.

Options available depend on selected collection (next slide)

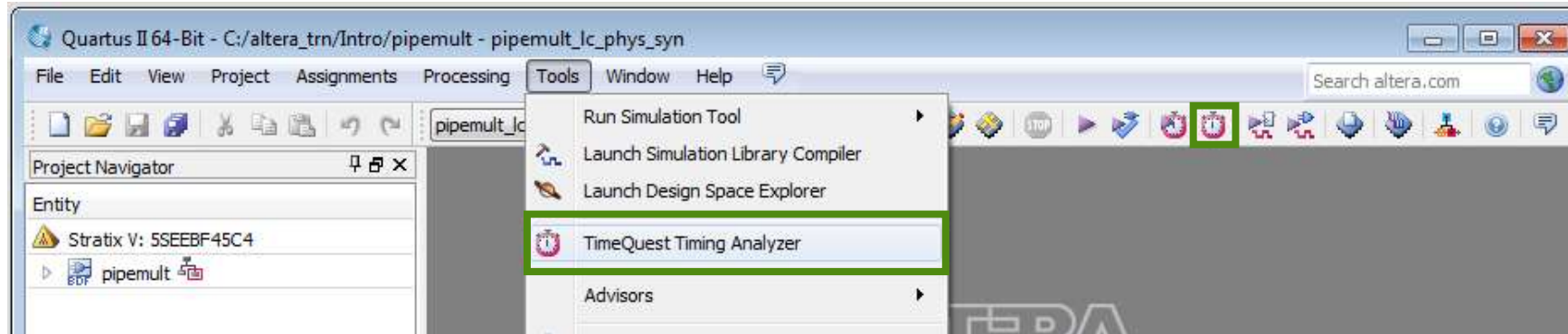
Select collection to search

Access from browse button in any GUI dialog box (similar to Quartus Node Finder)

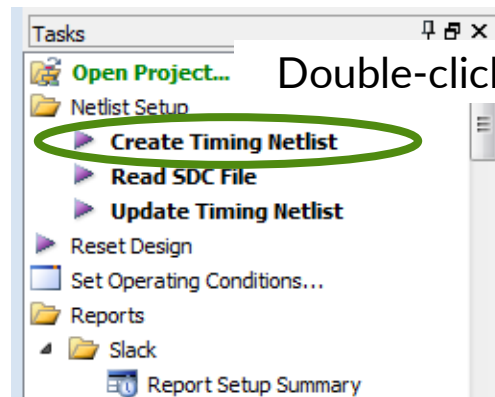
Edit command here or in .sdc file to use wildcards

CREATING A NETLIST FOR THE NAME FINDER

Open TimeQuest using the **Tools** menu or Toolbar button



Create a timing netlist (i.e. database) based on compilation results and selected delay



Double-click in **Tasks** pane

Do not close TimeQuest until you are done with the Name Finder

TIMEQUEST GUI

The screenshot displays the TimeQuest Timing Analyzer interface. The **Report pane** on the left shows a tree view with 'Summary (Hold)' selected. The **View pane** on the right shows the 'Summary of Paths' table and a detailed view of 'Path #1: Setup slack is 0.928', including a waveform diagram. The **Tasks pane** on the left shows a list of tasks, with 'Report Setup Summary' and 'Report Hold Summary' highlighted. The **Console pane** at the bottom shows a Tcl script execution log.

Report pane

View pane

Tasks pane

Console pane

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	0.928	inst2[8]	q[8]	clk1	clk1_out_virt	10.000	-4.909	3.423
2	0.966	inst2[2]	q[2]	clk1	clk1_out_virt	10.000	-4.900	3.394
3	0.968	inst2[13]	q[13]	clk1	clk1_out_virt	10.000	-4.909	3.383
4	0.972	inst2[15]	q[15]	clk1	clk1_out_virt	10.000	-4.894	3.394
5	0.973	inst2[9]	q[9]	clk1	clk1_out_virt	10.000	-4.904	3.383

	Total	Incr	RF	Type	Fanout	Log
1	0.000	0.000				
2	4.909	4.909				
1	0.000	0.000				
2	0.000	0.000				

	Total	Incr	RF	Type	Fanout	Location
1	10.000	10.000				
2	10.000	0.000				
1	10.000	0.000	R			
3	9.960	-0.040				
4	9.260	-0.700	F	oExt	0	PIN_AJ

```
tcl> create_timing_summary -hold -panel_name "Summary (Hold)"
> Timing requirements not met
tcl>
```


SDC TIMING CONSTRAINTS

- Clocks 
- I/O
- Asynchronous paths
- Timing exceptions
 - False paths
 - Multicycle paths

CLOCK CONSTRAINTS OBJECTIVES

- Create constraints for base clocks, virtual clocks, generated clocks, & PLL clocks
- Automatically detect and create clock constraints
- Constrain clock latency
- Constrain clock uncertainty

WHAT ARE CLOCKS IN SDC?

Defined, repeating signal characteristics applied to a point anywhere in the design

- Internal: applied to a specific node being used as a clock in design (port or pin)
- “Virtual”: no real source in, or direct interaction with design

Example: clocks on external devices that feed or are fed by the FPGA design, required for I/O analysis

Name clocks after node to which they are applied or something more meaningful

CLOCKS IN SDC (CONT.)

Two types

- Clock
 - Absolute/base clock or virtual clock
- Generated clock
 - Timing derived from another clock in design
 - Must have defined relation with source clock
 - Apply to output of logic function that modifies clock input
 - PLLs, clock dividers, output clocks, ripple clocks, etc.

All clocks are related by default

- Cross-domain transfers analyzed
- May need to “cut” clock domains from each other using timing exceptions (*discussed later*)

CREATING AND ABSOLUTE/BASE/VIRTUAL CLOCK

Command: create_clock

Options

[-name <clock_name>]

-period <time>

[-waveform {<rise_time> <fall_time>}]

[<target>]

[-add]

[] = optional

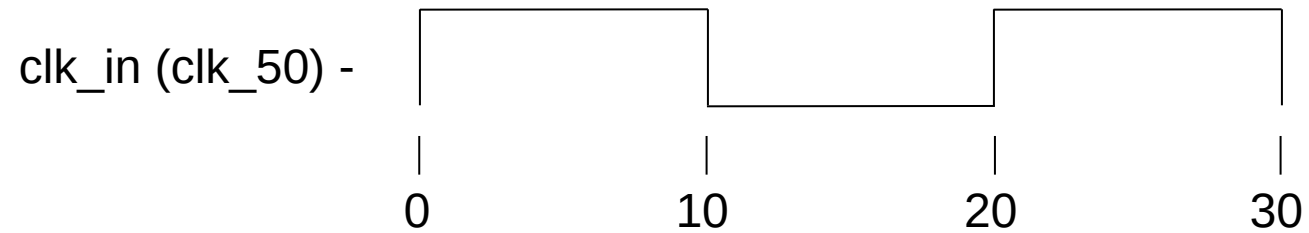
Note: In general, the more options added to a constraint command, the more specific the constraint is. When options are not specified, the constraint is more generalized and pertains to more of the target.

CREATE_CLOCK NOTES

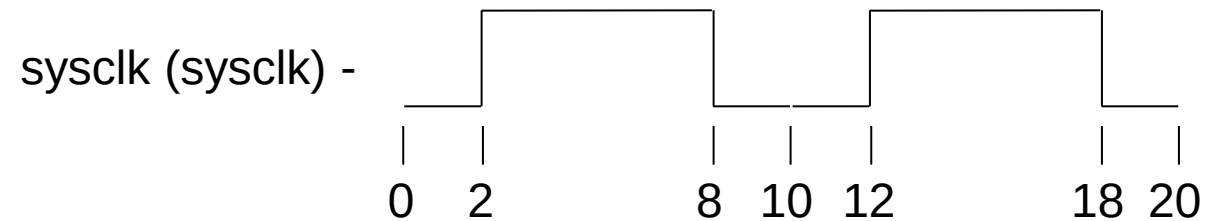
- name: assigns name to the clock to be used in other commands & reports when referring to clock
 - Optional for base clocks; defaults to target name if not specified
 - Required for virtual clocks
- waveform: indicates clock offset or non-50% duty cycle clocks
 - 50% duty cycle (rise at 0 ns, fall halfway through period) is assumed unless otherwise indicated
- <targets>: target port or pin for clock
 - Usually the device clock input port
 - Virtual clock created if no target specified
- add: adds clock to node with existing clock
 - Without -add, warning given that former clock constraint is overwritten

CREATE_CLOCK EXAMPLES

```
create_clock -period 20.0 -name clk_50 [get_ports clk_in]
```



```
create_clock -period 10.0 -waveform {2.0 8.0} [get_ports sysclk]
```



CREATE_CLOCK USING GUI

TimeQuest: **Constraints** menu → **Create Clock**
SDC Editor: **Edit** menu → **Insert Constraint** → **Create Clock**

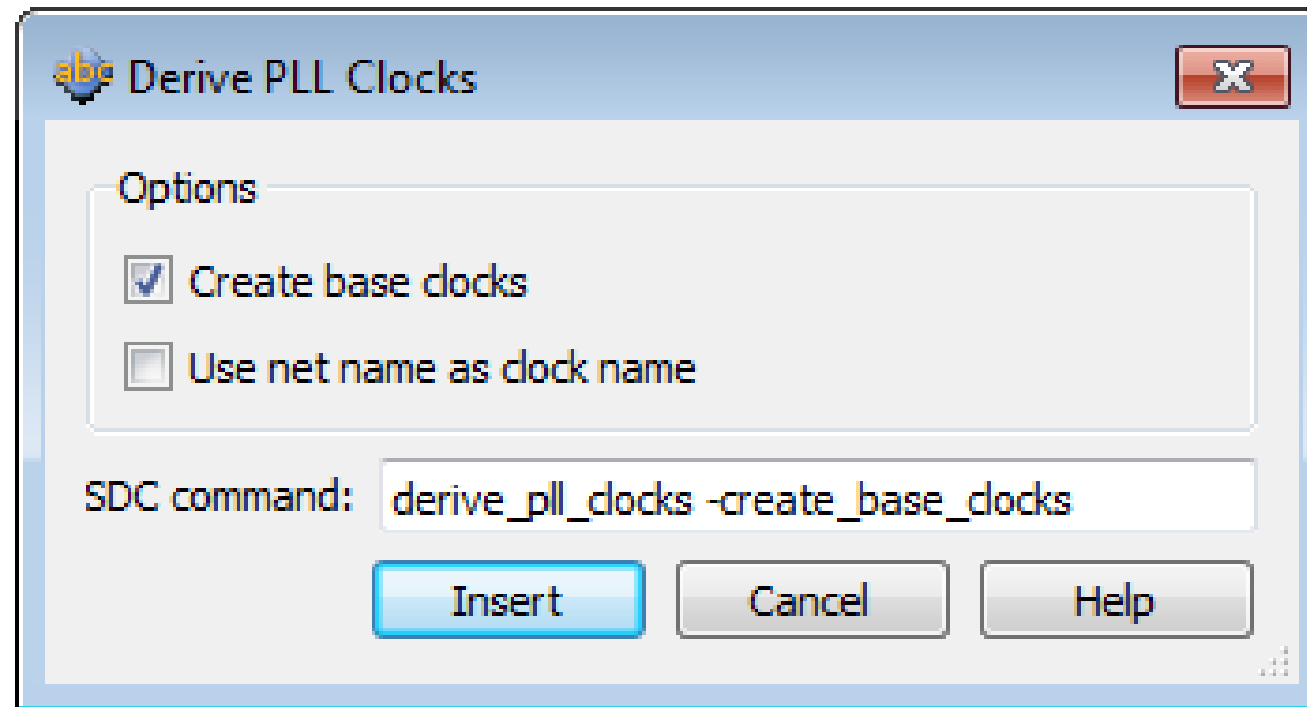
Name Finder

Edit any field
(change values; use wildcards in targets or command)

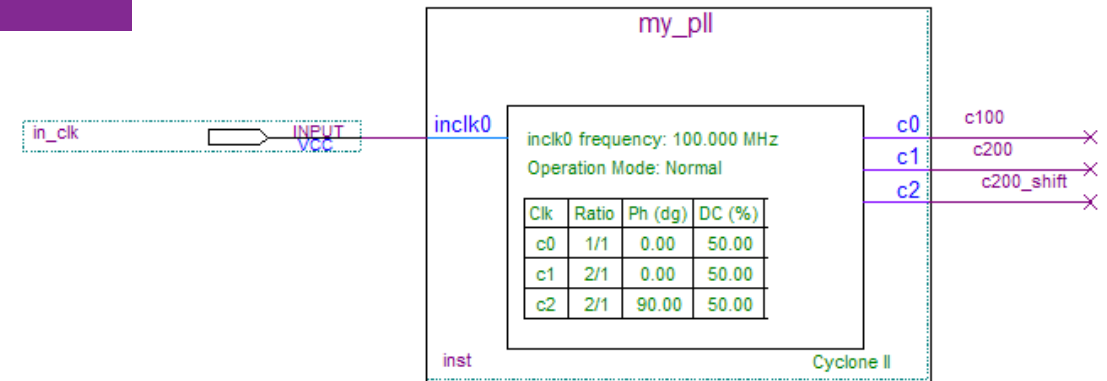
DERIVE PLL CLOCKS

- Command: `derive_pll_clocks`
 - `[-create_base_clocks]`: generates `create_clock` constraint(s) for PLL input clocks
- Create generated clocks on all PLL outputs
 - Based on input clock & PLL settings
- Requires defining PLL input as clock unless `-create_base_clocks` is used
- Automatically updates generated clocks on PLL outputs as changes made to PLL design
- `write_sdc -expand` expands constraint into standard `create_clock` and `create_generated_clock` commands

DERIVE PLL CLOCKS USING GUI



DERIVE_PLL_CLOCKS EXAMPLE



Using generated clock commands

```
create_clock -period 10.0 [get_ports in_clk]
create_generated_clock -name c100 \
    -source [get_pins {inst|altpll_component|pll|inclk[0]}] \
    -divide_by 1 \
    [get_pins {inst|altpll_component|pll|clk[0]}]
create_generated_clock -name c200 \
    -source [get_pins {inst|altpll_component|pll|inclk[0]}] \
    -multiply_by 2 \
    [get_pins {inst|altpll_component|pll|clk[1]}]
create_generated_clock -name c200_shift \
    -source [get_pins {inst|altpll_component|pll|inclk[0]}] \
    -multiply_by 2 \
    -phase 90 \
    [get_pins {inst|altpll_component|pll|clk[2]}]
```

Using derive PLL command

```
create_clock -period 10.0 \
    [get_ports in_clk]
derive_pll_clocks

# or simply:

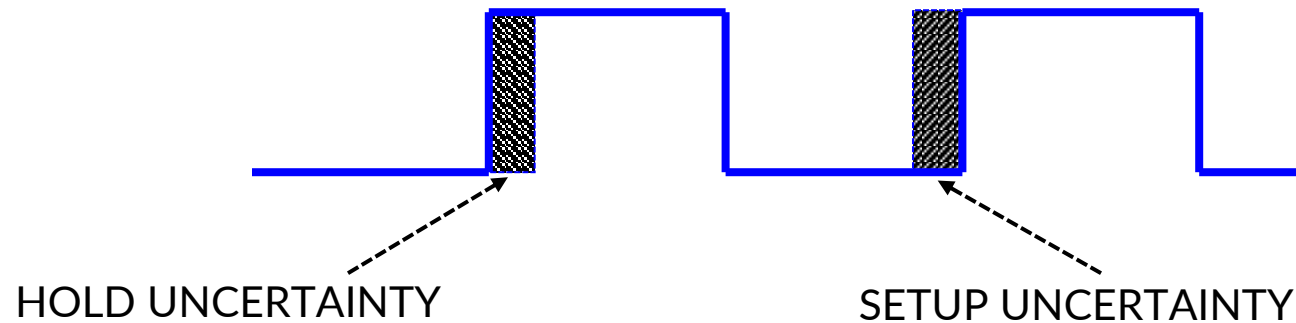
derive_pll_clocks \
    -create_base_clocks

# Note the clock names for
# the generated clocks
# will be the names of
# the PLL output pins
```

CLOCK UNCERTAINTY

Setup uncertainty decreases setup required time

Hold uncertainty increases hold required time



Ex. To add a 0.5-ns guardband around clock, use 250 ps of setup uncertainty and 250 ps of hold uncertainty.

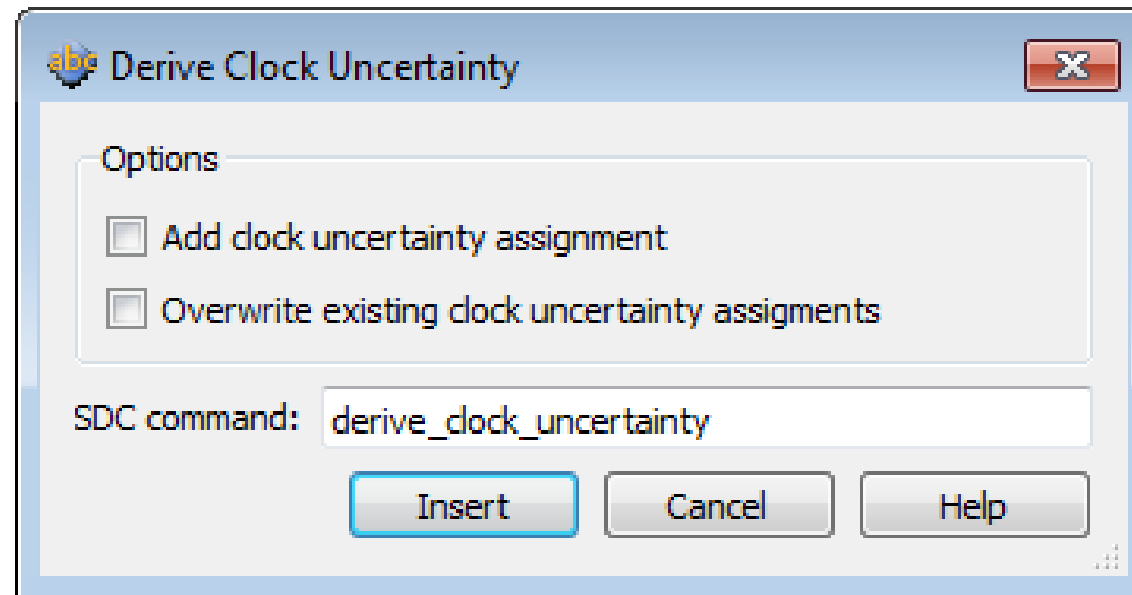
Used to model jitter, guard band, or skew

- Allows generation of clocks that are non-ideal

AUTOMATICALLY DERIVED UNCERTAINTIES

- Command: *derive_clock_uncertainty*
- Options
 - *[-overwrite]*: overwrites any existing uncertainty constraints
 - *[-add]*: adds derived uncertainties to existing constraints
- SDC extension expanded with *write_sdc -expand*
- View derived uncertainties with *report_sdc*
- *set_clock_uncertainty* have higher precedence than derived uncertainties by default
- *Use is recommended for all designs*
- *Automatically performed on new projects*
- *Results only seen after place and route*

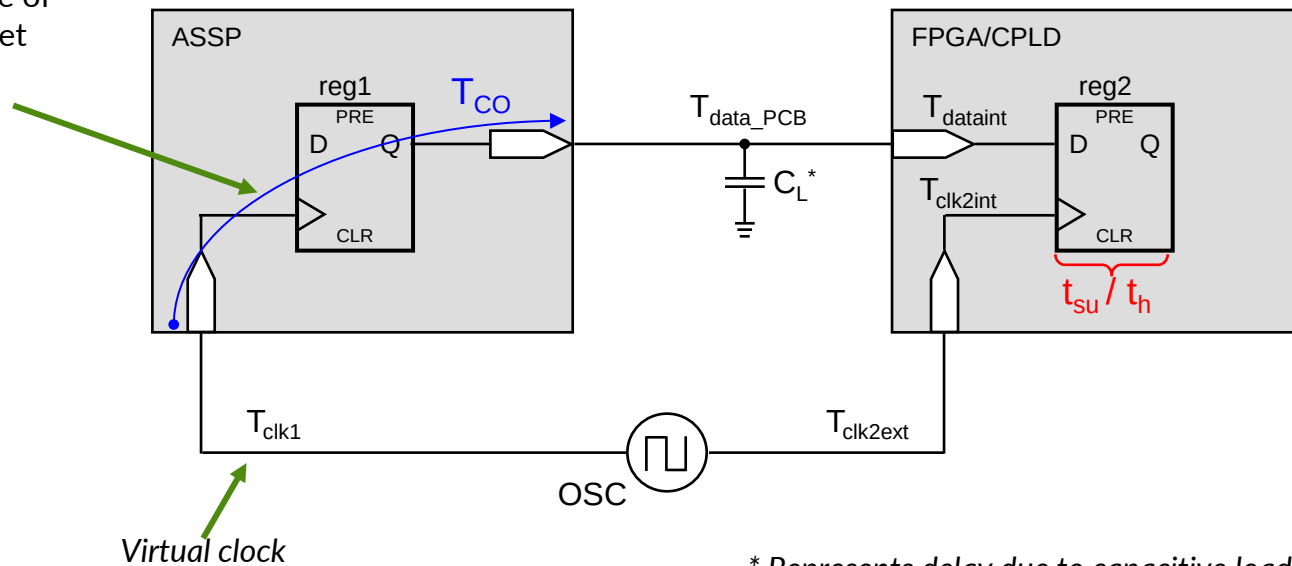
DERIVED CLOCK UNCERTAINTY (GUI)



SYNCHRONOUS INPUTS

- Need to specify timing relationship from ASSP to FPGA/CPLD to guarantee setup/hold in FPGA/CPLD
- Adjust placement and routing of input register to meet timing

T_{co} represents total clock-to-output time of ASSP (i.e. datasheet spec)



* Represents delay due to capacitive loading

CONSTRAINING SYNCHRONOUS INPUTS

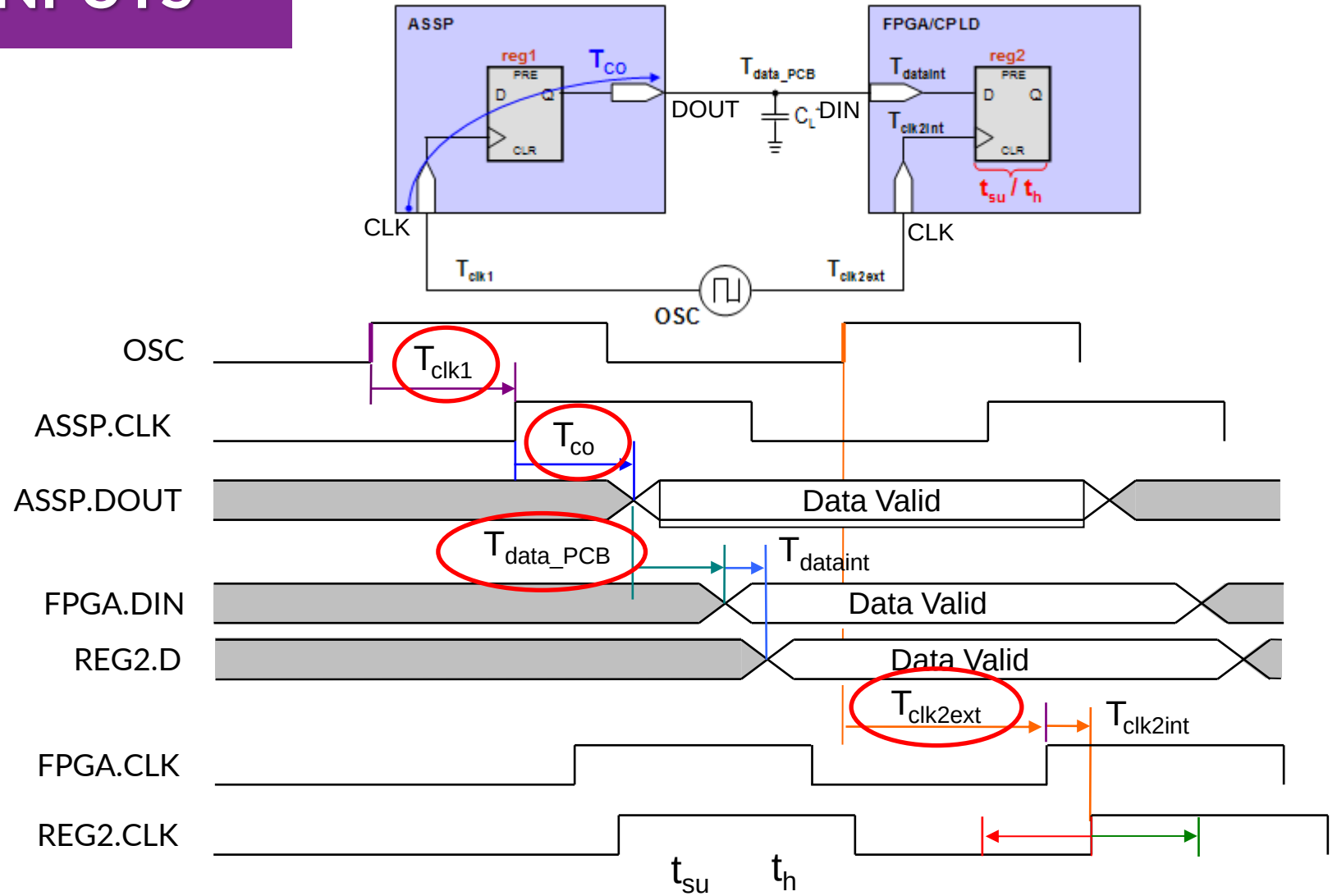
Use *set_input_delay* (-max option) command to constrain input setup time (maximum time to arrive and still meet T_{su})

- Calculated maximum input delay value representing all delays *external* to device

Use *set_input_delay* (-min option) command to constrain input hold time (minimum time to stay active and still meet T_h)

- Calculated minimum input delay value representing all delays *external* to device

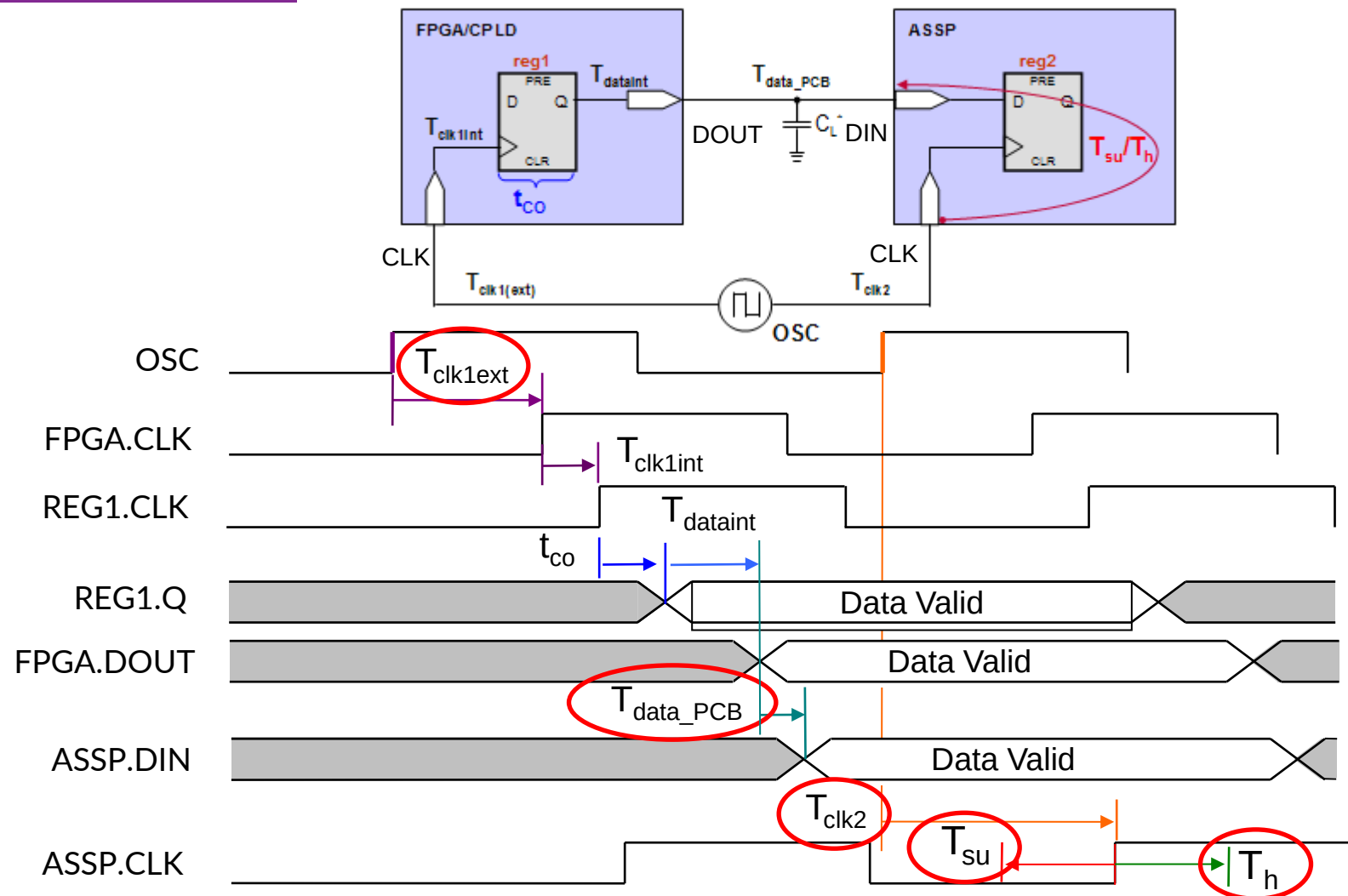
SYNCHRONOUS INPUTS



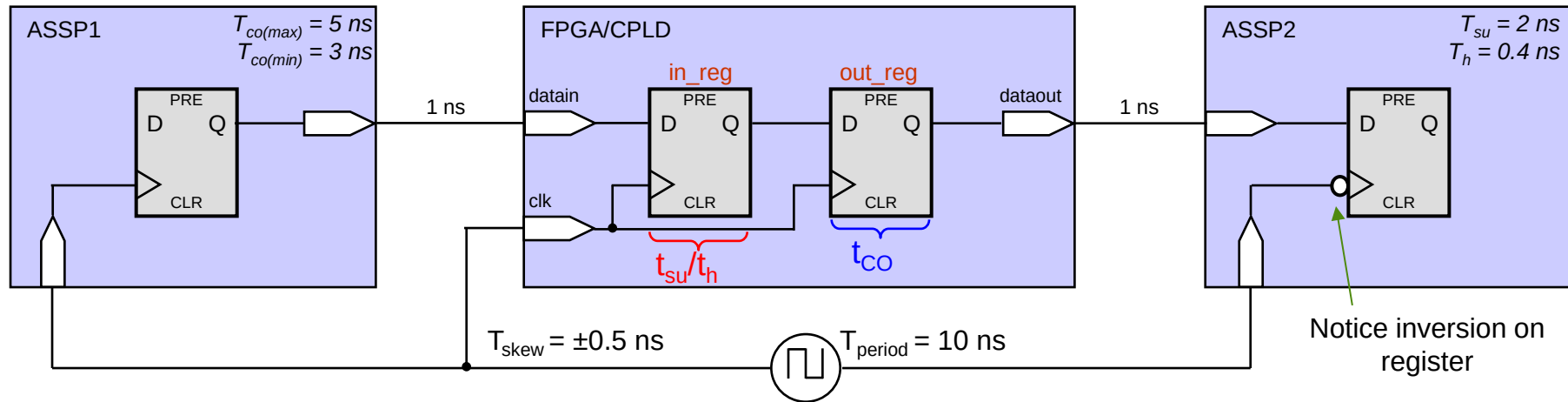
CONSTRAINING SYNCHRONOUS OUTPUTS

- Use *set_output_delay* (-max option) command to constrain maximum clock-to-output (maximum time to arrive and still meet ASSP's T_{su})
 - Calculated maximum output delay value represents all delays *external* to device
- Use *set_output_delay* (-min option) command to constrain minimum clock-to-output (minimum time to stay active and still meet ASSP's T_h)
 - Calculated minimum output delay value represents all delays *external* to device

SYNCHRONOUS OUTPUTS



SYNCHRONOUS I/O EXAMPLE



```
create_clock -period 10 -name clk [get_ports clk]
create_clock -period 10 -name clk_v_in; # virtual clock for input constraint
create_clock -period 10 -name clk_v_out; # virtual clock for output constraint

set_input_delay -clock clk_v_in -max [expr 1 - (-0.5) + 5] [get_ports datain]
set_input_delay -clock clk_v_in -min [expr 1 - 0.5 + 3] [get_ports datain]

set_output_delay -clock clk_v_out -max [expr 1 - (-0.5) + 2] \
    -clock_fall [get_ports dataout]
set_output_delay -clock clk_v_out -min [expr 1 - 0.5 - 0.4] \
    -clock_fall [get_ports dataout]
```


CONSTRAIN I/O USING TcI VARIABLES

Use Tcl variables in .sdc file for clarity and ease of reuse when constraining I/Os

```
# Enter device and PCB info related to the I/O
set Tco_max 5.000
set Tco_min 3.000
set Td_max 1.0
set Td_min 1.0
set longest_src_clk 1.0
set shortest_src_clk 0.5
set longest_dest_clk 1.0
set shortest_dest_clk 0.5

# Calculate the clock skew
set Tcs_smallest [expr $shortest_dest_clk - $longest_src_clk]
set Tcs_largest  [expr $longest_dest_clk - $shortest_src_clk]

# Calculate the input min and max values
set input_max [expr $Td_max + $Tco_max - $Tcs_smallest]
set input_min [expr $Td_min + $Tco_min - $Tcs_largest]

# Create the input delay constraint
set_input_delay -max -clock vir_clk_in $input_max [get_ports datain]
set_input_delay -min -clock vir_clk_in $input_min [get_ports datain]
```

TWO METHODS TO CREAT FALSE PATHS

- *set_false_path* command
 - Use when particular nodes are involved
 - Examples
 - All paths from an input pin to a set of registers
 - All paths from a register to another clock domain

- *set_clock_groups* command
 - Use when just clock domains are involved

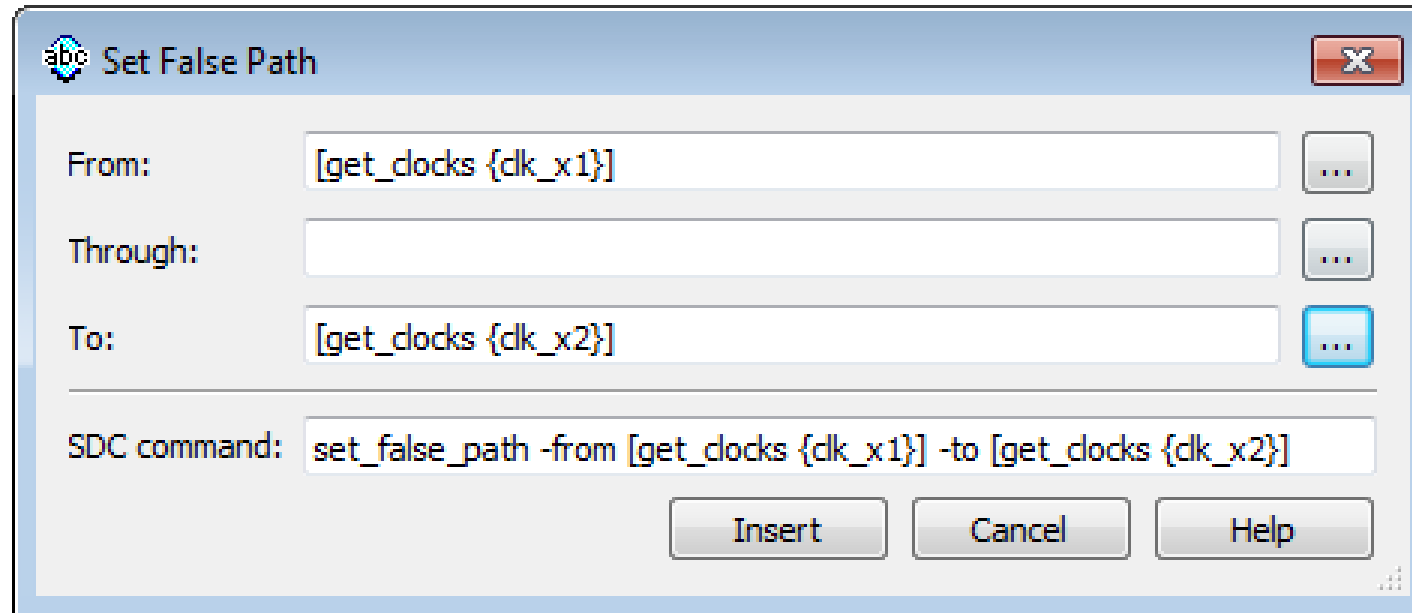
SET_FALSE_PATH COMMAND

- Indicates paths that should be ignored during fitting and timing analysis
- Options
 - [-fall_from <clocks>]
 - [-rise_from <clocks>]
 - [-from <names>]
 - [-through <names>]
 - [-to <names>]
 - [-fall_to <clocks>]
 - [-rise_to <clocks>]
 - [-setup]
 - [-hold]

SET_FALSE_PATH NOTES

- *-from* & *-to*: specify source & target nodes
 - Nodes can be clocks, registers, ports, pins, or cells
 - For registers, *-from* should be source register clock input pin, not **q** output
 - Specify a clock name to constrain all paths going into or out of its domain
 - Constrains both rising and falling edge clock transitions
 - More efficient than specifying individual nodes
- *-rise_from* & *-fall_from*: no analysis should be performed on the targeted path at the rising or falling edge transition of the path's launch clock; *not in GUI*
- *-rise_to* & *-fall_to*: no analysis should be performed on the targeted path at the rising or falling edge transition of the path's latch clock; *not in GUI*
- *-setup* & *-hold*: no setup/recovery or hold/removal analysis should be performed on the targeted path; *not in GUI*

SET FALSE PATH (GUI)



The image shows a 'Set False Path' dialog box with a title bar containing an 'abc' icon and a close button. The dialog has three input fields: 'From:', 'Through:', and 'To:'. The 'From:' field contains '[get_clocks {clk_x1}]', the 'Through:' field is empty, and the 'To:' field contains '[get_clocks {clk_x2}]'. Each field has a browse button (three dots) to its right. Below these fields is a text area for the 'SDC command:' which contains the command 'set_false_path -from [get_clocks {clk_x1}] -to [get_clocks {clk_x2}]'. At the bottom are three buttons: 'Insert', 'Cancel', and 'Help'.

abc Set False Path

From: [get_clocks {clk_x1}] ...

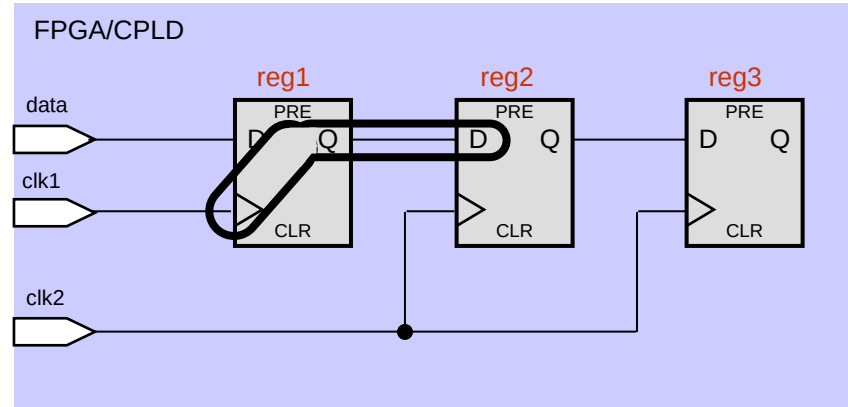
Through: ...

To: [get_clocks {clk_x2}] ...

SDC command: set_false_path -from [get_clocks {clk_x1}] -to [get_clocks {clk_x2}]

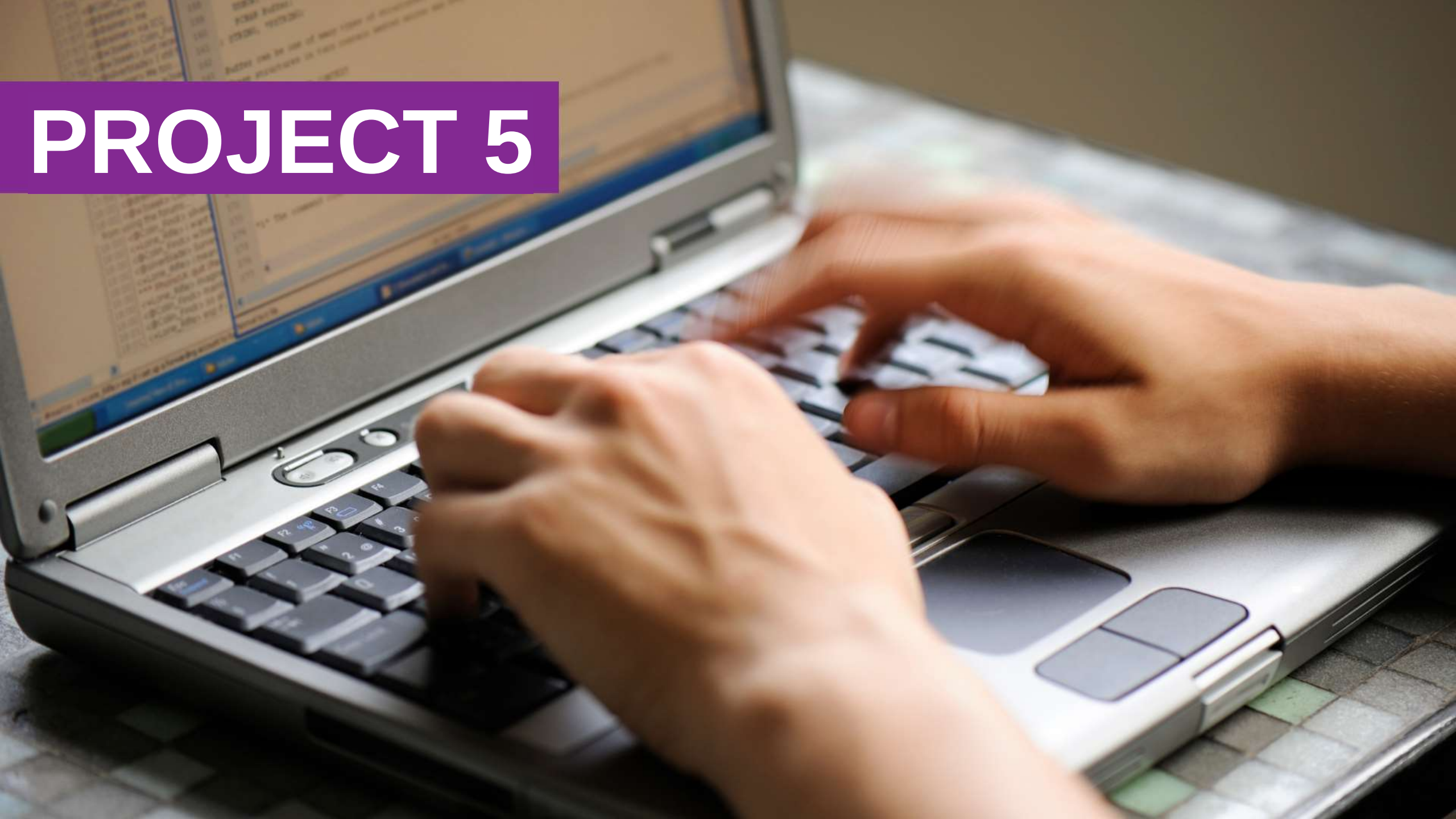
Insert Cancel Help

FALSE PATH EXAMPLE 1



Simple synchronizer
circuit between two
asynchronous clock
domains

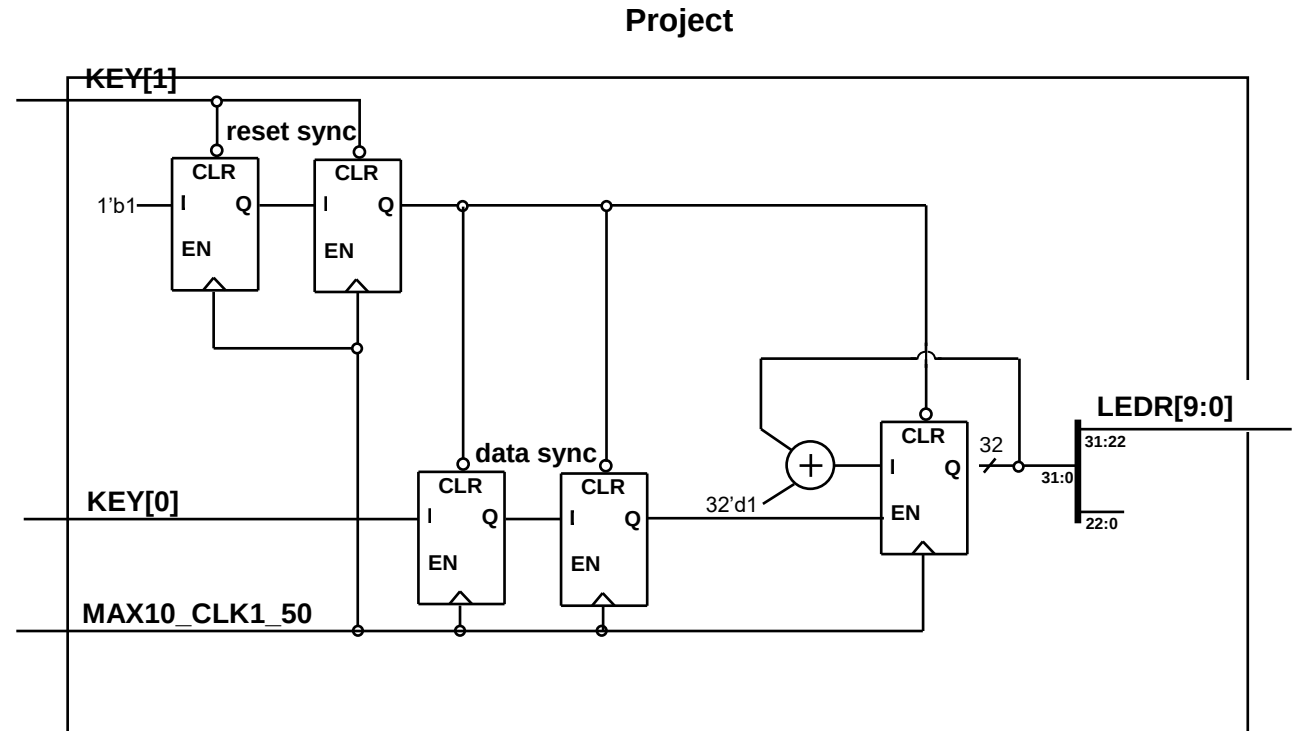
```
set_false_path -from [get_cells reg1] -to [get_cells reg2]
```



PROJECT 5

PROJECT 5

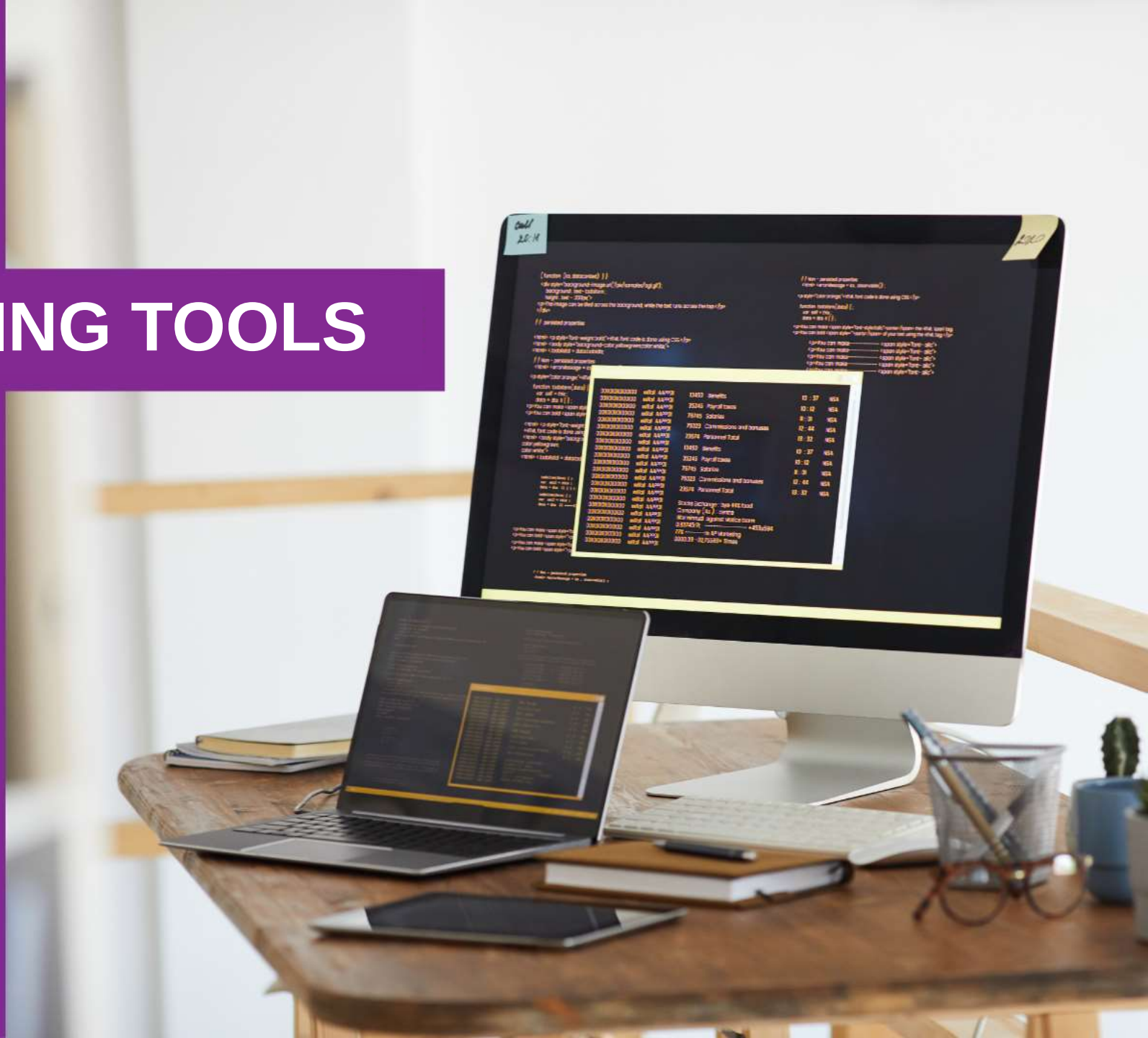
- Open the projeto-05 and compile it
- Create a reset synchronizer
- Create an enable synchronizer
- Create an SDC file defining a clock and false paths if necessary



QUARTUS DEBUGGING TOOLS

**EMBEDDED LOGIC
ANALYZER:**

***SIGNAL TAP LOGIC
ANALYZER***



SIGNAL TAP EMBEDDED LOGIC ANALYZER

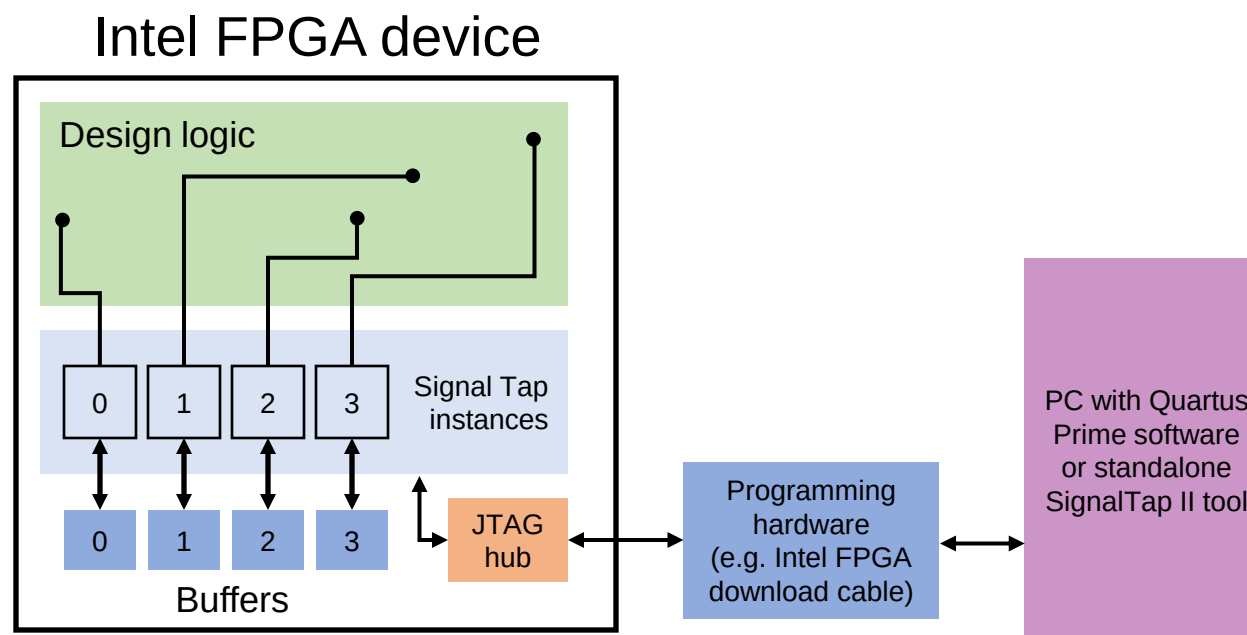


An Embedded Logic Analyzer!

SIGNAL TAP LOGIC ANALYZER

- Captures the logic state of FPGA internal signals using a defined clock signal
- Monitor buried signals in real-time
- Connects to the Quartus® Prime software through FPGA JTAG connection
- Available for free from the Intel FPGA Download Center
 - <https://www.intel.com/content/www/us/en/programmable/downloads/download-center.html>
 - Installed with the Quartus Prime software (all editions)
 - Installed with the stand-alone programmer
 - At Download Center, select **Programming Software** from **Select by Software** tab

WHAT IS THE SIGNAL TAP?



WHEN TO USE THE SIGNAL TAP

- No external equipment available
- Additional device resources available
 - Logic blocks
 - Memory blocks
- JTAG connection available
 - Local or remote over network

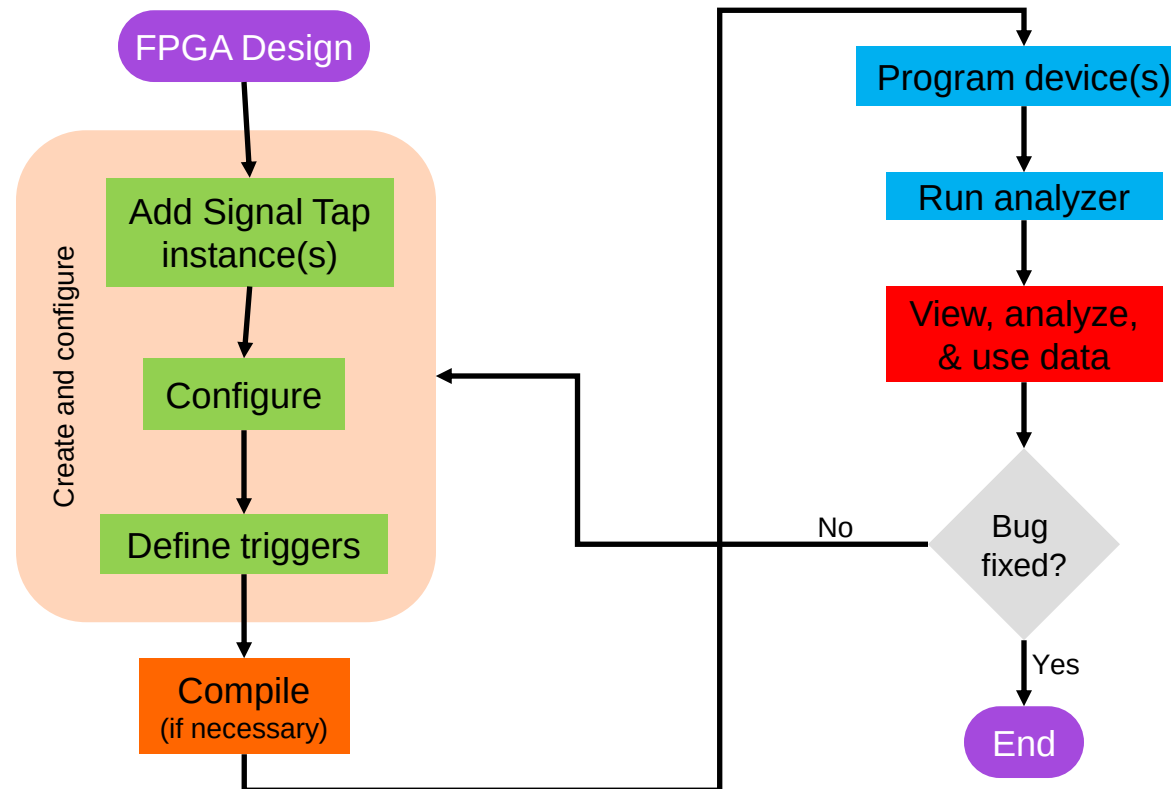
SIGNAL TAP RESOURCE UTILIZATION

- Logic elements
 - Number of channels
 - Number and complexity of trigger conditions
- Memory blocks
 - Number of channels
 - Sample depth
 - Selectable trade-off between depth & number of channels
 - 128K sample depth with 2048 channels not practical = 13107 M20K blocks
 - Largest Stratix® 10 device has ~11000 M20K blocks
- Estimated resource usage displayed and update during Signal Tap configuration

FEATURE OVERVIEW

Feature	Benefit
Multiple SignalTap II cores	Supports multiple clock domains or functional blocks in single device
Rapid Recompile	Perform incremental routing to minimize recompile time when adding/deleting/changing node connections
Up to 2048 data channels/128K samples per channel	View large samples of data
Up to 10 trigger conditions	Watch for many sequential (or non-sequential) events
State-based triggering flow	State machine controlled sequence of trigger conditions to ease capture of non-sequential events
External triggers	Interface Signal Tap instances with other devices and logic analyzers, including the Hard Processor System (HPS) on System on Chip (SoC) devices
Basic & advanced trigger support	Create simple (Boolean AND or OR) triggers or more complicated expressions
Data storage qualification	Only store data samples of interest in the buffer to effectively use limited buffer space
Store data in multiple output file formats	Allows sampled data to be used for simulation, data analysis, or documentation

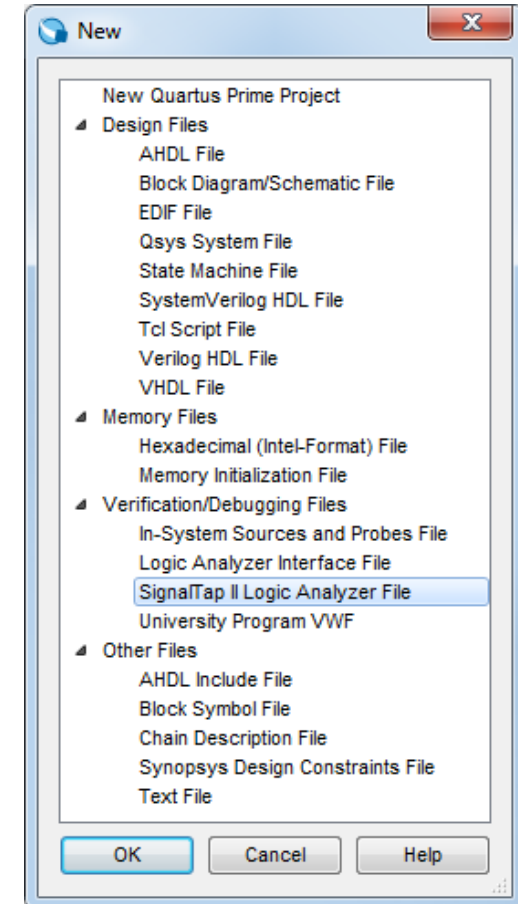
TYPICAL SIGNAL TAP DEBUGGING FLOW



ADDING SIGNAL TAP ELA TO A DESIGN:

CREATE .stp FILE

- Use Signal Tap file (.stp)
 - Use Quartus Prime GUI
 - Configure .stp file details manually
 - Easily enable or disable in project settings
- Method 1
 - **Tools** menu → **Signal Tap Logic Analyzer**
- Method 2
 - Select **File** menu → **New**
 - **Verification/Debugging Files** category
 - **Signal Tap Logic Analyzer File**



SIGNAL TAP LOGIC ANALYZER WINDOW

The screenshot shows the SignalTap II Logic Analyzer interface. At the top, the title bar reads "SignalTap II Logic Analyzer - C:/altera_trn/Quartus_II_Debug/QIID14_1/Ex2/top_counter - top_counter - [counter.stp]". The menu bar includes File, Edit, View, Project, Processing, Tools, Window, and Help. Below the menu bar is a toolbar with icons for file operations, analysis, and help.

Instance Manager: A table showing the status of the logic analyzer instance.

Instance	Status	Enabled	LEs: 584	Memory: 7168	Small: 0/415480	Med
instance_one	Not running	☒	584 cells	7168 bits	0 blocks	1 blk

JTAG Chain Configuration: A panel showing the JTAG chain configuration. It includes a "Hardware" dropdown, a "Device" dropdown (currently "None Detected"), and a "Setup..." button. A red banner at the top says "No devices detected".

Signal Configuration: A panel for configuring the signal analysis. It includes a "Clock" dropdown (set to "clk"), "Data" settings (Sample depth: 1 K, RAM type: Auto), "Segmented" settings (2 512 sample segments), "Nodes Allocated" (Auto/Manual), "Pipeline Factor" (0), "Storage qualifier", and "Type" (Continuous).

Node List: A table listing the nodes being analyzed.

Name	Data Enable	Trigger Enable	Trigger Conditions
u1[one_led_out{0..6}]	☒	☒	eight
u1[one_led_out{0}]	☒	☒	0
u1[one_led_out{1}]	☒	☒	0
u1[one_led_out{2}]	☒	☒	0
u1[one_led_out{3}]	☒	☒	0
u1[one_led_out{4}]	☒	☒	0
u1[one_led_out{5}]	☒	☒	0
u1[one_led_out{6}]	☒	☒	0

Trigger conditions & storage qualifiers: A label pointing to the "Trigger Conditions" column in the Node List table.

Design Hierarchy: A panel showing the design hierarchy. It includes a "Hierarchy Display" tree with "top_counter" and "u1" selected, and a "Data Log" section showing "instance_one".

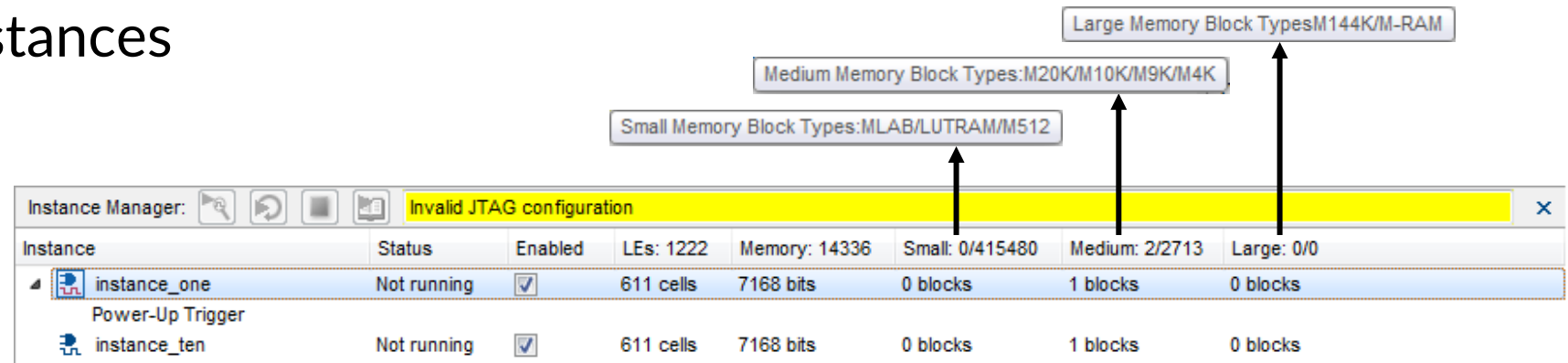
Data Log: A panel showing the data log. It includes a "Data Log" button and a "Data Log" section showing "instance_one".

Annotations:

- A yellow banner at the top of the Instance Manager says "Invalid JTAG configuration".
- A red banner at the top of the JTAG Chain Configuration says "No devices detected".
- A red circle highlights the "Data" and "Setup" buttons in the Design Hierarchy panel.

INSTANCE MANAGER

- Select logic analyzer instance or power-up trigger (*described later*) to setup/view/control
- Create new instances (right-click)
- Displays the current status of each instance
- Displays size (resource usage) of each instance
- Run and control instances



Invalid JTAG configuration

Instance	Status	Enabled	LEs: 1222	Memory: 14336	Small: 0/415480	Medium: 2/2713	Large: 0/0
instance_one	Not running	☒	611 cells	7168 bits	0 blocks	1 blocks	0 blocks
Power-Up Trigger							
instance_ten	Not running	☒	611 cells	7168 bits	0 blocks	1 blocks	0 blocks

Small Memory Block Types:MLAB/LUTRAM/M512

Medium Memory Block Types:M20K/M10K/M9K/M4K

Large Memory Block Types:M144K/M-RAM

NODE LIST

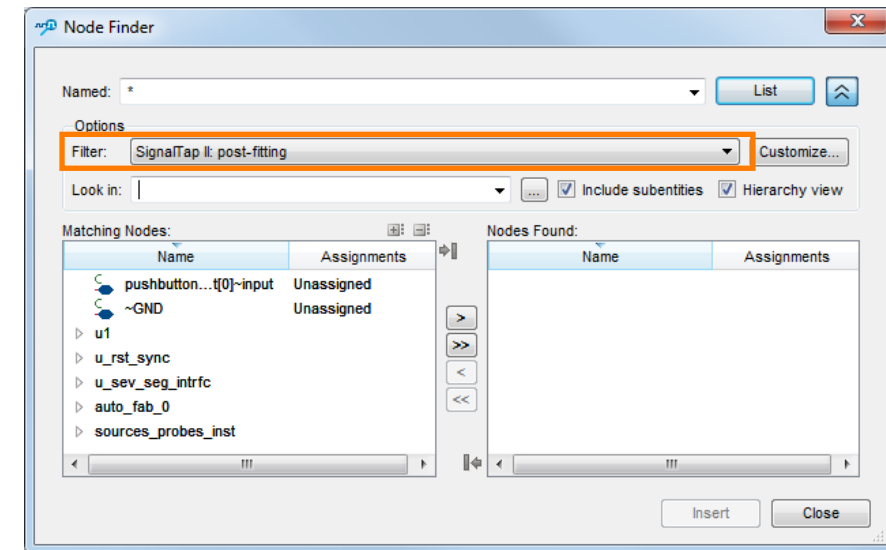
- Add and configure signals for data capturing and triggering
- Double-click empty space to add signals with Quartus Node Finder
- Busses grouped automatically; manually create custom groups

Grouped nodes (bus or right-click Group command)

trigger: 2016/07/08 11:08:17 #1			Lock mode: Allow all changes			
Node			Data Enable	Trigger Enable	Trigger Conditions	
Type	Alias	Name	7	7	1 ☒ Basic AND	2 ☒ Basic AND
		u1 one_led_out[0..6]	☒	☒	eight	nine
		u1 one_led_out[0]	☒	☒	0	0
		u1 one_led_out[1]	☒	☒	0	0
		u1 one_led_out[2]	☒	☒	0	0
		u1 one_led_out[3]	☒	☒	0	0
		u1 one_led_out[4]	☒	☒	0	1
		u1 one_led_out[5]	☒	☒	0	0
		u1 one_led_out[6]	☒	☒	0	0

NODE FINDER WITH SIGNAL TAP

- **Signal Tap: pre-synthesis:** Nodes available *after* analysis, but *before* synthesis (RTL signals)
 - Connections to SignalTap made before synthesis as if done in HDL
 - Colored **black** in Node List
- **Signal Tap: post-fitting:** Nodes available *after* fitter optimizations and place-and-route
 - Connections made only if routing resources available in post-fit netlist
 - Colored **blue** in Node List
- Preserve signal names through compilation with **keep** and **preserve** HDL attributes

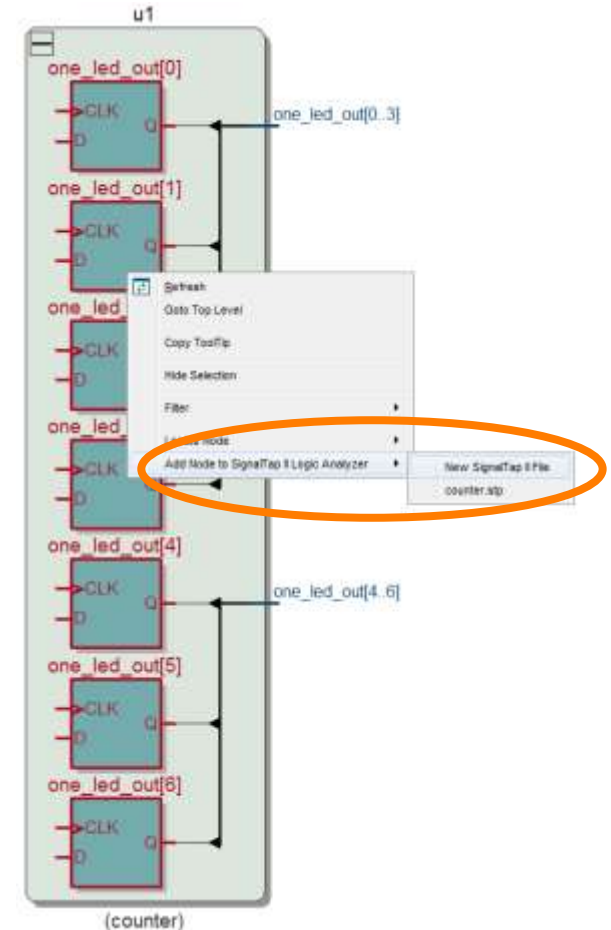


TAPPING FROM TECHNOLOGY MAP VIEWER

- Select post-fit nodes in Technology Map Viewer
- Quickly add to new or existing Signal Tap file

Tools menu → Netlist Viewers →
Technology Map Viewer (Post Fitting)

Select and right-click nodes



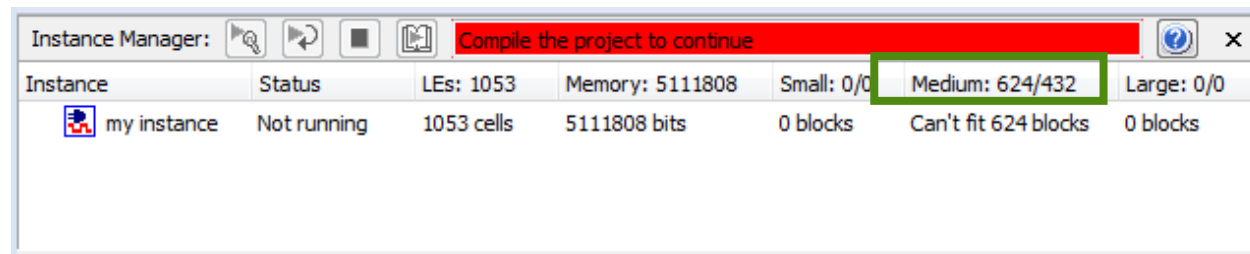
SET UP NODE LIST

- **Data Enable** column controls whether signals captured as data
 - Turn off to reduce memory resource usage
- **Trigger Enable** controls whether signals are part of the trigger condition
 - Turn off to reduces logic cell resource usage
- Monitor Instance Manager to see effects

trigger: 2016/07/08 11:08:17 #1			Lock mode: Allow all changes			
Node			Data Enable	Trigger Enable	Trigger Conditions	
Type	Alias	Name	7	7	1 Basic AND	2 Basic AND
		u1 one_led_out[0..6]			eight	nine
		u1 one_led_out[0]			<u>0</u>	<u>0</u>
		u1 one_led_out[1]			<u>0</u>	<u>0</u>
		u1 one_led_out[2]			<u>0</u>	<u>0</u>
		u1 one_led_out[3]			<u>0</u>	<u>0</u>
		u1 one_led_out[4]			<u>0</u>	<u>1</u>
		u1 one_led_out[5]			<u>0</u>	<u>0</u>
		u1 one_led_out[6]			<u>0</u>	<u>0</u>

FITTING PREDICTION

- Predict whether current configuration can fit before lengthy compilation
- Based on device resource capacity and resources required by logic analyzer instance(s) and user design



The screenshot shows the 'Instance Manager' window. At the top, there is a red error bar with the text 'Compile the project to continue'. Below this is a table with columns: Instance, Status, LEs: 1053, Memory: 5111808, Small: 0/0, Medium: 624/432, and Large: 0/0. The 'Medium' column header is highlighted with a green box. The table contains one row for 'my instance' with the status 'Not running'. The 'Medium' column value for this instance is 'Can't fit 624 blocks'.

Instance	Status	LEs: 1053	Memory: 5111808	Small: 0/0	Medium: 624/432	Large: 0/0
my instance	Not running	1053 cells	5111808 bits	0 blocks	Can't fit 624 blocks	0 blocks

BASIC TRIGGERING

trigger: 2016/07/08 11:08:17 #1

Lock mode: Allow all changes

Node		Data Enable	Trigger Enable	Trigger Conditions		
Type	Alias	Name	7	7	1 Basic AND	2 Basic AND
		u1 one_led_out[0..6]			eight	nine
		u1 one_led_out[0]			<u>0</u>	<u>0</u>
		u1 one_led_out[1]			<u>0</u>	<u>0</u>
		u1 one_led_out[2]			<u>0</u>	<u>0</u>
		u1 one_led_out[3]			<u>0</u>	<u>0</u>
		u1 one_led_out[4]			<u>0</u>	<u>1</u>
		u1 one_led_out[5]			<u>0</u>	<u>0</u>
		u1 one_led_out[6]			<u>0</u>	<u>0</u>

Condition type

Right-click to set value

TRIGGER TYPES AND OPTIONS

Disable individual conditions to skip
(sequential flow only; described later)

trigger: 2016/07/08 11:08:17 #1 Lock mode: Allow all changes

Node		Data Enable	Trigger Enable	Trigger Conditions	
Type	Alias	Name	7	7	
		u1 one_led_out[0..6]	☒	☒	Basic AND
		u1 one_led_out[0]	☒	☒	eight
		u1 one_led_out[1]	☒	☒	0
		u1 one_led_out[2]	☒	☒	0
		u1 one_led_out[3]	☒	☒	0
		u1 one_led_out[4]	☒	☒	0
		u1 one_led_out[5]	☒	☒	0
		u1 one_led_out[6]	☒	☒	0

Basic AND: *all* signal levels/transitions must be true

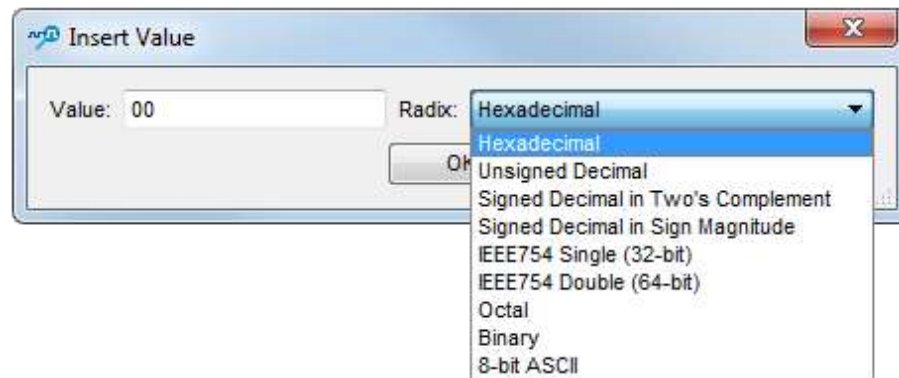
Basic OR: *any* levels/transitions true; bits in groups are OR reduced unless condition is a numerical value

Comparison: extension of **Basic OR**; compare groups to expected integer value(s)

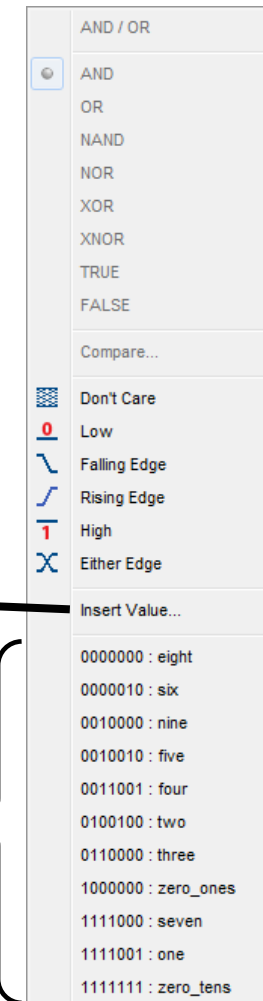
Advanced

CONDITION VALUE OPTIONS

- Right-click condition value cell to access
- Options available depend on type of trigger condition and whether setting up individual signal or group



Mnemonic trigger values



Basic OR options

Basic AND and OR values

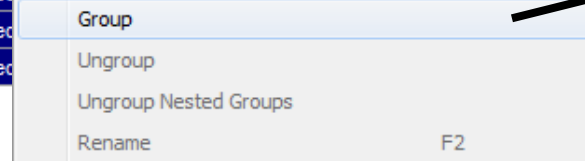
GROUPS & NESTED GROUPS

- Tapped nodes grouped by selecting and right-clicking
- Create *nested* groups for organization and customizable OR triggering
- Top group is referred to as the *parent*
- **Basic AND** trigger type behaves the same

trigger: 2016/07/08 11:08:17 #1

Node		
Type	Alias	Name
R		u1 one_led_out[0..6]
R		u1 one_led_out[0]
R		u1 one_led_out[1]
R		u1 one_led_out[2]
R		u1 one_led_out[3]
R		u1 one_led_out[4]
R		u1 one_led_out[5]
R		u1 one_led_out[6]

Parent group



trigger: 2016/07/08 11:08:17 #1

Node		
Type	Alias	Name
R		u1 one_led_out[0..6]
R		u1 one_led_out[0]
R		u1 one_led_out[1]
R		u1 one_led_out[2]
R		u1 one_led_out[3..6]
R		u1 one_led_out[3]
R		u1 one_led_out[4]
R		u1 one_led_out[5]
R		u1 one_led_out[6]

Nested group (child)

BASIC OR TRIGGERING WITH GROUPS & NESTED GROUPS

- Creates an *advanced* trigger condition for condition checking between the parent and nested group(s)
- Lowest level nested group evaluated first, then evaluated against nodes in next higher level

trigger: 2016/07/08 11:08:17 #1			Lock mode: Allow all changes			
Node			Data Enable	Trigger Enable	Trigger Conditions	
Type	Alias	Name	7	7	1 Basic AND	2 Basic OR
		 u1 one_led_out[0..6]			AND	 OR
		 u1 one_led_out[0]			<u>0</u>	<u>0</u>
		 u1 one_led_out[1]			<u>0</u>	<u>1</u>
		 u1 one_led_out[2]			<u>0</u>	<u>0</u>
		  u1 one_led_out[3..6]			0h	 2h (XOR)

Select evaluation method between nodes within nested group

Select evaluation method between parent group nodes and result of nested group

AND / OR

AND

OR

NAND

NOR

☒ XOR

XNOR

TRUE

FALSE

SIGNAL CONFIGURATION

- Manages data capture and all other Signal Tap options
 - Sample clock
 - Sample depth
 - Buffer type
 - Storage qualification
 - Trigger flow
 - Trigger position
 - Trigger-in & trigger-out
 - Node allocation for Rapid Recompile (*discussed later*)

The screenshot shows the 'Signal Configuration' dialog box, which is used to configure the Signal Tap feature. The dialog is divided into two main sections: 'Data' and 'Trigger'.

Data Section:

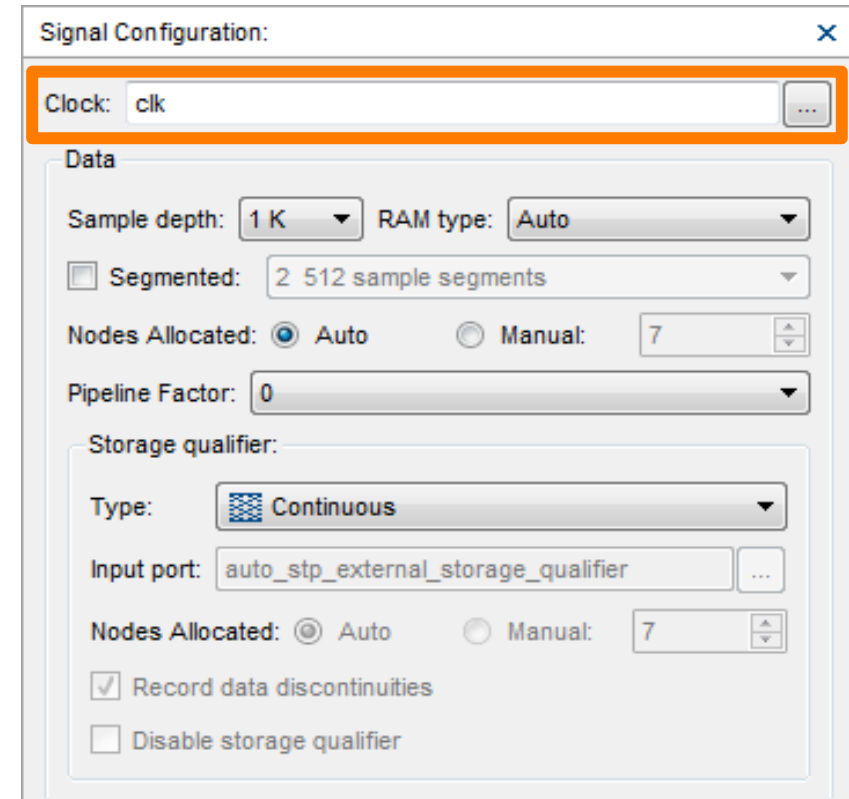
- Clock:** Set to 'clk'.
- Sample depth:** Set to '1 K'.
- RAM type:** Set to 'Auto'.
- Segmented:** Set to '2 512 sample segments'.
- Nodes Allocated:** Set to 'Auto' (radio button selected).
- Manual:** Set to '7' (spin box).
- Pipeline Factor:** Set to '0'.
- Storage qualifier:**
 - Type:** Set to 'Continuous'.
 - Input port:** Set to 'auto_stp_external_storage_qualifier'.
 - Nodes Allocated:** Set to 'Auto' (radio button selected).
 - Manual:** Set to '7' (spin box).
 - ☒ Record data discontinuities
 - ☐ Disable storage qualifier

Trigger Section:

- Nodes Allocated:** Set to 'Auto' (radio button selected).
- Manual:** Set to '7' (spin box).
- Trigger flow control:** Set to 'Sequential'.
- Trigger position:** Set to 'Center trigger position'.
- Trigger conditions:** Set to '2'.
- Trigger in:**
 - ☐ Pin:
 - ☒ Node:
 - Instance:** Set to 'instance_ten'.
 - ☐ Hard Processor System (HPS) trigger out
 - Pattern:** Set to '1 High'.
- Trigger out:**
 - ☐ Pin:
 - ☐ Instance:
 - ☐ Hard Processor System (HPS) trigger in
 - ☐ Hard Processor System (HPS) event: Set to '0'.
 - Level:** Set to 'Active High'.
 - Latency delay:** Set to '5 cycles'.

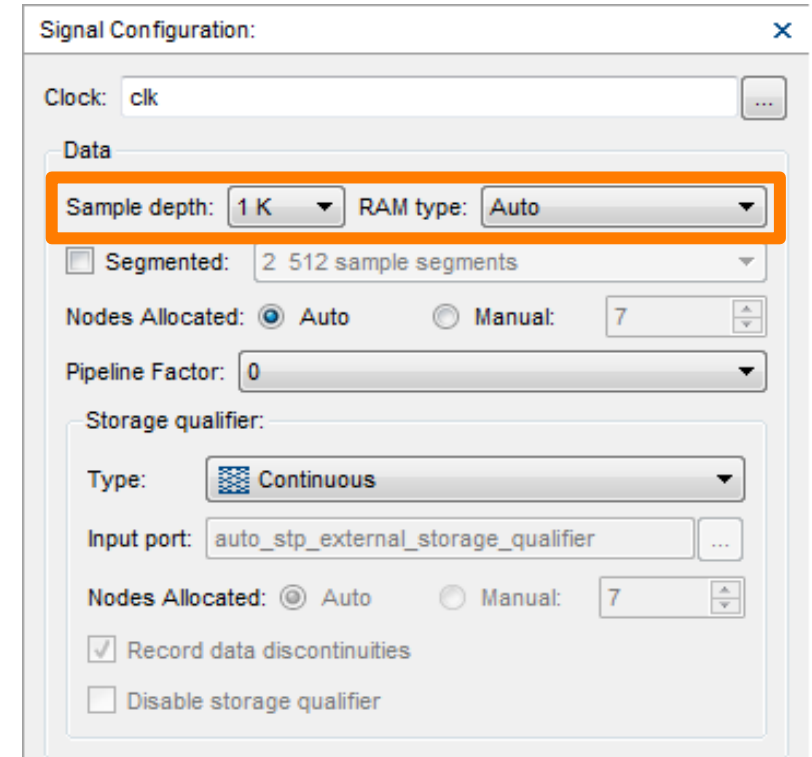
SAMPLING CLOCK

- Used to clock the Signal Tap logic
 - Use clock for domain being analyzed or global, fastest clock available for best results
 - Data captured on every rising edge
 - Clock signal itself cannot be monitored as data
 - External clock pin created automatically if clock unassigned
 - Named auto_stp_external_clk by default



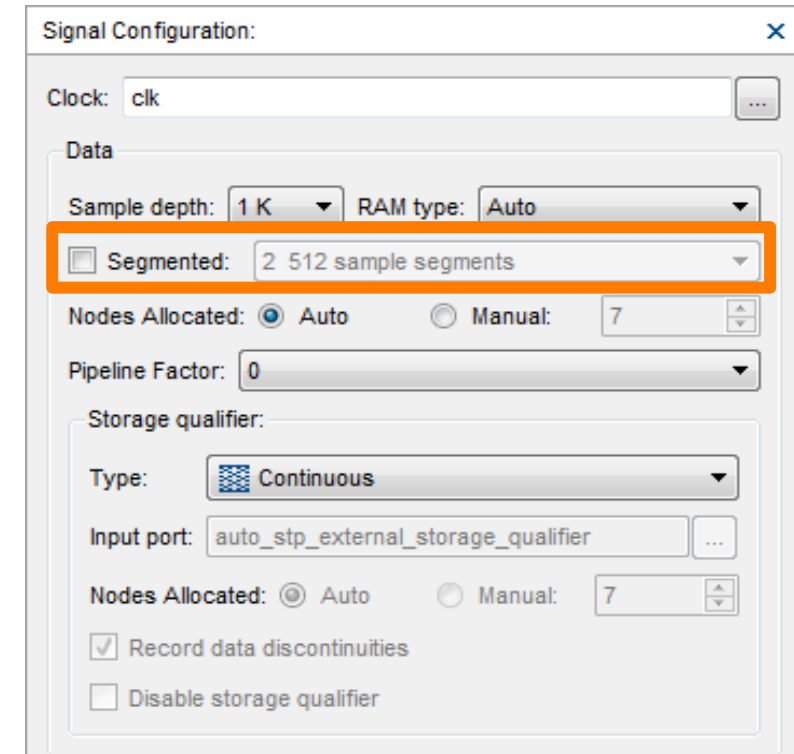
SAMPLE DEPTH & RAM TYPE

- Sample depth
 - Set number of samples to be stored for each data signal
 - 0 to 128K sample depth
- Select RAM type for supported FPGAs
 - Useful for reserving a specific memory type for debugging use
 - Defaults to **Auto** on unsupported device families



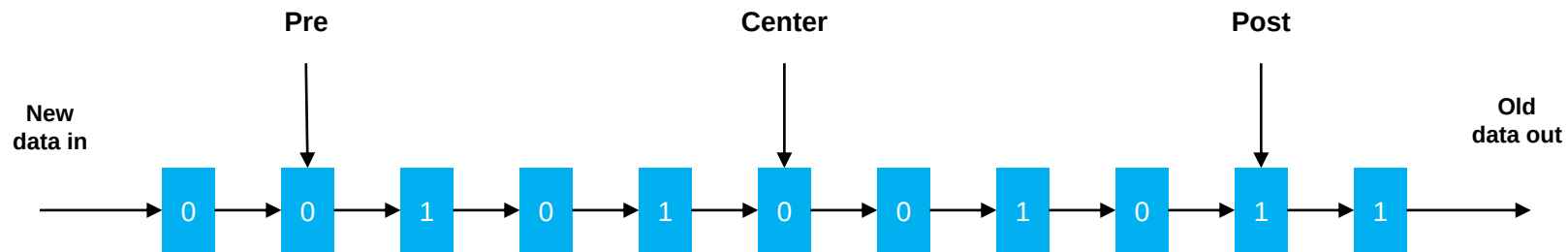
BUFFER TYPE

- Circular (default)
 - Segmented option turned off
- Segmented
 - Specify number of segments and segment sample depth



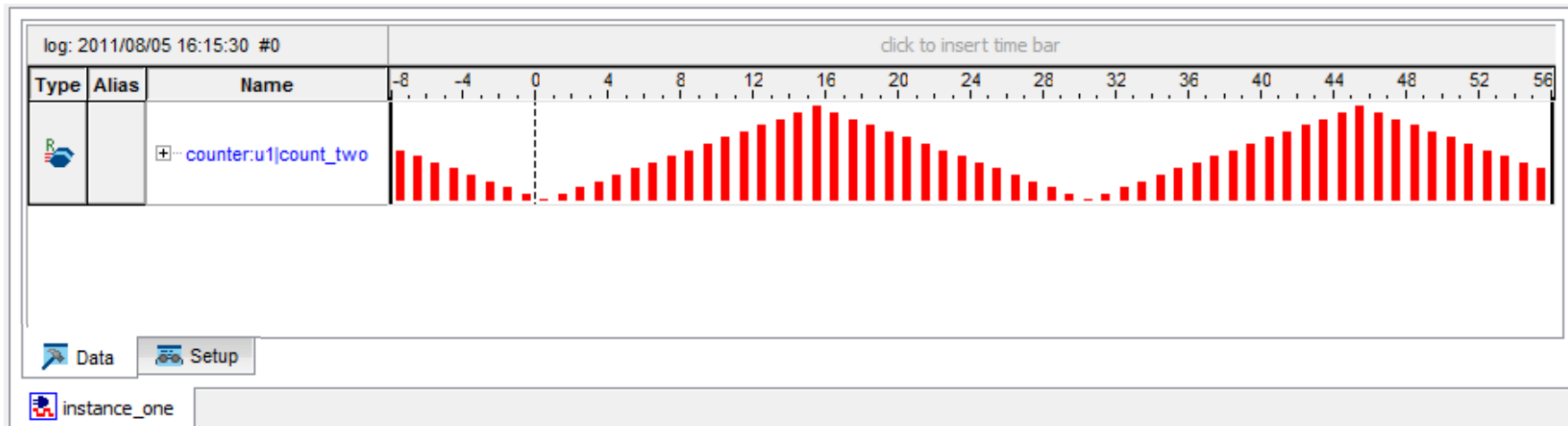
CIRCULAR BUFFER

- Data is circled (more like FIFO) through acquisition buffer until trigger event occurs
- After all defined trigger events occur, post-trigger data is collected until buffer fills up



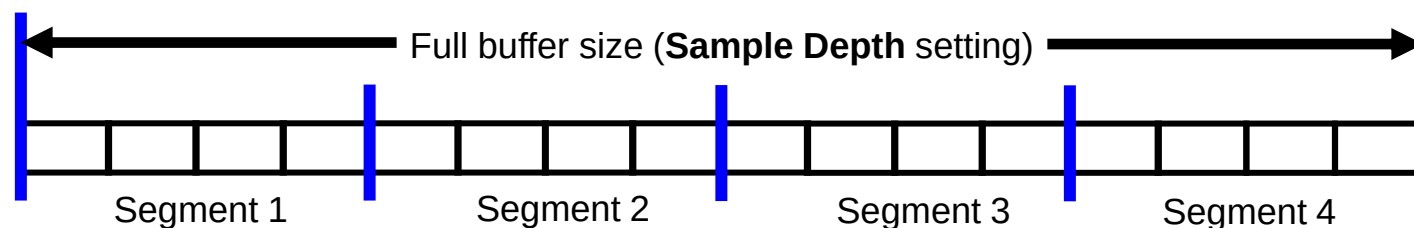
EXAMPLE: CIRCULAR BUFFER

Pre-trigger position;
data displayed as bar graph



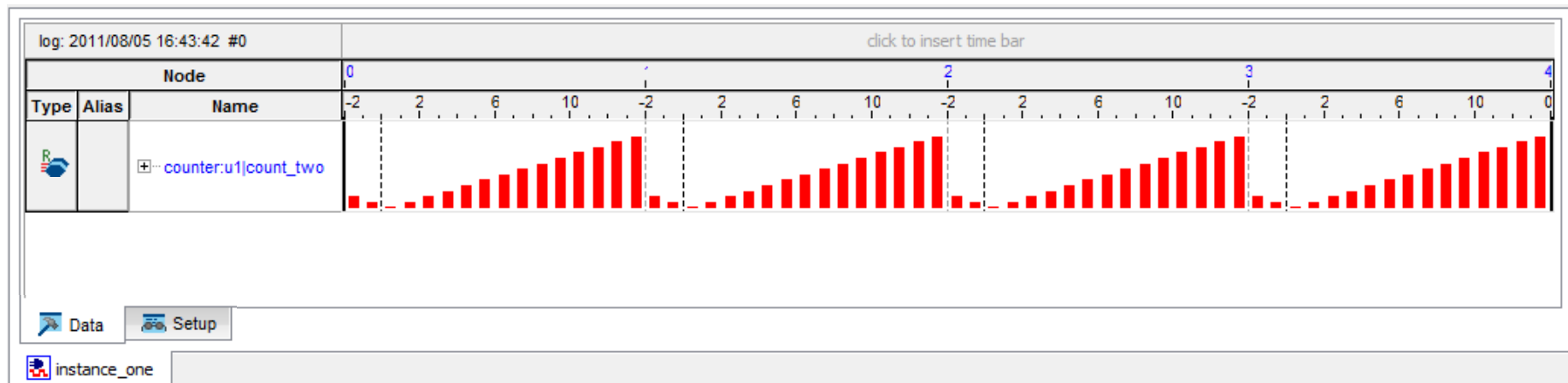
SEGMENTED BUFFER

- Buffer segmented into smaller, user-defined blocks
 - Each segment behaves like individual circular buffer
- Trigger position setting applies to each segment
- When trigger occurs, post-trigger data is collected until **single segment** fills up
- Process repeats with final trigger condition *only* until all segments are filled
- Use for recurring trigger events



EXAMPLE: SEGMENTED BUFFER

Four 16-sample segments;
data displayed as bar graph



TRIGGER POSITION

- Location in buffer (or buffer segment) where trigger sample located
 - Three preset positions
 - **Pre** (12% before trigger, 88% after)
 - **Center** (50% before, 50% after)
 - **Post** (88% before, 12% after)

Trigger

Nodes Allocated: ☒ Auto ☐ Manual: 7

Trigger flow control: Sequential

Trigger position: **Center trigger position**

Trigger conditions: 2

☐ Trigger in

☐ Pin:

☒ Node:

☐ Instance: instance_ten

☐ Hard Processor System (HPS) trigger out

Pattern: 1 High

☐ Trigger out

☐ Pin:

☐ Instance:

☐ Hard Processor System (HPS) trigger in

☐ Hard Processor System (HPS) event: 0

Level: Active High

Latency delay: 5 cycles

TRIGGER FLOW CONTROL

▪ Sequential

- Trigger conditions evaluated in numerical order
- Trigger in condition followed by conditions 1, 2, 3, etc.
- Buffer or 1st buffer segment filled on final condition
- Subsequent segments filled only when final condition reoccurs

▪ State-based

- Conditions evaluated in any order desired
- Controlled by user-defined state machine
- Counters and flags available for greater control

Trigger

Nodes Allocated: ☒ Auto ☐ Manual: 7

Trigger flow control: Sequential

Trigger position: Center trigger position

Trigger conditions: 2

☐ Trigger in

☐ Pin:

☒ Node:

☐ Instance: instance_ten

☐ Hard Processor System (HPS) trigger out

Pattern: 1 High

☐ Trigger out

☐ Pin:

☐ Instance:

☐ Hard Processor System (HPS) trigger in

☐ Hard Processor System (HPS) event: 0

Level: Active High

Latency delay: 5 cycles

TRIGGER CONDITIONS

- Create up to 10 trigger conditions for each SignalTap II instance
- Conditions appear as separate columns in Node List
- Order of analysis of conditions depends on flow control setting

The screenshot shows the 'Trigger' configuration window. At the top, 'Nodes Allocated' is set to 'Auto' and 'Manual' is set to '7'. 'Trigger flow control' is set to 'Sequential'. 'Trigger position' is set to 'Center trigger position'. The 'Trigger conditions' dropdown is highlighted with an orange box and shows the value '2'. Below this, the 'Trigger in' section is expanded, showing options for 'Pin', 'Node', 'Instance' (set to 'instance_ten'), and 'Hard Processor System (HPS) trigger out'. The 'Pattern' is set to '1 High'. The 'Trigger out' section is also expanded, showing options for 'Pin', 'Instance', 'Hard Processor System (HPS) trigger in', 'Hard Processor System (HPS) event' (set to '0'), 'Level' (set to 'Active High'), and 'Latency delay' (set to '5 cycles').

SIGNALTAP GUI VIEWS

The screenshot shows the SignalTap II Logic Analyzer interface. At the top, the title bar reads "SignalTap II Logic Analyzer - C:/altera_trn/Quartus_Debug/QD16_1/Ex2/top_counter - top_counter - [counter.stp]". The menu bar includes File, Edit, View, Project, Processing, Tools, Window, and Help. Below the menu bar is a toolbar with icons for file operations, analysis, and help.

The **Instance Manager** pane on the left lists two instances: **instance_one** and **instance_ten**, both with a status of "Not running". A yellow banner at the top of this pane displays the message "Invalid JTAG configuration". To the right, the **JTAG Chain Configuration** pane shows "No devices detected" in a red banner, with buttons for "Setup...", "Scan Chain", and "SOF Manager".

The main workspace is divided into several sections. The top section shows a trigger configuration for "2016/11/15 21:26:38 #0" with a lock mode of "Allow all changes". Below this is a table of nodes for the selected instance:

Type	Alias	Name	Data Enable	Trigger Enable	Trigger Conditions
Signal		counter:u1[one_led_out[0..6]	☒	☒	
Signal		counter:u1[one_led_out[0]	☒	☒	0
Signal		counter:u1[one_led_out[1]	☒	☒	0
Signal		counter:u1[one_led_out[2]	☒	☒	0

Below the node table are three tabs: **Data**, **Setup**, and **State-Based Trigger Flow**. The **Signal Configuration** pane on the right shows the clock set to "clk", a sample depth of "1 K", and "2 512 sample segments" segmented.

At the bottom left, the **Hierarchy Display** pane shows a tree structure with "top_counter" expanded to show "counter:u1". Below this, two buttons for "instance_one" and "instance_ten" are visible.

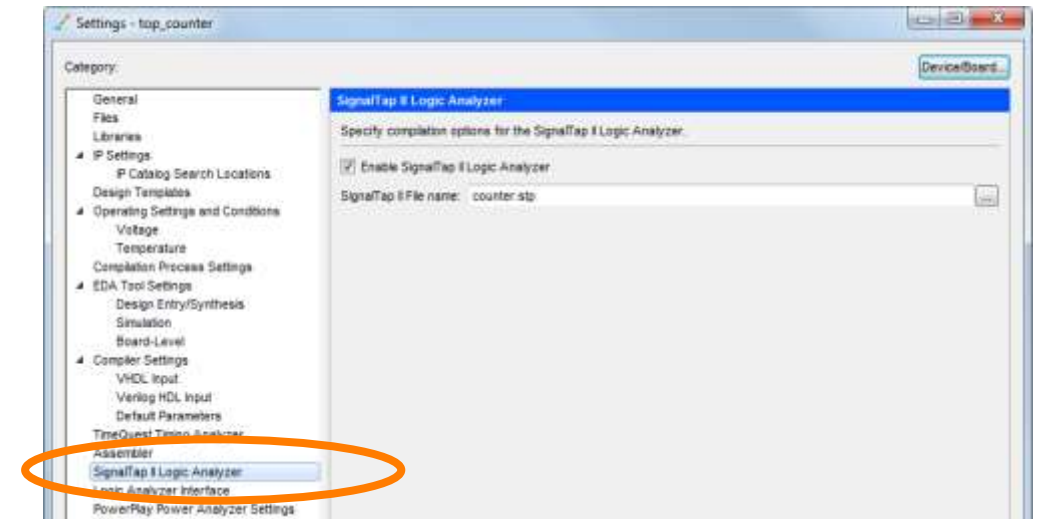
Annotations with arrows point to specific areas:

- A box labeled "Double-click in empty space to add nodes" points to the empty space above the node table.
- A box labeled "Setup and data analysis tabs" points to the "Data" and "Setup" tabs.
- A box labeled "Select instance to setup and run (or double-click in Instance Manager)" points to the "instance_one" and "instance_ten" buttons.

The bottom right corner of the window shows a progress bar at "0%" and a timer at "00:00:00".

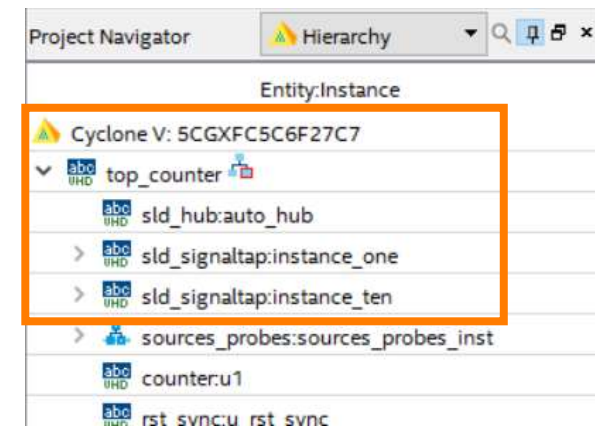
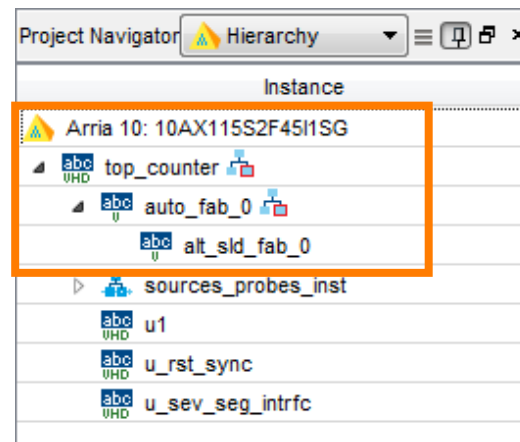
ENABLE & SPECIFY .stp FILE FOR PROJECT

- SignalTap II Logic Analyzer control in Quartus project settings
 - **Assignments** menu → **Settings**
 - Specify SignalTap II file to compile with project
 - May also enable by saving .stp file and agree to enable in project



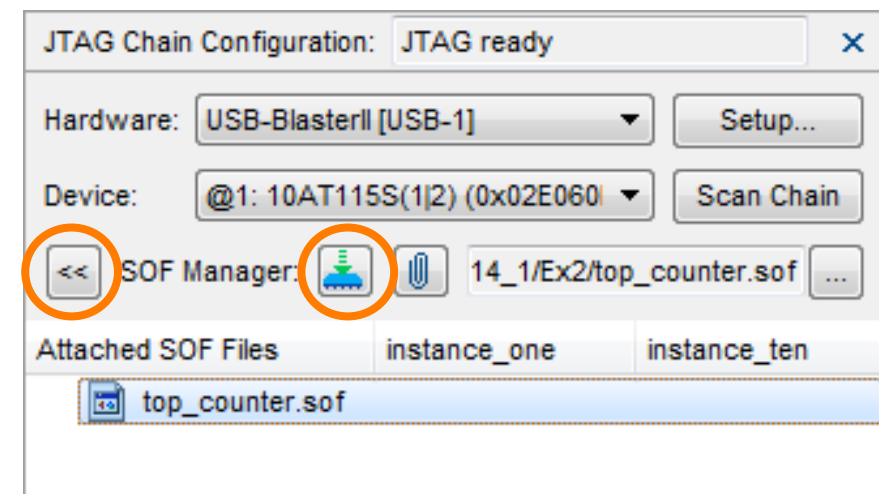
COMPILE PROJECT WITH SIGNAL TAP

- Full compilation (**Processing** menu → **Start Compilation**)
- Logic added to project to implement logic analyzer and JTAG hub connection
 - sld_hub
 - sld_signaltap

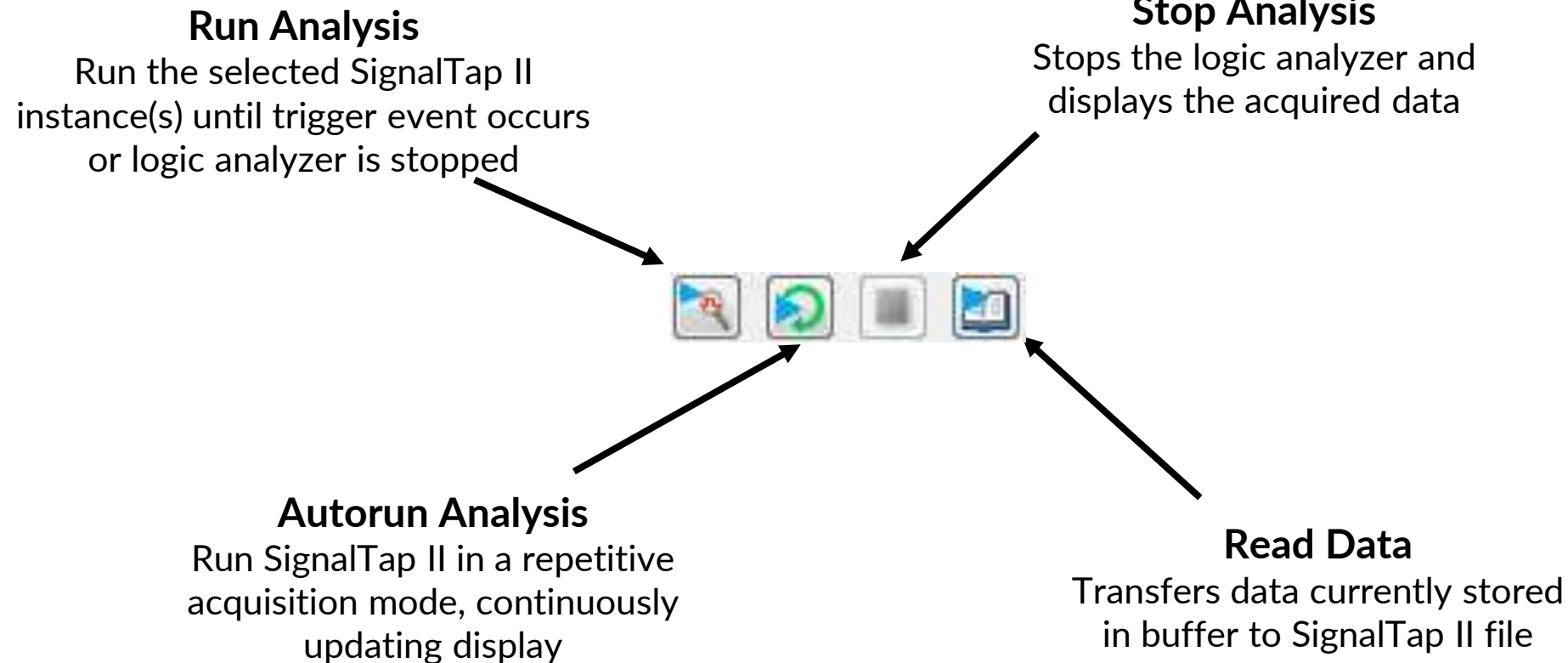


PROGRAM DEVICE(S)

- Use Quartus Prime Programmer or SignalTap II file
 - Program button in the SignalTap II interface configures the selected device in chain
 - Use Quartus Prime Programmer to program multiple devices
- Guarantee **.sof/.stp** compatibility by attaching **.sof** in **SOF Manager**

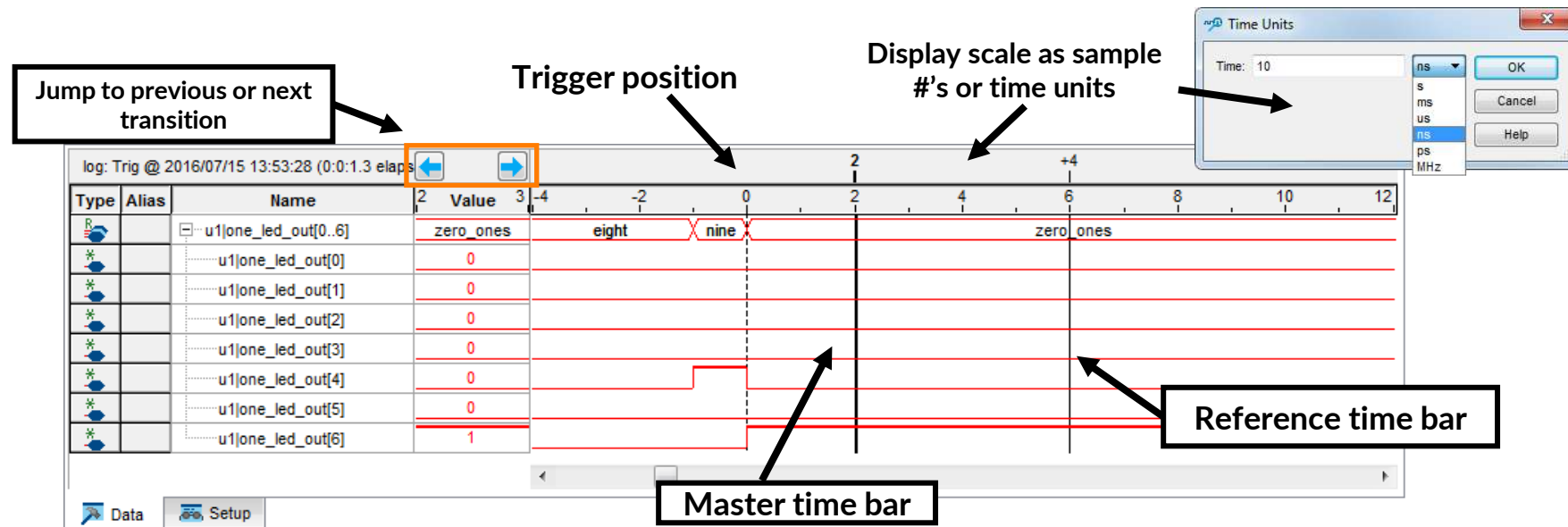


RUNNING & CONTROLLING THE SIGNAL TAP



DISPLAYING ACQUIRED DATA

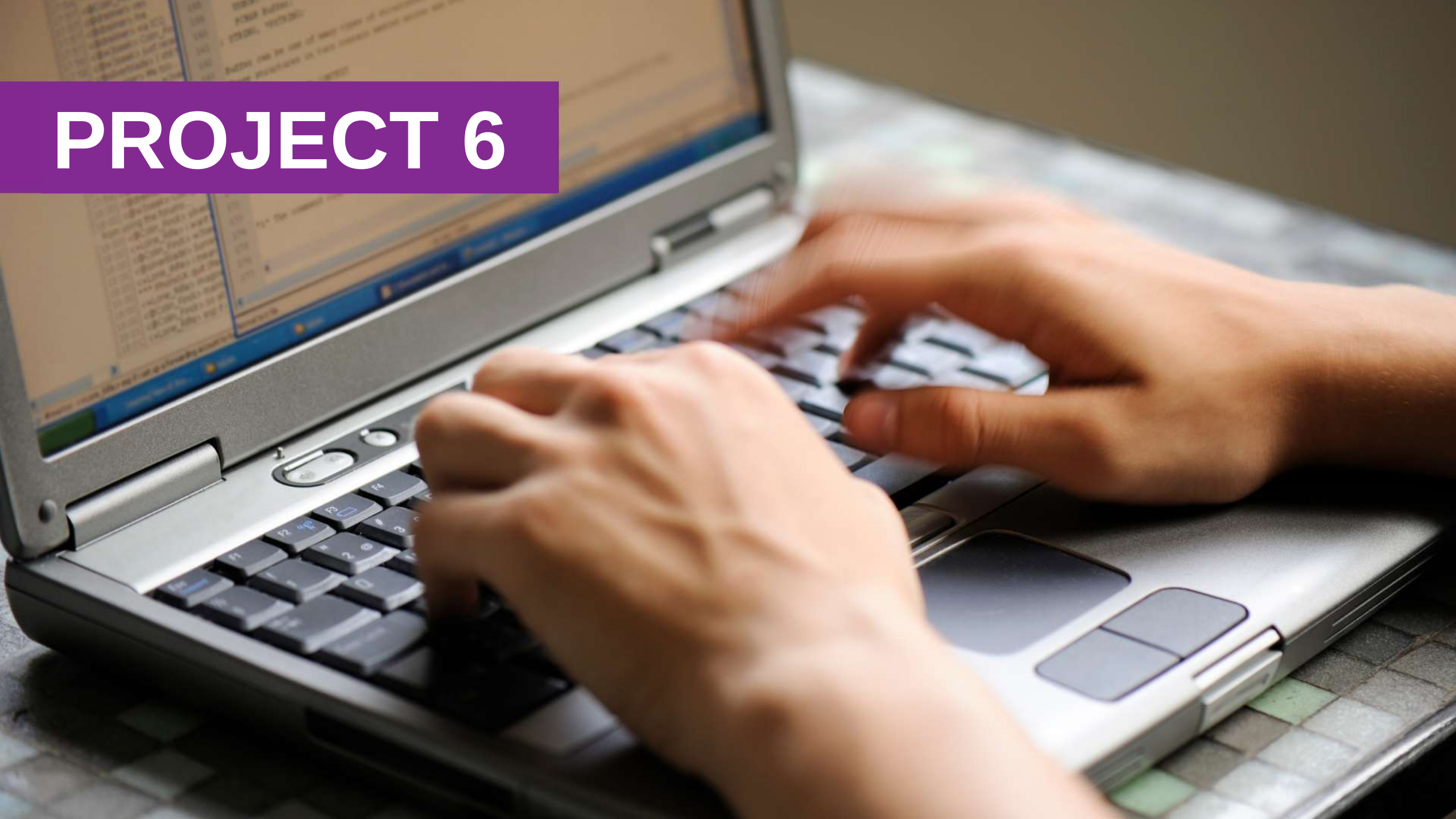
- Captured data displayed in the Data tab
 - Display signal groups as standard waveforms in selected radix or as a bar or line chart (right-click group on **Data** tab)



USING .stp FILE (REVIEW)

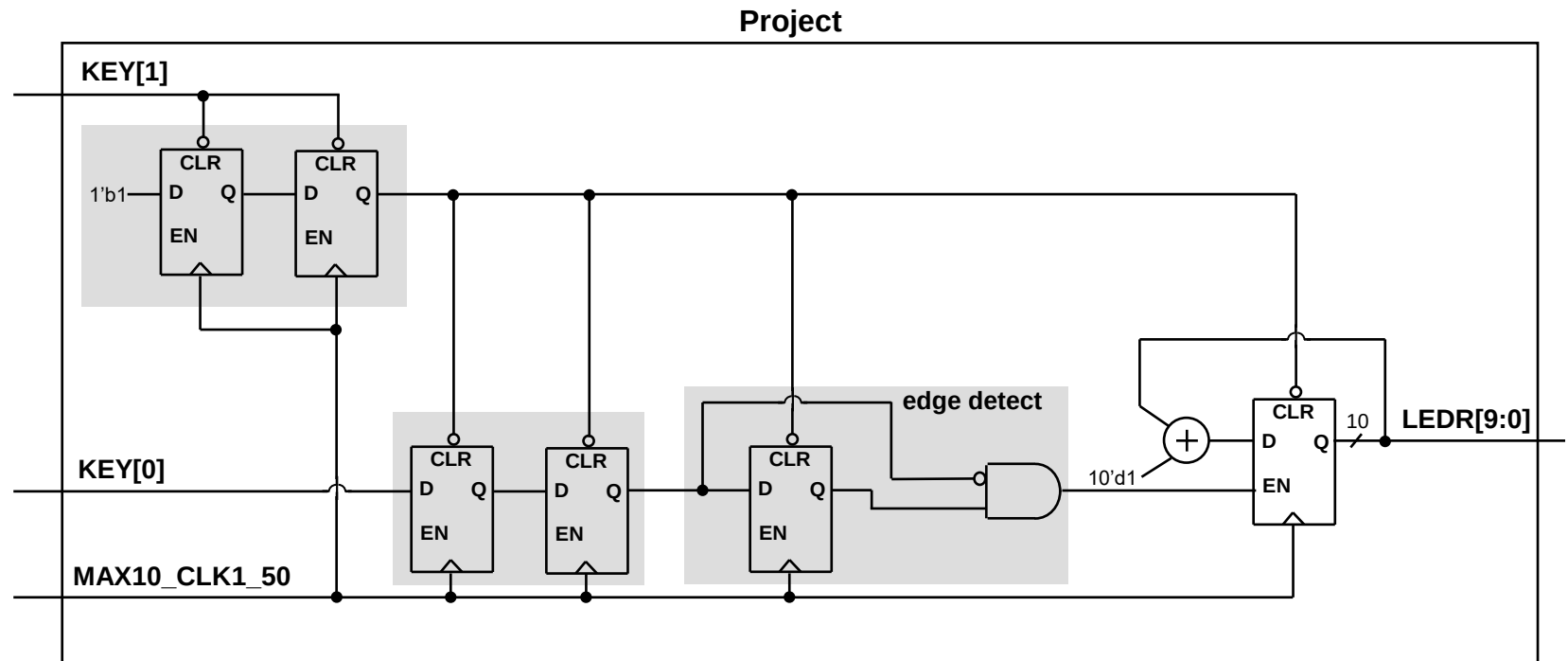
1. Create .stp file with Signal Tap GUI
 - Assign sample clock
 - Specify sample depth
 - Tap signals to analyze
 - Specify triggering conditions and flow
 - Setup JTAG
2. Save .stp file & compile with design
3. Program device
4. Acquire data

PROJECT 6



PROJECT 6

- Open the projeto-06 and change the counter to 10-bits
- Connect the 10 output bits into the 10 LEDs
- Create an edge detector which generates a single enable pulse
- Check the final result in the Signal Tap



VERILOG:

HIERARCHICAL DESIGN: COMPONENT INSTANCE

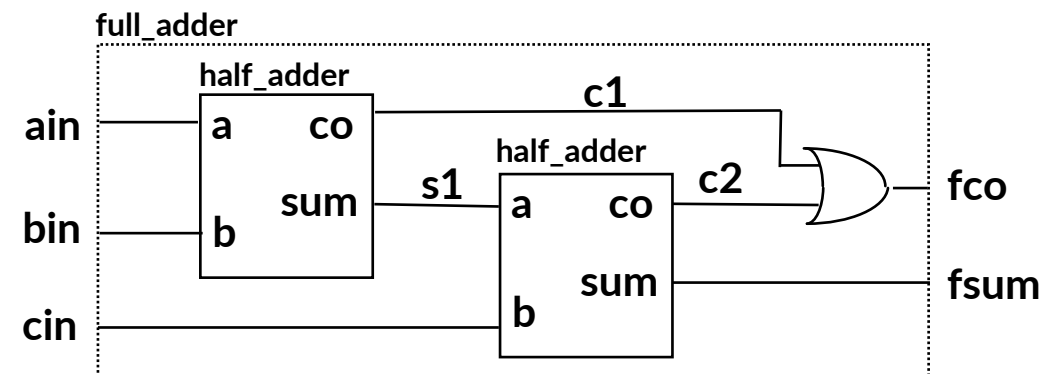


CONNECTING MODULE INSTANTIATION PORTS

- Module declaration defines input, output and bidirectional ports
- Ports from the top-level module are connected directly to the device pins
- Top level module can **instantiate** lower level modules
- Ports from lower level modules can be connected to device pins, to other modules pins or to other logic in the higher level module

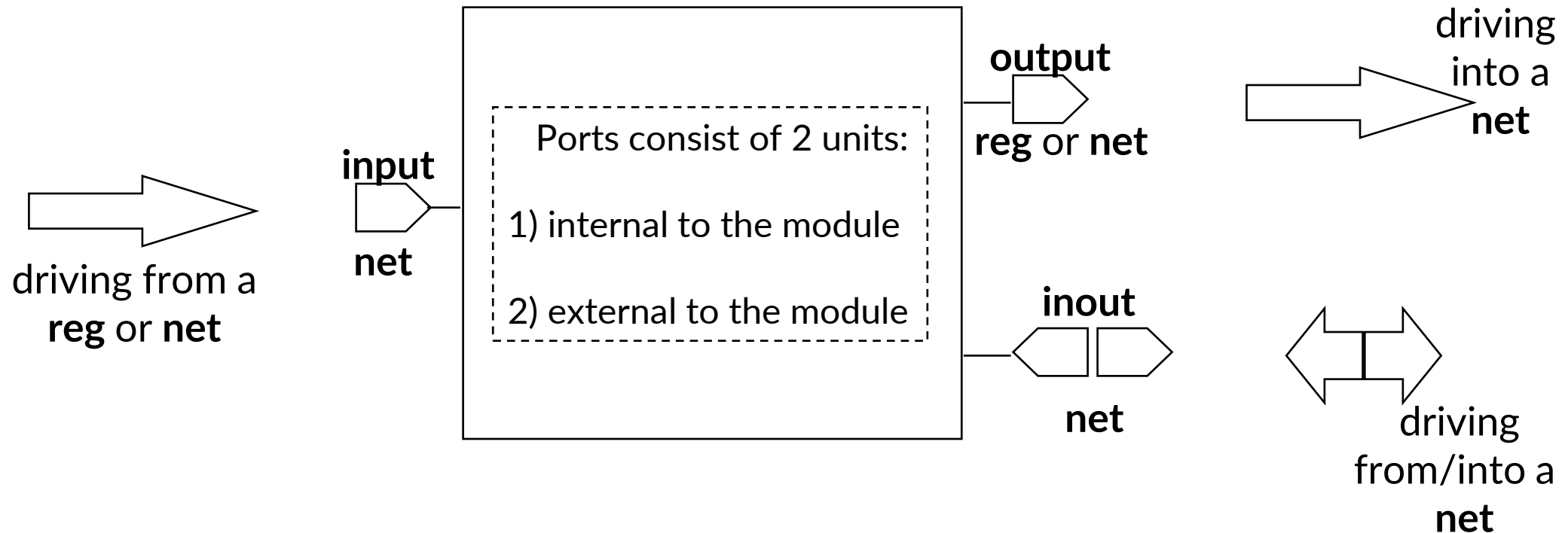
```
module full_adder (
    output fco, fsum,
    input ain, bin, cin
);

module half_adder (
    output co, sum,
    input a, b
);
```



PORT CONNECTION RULES

These are the requirements for port connections when modules are instantiated within other modules



CONNECTING MODULE INSTANTIATION PORTS

```
module full_adder (
    output fco, fsum,
    input ain, bin, cin
);
```

```
    half_adder u1 (
        .co (c1),
        .sum (s1),
        .a (ain),
        .b (bin)
    );
```

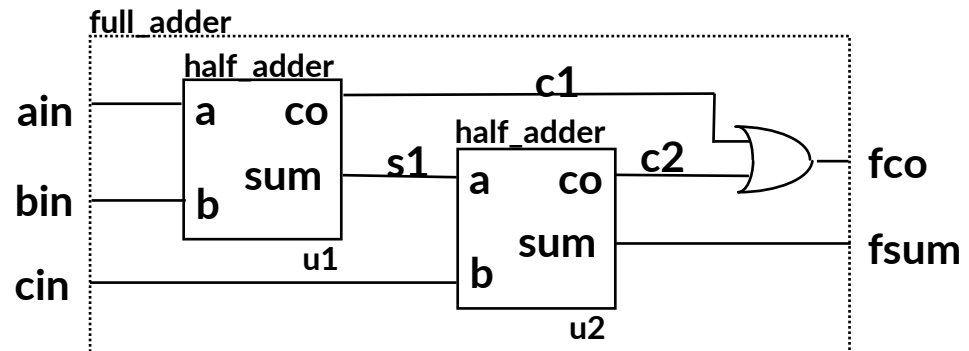
```
    half_adder u2 (
        c2, fsum, s1, cin
    );
```

```
    assign fco = c1 | c2;
```

```
endmodule
```

```
module half_adder (
    output co,
    output sum,
    input a,
    input b
);
```

Lower level module ports on the left, preceded by dot (.)
Local variables and nets on the right, between parenthesis



Positional connections are valid but not recommended

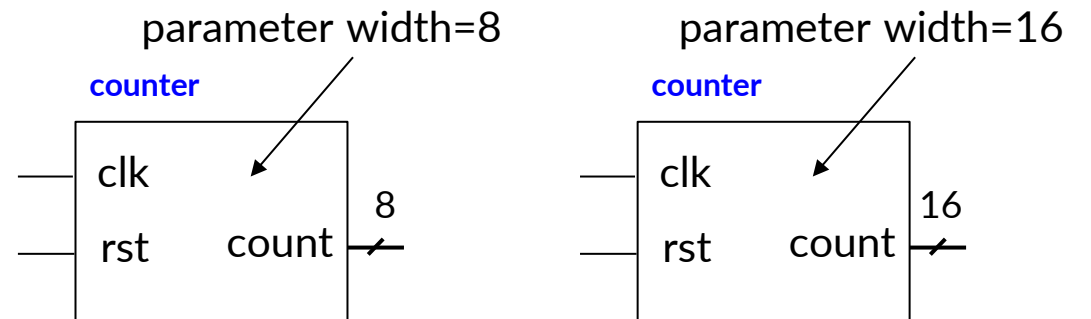
VERILOG:

COMPONENTS
PARAMETERIZATION



PARAMETERS

- Can be used to make reusable modules
- Must be resolved to a constant at compile time
- Each module instance can have different parameter values
- Module interface and behavior can be affected by parameter values



MODULE / PORT / PARAMETER DECLARATION

Module, port and parameter declarations can be combined

```
module counter
#(
    parameter width = 8,
    parameter modulus = 100
)
(
    input clk, rst,
    output [width-1:0] count
);
```

Parameter name

Parameter default value

*If parameter value is not assigned
on module instantiation, the default
value is used.*

PARAMETER DECLARATION AND DEFAULT VALUE

```
module counter
#(
    parameter width = 8,
    parameter modulus = 100
)
(
    input clk, rst,
    output reg [width-1:0] count
);

    always @ (posedge clk, posedge rst) begin
        if (rst)
            count <= { width{1'b0} };
        else
            begin
                count <= count + { {width-1{1'b0}}, 1'b1 };
                if (count == modulus-1)
                    count <= { width{1'b0} };
            end
        end
    end
endmodule
```

MODULE INSTANCE PARAMETER ASSIGNMENT

Parameter definition occurs during module instantiation

```
module two_counter (
    input clock_50,
    input reset,
    output [3:0] counter10,
    output [6:0] counter100
);
```

```
    counter #(
        .width (4),
        .modulus (10)
```

```
    ) u1 (
        .clk (clock_50),
        .rst (reset),
        .count (counter10)
    );
```

```
    counter #(7, 100)
        u2 (clock_50, reset, counter100);
```

```
endmodule
```

To just use the (default) value defined in the lower-level module, in the name method, use empty value (e.g. .width()) or leave parameter assignment out entirely

Recommended parameter assignment syntax, similar to port connections

Positional parameter assignment is valid but not recommended

QUARTUS:

**INTEGRATED
COMPONENTS:
IP CATALOG**



INTEL / ALTERA IP CORES

- Pre-made design blocks
- Benefits
 - Configurable, parameterized settings add flexibility & portability
 - “Drop-in” support to accelerate design entry
 - Pre-optimized for Altera architecture
- Many basic non-encrypted functions included for free in web and subscription editions
- More advanced IP available in two forms
 - IP Base Suite
 - MegaCore IP

IP BASE SUITE

- Free & installed with the Quartus Prime software
 - License required for Lite edition
 - Otherwise operate in OpenCore® Plus mode (*discussed in a moment*)
 - HDL simulation models installed in Quartus Prime libraries
- Included in IP Base Suite
 - FIR Compiler
 - Numerically Controlled Oscillator
 - Fast Fourier Transform Compiler
 - AXI BFM (bus functional models) for simulation
 - DDR2/3/QDRII/LPDDR2/RLDRAMII SDRAM Controllers with UniPHY
 - See <https://www.intel.com/content/www/us/en/programmable/products/intellectual-property/design/ip-base-suite.html> for details

IP MEGACORES

- Must purchase license (except IP Base Suite)
 - Logic for IP function is encrypted
- Two types
 - MegaCore IP: developed by Altera
 - Altera Megafunctions Partner Program (AMPP) IP
- All MegaCore functions & some AMPP functions support OpenCore Plus feature
 - Develop design using free version of core
 - HDL simulation models provided with IP
 - Generate time-limited, JTAG-tethered configuration/programming files
 - See [AN320: OpenCore Plus Evaluation of Megafunctions](#)

BEST-IN-CLASS MEGACORE IP EXAMPLES

- Triple-Speed Ethernet MAC
- 10Gb Ethernet MAC
- CRC Compiler
- IP Compiler for PCI Express® soft core
 - Note: Hard IP for PCI Express does not required a license
- Video and Image Processing Suite

See <https://www.intel.com/content/www/us/en/products/programmable/intellectual-property.html> for a complete list of Intel/Altera IP solutions

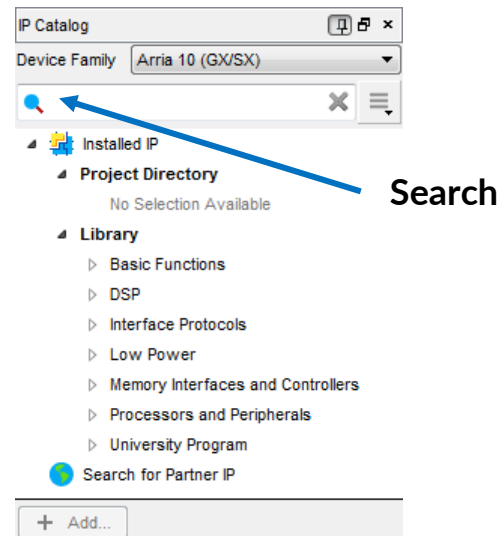
IP CATALOG

- Centralized tool for the implementation, configuration, & generation of IP cores optimized for a specific device family
- Includes listing of built-in and custom IP

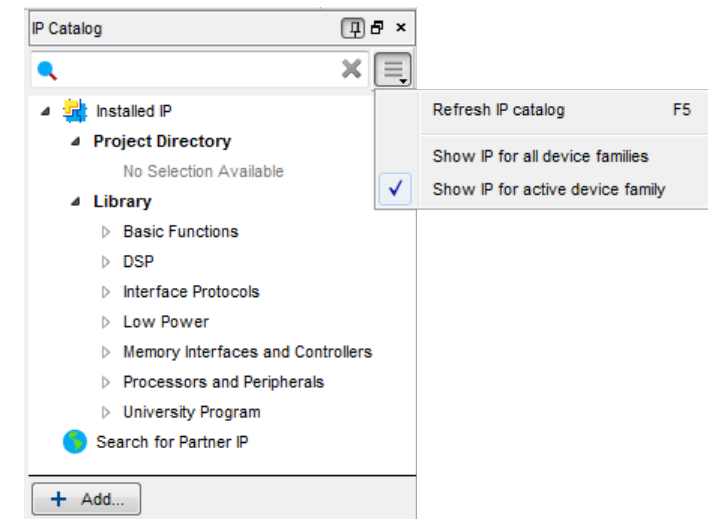


Tools menu → IP Catalog or
View menu → Utility Windows → IP Catalog

No project open
Generate IP for any device family

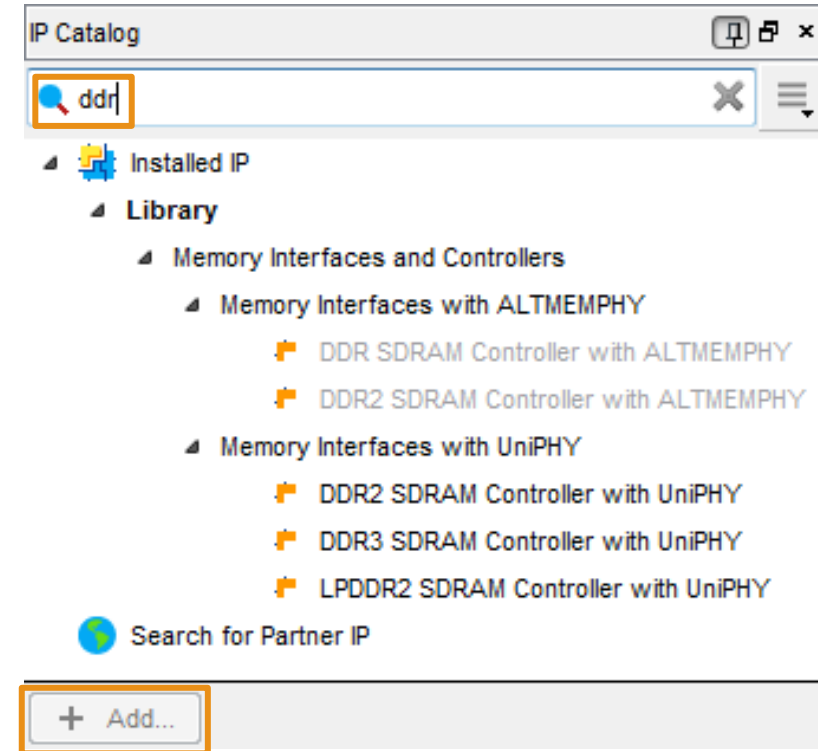


Project open
Generate IP for project device or any device family



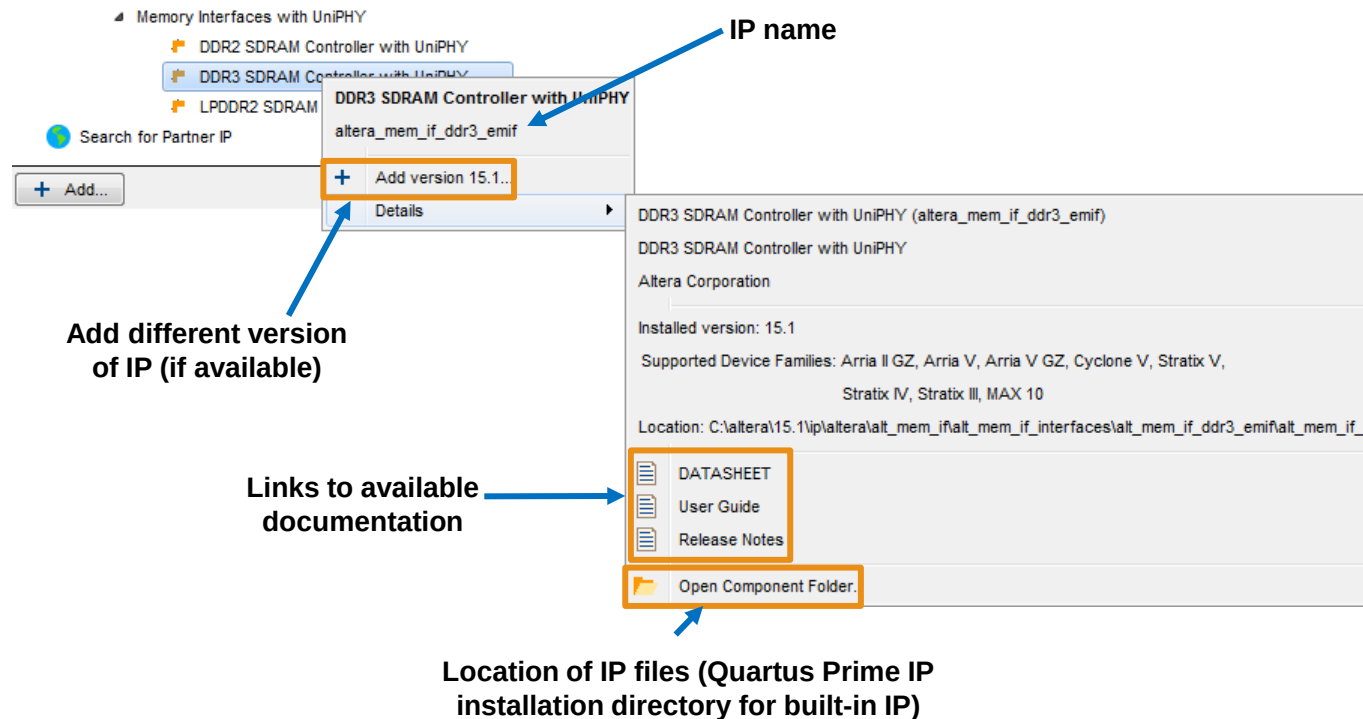
USING THE IP CATALOG

1. Select IP to create and click **Add**, or double-click IP
2. Name IP (for output file names)
3. Specify IP parameters
4. Generate IP files



IP OPTIONS AND DOCUMENTATION

Right-click IP in IP Catalog to add different versions or get help and documentation



CREATING AN IP VARIATION WITH THE IP PARAMETER EDITOR

- For all IP cores in the Quartus Prime Pro Edition (and some in Standard Edition)
- For Arria 10 and newer devices

Set top-level IP name and output file location (.qsys file and others)

The screenshot shows the 'New IP Variation' dialog box. It has a title bar with a close button. The main area contains the following fields and sections:

- A message: "Your IP settings will be saved in a .qsys file."
- A section titled "Create IP Variation" with two input fields:
 - "Entity name:" with the text "mult" entered.
 - "Save in folder:" with the path "C:\altera_trn\Quartus_Prime_Software_Foundation\QPF16_0\Verilog" and a browse button (three dots).
- A section titled "Target Device" with two dropdown menus:
 - "Family:" set to "Arria 10".
 - "Device:" set to "10AX115S2F45I1SG".
- An information bar at the bottom: "Info: Your system will be saved in C:\altera_trn\Quartus_Prime_Software_Foundation\QPF16_0\Verilog\mult.qsys." with an "OK" button.

IP PARAMETER EDITOR

Generates a Qsys system for each parameterized IP core saved as .qsys file

The screenshot displays the IP Parameter Editor interface for a dual_ram IP core. The main window is titled "IP Parameter Editor - dual_ram* (C:\altera_tm\Quartus_Prime_Software_Foundation\QPF16_0\Verilog\dual_ram.qsys)". The "Parameters" tab is active, showing the "RAM: 2-PORT" configuration. The "Operation Mode" is set to "With one read port and one write port". The "Type" is set to "As a number of words". The "Block Symbol" tab shows a block diagram of the ram_2port_0 block with inputs data[7..0], wraddress[4..0], rdaddress[4..0], wren, clock, and outputs datain, dataout, and q[7..0]. The "Presets" tab shows a list of presets for the RAM: 2-PORT 16.0, with a "Select available presets (if any)" button. The "System Messages" tab shows a table of messages:

Type	Path	Message
Info	2 Info Messages	
Info	dual_ram.ram_2port_0	Targeting device family: Arria 10.
Info	dual_ram.ram_2port_0	Tab Output2 is unavailable while using one read port and one write port.

The "Generate HDL..." button is highlighted with an orange box. The status bar at the bottom indicates "0 Errors, 0 Warnings".

Parameter options

Block diagram with signals/interfaces

Messages

Select available presets (if any)

IP PARAMETER EDITOR: GENERATE HDL

Choose to generate files for synthesis and/or simulation

The screenshot shows the 'Generation' dialog box in the IP Parameter Editor. It is divided into three main sections: Synthesis, Simulation, and Output Directory.

- Synthesis:** This section explains that synthesis files are used to compile the system in a Quartus Prime project. It includes a dropdown menu for 'Create HDL design files for synthesis:' set to 'Verilog'. There are two checkboxes: 'Create timing and resource estimates for third-party EDA synthesis tools.' (unchecked) and 'Create block symbol file (.bsf)' (checked).
- Simulation:** This section explains that the simulation model contains generated HDL files for the simulator and may include simulation-only features. It states that simulation scripts will be generated in a vendor-specific sub-directory. It provides guidance on using `ip-setup-simulation` and `ip-make-simscript` command-line utilities. The 'Create simulation model:' dropdown is set to 'None'.
- Output Directory:** This section shows the 'Path:' field set to `C:/altera_trn/Quartus_Prime_Software_Foundation/QPF16_0/Verilog/mult`. It notes that the path is fixed relative to the .qsys file. There is an unchecked checkbox for 'Clear output directories for selected generation targets.'

At the bottom right, there are 'Generate' and 'Cancel' buttons.

OUTPUT FILES

- Top-level folder files

- `<IP_name>.qsys`
 - Main Qsys system file; can be opened using IP Parameter Editor or Qsys system design tool
- `<IP_name>.sopcinfo`
 - XML file describing IP used for software development tools

- Located in `<IP_name>` folder

- `<IP_name>.qip`
 - Script file that adds all files needed for synthesis to Quartus Prime project
- `<IP_name>.bsf`
 - Symbol file for Quartus Prime schematic editor (optional)
- `<IP_name>.html`
 - HTML generation report
- Instantiation templates

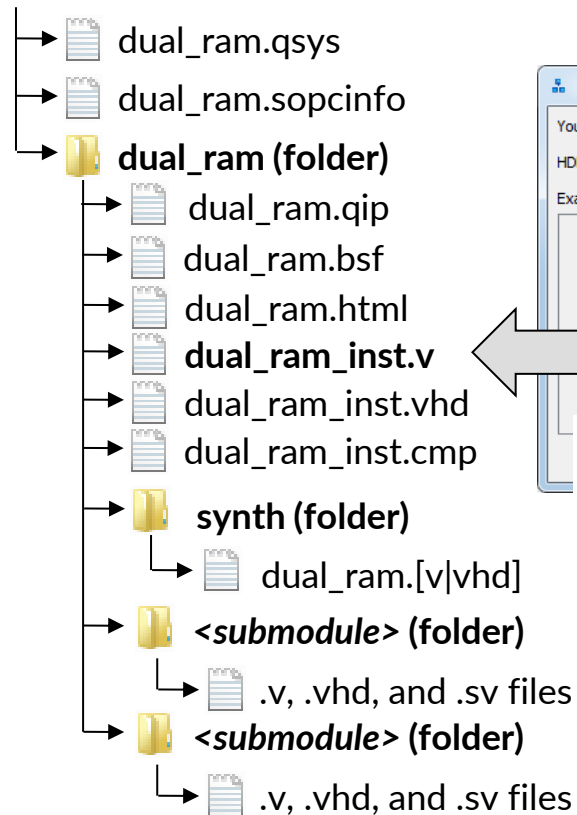
- Located in `<IP_name>/synth` folder

- `<IP_name>.[v|vhd]`
 - IP core top-level file connecting all components together
- Submodule library folders for synthesis
 - Unique folder for each submodule (if any)
 - Combination of Verilog, SystemVerilog, and/or VHDL files representing submodule

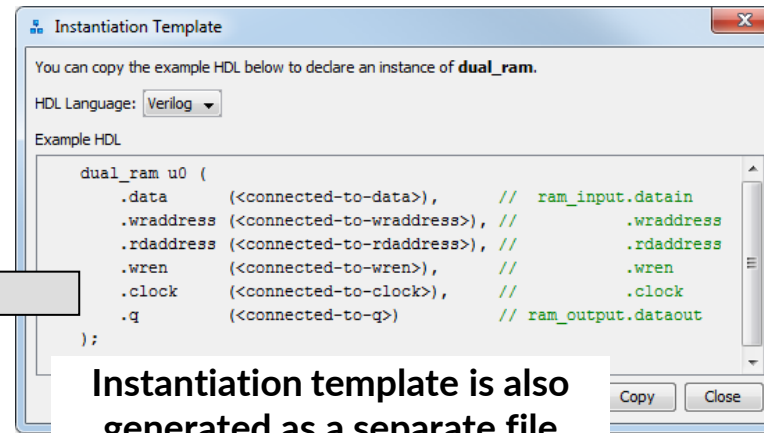
- Files for simulation (*optional*)

IP PARAMETER EDITOR OUTPUT FILES EXAMPLE

project (folder)



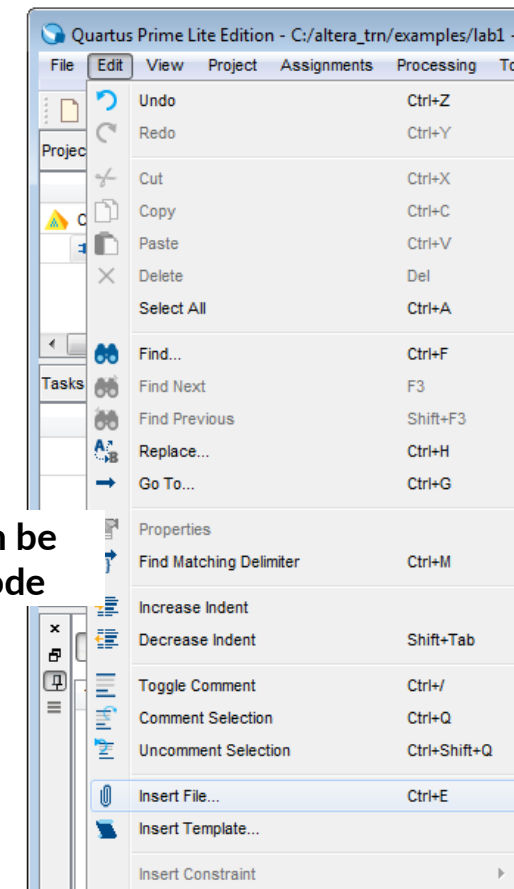
IP Parameter Editor **Generate** menu, **Show Instantiation Template**



Instantiation template is also generated as a separate file

Instantiation example file can be inserted directly into user code

Quartus Editor **Edit** menu, **Insert File**



QUARTUS:

INTEGRATED
COMPONENTS:
IP CATALOG

MEMORY
INITIALIZATION



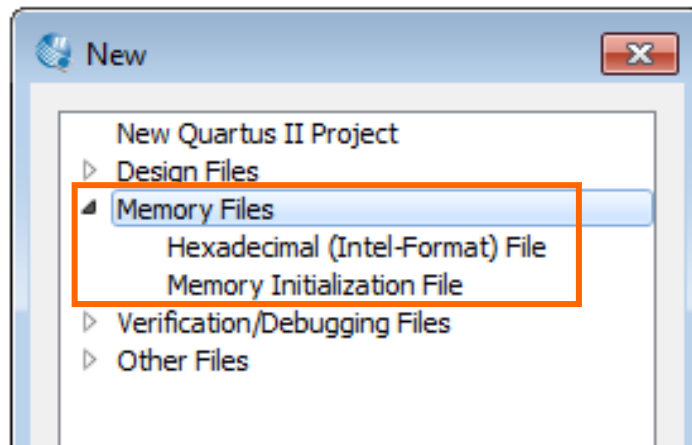
MEMORY EDITOR

- Create or edit memory initialization files
 - Intel HEX (.hex)
 - Preferred and compatible with 3rd party tools
 - Altera-specific (.mif) format
- Design entry
 - Initialization file data sent to device during device programming to initialize memory blocks
- Simulation
 - Use to initialize memory blocks before simulation or after breakpoints

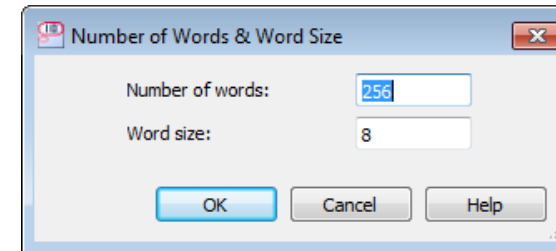
CREATE MEMORY INITIALIZATION FILE

File → New or Tasks window

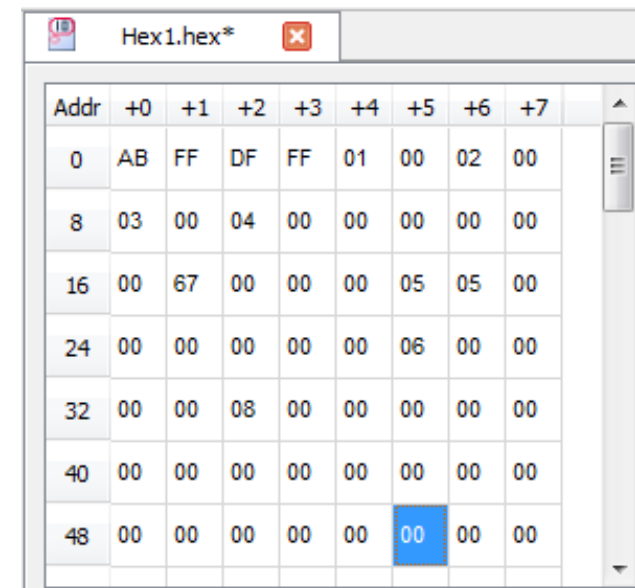
1) HEX or MIF format



2) Select memory size



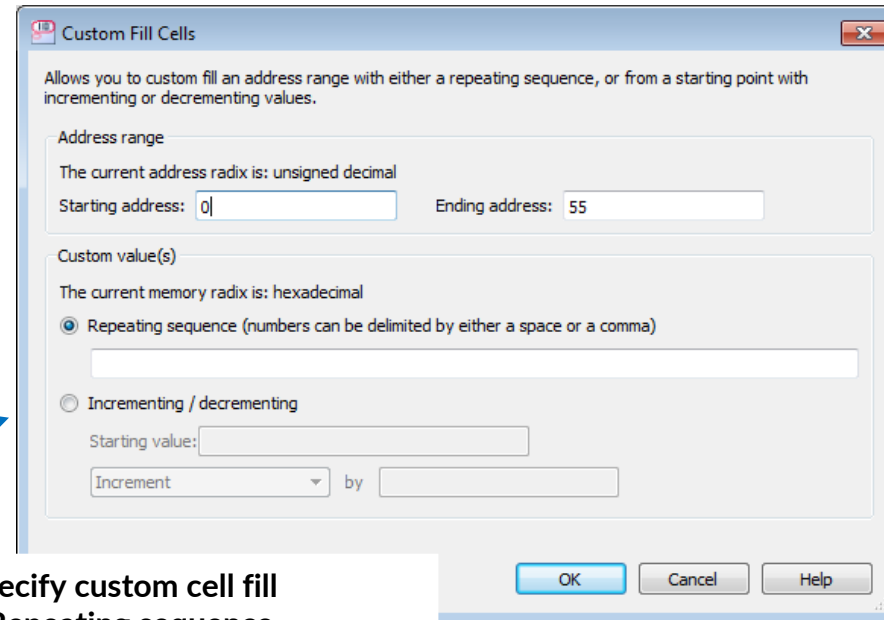
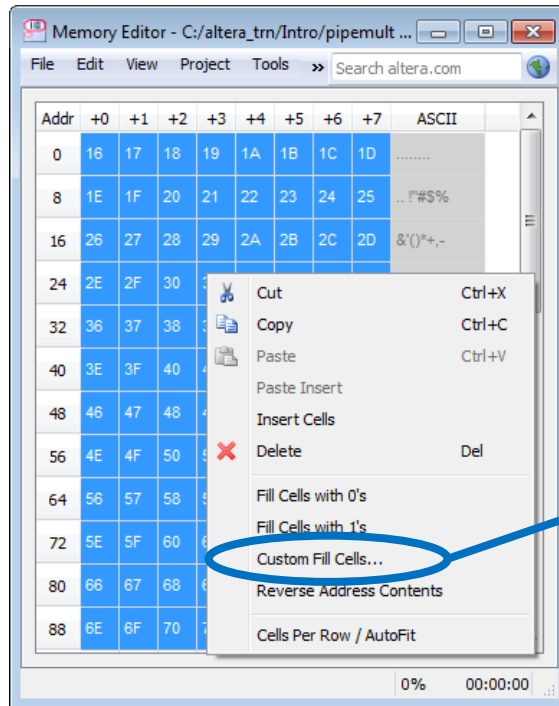
3) Memory space editor opens



EDIT CONTENTS

Edit contents of memory file
Save memory file as **.hex** or **.mif** file

- Select address location & type in a value
- Select the address & right-click to select fill option from menu
- Copy & paste from spreadsheet



Specify custom cell fill

- Repeating sequence
- Increment/decrement count

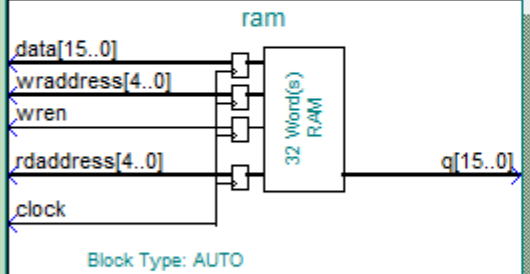
USING MEMORY FILE IN DESIGN

MegaWizard Plug-In Manager [page 8 of 10]

RAM: 2-PORT

1 Parameter Settings 2 EDA 3 Summary

General > Widths/Blk Type > Clks/Rd, Byte En > Regs/Clocks/Adrs > Output1 > **Mem Init**



Block Type: AUTO

Do you want to specify the initial content of the memory?

☐ No, leave it blank

☐ Initialize memory content data to XX..X on power-up in simulation

☒ Yes, use this file for the memory content data
(You can use a Hexadecimal (Intel-format) File [.hex] or a Memory Initialization File [.mif])

Browse...

File name: ram.hex

The initial content file should conform to which port's dimensions? PORT_B

Specify .mif or .hex file in
IP Parameter Editor or MegaWizard
Plug-In Manager

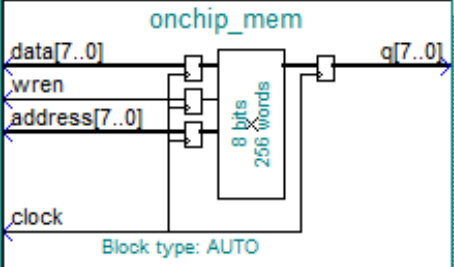
May also specify .mif file in HDL using the
ram_init_file synthesis attribute

USING MEMORY FILE IN DESIGN

MegaWizard Plug-In Manager [page 6 of 6]

RAM: 1-PORT

1 Parameter Settings 2 EDA 3 Summary



Turn on the files you wish to generate. A gray checkmark indicates a file that is automatically generated, and a green checkmark indicates an optional file. Click Finish to generate the selected files. The state of each checkbox is maintained in subsequent MegaWizard Plug-In Manager sessions.

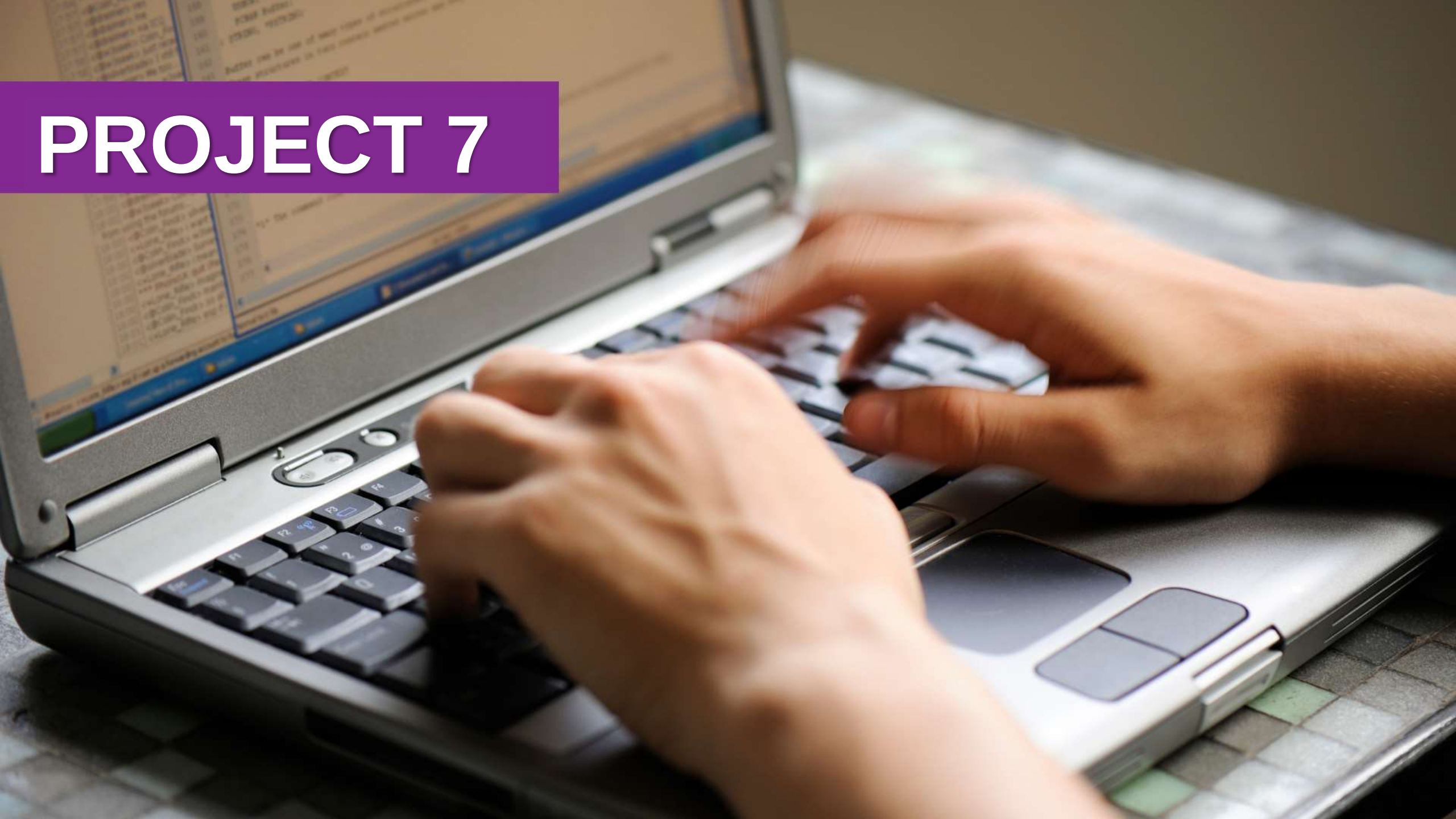
The MegaWizard Plug-In Manager creates the selected files in the following directory:
C:\altera_trn\examples\

File	Description
☒ onchip_mem.v	Variation file
☐ onchip_mem.inc	AHDL Include file
☐ onchip_mem.cmp	VHDL component declaration file
☐ onchip_mem.bsf	Quartus Prime symbol file
☒ onchip_mem_inst.v	Instantiation template file
☐ onchip_mem_bb.v	Verilog HDL black-box file

Enables the generation of an "Instantiation template file"

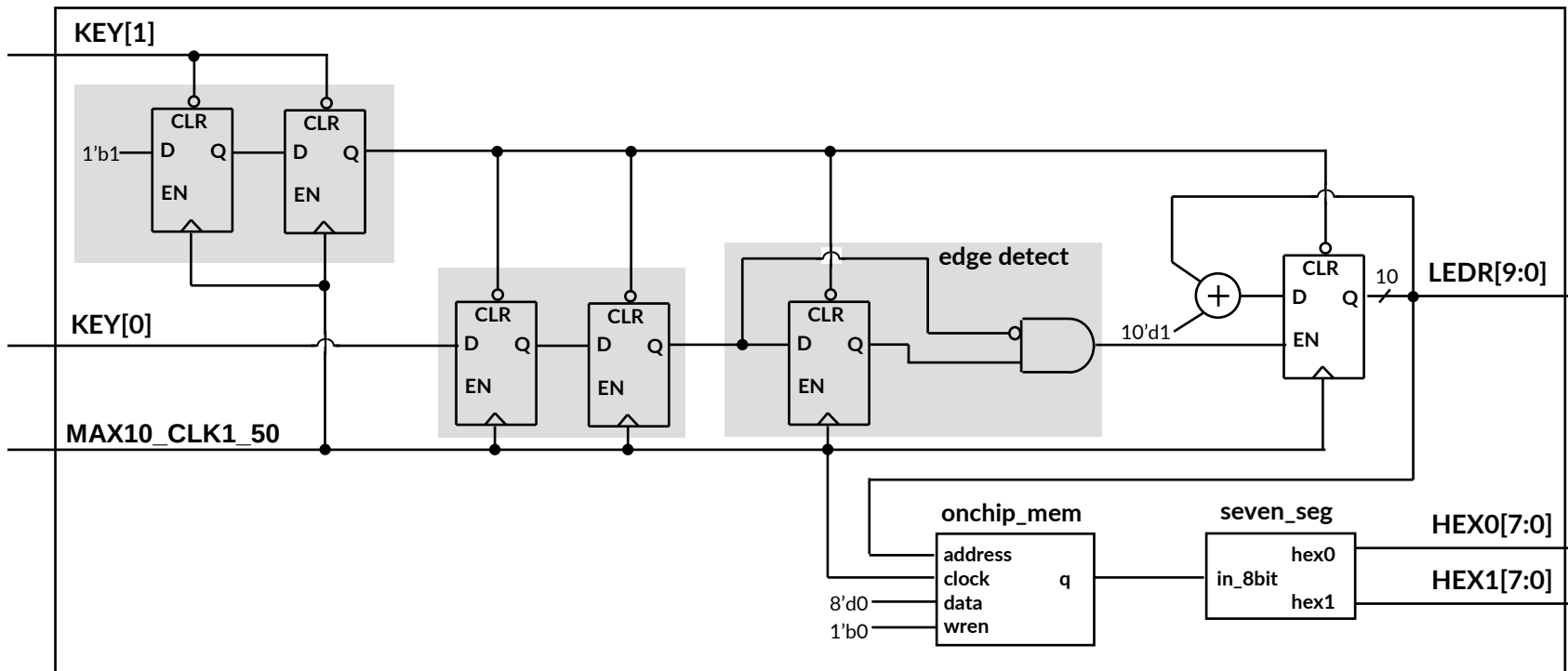
May also specify .mif file in HDL using the
ram_init_file synthesis attribute

PROJECT 7



PROJECT 7

- Open the projeto-07 file:
 - Create a memory initialization file with 1024 words and 8 bits.
 - Create a memory with the same size. Use On-Chip RAM 1-Port from IP Catalog.
 - Enable the memory initialization and select the new file created.
- Add a 7 segment display decoder (file seven_seg.v).



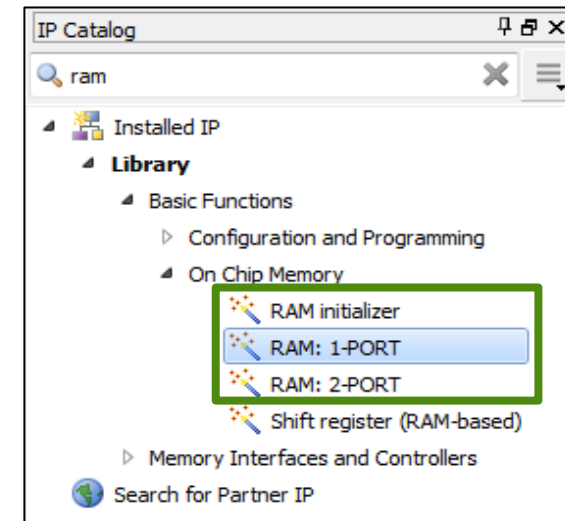
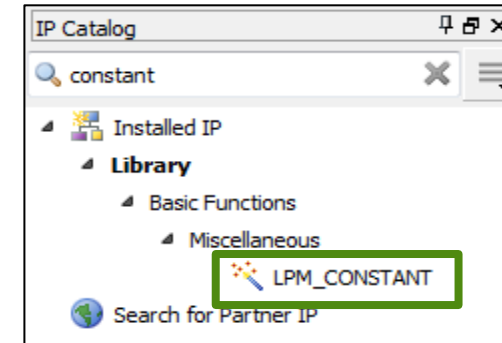


IN-SYSTEM MEMORY CONTENT EDITOR

- Allows
 - Viewing contents of memories & constants via JTAG
 - Updating contents of memories & constants via JTAG
- Example uses
 - Correct stored image/video data
 - Try different filter coefficients to change filter characteristics
 - View/correct parity bits
 - Insert data errors to check reliability

WHEN TO USE ISMCE

- Embedded memory blocks storing data
 - Verify memory contents during device operation
 - Update memory contents during device operation
- JTAG connection required
- No external equipment available
- Supported IP
 - Constants (**LPM_CONSTANT**)
 - Single-port ROM (**ROM: 1-PORT**)
 - Single-port RAM (**RAM: 1-PORT**)

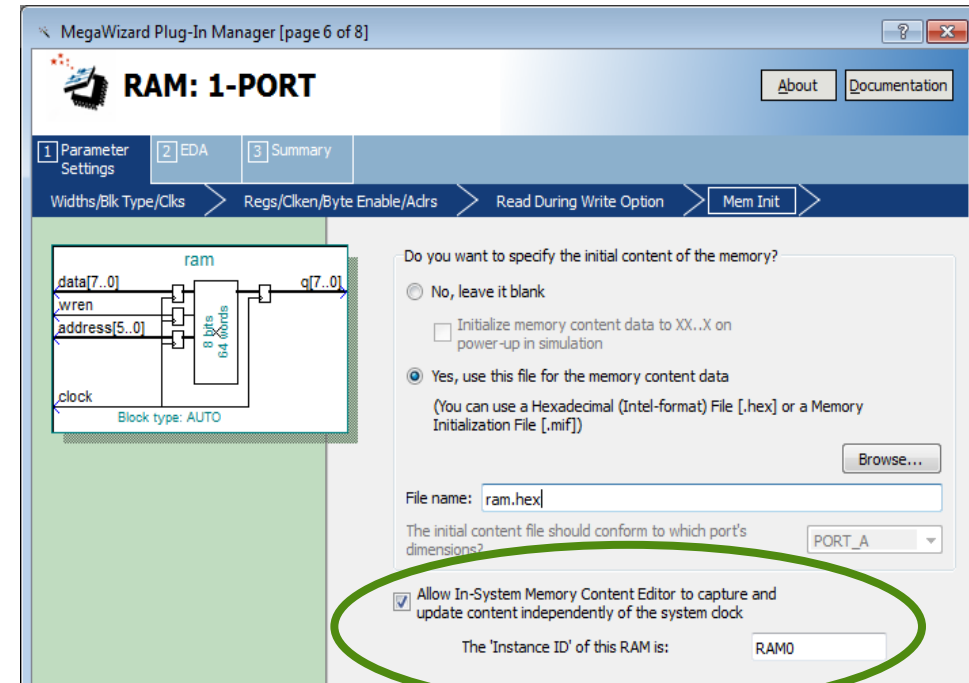


STEPS TO USING THE ISMCE

1. Enable in-system content editing for each memory & constant
2. Perform a full compilation & program device
3. Launch In-System Memory Content Editor
4. Perform reads/writes from/to in-system memories & constants

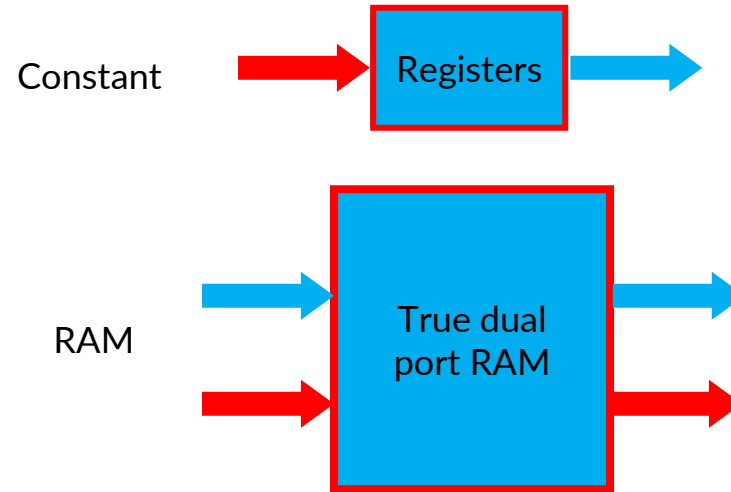
1) ENABLE IN-SYSTEM CONTENT EDITING

- MegaWizard™ Plug-In Manager
via the IP Catalog
 - Parameter Settings → **Mem Init**
tab
- Can also be enabled using
synthesis attributes in HDL code
 - See Quartus Handbook for
details



2) FULL COMPILATIONG & PROGRAM

Implementation changes to allow access to constants & memories



3) LAUNCH IN-SYSTEM MEMORY CONTENT EDITOR

Launch from the Quartus Tools Menu

Instance Manager

JTAG Chain Configuration

The screenshot shows the 'In-System Memory Content Editor' window. The title bar indicates the path: 'C:/altera_trn/Quartus_Debug/QD16_1/Ex1/top - top'. The menu bar includes File, Edit, View, Processing, Tools, Window, and Help. A search bar in the top right corner is labeled 'Search altera.com'.

Two panels are highlighted with green boxes and arrows:

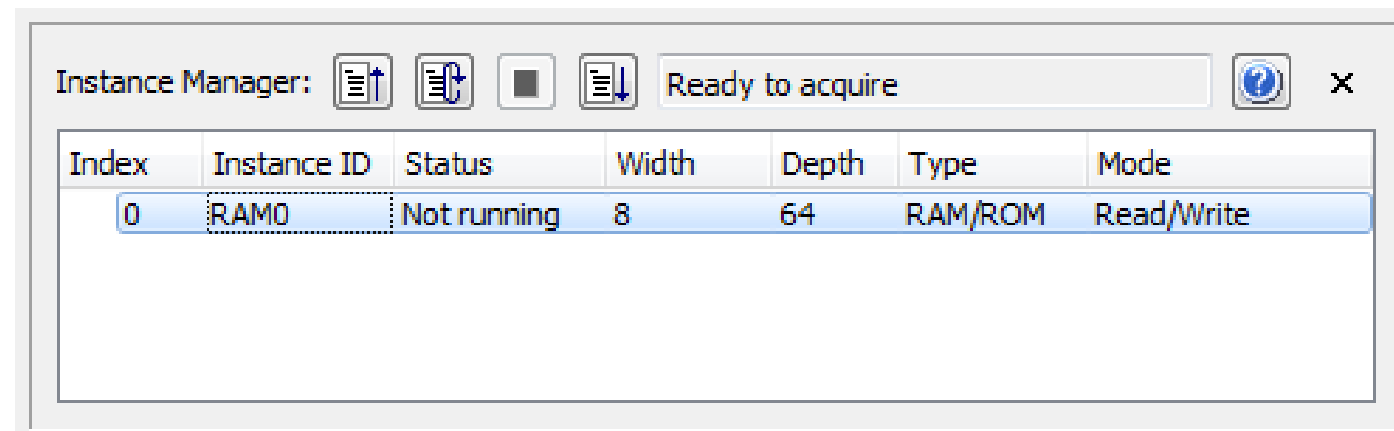
- Instance Manager:** This panel shows a table with columns: Index, Instance ID, Status, Width, Depth, Type, and Mode. The first row is selected, showing Index 0, Instance ID RAM0, Status Not running, Width 8, Depth 64, Type RAM/ROM, and Mode Read/Write.
- JTAG Chain Configuration:** This panel shows the JTAG Chain Configuration status as 'JTAG ready'. It includes fields for Hardware (USB-Blaster [USB-0]), Device (@1: 5CGTFD5(C5|F5)/5CGXBC5C(6|7)), and File. Buttons for 'Setup...', 'Scan Chain', and a file selection icon are also present.

The main area of the editor displays a hex editor view for 'Instance 0: RAM0'. It shows a grid of memory addresses and their corresponding hex values, along with a text representation of the data. The status bar at the bottom indicates '0%' progress, '00:00:00' time, 'Instance 0: RAM0', 'Word: 0x000025', and 'Bit: 0x000007'.

Hex editor

INSTANCE MANAGER

- Instance ID: user-assigned ID (4 characters max.)
- Memory/constant size (Width & Depth)
- Type (RAM/ROM or Constant)
- Mode (Read, Write, Read/Write)

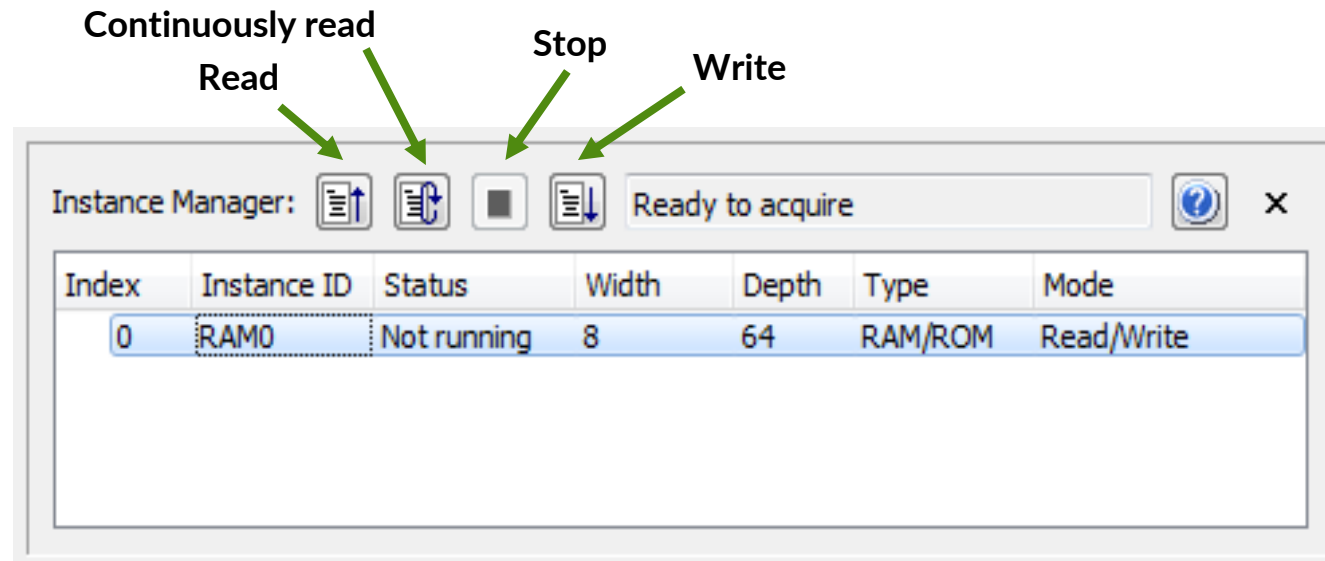


The screenshot shows the 'Instance Manager' window. At the top, there are icons for list manipulation and a status bar that says 'Ready to acquire'. Below this is a table with the following data:

Index	Instance ID	Status	Width	Depth	Type	Mode
0	RAM0	Not running	8	64	RAM/ROM	Read/Write

4) PERFORM READS/WRITES: INSTANCE MANAGER

- Select one or more instances in Instance Manager
- Click on command



5) PERFORM READS/WRITES: HEX EDITOR

- Use hex editor
 - Highlight a set of words or just type over existing words
 - Right-click words and select options
 - Fill with 0's
 - Fill with 1's
 - Random data
- Text color indicates memory data status
 - **Black:** current contents
 - **Red:** displayed contents are not current
 - **Blue:** data to be written to memory

IN-SYSTEM EDITOR OPTIONS

- Import memory data file to update memories or constant
 - .hex & .mif
- Export data displayed in hex editor
 - .rif, .hex, .mif, .vcd
- Open multiple editors to access multiple devices in JTAG chain
- Tcl scripting support

A wooden desk with a laptop, books, and a pen, with a blurred background of a room.

QUARTUS DEBUGGING

IN-SYSTEM SOURCES AND PROBES - ISSP

IN-SYSTEM SOURCES AND PROBES - ISSP



WHEN TO USE SOURCES & PROBES

- Simple functional debug needed
- Logic resources available for ISSP IP instance(s)
- JTAG connection required
- No external equipment available or needed

EXAMPLE USES

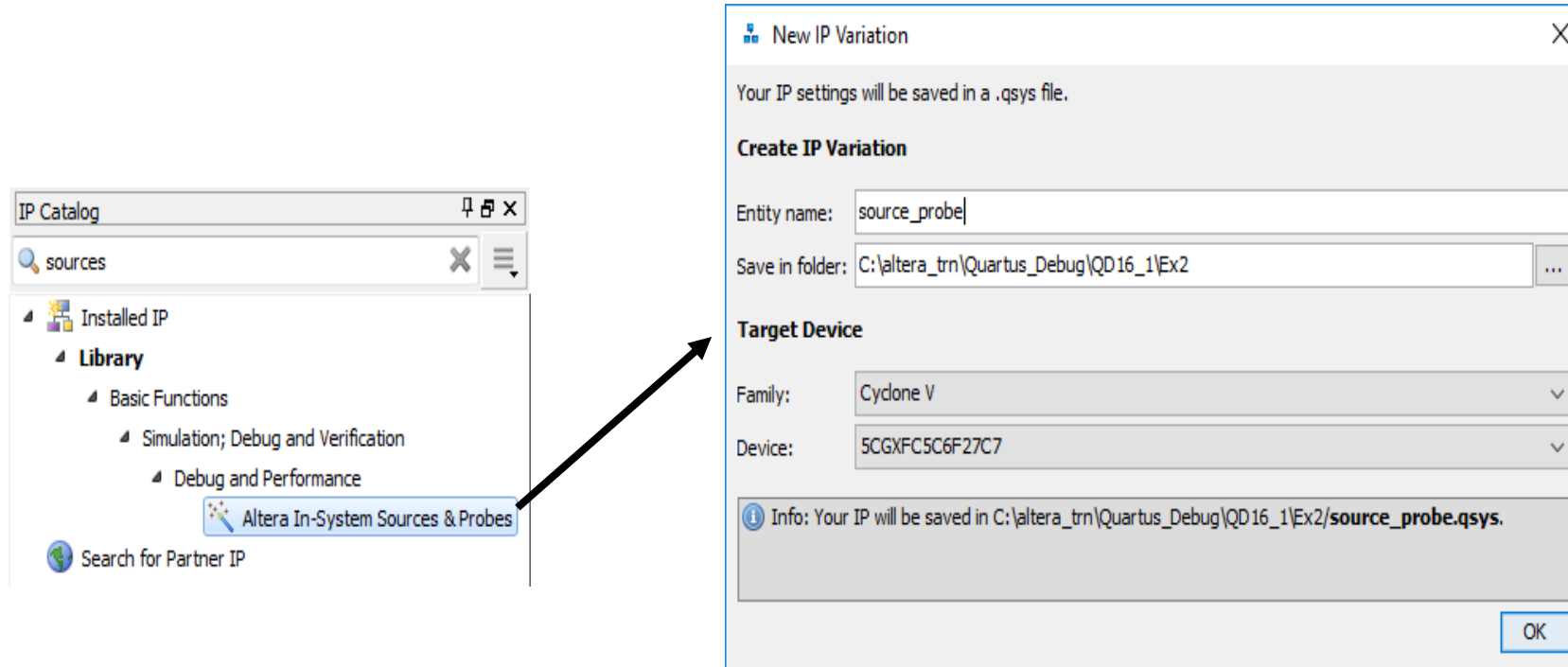
- Virtual push button control of signals in design
- Monitor results of changing design constants
- Extensive Tcl scripting support to create custom automated design control interfaces
- Force trigger conditions for debugging design with the Signal Tap Logic Analyzer

USING IN-SYSTEM SOURCES & PROBES

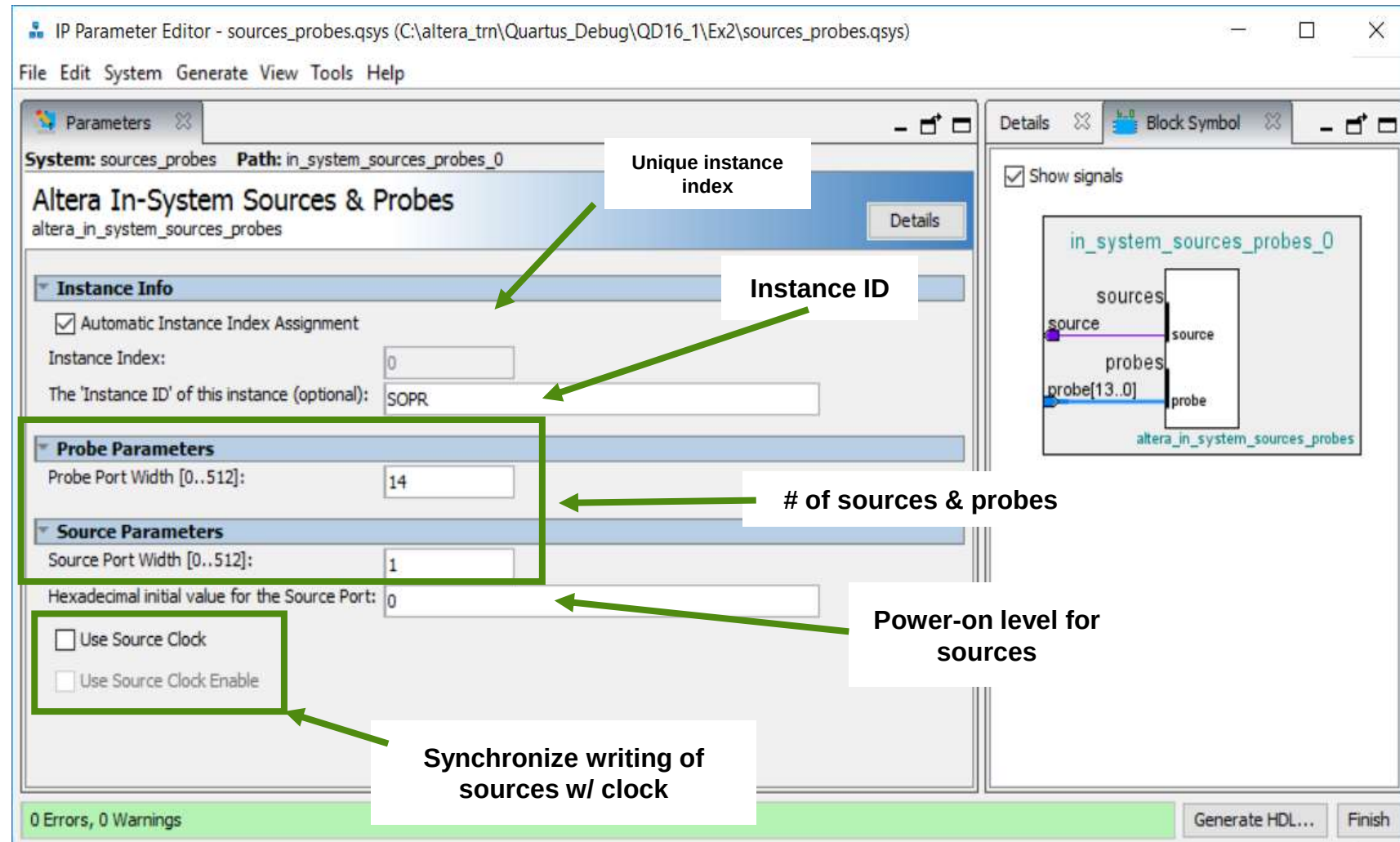
1. Create In-System Sources & Probes IP instance(s) using the IP Parameter Editor or MegaWizard Plug-In Manager (device-dependent) (through the IP Catalog)
2. Instantiate in design and compile project
3. Program target device(s)
4. Create and use In-System Sources & Probes Editor (.spf file) to control sources and probes

CREATE SOURCES & PROBES MEGAFUNCTION

- In-System Sources and Probes IP found in IP Catalog
- Select IP name and target device



IP PARAMETER EDITOR FOR ISSP



IN-SYSTEM SOURCES & PROBES EDITOR (.spf)

Quartus Tools menu → In-System Sources & Probes Editor

The screenshot displays the 'In-System Sources and Probes Editor' window. The top section contains the 'Instance Manager' with settings for 'Probe read interval' (Automatic, 0 samples per second) and 'Event log' (Maximum size: 64, Save data to event log checked). The 'JTAG Chain Configuration' panel on the right shows 'JTAG ready' status and hardware details (USB-Blaster [USB-0], Device: @ 1: EP3C120/EP4CE115 [0x02B]).

The main area shows a table of sources and probes. A green bracket labeled 'Probes' points to the list of probes (P13, P12, P11, P10, P9, P8, P7) under the 'one_seg' alias. A green arrow labeled 'Source' points to the 'reset_n' source (S0) under the 'ten_seg' alias.

The bottom section displays a waveform log (waveform viewer) showing the data for the selected sources and probes over time. The log includes a table with columns for Index, Type, Alias, Name, and Data, followed by a timeline view showing the signal levels for each probe and source.

Index	Type	Alias	Name	Data
P[13..7]	Probe	one_seg	one_seg[6]	00h
P13	Probe	one_seg	one_seg[6]	0
P12	Probe	one_seg	one_seg[5]	0
P11	Probe	one_seg	one_seg[4]	0
P10	Probe	one_seg	one_seg[3]	0
P9	Probe	one_seg	one_seg[2]	0
P8	Probe	one_seg	one_seg[1]	0
P7	Probe	one_seg	one_seg[0]	0
P[6..0]	Probe	ten_seg	ten_seg[6]	00h
P6	Probe	ten_seg	ten_seg[6]	0
P5	Probe	ten_seg	ten_seg[5]	0
P4	Probe	ten_seg	ten_seg[4]	0
P3	Probe	ten_seg	ten_seg[3]	0
P2	Probe	ten_seg	ten_seg[2]	0
P1	Probe	ten_seg	ten_seg[1]	0
P0	Probe	ten_seg	ten_seg[0]	0
S0	Source	reset_n	reset_n	0

SOURCES & PROBES INSTANCE MANAGER

- Select one or more instances
- Click on command

The screenshot shows the 'Instance Manager' window with several annotations pointing to specific controls:

- Read probe(s)**: Points to the first icon in the toolbar (a document with a magnifying glass).
- Continuously read**: Points to the second icon in the toolbar (a document with a magnifying glass and a refresh symbol).
- Stop**: Points to the third icon in the toolbar (a square with a diagonal line).
- Write source(s)**: Points to the fourth icon in the toolbar (a document with a magnifying glass and a write symbol).
- Refresh source data values**: Points to the fifth icon in the toolbar (a circular arrow).
- Number of samples in log**: Points to the 'Maximum size' dropdown menu in the 'Event log' section, which is currently set to 64.
- Write once or continuously**: Points to the 'Write source data' dropdown menu, which is currently set to 'Continuously'.

The window also displays the following settings and table:

Probe read interval
Current interval: 0 samples per second
☒ Automatic
☐ Manual 1 s

Event log
Maximum size: 64
☒ Save data to event log
 Write source data: Continuously

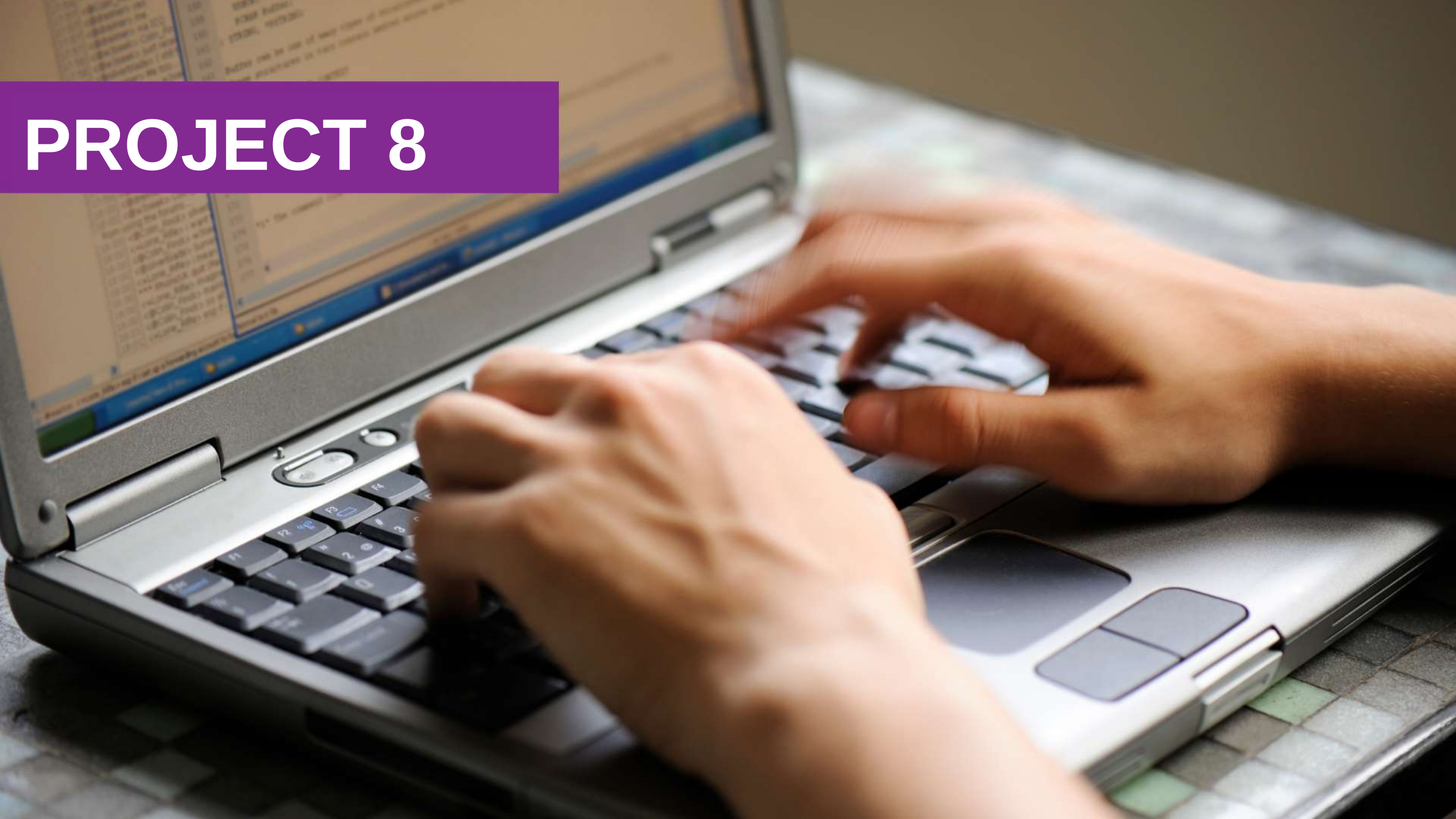
Index	Instance ID	Status	Sources: 1	Probes: 14	Name
0	SOPR	Unexpected JTA...	1	14	sources_probes:sources_probes_inst al...

INSTANCE CONTROL AND RESULTS

- Right-click multiple selected sources or probes to group and set display format
- Click Data column to change source value
 - **Red** number is value to be written (**Write source** button must be clicked)
 - **Black** number is current value
- Click or right-click in Log to zoom in or out
- Log saved in .spf file

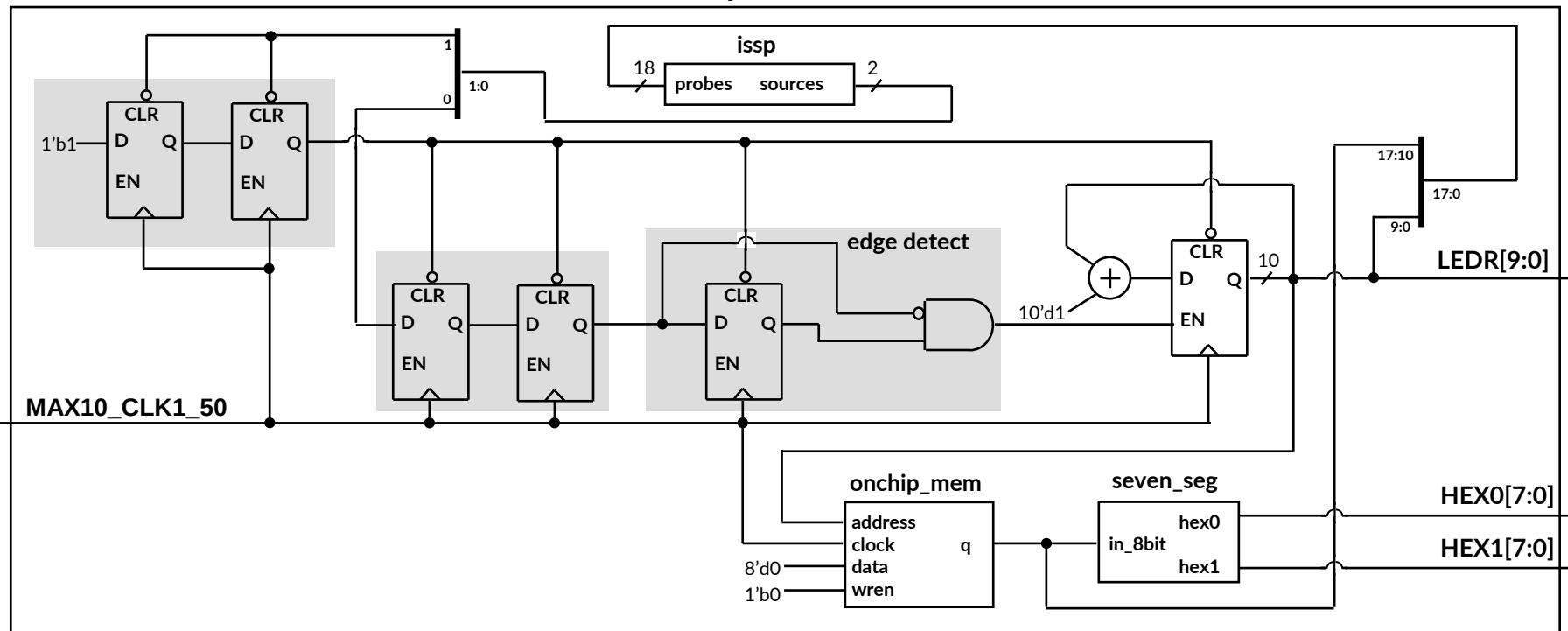
P1			ten_seg[1]	1	
P0			ten_seg[0]	0	
S0			reset_n	1	

Click to change source value/level



PROJECT 8

- ## Project



SYNCHRONOUS DESIGN

FINITE-STATE MACHINE: CONCEPT AND MODELING

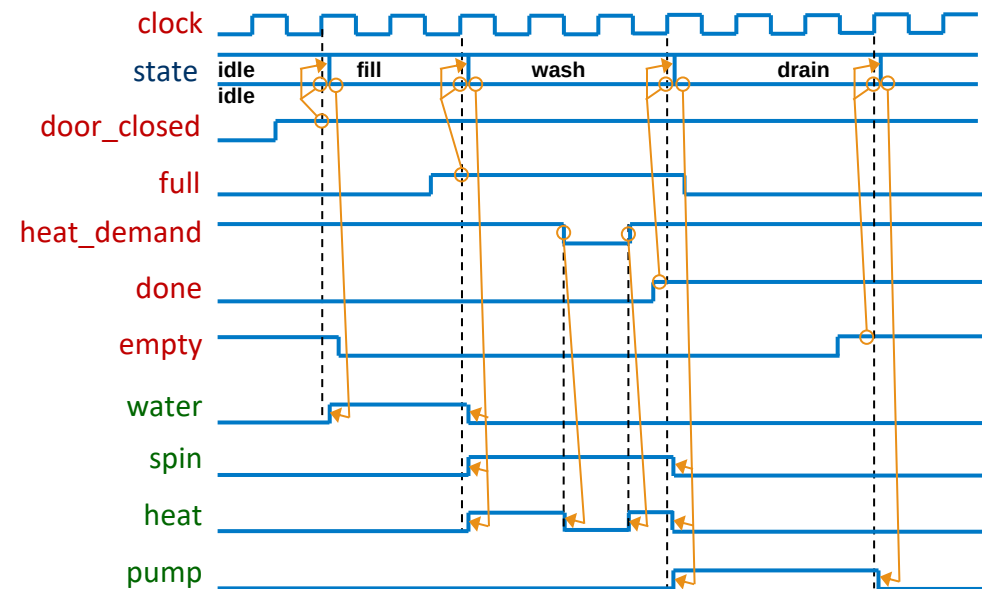
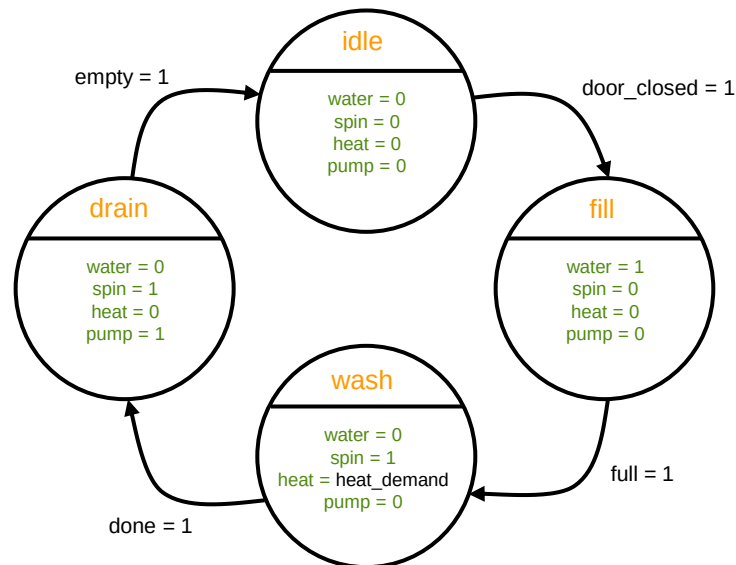


SYNCHRONOUS SEQUENTIAL CIRCUITS

- A kind of frequently used synchronous sequential circuits is the Finite-State Machine (FSM)
 - Well-defined method to implement any sequential circuit
 - Can be used to design several different types of circuits
- Example:
 - Protocol implementation. States:
 - Header sending
 - Data sending
 - CRC calculation
 - DMA (Direct Memory Access). States:
 - Data read request
 - Data reading
 - Data writing
 - Washing Machine. States:
 - Stand by - Idle
 - Water filling
 - Washing
 - Water draining

SYNCHRONOUS SEQUENTIAL CIRCUITS

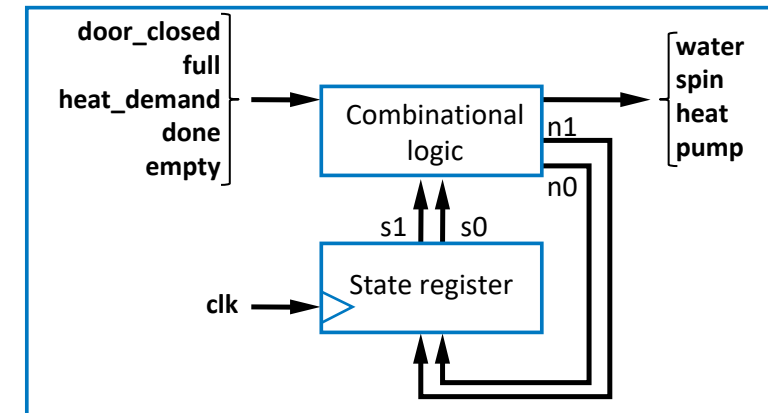
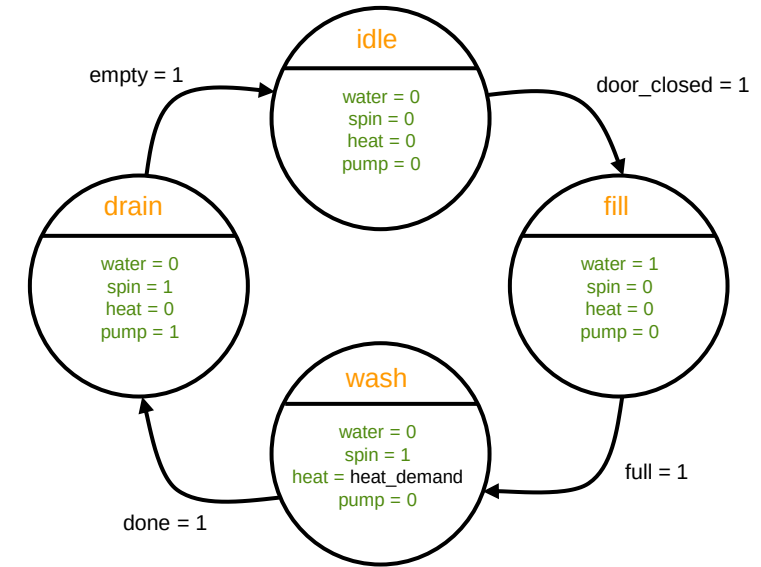
- A finite-state machine consists of some states and the state transitions
- The output can be set by the current state, next state and inputs, or any combinational logic using the current state as an input
- In the state diagram, each state name is shown in the upper half of the state circle, the outputs in the lower half of the state circle, and the transition condition from one state to another is placed at the arrows.
 - Whenever a transition condition isn't specified, this transition will occur in the next clock cycle.



SYNCHRONOUS SEQUENTIAL CIRCUITS

- How to implement a finite-state machine?
 - There are some architectures, it must be chosen the one which best fit the case
 - All architectures separate the State Register (current state) and the Combinational Logic, which defines the next state and the outputs

- In the washing machine example, 2 bits are needed to represent all 4 states (two registers, or “a two bit register”).

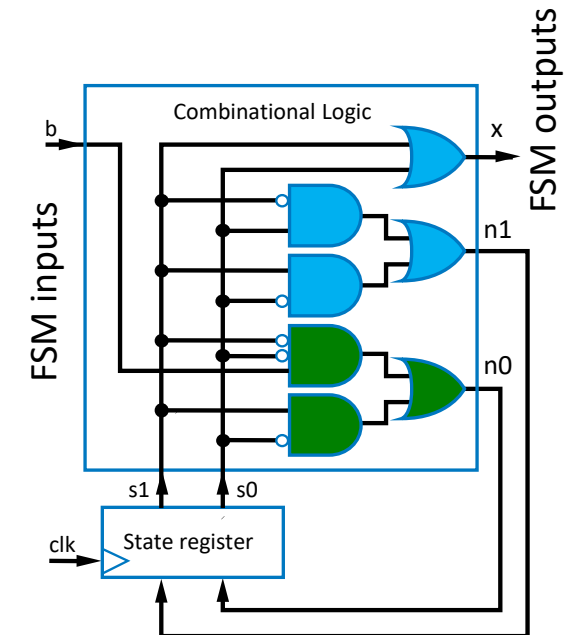


SYNCHRONOUS SEQUENTIAL CIRCUITS

- After defining the architecture (inputs, outputs and connections between the sequential and combinational blocks) is necessary:
 - Define the states transition code
 - Find the logic function for the combinational circuit
- It's not necessary when it's made at high-level (HDL)
 - The tool chooses the state transition code
 - The logic is described at high-level

	Inputs			Outputs		
	s1	s0	b	x	n1	n0
Off	0	0	0	0	0	0
	0	0	1	0	0	1
On1	0	1	0	1	1	0
	0	1	1	1	1	0
On2	1	0	0	1	1	1
	1	0	1	1	1	1
On3	1	1	0	1	0	0
	1	1	1	1	0	0

$$\begin{aligned}
 x &= s1 + s0 \\
 n1 &= s1's0 + s1s0' \\
 n0 &= s1's0'b + s1s0'
 \end{aligned}$$



SYNCHRONOUS DESIGN

FINITE-STATE MACHINE:

VERILOG IMPLEMENTATION



STATE MACHINE CODING

- Local parameters used to define states

```
localparam idle=0, fill=1, heat_w=2, wash=3, drain=4;
```

- Local parameter “values” are replaced with state encoding values as chosen by synthesis tool
 - Use options/constraints in synthesis tool to control encoding style (e.g. binary, one-hot, safe, etc.)

- Registers are used to store state

```
reg [2:0] current_state, next_state;
```

- Do not bit-slice current_state or next_state (e.g. state[1:0])
- Separate sequential process from combinational process
- Sequential process should always include synchronous or asynchronous reset
- Use case to do the next-state logic

STATE DECLARATION

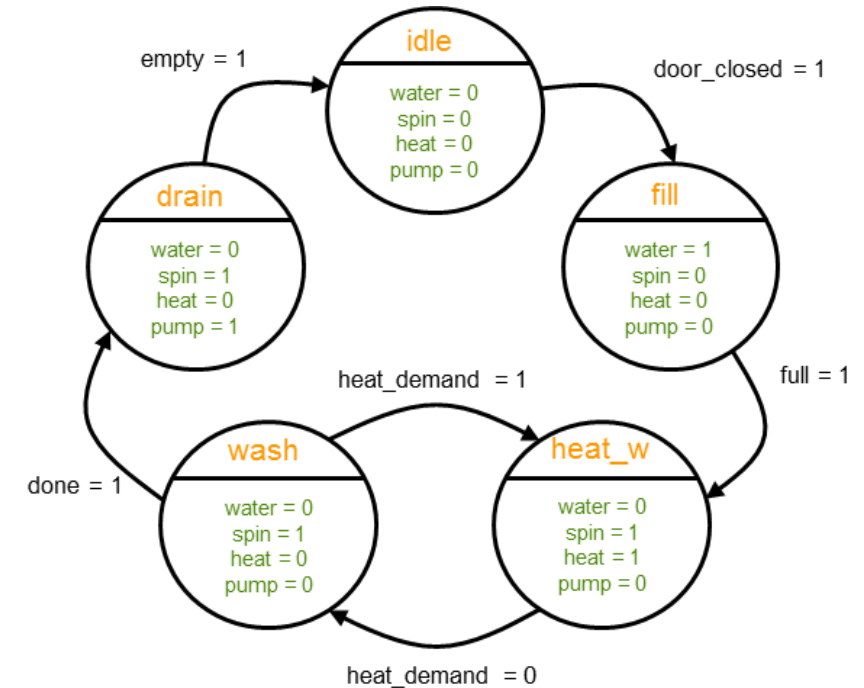
```
module state_machine (  
    input clk, reset, door_closed, full,  
    input heat_demand, done, empty,  
    output reg water, spin, heat, pump  
);
```

```
reg [2:0] current_state, next_state;
```

```
localparam idle=0, fill=1, heat_w=2,  
    wash=3, drain=4;
```

States

State registers



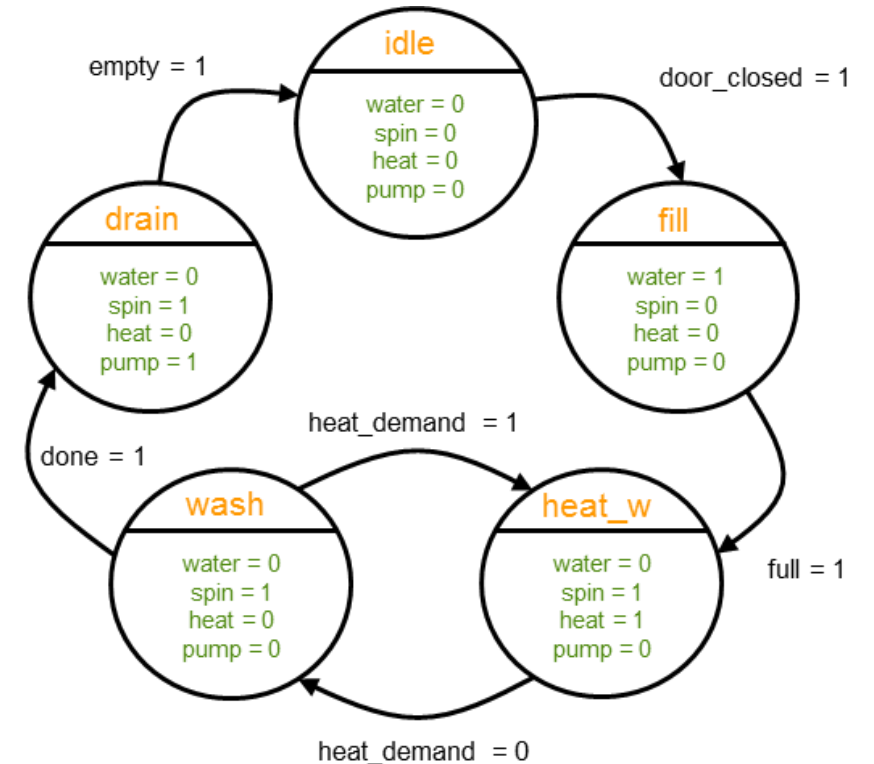
NEXT-STATE LOGIC

```
//State transitions
always @(posedge clk)
  if (reset)
    current_state <= idle;
  else
    current_state <= next_state;

//Next state logic
always @*
begin
  next_state <= current_state; //default condition
  case (current_state)
    idle: if (door_closed) next_state = fill;
    fill:  if (full) next_state = heat_w;
    heat_w: if (~heat_demand) next_state = wash;
    wash:  begin
              if (heat_demand) next_state = heat_w;
              if (done) next_state = drain;
            end
    drain: if (empty) next_state = idle;
  endcase
end
```

State transitions

Next state logic



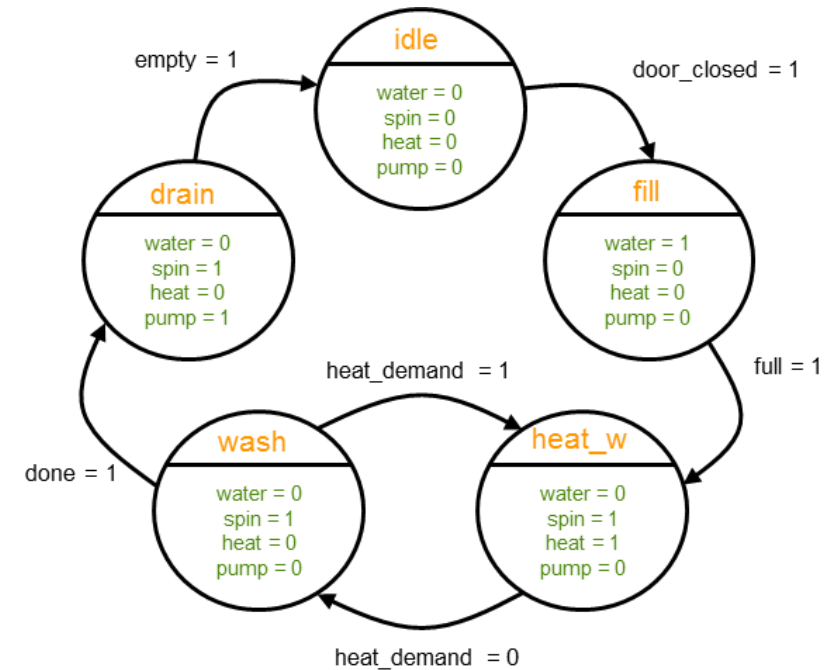
MOORE COMBINATORIAL OUTPUTS

```
//output logic
always @*
begin
```

```
  water <= 0;
  spin <= 0;
  heat <= 0;
  pump <= 0;
```

Default output conditions

```
  case (current_state)
    idle:      ;
    fill:      water <= 1;
    heat_w:    begin
                  spin <= 1;
                  heat <= 1;
                end
    wash:      spin <= 1;
    drain:     begin
                  spin <= 1;
                  pump <= 1;
                end
  endcase
end
```

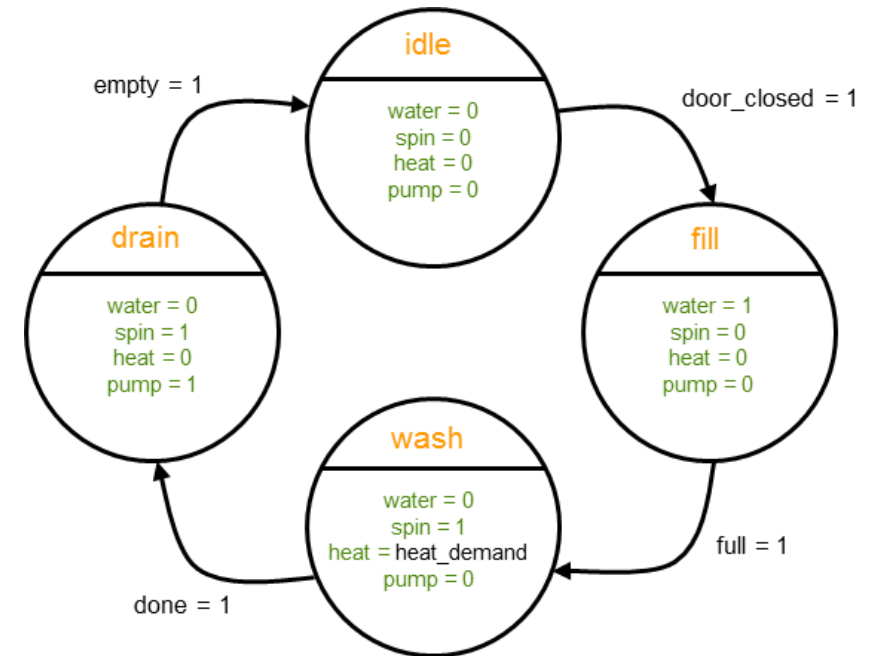


- Output logic function of state only

MEALY COMBINATORIAL OUTPUTS

```
//output logic
always @ *
begin
    water <= 0;
    spin <= 0;
    heat <= 0;
    pump <= 0;

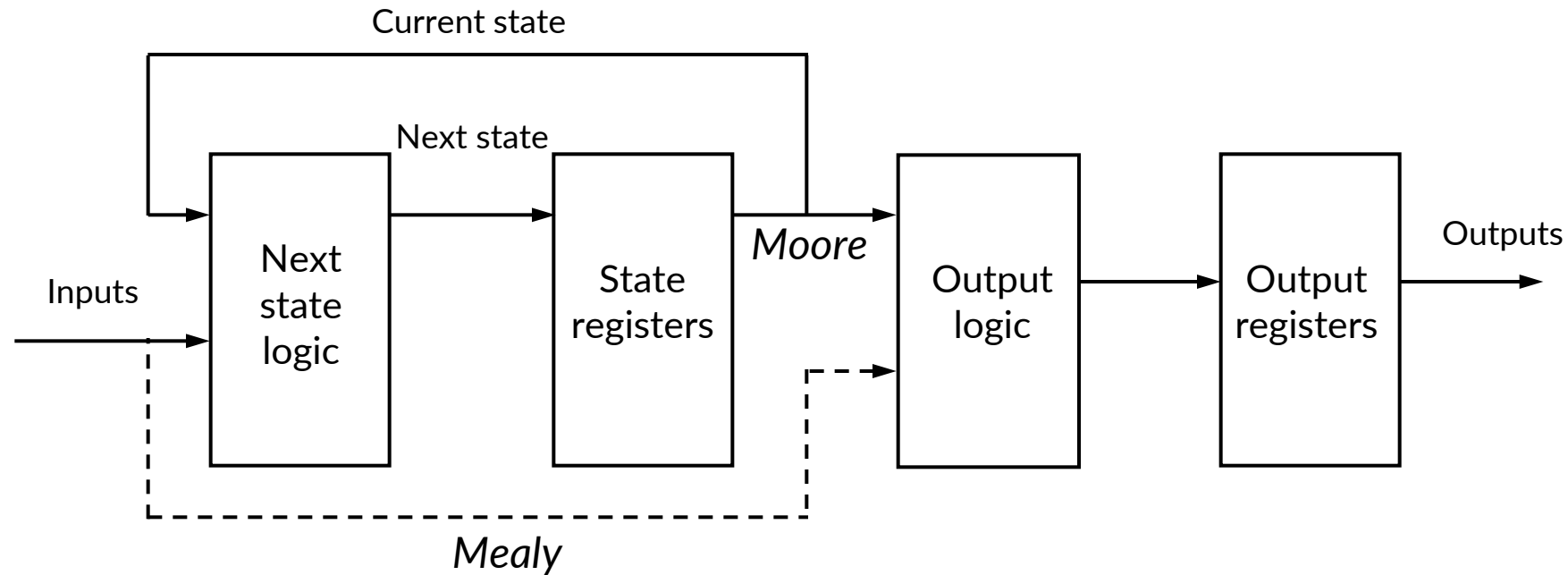
    case (current_state)
        idle:      ;
        fill:      water <= 1;
        wash:      begin
                        spin <= 1; heat <= heat_demand;
                    end
        drain:     begin
                        spin <= 1;
                        pump <= 1;
                    end
    endcase
end
```



- Output logic function of state and input(s)

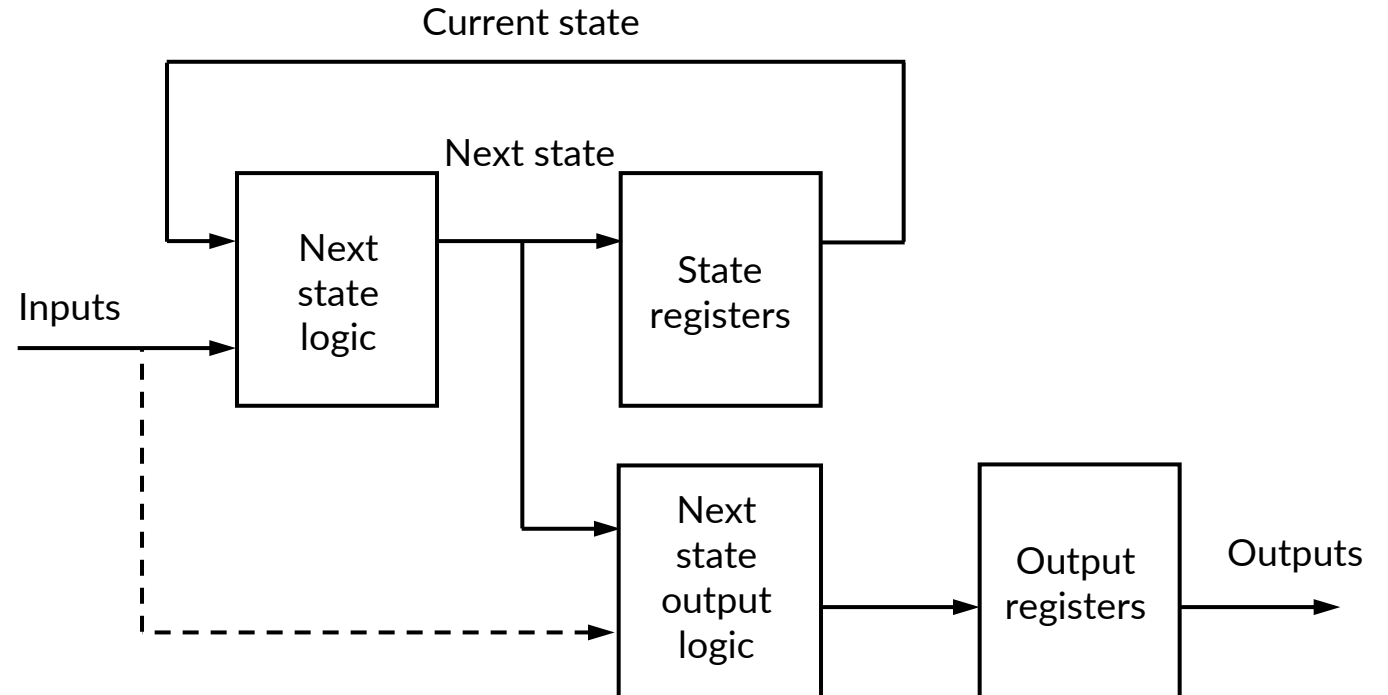
REGISTERED OUTPUTS

- Remove glitches by adding output registers
 - Adds a stage of latency



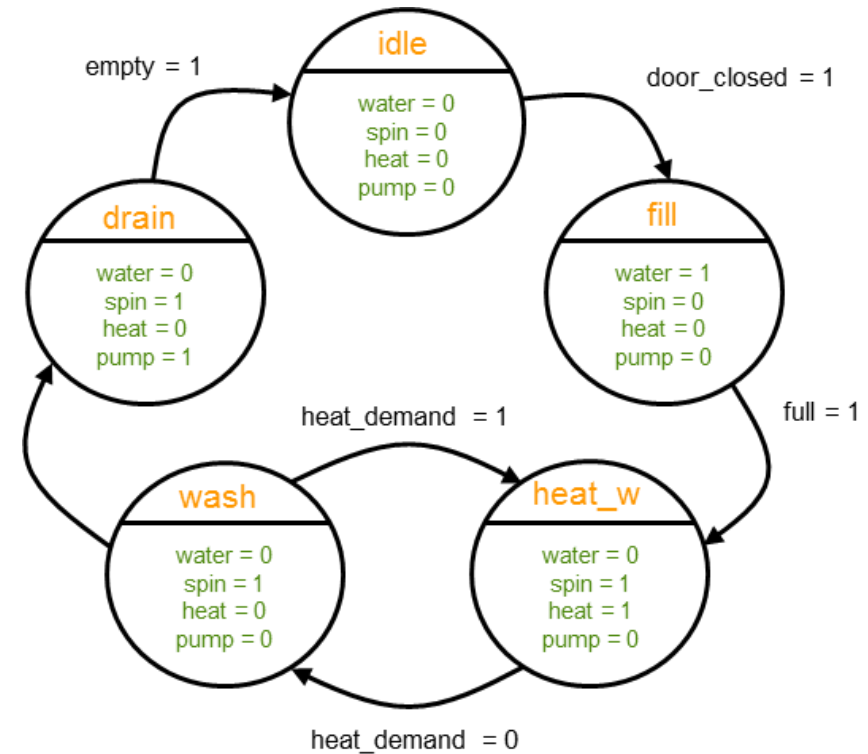
REGISTERED OUTPUTS WITHOUT LATENCY

- Base outputs on *next state* instead of current state
 - Output logic uses next state to determine what next outputs will be
 - On next rising edge, outputs change along with state registers



REGISTERED OUTPUTS WITHOUT LATENCY

```
//output logic
always @ (posedge clk)
begin
  water <= 0;
  spin <= 0;
  heat <= 0;
  pump = 0;
  case (next_state)
    idle:      ;
    fill:      water <= 1;
    heat_w:    begin
                  spin <= 1;
                  heat <= 1;
                end
    wash:      spin <= 1;
    drain:     begin
                  spin <= 1;
                  pump <= 1;
                end
  endcase
end
```



- Base output logic case statement on next state (instead of current state)
- Wrap output logic with a clocked process

TIP FOR WRITING EFFICIENT STATE MACHINES

- Remove counting, timing, arithmetic functions from state machine & implement externally
- Reduces overall logic & improves performance

STATE MACHINE VIEWER

Tools menu → Netlist Viewers → State Machine Viewer

Use menu to select from multiple state machines in design

State flow diagram

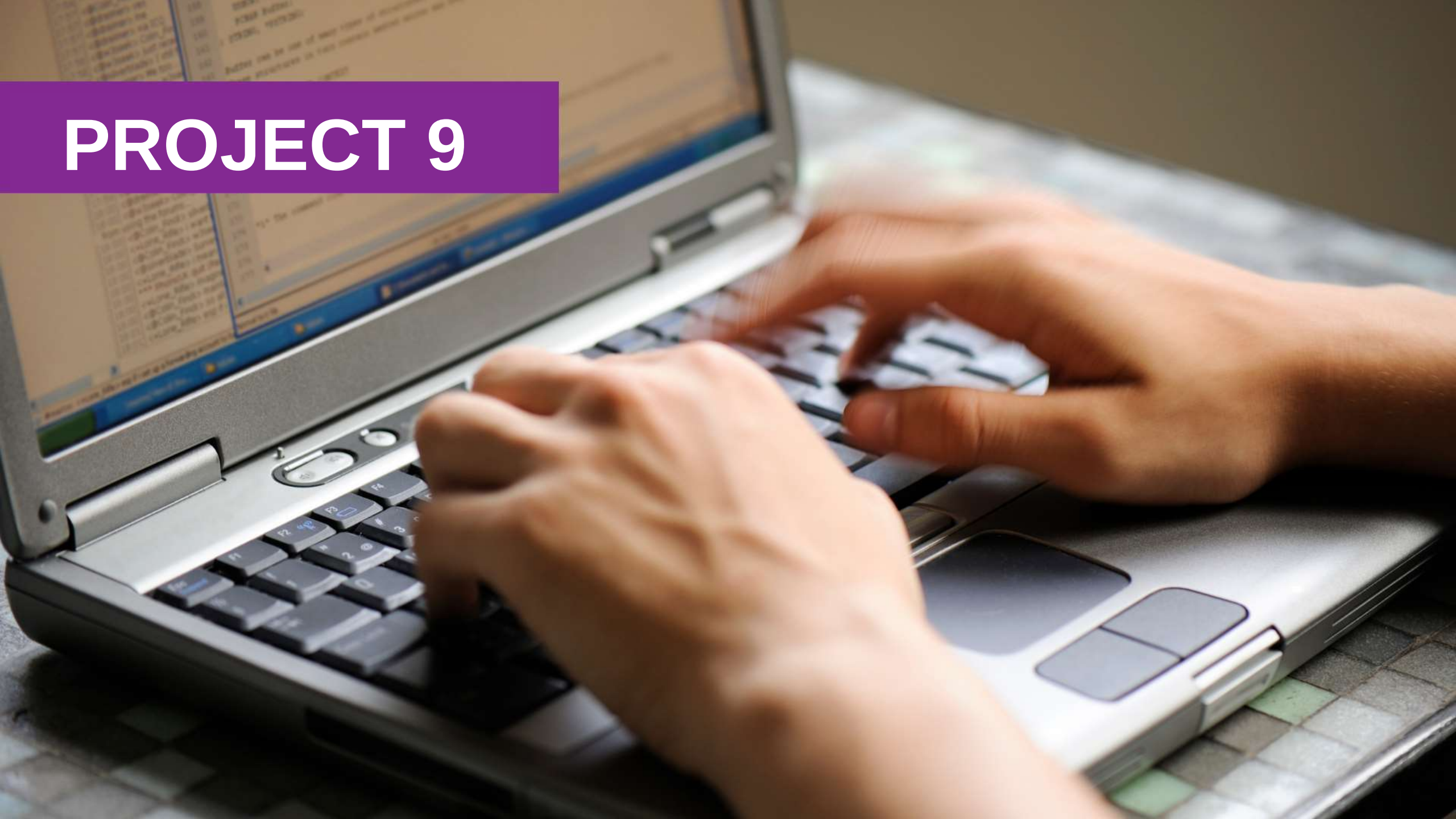
Highlight state in table or diagram to highlight in other

State transition/encoding table

The screenshot shows the State Machine Viewer interface. At the top, a dropdown menu is set to 'latches\$current_state'. Below this is a state flow diagram with five states: 'idle' (yellow), 'fill' (red), 'wash' (yellow), 'heat_w' (yellow), and 'drain' (yellow). Transitions are labeled with conditions like '(!empty)', '(!full)', '(!reset)', 'heat_demand', and 'door_closed'. At the bottom is a table with columns 'Source State', 'Destination State', and 'Condition'. The table lists 9 transitions, with rows 3, 4, 5, 8, and 9 highlighted in blue. The 'fill' state in the diagram is also highlighted in red, corresponding to the highlighted rows in the table.

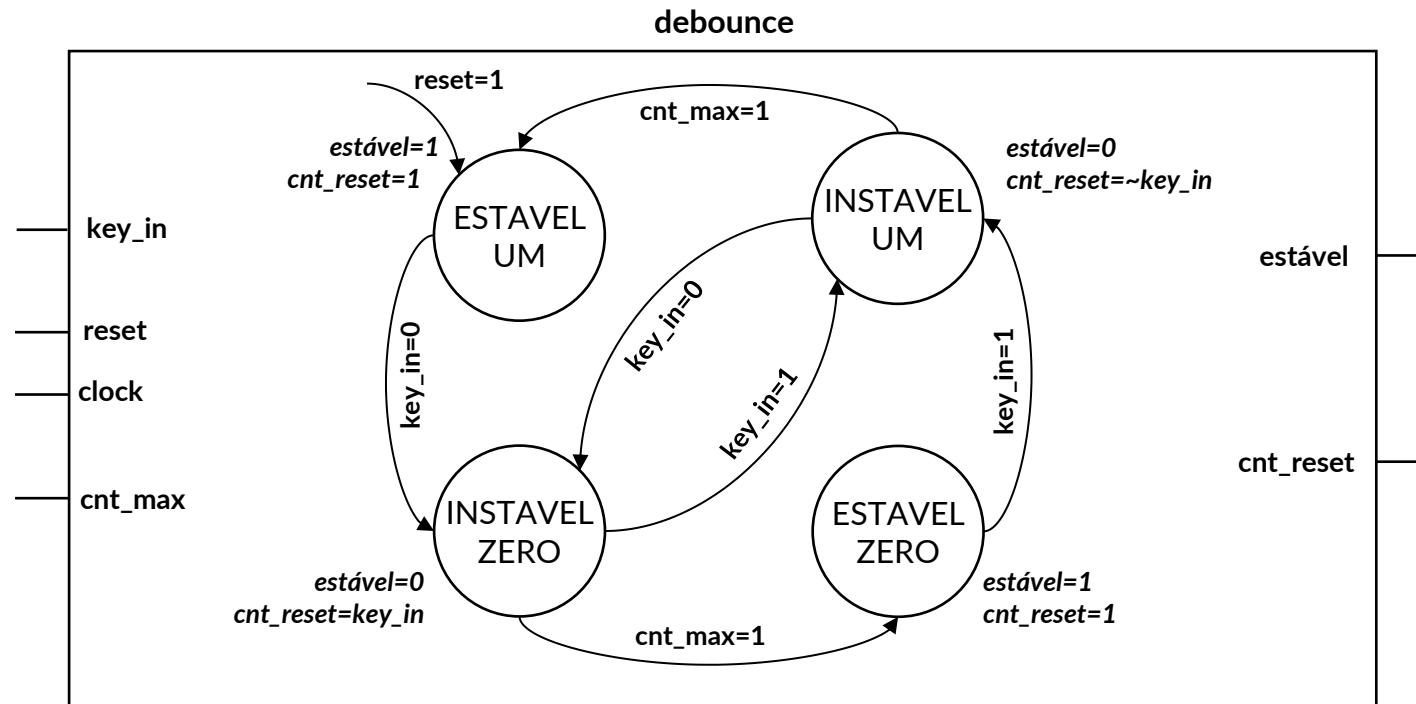
	Source State	Destination State	Condition
1	drain	idle	(!empty).(!reset) + (!empty)
2	drain	drain	(!empty).(!reset)
3	fill	fill	(!full).(!reset)
4	fill	idle	(reset)
5	fill	heat_w	(full).(!reset)
6	heat_w	idle	(reset)
7	heat_w	wash	(heat_demand).(!reset)
8	heat_w	heat_w	(!heat_demand).(!reset)
9	idle	fill	(door_closed).(!reset)

PROJECT 9



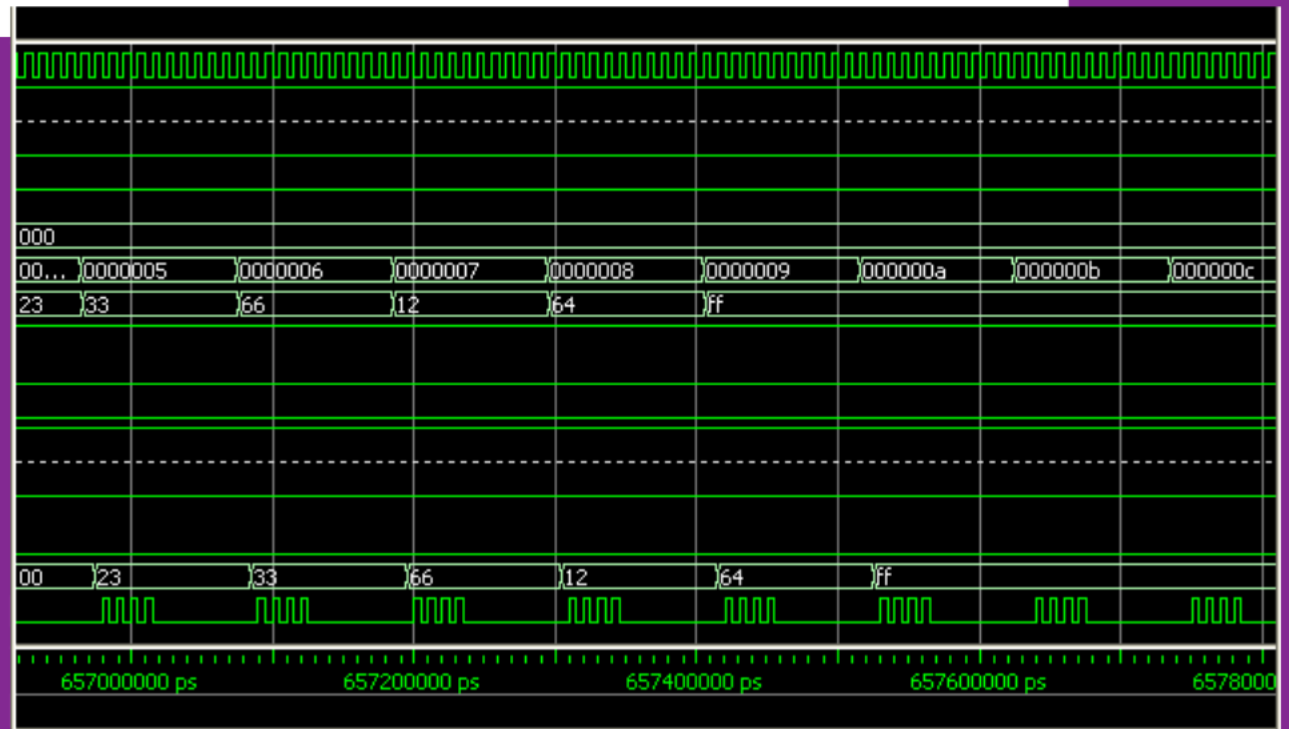
PROJECT 9

- Write a Verilog module to make an input signal debounce, using a finite-state machine according to the figure
- The finite-state machine must have an interface with an external counter determining when the debounce expires

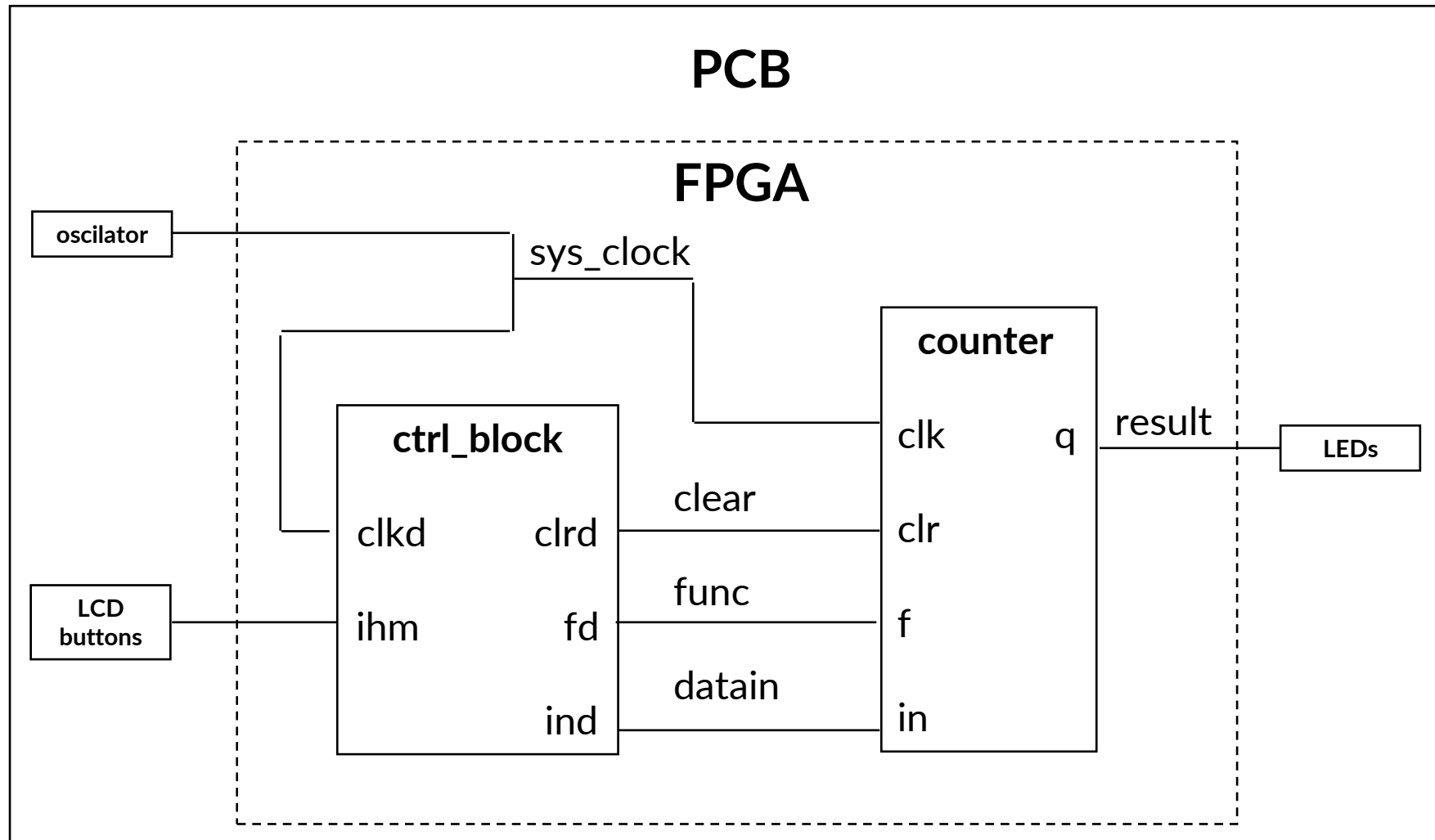


SIMULATION

MOTIVATION AND CONCEPT

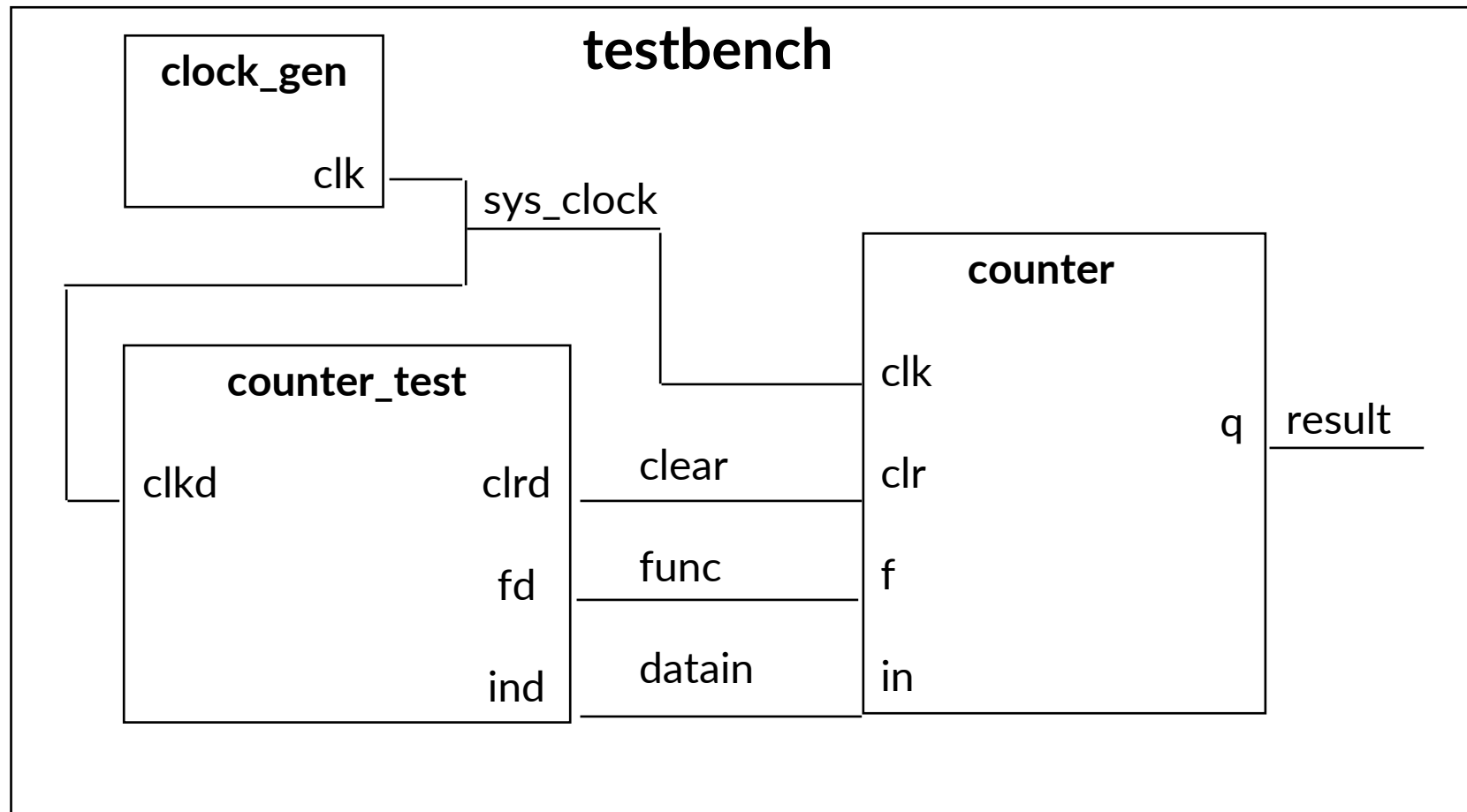


EXAMPLE – SYSTEM RUNNING ON HARDWARE



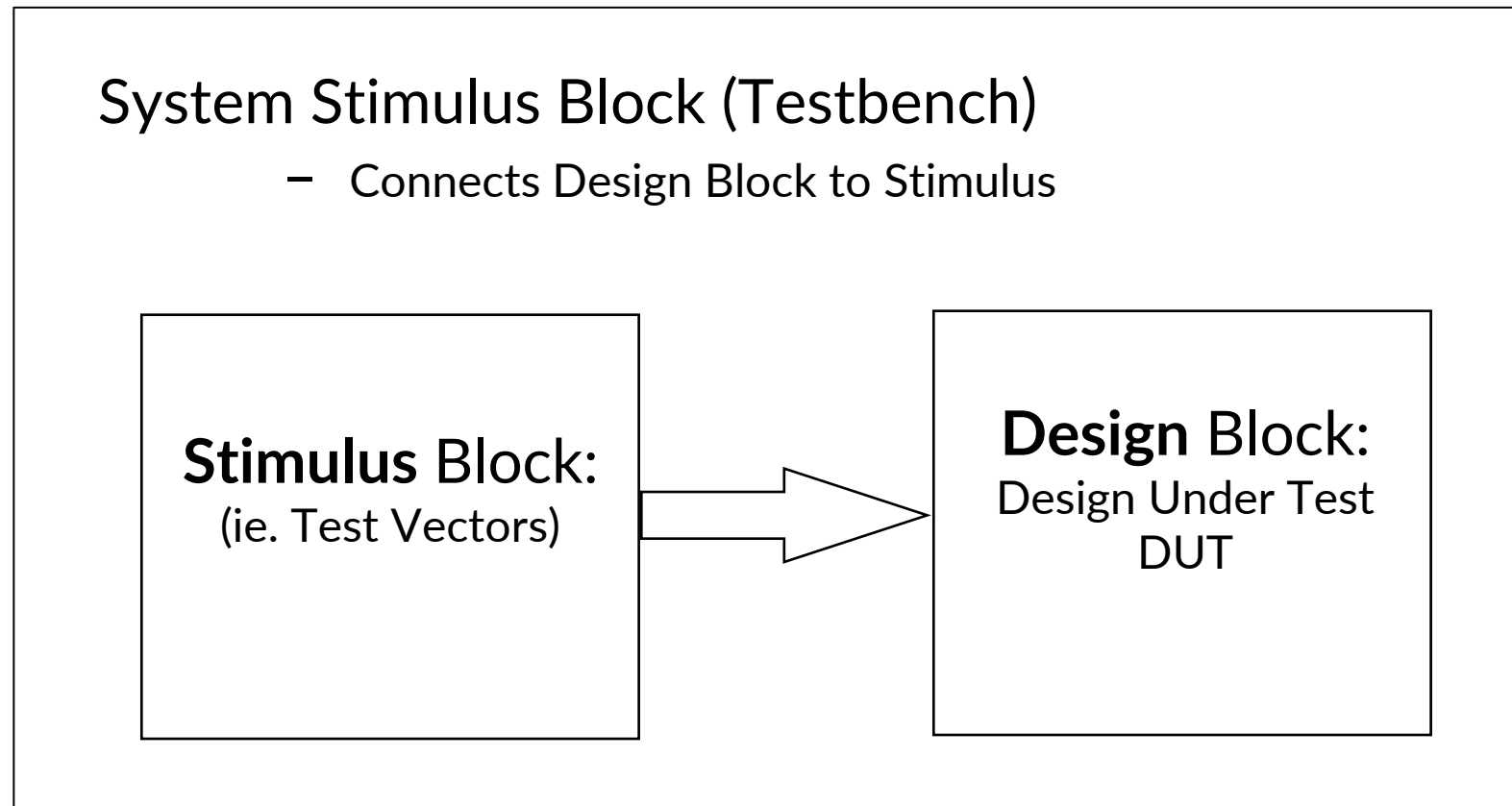
EXAMPLE – VERILOG SIMULATION SETUP

There is no actual hardware, so we need a *testbench*!



VERILOG SIMULATION ENVIRONMENT

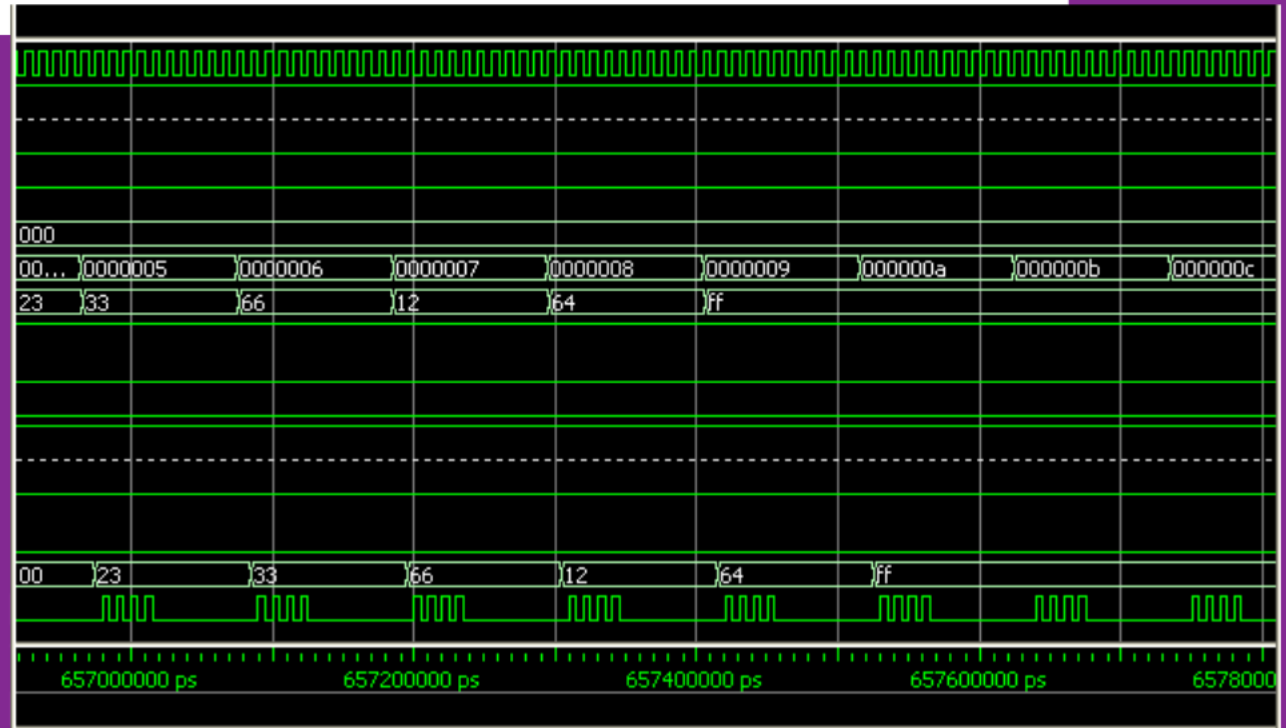
Contains a Stimulus Block and a Design Block



SIMULATION

STATEMENTS FOR SIMULATION:

- INITIAL



TWO PROCEDURAL BLOCKS

- **initial**
 - Used to initialize behavioral statements for simulation
 - **always**
 - Used to describe the circuit functionality using behavioral statements
- ⇒ Each **always** and **initial** block represents a separate process
- ⇒ Processes run in parallel and start at simulation time 0
- ⇒ Statements inside a process execute sequentially
- ⇒ **always** and **initial** blocks cannot be nested

INITIAL BLOCK

- Consists of behavioral statements
 - Each **initial** block executes concurrently starting at time 0, executes only once and then does not execute again
 - Must use keywords **begin** and **end** to group behavioral statements when **initial** block contains more than one behavioral statement
 - Example uses
 - Initialization
 - Monitoring
 - Any functionality that needs to be turned on just once
- ⇒ Note that though the **initial** block executes only once, the duration of **initial** block may be infinite (i.e. functionality inside may continue running for the duration of the model execution)

INITIAL BLOCK EXAMPLE

```

module system ( );

    reg a, b, c, d;

    // single statement
    initial a = 1'b0;

    /* multiple statements:
       needs to be grouped */
    initial begin
        b = 1'b1;
        #5 c = 1'b0;
        #10 d = 1'b0;
    end

    initial #20 $finish;

endmodule

```

Time	Statement(s) Executed
0	a = 1'b0; b = 1'b1
5	c = 1'b0;
15	d = 1'b0;
20	\$finish

NAMING PROCEDURAL BLOCKS

- Procedural blocks may be named by adding : <name> after **begin**
- Advantages
 - Allows the procedural block to be referenced in other places within the code by name
 - Allows declaration of objects local to the procedural block
 - Allows monitoring of procedural block by name in simulation tools

```
initial
begin : clock_init
    clk = 1'b0;
end

always
begin : clock_proc
    #(period/2) clk = ~clk;
end
```

ALWAYS BLOCK EXAMPLE

```

module clk_gen
  #(parameter period = 50)
  (
    output reg clk
  );

  initial clk = 1'b0;

  always
    #(period/2) clk = ~clk;

  initial #100 $finish;

endmodule
  
```

Time	Statement(s) Executed
0	clk = 1'b0;
25	clk = 1'b1;
50	clk = 1'b0;
75	clk = 1'b1;
100	\$finish;

VERILOG:

SIMULATION BUILDER:

ASSIGNMENT WITH DELAY (#)



TIMESCALE COMPILER DIRECTIVE PREVIEW

Time unit for
delays in module

``timescale 1 ns / 10 ps`

Precision for delay
values used by
simulator

```
module mult_acc (  
    input [7:0] ina, inb,  
    input clk, clr,  
    output [15:0] out  
);  
    ...  
    assign #5 ...  
    assign #25 ...
```

Note: Other compiler directives discussed later.

CONTINUOUS ASSIGNMENT DELAY

- Use #<value> notation to delay updating LHS
 - Models propagation delay

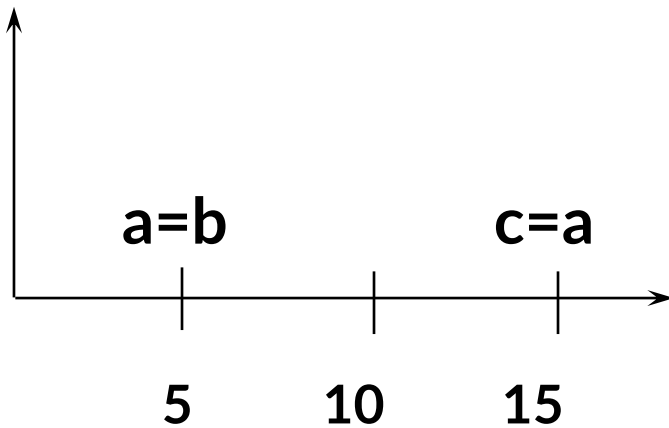
```
assign #25 adder_out = mult_out + out;
```

- Behavior
 - 1) Operand on RHS changes
 - 2) RHS reads input(s) and performs expression
 - 3) If $RHS \neq LHS$, LHS scheduled to be updated with new value after delay expires
 - 4) If another RHS operand changes before delay expires and a new value for LHS is calculated, then current value scheduled for LHS is cancelled and the new value for LHS is scheduled to be updated (if needed) after new delay period expires (inertial delay)
 - Requires value of RHS expression be stable for length of delay
- Ignored by synthesis

BLOCKING VS. NONBLOCKING ASSIGNMENTS

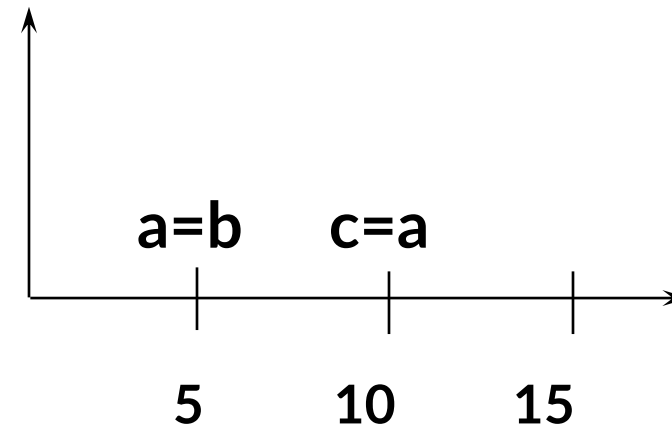
Blocking (=)

```
initial begin
  a = #5 b;
  c = #10 a;
end
```



Nonblocking (<=)

```
initial begin
  a <= #5 b;
  c <= #10 a;
end
```



EVALUATING BLOCKING & NONBLOCKING

initial begin

```
a = 1'b0; //Assgnmnt0  
b = 1'b1; //Assgnmnt1  
a <= #5 b; //Assgnmnt2  
b <= #5 a; //Assgnmnt3
```

end

This may seem confusing but it is perfectly valid Verilog and is here to illustrate the behavior of blocking and nonblocking statements

1. Assignment 0 executes and assigns **a** to 0 blocking remaining assignments
2. Assignment 1 executes and assigns **b** to 1 blocking remaining assignments
3. Assignment 2 executes and schedules **a** to take on the current value of **b** (1) after 5 time steps
4. Assignment 3 executes and schedules **b** to take the current value of **a** (0) after 5 time steps
5. Process ends and 5 time steps later **a** is updated to 1 and **b** is updated to 0

LAST NOTE ON NONBLOCKING

Unlike with continuous assignments, non-blocking assignments allow scheduling of any number of events to the same variable at different times without cancelling or overwriting prior scheduled events

```
initial begin
```

```
  a <= 1'b0;
```

```
  a <= #2 1'b1;
```

```
  a <= #4 1'b0;
```

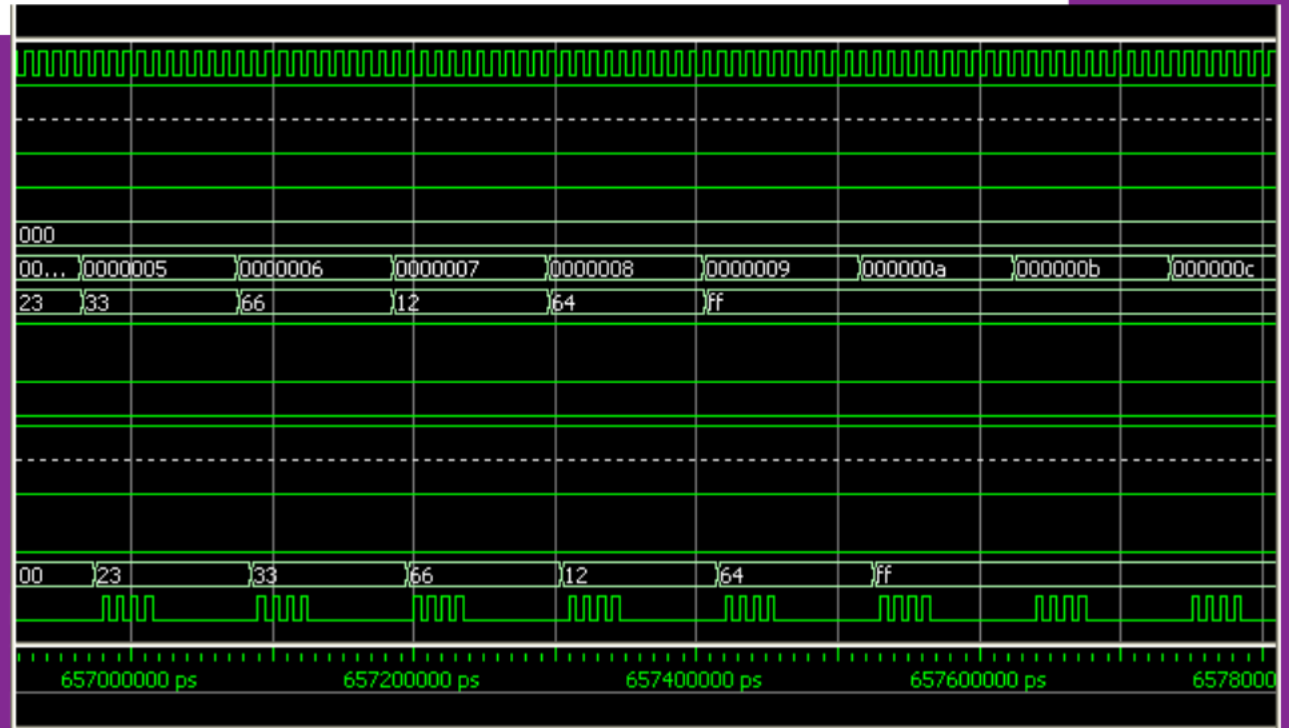
```
  a <= #6 1'b1;
```

```
end
```

*Each nonblocking statement schedules **a** to a value at some time in the future without disturbing the scheduling of the other statements*

SIMULATION

SIGNAL MONITORING



EVENT CONTROL

- Provides edge-sensitive control
- Symbolized by the @ symbol

@(expression)
- Pauses execution of procedural statements until event occurs (i.e. expression changes value)
- Inter-assignment delay occurs before expression is evaluated, intra-assignment delay occur before expression is assigned
- To test for multiple events (logical OR of events list)
 - Comma (,)
 - or

Examples*

```

initial begin
    // Rising edge control (inter-assignment)
    @ (posedge clk) r1 = r2;

    // Falling edge control (intra-assignment)
    r3 = @(negedge clk) r4;

    // Either edge control
    @ (a) r5 = r6;

    // Either edge control using more
    //   complex expression
    @ (a ^ b & c) r7 = r8;

    // Logical OR of two events using comma
    @ (posedge clk, enable) r9 = r10;

    // Logical OR of two events using
    //   "or" keyword
    r11 = @ (posedge clk or enable) r12;
end
  
```

**Note: This usage not supported by synthesis.*

WAIT STATEMENT

- Provides level-sensitive event control
- Uses wait keyword **wait** (*expression*)
- Pauses execution of the procedural block until wait statement is satisfied
 - If level test not satisfied, procedural block pauses until it is
 - If level test already satisfied, procedural block continues execution without pausing
- Not supported by synthesis

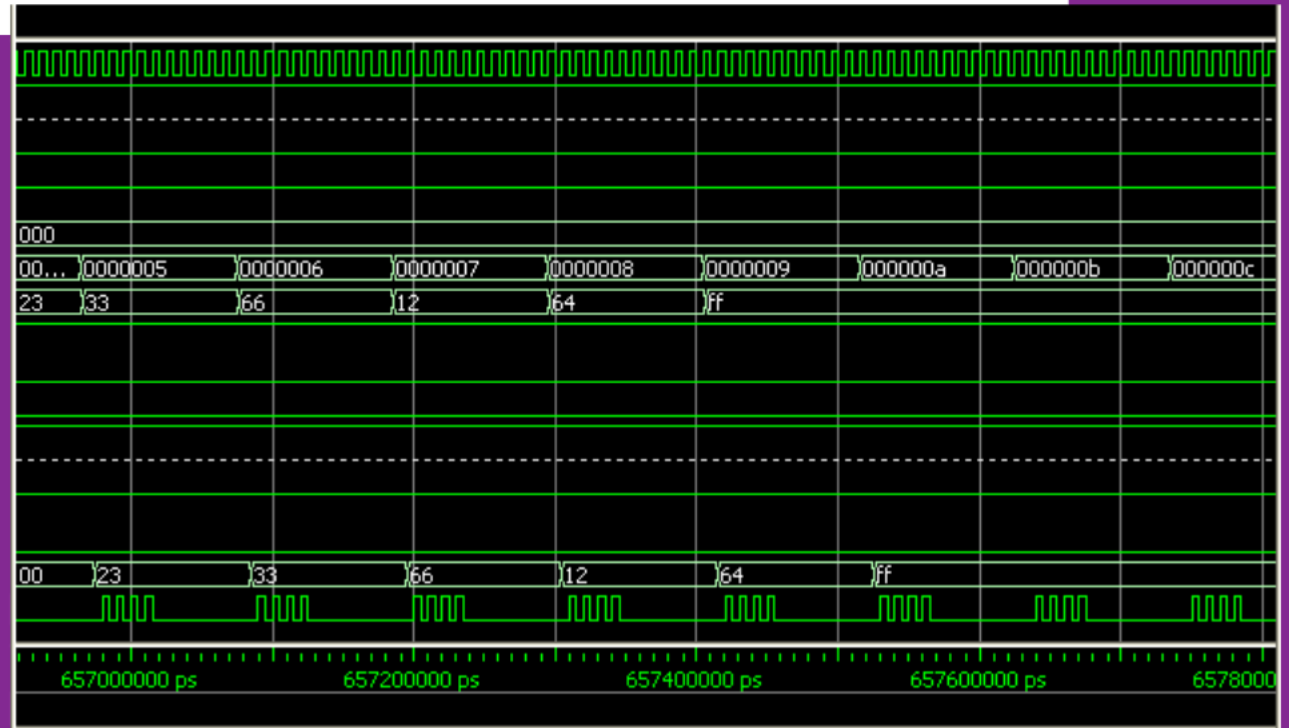
Example

```
initial begin
    ...
    wait (gate) r1 = r2; //Assignment0
    wait (!gate) r3 = r4; //Assignment1
    ...
end
```

- *Assignment0 must pause until gate is true (1) before r1 takes on value of data. Statement does not pause if gate already true*
- *Assignment1 must pause until gate is false (0) before r3 takes on value of data. Statement does not pause if gate already false*

SIMULATION

LOOP STATEMENTS



FOREVER AND REPEAT LOOPS

forever loop - executes continually

```
initial begin  
    clk = 0;  
    forever #25 clk = ~clk;  
end
```

Clock with period
of 50 time units

Not synthesizable!

repeat loop - executes a fixed number of times

```
if (rotate == 1)  
    repeat (8) begin  
        tmp = data[15];  
        data = {data << 1, tmp};  
    end
```

Repeats a rotate
operation 8 times

WHILE LOOP

while loop - executes if expression is true

```
initial begin
    count = 0;
    while (count < 101) begin
        $display ("Count = %d", count);
        count = count + 1;
    end
end
```

Counts from 0 to 100
Exits loop at count 101

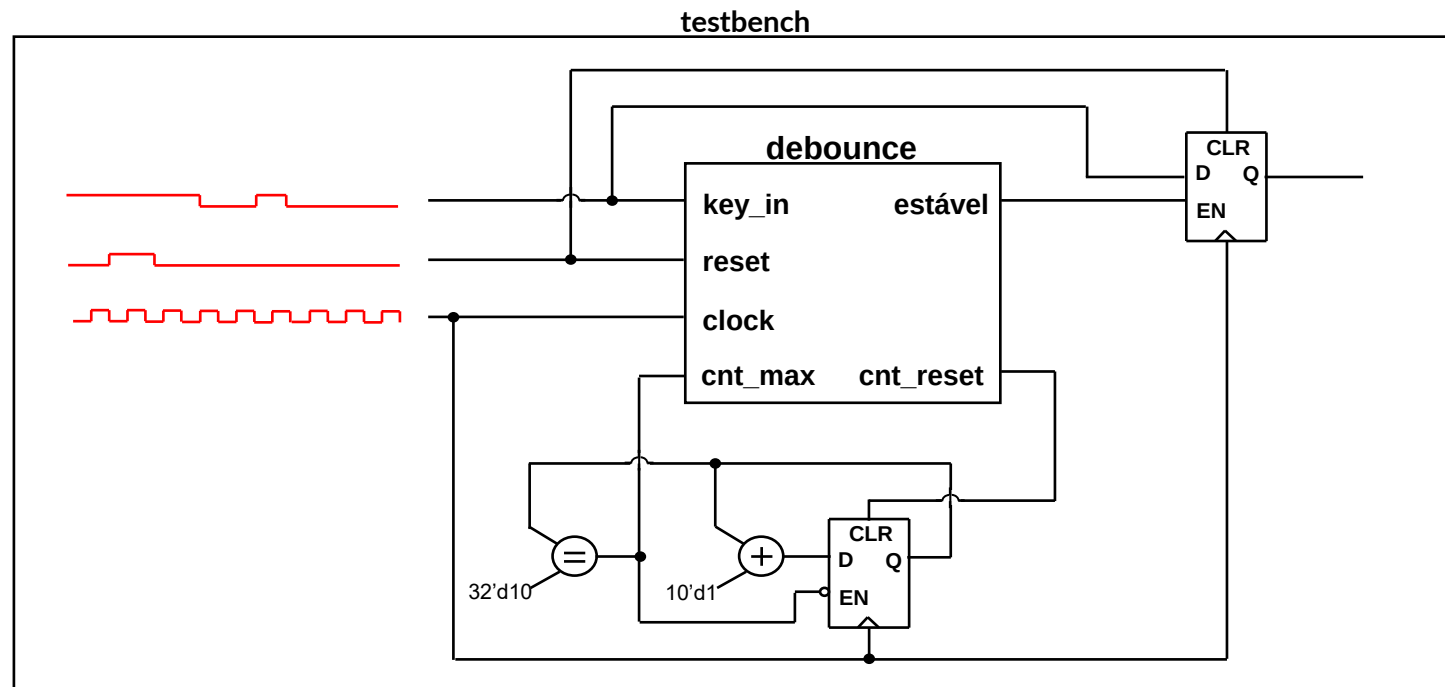
Not synthesizable!

PROJECT 10



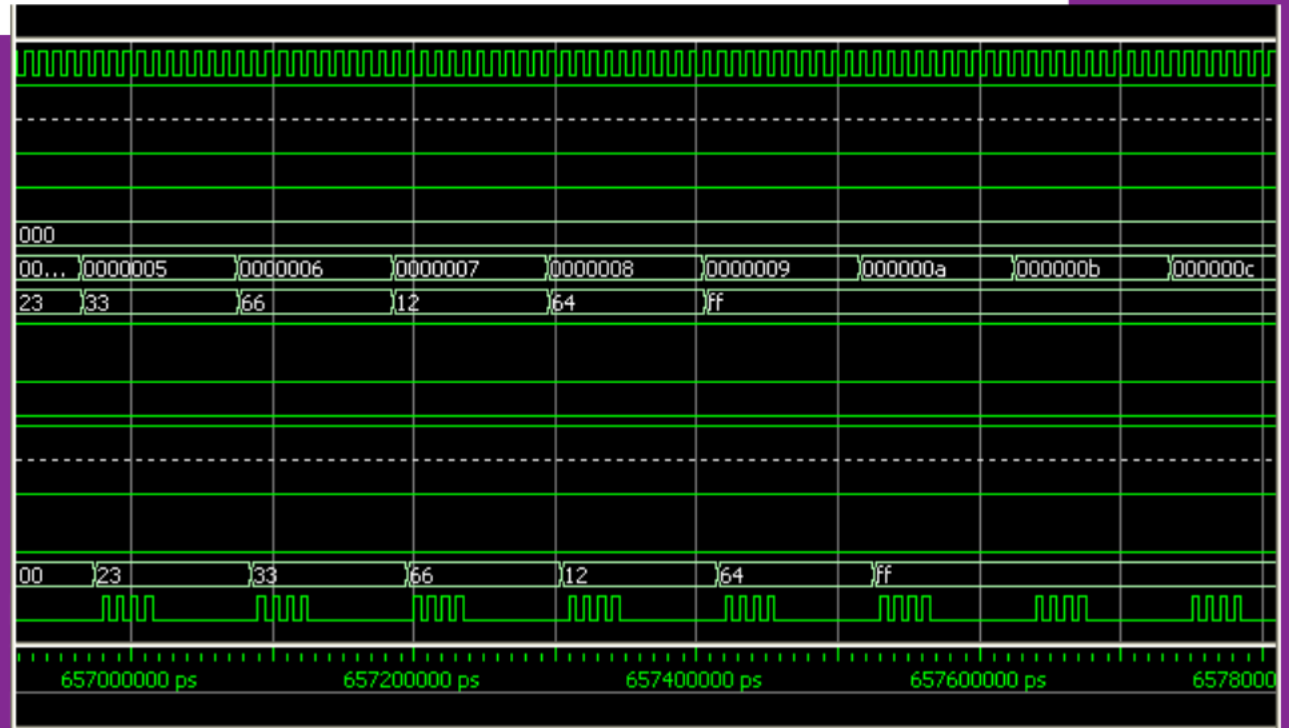
PROJECT 10

- Write Verilog code for a testbench that generates a clock signal and a reset signal
- Describe a signal that will simulate a key with noise
- Implement a counter with a reset signal and a maximum value
- Instantiate the debounce module



SIMULATION

MODELSIM



MODELSIM SIMULATION TOOL

- Developed by Mentor Graphics
- One of the industry's most popular simulators
- Simulates both Verilog & VHDL
 - Also allows for mixed language
- ModelSim-Altera Starter Edition included with Quartus software
- Optional ModelSim-Altera Edition

MODELSIM-ALTERA STARTER EDITION FEATURES

- Easy-to-use interface
 - Common across platforms
 - Available for PC, UNIX, and Linux environments
- NativeLink integration
 - Quartus software can automatically invoke ModelSim after place and route is finished

MODELSIM USER INTERFACE

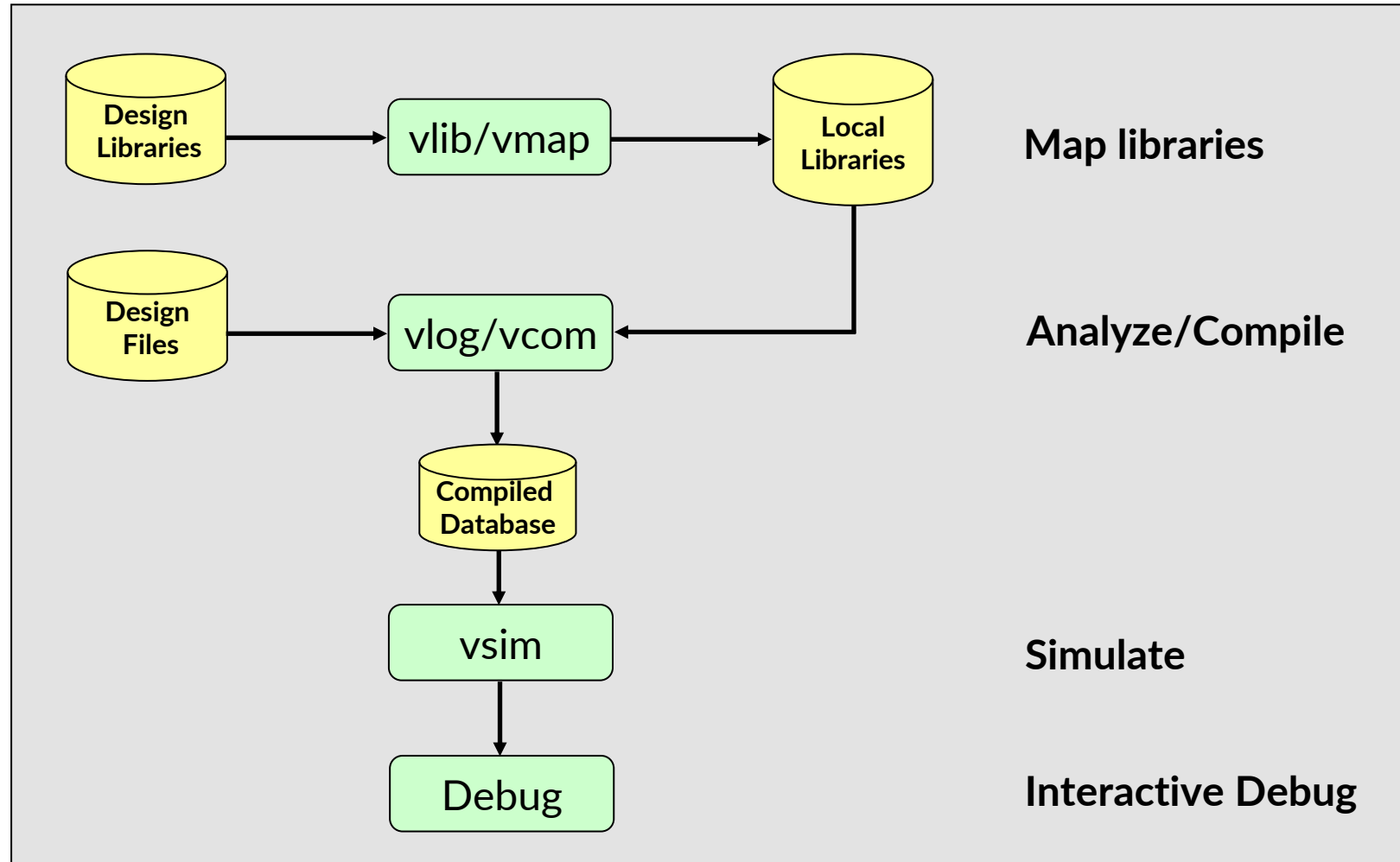
The screenshot displays the ModelSim user interface with several windows and panels labeled:

- Main Window:** The central area showing the source code for a Verilog module named `mult`.
- Workspace:** The left-hand pane showing the project hierarchy, including files like `filter`, `u1`, and `u2`.
- Objects:** A panel showing a list of objects and their values, such as `clk` (S10), `d` (00010000), and `reset` (S10).
- Source:** The main text area displaying the Verilog code for the `mult` module.
- Processes:** A window showing active processes, currently displaying `main`.
- Transcript:** A window showing the command history and simulation output, including commands like `run` and `view dataflow`.
- List Window:** A window showing a list of signals and their values, including `ns`, `delta`, and various `St0` and `St1` signals.
- Wave Window:** A window showing a timing diagram with multiple signals plotted against time, including `clk`, `d`, `reset`, `yn`, and `yvalid`.
- Dataflow:** A window showing the dataflow graph for the selected module, including a block labeled `#ASSIGN#7`.

MODELSIM USER INTERFACE FEATURES

- Windows and panes: Main (Workspace), Active Process, Dataflow, List, Locals, Memory, Objects, Source, & Wave
- Open from the View menu
- Drag & drop
 - Selecting HDL items in one window, drag from one window to another
 - Drag from the Dataflow, Objects, Source, Structure, Locals, and Wave windows
 - Drop into Wave or List window to see simulation results

MODELSIM-ALTERA PROJECT FLOW



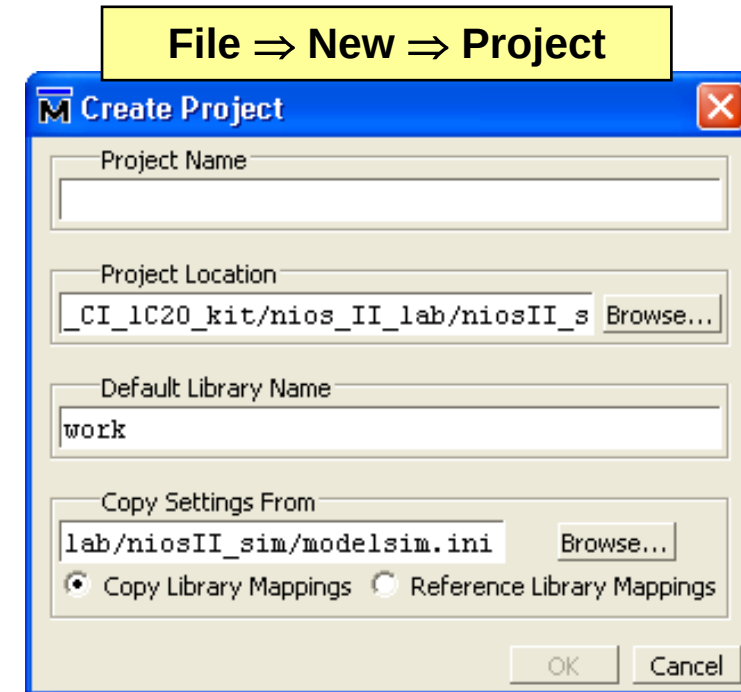
my_project

MODELSIM-ALTERA PROJECT FLOW

1. Create a project (including **work** library)
2. Add design and testbench files to the project
3. Create ModelSim libraries (optional)
4. Map existing libraries to physical directories (optional)
5. Compile design files
6. Start simulator
7. Create a simulation configuration (optional)
8. Advance simulator

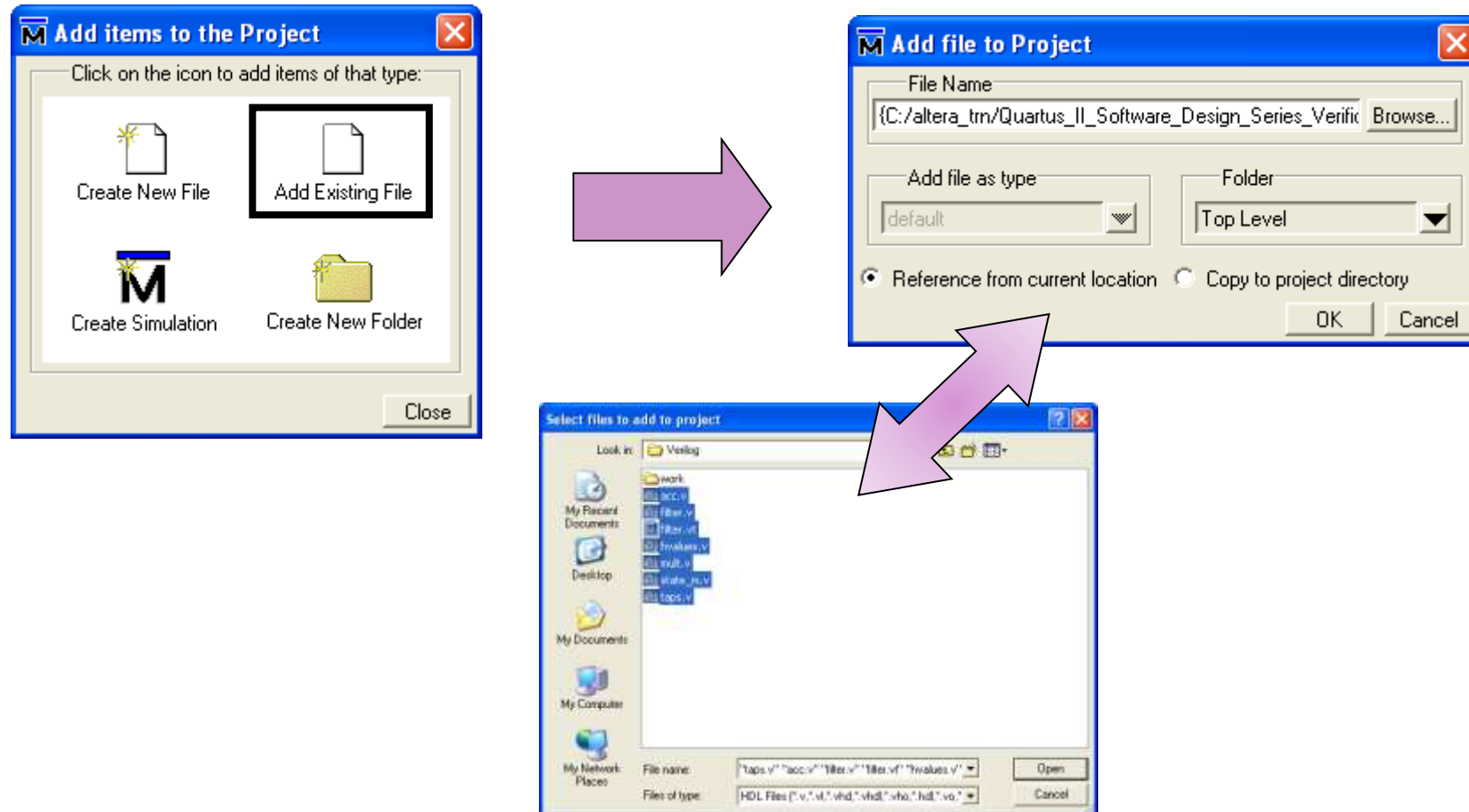
1) CREATE A PROJECT

- Name project, set location, and name default library (default name is **work**)
- Creates metadata file (.mpf) in project location to store all project settings



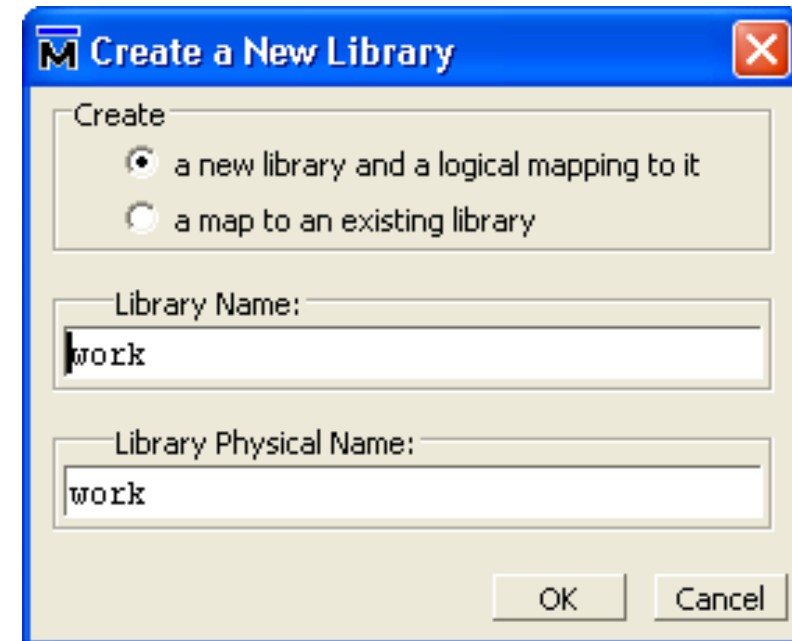
2) ADD FILES TO THE PROJECT

Add any existing design and testbench (HDL that defines simulation stimulus) files

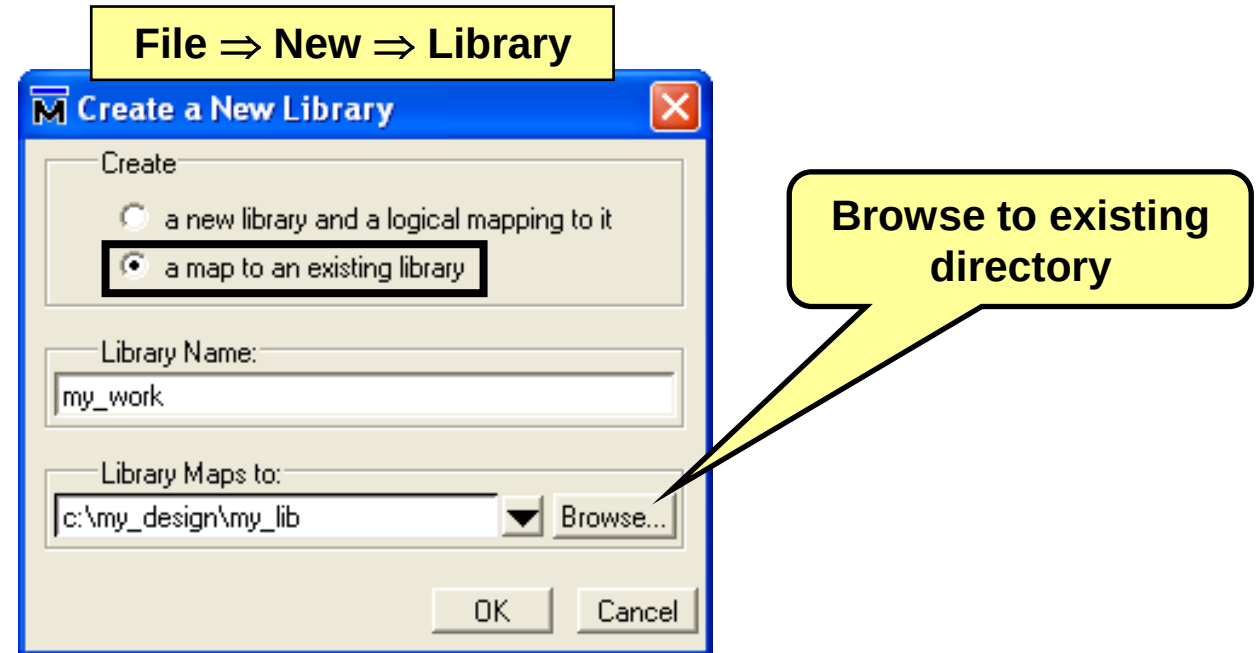


3) CREATE MODELSIM LIBRANES (OPTIONAL)

- From Main window
 - File ⇒ New ⇒ Library...
- Transcript command
 - ModelSim> **vlib** <library name>
 - Example: **vlib** my_work



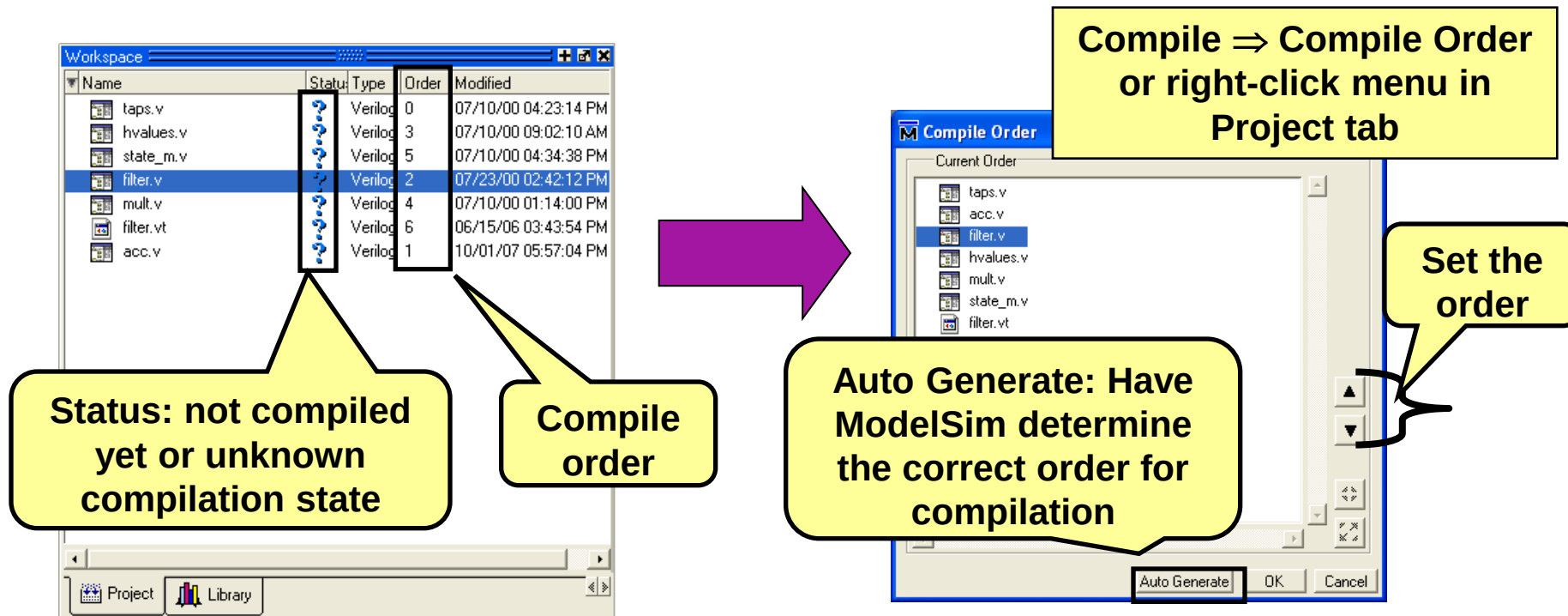
4) MAP EXISTING LIBRARIES TO PHYSICAL DIRECTORIES (OPTIONAL)



- Transcript command
 - ModelSim> **vmap** <logical_name> <directory_path>
 - Example: **vmap** my_work c:\my_design\mylib

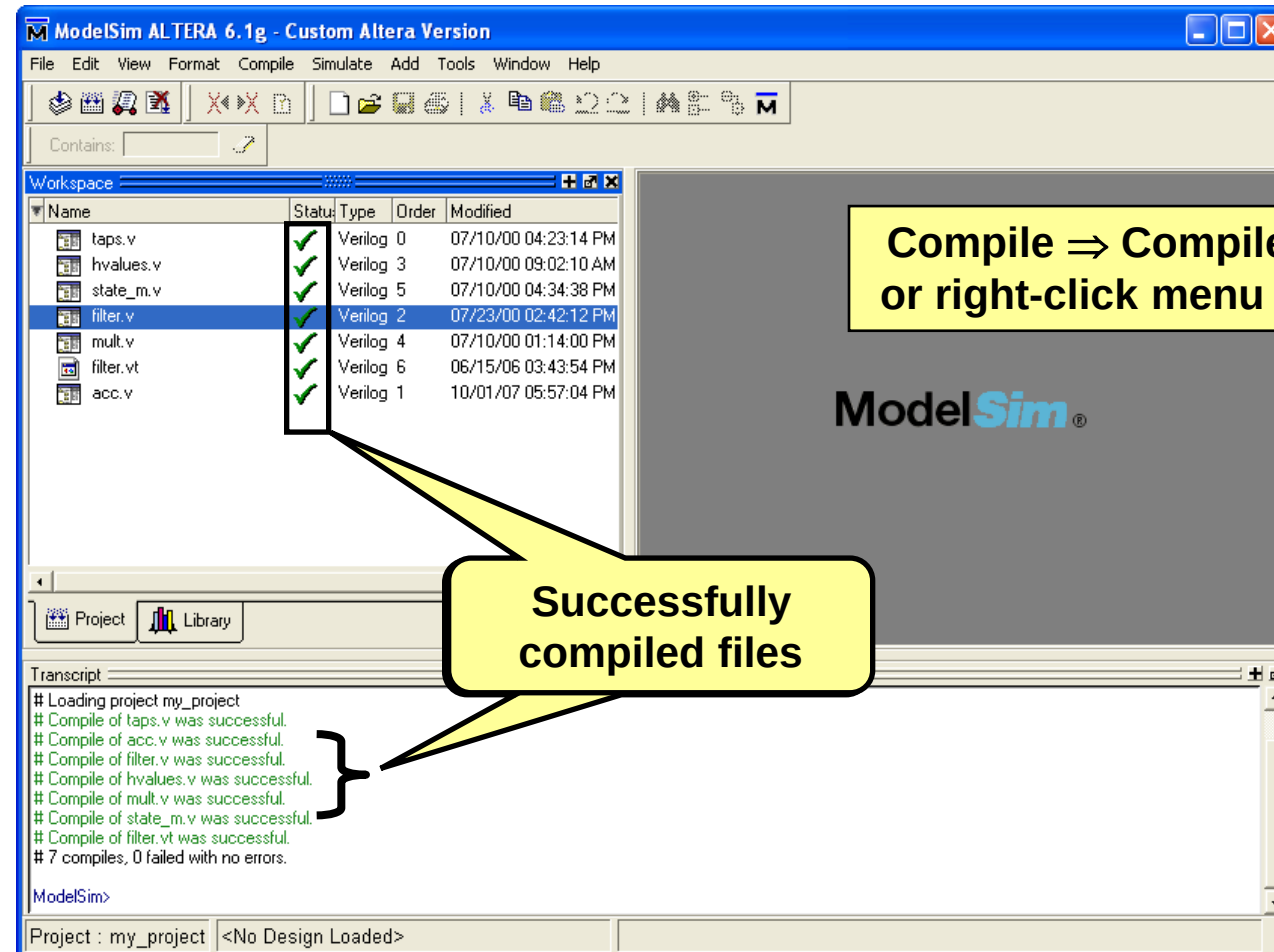
5) COMPILE DESIGN FILES

- Project tab in Workspace displays all design files
- Set compilation order and store with project
 - Required for VHDL dependencies; optional for Verilog
 - Useful in full version of ModelSim with mixed Verilog/VHDL



5) COMPILE DESIGN FILES (CONT.)

Compile all files or
only selected ones



6) START SIMULATOR (GUI)

Select options and start the simulation

Simulate ⇒ Start Simulation

Select and expand library

Select top-level module or entity/architecture to simulate

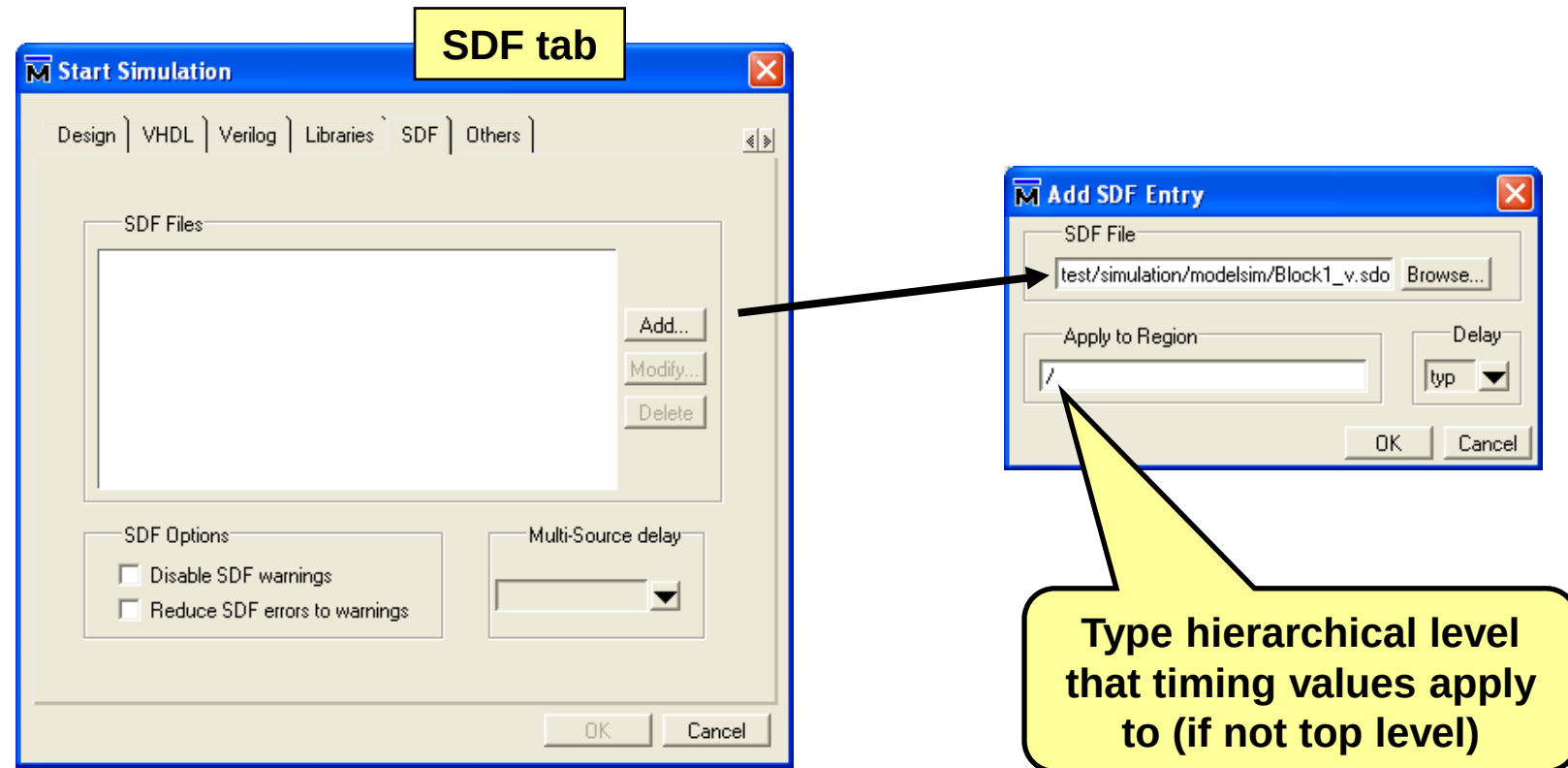
Choose simulator resolution

Note: Most windows described earlier are not populated (or even appear, like the *sim* tab) until the simulator is started.

6) START SIMULATOR (GUI) (CONT.)

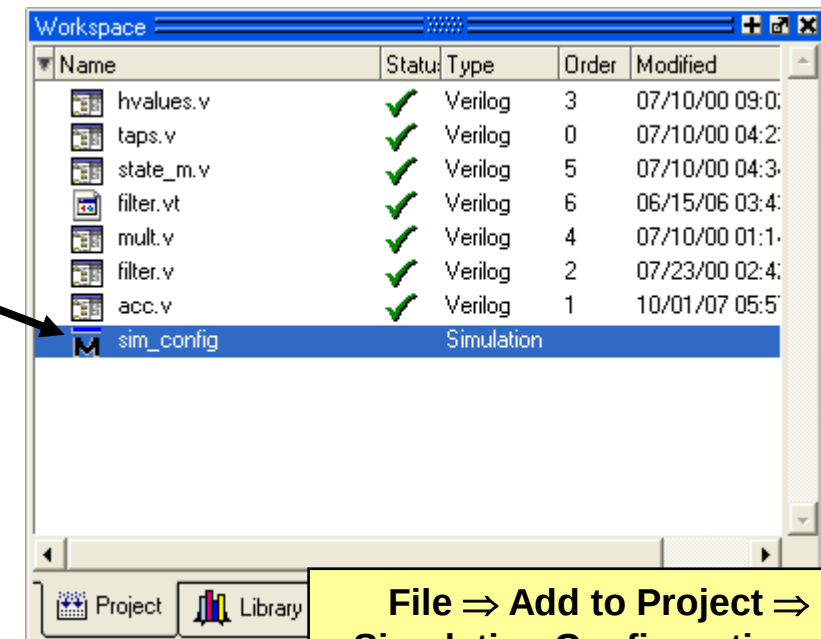
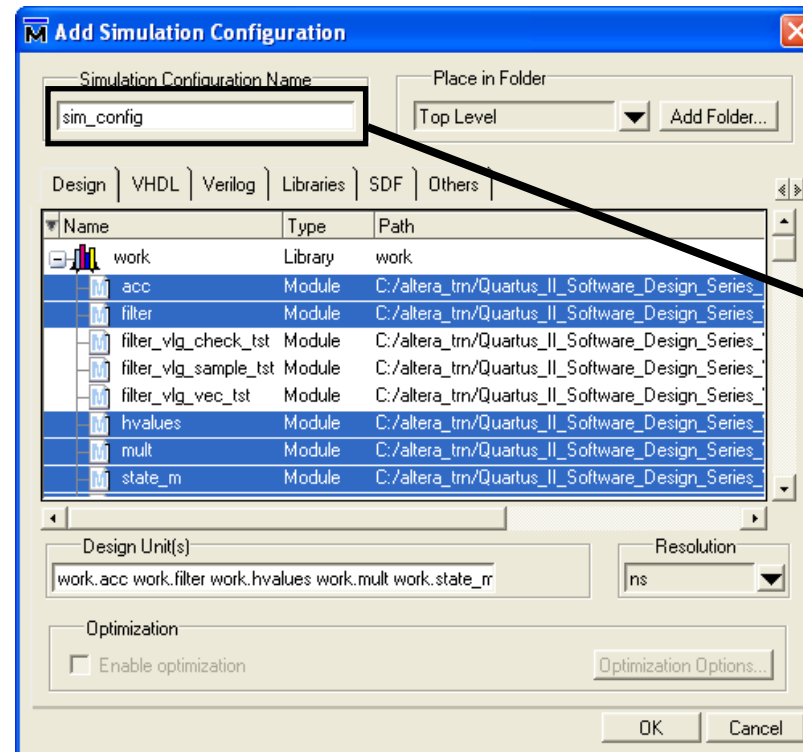
For gate-level, post-fit (timing) simulation, add **SDF** or **SDO** file(s)

See on-line help for more information about post-processing simulations



7) CREATE A SIMULATION CONFIGURATION (OPTIONAL)

- Simulation configuration holds all settings and simulation options
- Double-click to start simulation (instead of step 6)



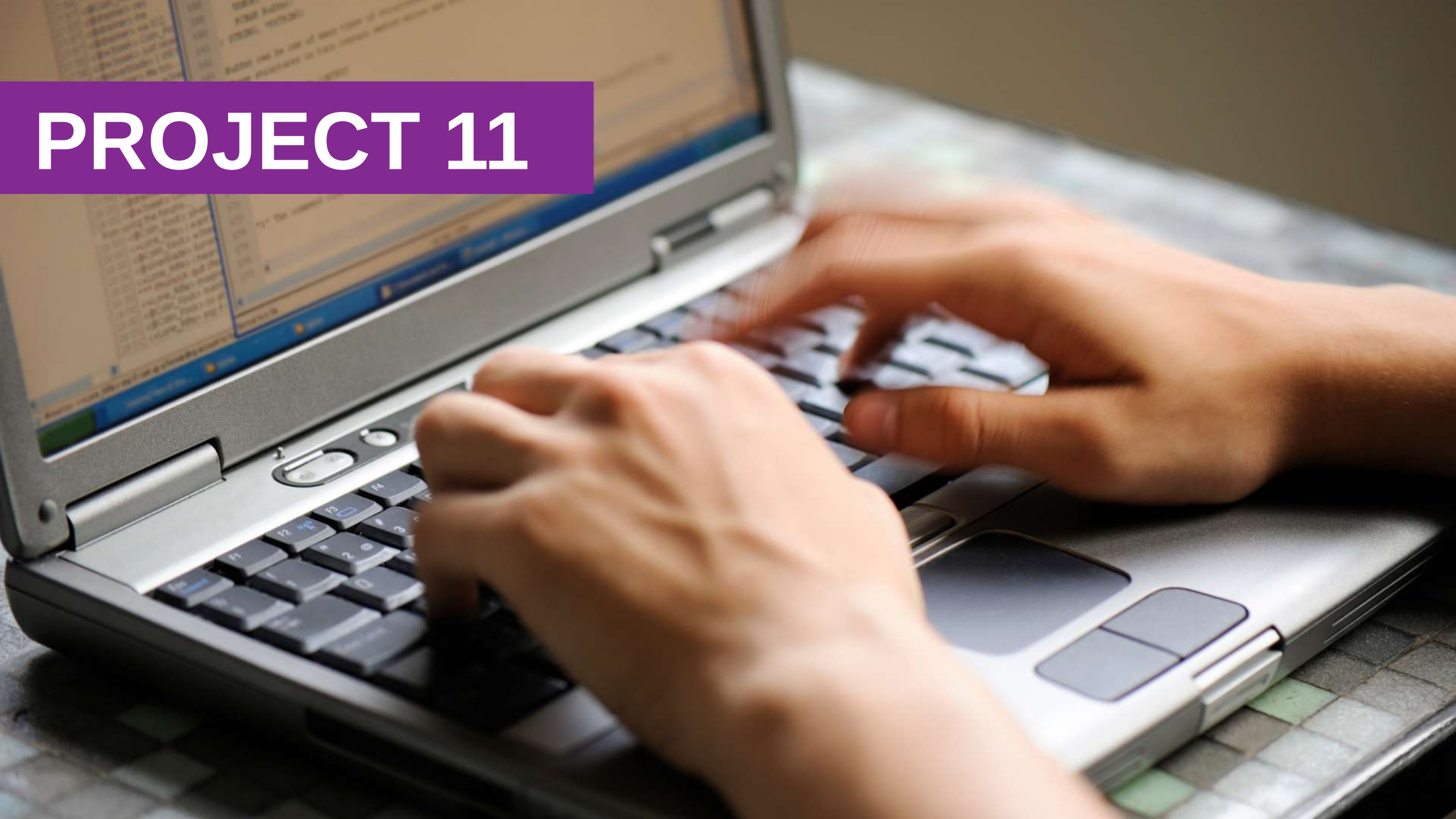
**File ⇒ Add to Project ⇒
Simulation Configuration or
right-click menu in Project tab**

8) ADVANCE SIMULATOR (GUI)

- **Simulate ⇒ Run ⇒ <run_option>**
 - Advance simulation by selected number of time steps
 - Continue running a paused simulation
- **Simulate ⇒ Run ⇒ Restart**
 - Select simulation result data to keep for next run
 - Reload any edited design elements
 - Reset simulation time to 0
 - Transcript: **restart**

COMMAND EXAMPLES (RUN)

- **run 1000**
 - Advances the simulator 1000 timesteps from the current position
- **run 2500 ns**
 - Advances the simulator the number of timesteps corresponding to 2500 ns from the current position
- **run @3000**
 - Advances the simulator to timestep 3000 from the current position
- **run @7000 ns**
 - Advances the simulator to 7000 ns from the current position
- **restart (or restart -force)**
 - Resets the simulator to time 0



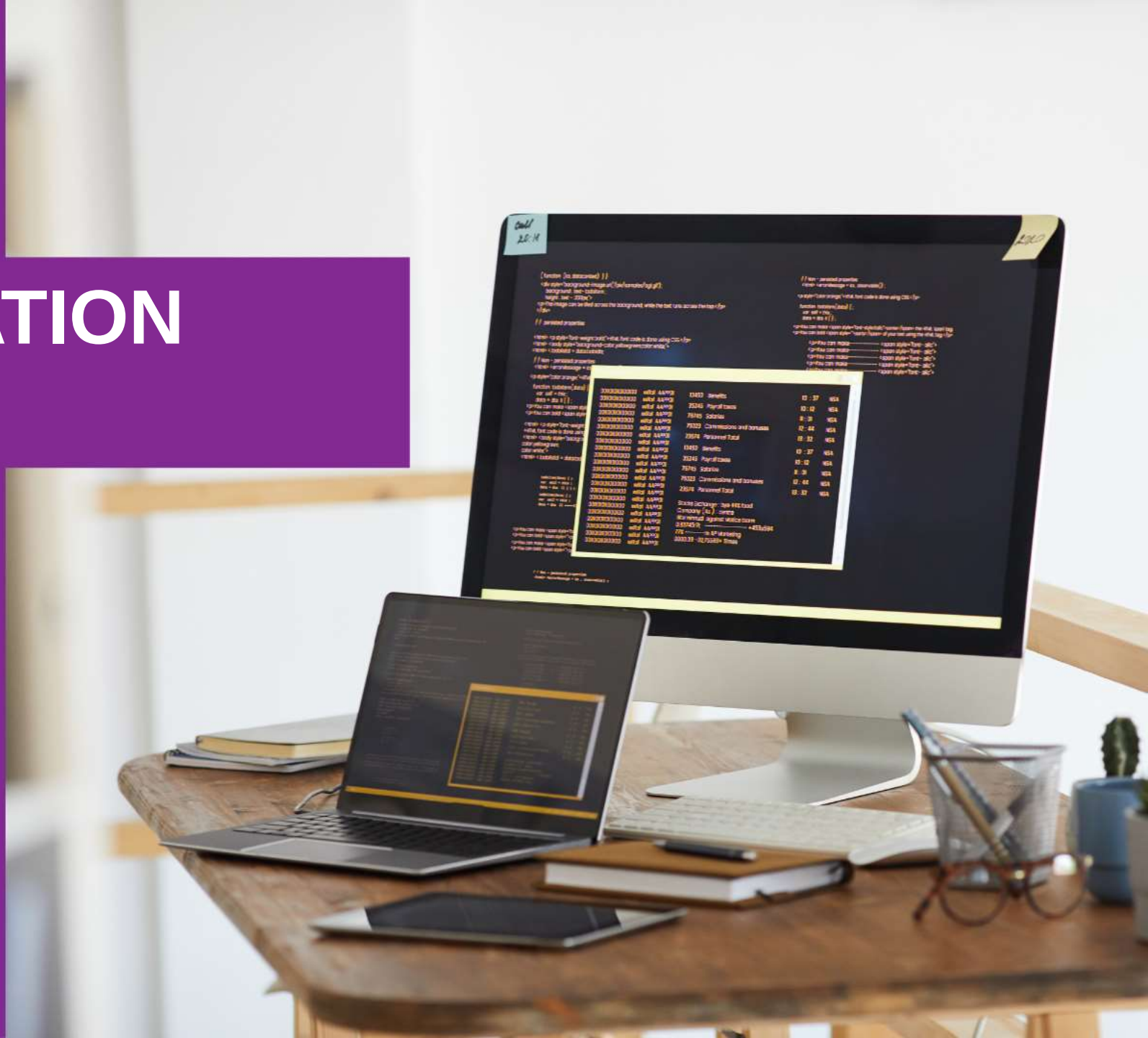
PROJECT 11

PROJECT 11

- Create a ModelSim project
- Add both the Testbench and the Debounce files to the project
- Compile and Run the simulation

SYSTEM INTEGRATION TOOLS

PLATFORM DESIGNER
(Qsys)

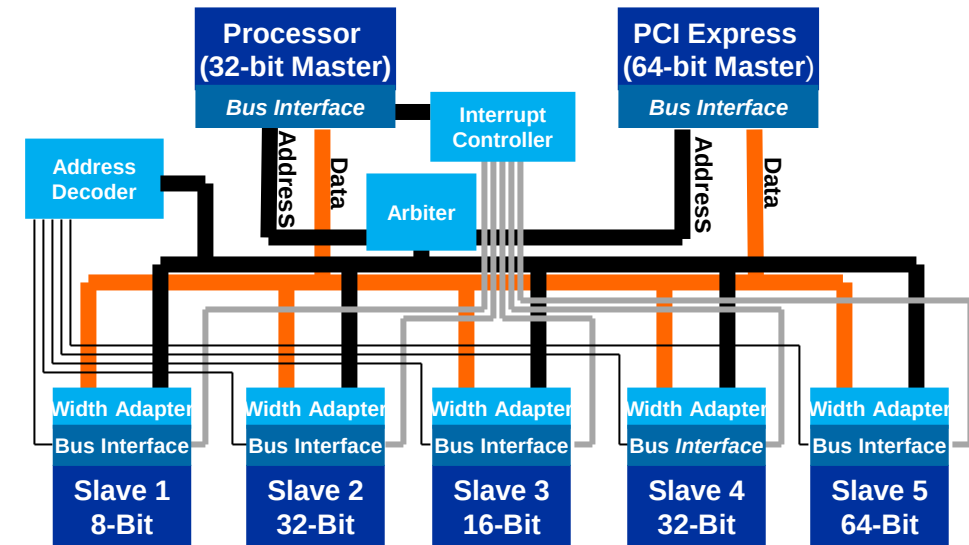


SYSTEM DESIGN ENTRY WITH PLATFORM DESIGN

- Simplifies complex system development
 - Automatic interconnect generation
- Raises the level of design abstraction
 - High level design and system visualization
- Provides a standard platform: IP integration, custom IP authoring, IP verification
- Enables design re-use
- Scales easily to meet the needs of end product
- Reduces time to market
 - Reduces design development time
 - Eases verification

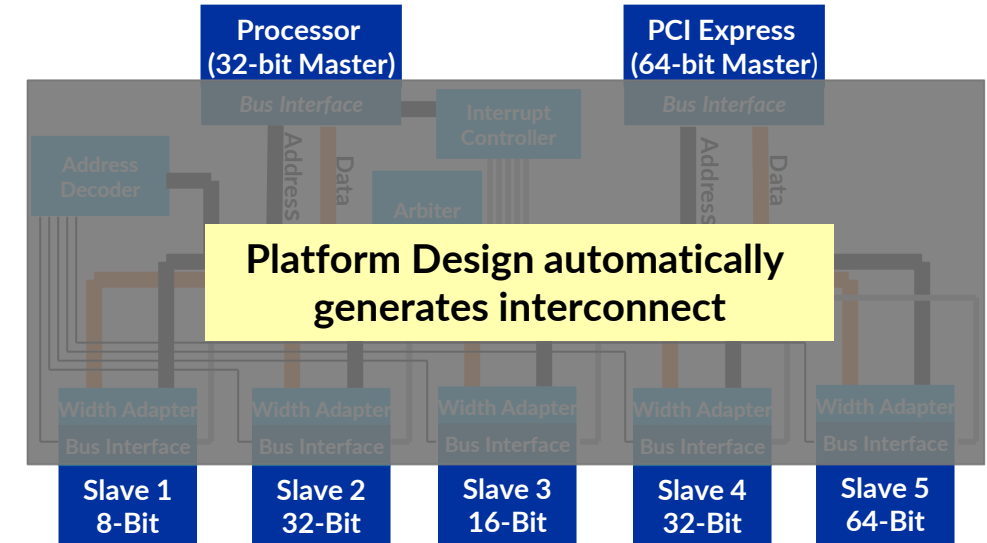
BUT WHAT EXACTLY IS PLATFORM DESIGN?

- Components in system use different interfaces to communicate (some standard, some non-standard)
- Typical system requires significant engineering work to design custom interface logic
- Integrating design blocks and intellectual property (IP) is tedious and error-prone



AUTOMATIC INTERCONNECT GENERATION

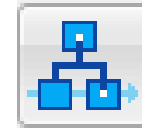
- Avoids error-prone integration
- Saves development time with automatic logic & HDL generation
- Enables you to focus on value-add blocks



Platform Design improves productivity by automatically generating the system interconnect logic

THE PLATFORM DESIGN GUI

Access in Tools menu, toolbar, or Tasks window

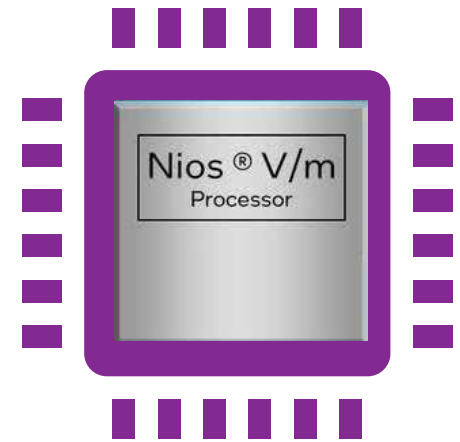


The screenshot displays the Altera Platform Design GUI with several key components labeled:

- IP Catalog:** A window on the left showing a list of IP cores categorized by type (e.g., DSP, Interface Protocols, Memory Interfaces and Controllers, PL, Processors and Peripherals, Qsys Interconnect).
- System Contents:** A central window showing a hierarchical diagram of the system components and their interconnections. A blue circle highlights the tabs at the top of this window, with an arrow pointing to the text "Draggable, detachable tabs".
- Hierarchy:** A window on the bottom left showing a detailed tree view of the system components, including clocks, buses, and memory blocks.
- Messages:** A window at the bottom right showing a list of messages (warnings and errors) with columns for Type, Path, and Message.

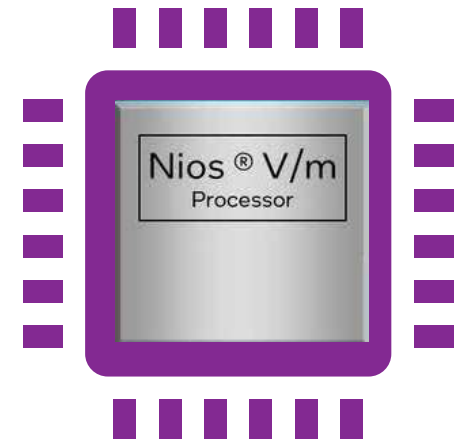
NIOS V PROCESSOR

- Nios V is a softcore processor
 - Implemented using FPGA's programmable logic resources
 - Allows customization based on specific project needs
- Based on RISC-V Architecture
 - Open and extensible instruction set architecture
 - Promotes flexibility and innovation in custom solutions
- Evolution of Nios Family
 - Successor to the Nios II processor
 - Aligns with the trend towards open architecture (RISC-V)



NIOS V PROCESSOR

- Integration with Altera FPGAs
 - Designed to work seamlessly with Altera FPGAs
 - Ideal where flexibility and customization are key
 - Adjustable features: bus width, cache size, and more
- Applications
 - IoT devices, industrial automation, telecommunications
 - Ideal for embedded systems, signal processing, control systems



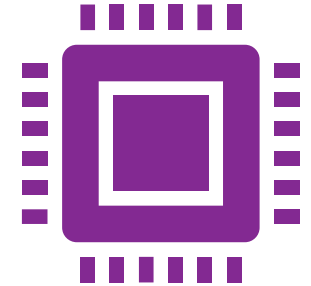
TREINAMENTO ETAPA 02

■ Platform Designer

- Introduction to Platform Designer
- Getting Started with Platform Designer
- Platform Designer GUI
- Platform Designer Interconnect
- IP Catalog "Off-the-Shelf" Components
- Platform Designer Standard Interfaces & Signaling
- Custom Components
- System Console & Unified Toolkit (UTK) Framework

■ NIOS V Processor

- Introduction to Nios® V Processor Architecture
- FPGA Hardware Design Flow
- Software Build Tools and System Design with Nios® V Processor
- Booting and Configuration Options



TREINAMENTO ETAPA 02

Curso online e ao vivo de nível Avançado



Data: **17 a 21 de Março**



Horário: **das 14h às 18h**



Inscrição até **07 de Março**

- Carga horária: 20 horas
- Siga utilizando o seu kit DE10-Lite
- Ementa Completa: <https://www.macnicadhw.com.br/capacitacao/treinamento-fpga-intel/treinamento-fpga-avancado/ementa-completa>





macnica
Macnica DHW



[/macnicadhw](#)



[Macnica DHW](#)



[/macnicadhw](#)



+55 (48) 3225-5052



faleconosco@macnicadhw.com.br



www.macnicadhw.com.br



Florianópolis/SC - Brasil
Rua Patrício Farias 101, 8º andar
Bairro Itacorubi - CEP 88034-132